

Atlas 200 AI Accelerator Module

Application Software Development Guide (Models 3000)

Issue 01
Date 2019-09-30



Copyright © Huawei Technologies Co., Ltd. 2020. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions



HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base
Bantian, Longgang
Shenzhen 518129
People's Republic of China

Website: <https://www.huawei.com>

Email: support@huawei.com

Contents

1 About This Document.....	1
2 Development Process.....	2
3 DDK Environment Setup.....	4
3.1 DDK Installation Guide (CentOS, x86).....	4
3.1.1 Introduction.....	4
3.1.2 Obtaining Software Packages.....	4
3.1.3 Environment Preparation.....	5
3.1.4 Installing the DDK.....	10
3.1.4.1 Preparing for Installation.....	11
3.1.4.2 Installation.....	12
3.1.4.3 Common Operations.....	12
3.1.4.3.1 Uninstalling the DDK.....	12
3.1.4.3.2 Querying the DDK Version.....	13
3.1.4.3.3 Configuring the OpenPGP Public Key.....	13
3.1.5 FAQs.....	15
3.1.5.1 What Do I Do If the Message "Import Error:cannot import name main" Is Displayed When the pip3/pip3.5 -V Command Is Run after pip3/pip3.5 Upgrade?.....	16
3.1.5.2 What Do I Do If epel-release Installation Fails?.....	16
3.1.5.3 What Do I Do If the Message "Warning: DDK functions except TE have been successfully installed." Is Displayed During DDK Installation?.....	17
3.1.5.4 What Do I Do If a WARNING or FAIL Result Is Returned in the Integrity Verification of a Software Package?.....	17
3.1.6 Appendix.....	19
3.1.6.1 Overview of Software Packages.....	19
3.1.6.2 Open Source Third-Party Libraries.....	20
3.2 DDK Installation Guide (Ubuntu, x86).....	21
3.2.1 Introduction.....	21
3.2.2 Obtaining Software Packages.....	21
3.2.3 Environment Preparation.....	22
3.2.4 Installing the DDK.....	25
3.2.4.1 Preparing for Installation.....	25
3.2.4.2 Installation.....	26
3.2.4.3 Common Operations.....	27

3.2.4.3.1 Uninstalling the DDK.....	27
3.2.4.3.2 Querying the DDK Version.....	27
3.2.4.3.3 Configuring the OpenPGP Public Key.....	28
3.2.5 FAQs.....	30
3.2.5.1 What Do I Do If a Message Is Displayed Indicating pip2 or pip Unavailability During Mind Studio or DDK Installation?.....	30
3.2.5.2 What Do I Do If a WARNING or FAIL Result Is Returned in the Integrity Verification of a Software Package?.....	31
3.2.6 Appendix.....	33
3.2.6.1 Overview of Software Packages.....	33
3.2.6.2 Open Source Third-Party Libraries.....	34
4 Model Conversion.....	36
4.1 Introduction.....	36
4.2 Caffe/TensorFlow Mode Conversion.....	38
4.2.1 Constraints and Parameters.....	38
4.2.2 Model Conversion Using OMG.....	48
4.3 AIPP Configuration.....	49
4.3.1 Overview.....	50
4.3.2 Configuration File Template.....	51
4.3.3 CSC Configuration.....	54
4.3.4 Crop/Padding Configuration.....	58
4.3.5 AIPP Configurations According to Model Inputs.....	59
4.3.6 AIPP Verification of the Model Input Size.....	59
4.3.7 Parameter Structure for Dynamic AIPP.....	60
4.4 Quantization Configuration.....	61
4.4.1 Overview.....	61
4.4.2 Configuration File Template.....	62
4.4.3 Description of the Configuration Parameters.....	64
4.4.4 Parameter Tuning.....	73
4.5 FAQs.....	77
4.5.1 What Do I Do If No Log Is Output to the Screen When the DDK Installation User Is HwHiAiUser?.....	77
4.5.2 What Do I Do If Model Conversion Takes Too Long When the OS and Architecture Configuration of the DDK Server is Arm (aarch64)?.....	77
4.5.3 What Do I Do If the Error "Unrecognized layer:xxx, layer type xxx" Is Reported During Model Conversion?.....	77
4.5.4 What Do I Do If the Error "It is recommended to convert layers-structure to layer-structure by caffe tool" Is Reported During Model Conversion?.....	78
5 Operator Development.....	80
5.1 Before You Start.....	80
5.1.1 Overview.....	80
5.1.2 Intended Audience.....	83
5.2 Precautions for Operator Development.....	84

5.3 Environment Preparation.....	84
5.4 Overall Development Process.....	84
5.5 Setting Environment Variables.....	85
5.6 Creating a Custom Operator Development Project.....	86
5.7 Developing a Custom Operator.....	88
5.7.1 Operator Basics.....	88
5.7.2 Implementing an Operator.....	92
5.7.2.1 Procedure.....	93
5.7.2.2 Importing Python Modules.....	93
5.7.2.3 Implementing an Operator.....	94
5.7.2.4 Scheduling and Building an Operator.....	95
5.7.2.5 Running an Operator.....	96
5.7.3 Code Examples.....	96
5.7.4 Operator Running Verification.....	98
5.7.4.1 Compiling an Operator.....	98
5.7.4.2 Building an Input Data File.....	99
5.7.4.3 Running a Single Operator.....	100
5.8 caffe.proto File Operator Definition (Optional).....	102
5.9 Developing the Plug-In of a Custom Operator.....	103
5.9.1 Implementing a Plug-In.....	103
5.9.1.1 Implementing the Plug-In.....	103
5.9.1.2 Including Header Files.....	104
5.9.1.3 Parsing an Operator.....	106
5.9.1.4 Inferring the Output Tensor Description of an Operator.....	110
5.9.1.5 Building an Operator.....	113
5.9.1.6 Registering an Operator.....	114
5.9.2 Plug-In Code Example.....	116
5.9.3 Compiling the Operator Plug-In.....	119
5.10 Loading the Plug-In for Model Conversion.....	120
5.11 Function Reference for Operator Plug-In Development.....	121
5.11.1 ParseParamsFn.....	121
5.11.2 InferShapeAndTypeFn.....	122
5.11.3 TEBinBuildFn.....	123
5.11.4 WeightFormats.....	124
5.11.5 Registration Macro of the Operator Building Function.....	125
5.12 Operator List.....	126
5.12.1 Parameter Description.....	126
5.12.2 Operator Boundaries.....	127
5.12.2.1 General Restrictions.....	127
5.12.2.2 Caffe Operator Boundaries.....	127
5.12.2.3 TensorFlow Operator Boundaries.....	156
6 Application Software Development.....	207

6.1 Before You Start.....	207
6.1.1 Application Scenarios.....	207
6.1.2 Key Concepts.....	208
6.1.3 Service Development Constraints.....	209
6.2 Software Development Description (Important).....	211
6.3 Preparations.....	212
6.3.1 Obtaining the Sample Program.....	212
6.4 Development Environment.....	213
6.4.1 DDK Directory Distribution.....	213
6.4.2 Header Files and Libraries.....	213
6.4.3 Compilation Tool.....	214
6.4.4 DDK Tools.....	215
6.5 Inference Service Flow.....	215
6.5.1 Software Service Flow.....	215
6.5.2 Example Mapping Between Software and Hardware Modules.....	216
6.6 Code Execution Sample (HelloDavinci).....	217
6.6.1 Obtaining the HelloDavinci Code.....	217
6.6.2 HelloDavinci Process Framework.....	217
6.6.3 HelloDavinci Compilation and Running.....	218
6.7 Software Code Development.....	219
6.7.1 Configuring the Matrix Framework.....	219
6.7.1.1 Configuring, Creating, and Destroying a Graph.....	220
6.7.1.2 Configuring Engine.....	222
6.7.1.3 Configuring Data Transmission.....	223
6.7.2 Using DVPP APIs.....	224
6.7.2.1 Using DVPP APIs.....	224
6.7.2.2 Applying for DVPP Memory.....	226
6.7.3 Offline Model Inference.....	226
6.7.3.1 Configuring AIPs.....	226
6.7.3.2 Converting an Offline Model.....	228
6.7.3.3 Performing Model Inference.....	229
6.7.4 Optimizing Software Performance.....	231
6.7.5 Commissioning Software Logs.....	231
6.7.5.1 Registering Log Levels.....	231
6.7.5.2 Printing Logs.....	232
6.7.5.3 Viewing Logs.....	232
6.7.5.4 Restarting Log-related Processes.....	234
6.7.6 Service Software Compilation.....	234
6.8 Graph Keywords.....	235
7 High Performance Application Programming.....	241
7.1 Overview.....	241
7.2 Key Points.....	241

7.2.1 Memory Management.....	242
7.2.1.1 Memory Management APIs Provided by the Native Language.....	242
7.2.1.2 Memory Management APIs Provided by the Matrix Module.....	242
7.2.2 Host-Device Data Transmission.....	248
7.2.3 Usage of DVPP.....	251
7.2.3.1 Image and Video Encoding and Decoding.....	251
7.2.3.2 Image Cropping and Resizing.....	251
7.2.4 Model Conversion Pre-Processing Configuration.....	253
7.2.5 Batch and Timeout.....	253
7.2.6 Processing the Input and Output Data of Algorithm Inference.....	254
7.2.7 Optimization of Data Backhaul.....	254
7.3 Operator Usage Suggestions.....	255
7.4 Samples.....	256
7.4.1 Data Flow.....	256
7.4.2 Zero-Copy.....	257
7.5 FAQs.....	257
7.5.1 What Do I Do If an Exception Occurs Because a Smart Pointer Is Used to Automatically Free Memory but the Destructor Is Not Empty?.....	258
7.5.2 What Do I Do If an Exception Occurs Because a Smart Pointer Is Used to Manually Free Memory but the Destructor Is Not HIAI_DFree?.....	258
8 API Reference.....	260
8.1 Framework API Reference.....	260
8.1.1 Introduction.....	260
8.1.2 Non-Class Member Functions.....	260
8.1.2.1 custom_op_run.....	261
8.1.2.2 custom_op_compare.....	262
8.1.2.3 GetContext.....	263
8.1.2.4 AutoMappingFn.....	263
8.1.2.5 aicpu_run_func.....	264
8.1.3 Class Member Functions.....	265
8.1.3.1 Class StatusFactory.....	265
8.1.3.1.1 Instance.....	265
8.1.3.1.2 RegisterErrorNo.....	265
8.1.3.1.3 GetErrDesc.....	266
8.1.3.1.4 StatusFactory.....	266
8.1.3.1.5 ~StatusFactory.....	266
8.1.3.2 Class ErrorNoRegistrar.....	267
8.1.3.2.1 ErrorNoRegistrar.....	267
8.1.3.2.2 ~ErrorNoRegistrar.....	267
8.1.3.3 Class OpTypeContainer.....	267
8.1.3.3.1 Instance.....	268
8.1.3.3.2 Register.....	268
8.1.3.3.3 IsExisting.....	268

8.1.3.3.4 OpTypeContainer.....	269
8.1.3.4 Class OpTypeRegistrar.....	269
8.1.3.4.1 OpTypeRegistrar.....	269
8.1.3.4.2 ~OpTypeRegistrar.....	269
8.1.3.5 Class OpRegistrationData.....	270
8.1.3.5.1 OpRegistrationData.....	270
8.1.3.5.2 ~OpRegistrationData.....	270
8.1.3.5.3 FrameworkType.....	270
8.1.3.5.4 OriginOpType.....	271
8.1.3.5.5 ParseParamsFn.....	271
8.1.3.5.6 InferShapeAndTypeFn.....	272
8.1.3.5.7 UpdateOpDescFn.....	273
8.1.3.5.8 GetWorkspaceSizeFn.....	274
8.1.3.5.9 TEBinBuildFn.....	275
8.1.3.5.10 ImpliedType.....	276
8.1.3.5.11 Formats.....	277
8.1.3.5.12 WeightFormats.....	278
8.1.3.5.13 Finalize.....	278
8.1.3.6 Class OpRegistry.....	279
8.1.3.6.1 Instance.....	279
8.1.3.6.2 Register.....	279
8.1.3.6.3 GetImpliedType.....	279
8.1.3.6.4 GetOpTypeByImpliedType.....	280
8.1.3.6.5 GetFormats.....	280
8.1.3.6.6 GetWeightFormats.....	281
8.1.3.6.7 GetParseParamFunc.....	281
8.1.3.6.8 GetInferShapeFunc.....	282
8.1.3.6.9 GetGetWorkspaceSizeFunc.....	282
8.1.3.6.10 GetUpdateOpDescFunc.....	282
8.1.3.6.11 GetBuildTeBinFunc.....	283
8.1.3.6.12 GetTransWeightFunc.....	283
8.1.3.7 Class OpReceiver.....	283
8.1.3.7.1 OpReceiver.....	284
8.1.3.7.2 ~OpReceiver.....	284
8.1.4 Macro Definitions.....	284
8.1.4.1 Macro for Generating Error Codes.....	284
8.1.4.2 Macro for Generating Error Codes of the Common Module.....	285
8.1.4.3 Macro for Generating Error Codes of the OMG Module.....	285
8.1.4.4 Macro for Generating Error Codes of the OME Module.....	286
8.1.4.5 Macro for Generating Error Codes of the CALIBRATION Module.....	286
8.1.4.6 Macro for Obtaining Error Code Description.....	287
8.1.4.7 Macro for Registering the Operator Type.....	287

8.1.4.8 Macro Indicating Whether an Operator Type Exists.....	288
8.1.4.9 Macro for Registering an Operator.....	288
8.1.4.9.1 REGISTER_CUSTOM_OP.....	288
8.1.4.10 Macros for Marking External APIs.....	288
8.1.4.10.1 FMK_FUNC_HOST_VISIBILITY.....	289
8.1.4.10.2 FMK_FUNC_DEV_VISIBILITY.....	289
8.2 Matrix API Reference.....	289
8.2.1 Overview.....	289
8.2.2 Process Orchestration APIs.....	290
8.2.2.1 Process Connection APIs (C Language).....	290
8.2.2.1.1 HIAI_Init (Common with C++).....	291
8.2.2.1.2 HIAI_CreateGraph.....	291
8.2.2.1.3 HIAI_DestroyGraph.....	292
8.2.2.2 Process Connection APIs (C++ Language).....	293
8.2.2.2.1 Graph::GetInstance.....	293
8.2.2.2.2 Graph::CreateGraph (Creating a Graph Based on the Configuration File).....	293
8.2.2.2.3 Graph::CreateGraph (Creating a Graph Based on the Configuration File and Writing the Generated Graph Back to the List).....	294
8.2.2.2.4 Graph::CreateGraph (Creating a Graph Based on the Protobuf Data Format).....	295
8.2.2.2.5 Graph::DestroyGraph.....	296
8.2.2.2.6 Graph::UpdateEngineConfig.....	297
8.2.2.3 Data Sending APIs (C Language).....	298
8.2.2.3.1 HIAI_SetDataRecvFunctor.....	298
8.2.2.3.2 HIAI_C_SendData.....	299
8.2.2.4 Data Sending APIs (C++ Language).....	300
8.2.2.4.1 Graph::SetDataRecvFunctor.....	300
8.2.2.4.2 Graph::SendData.....	301
8.2.2.4.3 HIAI_SendData.....	303
8.2.2.5 API Usage Example.....	304
8.2.2.5.1 Example of Using the C APIs.....	304
8.2.2.5.2 Example of Using C++ APIs (Creating a Graph Based on the Configuration File).....	306
8.2.2.5.3 Usage Example for C++ APIs (Creating a Graph Based on the Configuration File and Writing the Generated Graph to the List).....	308
8.2.3 Engine Implementation APIs (C++ Language).....	310
8.2.3.1 Constructor and Destructor Functions of an Engine.....	310
8.2.3.2 Engine::Init.....	310
8.2.3.3 Macro: HIAI_DEFINE_PROCESS.....	312
8.2.3.4 Macro: HIAI_IMPL_ENGINE_PROCESS.....	312
8.2.3.5 Engine::SetDataRecvFunctor.....	314
8.2.3.6 Engine::SendData.....	315
8.2.3.7 Calling Example.....	316
8.2.4 Model Manager APIs (C++ Language).....	318
8.2.4.1 Offline Model Manager.....	318

8.2.4.1.1 AIModelManager::Init.....	318
8.2.4.1.2 AIModelManager::SetListener.....	320
8.2.4.1.3 AIModelManager::Process.....	321
8.2.4.1.4 AIModelManager::CreateOutputTensor.....	322
8.2.4.1.5 AIModelManager::CreateInputTensor.....	322
8.2.4.1.6 AIModelManager::IsPreAllocateOutputMem.....	323
8.2.4.1.7 AIModelManager::GetModelIOTensorDim.....	323
8.2.4.1.8 AIModelManager::GetMaxUsedMemory.....	324
8.2.4.1.9 AISimpleTensor::SetBuffer.....	325
8.2.4.1.10 AITensorFactory::CreateTensor.....	326
8.2.4.1.11 Calling Example.....	327
8.2.4.2 AIPP Configuration APIs.....	328
8.2.4.2.1 Overview.....	329
8.2.4.2.2 SetDynamicInputIndex.....	329
8.2.4.2.3 SetDynamicInputEdgeIndex.....	330
8.2.4.2.4 SetInputFormat.....	330
8.2.4.2.5 SetCscParams (Setting the Default Parameter Value).....	331
8.2.4.2.6 SetCscParams (Setting a Parameter Value as Required).....	333
8.2.4.2.7 SetRbuvSwapSwitch.....	335
8.2.4.2.8 SetAxSwapSwitch.....	336
8.2.4.2.9 SetSrcImageSize.....	337
8.2.4.2.10 SetCropParams.....	337
8.2.4.2.11 SetPaddingParams.....	338
8.2.4.2.12 SetDtcPixelMean.....	340
8.2.4.2.13 SetDtcPixelMin.....	341
8.2.4.2.14 SetPixelVarReci.....	342
8.2.4.2.15 SetInputDynamicAIPP.....	343
8.2.4.2.16 GetDynamicInputIndex.....	344
8.2.4.2.17 GetDynamicInputEdgeIndex.....	345
8.2.4.2.18 Calling Example.....	346
8.2.4.3 Data Types.....	348
8.2.4.3.1 AIConfigItem.....	348
8.2.4.3.2 AIConfig.....	348
8.2.4.3.3 AITensorParaDescription.....	348
8.2.4.3.4 AITensorDescription.....	349
8.2.4.3.5 AITensorDescriptionList.....	349
8.2.4.3.6 AIModelDescription.....	349
8.2.4.3.7 AINNNodeDescription.....	350
8.2.4.3.8 AINNNodeDescriptionList.....	350
8.2.4.3.9 AIAPIDescription.....	350
8.2.4.3.10 AIAPIDescriptionList.....	350
8.2.4.3.11 AIOPDescription.....	351

8.2.4.3.12 AIOPDescriptionList.....	351
8.2.4.3.13 NodeDesc.....	351
8.2.4.3.14 EngineDesc.....	351
8.2.4.3.15 GraphInitDesc.....	352
8.2.4.3.16 GeneralFileBuffer.....	352
8.2.4.3.17 AIContext.....	352
8.2.4.3.18 TensorDimension.....	353
8.2.4.3.19 IAIListener.....	354
8.2.4.3.20 IAITensor.....	354
8.2.4.4 Other Compilation Dependent APIs.....	355
8.2.4.4.1 AIAlgAPIFactory.....	355
8.2.4.4.2 AIAlgAPIRegistrar.....	356
8.2.4.4.3 REGISTER_ALG_API_UNIQUE.....	356
8.2.4.4.4 REGISTER_ALG_API.....	356
8.2.4.4.5 AIModelManager.....	356
8.2.4.4.6 getModelInfo.....	358
8.2.4.4.7 IAINNNode.....	358
8.2.4.4.8 AINNNodeFactory.....	359
8.2.4.4.9 AINNNodeRegistrar.....	360
8.2.4.4.10 REGISTER_NN_NODE.....	361
8.2.4.4.11 AITensorGetBytes.....	361
8.2.4.4.12 AITensorFactory.....	361
8.2.4.4.13 REGISTER_TENSOR_CREATER_UNIQUE.....	362
8.2.4.4.14 REGISTER_TENSOR_CREATER.....	362
8.2.4.4.15 AISimpleTensor.....	362
8.2.4.4.16 AINeuralNetworkBuffer.....	364
8.2.4.4.17 AllImageBuffer.....	366
8.2.4.4.18 HIAILog.....	367
8.2.4.4.19 HIAI_ENGINE_LOG.....	368
8.2.4.5 Exception Handling.....	369
8.2.5 Auxiliary APIs.....	369
8.2.5.1 Data Receiving APIs (C++ Language).....	369
8.2.5.1.1 Obtaining the Number of Devices.....	369
8.2.5.1.2 Obtaining the ID of the First Device.....	370
8.2.5.1.3 Obtaining the ID of the First Graph.....	370
8.2.5.1.4 Obtaining the ID of the Next Device.....	371
8.2.5.1.5 Obtaining the ID of the Next Graph.....	372
8.2.5.1.6 Obtaining the Engine Pointer.....	373
8.2.5.1.7 Obtaining the Graph ID.....	373
8.2.5.1.8 Obtaining the Device ID of a Graph.....	373
8.2.5.1.9 Obtaining the Graph ID of an Engine.....	374
8.2.5.1.10 Obtaining the Maximum Queue Size of an Engine.....	374

8.2.5.1.11 Obtaining the Current Queue Size of a Specified Port of the Engine.....	374
8.2.5.1.12 Parsing the Matrix Configuration File.....	375
8.2.5.1.13 Obtaining the PCIe Information.....	375
8.2.5.1.14 Obtaining the Version Number.....	376
8.2.5.1.15 Obtaining the OamConfig Smart Pointer.....	377
8.2.5.2 Data Type Serialization and Deserialization (C++ Language).....	377
8.2.5.2.1 Macro: HIAI_REGISTER_DATA_TYPE.....	377
8.2.5.2.2 Macro: HIAI_REGISTER_TEMPLATE_DATA_TYPE.....	378
8.2.5.2.3 Macro: HIAI_REGISTER_SERIALIZE_FUNC.....	379
8.2.5.2.4 Graph::ReleaseDataBuffer.....	381
8.2.5.2.5 API Usage Example.....	381
8.2.5.3 Memory Management (C Language).....	384
8.2.5.3.1 HIAI_DMAlloc.....	384
8.2.5.3.2 HIAI_DFree.....	386
8.2.5.3.3 HIAI_DVPP_DMAlloc.....	387
8.2.5.3.4 HIAI_DVPP_DFree.....	388
8.2.5.4 Memory Management (C++ Language).....	389
8.2.5.4.1 HIAIMemory::HIAI_DMAlloc.....	389
8.2.5.4.2 HIAIMemory::HIAI_DFree.....	392
8.2.5.4.3 HIAIMemory::IsDMAlloc.....	393
8.2.5.4.4 HIAIMemory::HIAI_DVPP_DMAlloc.....	393
8.2.5.4.5 HIAIMemory::HIAI_DVPP_DFree.....	395
8.2.5.5 Logs (C++).....	396
8.2.5.5.1 Error Code Registration.....	396
1. HIAI_DEF_ERROR_CODE.....	396
2. API Usage Example.....	396
8.2.5.5.2 Log Printing.....	397
1. Log Printing Format 1.....	397
2. Log Printing Format 2.....	398
3. Log Printing Format 3.....	398
4. Log Printing Format 4.....	399
5. Log Printing Format 5.....	399
6. Log Printing Format 6.....	400
7. Log Printing Format 7.....	401
8. Log Printing Format 8.....	401
8.2.5.6 MultiTypeQueue API for Queue Management (C++ Language).....	402
8.2.5.6.1 MultiTypeQueue Constructor.....	402
8.2.5.6.2 PushData.....	403
8.2.5.6.3 FrontData.....	403
8.2.5.6.4 PopData.....	404
8.2.5.6.5 PopAllData.....	404
8.2.5.6.6 API Usage Example.....	405

8.2.5.7 Event Registration API (C++ Language).....	405
8.2.5.7.1 Graph::RegisterEventHandle.....	405
8.2.5.8 Others (C++ Language).....	406
8.2.5.8.1 DataRecvInterface::RecvData.....	406
8.2.5.8.2 Graph::SetPublicKeyForSignatureEncryption.....	407
8.2.6 Python APIs.....	408
8.2.7 Appendix.....	411
8.2.7.1 APIs to Be Deleted in Later Versions.....	411
8.2.7.1.1 Low-Power APIs (C++ Language).....	411
1. PowerState::SubscribePowerStateEvent.....	411
2. PowerState::UnsubscribePowerStateEvent.....	411
8.2.7.1.2 Data Types.....	412
1. Enumeration: PowerState::DEVICE_POWER_STATE.....	412
2. PowerState::PowerStateNotifyCallbackT.....	412
8.2.7.2 Matrix Data Types.....	412
8.2.7.2.1 Protobuf Data Types.....	412
8.2.7.2.2 Matrix Custom Data Types.....	417
8.2.7.3 Data Structures Registered in the Matrix.....	417
8.2.7.4 Example.....	432
8.2.7.4.1 Orchestration Configuration Example.....	433
8.2.7.4.2 Example of Data Transmission Based on Optimized Performance.....	434
8.3 DVPP API Reference.....	435
8.3.1 Overview.....	435
8.3.1.1 API Introduction.....	435
8.3.1.2 API List.....	436
8.3.1.3 VPC Function.....	438
8.3.1.4 JPEGE Function.....	444
8.3.1.5 JPEGD Function.....	445
8.3.1.6 PNGD Function.....	446
8.3.1.7 VDEC Function.....	447
8.3.1.8 VENC Function.....	449
8.3.1.9 Input and Output Memory.....	449
8.3.2 VPC/JPEGE/JPEGD/PNGD Function Interfaces.....	451
8.3.2.1 CreateDvppApi.....	451
8.3.2.2 DvppCtl.....	452
8.3.2.2.1 API Description.....	452
8.3.2.2.2 VPC Parameters.....	454
8.3.2.2.3 JPEGE Parameters.....	461
8.3.2.2.4 JPEGD Parameters.....	463
8.3.2.2.5 PNGD Parameters.....	468
8.3.2.2.6 Parameters for Querying the DVPP Engine.....	470
8.3.2.3 DestroyDvppApi.....	473

8.3.2.4 DvppGetOutParameter.....	474
8.3.3 VDEC Function Interfaces.....	475
8.3.3.1 General Description.....	475
8.3.3.2 CreateVdecApi.....	475
8.3.3.3 VdecCtl.....	476
8.3.3.4 DestroyVdecApi.....	480
8.3.4 VENC Function Interfaces.....	481
8.3.4.1 General Description.....	481
8.3.4.2 CreateVenc.....	481
8.3.4.3 RunVenc.....	483
8.3.4.4 DestroyVenc.....	485
8.3.5 Compatibility with the Functions of Earlier Versions.....	485
8.3.5.1 Compatibility.....	485
8.3.5.2 VPC Functions and Parameters.....	486
8.3.5.3 CMDLIST Functions and Parameters.....	502
8.3.5.4 VENC Functions and Parameters.....	508
8.3.6 Calling Example.....	510
8.3.6.1 Implementing the VPC Function.....	510
8.3.6.2 Implementing the JPEGE Function.....	519
8.3.6.3 Implementing the JPEGD Function.....	522
8.3.6.4 Implementing the PNGD Function.....	525
8.3.6.5 Implementing the VDEC Function.....	526
8.3.6.6 Implementing the VENC Function.....	530
8.3.7 Data Types.....	532
8.3.7.1 Structures in VpcUserImageConfigure.....	532
8.3.7.2 Structures and Classes in vdec_in_msg.....	535
8.3.7.3 Structures in IMAGE_CONFIG.....	538
8.3.7.4 Structures in dvpp_engine_capability_stru.....	540
8.3.7.5 Structures in vpc_in_msg.....	548
8.3.7.5.1 RDMACHANNEL Structure.....	548
8.3.7.5.2 VpcTurningReverse Structure.....	548
8.3.8 Appendix.....	548
8.3.8.1 Auxiliary Function APIs.....	549
8.3.8.2 List of APIs that Are Not Recommended.....	550
8.3.8.3 Sample Usage of the DVPP Executor.....	551
8.3.8.3.1 Environment Preparation.....	551
8.3.8.3.2 Sample Input Parameters of the DVPP Executor.....	551
8.3.8.3.3 VPC Instructions.....	554
1. VPC Basic Function 1.....	554
2. VPC Basic Function 2.....	555
8.3.8.3.4 VDEC Usage.....	555
8.3.8.3.5 Usage of the VENC.....	556

8.3.8.3.6 Usage of the JPEG	556
8.3.8.3.7 Usage of the JPEGD	557
8.3.8.3.8 Usage of PNGD	557
8.3.8.3.9 JPEGD+VPC+JPEG Cascading	558
8.3.8.4 Exception Handling	558
8.3.8.5 Acronyms	558
8.4 GE API Reference	559
8.4.1 Introduction	559
8.4.2 Attribute APIs	560
8.4.2.1 SetAttr	560
8.4.2.2 GetAttr	561
8.4.2.3 HasAttr	561
8.4.2.4 DelAttr	562
8.4.2.5 SetName	563
8.4.2.6 GetName	564
8.4.2.7 GetItem	564
8.4.2.8 GetValueType	565
8.4.2.9 IsEmpty	565
8.4.2.10 Copy	566
8.4.2.11 MutableTensor	567
8.4.2.12 MutableListTensor	567
8.4.2.13 InitDefault	568
8.4.3 Buffer APIs	569
8.4.3.1 ClearBuffer	569
8.4.3.2 GetData	569
8.4.3.3 GetSize	570
8.4.3.4 CopyFrom	571
8.4.4 Graph APIs	571
8.4.4.1 SetInputs	571
8.4.4.2 SetOutputs	572
8.4.4.3 IsValid	573
8.4.4.4 AddOp	573
8.4.4.5 FindOpByName	574
8.4.4.6 CheckOpByName	575
8.4.4.7 GetAllOpName	576
8.4.5 Model APIs	576
8.4.5.1 SetName	577
8.4.5.2 GetName	577
8.4.5.3 SetVersion	578
8.4.5.4 GetVersion	578
8.4.5.5 SetPlatformVersion	579
8.4.5.6 GetPlatformVersion	579

8.4.5.7 GetGraph.....	580
8.4.5.8 SetGraph.....	581
8.4.5.9 Save.....	581
8.4.5.10 Load.....	582
8.4.5.11 IsValid.....	583
8.4.6 Operator APIs.....	583
8.4.6.1 Operator Constructor.....	583
8.4.6.2 GetName.....	584
8.4.6.3 SetInput (srcOprt Has Only One Output).....	585
8.4.6.4 SetInput (srcOprt Has Multiple Outputs).....	585
8.4.6.5 GetInputDesc.....	586
8.4.6.6 TryGetInputDesc.....	587
8.4.6.7 UpdateInputDesc.....	588
8.4.6.8 GetOutputDesc.....	589
8.4.6.9 UpdateOutputDesc.....	590
8.4.6.10 GetDynamicInputDesc.....	591
8.4.6.11 UpdateDynamicInputDesc.....	592
8.4.6.12 GetDynamicOutputDesc.....	593
8.4.6.13 UpdateDynamicOutputDesc.....	594
8.4.6.14 SetAttr.....	595
8.4.6.15 GetAttr.....	597
8.4.7 Shape APIs.....	598
8.4.7.1 GetDimNum.....	598
8.4.7.2 GetDim.....	598
8.4.7.3 GetDims.....	599
8.4.7.4 SetDim.....	600
8.4.7.5 GetShapeSize.....	600
8.4.8 Tensor APIs.....	601
8.4.8.1 GetTensorDesc.....	601
8.4.8.2 MutableTensorDesc.....	602
8.4.8.3 SetTensorDesc.....	602
8.4.8.4 GetData.....	603
8.4.8.5 MutableData.....	604
8.4.8.6 SetData.....	604
8.4.8.7 Clone.....	605
8.4.9 TensorDesc APIs.....	606
8.4.9.1 Update.....	606
8.4.9.2 GetShape.....	608
8.4.9.3 MutableShape.....	608
8.4.9.4 SetShape.....	609
8.4.9.5 GetFormat.....	609
8.4.9.6 SetFormat.....	610

8.4.9.7 GetDataType.....	611
8.4.9.8 SetDataType.....	611
8.4.9.9 Clone.....	612
8.4.9.10 IsValid.....	612
8.4.10 Operator Registration APIs	613
8.4.10.1 REG_OP.....	614
8.4.10.2 ATTR.....	614
8.4.10.3 REQUIRED_ATTR.....	617
8.4.10.4 INPUT.....	619
8.4.10.5 OPTIONAL_INPUT.....	620
8.4.10.6 DYNAMIC_INPUT.....	621
8.4.10.7 OUTPUT.....	622
8.4.10.8 DYNAMIC_OUTPUT.....	623
8.4.10.9 INFER_SHAPE_AND_TYPE.....	624
8.4.10.10 ATTR_ALL_VERIFY.....	626
8.4.10.11 OP_END.....	627
8.4.10.12 List of Built-in Operators.....	627
8.4.11 Model Building APIs.....	628
8.4.11.1 CreateModelBuff.....	628
8.4.11.2 BuildIRModel.....	629
8.4.11.3 ReleaseModelBuff.....	630
8.5 TE API Reference.....	631
8.5.1 Introduction.....	631
8.5.2 Description.....	631
8.5.3 Compute APIs.....	637
8.5.3.1 te.lang.cce.vadd(lhs, rhs).....	637
8.5.3.2 te.lang.cce.vsub(lhs, rhs).....	637
8.5.3.3 te.lang.cce.vmul(lhs, rhs).....	638
8.5.3.4 te.lang.cce.vmin(lhs, rhs).....	638
8.5.3.5 te.lang.cce.vmax(lhs, rhs).....	638
8.5.3.6 te.lang.cce.vor(lhs, rhs).....	639
8.5.3.7 te.lang.cce.vand(lhs, rhs).....	639
8.5.3.8 te.lang.cce.vadds(raw_tensor, scalar).....	640
8.5.3.9 te.lang.cce.vmults(raw_tensor, scalar).....	640
8.5.3.10 te.lang.cce.vlog(raw_tensor).....	641
8.5.3.11 te.lang.cce.vexp(raw_tensor).....	641
8.5.3.12 te.lang.cce.vabs(raw_tensor).....	641
8.5.3.13 te.lang.cce.vrec(raw_tensor).....	642
8.5.3.14 te.lang.cce.cast_to(data, dtype, f1628IntegerFlag=False).....	642
8.5.3.15 te.lang.cce.vrelu(raw_tensor).....	643
8.5.3.16 te.lang.cce.vnot(raw_tensor).....	644
8.5.3.17 te.lang.cce.vaxpy(lhs, rhs, scalar).....	644

8.5.3.18 te.lang.cce.vmla(x, y, z).....	645
8.5.3.19 te.lang.cce.vmadd(x, y, z).....	645
8.5.3.20 te.lang.cce.vmaddrelu(x, y, z).....	646
8.5.3.21 te.lang.cce.ceil(raw_tensor).....	646
8.5.3.22 te.lang.cce.floor(raw_tensor).....	646
8.5.3.23 te.lang.cce.round(raw_tensor).....	647
8.5.3.24 te.lang.cce.sum(raw_tensor, axis, keepdims=False).....	647
8.5.3.25 te.lang.cce.reduce_min(raw_tensor, axis, keepdims=False).....	648
8.5.3.26 te.lang.cce.reduce_max(raw_tensor, axis, keepdims=False).....	648
8.5.3.27 te.lang.cce.reduce_prod(raw_tensor, axis, keepdims=False).....	649
8.5.3.28 te.lang.cce.broadcast(var, shape, output_dtype=None).....	649
8.5.3.29 te.lang.cce.unsorted_segment_sum(tensor, segment_ids, num_segments, init_value=0).....	650
8.5.3.30 te.lang.cce.unsorted_segment_mean(tensor, segment_ids, num_segments, init_value=0).....	652
8.5.3.31 te.lang.cce.unsorted_segment_prod(tensor, segment_ids, num_segments, init_value=0).....	653
8.5.3.32 te.lang.cce.unsorted_segment_min(tensor, segment_ids, num_segments, init_value=0).....	654
8.5.3.33 te.lang.cce.unsorted_segment_max(tensor, segment_ids, num_segments, init_value=0).....	656
8.5.3.34 te.lang.cce.concat(raw_tensors, axis).....	657
8.5.3.35 te.lang.cce.conv(*args).....	658
8.5.3.36 te.lang.cce.compute_four2five(input, raw_shape_4D).....	659
8.5.3.37 te.lang.cce.compute_five2four(input, raw_shape_4D).....	660
8.5.3.38 te.lang.cce.matmul(tensor_a, tensor_b, trans_a=False, trans_b=False, alpha_num=1.0, beta_num=0.0, tensor_c=None).....	660
8.5.4 Build APIs.....	661
8.5.4.1 te.lang.cce.auto_schedule(outs).....	661
8.5.4.2 te.lang.cce.cce_build_code(sch, config_map = {}).....	662
8.5.5 Convergence API.....	662
8.5.5.1 bool BuildTeCustomOp(std::string ddkVer, std::string opName, std::string opPath, std::string opFuncName, const char *format, ...).....	663
8.5.6 APIs that Compilation Depends on.....	663
8.5.7 Instructions.....	664
8.5.7.1 Usage Example.....	664
8.5.7.2 Exception Handling.....	665
9 Command Reference.....	666
9.1 IDE-daemon-client Command Reference.....	666
9.1.1 Introduction.....	666
9.1.2 Sending a Command to the Device by Using IDE Daemon.....	667
9.1.3 Sending a Command to the Device by Using IDE Daemon.....	676
9.1.4 Transferring a File to the Device by Using IDE Daemon.....	685
9.1.5 Transferring a File to the Host by Using IDE Daemon.....	688
9.1.6 Obtaining a File from the Host by Using IDE Daemon.....	690
9.1.7 Detecting the Heartbeat Between IDE Daemon and the Host.....	692
9.1.8 Setting Log Levels.....	693
9.1.9 Sending Profiling Items to the Host by Using IDE Daemon.....	697

9.1.10 Collecting Dump Information Recorded by the Black Box on the Host..... 698

9.1.11 Obtaining the Information About the Device Connected to the Host..... 702

9.1.12 Generating a Character String Encryption Files..... 704

9.1.13 Synchronizing the Time Between IDE Daemon and the Host..... 704

9.1.14 Viewing the Help Information..... 706

10 Appendix..... 707

10.1 Getting Help..... 707

10.1.1 Preparing for Contacting Huawei..... 707

10.1.2 Contacting Huawei Technical Support..... 707

10.1.3 Preparing for Debugging..... 707

10.1.4 Using Product Documentation..... 708

10.1.5 Getting Help from Website..... 708

10.1.6 Ways to Contact Huawei..... 708

10.2 General Disclaimer..... 709

1 About This Document

Symbol Conventions

The symbols that may be found in this document are defined as follows.

Symbol	Description
	Indicates a hazard with a high level of risk which, if not avoided, will result in death or serious injury.
	Indicates a hazard with a medium level of risk which, if not avoided, could result in death or serious injury.
	Indicates a hazard with a low level of risk which, if not avoided, could result in minor or moderate injury.
	Indicates a potentially hazardous situation which, if not avoided, could result in equipment damage, data loss, performance deterioration, or unanticipated results. NOTICE is used to address practices not related to personal injury.
 NOTE	Supplements the important information in the main text. NOTE is used to address information not related to personal injury, equipment damage, and environment deterioration.

Change History

Issue	Release Date	Description
01	2019-09-30	This issue is the first official release.

2 Development Process

This document describes the process of developing application software, including DDK environment setup, model conversion, operator development, and development.

DDK Environment Setup

The DDK provides users with algorithm development tool kit based on Ascend AI processors. The tool kit can be used to build the compilation environment of related projects. For details, see [3 DDK Environment Setup](#).

Model Conversion

The offline model generator (OMG) converts the network models of open-source frameworks, such as Caffe and TensorFlow, into offline models supported by Ascend AI processors. During model conversion, operator scheduling tuning, weight data rearrangement, quantization compression, and memory usage tuning can be implemented, and model preprocessing can be completed without using devices. For details, see [4 Model Conversion](#).

Operator Development

If you want to develop operators to improve computing performance or the Ascend AI software stack does not support operators in a model, you can develop operators by yourself. For details about the development process, see [5 Operator Development](#). For details about the APIs of the TE custom operator development framework, see [8.5 TE API Reference](#). For details about the APIs of the custom operator plug-in development, see [8.4 GE API Reference](#).

Application Software Development

After the preceding preparations are complete, you can develop application software based on the Ascend AI software stack. For details about the development process and method, see [6 Application Software Development](#).

During development, you need to invoke the Ascend AI software stack components shown in [Table 2-1](#) to implement application development and model inference.

Table 2-1 Ascend AI software stack components

Component	Description	Reference
Framework	Converts the neural network model into a form supported by the Ascend AI processor, integrates the converted model with the Ascend AI processor, and guides the neural network to achieve its optimal performance.	8.1 Framework API Reference
Process orchestrator (Matrix)	Implements the neural network on Ascend AI processors and coordinates the entire neural network validation process.	8.2 Matrix API Reference
Digital vision pre-processing (DVPP)	The Da Vinci architecture has fixed requirements on the input data format. If the data does not meet the input format and resolution requirements, the DVPP needs to be called to convert the format to ensure subsequent inference services.	8.3 DVPP API Reference
Data backhaul tool (IDE-daemon-hiai)	Provides APIs for the Matrix and Framework to invoke for the DVPP to preprocess data and return neural network data.	9.1 IDE-daemon-client Command Reference

To ensure high performance in data transmission, DVPP processing, and model inference during application development, you can develop applications based on the suggestions provided in [7 High Performance Application Programming](#) to quickly build your own high-performance applications.

In addition, to improve code development efficiency, some code samples and tools are open-source for software components and inference services of Ascend AI processors. You can log in to <https://www.huaweicloud.com/intl/en-us/ascend/resources/CodeSamples> to obtain the code samples and tools.

3 DDK Environment Setup

[3.1 DDK Installation Guide \(CentOS, x86\)](#)

[3.2 DDK Installation Guide \(Ubuntu, x86\)](#)

3.1 DDK Installation Guide (CentOS, x86)

3.1.1 Introduction

This document describes the method of separately installing the device development kit (DDK) and troubleshooting of common problems. It is applicable to development in command line interface (CLI) mode.

The DDK is an NPU-based digital developer kit that can be used to build a compilation environment for projects. Different release packages integrate the DDKs of different NPU forms. The DDK of the current version contains the dependency libraries and header files of components such as the TE, DVPP, and process orchestration. Users can compile the project files by using **makefile**.

3.1.2 Obtaining Software Packages

Before installing the tool, prepare the following software package:

DDK installation package: Visit <https://support.huawei.com/enterprise>, input the product name in the search box, and press **Enter** or click the search icon. In the displayed window, click the **Software Download** tab to obtain the desired installation package.

Table 3-1 describes the required software package.

Table 3-1 Overview of the software package

Installation Package	Description	Integrity Verification File
MSpore_DDK-{version}-x86_64.centOS7.4-x86_64.centOS7.4-aarch64.miniOS.tar.gz	DDK installation package	MSpore_DDK-{version}-x86_64.centOS7.4-x86_64.centOS7.4-aarch64.miniOS.tar.gz.asc

 NOTE

{version} indicates the DDK version number.

3.1.3 Environment Preparation

Environment Requirements

The DDK installation must meet the following environment requirements on the hardware and operating system (OS).

Table 3-2 CentOS version information

Category	Version	Obtaining Method	Precautions
OS	7.4	Download the required version from http://vault.centos.org/7.4.1708/isos/x86_64/ . The recommended version is CentOS-7-x86_64-DVD-1708.iso	The DDK installation and offline installation packages are adapted and tested based on the Infrastructure Server of CentOS. Therefore, you must use the Infrastructure Server software for installation. Other software, for example, Minimal Install or Development and Creative Workstation, cannot be used.
Python	Python2 : 2.7+ Python3 : 3.5+	For details, see Installing Dependencies .	-

(Optional) Preparing the DDK Installation User

You can install the DDK as any user (**root** or non-**root**).

- If the **root** user is used for installation, skip this section.
- If you use an existing non-**root** user to install the DDK, run the following command as the **root** user to grant the existing user with permission 750 on the **\$HOME** directory:
`chmod 750 /home/username`
- If you want to install the DDK as a new non-**root** user, perform the following steps as the **root** user: This document describes how to install the DDK as a new non-**root** user.

- a. Run the following command to create a user for DDK installation and configure the **\$HOME** directory of the user:

```
useradd -d /home/username -m username
```

- b. Run the following command to set the password:
`passwd username`

- c. Run the following command to set the permission:
`chmod 750 /home/username`

NOTE

username indicates the user name for installing the DDK. The **umask** value of the user cannot be greater than **0027**.

- To view the **umask** value, run the **umask** command.
- To change the **umask** value, run the **umask New value** command.

If the created non-**root** user is **HwHiAiUser**, the DDK is deployed on the host server, and **HwHiAiUser** is used for offline model conversion, logs are stored in the **var/dlog** directory on the host by default. To enable the output of logs to the screen by default, you can set the following environment variable:

```
export SLOG_PRINT_TO_STDOUT=1
```

- Change the size of the file descriptor.
 - a. Open the system configuration file **/etc/security/limits.conf** as the **root** user by running the following command:
`vi /etc/security/limits.conf`
 - b. Add the following content to the end of the **limits.conf** file:

```
* soft nproc 65535
* hard nproc 65535
* soft nofile 65535
* hard nofile 65535
```

NOTE

* indicates all users. You are advised to replace it with the name of the user who installs the Mind Studio project.

- c. Run the **:wq!** command to save the file and exit.

(Optional) Configuring Permissions of the DDK Installation User

When a non-**root** user is used for installation, perform the following operations. Skip the following operations if the **root** user is used for installation.

Before installing the DDK, you need to download dependent software with the **sudo yum** permission. Perform the following operations as the **root** user:

1. Run the following commands to open the **/etc/sudoers** file:

```
chmod u+w /etc/sudoers  
vi /etc/sudoers
```
2. Under **root ALL=(ALL) ALL** in the file, add the following content:

```
username ALL=(ALL:ALL) NOPASSWD:SETENV: /usr/bin/yum
```

Replace **username** with the name of the common user who executes the installation script.
3. Run the **:wq!** command to save the file.
4. Run the following command to revoke the write permission on the **/etc/sudoers** file:

```
chmod u-w /etc/sudoers
```

NOTE

Ensure that the last line of the **/etc/sudoers** file is **#includedir /etc/sudoers.d**. Otherwise, add it manually.

Configuring the Network

If CentOS cannot be connected to the network through a remote terminal after CentOS is started, perform the following steps to configure the network adapter:

Run the following command as the **root** user to open the NIC configuration file:

```
vi /etc/sysconfig/network-scripts/ifcfg-enp2s0f0
```

NOTE

Replace **ifcfg-enp2s0f0** with the actual name of the network adapter in use.

Change the value of **ONBOOT** to **yes**, and set network parameters as shown in the following figure.

Figure 3-1 Setting network parameters

```
TYPE=Ethernet
PROXY_METHOD=none
BROWSER_ONLY=no
BOOTPROTO=none
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
IPV6_ADDR_GEN_MODE=stable-privacy
NAME=enp2s0f0
UUID=71b3c94b-e746-44f8-a2e8-21b452746dba
DEVICE=enp2s0f0
ONBOOT=yes
IPADDR=10.174.28.173
PREFIX=22
GATEWAY=10.174.28.1
DNS1=10.129.2.34
IPV6_PRIVACY=no
```

NOTE

BOOTPROTO indicates the IP address type of the device. For a static IP address, set this parameter to **static** or **none**. For a dynamic IP address, set this parameter to **dhcp** so that the IP address can be obtained automatically.

Save the file and exit. Run the following command to restart the network service:

```
service network restart
```

Configuring the Sources

DDK installation requires related dependencies, which must be downloaded in advance. Ensure that the server where the DDK is installed can successfully connect to the network. Perform the following steps as the **root** user:

Step 1 Run the following command to check source validity:

```
yum makecache
```

If an error is reported during the command execution, check whether the network connection is normal or replace the sources in the **/etc/yum.repos.d/CentOS-Base.repo** file with valid ones.

Step 2 Configure special sources.

- Install epel-release to support Python download and installation before installing ius-release.
 - If it is the first installation, run the following command:

```
yum install -y epel-release
```

- If it is not the first installation, run the following command:
`yum reinstall -y epel-release`

NOTE

If **epel-release** fails to be installed by using the **yum** source, rectify the fault by referring to **FAQs > What Do I Do If epel-release Installation Fails?**

- Run the following commands to install **ius-release**:
`wget https://centos7.iuscommunity.org/ius-release.rpm --no-check-certificate`
`rpm -vih ius-release.rpm`

If python35u cannot be installed during dependency installation, perform the following steps to replace **ius.repo**:

Configure the **ius** source to support python35u download and installation. The **ius** source is in the **/etc/yum.repos.d/ius.repo** file. Replace the following content with that in the **ius.repo** file:

```
[ius]
name=IUS Community Packages for Enterprise Linux 7 - $basearch
#baseurl=https://dl.iuscommunity.org/pub/ius/stable/CentOS/7/$basearch
mirrorlist=https://mirrors.iuscommunity.org/mirrorlist?repo=ius-centos7&arch=
$basearch&protocol=http
failovermethod=priority
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-IUS-7

[ius-debuginfo]
name=IUS Community Packages for Enterprise Linux 7 - $basearch - Debug
#baseurl=https://dl.iuscommunity.org/pub/ius/stable/CentOS/7/$basearch/debuginfo
mirrorlist=https://mirrors.iuscommunity.org/mirrorlist?repo=ius-centos7-debuginfo&arch=
$basearch&protocol=http
failovermethod=priority
enabled=0
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-IUS-7

[ius-source]
name=IUS Community Packages for Enterprise Linux 7 - $basearch - Source
#baseurl=https://dl.iuscommunity.org/pub/ius/stable/CentOS/7/SRPMS
mirrorlist=https://mirrors.iuscommunity.org/mirrorlist?repo=ius-centos7-
source&arch=source&protocol=http
failovermethod=priority
enabled=0
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-IUS-7
```

Step 3 Run the following commands to update the software information:

```
yum clean all
yum makecache
```

----End

Installing Dependencies

Run the **su - username** command to switch to the DDK installation user and perform the following operations to install components such as the GCC and Python on which the DDK depends:

Step 1 Run the following command to install the dependencies:

```
sudo -E yum install -y gcc-c++ cmake make python python-devel python2-pip python35u-devel python35u-pip
```

After the dependencies are installed, run the **python3** command to check whether Python 3 exists. If not, run the following commands to create a soft link from Python 3 to Python 3.5:

1. **which python3.5**: checks the installation path of Python 3.5.
2. **sudo ln -s /xxx/python3.5 /xxx/python3**: /xxx indicates the absolute path of Python 3.5 or Python 3.

NOTE

If a message is displayed indicating that you do not have the permission when you run the command, switch to the **root** user to configure a soft link, and then switch back to the common user.

Step 2 Install the Python dependencies.

- Before installing the software on which Python 2.7 depends, ensure that the preceding operations are successful. Run the **pip -V** command to check the pip version. If the following information is displayed, it indicates that Python 2.7 has been installed correctly:

```
pip 8.1.2 from /usr/lib/python2.7/site-packages (python 2.7)
```

The preceding information indicates that pip has been installed correctly. Run the following commands to install the software dependencies required by Python 2.7 (NumPy versions 1.11.0–1.16.0 are supported and the following uses **1.16.0** as an example):

```
pip install decorator --user  
pip install numpy==1.16.0 --user  
pip install setuptools --upgrade --user
```

- Before installing the software on which Python 3.5 depends, run the **pip3.5 -V** command to check the version information. If the following information is displayed, it indicates that Python 3.5 has been installed correctly:

```
pip 9.0.1 from /usr/lib/python3.5/site-packages (python 3.5)
```

Run the following commands to install the software on which Python 3.5 depends:

```
pip3.5 install numpy --user  
pip3.5 install decorator --user  
pip3.5 install setuptools --upgrade --user
```

NOTE

If the message "Import Error:cannot import name main" is displayed when you run the **pip3.5 -V** command, rectify the fault by referring to [3.1.5.1 What Do I Do If the Message "Import Error:cannot import name main" Is Displayed When the pip3/pip3.5 -V Command Is Run after pip3/pip3.5 Upgrade?](#)

----End

3.1.4 Installing the DDK

This section describes the installation method of the DDK and common operations.

3.1.4.1 Preparing for Installation

Uploading Installation Packages

Upload the following files to any directory under **\$HOME** as the DDK installation user:

- **MSpore_DDK-{version}-x86_64.centOS7.4-x86_64.centOS7.4-aarch64.miniOS.tar.gz**: DDK installation package
- **MSpore_DDK-{version}-x86_64.centOS7.4-x86_64.centOS7.4-aarch64.miniOS.tar.gz.asc**: file for verifying the DDK installation package

NOTE

Ensure that the preceding two files are in the same directory.

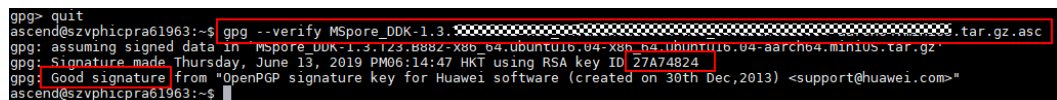
Verifying Software Package Integrity

Before installing a software package, you are advised to check whether the software package is incomplete or damaged due to network or storage device faults.

In the directory where the DDK installation package is stored, perform the following operations:

- Step 1** Configure the OpenPGP public keys. For details, see [3.1.4.3.3 Configuring the OpenPGP Public Key](#).
- Step 2** Run the following command as the DDK installation user to check the validity and integrity of the software package, as shown in [Figure 3-2](#):
- ```
gpg --verify "MSpore_DDK****.tar.gz.asc"
```

**Figure 3-2** Verifying the software package integrity



```
ascend@szvphicpra61963:~$ gpg --verify MSpore_DDK-1.3.123.B882-x86_64.ubuntu16.04-x86_64.ubuntu16.04-aarch64.miniOS.tar.gz.asc
gpg: assuming signed data in 'MSpore_DDK-1.3.123.B882-x86_64.ubuntu16.04-x86_64.ubuntu16.04-aarch64.miniOS.tar.gz'
gpg: Signature made Thursday, June 13, 2019 PM06:14:47 HKT using RSA key ID 27A74824
gpg: Good signature from 'OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>'
ascend@szvphicpra61963:~$
```

- In the returned information, **27A74824** indicates the public key ID.
- If the message **Good signature** is displayed without **WARNING** or **FAIL**, the signature is valid and the integrity verification is passed.
- If **WARNING** or **FAIL** is displayed, the verification fails. Rectify the fault by referring to the handling suggestions described in [3.1.5.4 What Do I Do If a WARNING or FAIL Result Is Returned in the Integrity Verification of a Software Package?](#).

#### NOTE

- In actual operations, *MSpore\_DDK\*\*\*\*.tar.gz.asc* must be replaced with the verification file of the actual installation package.
- The integrity verification can be performed only when the software package and the .asc file are stored in the same path.

----End

### 3.1.4.2 Installation

#### Prerequisites

The operations required in [3.1.3 Environment Preparation](#) and [3.1.4.1 Preparing for Installation](#) have been completed.

#### Procedure

Switch to the DDK installation user and perform the following operations:

**Step 1** Run the following command to decompress the installation package:

```
tar -zxvf MSpore_DDK****tar.gz
```

In actual operations, replace *MSpore\_DDK\*\*\*\*tar.gz* with the actual installation package. For details about the decompressed files, see [Table 3-4](#).

**Step 2** Run the following command to install the DDK:

```
bash install.sh DDK installation directory
```

The DDK installation directory is automatically created during the installation. For example, if the installation directory is set to **\$HOME/tools/che/ddk**, the **tools/che/ddk** directory is automatically created during the installation. Alternatively, you can specify an installation path.

If the message "Successfully installed the DDK!" is displayed, the DDK has been installed successfully.

#### NOTE

- If the message "Warning: DDK functions except TE have been successfully installed." is displayed during the installation, rectify the fault by referring to [3.1.5.3 What Do I Do If the Message "Warning: DDK functions except TE have been successfully installed." Is Displayed During DDK Installation?](#).
- After the DDK is installed, the profiling function under the installation path **\$HOME/tools/che/ddk/ddk/toolchains/profiler** is unavailable. If you need to use the profiling function, install Mind Studio and the DDK by referring to *Ascend 310 Mind Studio Installation Guide* (select an installation guide based on the Mind Studio server).

----End

### 3.1.4.3 Common Operations

This section describes how to install and use the DDK.

#### 3.1.4.3.1 Uninstalling the DDK

To update the DDK version, uninstall the current version and then install a new version according to the installation procedure described in 5.1. This section describes how to uninstall the DDK.

Go to the DDK installation path, for example, **\$HOME/tools/che/ddk/ddk/scripts**. Run the following command as the DDK installation user to uninstall the DDK:

```
./uninstall.sh
```

If the following information is displayed, the DDK is uninstalled successfully:

```
Info: ide_daemon pem uninstall succ
Starting to remove ddk dir.
```

### 3.1.4.3.2 Querying the DDK Version

After the DDK is installed, you can view the DDK version number in the DDK installation directory.

For example, if the installation directory of the DDK is `$HOME/tools/che/ddk`, run the following command in the `$HOME/tools/che/ddk` directory to view the DDK version number:

```
cat ddk_info
```

Figure 3-3 shows the return result.

Figure 3-3 Querying the DDK version

```
ascend@szvphicpra61963:~/tools/che/ddk/ddk$ cat ddk_info
{
 "VERSION": "1.3.113.882",
 "NAME": "DDK",
 "TARGET": "ASIC"
}ascend@szvphicpra61963:~/tools/che/ddk/ddk$
```

In Figure 3-3:

- **VERSION:** Indicates the version number of the DDK. The query result varies according to the actual situations.
- **TARGET:** Indicates the running environment of the DDK, which is **ASIC** or **Atlas DK**.

### 3.1.4.3.3 Configuring the OpenPGP Public Key

#### Prerequisites

- The public key is configured by the DDK installation user.
- The GnuPG tool is installed on Linux.

#### Verification:

- If the GnuPG tool has been installed, run the **gpg --version** command in the shell. Information in Figure 3-4 is displayed:

Figure 3-4 Command output

```
ascend@ascend-HP-ProDesk-600-G4-PCI-MT:~$ gpg --version
gpg (GnuPG) 1.4.20
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: ~/.gnupg
Supported algorithms:
Pubkey: RSA, RSA-E, RSA-S, ELG-E, DSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: MD5, SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2
```



- If the GnuPG tool is not installed, install the tool by following the instructions provided in its official website <https://www.gnupg.org/>.

## Configuring a Public Key

**Step 1** Obtain the public key file.

Go to the [OpenPGP download page](#) and click the download link, as shown in [Figure 3-5](#). The file download page is displayed.

**Figure 3-5** Click to download the file.

| Version                     | Publication Date | Expired |
|-----------------------------|------------------|---------|
| <a href="#">V100R001C00</a> | 2019-01-21       | Active  |

The **KEYS** file is the public key file, as shown in [Figure 3-6](#).

**Figure 3-6** Selecting the KEYS file

| <input type="checkbox"/> Software Name                            | Size   | Publication Date | Download          |
|-------------------------------------------------------------------|--------|------------------|-------------------|
| <input type="checkbox"/> OpenPGP Signature Verification Guide.pdf | 1.58MB | 2019-01-21       | <a href="#">↓</a> |
| <input type="checkbox"/> KEYS.txt                                 | 1.26KB | 2019-01-21       | <a href="#">↓</a> |
| <input type="checkbox"/> VerificationTools.rar                    | 3.44MB | 2019-01-21       | <a href="#">↓</a> |
| <a href="#">Download</a>                                          |        |                  |                   |

### NOTE

If you want to switch to the English interface, click  **Worldwide** in the upper right corner.

**Step 2** Upload the downloaded **KEYS.txt** file to the Linux OS where the DDK is located, for example, **/home/username/openpgp/keys**.

**Step 3** Import the public key file.

Run the following command to go to the directory that contains the public key file **KEYS**.

```
gpg --import "/home/username/openpgp/keys/KEYS.txt"
```

**Figure 3-7** Import the public key file.

```
ascend@szvphicpra61963:~$ gpg --import "/home/ascend/openpgp/keys/KEYS.txt"
gpg: keyring "/home/ascend/.gnupg/secring.gpg" created
gpg: /home/ascend/.gnupg/trustdb.gpg: trustdb created
gpg: key 27A74824: public key "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>" imported
gpg: Total number processed: 1
gpg: imported: 1 (RSA: 1)
```

### NOTE

**/home/username/openpgp/keys** indicates the absolute path of the public key file **KEYS**. **username** must be replaced with the name of the DDK installation user.

**Step 4** Run the following command to view the import result:

```
gpg --fingerprint
```

Figure 3-8 Checking the import result

```
ascend@szvphicpra61963:~$ gpg --fingerprint
/home/ascend/.gnupg/pubring.gpg

pub 2048R/27A74824 2013-12-30
Key fingerprint = B100 0AC3 8C41 525A 19BD C087 99AD 81DF 27A7 4824
uid OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>
```

**Step 5** Verify the public key.

- The validity of the OpenPGP public key must be verified based on the public key ID, fingerprint, UID, and the publisher of the public key. The published information of the OpenPGP public key is as follows:
  - Public key ID: 27A74824
  - Key fingerprint: B100 0AC3 8C41 525A 19BD C087 99AD 81DF 27A7 4824
  - UID: OpenPGP signature key for Huawei software (created on 30th Dec, 2013) <support@huawei.com>

After the information is verified, you can set the trust level for the public key

- by running the following command:  
gpg --edit-key "OpenPGP signature key for Huawei software" trust  
Information similar to the following is displayed. Enter **5** after **Your decision?**, which indicates **I trust ultimately**. Enter **y** after **Do you really want to set this key to ultimate trust? (y/N)**.

Figure 3-9 Setting the trust level for the public key

```
ascend@szvphicpra61963:~$ gpg --edit-key "OpenPGP signature key for Huawei" trust
gpg (GnuPG) 1.4.20; Copyright (C) 2015 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

pub 2048R/27A74824 created: 2013-12-30 expires: never usage: SC
trust: unknown validity: unknown
[unknown] (1). OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>

pub 2048R/27A74824 created: 2013-12-30 expires: never usage: SC
trust: unknown validity: unknown
[unknown] (1). OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>

Please decide how far you trust this user to correctly verify other users' keys
(by looking at passports, checking fingerprints from different sources, etc.)

 1 = I don't know or won't say
 2 = I do NOT trust
 3 = I trust marginally
 4 = I trust fully
 5 = I trust ultimately
 m = back to the main menu

Your decision? 5
Do you really want to set this key to ultimate trust? (y/N) y

pub 2048R/27A74824 created: 2013-12-30 expires: never usage: SC
trust: ultimate validity: unknown
[unknown] (1). OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>
Please note that the shown key validity is not necessarily correct
unless you restart the program.

gpg> quit
```

**Step 6** Run the **quit** command to exit.

----End

## 3.1.5 FAQs

### 3.1.5.1 What Do I Do If the Message "Import Error:cannot import name main" Is Displayed When the pip3/pip3.5 -V Command Is Run after pip3/pip3.5 Upgrade?

#### Symptom

After pip3/pip3.5 upgrade, the Python library functions are changed. Therefore, the "Import Error:cannot import name main" error occurs upon a pip3/pip3.5 command.

#### Solution 1

Run the following upgrade command:

- For Python 3:  
`python3 -m pip install --upgrade pip --user`
- For Python 3.5:  
`python3.5 -m pip install --upgrade pip --user`

If the fault persists, follow [Solution 2](#).

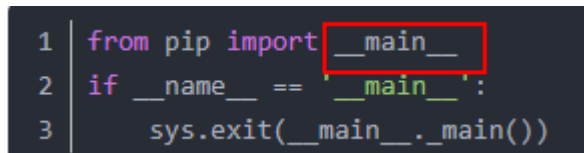
#### Solution 2

For Python 3:

Open `vim /usr/bin/pip3`.

Modify the code as shown in [Figure 3-10](#).

**Figure 3-10** Modifying pip3 code



```
1 | from pip import __main__
2 | if __name__ == '__main__':
3 | sys.exit(__main__.__main__())
```

The solution for Python 3.5 is the same as that for Python 3.

### 3.1.5.2 What Do I Do If epel-release Installation Fails?

If epel-release fails to be installed using the `sudo -E yum install -y epel-release` command, perform the following steps to manually install EPEL:

1. Run the following command to check if the system version matches the EPEL version:  
`cat /etc/redhat-release`  
If "CentOS Linux release 7.\*\*\*" is returned, EPEL 7 must be installed.
2. Run the following command to download the EPEL installation package of the required version from the EPEL website <https://dl.fedoraproject.org/pub/epel>:  
`wget https://dl.fedoraproject.org/pub/epel/7Server/x86_64/Packages/e/epel-release-7-11.noarch.rpm --no-check-certificate`

3. Run the following command to install EPEL (for example, EPEL 7):  
`rpm -vih epel-release-7-11.noarch.rpm`
4. Run the following command to update makecache:  
`yum clean all && yum makecache`

### 3.1.5.3 What Do I Do If the Message "Warning: DDK functions except TE have been successfully installed." Is Displayed During DDK Installation?

#### Symptom

After the DDK installation script is executed, messages such as "error: install te-0.4.0.egg failed" and "error: install topi-0.4.0.egg failed" are displayed during the installation. However, the message "Warning: DDK functions except TE have been successfully installed." is displayed when the installation is complete. The DDK has been successfully installed, but TE functions are unavailable.

#### Possible Cause

The **root** user adopts an earlier version (0.9.8) of **setuptools**. When the **root** user is switched to the DDK installation user to install the DDK, the system still reads the **setuptools** path of the **root** user. As a result, **setuptools** used by the DDK installation user does not take effect.

#### Solution

Run the following command as the DDK installation user to set the environment variable for the **setuptools** run by the DDK installation user to take effect, and then reinstall the DDK:

```
export PATH=/usr/local/python3/bin:$HOME/bin:$HOME/.local/bin:$PATH
```

### 3.1.5.4 What Do I Do If a WARNING or FAIL Result Is Returned in the Integrity Verification of a Software Package?

If a WARNING or FAIL result is returned in the integrity verification of a software package, the verification fails. Rectify the fault by referring to the handling suggestions described in [Table 3-3](#).

**Table 3-3** Verification result examples

| Verification Result Description                                                         | Displayed Information                                                                                                                                                                                                                                                                                                                                                                                                        | Verification Result | Solution                                                                                                                                                          |
|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The signature verification is successful without exception.                             | gpg: Signature made Thu Jan 9 15:29:06 2014 CST using RSA key ID 27A74824<br>gpg: Good signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>"                                                                                                                                                                                                                            | PASS                | N/A                                                                                                                                                               |
| The signature verification fails.                                                       | gpg: Signature made Thu Jan 9 15:29:06 2014 CST using RSA key ID 27A74824<br>gpg: BAD signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>"                                                                                                                                                                                                                             | FAIL                | Download the target file again.                                                                                                                                   |
| The public key cannot be found.                                                         | gpg: Signature made Thu Jan 9 15:20:01 2014 CST using RSA key ID 27A74824<br>gpg: Can't check signature: public key not found                                                                                                                                                                                                                                                                                                | FAIL                | Download the public key again. For details, see <b>Configuring OpenPGP Public Keys</b> .                                                                          |
| The signature verification is successful, but the public key is not ultimately trusted. | gpg: Signature made Thu Jan 9 15:29:06 2014 CST using RSA key ID 27A74824<br>gpg: Good signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>"<br>gpg: WARNING: This key is not certified with a trusted signature!<br>gpg: There is no indication that the signature belongs to the owner.<br>Primary key fingerprint: B100 0AC3 8C41 525A 19BD C087 99AD 81DF 27A7 4824 | WARNING             | After confirming that key ID <b>(27A74824)</b> , set the trust level of Huawei public key to <b>5</b> . For details, see <b>Configuring OpenPGP Public Keys</b> . |
| The corresponding source file cannot be found.                                          | gpg: no signed data<br>gpg: can't hash datafile: No data                                                                                                                                                                                                                                                                                                                                                                     | FAIL                | Download the target file again.                                                                                                                                   |

| Verification Result Description                                                | Displayed Information                                                                                                                                                                                                                                                                                                                              | Verification Result | Solution                                                      |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|---------------------------------------------------------------|
| The signature has expired.                                                     | gpg: Signature made 04/24/13 10:50:29 CST using RSA key ID 133B64E5<br>gpg: Expired signature from " OpenPGP signature test key <support@huawei.com>"<br>gpg: Signature expired 04/25/13 10:50:29 CST                                                                                                                                              | FAIL                | Download an updated target file.                              |
| The signature verification is successful, but the public key has been revoked. | gpg: Signature made 06/13/13 11:14:49 CST using RSA key ID 133B64E5<br>gpg: Good signature from " OpenPGP signature test key <support@huawei.com>"<br>gpg: WARNING: This key has been revoked by its owner!<br>gpg: This could mean that the signature is forged.<br>gpg: reason for revocation: Key is no longer used<br>gpg: revocation comment: | WARNING             | Download the latest public key and an updated target file.    |
| The corresponding signature file cannot be found in the source file.           | None                                                                                                                                                                                                                                                                                                                                               | WARNING             | Download the signature file corresponding to the target file. |

## 3.1.6 Appendix

### 3.1.6.1 Overview of Software Packages

For details about the decompressed DDK installation package, see [Table 3-4](#).

**Table 3-4** Overview of the software packages

| Installation Package                                                         | Content    | Description              |
|------------------------------------------------------------------------------|------------|--------------------------|
| MSpore_DDK-{version}-<uihost arch.os>-<host arch.os>-<device arch.os>.tar.gz | ddk.tar.gz | DDK installation package |
|                                                                              | install.sh | Installation script      |

| Installation Package                                                         | Content      | Description                                                                                                                                                            |
|------------------------------------------------------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| For details about the parameter description, see <a href="#">Table 3-5</a> . | check_sha.sh | Script used to verify the integrity of the preceding files. This script is automatically called to perform integrity check during the execution of <b>install.sh</b> . |

**Table 3-5** Naming rules of the DDK installation package

| Parameter        | Description                                                                                                                                                                                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| {version}        | Version number                                                                                                                                                                                                                                                                                                                                     |
| <uihost arch.os> | CPU architecture, OS, and version on the UI host side, for example, x86_64.ubuntu16.04 and x86_64.centOS7.4                                                                                                                                                                                                                                        |
| <host arch.os>   | CPU architecture, OS, and version on the host side, for example, x86_64.ubuntu16.04 and x86_64.centOS7.4                                                                                                                                                                                                                                           |
| <device arch.os> | CPU architecture, OS, and version on the device side, for example, aarch64.ubuntu16.04 and aarch64.miniOS <ul style="list-style-type: none"> <li>Device-side architecture of the DK form (Atlas 200 DK), for example, aarch64.ubuntu16.04</li> <li>Device-side architecture of the non-DK form (Atlas 300), for example, aarch64.miniOS</li> </ul> |

### 3.1.6.2 Open Source Third-Party Libraries

#### cereal

cereal is a header-only BSD-licensed C++11 serialization library. It is designed to be fast, light-weight, and easy to extend. cereal takes arbitrary data types and reversibly turns them into different representations, such as compact binary encodings, XML, or JSON. The version currently used is 1.2.2.

For details, visit the cereal official website at <http://usclab.github.io/cereal/>.

#### gflags

Google GFlags (GFlags), the Global Flags Editor, contains a Python-friendly C++ library that implements command-line flags processing, replacing systems like **getopt()**. The version currently used is 2.2.1.

For details, visit the GFlags official website at <https://github.com/gflags/gflags>.

## **glog**

Google glog is a library that implements application-level logging. This library provides logging APIs based on C++-style streams and various helper macros. Similar to `assert` defined in the standard C library, it provides more output information and flexibility.

For details, visit the glog official website at <https://github.com/google/glog>.

## **opencv**

OpenCV (short for Open Source Computer Vision Library) is a library of programming functions mainly aimed at real-time computer vision. This cross-platform library sets its focus on real-time image processing, computer vision, and pattern recognition programs. The version currently used is 3.4.2.

For details, visit the OpenCV official website at <https://opencv.org/>.

## **Protobuf**

Protobuf (short for Protocol buffers) are Google's language-neutral, platform-neutral, extensible mechanism for serializing structured data. It is useful in developing programs to communicate with each other over a wire or for storing data. The version currently used is 3.5.1.

For details, visit the official website at <https://developers.google.com/protocol-buffers/>.

# **3.2 DDK Installation Guide (Ubuntu, x86)**

## **3.2.1 Introduction**

This document describes the method of separately installing the device development kit (DDK) and troubleshooting of common problems. It is applicable to development in command line interface (CLI) mode.

The DDK is an NPU-based digital developer kit that can be used to build a compilation environment for projects. Different release packages integrate the DDKs of different NPU forms. The DDK of the current version contains the dependency libraries and header files of components such as the TE, DVPP, and process orchestration. Users can compile the project files by using **makefile**.

## **3.2.2 Obtaining Software Packages**

Before installing the tool, prepare the following software packages:

DDK installation package: Visit <https://support.huawei.com/enterprise>, input the product name in the search box, and press **Enter** or click the search icon. In the displayed window, click the **Software Download** tab to obtain the desired installation package.

The software packages are described as follows.



**Table 3-6** Overview of the software packages

| Installation Package                                                                   | Description                                                                    | Integrity Verification File                                                                |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| MSpore_DDK-{version}-x86_64.ubuntu16.04-x86_64.ubuntu16.04-aarch64.miniOS.tar.gz       | DDK installation package for non-developer board scenarios (such as Atlas 300) | MSpore_DDK-{version}-x86_64.ubuntu16.04-x86_64.ubuntu16.04-aarch64.miniOS.tar.gz.asc       |
| MSpore_DDK-{version}-x86_64.ubuntu16.04-aarch64.ubuntu16.04-aarch64.ubuntu16.04.tar.gz | DDK installation package for developer board scenarios (such as Atlas 200 DK)  | MSpore_DDK-{version}-x86_64.ubuntu16.04-aarch64.ubuntu16.04-aarch64.ubuntu16.04.tar.gz.asc |

 **NOTE**

{version} indicates the DDK version number.

## 3.2.3 Environment Preparation

### Environment Requirements

The DDK installation must meet the following environment requirements on the hardware and operating system (OS).

**Table 3-7** Ubuntu system version information

| Category | Version                        | Obtaining Method                                                                                                                                                                                                                                                                     | Precautions |
|----------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| OS       | 16.04.3                        | Download the required version from <a href="http://old-releases.ubuntu.com/releases/16.04.3/">http://old-releases.ubuntu.com/releases/16.04.3/</a> . You can download the desktop version <b>ubuntu-XXX-desktop-amd64.iso</b> or server version <b>ubuntu-XXX-server-amd64.iso</b> . | -           |
| Python   | Python2: 2.7+<br>Python3: 3.5+ | For details, see <a href="#">Installing Dependencies</a> .                                                                                                                                                                                                                           | -           |

### (Optional) Preparing the DDK Installation User

You can install the DDK as any user (**root** or non-**root**).

- If the **root** user is used for installation, skip this section.
- If you use an existing non-**root** user to install the DDK, run the following command as the **root** user to grant the existing user with permission 750 on the **\$HOME** directory:  

```
chmod 750 /home/username
```
- If you want to install the DDK as a new non-**root** user, perform the following steps as the **root** user: This document describes how to install the DDK as a new non-**root** user.
  - a. Run the following command to create a user for DDK installation and configure the **\$HOME** directory of the user:  

```
useradd -d /home/username -m username
```
  - b. Run the following command to set the password:  

```
passwd username
```
  - c. Run the following command to set the permission:  

```
chmod 750 /home/username
```

#### NOTE

*username* indicates the user name for installing the DDK. The **umask** value of the user cannot be greater than **0027**.

- To view the **umask** value, run the **umask** command.
- To change the **umask** value, run the **umask New value** command.

If the created non-**root** user is **HwHiAiUser**, the DDK is deployed on the host server, and **HwHiAiUser** is used for offline model conversion, logs are stored in the **var/dlog** directory on the host by default. To enable the output of logs to the screen by default, you can set the following environment variable:  

```
export SLOG_PRINT_TO_STDOUT=1
```

## (Optional) Configuring Permissions of the DDK Installation User

When a non-**root** user is used for installation, perform the following operations. Skip the following operations if the **root** user is used for installation.

Before installing the DDK, you need to download dependent software with the **sudo apt-get** permission. Perform the following operations as the **root** user:

1. Open the **/etc/sudoers** file.  

```
chmod u+w /etc/sudoers
vi /etc/sudoers
```
2. Add the following content under **# User privilege specification** of the file:  

```
username ALL=(ALL:ALL) NOPASSWD:SETENV:usr/bin/apt-get
```

Replace **username** with the name of the common user who executes the installation script.

#### NOTE

Ensure that the last line of the **/etc/sudoers** file is **#includedir /etc/sudoers.d**. Otherwise, add it manually.

3. Run the **:wq!** command to save the file.
4. Run the following command to revoke the write permission on the **/etc/sudoers** file:  

```
chmod u-w /etc/sudoers
```

## Checking the Source Availability

DDK installation requires related dependencies, which must be downloaded in advance. Ensure that the server where the DDK is installed can successfully connect to the network.

Run the following command as the **root** user to check whether the source is available:

```
apt-get update
```

### NOTE

If an error is reported during the command execution, check whether the network connection is normal or replace the source in the `/etc/apt/sources.list` file with a valid one.

## Installing Dependencies

Run the **su - *username*** command to switch to the DDK installation user and perform the following operations to install components such as the GCC and Python on which the DDK depends:

**Step 1** Run the following command to install the DDK dependencies:

```
sudo -E apt-get install gcc g++ cmake make python-pip python3-pip python3 python
```

**Step 2** Install the Python development environment.

1. Install the Python 2 environment.

- Environment deployment depends on pip. Install dependencies of the TE software package, including NumPy (versions 1.11.0–1.16.0 are supported and the following uses **1.16.0** as an example) and decorator. Run the following commands:

```
pip2 install numpy==1.16.0 --user
pip2 install decorator --user
```

- Check the Python 2 environment.

Run the following commands to check the Python, pip, and Setuptools respectively:

```
python2 --version
pip2 --version
easy_install --version
```

If the following information is displayed respectively for the preceding three commands, the installation is successful:

```
Python 2.7.12
pip 19.0.1 from /usr/local/lib/python2.7/dist-packages/pip (python 2.7)
setuptools 20.7.0 from /usr/lib/python2.7/dist-packages (Python 2.7)
```

2. Install the Python3 (3.5+) environment.

- Environment deployment depends on the pip3. Run the following commands to install the dependencies (NumPy and decorator) of the TE software package respectively:

```
pip3 install numpy --user
pip3 install decorator --user
```

- Check the Python 3 environment.

Run the following commands to check the Python 3, pip3, and Setuptools, respectively:

```
python3 --version
pip3 --version
easy_install3 --version
```

If the following information is displayed respectively for the preceding three commands, the installation is successful:

```
Python 3.5.2
pip 8.1.1 from /usr/lib/python3/dist-packages (python 3.5)
setuptools 20.7.0 from /usr/lib/python3/dist-packages (Python 3.5)
```

----End

#### NOTE

- If the pip installation is abnormal during the dependency installation, rectify the fault by referring to [3.2.5.1 What Do I Do If a Message Is Displayed Indicating pip2 or pip Unavailability During Mind Studio or DDK Installation?](#).
- The version numbers in the command outputs are only for reference, which are subject to the actual situations.

## 3.2.4 Installing the DDK

This section describes the installation method of the DDK and common operations.

### 3.2.4.1 Preparing for Installation

#### Uploading Installation Packages

Upload the following files to any directory under **\$HOME** as the DDK installation user:

- **MSpore\_DDK\*\*\*\*.tar.gz**: DDK installation package
- **MSpore\_DDK\*\*\*\*.tar.gz.asc**: file for verifying the DDK installation package

#### NOTE

For details about DDK packages, see [Table 3-6](#). \* indicates the specific package. Ensure that the preceding two files are in the same directory.

#### Verifying Software Package Integrity

Before installing a software package, you are advised to check whether the software package is incomplete or damaged due to network or storage device faults.

In the directory where the DDK installation package is stored, perform the following operations:

**Step 1** Configure the OpenPGP public keys. For details, see [3.2.4.3.3 Configuring the OpenPGP Public Key](#).

**Step 2** Run the following command as the DDK installation user to check the validity and integrity of the software package, as shown in [Figure 3-11](#):

```
gpg --verify "MSpore_DDK****.tar.gz.asc"
```

**Figure 3-11** Verifying the software package integrity

```
gpg> quit
ascend@szvphicpra61963:~$ gpg --verify MSpore_DDK-1.3.123.B88Z-x86_64-ubuntu16.04-x86_64-ubuntu16.04-aarch64.minios.tar.gz.asc
gpg: assuming signed data in 'MSpore_DDK-1.3.123.B88Z-x86_64-ubuntu16.04-x86_64-ubuntu16.04-aarch64.minios.tar.gz'
gpg: Signature made Thursday, June 13, 2019 PM06:14:47 HKT using RSA key ID 27A74824
gpg: Good signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>"
ascend@szvphicpra61963:~$
```

- In the returned information, **27A74824** indicates the public key ID.
- If the message **Good signature** is displayed without **WARNING** or **FAIL**, the signature is valid and the integrity verification is passed.
- If **WARNING** or **FAIL** is displayed, the verification fails. Rectify the fault by referring to the handling suggestions described in [3.2.5.2 What Do I Do If a WARNING or FAIL Result Is Returned in the Integrity Verification of a Software Package?](#).

#### NOTE

- In actual operations, *MSpore\_DDK\*\*\*\*.tar.gz.asc* must be replaced with the verification file of the actual installation package.
- The integrity verification can be performed only when the software package and the .asc file are stored in the same path.

----End

## 3.2.4.2 Installation

### Prerequisites

The operations required in [3.2.3 Environment Preparation](#) and [3.2.4.1 Preparing for Installation](#) have been completed.

### Procedure

Switch to the DDK installation user and perform the following operations:

- Step 1** Run the following command to decompress the installation package:

```
tar -zxvf MSpore_DDK****tar.gz
```

In actual operations, *MSpore\_DDK\*\*\*\*.tar.gz.asc* must be replaced with the actual installation package. For details about the decompressed files, see [Table 3-9](#).

- Step 2** Run the following command to install the DDK:

```
bash install.sh DDK installation directory
```

The DDK installation directory is automatically created during the installation. For example, if the installation directory is set to **\$HOME/tools/che/ddk**, the **tools/che/ddk** directory is automatically created during the installation. Alternatively, you can specify an installation path.

If the message "Successfully installed the DDK!" is displayed, the DDK has been installed successfully.

#### NOTE

After the DDK is installed, the profiling function under the installation path **\$HOME/tools/che/ddk/ddk/toolchains/profiler** is unavailable. If you need to use the profiling function, install Mind Studio and the DDK by referring to *Ascend 310 Mind Studio Installation Guide* (select an installation guide based on the Mind Studio server).

----End

### 3.2.4.3 Common Operations

This section describes how to install and use the DDK.

#### 3.2.4.3.1 Uninstalling the DDK

To update the DDK version, uninstall the current version and then install a new version according to the installation procedure described in 5.1. This section describes how to uninstall the DDK.

Go to the DDK installation path, for example, **\$HOME/tools/che/ddk/ddk/scripts**. Run the following command as the DDK installation user to uninstall the DDK:

```
./uninstall.sh
```

If the following information is displayed, the DDK is uninstalled successfully:

```
Info: ide_daemon pem uninstall succ
Starting to remove ddk dir.
```

#### 3.2.4.3.2 Querying the DDK Version

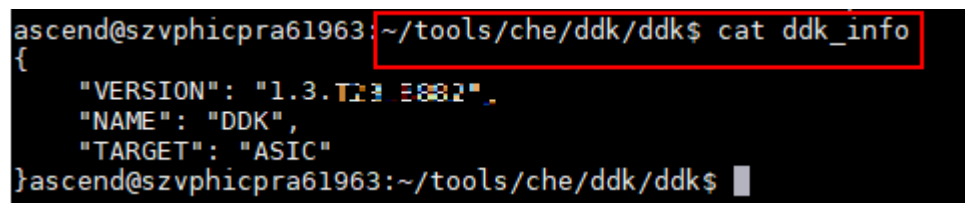
After the DDK is installed, you can view the DDK version number in the DDK installation directory.

For example, if the installation directory of the DDK is **\$HOME/tools/che/ddk**, run the following command in the **\$HOME/tools/che/ddk** directory to view the DDK version number:

```
cat ddk_info
```

**Figure 3-12** shows the return result.

**Figure 3-12** Querying the DDK version



```
ascend@szvphicpra61963:~/tools/che/ddk/ddk$ cat ddk_info
{
 "VERSION": "1.3.123 8882",
 "NAME": "DDK",
 "TARGET": "ASIC"
}ascend@szvphicpra61963:~/tools/che/ddk/ddk$
```

In **Figure 3-12**:

- **VERSION**: Indicates the version number of the DDK. The query result varies according to the actual situations.
- **TARGET**: Indicates the running environment of the DDK, which is **ASIC** or **Atlas DK**.

### 3.2.4.3.3 Configuring the OpenPGP Public Key

#### Prerequisites

- The public key is configured by the DDK installation user.
- The GnuPG tool is installed on Linux.

**Verification:**

- If the GnuPG tool has been installed, run the **gpg --version** command in the shell. Information in [Figure 3-13](#) is displayed:

**Figure 3-13** Command output

```
ascend@ascend-HP-ProDesk-600-G4-PCI-MT:~$ gpg --version
gpg (GnuPG) 1.4.20
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: ~/.gnupg
Supported algorithms:
Pubkey: RSA, RSA-E, RSA-S, ELG-E, DSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
 CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: MD5, SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2
```

- If the GnuPG tool is not installed, install the tool by following the instructions provided in its official website <https://www.gnupg.org/>.

#### Configuring a Public Key

**Step 1** Obtain the public key file.

Go to the [OpenPGP download page](#) and click the download link, as shown in [Figure 3-14](#). The file download page is displayed.

**Figure 3-14** Click to download the file.

| Version                     | Publication Date | Expired |
|-----------------------------|------------------|---------|
| <a href="#">V100R001C00</a> | 2019-01-21       | Active  |

The **KEYS** file is the public key file, as shown in [Figure 3-15](#).

**Figure 3-15** Selecting the KEYS file

| <input type="checkbox"/> Software Name                            | Size   | Publication Date | Download          |
|-------------------------------------------------------------------|--------|------------------|-------------------|
| <input type="checkbox"/> OpenPGP Signature Verification Guide.pdf | 1.58MB | 2019-01-21       | <a href="#">↓</a> |
| <input type="checkbox"/> KEYS.txt                                 | 1.26KB | 2019-01-21       | <a href="#">↓</a> |
| <input type="checkbox"/> VerificationTools.rar                    | 3.44MB | 2019-01-21       | <a href="#">↓</a> |
| <div>Download</div>                                               |        |                  |                   |

 **NOTE**

If you want to switch to the English interface, click  in the upper right corner.

**Step 2** Upload the downloaded **KEYS.txt** file to the Linux OS where the DDK is located, for example, **/home/username/openpgp/keys**.

**Step 3** Import the public key file.

Run the following command to go to the directory that contains the public key file **KEYS**.

```
gpg --import "/home/username/openpgp/keys/KEYS.txt"
```

**Figure 3-16** Import the public key file.

```
ascend@szvphicpra61963:~$ gpg --import "/home/ascend/openpgp/keys/KEYS.txt"
gpg: keyring '/home/ascend/.gnupg/secring.gpg' created
gpg: /home/ascend/.gnupg/trustdb.gpg: trustdb created
gpg: key 27A74824: public key "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>" imported
gpg: Total number processed: 1
gpg: imported: 1 (RSA: 1)
```

#### NOTE

**/home/username/openpgp/keys** indicates the absolute path of the public key file **KEYS**. **username** must be replaced with the name of the DDK installation user.

**Step 4** Run the following command to view the import result:

```
gpg --fingerprint
```

**Figure 3-17** Checking the import result

```
ascend@szvphicpra61963:~$ gpg --fingerprint
/home/ascend/.gnupg/pubring.gpg

pub 2048R/27A74824 2013-12-30
 Key fingerprint = B100 0AC3 8C41 525A 19BD C087 99AD 81DF 27A7 4824
uid OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>
```

**Step 5** Verify the public key.

- The validity of the OpenPGP public key must be verified based on the public key ID, fingerprint, UID, and the publisher of the public key. The published information of the OpenPGP public key is as follows:
  - Public key ID: 27A74824
  - Key fingerprint: B100 0AC3 8C41 525A 19BD C087 99AD 81DF 27A7 4824
  - UID: OpenPGP signature key for Huawei software (created on 30th Dec, 2013) <support@huawei.com>

After the information is verified, you can set the trust level for the public key

- by running the following command:

```
gpg --edit-key "OpenPGP signature key for Huawei software" trust
```

Information similar to the following is displayed. Enter **5** after **Your decision?**, which indicates **I trust ultimately**. Enter **y** after **Do you really want to set this key to ultimate trust? (y/N)**.



Figure 3-18 Setting the trust level for the public key

```
ascend@szvphicpra61963:~$ gpg --edit-key "OpenPGP signature key for Huawei" trust
gpg (GnuPG) 1.4.20; Copyright (C) 2015 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

pub 2048R/27A74824 created: 2013-12-30 expires: never usage: SC
trust: unknown validity: unknown
[unknown] (1). OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>

pub 2048R/27A74824 created: 2013-12-30 expires: never usage: SC
trust: unknown validity: unknown
[unknown] (1). OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>

Please decide how far you trust this user to correctly verify other users' keys
(by looking at passports, checking fingerprints from different sources, etc.)

 1 = I don't know or won't say
 2 = I do NOT trust
 3 = I trust marginally
 4 = I trust fully
 5 = I trust ultimately
 m = back to the main menu

Your decision? 5
Do you really want to set this key to ultimate trust? (y/N) y

pub 2048R/27A74824 created: 2013-12-30 expires: never usage: SC
trust: ultimate validity: unknown
[unknown] (1). OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>
Please note that the shown key validity is not necessarily correct
unless you restart the program.

gpg> quit
```

Step 6 Run the **quit** command to exit.

----End

## 3.2.5 FAQs

### 3.2.5.1 What Do I Do If a Message Is Displayed Indicating pip2 or pip Unavailability During Mind Studio or DDK Installation?

#### Symptom

During the installation of Mind Studio or a DDK, the system displays a message indicating that the pip2 or pip is unavailable and exits the installation, as shown in [Figure 3-19](#) and [Figure 3-20](#).

Figure 3-19 Message indicating pip2 unavailability

```
Python 2.7.12
Traceback (most recent call last):
 File "/usr/bin/pip2", line 9, in <module>
 load_entry_point('pip==8.1.1', 'console_scripts', 'pip2')()
 File "/usr/local/lib/python2.7/dist-packages/setuptools-0.6c11-py2.7.egg/pkg_resources.py", line 318, in load_entry_point
 File "/usr/local/lib/python2.7/dist-packages/setuptools-0.6c11-py2.7.egg/pkg_resources.py", line 2221, in load_entry_point
 File "/usr/local/lib/python2.7/dist-packages/setuptools-0.6c11-py2.7.egg/pkg_resources.py", line 1954, in load
 File "/usr/lib/python2.7/dist-packages/pip/_init_.py", line 18, in <module>
 from pip.commands import get_summaries, get_similar_commands
 File "/usr/lib/python2.7/dist-packages/pip/commands/__init__.py", line 8, in <module>
 from pip.commands.freeze import FreezeCommand
 File "/usr/lib/python2.7/dist-packages/pip/commands/freeze.py", line 8, in <module>
 from pip.operations.freeze import freeze
 File "/usr/lib/python2.7/dist-packages/pip/operations/freeze.py", line 11, in <module>
 from pip._vendor.pkg_resources import RequirementParseError
ImportError: cannot import name RequirementParseError
Your pip2 has problem, please solve it before install MindStudio, Exit 1
```

Figure 3-20 Message indicating pip unavailability

```
ascend@szvphicpra61963:~$ pip
Traceback (most recent call last):
 File "/usr/bin/pip", line 9, in <module>
 from pip import main
ImportError: cannot import name main
```

## Possible Cause

pip2 is not updated during pip re-installation.

## Solution 1

- Step 1** Run the **su root** command to switch to the **root** user and run the **pip list** command. If no error message is displayed, the pip is available. If an error message is displayed after the **pip2 list** command is executed, the pip2 is unavailable.
- Step 2** Run the **rm /usr/bin/pip2** command as the **root** user to delete the pip2.
- Step 3** Run the **ln -s pip pip2** command to create a soft link from the pip2 to pip.
- Step 4** Run the **pip2 list** command again. If no error message is displayed, it indicates that the fault has been rectified.

If the pip and pip2 are still unavailable, see [Solution 2](#).

----End

## Solution 2

If the pip installation is abnormal during the dependency installation, run the following commands in sequence:

```
sudo apt-get remove python-pip python3-pip
wget https://bootstrap.pypa.io/get-pip.py
python get-pip.py --user
python3 get-pip.py --user
```

### 3.2.5.2 What Do I Do If a WARNING or FAIL Result Is Returned in the Integrity Verification of a Software Package?

If a WARNING or FAIL result is returned in the integrity verification of a software package, the verification fails. Rectify the fault by referring to the handling suggestions described in [Table 3-8](#).

**Table 3-8** Verification result examples

| Verification Result Description                             | Displayed Information                                                                                                                                                                             | Verification Result | Solution |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|----------|
| The signature verification is successful without exception. | gpg: Signature made Thu Jan 9 15:29:06 2014 CST using RSA key ID 27A74824<br>gpg: Good signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>" | PASS                | N/A      |

| Verification Result Description                                                         | Displayed Information                                                                                                                                                                                                                                                                                                                                                                                                        | Verification Result | Solution                                                                                                                                                          |
|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The signature verification fails.                                                       | gpg: Signature made Thu Jan 9 15:29:06 2014 CST using RSA key ID 27A74824<br>gpg: BAD signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>"                                                                                                                                                                                                                             | FAIL                | Download the target file again.                                                                                                                                   |
| The public key cannot be found.                                                         | gpg: Signature made Thu Jan 9 15:20:01 2014 CST using RSA key ID 27A74824<br>gpg: Can't check signature: public key not found                                                                                                                                                                                                                                                                                                | FAIL                | Download the public key again. For details, see <b>Configuring OpenPGP Public Keys</b> .                                                                          |
| The signature verification is successful, but the public key is not ultimately trusted. | gpg: Signature made Thu Jan 9 15:29:06 2014 CST using RSA key ID 27A74824<br>gpg: Good signature from "OpenPGP signature key for Huawei software (created on 30th Dec,2013) <support@huawei.com>"<br>gpg: WARNING: This key is not certified with a trusted signature!<br>gpg: There is no indication that the signature belongs to the owner.<br>Primary key fingerprint: B100 0AC3 8C41 525A 19BD C087 99AD 81DF 27A7 4824 | WARNING             | After confirming that key ID <b>(27A74824)</b> , set the trust level of Huawei public key to <b>5</b> . For details, see <b>Configuring OpenPGP Public Keys</b> . |
| The corresponding source file cannot be found.                                          | gpg: no signed data<br>gpg: can't hash datafile: No data                                                                                                                                                                                                                                                                                                                                                                     | FAIL                | Download the target file again.                                                                                                                                   |
| The signature has expired.                                                              | gpg: Signature made 04/24/13 10:50:29 CST using RSA key ID 133B64E5<br>gpg: Expired signature from " OpenPGP signature test key <support@huawei.com>"<br>gpg: Signature expired 04/25/13 10:50:29 CST                                                                                                                                                                                                                        | FAIL                | Download an updated target file.                                                                                                                                  |

| Verification Result Description                                                | Displayed Information                                                                                                                                                                                                                                                                                                                              | Verification Result | Solution                                                      |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|---------------------------------------------------------------|
| The signature verification is successful, but the public key has been revoked. | gpg: Signature made 06/13/13 11:14:49 CST using RSA key ID 133B64E5<br>gpg: Good signature from " OpenPGP signature test key <support@huawei.com>"<br>gpg: WARNING: This key has been revoked by its owner!<br>gpg: This could mean that the signature is forged.<br>gpg: reason for revocation: Key is no longer used<br>gpg: revocation comment: | WARNING             | Download the latest public key and an updated target file.    |
| The corresponding signature file cannot be found in the source file.           | None                                                                                                                                                                                                                                                                                                                                               | WARNING             | Download the signature file corresponding to the target file. |

## 3.2.6 Appendix

### 3.2.6.1 Overview of Software Packages

For details about the decompressed DDK installation package, see [Table 3-9](#).

**Table 3-9** Overview of the software packages

| Installation Package                                                                                                                                          | Content      | Description                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MSpore_DDK-{version}-<uihost arch.os>-<host arch.os>-<device arch.os>.tar.gz<br>For details about the parameter description, see <a href="#">Table 3-10</a> . | ddk.tar.gz   | DDK installation package                                                                                                                                               |
|                                                                                                                                                               | install.sh   | Installation script                                                                                                                                                    |
|                                                                                                                                                               | check_sha.sh | Script used to verify the integrity of the preceding files. This script is automatically called to perform integrity check during the execution of <b>install.sh</b> . |

**Table 3-10** Naming rules of the DDK installation package

| Parameter        | Description                                                                                                                                                                                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| {version}        | Version number                                                                                                                                                                                                                                                                                                                                     |
| <uihost arch.os> | CPU architecture, OS, and version on the UI host side, for example, x86_64.ubuntu16.04 and x86_64.centOS7.4                                                                                                                                                                                                                                        |
| <host arch.os>   | CPU architecture, OS, and version on the host side, for example, x86_64.ubuntu16.04 and x86_64.centOS7.4                                                                                                                                                                                                                                           |
| <device arch.os> | CPU architecture, OS, and version on the device side, for example, aarch64.ubuntu16.04 and aarch64.miniOS <ul style="list-style-type: none"> <li>Device-side architecture of the DK form (Atlas 200 DK), for example, aarch64.ubuntu16.04</li> <li>Device-side architecture of the non-DK form (Atlas 300), for example, aarch64.miniOS</li> </ul> |

### 3.2.6.2 Open Source Third-Party Libraries

#### cereal

cereal is a header-only BSD-licensed C++11 serialization library. It is designed to be fast, light-weight, and easy to extend. cereal takes arbitrary data types and reversibly turns them into different representations, such as compact binary encodings, XML, or JSON. The version currently used is 1.2.2.

For details, visit the cereal official website at <http://uscilab.github.io/cereal/>.

#### gflags

Google GFlags (GFlags), the Global Flags Editor, contains a Python-friendly C++ library that implements command-line flags processing, replacing systems like **getopt()**. The version currently used is 2.2.1.

For details, visit the GFlags official website at <https://github.com/gflags/gflags>.

#### glog

Google glog is a library that implements application-level logging. This library provides logging APIs based on C++-style streams and various helper macros. Similar to assert defined in the standard C library, it provides more output information and flexibility.

For details, visit the glog official website at <https://github.com/google/glog>.

#### opencv

OpenCV (short for Open Source Computer Vision Library) is a library of programming functions mainly aimed at real-time computer vision. This cross-platform library sets its focus on real-time image processing, computer vision, and pattern recognition programs. The version currently used is 3.4.2.

For details, visit the OpenCV official website at <https://opencv.org/>.

## Protobuf

Protobuf (short for Protocol buffers) are Google's language-neutral, platform-neutral, extensible mechanism for serializing structured data. It is useful in developing programs to communicate with each other over a wire or for storing data. The version currently used is 3.5.1.

For details, visit the official website at <https://developers.google.com/protocol-buffers/>.

# 4 Model Conversion

---

- [4.1 Introduction](#)
- [4.2 Caffe/TensorFlow Mode Conversion](#)
- [4.3 AIPP Configuration](#)
- [4.4 Quantization Configuration](#)
- [4.5 FAQs](#)

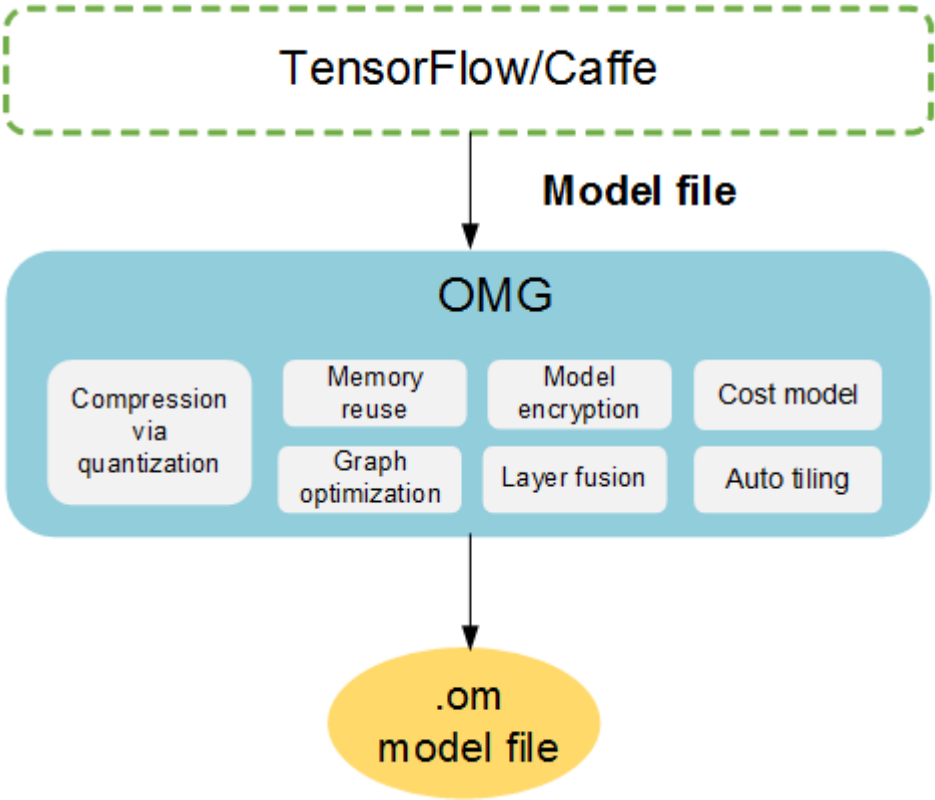
## 4.1 Introduction

Trained models under frameworks such as Caffe and TensorFlow can be converted into offline models supported by the Ascend AI processor by using the offline model generator (OMG). During offline model conversion, you can enable operator scheduling optimization, data re-orchestration in the weight file, model compression via quantization, and memory usage optimization, thereby pre-processing the model without depending on the device.

### Functional Architecture

[Figure 4-1](#) shows the functional architecture of OMG.

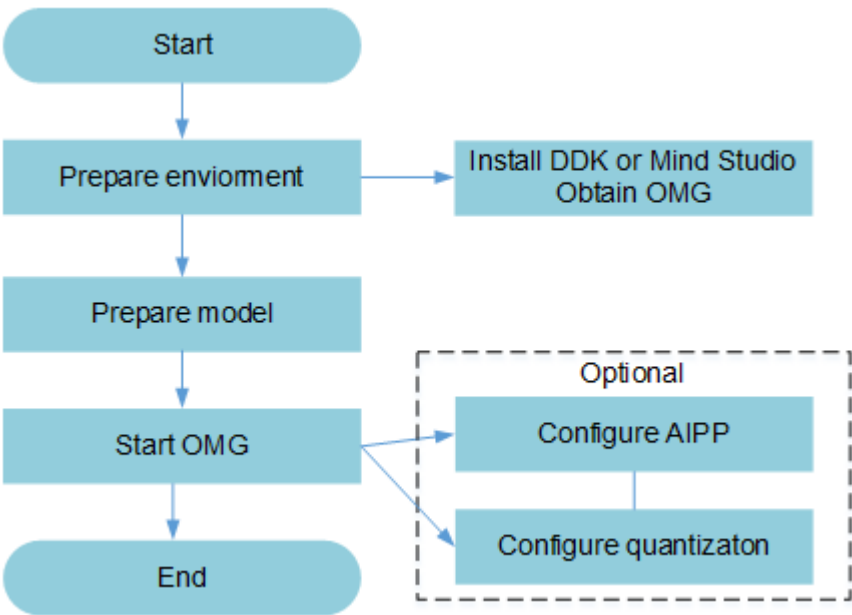
Figure 4-1 OMG functional architecture



OMG Workflow

Figure 4-2 shows the model conversion workflow of OMG.

Figure 4-2 OMG workflow



The OMG workflow is described as follows:



1. Before using OMG, install the DDK or the Mind Studio IDE on the server and obtain the OMG tool from the specified path. For details, see [Preparations](#).
2. Upload the model to be converted to the server where the DDK is located. For details, see [Sample](#).
3. Start OMG to convert the model. Enable artificial intelligence pre-processing (AIPP) or quantization as required. For details, see [4.3 AIPP Configuration](#) and [4.4 Quantization Configuration](#).
  - a. The Ascend AI processor introduces the AIPP module for hardware-based image pre-processing including color space conversion (CSC), image normalization (by subtracting the mean value or multiplying a factor), image cropping (by specifying the crop start and cropping the image to the size required by the neural network), and much more. Images output by the DVPP module are aligned images in YUV420SP format. The digital vision pre-processing (DVPP) module does not output RGB images. Therefore, the AIPP module is introduced to convert the aligned YUV420SP images and then crop them to the size required by the model.
  - b. Quantization refers to low-bit quantization of high-precision data, suitable for scenarios that are model-size and performance demanding. This feature results in lightweight models, with reduced storage spaces and transfer latency but improved computation efficiency.

## 4.2 Caffe/TensorFlow Mode Conversion

This topic describes how to use the OMG tool to convert a Caffe or TensorFlow model to an offline model supported by the Ascend AI processor.

### 4.2.1 Constraints and Parameters

#### Restrictions

Before model conversion, pay attention to the following restrictions:

- Only a Caffe or TensorFlow model can be converted. For a Caffe model, the input data must be of the FLOAT type. For a TensorFlow model, the input data must be of INT32, BOOL, UINT8, or FLOAT type.
- For a Caffe model, **op name** and **op type** in the model file (.prototxt) and weight file (.caffemodel) must be the consistent (case sensitive).
- For a Caffe model, the **top** names of all layers must be the same except for the layers with the same **top** and **bottom** (such as BatchNorm, Scale, and ReLU).
- For a TensorFlow model, only the FrozenGraphDef format is supported.
- Inputs with dynamic shapes are not supported, for example, NHWC = [?, ?, ?, 3]. The dimension sizes must be static.
- The input can be up to 4-dimensional. Operators involving dimension changes (such as reshape and expanddim) cannot output five dimensions.
- Except the const operator, the input and output of operators at all layers in the model must meet the requirements of **dim! = 0**.
- Model conversion does not support models that contain operators used for model training.

- A UINT8 quantized model cannot be converted.
- Model operators support only 2D convolution, and do not support 3D convolution.
- Only the operators listed in *Operator List* are supported. The defined restrictions on the operators must be met.

## Parameter Description

| Parameter | Description                                                                                                                                                                                                                                                                                                                                   | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --mode    | <p>Operating mode</p> <ul style="list-style-type: none"> <li>• 0: Generate an offline model supported by the Ascend AI processor</li> <li>• 1: Convert the offline model or model file to the JSON format.</li> <li>• 3: Perform precheck only to check the validity of the model file.</li> </ul>                                            | No                                                                  | 0             |
| --model   | <p>Path of the source model file</p> <p><b>NOTE</b><br/>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                 | Yes                                                                 | N/A           |
| --weight  | <p>Path of the weight file</p> <p>This parameter needs to be specified when the source model framework is Caffe.</p> <p><b>NOTE</b><br/>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p> | No                                                                  | N/A           |

| Parameter                 | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --framework               | <p>Framework of the source model</p> <ul style="list-style-type: none"> <li>0: Caffe</li> <li>3: TensorFlow</li> </ul> <p><b>NOTE</b></p> <ul style="list-style-type: none"> <li>This parameter is not mandatory when mode is set to <b>1</b>. If this parameter is not set, the offline model is converted to the JSON format by default (ensure that <b>--om</b> and <b>--framework</b> are consistent).<br/>--framework=0 --om=/home/username/test/resnet18.prototxt</li> <li>This parameter is mandatory when <b>mode</b> is set to <b>0</b> or <b>3</b>.</li> </ul> | Yes                                                                 | N/A           |
| --output                  | <p>Path for storing the converted offline model (including the file name), for example, <b>out/caffe_resnet18</b>.</p> <p>The converted offline model is automatically suffixed with <b>.om</b>.</p> <p><b>NOTE</b><br/>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                                                                            | Yes                                                                 | N/A           |
| --encrypt_mode (reserved) | <p>Encryption mode</p> <ul style="list-style-type: none"> <li>0: encrypted</li> <li>-1: not encrypted</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                         | No                                                                  | -1            |

| Parameter                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --encrypt_key<br>(reserved)  | <p>Path of the random number file used for encryption</p> <p>This parameter is mandatory in encryption mode.</p> <p><b>NOTE</b></p> <ul style="list-style-type: none"> <li>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</li> <li>During the test, you can run the <b>openssl rand 32 -out ek_key</b> command to generate a random number file. In commercial use, a random number file can be generated by using other tools as required.</li> </ul> | No                                                                  | N/A           |
| --hardware_key<br>(reserved) | <p>Path of the encrypted ISV hardware key file</p> <p>This parameter is mandatory in encryption mode.</p> <p><b>NOTE</b></p> <p>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                                                                                                                                                                                                     | No                                                                  | N/A           |
| --certificate<br>(reserved)  | <p>Path of the ISV certificate file used for encryption</p> <p>This parameter is mandatory in encryption mode.</p> <p><b>NOTE</b></p> <p>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                                                                                                                                                                                            | No                                                                  | N/A           |
| --private_key<br>(reserved)  | <p>Path of the ISV private key file used for encryption</p> <p>This parameter is mandatory in encryption mode.</p> <p><b>NOTE</b></p> <p>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                                                                                                                                                                                            | No                                                                  | N/A           |

| Parameter      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-------------------|
| --cal_conf     | <p>Path of the quantization configuration file</p> <p><b>NOTE</b></p> <ul style="list-style-type: none"> <li>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</li> <li>The following is an example of the quantization configuration file:<br/>device: USE_CPU<br/>bin: 150<br/>type: JSD<br/>quantize_algo: NON_OFFSET<br/>inference_with_data_quantized: true<br/>inference_with_weight_quantized: true</li> <li>For details about the quantization configuration file, see <a href="#">4.4 Quantization Configuration</a>.</li> </ul> | No                                                                  | N/A               |
| --check_report | <p>Path of the precheck result file. If this path is not specified, the precheck result is saved in the current path when the model conversion fails or <b>mode</b> is set to <b>3</b> (precheck only).</p> <p><b>NOTE</b></p> <p>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                                                                                                                                                                                   | No                                                                  | check_result.json |
| --h or --help  | Help information                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | No                                                                  | N/A               |
| --input_format | <p>Input data format, either <b>NCHW</b> or <b>NHWC</b></p> <ul style="list-style-type: none"> <li>For TensorFlow, the default value is <b>NHWC</b>. To use the NCHW format, you need to specify <b>NCHW</b>.</li> <li>For Caffe, only <b>NCHW</b> is supported.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                               | No                                                                  | N/A               |

| Parameter          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --input_fp16_nodes | <p>This parameter is used in conjunction with the --is_output_fp16 parameter.</p> <p>Name of the FP16 input node to the second network in the event of network cascade</p> <p>Example: <b>node_name1;node_name2</b></p> <p>For example: Two networks net1 and net2 are cascaded. The output of net1 serves as the input to net2. This parameter is used to specify the name of the FP16 input node to the second network in the event of network cascade.</p>                     | No                                                                  | N/A           |
| --input_shape      | <p>Shape of the input data</p> <p>Example:<br/><b>input_name1:n1,c1,h1,w1;input_name2:n2,c2,h2,w2</b></p> <p><b>input_name</b> must be the node name in the network model before model conversion.</p> <p>If the source model is of a dynamic shape, for example, <b>input_name1:? ,h,w,c</b>. This parameter is mandatory.</p> <p>Replace ? with the actual batch size. It is used to convert the source model with a dynamic shape into a offline model with a fixed shape.</p> | No                                                                  | N/A           |
| --is_output_fp16   | <p>Whether the output data type of the first network is FP16 in the event of network cascade</p> <p>For example: <b>false,true,false,true</b></p> <p>For example: Two networks net1 and net2 are cascaded. The output of net1 serves as the input to net2. This parameter is used to specify the output data type of net1 as FP16.</p>                                                                                                                                            | No                                                                  | false         |

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --json    | <p>Path of the .json file converted from the offline model</p> <p><b>NOTE</b><br/>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p>                                                                                                                                                                                                   | No                                                                  | N/A           |
| --om      | <p>This parameter is mandatory when <b>mode</b> is set to 1.</p> <p>Path of the offline model or model file to be converted to the JSON format For example: <b>/home/username/test/out/caffe_resnet18.om</b> or <b>/home/username/test/resnet18.prototxt</b></p> <p><b>NOTE</b><br/>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</p> | No                                                                  | N/A           |

| Parameter     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --op_name_map | <p>Path of the operator mapping configuration file. This parameter must be specified when the DetectionOutput algorithm is used on the network.</p> <p>For example, the DetectionOutput operator can play different roles in different networks. It can specify the mapping from DetectionOutput (of a Da Vinci model) to the following operators:</p> <ul style="list-style-type: none"> <li>FSRDetectionOutput (of the Faster R-CNN network)</li> <li>SSDDetectionOutput (of the SSD network)</li> <li>RefinedetDetectionOutput (of the RefineDet network)</li> </ul> <p><b>NOTE</b></p> <ul style="list-style-type: none"> <li>The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</li> <li>The following is an example of the operator mapping configuration file:<br/><b>DetectionOutput: SSDDetectionOutput</b></li> </ul> | No                                                                  | N/A           |



| Parameter     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --out_nodes   | <p>Output node</p> <p>If the output node (operator name) is not specified, the output of the last operator layer serves as the model output by default. To check the parameters of a specific operator layer, specify the operator layer by using this parameter. After the model is converted, you can view the parameter information of the specified operator at the end of the .om model file or the JSON file converted from the .om model file.</p> <p>Example:<br/><b>node_name1:0;node_name1:1;node_name2:0</b></p> <p><b>node_name</b> must be the node name in the network model before model conversion. The number after the colon indicates the output index. For example, <b>node_name1:0</b> indicates output 0 of the node named <b>node_name1</b>.</p> | No                                                                  | N/A           |
| --plugin_path | <p>Paths of the custom operator plug-ins</p> <p>Example: <b>/home/a1/b1;/home/a2/b2;/home/a3/b3</b></p> <p><b>NOTE</b><br/>Separate multiple paths by semicolons (;). However semicolons (;) are not allowed in a path; otherwise, path parsing fails.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | No                                                                  | ./plugin      |
| --target      | <p>This parameter can only be set to <b>mini</b>.</p> <p><b>mini</b>: The eltwise operator in quantization supports dual outputs. The roipooling operator in quantization supports int8 output. The conv operator in quantization supports hybrid precision.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | No                                                                  | mini          |
| --ddk_version | Version of the DDK environment to be matched for running a custom operator                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | No                                                                  | N/A           |

| Parameter        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --net_format     | <p>Preferred data format for network operators: <b>ND</b> (<math>N \leq 4</math>) or <b>5D</b>. This parameter is valid only when the input data of the operator on the network supports both the ND and 5D formats.</p> <ul style="list-style-type: none"> <li>• <b>ND</b>: The operators in the model are converted into the common format NCHW.</li> <li>• <b>5D</b>: The operators in the model are converted into Huawei-developed format 5D.</li> </ul>                                                                                                                                                                                                                                                               | No                                                                  | N/A           |
| --insert_op_conf | <p>Configuration file of the preprocessing operator, for example, aipp operator</p> <p><b>NOTE</b></p> <ul style="list-style-type: none"> <li>• The path can contain uppercase letters, lowercase letters, digits, and underscores (_). The file name can contain uppercase letters, lowercase letters, digits, underscores (_), and periods (.).</li> <li>• The following is an example of the configuration file:<br/> <pre>aipp_op {   aipp_mode: static   input_format: YUV420SP_U8   csc_switch: true   var_reci_chn_0: 0.00392157   var_reci_chn_1: 0.00392157   var_reci_chn_2: 0.00392157 }</pre> </li> <li>• For details about the AIPP configuration file, see <a href="#">4.3 AIPP Configuration</a>.</li> </ul> | No                                                                  | N/A           |

| Parameter          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Mandatory or Not (Depending on Whether the Value of Mode Is 0 or 3) | Default Value |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------|
| --fp16_high_prec   | <p>Whether to generate a high-accuracy FP16 Da Vinci model.</p> <ul style="list-style-type: none"> <li><b>0</b> (default): Generate a common FP16 Da Vinci model, which shows better inference performance.</li> <li><b>1</b>: Generate a high-accuracy FP16 Da Vinci model, which offers higher inference accuracy.</li> </ul> <p>High-precision model generation supports only the following operators: Convolution, Pooling, and FullConnection of Caffe and tf.nn.conv2d and tf.nn.max_pool of TensorFlow.</p> | No                                                                  | 0             |
| --output_type      | <p>Network output data type</p> <ul style="list-style-type: none"> <li><b>FP32</b> (default): It is recommended for classification and detection networks.</li> <li><b>UINT8</b>: It is recommended for image super-resolution networks for better inference performance.</li> </ul>                                                                                                                                                                                                                               | No                                                                  | FP32          |
| --enable_l2dynamic | <p>L2 dynamic optimization enable. This parameter may affect the inference performance of the network model. If the performance does not meet the requirement, you can disable this switch to verify the impact on the performance.</p> <ul style="list-style-type: none"> <li><b>true</b>: L2 dynamic optimization enabled</li> <li><b>false</b>: L2 dynamic optimization disabled</li> </ul>                                                                                                                     | No                                                                  | true          |

## 4.2.2 Model Conversion Using OMG

### Preparations

- **Environment preparation**

This document describes how to convert models in command line interface (CLI) mode. For details about how to convert models in the Mind Studio IDE, see *Ascend 310 Mind Studio Basic Operations*.

The following describes the scenario where the DDK is independently installed.

If the DDK installation user is **HwHiAiUser** and the log fails to be output to the screen, see [4.5.1 What Do I Do If No Log Is Output to the Screen When the DDK Installation User Is HwHiAiUser?](#). If the OS and architecture configuration of the DDK server is Arm (aarch64) and model conversion takes too long, see [4.5.2 What Do I Do If Model Conversion Takes Too Long When the OS and Architecture Configuration of the DDK Server is Arm \(aarch64\)?](#).

- Upload the model file (\*.prototxt), weight file (\*.caffemodel) required by model conversion to the **/home/username/test/** directory on the DDK server as the DDK installation user.

## Sample

**Step 1** Log in to the DDK server as the DDK installation user.

**Step 2** Set the following environment variable:

```
export LD_LIBRARY_PATH=DDKInstallationDirectory/ddk/uihost/lib
```

**Step 3** Find OMG in the **ddk/uihost/bin** directory under the DDK installation directory. Run the following command to generate a model file. (The directories and files in the command are for reference only.)

```
./omg --model=/home/username/test/resnet18.prototxt --weight=/home/username/test/
resnet18.caffemodel --framework=0 --output=/home/username/test/out/caffe_resnet18
```

After the command is executed successfully, you can view the model file (for example, **caffe\_resnet18.om**).

### NOTE

A command that exceeds one line will be automatically wrapped due to the restriction of the PDF document format. Therefore, if you want to use the command in the example directly, you need to manually merge the lines into one line and separate the parameters with spaces.

For details about how to fix the following errors during model conversion (**It is recommended to convert layers-structure to layer-structure by caffe tool and Type XXX unsupported**), see [4.5 FAQs](#).

**Step 4** (Optional) If an output node is specified (that is, the **--out\_nodes** parameter is set) during model conversion and the information of the last operator layer fails to be viewed in the converted .om model, convert the .om model file to the JSON format by running the following command and check the .json file:

```
omg --mode=1 --om=/home/username/test/caffe_resnet18.om --json=/home/username/test/out/resnet.json
```

----End

## 4.3 AIPP Configuration

## 4.3.1 Overview

AIPP involves image resizing, CSC, and mean value subtraction and factor multiplication (for pixel changing). All these functions are implemented by the AI Core.

Static AIPP and dynamic AIPP modes are supported. However, the two modes are mutually exclusive.

- Static AIPP: During model conversion, set the AIPP mode to static and set the AIPP parameters. After the Da Vinci model is generated, the AIPP parameter values are saved in the model. The same AIPP parameter configurations are used in each model inference phase.

If the static AIPP mode is used, batches share the same set of AIPP parameters.

- Dynamic AIPP: During model conversion, set the AIPP mode to dynamic. The AIPP parameters must be set in the code of the inference engine before each model inference. In this way, different sets of AIPP configurations can be used for different model inference phases. For details about the dynamic AIPP APIs and usage example, see **AIPP Configuration APIs** in *Matrix API Reference*.

In dynamic AIPP mode, batches use separate parameter configurations (such as the crop and resize attributes) defined by the dynamic parameter structure. For details about the dynamic parameter structure, see [4.3.7 Parameter Structure for Dynamic AIPP](#).

AIPP supports the following image input formats: **YUV420SP\_U8**, **XRGB8888\_U8**, **RGB888\_U8**, and **YUV400\_U8**.

- YUV420SP\_U8 is classified into YUV420SP\_UV(NV12) and YUV420SP\_VU(NV21) based on the UV sequence. The default format is YUV420SP\_UV(NV12).
  - If **rbuf\_swap\_switch** in [4.3.2 Configuration File Template](#) is set to **false**, the AIPP output format is YUV420SP\_UV(NV12)
  - If **rbuf\_swap\_switch** in [4.3.2 Configuration File Template](#) is set to **true**, the AIPP output format is YUV420SP\_UV(NV12).
- For RGB888\_U8, the AIPP output format varies according to the value of **rbuf\_swap\_switch**.
  - If **rbuf\_swap\_switch** in [4.3.2 Configuration File Template](#) is set to **false**, the AIPP output format is RGB888\_U8.
  - If **rbuf\_swap\_switch** in [4.3.2 Configuration File Template](#) is set to **true**, the AIPP output format is BGR888\_U8.

#### NOTICE

- The input format of AIPP is **YUV420SP\_U8** (the default format is **YUV420SP\_UV**). If the format is **YUV420SP\_VU**, set **rbuv\_swap\_switch** to **false**. Otherwise, the output result will be affected.
- When AIPP is enabled, the model input is in **RGB888\_U8** or **BGR888\_U8** format. The two formats correspond to different CSC matrices.
- Pay attention to the following requirements on the input images:
  - If AIPP is enabled for model conversion and the input image format is **XRGB8888\_U8** or **RGB888\_U8**, the converted model accepts NHWC input only.
  - If AIPP is disabled for model conversion, the converted model accepts NCHW input only for inference. Therefore, you need to manually convert the NHWC data to NCHW data.

## 4.3.2 Configuration File Template

The following describes the configuration file.

```
The AIPP configuration starts with the aipp_op flag, which indicates that it is the configuration of an
AIPP operator. Multiple aipp_op flags can be configured.
aipp_op {
#
AIPP provides the following features: CSC, cropping, mean subtraction, multiplication factor, data
exchange between channels, and single-line mode.
The input images must in the UINT8 data type.
To use this configuration file, delete the comments of parameters to be configured and set the parameter
values as required.
The parameter values in the template are default values. The input_format attribute is mandatory, while
other attributes are optional.
#===== Global settings (start)
=====
aipp_mode specifies the AIPP mode. This parameter is mandatory.
Type: enum
Value range: dynamic or static (dynamic indicates dynamic AIPP, and static indicates static AIPP.)
aipp_mode:

related_input_rank is optional. It indicates the processing start of the inputs. The default value is 0,
which indicates that AIPP processing starts from input 0. For example, if the model has two inputs, to have
the AIPP processing start from the second input, set related_input_rank to 1.
Type: int
Value range: ≥ 0
related_input_rank: 0

input_edge_idx is optional. If a model input is shared by multiple operators, that is, the data operator is
followed by more operators, set this parameter to perform different AIPP processing for different output
dimensions of the Data operator.
Type: int
Value range: ≥ 0
input_edge_idx: 0
#===== Global settings (end)
=====

#===== Dynamic AIPP settings (start)
=====
Maximum size of the input image (must be greater than or equal to that of the source image)
Type: int
max_src_image_size: 0
```

```
Whether to enable rotation. This parameter is reserved. Rotation is not supported currently.
Type: bool
Value range: true (yes) or false (no)
support_rotation: false
===== Dynamic AIPP settings (end)
=====

===== Static AIPP settings (start)
=====
=====
Input image format
Type: enum
Value: YUV420SP_U8, XRGB8888_U8, RGB888_U8, or YUV400_U8
input_format :

Image width and height
Type: uint13
Value range: [0, 4096]. For YUV420SP_U8 images, the value must be an even number.
Note: Set src_image_size_w and src_image_size_h to the actual image width and height respectively. If
they are not set or set to 0, the width and height defined in the network input are used.
src_image_size_w: 0
src_image_size_h: 0

c_padding_value: 0.0
===== Crop settings (For the configuration example, see section AIPP Configuration > Crop/
Padding Configuration.) =====
Whether to enable crop in AIPP
Type: bool
Value range: true (yes) or false (no)
crop :false

Horizontal and vertical coordinates of the crop start. W and H defined in the network input are used as
the size of the cut out image.
Type: uint13
Value range: [0, 4096]. For YUV420SP_U8 images, the value must be an even number.
Note: load_start_pos_w plus W defined in the network input must be less than or equal to
src_image_size_w. load_start_pos_h plus H defined in the network input must be less than or equal to
src_image_size_h.
load_start_pos_w: 0
load_start_pos_h: 0

Size of the cut out image
Type: uint13
Value range: even number within [0, 4096], load_start_pos_w + crop_size_w ≤ src_image_size_w, and
load_start_pos_h + crop_size_h ≤ src_image_size_h
crop_size_w: 0
crop_size_h: 0

===== Resize settings =====
Whether to enable resize. This parameter is reserved. Rotation is not supported currently.
Type: bool
Value range: true (yes) or false (no)
resize :false

Width and height of the resized image. This parameter is reserved. Resize is not supported currently.
Type: uint13
Value range: even number within [0, 4096], less than src_image_size
resize_output_w: 0
resize_output_h: 0

===== Padding settings (For the configuration example, see section AIPP Configuration > Crop/
Padding Configuration.) =====
Whether to enable padding in AIPP
Type: bool
Value range: true (yes) or false (no)
```

```
padding: false

Padding in the C direction, for static AIPP
Type: float16
left_padding_size: 0
right_padding_size: 0
top_padding_size: 0
bottom_padding_size: 0

#=====
Rotation settings =====
Rotation angle. This parameter is reserved. Rotation is not supported currently.
Type: uint8
Value range: {0, 1, 2, 3}, where, 0 indicates zero rotation, 1 indicates 90° clockwise rotation, 2 indicates
180° clockwise rotation, and 3 indicates 270° clockwise rotation.
rotation_angle :0

#=====
CSC settings (For the configuration example, see section AIPP Configuration > CSC
Configuration.) =====
Whether to enable CSC in static AIPP
Type: bool
Value: true or false. The value true indicates enabled, and the value false indicates disabled.
csc_switch :false

Whether to enable R/B or U/V channel swap before CSC
Type: bool
Value: true or false. The value true indicates enabled, and the value false indicates disabled.
rbuv_swap_switch: false

Whether to enable single line mode (in this mode, only the first line of the cut out image is processed)
Type: bool
Value: true or false. The value true indicates enabled, and the value false indicates disabled.
single_line_mode: false

If the CSC is disabled (false), this function is bypassed.
If the input image has four channels, the first channel is ignored.
YUV2BGR conversion:
| B | | matrix_r0c0 matrix_r0c1 matrix_r0c2 | | Y - input_bias_0 |
| G | = | matrix_r1c0 matrix_r1c1 matrix_r1c2 | | U - input_bias_1 | >> 8
| R | | matrix_r2c0 matrix_r2c1 matrix_r2c2 | | V - input_bias_2 |
BGR2YUV conversion:
| Y | | matrix_r0c0 matrix_r0c1 matrix_r0c2 | | B | | output_bias_0 |
| U | = | matrix_r1c0 matrix_r1c1 matrix_r1c2 | | G | >> 8 + | output_bias_1 |
| V | | matrix_r2c0 matrix_r2c1 matrix_r2c2 | | R | | output_bias_2 |

Elements in a 3 x 3 CSC matrix
Type: int16
Value range: [-32768, +32767]
matrix_r0c0 :298
matrix_r0c1 :516
matrix_r0c2 :0
matrix_r1c0 :298
matrix_r1c1 :-100
matrix_r1c2 :-208
matrix_r2c0 :298
matrix_r2c1 :0
matrix_r2c2 :409

Output bias for RGB2YUV conversion
Type: uint8
Value range: [0, 255]
output_bias_0: 16
output_bias_1: 128
output_bias_2: 128

Input bias for YUV2RGB conversion
Type: uint8
Value range: [0, 255]
```



```
input_bias_0: 16
input_bias_1: 128
input_bias_2: 128

#===== Mean subtraction and multiplication factor settings
#=====
The computation rules are as follows:
For uint8->uint8, this function is bypassed.
For uint8->int8: pixel_out_chx(i) = pixel_in_chx(i) - mean_chn_i
For uint8->fp16: pixel_out_chx(i) = [pixel_in_chx(i) - mean_chn_i - min_chn_i] * var_reci_chn

Mean value of channel n
Type: uint8
Value range: [0, 255]
mean_chn_0: 0
mean_chn_1: 0
mean_chn_2: 0

Minimum value of channel n
Type: float16
Value range: [-65504, +65504]
min_chn_0 :0.0
min_chn_1 :0.0
min_chn_2 :0.0

Standard deviation of channel n or reciprocal of (Max. - Min.)
Type: float16
Value range: [-65504, +65504]
var_reci_chn_0: 1.0
var_reci_chn_1: 1.0
var_reci_chn_2: 1.0
}
#===== Static AIPP settings (end)
#=====
#=====
```

### 4.3.3 CSC Configuration

After confirming the image formats before and after AIPP (that is, the image format of the model input and image format during model training), you can determine the CSC parameter values. (The value of **matrix\_r\*c\*** is fixed and does not need to be adjusted.) The following describes some configuration references.

Before AIPP, the following two typical CSC configurations are provided for the images or videos (in various color encoding modes such as YUV420SP\_U8, XRGB8888\_U8, RGB888\_U8, and YUV400\_U8) input to the model:

- For JPEG image files (such as JPG, JPEG, JPG, and JPEG), you can select from the CSC configurations in the JPEG columns in the following tables.
- For the decoded video data, the CSC parameters can be configured according to different color video digital standards (such as BT.601 and BT.709).

#### YUV420SP\_U8 to YVU444SP\_U8

```
aipp_op {
 aipp_mode: static
 input_format : YUV420SP_U8
 csc_switch : false
 rbuv_swap_switch : true
}
```

## YUV420SP\_U8 to RGB888\_U8

**Table 4-1** YUV420SP\_U8 to RGB888\_U8

| JPEG                                                                                                                                                                                                                                                                                                                                                                 | BT-601NARROW                                                                                                                                                                                                                                                                                                                                                           | BT-709NARROW                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>aipp_op {   aipp_mode: static   input_format : YUV420SP_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 256   matrix_r0c1 : 0   matrix_r0c2 : 359   matrix_r1c0 : 256   matrix_r1c1 : -88   matrix_r1c2 : -183   matrix_r2c0 : 256   matrix_r2c1 : 454   matrix_r2c2 : 0   input_bias_0 : 0   input_bias_1 : 128   input_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : YUV420SP_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 298   matrix_r0c1 : 0   matrix_r0c2 : 409   matrix_r1c0 : 298   matrix_r1c1 : -100   matrix_r1c2 : -208   matrix_r2c0 : 298   matrix_r2c1 : 516   matrix_r2c2 : 0   input_bias_0 : 16   input_bias_1 : 128   input_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : YUV420SP_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 298   matrix_r0c1 : 0   matrix_r0c2 : 459   matrix_r1c0 : 298   matrix_r1c1 : -55   matrix_r1c2 : -136   matrix_r2c0 : 298   matrix_r2c1 : 541   matrix_r2c2 : 0   input_bias_0 : 16   input_bias_1 : 128   input_bias_2 : 128 }</pre> |

## YUV420SP\_U8 to BGR888\_U8

**Table 4-2** YUV420SP\_U8 to BGR888\_U8

| JPEG                                                                                                                                                                                                                                                                                                                                                                 | BT-601NARROW                                                                                                                                                                                                                                                                                                                                                           | BT-709NARROW                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>aipp_op {   aipp_mode: static   input_format : YUV420SP_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 256   matrix_r0c1 : 454   matrix_r0c2 : 0   matrix_r1c0 : 256   matrix_r1c1 : -88   matrix_r1c2 : -183   matrix_r2c0 : 256   matrix_r2c1 : 0   matrix_r2c2 : 359   input_bias_0 : 0   input_bias_1 : 128   input_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : YUV420SP_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 298   matrix_r0c1 : 516   matrix_r0c2 : 0   matrix_r1c0 : 298   matrix_r1c1 : -100   matrix_r1c2 : -208   matrix_r2c0 : 298   matrix_r2c1 : 0   matrix_r2c2 : 409   input_bias_0 : 16   input_bias_1 : 128   input_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : YUV420SP_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 298   matrix_r0c1 : 541   matrix_r0c2 : 0   matrix_r1c0 : 298   matrix_r1c1 : -55   matrix_r1c2 : -136   matrix_r2c0 : 298   matrix_r2c1 : 0   matrix_r2c2 : 459   input_bias_0 : 16   input_bias_1 : 128   input_bias_2 : 128 }</pre> |

## YUV420SP\_U8 to Gray

```
aipp_op {
 aipp_mode: static
 input_format : YUV420SP_U8
 csc_switch : true
 rbuv_swap_switch : false
 matrix_r0c0 : 256
 matrix_r0c1 : 0
 matrix_r0c2 : 0
}
```

```
matrix_r1c0 : 0
matrix_r1c1 : 0
matrix_r1c2 : 0
matrix_r2c0 : 0
matrix_r2c1 : 0
matrix_r2c2 : 0
input_bias_0 : 0
input_bias_1 : 0
input_bias_2 : 0
}
```

## XRGB8888\_U8 to YUV444SP\_U8

Table 4-3 XRGB8888\_U8 to YUV444SP\_U8

| JPEG                                                                                                                                                                                                                                                                                                                                                                      | BT-601NARROW                                                                                                                                                                                                                                                                                                                                                              | BT-709NARROW                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>aipp_op {   aipp_mode: static   input_format : XRGB8888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 77   matrix_r0c1 : 150   matrix_r0c2 : 29   matrix_r1c0 : -43   matrix_r1c1 : -85   matrix_r1c2 : 128   matrix_r2c0 : 128   matrix_r2c1 : -107   matrix_r2c2 : -21   output_bias_0 : 0   output_bias_1 : 128   output_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : XRGB8888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 66   matrix_r0c1 : 129   matrix_r0c2 : 25   matrix_r1c0 : -38   matrix_r1c1 : -74   matrix_r1c2 : 112   matrix_r2c0 : 112   matrix_r2c1 : -94   matrix_r2c2 : -18   output_bias_0 : 16   output_bias_1 : 128   output_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : XRGB8888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 47   matrix_r0c1 : 157   matrix_r0c2 : 16   matrix_r1c0 : -26   matrix_r1c1 : -87   matrix_r1c2 : 112   matrix_r2c0 : 112   matrix_r2c1 : -102   matrix_r2c2 : -10   output_bias_0 : 16   output_bias_1 : 128   output_bias_2 : 128 }</pre> |

## XRGB8888\_U8 to YVU444SP\_U8

Table 4-4 XRGB8888\_U8 to YVU444SP\_U8

| JPEG                                                                                                                                                                                                                                                                                                                                                                      | BT-601NARROW                                                                                                                                                                                                                                                                                                                                                              | BT-709NARROW                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>aipp_op {   aipp_mode: static   input_format : XRGB8888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 77   matrix_r0c1 : 150   matrix_r0c2 : 29   matrix_r1c0 : 128   matrix_r1c1 : -107   matrix_r1c2 : -21   matrix_r2c0 : -43   matrix_r2c1 : -85   matrix_r2c2 : 128   output_bias_0 : 0   output_bias_1 : 128   output_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : XRGB8888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 66   matrix_r0c1 : 129   matrix_r0c2 : 25   matrix_r1c0 : 112   matrix_r1c1 : -94   matrix_r1c2 : -18   matrix_r2c0 : -38   matrix_r2c1 : -74   matrix_r2c2 : 112   output_bias_0 : 16   output_bias_1 : 128   output_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : XRGB8888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 47   matrix_r0c1 : 157   matrix_r0c2 : 16   matrix_r1c0 : 112   matrix_r1c1 : -102   matrix_r1c2 : -10   matrix_r2c0 : -26   matrix_r2c1 : -87   matrix_r2c2 : 112   output_bias_0 : 16   output_bias_1 : 128   output_bias_2 : 128 }</pre> |

**XRGB8888\_U8 to Gray**

```
aipp_op {
 aipp_mode: static
 input_format : XRGB8888_U8
 csc_switch : true
 rbuf_swap_switch : false
 matrix_r0c0 : 76
 matrix_r0c1 : 150
 matrix_r0c2 : 30
 matrix_r1c0 : 0
 matrix_r1c1 : 0
 matrix_r1c2 : 0
 matrix_r2c0 : 0
 matrix_r2c1 : 0
 matrix_r2c2 : 0
 output_bias_0 : 0
 output_bias_1 : 0
 output_bias_2 : 0
}
```

**RGB888\_U8 to BGR888\_U8**

```
aipp_op {
 aipp_mode: static
 input_format : RGB888_U8
 csc_switch : false
 rbuf_swap_switch : true
}
```

**RGB888\_U8 to YUV444SP\_U8**

**Table 4-5** RGB888\_U8 to YUV444SP\_U8

| JPEG                                                                                                                                                                                                                                                                                                                                                                                                                                        | BT-601NARROW                                                                                                                                                                                                                                                                                                                                                                                                                                | BT-709NARROW                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>aipp_op {<br/>  aipp_mode: static<br/>  input_format : RGB888_U8<br/>  csc_switch : true<br/>  rbuf_swap_switch : false<br/>  matrix_r0c0 : 77<br/>  matrix_r0c1 : 150<br/>  matrix_r0c2 : 29<br/>  matrix_r1c0 : -43<br/>  matrix_r1c1 : -85<br/>  matrix_r1c2 : 128<br/>  matrix_r2c0 : 128<br/>  matrix_r2c1 : -107<br/>  matrix_r2c2 : -21<br/>  output_bias_0 : 0<br/>  output_bias_1 : 128<br/>  output_bias_2 : 128<br/>}</pre> | <pre>aipp_op {<br/>  aipp_mode: static<br/>  input_format : RGB888_U8<br/>  csc_switch : true<br/>  rbuf_swap_switch : false<br/>  matrix_r0c0 : 66<br/>  matrix_r0c1 : 129<br/>  matrix_r0c2 : 25<br/>  matrix_r1c0 : -38<br/>  matrix_r1c1 : -74<br/>  matrix_r1c2 : 112<br/>  matrix_r2c0 : 112<br/>  matrix_r2c1 : -94<br/>  matrix_r2c2 : -18<br/>  output_bias_0 : 16<br/>  output_bias_1 : 128<br/>  output_bias_2 : 128<br/>}</pre> | <pre>aipp_op {<br/>  aipp_mode: static<br/>  input_format : RGB888_U8<br/>  csc_switch : true<br/>  rbuf_swap_switch : false<br/>  matrix_r0c0 : 47<br/>  matrix_r0c1 : 157<br/>  matrix_r0c2 : 16<br/>  matrix_r1c0 : -26<br/>  matrix_r1c1 : -87<br/>  matrix_r1c2 : 112<br/>  matrix_r2c0 : 112<br/>  matrix_r2c1 : -102<br/>  matrix_r2c2 : -10<br/>  output_bias_0 : 16<br/>  output_bias_1 : 128<br/>  output_bias_2 : 128<br/>}</pre> |

RGB888\_U8 to YVU444SP\_U8

Table 4-6 RGB888\_U8 to YVU444SP\_U8

| JPEG                                                                                                                                                                                                                                                                                                                                                                    | BT-601NARROW                                                                                                                                                                                                                                                                                                                                                            | BT-709NARROW                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>aipp_op {   aipp_mode: static   input_format : RGB888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 77   matrix_r0c1 : 150   matrix_r0c2 : 29   matrix_r1c0 : 128   matrix_r1c1 : -107   matrix_r1c2 : -21   matrix_r2c0 : -43   matrix_r2c1 : -85   matrix_r2c2 : 128   output_bias_0 : 0   output_bias_1 : 128   output_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : RGB888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 66   matrix_r0c1 : 129   matrix_r0c2 : 25   matrix_r1c0 : 112   matrix_r1c1 : -94   matrix_r1c2 : -18   matrix_r2c0 : -38   matrix_r2c1 : -74   matrix_r2c2 : 112   output_bias_0 : 16   output_bias_1 : 128   output_bias_2 : 128 }</pre> | <pre>aipp_op {   aipp_mode: static   input_format : RGB888_U8   csc_switch : true   rbuv_swap_switch : false   matrix_r0c0 : 47   matrix_r0c1 : 157   matrix_r0c2 : 16   matrix_r1c0 : 112   matrix_r1c1 : -102   matrix_r1c2 : -10   matrix_r2c0 : -26   matrix_r2c1 : -87   matrix_r2c2 : 112   output_bias_0 : 16   output_bias_1 : 128   output_bias_2 : 128 }</pre> |

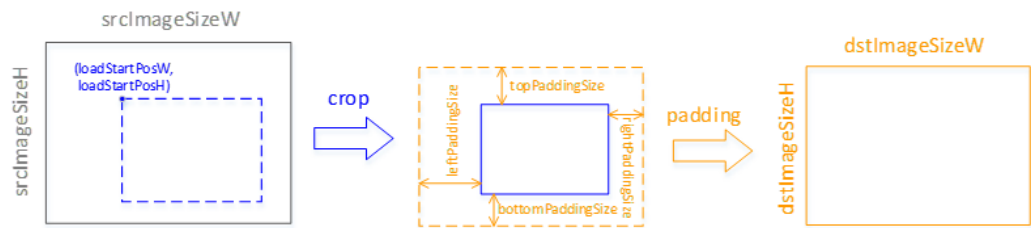
RGB888\_U8 to Gray

```
aipp_op {
 aipp_mode: static
 input_format : RGB888_U8
 csc_switch : true
 rbuv_swap_switch : false
 matrix_r0c0 : 76
 matrix_r0c1 : 150
 matrix_r0c2 : 30
 matrix_r1c0 : 0
 matrix_r1c1 : 0
 matrix_r1c2 : 0
 matrix_r2c0 : 0
 matrix_r2c1 : 0
 matrix_r2c2 : 0
 output_bias_0 : 0
 output_bias_1 : 0
 output_bias_2 : 0
}
```

4.3.4 Crop/Padding Configuration

Currently, AIPP supports image resize by crop and padding.

Figure 4-3 Image resize



For YUV420SP\_U8 images, the values of **load\_start\_pos\_w**, **load\_start\_pos\_h**, **crop\_size\_w**, and **crop\_size\_h** must be even numbers. The width/height of the cut out image must be the same as **W/ H** defined in the network input.

A configuration example is as follows:

```
aipp_op {
 aipp_mode: static
 input_format : YUV400_U8

 src_image_size_w :320
 src_image_size_h :240

 crop :true
 load_start_pos_w :10
 load_start_pos_h :20
 crop_size_w :50
 crop_size_h :60

 padding : true
 left_padding_size :10
 right_padding_size :20
 top_padding_size :10
 bottom_padding_size :20
}
```

### 4.3.5 AIPP Configurations According to Model Inputs

You can specify more than one set of AIPP configurations in the AIPP configuration file. Different AIPP processing is performed for different model inputs. Different sets of AIPP parameters are separated by the **aipp\_op** flags. For example:

```
aipp_op {
 related_input_rank: 0
 aipp_mode : dynamic
 max_src_image_size: 60000
}
aipp_op {
 related_input_rank : 1
 aipp_mode : dynamic
 max_src_image_size: 80000
}
```

The preceding configuration defines two sets of AIPP parameters, corresponding to AIPP on the first and second inputs of the model. Dynamic AIPP is adopted. The maximum image sizes allowed by the two inputs are 60000 bytes and 80000 bytes respectively.

### 4.3.6 AIPP Verification of the Model Input Size

If static or dynamic AIPP is configured, the size (**input\_size**) of the generated image received by the Da Vinci model is changed due to operations such as crop and padding. In the model inference phase, the OME verifies the size of the input image. In dynamic AIPP mode, the input size must be less than or equal to **max\_src\_image\_size**. In static AIPP mode, calculate the input size according to [Table 4-7](#).

For static AIPP, assume that the batch count of the model is **N**, the image width is **src\_image\_size\_w**, and the height width is **src\_image\_size\_h**. [Table 4-7](#) describes the verification formulas for the size of the model input.

**Table 4-7** input\_size verification formulas

| input_format | input_size                                                          |
|--------------|---------------------------------------------------------------------|
| YUV400_U8    | $N * \text{src\_image\_size\_w} * \text{src\_image\_size\_h}$       |
| YUV420SP_U8  | $N * \text{src\_image\_size\_w} * \text{src\_image\_size\_h} * 1.5$ |
| XRGB8888_U8  | $N * \text{src\_image\_size\_w} * \text{src\_image\_size\_h} * 4$   |
| RGB888_U8    | $N * \text{src\_image\_size\_w} * \text{src\_image\_size\_h} * 3$   |

The OMG adds a model input before a model with dynamic AIPP is generated, according to the following formula:

$\text{sizeof}(\text{kAippDynamicPara}) + (\text{batch\_count} - 1) \times \text{sizeof}(\text{kAippDynamicBatchPara})$

### 4.3.7 Parameter Structure for Dynamic AIPP

```
typedef struct tagAippDynamicBatchPara
{
 int8_t cropSwitch; //crop switch
 int8_t scfSwitch; //resize switch
 int8_t paddingSwitch; // 0: unable padding,
 // 1: padding config value,sfr_filling_hblank_ch0 ~ sfr_filling_hblank_ch2
 // 2: padding source picture data, single row/column copy
 // 3: padding source picture data, block copy
 // 4: padding source picture data, mirror copy
 int8_t rotateSwitch; //Rotation switch: 0 indicates zero rotation, 1 indicates 90° clockwise rotation, 2
 // indicates 180° clockwise rotation, and 3 indicates 270° clockwise rotation.
 int8_t reserve[4];
 int32_t cropStartPosW; //the start horizontal position of cropping
 int32_t cropStartPosH; //the start vertical position of cropping
 int32_t cropSizeW; //crop width
 int32_t cropSizeH; //crop height
 int32_t scfInputSizeW; //input width of scf
 int32_t scfInputSizeH; //input height of scf
 int32_t scfOutputSizeW; //output width of scf
 int32_t scfOutputSizeH; //output height of scf
 int32_t paddingSizeTop; //top padding size
 int32_t paddingSizeBottom; //bottom padding size
 int32_t paddingSizeLeft; //left padding size
 int32_t paddingSizeRight; //right padding size
 int16_t dtcPixelMeanChn0; //mean value of channel 0
 int16_t dtcPixelMeanChn1; //mean value of channel 1
 int16_t dtcPixelMeanChn2; //mean value of channel 2
 int16_t dtcPixelMeanChn3; //mean value of channel 3
#ifdef DAVINCI_TINY
 uint16_t dtcPixelMinChn0; //min value of channel 0
 uint16_t dtcPixelMinChn1; //min value of channel 1
 uint16_t dtcPixelMinChn2; //min value of channel 2
 uint16_t dtcPixelMinChn3; //min value of channel 3
 uint16_t dtcPixelVarReciChn0; //sfr_dtc_pixel_variance_reci_ch0
 uint16_t dtcPixelVarReciChn1; //sfr_dtc_pixel_variance_reci_ch1
 uint16_t dtcPixelVarReciChn2; //sfr_dtc_pixel_variance_reci_ch2
 uint16_t dtcPixelVarReciChn3; //sfr_dtc_pixel_variance_reci_ch3
 int8_t reserve1[16]; //32B assign, for ub copy
#endif
}kAippDynamicBatchPara;
typedef struct tagAippDynamicPara
{
 uint8_t inputFormat; //Input format: YUV420SP_U8, XRGB8888_U8, or RGB888_U8
 //uint8_t outDataType; //output data type: CC_DATA_HALF,CC_DATA_INT8, CC_DATA_UINT8
}
```

```
int8_t cscSwitch; //csc switch
int8_t rbuvSwapSwitch; //rb/ub swap switch
int8_t axSwapSwitch; //RGBA->ARGB, YUVA->AYUV swap switch
int8_t batchNum; //batch parameter number
int8_t reserve1[3];
int32_t srcImageSizeW; //source image width
int32_t srcImageSizeH; //source image height
int16_t cscMatrixR0C0; //csc_matrix_r0_c0
int16_t cscMatrixR0C1; //csc_matrix_r0_c1
int16_t cscMatrixR0C2; //csc_matrix_r0_c2
int16_t cscMatrixR1C0; //csc_matrix_r1_c0
int16_t cscMatrixR1C1; //csc_matrix_r1_c1
int16_t cscMatrixR1C2; //csc_matrix_r1_c2
int16_t cscMatrixR2C0; //csc_matrix_r2_c0
int16_t cscMatrixR2C1; //csc_matrix_r2_c1
int16_t cscMatrixR2C2; //csc_matrix_r2_c2
int16_t reserve2[3];
uint8_t cscOutputBiasR0; //output bias for RGB to YUV, element of row 0, unsigned number
uint8_t cscOutputBiasR1; //output bias for RGB to YUV, element of row 1, unsigned number
uint8_t cscOutputBiasR2; //output bias for RGB to YUV, element of row 2, unsigned number
uint8_t cscInputBiasR0; //input bias for YUV to RGB, element of row 0, unsigned number
uint8_t cscInputBiasR1; //input bias for YUV to RGB, element of row 1, unsigned number
uint8_t cscInputBiasR2; //input bias for YUV to RGB, element of row 2, unsigned number
uint8_t reserve3[2];
int8_t reserve4[16]; //32B assign, for ub copy
kAippDynamicBatchPara aippBatchPara; //allow transfer several batch para.
} kAippDynamicPara;
```

## 4.4 Quantization Configuration

### 4.4.1 Overview

Quantization herein refers to low-bit quantization performed on high-accuracy data, so as to save the network memory usage, reduce the transmission delay, and improve the operation efficiency.

Currently, quantization using the Convolution, Full Connection, and ConvolutionDepthwise operators is supported, including weight, offset, and data quantization. Two quantization modes are supported: non-offset mode and data-offset mode.

The quantized weight and offset are stored in the Da Vinci model during Da Vinci model generation. In model inference, the quantized weight and offset are used for data computation.

The concepts are described as follows:

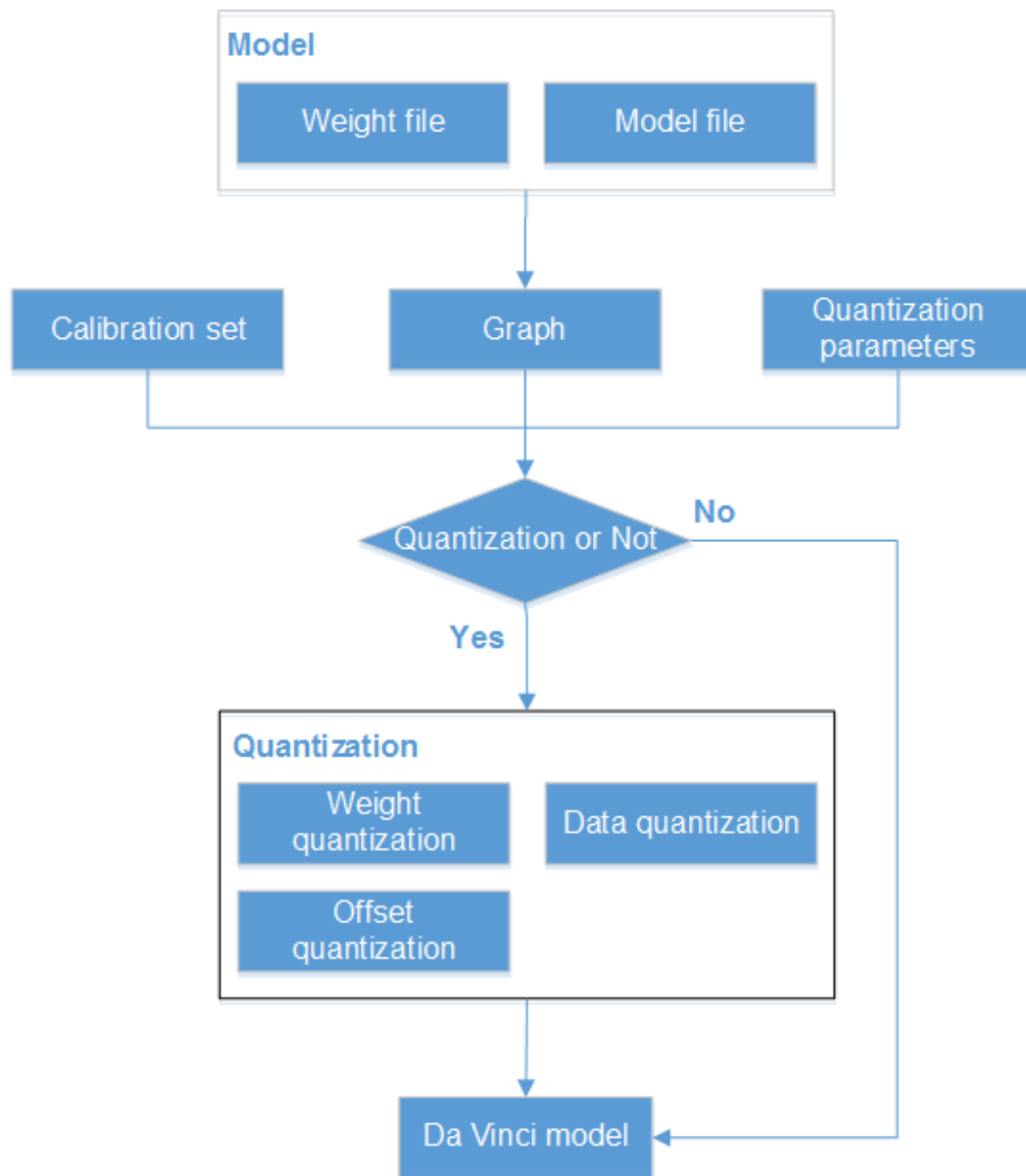
- Non-offset mode: Both the weight and data are offset.
- Data-offset mode: The data is offset while the weight is not.
- Weight quantification: Performs int8 quantization on the weight file according to the quantization algorithm. In with-offset quantization mode, the int8 weight, scale, and offset are output. In non-offset quantization mode, the int8 weight and scale are output. Currently, only the non-offset quantization mode is supported for weight quantization.
- Data quantification: Performs inference by using limited inputs (a calibration set, used to train quantization parameters and ensure the accuracy). The scale and offset of the quantized data are obtained in the data-offset quantization



mode or the scale of the quantized data is obtained in non-offset quantization mode, according to the frequency statistics and divergence algorithm.

- Offset quantization: Quantizes the FP32 offset data into the INT32 output based on the weight-quantized scale and data-quantized scale

**Figure 4-4** Quantization processing process



## 4.4.2 Configuration File Template

Quantization is supported in single-input and multi-input network scenarios. You can use this configuration file template directly or make modifications as required for adaptation. Note that some of the parameters in the configuration file are mandatory. [4.4.3 Description of the Configuration Parameters](#) describes the parameters in the configuration file and whether a parameter is mandatory.

The configuration file cannot contain Chinese characters.

## Single-Input Network Scenario

With the image input, the **omg** command specifies the quantization configuration. The following is an example of the configuration file:

```
device:USE_CPU
quantize_algo:HALF_OFFSET
weight_type:VECTOR_TYPE
bin:150
type:KL
inference_with_data_quantized:false
inference_with_weight_quantized:true
super_parameter:
{
 min_percentile:PERCENTILE_HIGH
 max_percentile:PERCENTILE_MID
 start_ratio:0.7
 end_ratio:1.3
 step_ratio:0.01
}
exclude_op:'fc1000'
batch_count:50
preprocess_parameter:
{
 input_type:IMAGE
 image_format:BGR
 input_file_path:'calibration/image_set'
 mean_value:104.0
 mean_value:117.0
 mean_value:123.0
 standard_deviation:1.0
}
```

## Multi-Input Network Scenario

With image+binary inputs, the **omg** command specifies the quantization configuration. The following is an example of the configuration file:

```
device:USE_CPU
quantize_algo:HALF_OFFSET
weight_type:VECTOR_TYPE
bin:150
type:KL
inference_with_data_quantized:false
inference_with_weight_quantized:true
super_parameter:
{
 min_percentile:PERCENTILE_HIGH
 max_percentile:PERCENTILE_MID
 start_ratio:0.7
 end_ratio:1.3
 step_ratio:0.01
}
exclude_op:'fc1000'
batch_count:50
preprocess_parameter:
{
 input_type:IMAGE
 image_format:BGR
 input_file_path:'calibration/image_set'
 mean_value:104.0
 mean_value:117.0
 mean_value:123.0
 standard_deviation:1.0
}
preprocess_parameter:
{
 input_type:BINARY
```

```
input_file_path:'calibration/img_info.bin'
}
```

### 4.4.3 Description of the Configuration Parameters

**Table 4-8** Description of the device parameter

|                              |                                              |
|------------------------------|----------------------------------------------|
| <b>Function</b>              | Inference mode                               |
| <b>Type</b>                  | enum                                         |
| <b>Value Range</b>           | USE_CPU                                      |
| <b>Parameter</b>             | <b>USE_CPU</b> : Uses the CPU for inference. |
| <b>Recommended Value</b>     | USE_CPU                                      |
| <b>Mandatory or Optional</b> | Mandatory                                    |

**Table 4-9** Description of the quantize\_algo parameter

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Quantization mode                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Type</b>                  | enum                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Value Range</b>           | <ul style="list-style-type: none"> <li>• NON_OFFSET</li> <li>• HALF_OFFSET</li> </ul>                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Parameter</b>             | <ul style="list-style-type: none"> <li>• <b>NON_OFFSET</b>: Both the weight and data are offset.</li> <li>• <b>HALF_OFFSET</b>: The data is offset while the weight is not.</li> </ul> <p>There are two quantization mapping formulas:</p> <ul style="list-style-type: none"> <li>• With offset: <math>q\_uint8 = \text{round}(d\_float/scale) - \text{offset}</math></li> <li>• Without offset: <math>q\_int8 = \text{round}(d\_float/scale)</math></li> </ul> |
| <b>Recommended Value</b>     | HALF_OFFSET                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

**Table 4-10** Description of the weight\_type parameter

|                    |                                                                                        |
|--------------------|----------------------------------------------------------------------------------------|
| <b>Function</b>    | Weight quantization mode                                                               |
| <b>Type</b>        | enum                                                                                   |
| <b>Value Range</b> | <ul style="list-style-type: none"> <li>• VECTOR_TYPE</li> <li>• SCALAR_TYPE</li> </ul> |

|                              |                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameter</b>             | <p>A convolution operator may have multiple filters, and the corresponding quantization parameters may be different.</p> <ul style="list-style-type: none"> <li>• <b>VECTOR_TYPE</b>: Each filter uses a separate group of quantization parameters</li> <li>• <b>SCALAR_TYPE</b>: The filters share the same group of quantization parameters.</li> </ul> |
| <b>Recommended Value</b>     | VECTOR_TYPE                                                                                                                                                                                                                                                                                                                                               |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                                 |

**Table 4-11** Description of the preprocess\_parameter parameters

|                              |                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Pre-processing and input parameters                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Type</b>                  | Struct                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Value Range</b>           | None                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Parameter</b>             | <p>Pre-processing and input settings. The number of <b>preprocess_parameter</b> parameters must be the same as the number of operators input to the network.</p> <p><b>preprocess_parameter</b> contains the following parameters:</p> <ul style="list-style-type: none"> <li>• input_type</li> <li>• image_format</li> <li>• input_file_path</li> <li>• mean_value</li> <li>• standard_deviation</li> </ul> |
| <b>Recommended Value</b>     | None                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                                                                                    |

**Table 4-12** Description of the input\_type parameter

|                    |                                                                             |
|--------------------|-----------------------------------------------------------------------------|
| <b>Function</b>    | Input data type                                                             |
| <b>Type</b>        | enum                                                                        |
| <b>Value Range</b> | <ul style="list-style-type: none"> <li>• IMAGE</li> <li>• BINARY</li> </ul> |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameter</b>             | <p>Input type. Currently, the image (<b>IMAGE</b>) and binary file (<b>BINARY</b>) formats are supported.</p> <ul style="list-style-type: none"> <li>• <b>IMAGE</b>: image format<br/>The following image formats are supported: .bmp, .dib, .jpeg, .jpg, .jpe, .jp2, .png, .webp, .pbm, .pgm, .ppm, .sr, .ras, .tiff, .tif, .BMP, .DIB, .JPEG, .JPG, .JPE, .JP2, .PNG, .WEBP, .PBM, .PGM, .PPM, .SR, .RAS, .TIFF, .TIF</li> <li>• <b>BINARY</b>: binary file format. For details, see <a href="#">Table 4-13</a>. For non-4D (num, channels, height, and width) data, the data needs to be padded to four dimensions and the value of a padded dimension is 1.</li> </ul> |
| <b>Recommended Value</b>     | None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

**Table 4-13** Binary file formats

| File Header/<br>Data            | Address<br>Offset | Type       | Value | Description                                                                                         |
|---------------------------------|-------------------|------------|-------|-----------------------------------------------------------------------------------------------------|
| File header (20 bytes in total) | 0000              | 32-bit int | 510   | Magic number.<br>When the magic number is <b>510</b> , it is used to verify the validity of a file. |
|                                 | 0004              | 32-bit int | 50    | Input num                                                                                           |
|                                 | 0008              | 32-bit int | 3     | Input channels                                                                                      |
|                                 | 0012              | 32-bit int | 28    | Input height                                                                                        |
|                                 | 0016              | 32-bit int | 28    | Input width                                                                                         |
| Data                            | ...               | float      | 126   | The number of data records is equal to the product of (num x channels x height x width).            |

**Table 4-14** Description of the input\_file\_path parameter

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <b>Function</b> | Input data address, that is, path of the calibration set images |
| <b>Type</b>     | string                                                          |

|                              |                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Value Range</b>           | None                                                                                                                                                                                                                                                                                                                                                   |
| <b>Parameter</b>             | Folder or file to be quantified. The path cannot contain Chinese characters, special characters (including   ; & \$ > < `), or spaces. Set this parameter based on the actual network configuration. <ul style="list-style-type: none"> <li>For <b>IMAGE</b> input, specify the folder.</li> <li>For <b>BINARY</b> input, specify the file.</li> </ul> |
| <b>Recommended Value</b>     | Set this parameter to an image folder or a binary file as required.                                                                                                                                                                                                                                                                                    |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                              |

**Table 4-15** Description of the image\_format parameter

|                              |                                                                                                                                                                                                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Order of the three channels of the image input                                                                                                                                                                                                                       |
| <b>Type</b>                  | enum                                                                                                                                                                                                                                                                 |
| <b>Value Range</b>           | <ul style="list-style-type: none"> <li>BGR</li> <li>RGB</li> </ul>                                                                                                                                                                                                   |
| <b>Parameter</b>             | Order of the three channels of the image input for model training<br>This parameter is mandatory when <b>input_type</b> is set to <b>IMAGE</b> . This parameter is determined based on the order of the input channels for network training. A typical order is BGR. |
| <b>Recommended Value</b>     | BGR                                                                                                                                                                                                                                                                  |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                            |

**Table 4-16** Description of the mean\_value parameter

|                    |                                    |
|--------------------|------------------------------------|
| <b>Function</b>    | Mean value for image preprocessing |
| <b>Type</b>        | float                              |
| <b>Value Range</b> | [0, 255.0]                         |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameter</b>             | <p>Mean value of a channel for image preprocessing</p> <p>This parameter is mandatory when <b>input_type</b> is set to <b>IMAGE</b>. The count of mean values is determined by the channel count. Three mean values need to be configured for the RGB channels respectively as follows:</p> <ul style="list-style-type: none"> <li>• mean_value: 104.0</li> <li>• mean_value: 117.0</li> <li>• mean_value: 123.0</li> </ul> |
| <b>Recommended Value</b>     | None                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                                                                                                   |

**Table 4-17** Description of the standard\_deviation parameter

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Standard deviation for image preprocessing                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Type</b>                  | float                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Value Range</b>           | [0, FLOAT_MAX]                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Parameter</b>             | <p>Standard deviation for image preprocessing</p> <p>This parameter is mandatory when <b>input_type</b> is set to <b>IMAGE</b>. Channels share the same standard deviation.</p> <p>If the input range is beyond the range that can be represented by the FLOAT type, the quantization precision of the model cannot be guaranteed.</p> <p>If <math>0 \leq \text{standard\_deviation} \leq 0.00001</math>, the value of <b>standard_deviation</b> is <b>1.0</b>.</p> |
| <b>Recommended Value</b>     | <p><b>1.0</b>: The image value range (Source image value – Mean value) does not change.</p> <p><b>255.0</b>: If the image value range is [0, 0,255], the value can be condensed to [0,1, 1.0].</p>                                                                                                                                                                                                                                                                  |
| <b>Mandatory or Optional</b> | Mandatory                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

**Table 4-18** Description of the bin parameter

|                    |                                     |
|--------------------|-------------------------------------|
| <b>Function</b>    | Data range of the mapping histogram |
| <b>Type</b>        | uint32                              |
| <b>Value Range</b> | [0, 1000]                           |

|                              |                                                                                                                                                                                                                                                                                      |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameter</b>             | Data range of the histogram statistics<br>The statistics histogram is required during the divergence computation. The value of this parameter determines the maximum value of the histogram. If this parameter is not set or set to <b>0</b> , the default value <b>150</b> is used. |
| <b>Recommended Value</b>     | 100/150/200/250                                                                                                                                                                                                                                                                      |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                                                             |

**Table 4-19** Description of the type parameter

|                              |                                                                                                                                                                                                                                                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Type of the divergence computation                                                                                                                                                                                                                                                                                        |
| <b>Type</b>                  | enum                                                                                                                                                                                                                                                                                                                      |
| <b>Value Range</b>           | <ul style="list-style-type: none"> <li>• KL</li> <li>• SYMKL</li> <li>• JSD</li> </ul>                                                                                                                                                                                                                                    |
| <b>Parameter</b>             | <ul style="list-style-type: none"> <li>• <b>KL</b>: Kullback-Leibler Divergence</li> <li>• <b>SYMKL</b>: Symmetric Kullback-Leibler Divergence</li> <li>• <b>JSD</b>: Jensen-Shannon Divergence</li> </ul> <p>Different divergence types correspond to different computation methods. The default value is <b>KL</b>.</p> |
| <b>Recommended Value</b>     | None                                                                                                                                                                                                                                                                                                                      |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                                                                                                  |

**Table 4-20** Description of the inference\_with\_data\_quantized parameter

|                          |                                                                                                                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>          | Whether to use the quantized data as the inference input                                                                                                                                     |
| <b>Type</b>              | bool                                                                                                                                                                                         |
| <b>Value Range</b>       | <ul style="list-style-type: none"> <li>• true</li> <li>• false</li> </ul>                                                                                                                    |
| <b>Parameter</b>         | Whether to use the quantized data as the inference input<br>When this parameter is set to <b>true</b> , the quantization of the input data is simulated. The default value is <b>false</b> . |
| <b>Recommended Value</b> | false                                                                                                                                                                                        |



|                              |          |
|------------------------------|----------|
| <b>Mandatory or Optional</b> | Optional |
|------------------------------|----------|

**Table 4-21** Description of the inference\_with\_weight\_quantized parameter

|                              |                                                                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Whether to use the quantized weight as the inference input                                                                                                                                                          |
| <b>Type</b>                  | bool                                                                                                                                                                                                                |
| <b>Value Range</b>           | <ul style="list-style-type: none"> <li>• true</li> <li>• false</li> </ul>                                                                                                                                           |
| <b>Parameter</b>             | Whether to use the quantized weight as the inference input<br>When this parameter is set to <b>true</b> , the quantization and de-quantization of the weight data are simulated. The default value is <b>true</b> . |
| <b>Recommended Value</b>     | true                                                                                                                                                                                                                |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                            |

**Table 4-22** Description of the super\_parameter parameters

|                              |                                                                                                                                                                                                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Search parameters                                                                                                                                                                                                                             |
| <b>Type</b>                  | Struct                                                                                                                                                                                                                                        |
| <b>Value Range</b>           | None                                                                                                                                                                                                                                          |
| <b>Parameter</b>             | Search parameters<br><b>super_parameter</b> contains the following parameters: <ul style="list-style-type: none"> <li>• min_percentile</li> <li>• max_percentile</li> <li>• start_ratio</li> <li>• end_ratio</li> <li>• step_ratio</li> </ul> |
| <b>Recommended Value</b>     | None                                                                                                                                                                                                                                          |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                      |

**Table 4-23** Description of the min\_percentile parameter

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Searched minimum number                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Type</b>                  | enum                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Value Range</b>           | <ul style="list-style-type: none"> <li>• PERCENTILE_HIGH</li> <li>• PERCENTILE_MID</li> <li>• PERCENTILE_LOW</li> </ul>                                                                                                                                                                                                                                                                                                                                  |
| <b>Parameter</b>             | <p>Indicates the minimum number to be considered as the search result among a group of numbers in ascending order. For example, if there are 100 numbers, the value <b>1.0</b> indicates that number 0 (<math>100 - 100 \times 1.0</math>) is considered as the minimum, that is, the smallest number.</p> <ul style="list-style-type: none"> <li>• PERCENTILE_HIGH: 1.0</li> <li>• PERCENTILE_MID: 0.99999</li> <li>• PERCENTILE_LOW: 0.9999</li> </ul> |
| <b>Recommended Value</b>     | PERCENTILE_HIGH                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

**Table 4-24** Description of the max\_percentile parameter

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Searched maximum number                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Type</b>                  | enum                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Value Range</b>           | <ul style="list-style-type: none"> <li>• PERCENTILE_HIGH</li> <li>• PERCENTILE_MID</li> <li>• PERCENTILE_LOW</li> </ul>                                                                                                                                                                                                                                                                                                                                  |
| <b>Parameter</b>             | <p>Indicates the maximum number to be considered as the search result among a group of numbers in descending order. For example, if there are 100 numbers, the value <b>1.0</b> indicates that number 0 (<math>100 - 100 \times 1.0</math>) is considered as the maximum, that is, the largest number.</p> <ul style="list-style-type: none"> <li>• PERCENTILE_HIGH: 1.0</li> <li>• PERCENTILE_MID: 0.99999</li> <li>• PERCENTILE_LOW: 0.9999</li> </ul> |
| <b>Recommended Value</b>     | PERCENTILE_MID/PERCENTILE_HIGH                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

**Table 4-25** Description of the start\_ratio, end\_ratio, and step\_ratio parameters

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Parameter                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Type</b>                  | float                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Value Range</b>           | end_ratio > start_ratio > 0, step_ratio > 0 (If the input range is beyond the range that can be represented by the FLOAT type, the quantization accuracy of the model cannot be guaranteed.)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Parameter</b>             | <ul style="list-style-type: none"> <li>• <b>start_ratio</b> indicates the search start.</li> <li>• <b>end_ratio</b> indicates the search end.</li> <li>• <b>step_ratio</b> indicates the search step.</li> </ul> <p>After <b>d_max</b> and <b>d_min</b> are found in the algorithm, the ranges taken before and after <b>d_max</b> and <b>d_min</b> are determined based on <b>start_ratio</b> and <b>end_ratio</b>. Then, the step incremented each time is determined by <b>step_ratio</b>. For example:</p> <p>If <b>d_max</b> is <b>100</b>, <b>start_ratio</b> is <b>0.8</b>, <b>end_ratio</b> is <b>1.2</b>, and <b>step_ratio</b> is <b>0.01</b>, the defined search range boundaries before and after <b>d_max</b> are <b>80</b> (100 x 0.8) and <b>120</b> (100 x 1.2) at a step of <b>1</b> (100 x 0.01). That is, there are 41 <b>d_max</b> search results in total.</p> |
| <b>Recommended Value</b>     | <p>The following two sets of configurations are recommended:</p> <ul style="list-style-type: none"> <li>• start_ratio: 0.7 end_ratio: 1.3 step_ratio: 0.01</li> <li>• start_ratio: 0.3 end_ratio: 1.7 step_ratio: 0.01</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

**Table 4-26** Description of the batch\_count parameter

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | Number of images in the quantization calibration set to be processed                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Type</b>        | uint32                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Value Range</b> | [0, UINT32_MAX)                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Parameter</b>   | <p>Number of images in the quantization calibration set to be processed</p> <p>If this parameter is not set or set to <b>0</b>, all images in the calibration set are used. If the value is greater than <b>0</b>, the smaller one in the total number of images in the calibration set path and the value of this parameter is used as the actual number of images in the calibration set. It is recommended that the number of images in the calibration set do not exceed <b>500</b>.</p> |

|                              |          |
|------------------------------|----------|
| <b>Recommended Value</b>     | 50       |
| <b>Mandatory or Optional</b> | Optional |

**Table 4-27** Description of the `exclude_op` parameter

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>              | Quantization operator blacklist                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Type</b>                  | string                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Value Range</b>           | Operator name                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Parameter</b>             | <p>Operators in the list are not used for quantization.</p> <ul style="list-style-type: none"> <li>Only the Convolution, Full Connection, and ConvolutionDepthwise operators are supported.</li> <li>To blacklist multiple operators, multiple <b><code>exclude_op</code></b> parameters need to be set. Each parameter occupies a line. For example:<br/> <b><code>exclude_op:'aaa'</code></b><br/> <b><code>exclude_op:'bbb'</code></b></li> </ul> |
| <b>Recommended Value</b>     | None                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Mandatory or Optional</b> | Optional                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## 4.4.4 Parameter Tuning

The weight, offset, and data are quantized based on the quantization parameters set during model conversion. If the accuracy does not meet requirements, you can perform the following steps to adjust the quantization parameters.

1. Calibrate the model data and parameters based on the default parameter values described in [4.4.2 Configuration File Template](#).
2. If the quantization accuracy obtained in Step 1 is as expected, the parameter tuning ends. Otherwise, go to Step 3.
3. Manually perform the following modifications and then perform quantization.
  - a. Modify the search range parameters.

**Table 4-28** Search scope parameters

| As It Is         | To Be            |
|------------------|------------------|
| start_ratio: 0.7 | start_ratio: 0.3 |
| end_ratio: 1.3   | end_ratio: 1.7   |
| step_ratio: 0.01 | step_ratio: 0.01 |

- b. Change the value of **bin** from **150** to **200** or **250**.
- c. For a detection network, you are advised to set **max\_percentile** to **PERCENTILE\_MID** (indicating **0.99999**).
4. If the quantization accuracy obtained in Step 3 is as expected, the parameter tuning ends. Otherwise, go to Step 5.
5. Traverse the configuration options of the quantization parameters described in Table 4-29. (You can adjust the search range based on the calibration time requirement.)

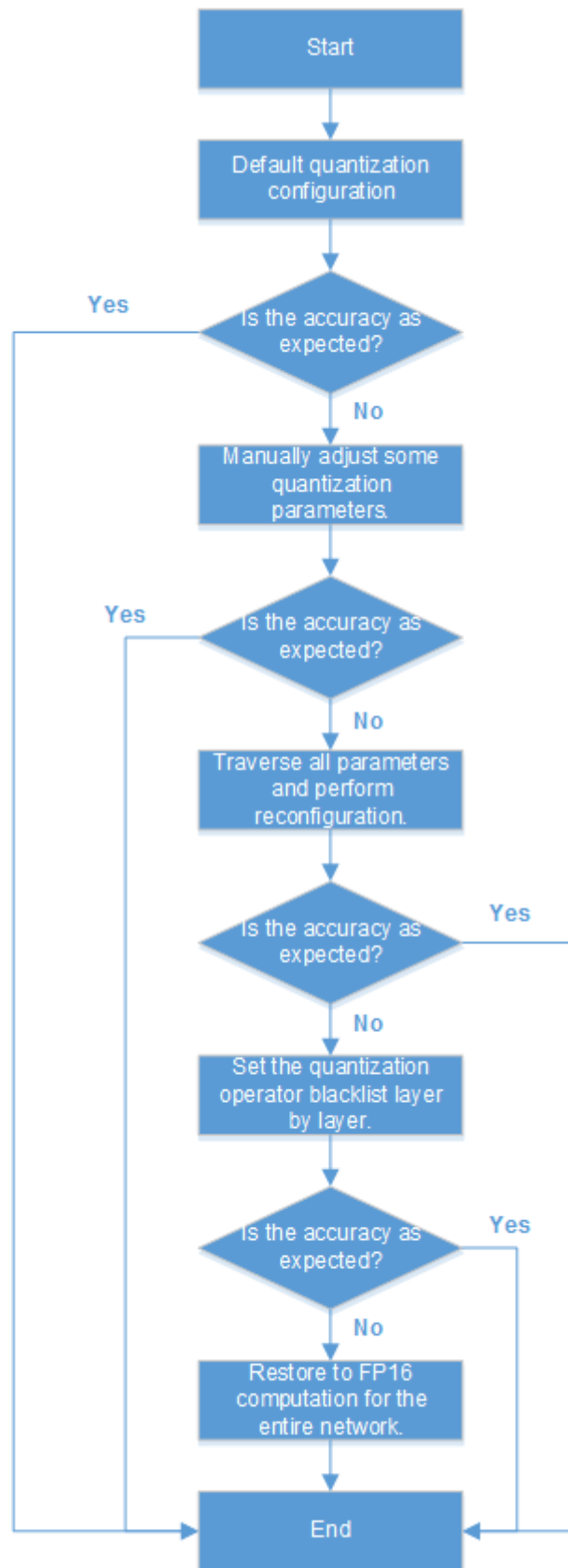
**Table 4-29** Parameter Settings

| Parameter                                                                                            | Configuration Description                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| quantize_algo                                                                                        | <b>HALF_OFFSET</b> : The data is offset while the weight is not.                                                                                                                                                                                                                                                                                                     |
| weight_type                                                                                          | <b>VECTOR_TYPE</b> : Each filter uses a separate group of quantization parameters.                                                                                                                                                                                                                                                                                   |
| bin                                                                                                  | The statistics histogram is required during the divergence computation. The value of this parameter determines the maximum value of the histogram. If this parameter is not set or set to <b>0</b> , the default value <b>150</b> is used. <b>100</b> , <b>150</b> , and <b>200</b> are recommended.                                                                 |
| type                                                                                                 | Different divergence types correspond to different computation methods. The default value is <b>KL</b> . <ul style="list-style-type: none"> <li><b>KL</b>: Kullback-Leibler Divergence</li> <li><b>SYMKL</b>: Symmetric Kullback-Leibler Divergence</li> <li><b>JSD</b>: Jensen-Shannon Divergence</li> </ul>                                                        |
| <ul style="list-style-type: none"> <li>start_ratio</li> <li>end_ratio</li> <li>step_ratio</li> </ul> | <b>start_ratio</b> indicates the search start. <b>end_ratio</b> indicates the search end. <b>step_ratio</b> indicates the search step.<br><br>The following two sets of configurations are recommended: <ul style="list-style-type: none"> <li>start_ratio: 0.7 end_ratio: 1.3 step_ratio: 0.01</li> <li>start_ratio: 0.3 end_ratio: 1.7 step_ratio: 0.01</li> </ul> |
| max_percentile                                                                                       | Maximum number to be considered as the search result among a group of numbers in descending order<br><b>1.0</b> and <b>0.99999</b> are recommended.                                                                                                                                                                                                                  |

| Parameter   | Configuration Description                                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| batch_count | Number of images in the quantization calibration set to be processed<br>The value can be an integer greater than 1. <b>10, 20, and 50</b> are recommended. |

6. If the quantization accuracy obtained in Step 5 is as expected, the parameter tuning ends. Otherwise, go to Step 7.
7. Use the **exclude\_op** parameter to set the quantization operator blacklist. Operators are blacklisted layer by layer from the first layer. Operators in the operator blacklist are not quantized, narrowing down the operators that affect the accuracy.  
Only the operators in the blacklist are not quantized. In this case, int8 computation and FP16 computation are not separated.
8. If the quantization accuracy obtained in Step 7 is as expected, the parameter tuning ends. Otherwise, it indicates that the quantization does not affect the accuracy, and the quantization configuration is not required. Remove the quantization configuration, and restore the network-wide FP16 computation.

**Figure 4-5** Parameter tuning process



## 4.5 FAQs

### 4.5.1 What Do I Do If No Log Is Output to the Screen When the DDK Installation User Is HwHiAiUser?

In the scenario where the DDK is installed by the **HwHiAiUser** user, the log is stored in the **/var/dlog** directory on the host by default. You can also set the following environment variable to enable log output to the screen:

```
export SLOG_PRINT_TO_STDOUT=1
```

If the DDK installation user is not **HwHiAiUser**, the log is output to the screen by default.

### 4.5.2 What Do I Do If Model Conversion Takes Too Long When the OS and Architecture Configuration of the DDK Server is Arm (aarch64)?

For an Arm (AArch64) DDK server, to accelerate model conversion, you can use the numactl tool to specify CPU cores for model conversion as follows:

1. Log in to the DDK server as the DDK installation user and run the **su root** command to switch to the **root** user.
2. Ensure that the host server is connected to the network, and run the following command to install the numactl tool:
3. Switch to the DDK installation user and run the **numactl -C** command to specify CPU cores 16–31 to process model conversion:

CPU cores 16–31 are recommended for better processing performance. You can also change the CPU cores based as required.

```
numactl -C 16-31 --localalloc omg <args>
```

### 4.5.3 What Do I Do If the Error "Unrecognized layer:xxx, layer type xxx" Is Reported During Model Conversion?

#### Symptom

When the omg command is run to convert a model, the error messages "Type XXX unsupported" and "Unrecognized layer:xxx, layer type XXX" are displayed, as shown in [Figure 4-6](#).



Figure 4-6 Model conversion error messages

```
[INFO] FMK:2019-07-04-06:47:01.467.332 CheckFlags:framework/domi/omg_main/main.cpp:289:"domi will run without encrypt!"
[INFO] FMK:2019-07-04-06:47:01.471.981 LoadCustomOpLib:framework/domi/omg_main/main.cpp:619:"plugin load lib_caffe_parser.so success."
[INFO] FMK:2019-07-04-06:47:01.476.887 Register:framework/domi/omg/./omg/parser/op_parser_factory.cpp:14:"register caffe parser adapter success"
[INFO] FMK:2019-07-04-06:47:01.477.319 LoadPluginLib:framework/domi/omg_main/main.cpp:655:"Load plugin libfusion_fe.so success."
[INFO] FMK:2019-07-04-06:47:01.477.429 Generate:framework/domi/omg/omg.cpp:707:"set rtContext RT_CTX_GEN MODE.."
[INFO] RUNTIME:2019-07-04-06:47:01.477.451 23143 runtime/feature/src/stream.cc:101 Setup:alloc stream id in default mode
[INFO] RUNTIME:2019-07-04-06:47:01.477.457 23143 runtime/feature/src/stream.cc:114 Setup:streamId=384, firstTaskId=65535, IsTaskSink=0
[INFO] RUNTIME:2019-07-04-06:47:01.477.485 23143 runtime/feature/src/stream.cc:101 Setup:alloc stream id in default mode
[INFO] RUNTIME:2019-07-04-06:47:01.477.491 23143 runtime/feature/src/stream.cc:114 Setup:streamId=640, firstTaskId=65535, IsTaskSink=0
[INFO] CCE:2019-07-04-06:47:01.477.512 cce/common/cce_sys.cc:34 cceSysInit cceSysInit begin!
[INFO] CCE:2019-07-04-06:47:01.478.065 cce/common/cce_sys.cc:43 cceSysInit cceSysInit success!
[INFO] CCE:2019-07-04-06:47:01.478.074 cce/optimizer/optimizer_sys.cc:29 optimizerSysInit optimizerSysInit begin!
[INFO] CCE:2019-07-04-06:47:01.478.202 cce/optimizer/optimizer_sys.cc:33 optimizerSysInit optimizerSysInit success!
[INFO] CCE:2019-07-04-06:47:01.478.209 cce/common/cce_sys.cc:46 cceSysInit optimizerSysInit success!
[INFO] FMK:2019-07-04-06:47:01.478.224 Parse:framework/domi/omg/parser/caffe/caffe_parser.cpp:733:"Caffe Parse model file deploy_mylenet-1.prototxt"
[INFO] FMK:2019-07-04-06:47:01.479.405 CheckTypeSupported:framework/domi/omg/./omg/pre_checker/pre_checker.cpp:312:"Type Reduction unsupported"
[INFO] FMK:2019-07-04-06:47:01.479.514 ConvertDim:framework/domi/common/op/ge_op_utils.cpp:2055:"Convert format , src size 4 < 3 ,not need convert"
[INFO] RUNTIME:2019-07-04-06:47:01.479.595 23143 runtime/feature/src/logger.cc:262 HostMalloc:host memory malloc, size = 92
[INFO] RUNTIME:2019-07-04-06:47:01.479.619 23143 runtime/feature/src/logger.cc:270 HostMalloc:host memory alloc ok
[ERROR] FMK:2019-07-04-06:47:01.480.280 AddNode:framework/domi/omg/parser/caffe/caffe_parser.cpp:233:"Unrecognized layer:reduction, layer type Reduction"
[ERROR] FMK:2019-07-04-06:47:01.480.299 Parse:framework/domi/omg/parser/caffe/caffe_parser.cpp:812:"Caffe parser add node fail."
[INFO] FMK:2019-07-04-06:47:01.480.590 SaveJsonToFile:framework/domi/omg/./omg/model/model_saver.cpp:40:"file check_result.json does not exist, it will be created."
[ERROR] FMK:2019-07-04-06:47:01.480.680 SaveJsonToFile:framework/domi/omg/./omg/model/model_saver.cpp:47:"open file failed, file path : check_result.json"
[ERROR] FMK:2019-07-04-06:47:01.480.686 Save:framework/domi/omg/./omg/pre_checker/pre_checker.cpp:293:"Save failed."
[ERROR] FMK:2019-07-04-06:47:01.480.619 Generate:framework/domi/omg/omg.cpp:724:"Generate pre-checking report failed."
[ERROR] FMK:2019-07-04-06:47:01.480.757 main:framework/domi/omg_main/main.cpp:815:"OMG Generate execute failed!"
OMG generate offline model failed, please see the log or pre-checking report for more details.
```

## NOTE

As shown in Figure 4-6, the Reduction operator is used as an example for illustration.

## Solution

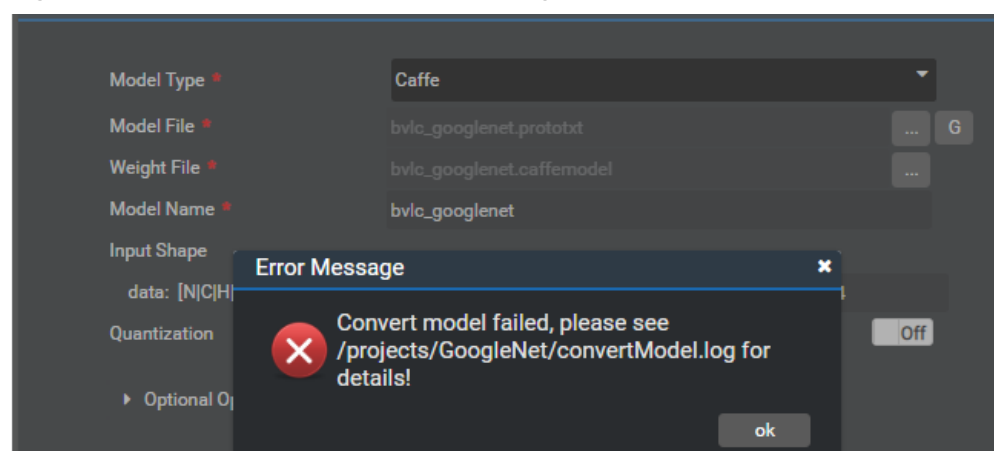
It is because the source model contains operators that cannot be identified by Ascend 310. You need to develop these operators, develop operator plug-ins, and load operator plug-ins to convert the model, according to *TE Custom Operator Development Guide*.

## 4.5.4 What Do I Do If the Error "It is recommended to convert layers-structure to layer-structure by caffe tool" Is Reported During Model Conversion?

## Symptom

The .prototxt and .caffemodel files of GoogLeNet downloaded from [https://github.com/BVLC/caffe/tree/master/models/bvlc\\_googlenet](https://github.com/BVLC/caffe/tree/master/models/bvlc_googlenet) fail to be imported, as shown in Figure 4-7.

Figure 4-7 Model conversion error message



The **convertModel.log** file shows the following error information.

```
[ERROR] FMK:2018-12-22-23:47:48.147.318 ConvertNetParameter:framework/domi/omg/parser/caffe/caffe_parser.cpp:776:"The weight file is consisted of layers-structure which is deprecated in caffe and unsupport in OMG. It is recommended to convert layers-structure to layer-structure by caffe tool. Error Code:0xFFFFFFFF(failed)"
```

## Solution

It is because the .prototxt and .caffemodel files have to be upgraded to the latest by using tools provided by Caffe. You can obtain the Caffe tools from [Link](#).

- Step 1** Download the upgrade\_net\_proto\_text and upgrade\_net\_proto\_binary tools to any directory on the Mind Studio server.
- Step 2** Compile and obtain tools upgrade\_net\_proto\_text and upgrade\_net\_proto\_binary by referring to [Link](#).
- Step 3** Use the upgrade\_net\_proto\_text tool to upgrade the .prototxt file.  
**upgrade\_net\_proto\_text** *model\_old.prototxt model\_new.prototxt*
- Step 4** Use the upgrade\_net\_proto\_binary tool to upgrade the .caffemodel file.  
**upgrade\_net\_proto\_binary** *model\_old.caffemodel model\_new.caffemodel*
- Step 5** Use the upgraded .prototxt file and .caffemodel file to import the model again.  
----End

# 5 Operator Development

---

- [5.1 Before You Start](#)
- [5.2 Precautions for Operator Development](#)
- [5.3 Environment Preparation](#)
- [5.4 Overall Development Process](#)
- [5.5 Setting Environment Variables](#)
- [5.6 Creating a Custom Operator Development Project](#)
- [5.7 Developing a Custom Operator](#)
- [5.8 caffe.proto File Operator Definition \(Optional\)](#)
- [5.9 Developing the Plug-In of a Custom Operator](#)
- [5.10 Loading the Plug-In for Model Conversion](#)
- [5.11 Function Reference for Operator Plug-In Development](#)
- [5.12 Operator List](#)

## 5.1 Before You Start

### 5.1.1 Overview

Ascend 310 provides a deep-learning framework that converts models under open-source frameworks such as Caffe and TensorFlow into models supported by Ascend 310. During model conversion, if operators in the models are not implemented in the built-in operator library, an error is reported during the conversion. In this case, the operators that are not implemented need to be customized. The custom operators can be added to the custom operator library to allow successful model conversion. You can also optimize existing operators in the current model, add the optimized operators to the custom operator library, and load the optimized operators for model conversion.

Ascend 310 provides the Tensor Engine (TE) operator development framework for developing custom operators. TE is a custom operator development framework

based on the Tensor Virtual Machine (TVM). It provides the DSL language based on the Python syntax for developing custom operators. Custom TE operators can run on the AI CPU and AI Core.

This document describes the development process of a custom Tensor Engine (TE) operator in command line interface (CLI) mode by using the custom `caffe_reduction_layer` operator (which is an extended built-in reduction operator in the Caffe network model LeNet-5) as an example. The process includes how to develop a custom TE operator, verify the operator, register the operator, and load the custom operator for model conversion.

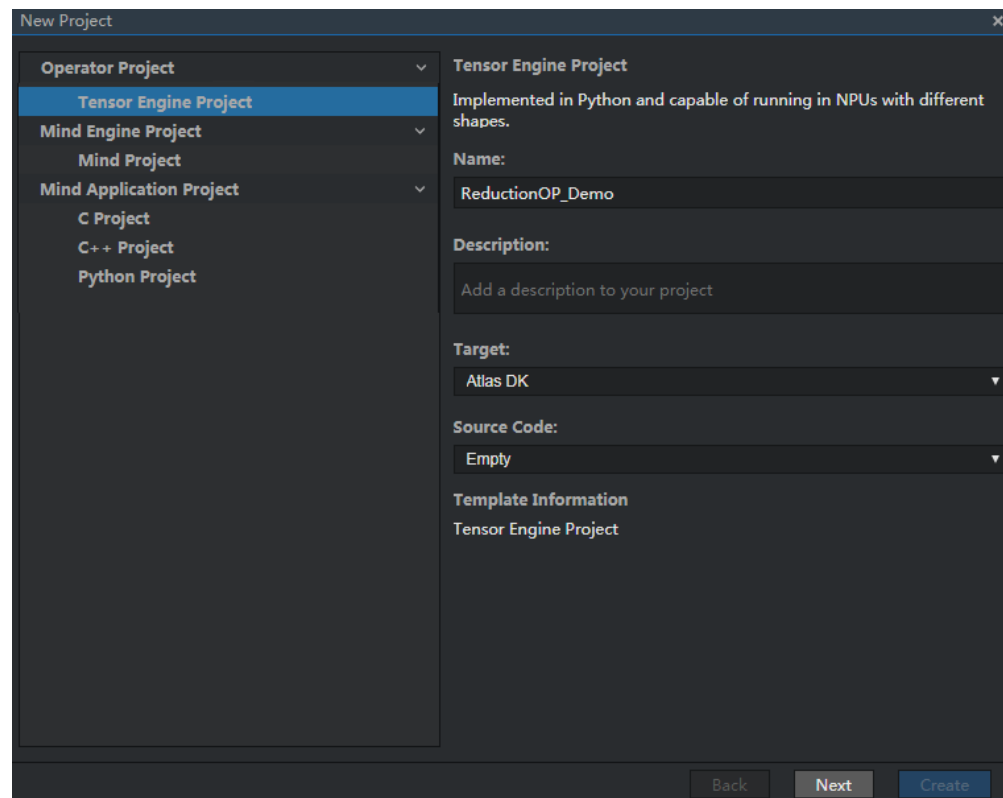
**Table 5-1** describes the custom TE operator development modes supported by Ascend 310. This document describes only the development in command line interface (CLI) mode.

**Table 5-1** Custom TE operator development modes supported by Ascend 310

| Development Mode | Tool Dependency                                                                                                                                                                                                                                                                                                                                        | Description                                                                                                                                                                                                     | Compilation Mode                                                          | Reference Document                                 |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------|
| CLI              | <p>This mode does not depend on Mind Studio. It only requires the device development kit (DDK) installation.</p> <p><b>NOTE</b><br/>The DDK provides the NPU-based digital development kit that contains project code samples and integrates related dependent libraries and header files. You can compile project files by using <b>Makefile</b>.</p> | <p>There is no UI or automatically generated framework code. The implementation code of all operators and operator plug-in code need to be developed by users or modified based on code samples in the DDK.</p> | <p>The <b>Makefile</b> file is compiled based on the DDK code sample.</p> | <p><i>TE Custom Operator Development Guide</i></p> |

| Development Mode | Tool Dependency                                                                   | Description                                                                                                                                                                                        | Compilation Mode                                                                                                          | Reference Document                                                   |
|------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Mind Studio      | This mode depends on Mind Studio. The following figure shows the operation entry. | The framework code is automatically generated based on the model file imported during project creation and selected operators. You only need to pay attention to the operator implementation code. | The <b>Makefile</b> file does not need to be compiled because it is generated based on the configurations on Mind Studio. | <i>Ascend 310 TE Custom Operator Development Guide (Mind Studio)</i> |

**Figure 5-1** Tensor engine project creation



## 5.1.2 Intended Audience

This document is intended for developers who program operators using Ascend 310. It aims to help you:

- Understand the end-to-end (E2E) development process of a custom TE operator, including developing a custom operator and integrating it into the network to run on E2E applications.
- Learn the operator development and operator plug-in development principles by referring to the reduction operator sample in this document, and be able to develop other custom operators.

To better understand this document, you should have:

- Capability of developing Python/C++/C language programs
- Good understanding of mathematical expressions
- Good understanding of machine learning and deep learning
- Good understanding of the Ascend 310 architecture
- Good understanding of TVM and open source framework Caffe

## 5.2 Precautions for Operator Development

### Precautions for Developing Custom Operators for an SSD Network with the Post-Processing Node

The PriorBox operator and detection output operator of the SSD network are automatically integrated during model conversion. If the implementation of the PriorBox operator of the SSD network has been customized, model conversion will fail due to the integration failure. In this situation, you need to delete the detection output operator from the network model file, and customize a post-processing node in engine orchestration to implement the function of the detection output operator.

#### Others

- Do not modify the implementation of the built-in operators of the framework (all files are saved in **ddk/include/inc/custom**). Otherwise, exceptions may occur. For example, the system fails to be started or the model cannot be converted.
- When developing an operator plug-in, customers shall be responsible for the source code to avoid backdoor implantation.

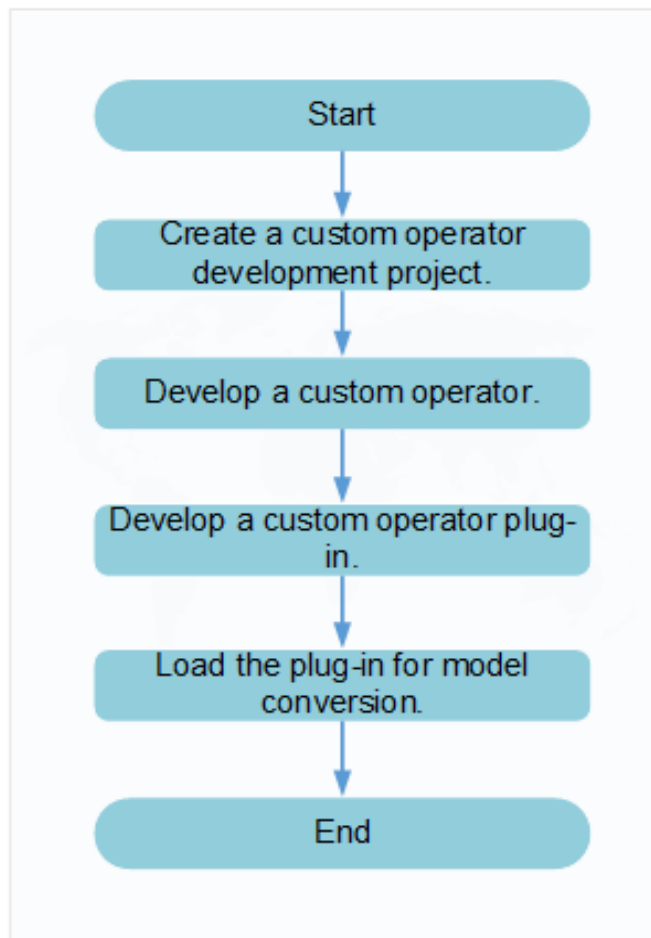
## 5.3 Environment Preparation

The DDK has been deployed. It provides developers with an Ascend 310-based digital development kit that contains project code samples and integrates related dependent libraries and header files. You can compile project files by using **Makefile**.

## 5.4 Overall Development Process

**Figure 5-2** shows the development process of a custom TE operator.

**Figure 5-2** Developing a custom TE operator



The operating procedure is as follows:

1. Create a custom operator development project.
2. Customize unimplemented operators, including developing operator code, and compiling, running, and verifying a single operator.
3. Develop the custom operator plug-in and register the custom operator with the Framework. After the operator plug-in is compiled, the \*.so plug-in file of the custom operator is generated.
4. Load the \*.so plug-in file to identify the custom operator and convert the model.

## 5.5 Setting Environment Variables

**Step 1** Log in to the DDK server as the DDK installation user *ascend*.

**Step 2** Set the environment variables.

- In the Atlas 200 DK environment, run the following commands in the CLI to set environment variables:  

```
export DDK_HOME=$HOME/tools/che/ddk/ddk
export LD_LIBRARY_PATH=$DDK_HOME/uihost/lib/
export PYTHONPATH=$DDK_HOME/site-packages/te-0.4.0.egg:$DDK_HOME/site-
```



```
packages/topi-0.4.0.egg
export PATH=$PATH:$DDK_HOME/toolchains/cccec-linux/bin
export TVM_AICPU_LIBRARY_PATH=$DDK_HOME/uihost/lib/:$DDK_HOME/uihost/
toolchains/cccec-linux/aicpu_lib
export TVM_AICPU_INCLUDE_PATH=$DDK_HOME/include/inc/tensor_engine
export TVM_AICPU_OS_SYSROOT=/usr/lib/aarch64-linux-gnu
```

- In the ASIC environment, run the following commands in the CLI to set environment variables:

```
export DDK_HOME=$HOME/tools/che/ddk/ddk
export LD_LIBRARY_PATH=$DDK_HOME/uihost/lib/
export PYTHONPATH=$DDK_HOME/site-packages/te-0.4.0.egg:$DDK_HOME/site-
packages/topi-0.4.0.egg
export PATH=$PATH:$DDK_HOME/toolchains/cccec-linux/bin
export TVM_AICPU_LIBRARY_PATH=$DDK_HOME/uihost/lib/:$DDK_HOME/uihost/
toolchains/cccec-linux/aicpu_lib
export TVM_AICPU_INCLUDE_PATH=$DDK_HOME/include/inc/tensor_engine
export TVM_AICPU_OS_SYSROOT=$DDK_HOME/uihost/toolchains/aarch64-linux-
gcc6.3/sysroot
```

#### NOTE

Change the path of **DDK\_HOME** to the actual installation path of the DDK.  
You can write the commands for setting environment variables to the operator customization script for future use.

----End

## 5.6 Creating a Custom Operator Development Project

**Step 1** Log in to the DDK server as the DDK installation user *ascend*.

**Step 2** Create a workspace directory.

```
mkdir $HOME/tools/projects
```

**Step 3** Create a directory for custom projects.

```
$ mkdir $HOME/tools/projects/customop_te
```

Create an operator code directory.

```
mkdir $HOME/tools/projects/customop_te/operator
```

Create a directory for storing operator plug-in code.

```
mkdir $HOME/tools/projects/customop_te/plugin
```

**Step 4** Copy the sample code in the DDK to the directory where the operator development project is stored.

#### NOTE

- This example describes how to develop a custom operator by using the code of the built-in reduction operator in the DDK and its plug-in. You can create related files for editing.
- In the example, the installation path of the DDK is **\$HOME/tools/che**. If the actual installation path is different from the example path, replace the path in the example command with the actual installation path.
- Copy the sample code file **reduction.py** of the reduction operator and the sample data generation script **data\_gen.py** used for operator verification in

the DDK to the directory where the operator code of the custom project is stored.

```
cp -rf $HOME/tools/che/ddk/ddk/sample/customop/python/reduction.py
$HOME/tools/projects/customop_te/operator/
```

```
cp -rf $HOME/tools/che/ddk/ddk/sample/customop/python/data_gen.py
$HOME/tools/projects/customop_te/operator/
```

- Copy the sample code file of the reduction operator plug-in and the **Makefile** file in the DDK to the directory where the operator plug-in code of the custom project is stored.

```
cp -rf $HOME/tools/che/ddk/ddk/sample/customop/
customop_caffe_demo/caffe_reduction_layer.cpp $HOME/tools/projects/
customop_te/plugin/
```

```
cp -rf $HOME/tools/che/ddk/ddk/sample/customop/
customop_caffe_demo/Makefile $HOME/tools/projects/customop_te/
plugin/
```

- Copy the Caffe network model file in the DDK used in this example to the custom project directory.

```
cp -rf $HOME/tools/che/ddk/ddk/sample/customop/
customop_caffe_demo/model/ $HOME/tools/projects/customop_te/
```

**Table 5-2** shows the directory structure of the custom operator development project.

**Table 5-2** Directory structure

| Sample Directory         | File                      | Description                                                                    |
|--------------------------|---------------------------|--------------------------------------------------------------------------------|
| customop_te/<br>operator | reduction.py              | Sample code of the reduction operator                                          |
|                          | data_gen.py               | File generated based on reduction operator sample data                         |
| customop_te/<br>plugin   | caffe_reduction_layer.cpp | Sample code of the reduction operator plug-in (needs to be modified)           |
|                          | Makefile                  | Compilation rule file of the reduction operator plug-in (needs to be modified) |
| customop_te/<br>model    | deploy_mylenet-1.prototxt | Original model file of the MyLeNet network                                     |
|                          | mylenet-1.caffemodel      | Pre-trained model file of the MyLeNet network                                  |

----End

## 5.7 Developing a Custom Operator

### 5.7.1 Operator Basics

A deep learning algorithm consists of multiple compute units, that is, operators (Ops). In Caffe, an operator describes the computation logic of the layer, for example, the convolution that performs convolution and the Fully-Connected (FC) layer that multiplies the input by a weight matrix.

The following introduces some basic terms about operators.

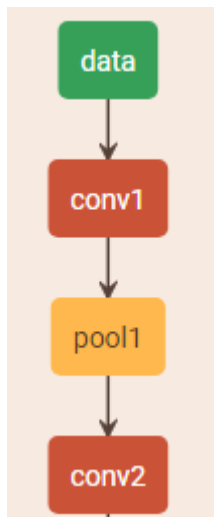
#### Operator Type

Type of an operator. For example, the type of a convolution operator is convolution. A network can have different operators of the same type.

#### Operator Name

The name of an operator identifies the operator on a network. An operator name must be unique on a network. As shown in the following figure, conv1, pool1, and conv2 are the names of operators on the network. They are of the same type, convolution. Each indicates a convolution operation.

**Figure 5-3** Network topology



#### Tensor

As data in a TE operator, the tensor includes the input data and output data. **TensorDesc** (the tensor descriptor) describes the input data and output data. [Table 5-3](#) describes the attributes of the **TensorDesc** struct.

**Table 5-3** Description of the TensorDesc attributes

| Attribute | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| name      | Indexes a tensor. The <b>name</b> of each tensor must be unique.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| shape     | Specifies the shape of a tensor, for example, <b>(10)</b> , <b>(1024,1024)</b> , or <b>(2,3,4)</b> . For details, see <a href="#">Shape</a> .<br>Default value: N/A<br>Format: <b>(i1, i2, ... in)</b> , where, <b>i1</b> to <b>in</b> are positive integers.                                                                                                                                                                                                                                                                                                 |
| dtype     | Specifies the data type of a tensor object.<br>Default value: N/A<br>Value range: <b>float16, int8, int16, int32, uint8, uint16, bool</b><br><b>NOTE</b> <ul style="list-style-type: none"> <li>The supported data types vary with the operation. For details, see the <i>TE API Reference</i>.</li> <li>TE APIs support both the float16 and float32 types. However, OMG converts the float32 type to the float16 type during model conversion. Therefore, the current version does not support the float32 type for custom operator development.</li> </ul> |
| format    | Specifies the data layout format. For details, see <a href="#">Format</a> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

- Shape**  
The shape of a tensor is described in the format of **(D0, D1, ..., Dn - 1)**, where, **D0** to **Dn** are positive integers.  
For example, the shape (3, 4) indicates a 3 x 4 matrix, where the first dimension has three elements, the second dimension has four elements.  
The number count in the bracket equals to the dimension count of the tensor. The first element of shape depends on the element count in the outer bracket of the tensor, and the second element of shape depends on the element count in the second bracket of the tensor starting from the left, and so on.

**Table 5-4** Tensor shape examples

| Tensor                         | Shape   |
|--------------------------------|---------|
| 1                              | (0,)    |
| [1,2,3]                        | (3,)    |
| [[1,2],[3,4]]                  | (2,2)   |
| [[[1,2],[3,4]], [[5,6],[7,8]]] | (2,2,2) |

- Format**  
In the deep learning framework, n-dimensional data is stored by using an n-dimensional array. For example, a feature graph of a convolutional neural

network is stored by using a four-dimensional array. The four dimensions are N, H, W and C, which stand for batch, height, width, and channels, respectively.

Data can be stored only in linear mode because the dimensions have a fixed order. Different deep learning frameworks store feature graph data in different sequences. For example, data in Caffe is stored in the order of [Batch, Channels, Height, Width], that is, NCHW. Data in TensorFlow is stored in the order of [Batch, Height, Width, Channels], that is, NHWC.

As shown in [Figure 5-4](#), an RGB picture is used as an example. In the NCHW order, the pixel values of each channel are clustered in sequence as RRRGGGBBB. In the NHWC order, the pixel values are interleaved as RGBRGBRG.

**Figure 5-4** NCHW and NHWC



To improve data access efficiency, the tensor data in the Ascend AI software stack is stored in the 5D format NC1HWC0. C0 is closely related to the micro architecture and is equal to the size of the matrix computing unit in AI Core. For the FP16 type, C0 is **16**; for the INT8 type, C0 is **32**. The C0 data needs to be stored consecutively. That is,  $C1 = (C + C0 - 1) / C0$ . If the division result is not an integer, the last data record is padded with zeros to be aligned with C0.

The NHWC-to-NC1HWC0 conversion process is as follows:

- Split the NHWC data along dimension C into C1 pieces of NHWC0.
- Arrange the C1 pieces of NHWC0 in the memory consecutively into NC1HWC0.

## Operator Attributes

Different operators have different attribute values. The following describes some common operator attributes.

- **Axis**

The axis represents the subscript of a dimension of a tensor. For a two-dimensional tensor with five rows and six columns, that is, with shape (5, 6), axis 0 represents the first dimension in the tensor, that is, the row; axis 1 represents the second dimension of tensor, that is, the column.

For example, for tensor  $[[[1,2],[3,4]], [[5,6],[7,8]]]$  with shape (2, 2, 2), axis 0 represents data in the first dimension, that is, matrices  $[[1,2],[3,4]]$  and  $[[5,6],[7,8]]$ , axis 1 represents data in the second dimension, that is, arrays  $[1,2]$ ,  $[3,4]$ ,  $[5,6]$ , and  $[7,8]$ , and axis 2 indicates the data in the third dimension, that is, numbers 1, 2, 3, 4, 5, 6, 7, and 8.

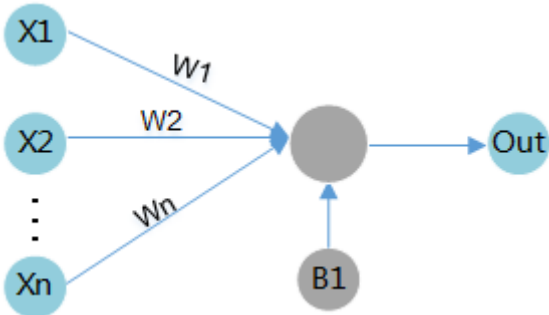
Negative **axis** is interpreted as a dimension counted from the end.

- **Bias**

The bias, along with the weight, is a linear component to be applied to the input data. It is applied to the result of multiplying the weight by the input data.

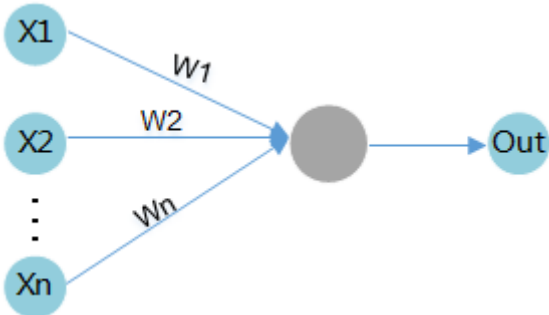
As shown in [Figure 5-5](#), assume that the input data is **X1**, the associated weight is **W1**, and the bias is **B1**. After the data passes the compute unit, the data changes to **(X1 \* W1 + B1)**.

**Figure 5-5** Bias computation example



- **Weight**  
The input data is multiplied by a weight value in the compute unit. For example, if an operator has two inputs, an associated weight value is allocated to each input. Generally, it is considered that relatively important data is assigned a relatively greater weight value, and a weight value of zero indicates a specific feature can be ignored.  
As shown in [Figure 5-6](#), assume that the input data is **X1** and the associated weight is **W1**. After the data passes the compute unit, the data changes to **(X1 \* W1)**.

**Figure 5-6** Weight computation example



**Sample Operator: Reduction**

- Reduction is a Caffe operator that reduces the specified axis and its subsequent axes of a multi-dimensional array.
- **Attributes of the reduction operator**
    - **ReductionOp**: operation type. Four operation types are supported.

**Table 5-5** Operation types supported by the reduction operator

| Operator Type | Description                          |
|---------------|--------------------------------------|
| SUM           | Sums the values of all reduced axes. |

| Operator Type | Description                                                |
|---------------|------------------------------------------------------------|
| ASUM          | Sums the absolute values of all reduced axes.              |
| SUMSQ         | Squares the values of all reduced axes and then sums them. |
| MEAN          | Averages the values of all reduced axes.                   |

- **axis**: first axis to reduce. The value range is  $[-N, N - 1]$ .  
For example, for an input tensor with shape (5, 6, 7, 8):
  - If axis = 3, the shape of the output tensor is (5, 6, 7).
  - If axis = 2, the shape of the output tensor is (5, 6).
  - If axis = 1, the shape of the output tensor is **(5)**.
  - If axis = 0, the shape of the output tensor is **(1)**.
- **coeff**: scalar, scaling coefficient for output. The value **1** indicates that the output is not scaled.
- Data of the reduction operator
  - Input data  
The input contains the tensor data and tensor description.  
Input data: tensor **x**  
The description of tensor **x** contains the following attributes:

**Table 5-6** Input of the reduction operator

| Input Parameter | Description                                                                                 |
|-----------------|---------------------------------------------------------------------------------------------|
| x               | Name of the input tensor, whose shape is determined by the <b>shape</b> parameter           |
| shape           | Shape of the input data, N-dimensional                                                      |
| dtype           | Type of the input data, either<br>Indicates the data type. The value is float16 or float32. |

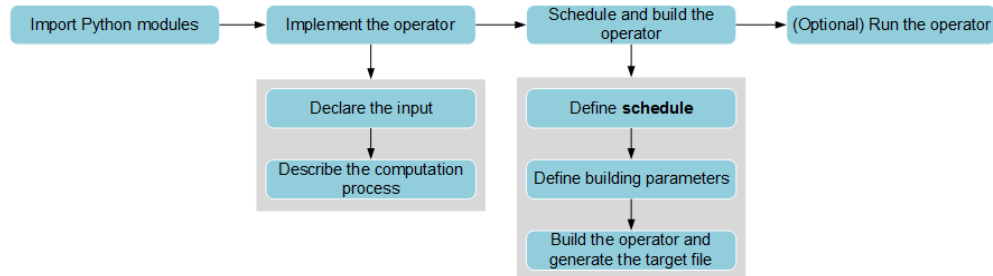
- Output data  
**y**: tensor of the identical data type as input **x**, whose shape is determined by the input tensor shape and the specified **axis**

## 5.7.2 Implementing an Operator

### 5.7.2.1 Procedure

The code of a TE operator is developed in Python. [Figure 5-7](#) shows the implementation procedure.

**Figure 5-7** Implementation process of a TE custom operator



#### NOTE

- The supported input data types for custom operators are as follows: float16, int8, int16, int32, uint8, uint16, and bool.
- The supported data types vary with the operation. For details, see the *TE API Reference*.
- TE APIs support both the float16 and float32 types. However, OMG converts the float32 type to the float16 type during model conversion. Therefore, the current version does not support the float32 type for custom operator development.
- TE provides sample code of some custom operators for user reference or direct use in `ddk/site-packages/topi-0.4.0.egg/topi/cce` in the DDK installation directory.

### 5.7.2.2 Importing Python Modules

Import the Python modules provided by the Ascend AI software stack. The sample code is provided as follows.

```
import te.lang.cce
from te import tvn
from topi import generic
```

In the preceding information:

- **te.lang.cce**: introduces the SDL APIs supported by TE, including common ones such as **vmuls**, **vadds**, and **matmul**.  
For details about the interface definition, see the Python functions in the `/site-packages/te-0.4.0.egg/te/lang/cce/` directory in the DDK installation path. For details about how to use the Python functions, see the *TE API Reference*.
- **te.tvn**: introduces the code generation mechanism of the TVM.  
For details about the interface definition, see the Python functions in the `/site-packages/te-0.4.0.egg/te/tvn` directory in the DDK installation path. For details about how to use the Python functions, visit <https://docs.tvn.ai/>.
- **topi.generic**: provides the API for automatic operator scheduling.  
For details about the interface definition, see the Python functions in the `/site-packages/topi-0.4.0.egg/topi/generic` directory in the DDK installation path. For details about how to use the Python functions, see the *TE API Reference*.



### 5.7.2.3 Implementing an Operator

#### Function Definition for Operator Implementation

As described below, the implementation function of an operator contains the input tensor shape, data type, operator attributes, kernel name, and build and print configurations. This function is called by plug-in code and is executed when OMG converts the model.

```
def operationname(shape, dtype, attribute1, attribute2, ..., kernel_name="KernelName", need_build=True, need_print=False)
```

In the preceding information:

- **shape**: input tensor shape. If an operator has multiple input tensors and each tensor has a unique shape, multiple shapes need to be defined as placeholders for the tensors. If multiple input tensors have a same shape, define one shape.
- **dtype**: data type of the input tensor.
- **attribute1, attribute2, ...**: operator attributes. Edit the code based on the operator definition.
- **kernel\_name**: name of the operator in the kernel, that is, the name of the generated binary file. The value is user-defined and unique. The value can contain only uppercase letters, lowercase letters, digits, and underscores (\_). Enter a maximum of 200 characters starting with a letter or underscore (\_).
- **need\_build**: build enable, either **True** or **False**
- **need\_print**: intermediate representation (IR) print enable, either **True** or **False**

Examples:

Reduction operator:

```
def reduction(shape, dtype, axis, operation, coeff, kernel_name="Reduction", need_build=True, need_print=False)
```

Matmul operator:

```
def matmul(shape_a, shape_b, dtype, kernel_name="matmul", trans_a=False, trans_b=False, need_build=False, need_print=False):
```

#### Operator Implementation Logic

The TE operator implementation logic is summarized as follows:

Define placeholders for input tensors, and then call the SDL interfaces in **te.lang.cce** to describe the computation process. The following is a code example:

```
data = tvm.placeholder(shape, name="data_input", dtype=inp_dtype)
with tvm.target.cce():
 cof = coeff
 data_tmp_input = te.lang.cce.vmults(data, cof) // Process the scaling parameter and multiply the input tensor by a scalar.
 tmp = data_tmp_input
 res_tmp = te.lang.cce.sum(tmp, axis=axis) // Perform the summation operation on the axis.
 res = te.lang.cce.cast_to(res_tmp, inp_dtype, f1628IntegerFlag=True) // Convert the data type.
```

In the preceding information:

- **data** indicates the input tensor, which is defined by using the **placeholder** interface of the TVM. A Tensor object is returned, indicating a group of input data.

If the operator has multiple input tensors, multiple tensor objects need to be defined. For example:

```
tensor_a = tvm.placeholder(shape_a, name='tensor_a', dtype=dtype)
tensor_b = tvm.placeholder(shape_b, name='tensor_b', dtype=dtype)
```

- **vmuls** (vector multiplication) and **sum** (summation) constitute the intermediate computation logic.
- **cast\_to** is used to convert the data type. The output tensor must be of the identical data type as the input tensor. If the data type is changed during computation, you need to use the **cast\_to** interface to convert the data type of the output tensor to that of the input tensor.

For example: If the data type of the input tensor is int8, it is converted to float16 for the vmuls operation. In this case, the **cast\_to** interface must be called to convert the data type of the output tensor from float16 to int8 after the computation logic is complete. **vmuls** converts int8 values to float16, padding the decimal part with zeros. Therefore, **f1628IntegerFlag** is set to **True**. The sample code is as follows:

```
res = te.lang.cce.cast_to(res_tmp, inp_dtype, f1628IntegerFlag=True)
```

For details about how to use the **te.lang.cce.cast\_to** API, see **Compute APIs** in *TE API Reference*.

- **res** indicates the output tensor, of the identical data type as the input tensor.

Before implementing the operator logic, you can customize the code for pre-processing the input data. The sample code is as follows:

```
basic check
check_list = ["float16", "float32"]
if not (dtype.lower() in check_list):
 raise RuntimeError("Reduction only support %s while dtype is %s" % (
 ",".join(check_list), dtype))
reduction_op = ("SUM", "ASUM", "SUMSQ", "MEAN")

axis parameter check
if type(axis) != int:
 raise RuntimeError("type of axis value should be int")
if axis >= len(shape) or axis < -len(shape):
 raise RuntimeError(
 "input axis is out of range, axis value can be from %d to %d" % (
 -len(shape), len(shape) - 1))
operation parameter check
if operation not in reduction_op:
 raise RuntimeError("operation can only be one of SUM, ASUM, SUMSQ, MEAN")
coeff parameter check
if type(coeff) != int and type(coeff) != float:
 raise RuntimeError("coeff must be a value")
Preprocess
if axis < 0:
 axis = len(shape) + axis
shape = list(shape)
shape1 = shape[:axis] + [reduce(lambda x, y: x * y, shape[axis:])]
inp_dtype = dtype.lower()
```

#### 5.7.2.4 Scheduling and Building an Operator

As shown in the following code, after the computation logic is defined, the Auto schedule mechanism performs auto scheduling. You can check the computation IR

through the print mechanism provided by the TVM. The configuration information includes the print switch status, build switch status, operator name in the kernel, and input and output tensors.

```
sch = generic.auto_schedule(res)
config = {
 "print_ir": need_print,
 "need_build": need_build,
 "name": kernel_name,
 "tensor_list": [data, res]
}
te.lang.cce.cce_build_code(sch, config)
```

- Use the **auto\_schedule** interface of **generic** to perform auto scheduling (defining **schedule**). The argument of the **auto\_schedule** interface is the output tensor of the operator.  
The schedule object defines how to efficiently execute the described computation process on hardware. That is, the related computations are mapped to the corresponding instructions on a hardware device. A schedule object contains an IR, which uses code similar to pseudo code to describe a computation process. You can print the object by using the parameter **need\_print**.
- The input and output tensors are stored in **tensor\_list**. The input and output tensors must be arranged in the input and output sequences of the operator.  
For example: "**tensor\_list**": [**tensor\_a**, **tensor\_b**, **res**], where, **tensor\_a** and **tensor\_b** are the input tensors, and **res** is the output tensor.
- The **cce\_build\_code** interface provided by **te.lang.cce** is used to build the operator based on scheduling and configuration. During operator building when OMG converts the model, a dedicated kernel is built based on the input data shape, type, and operator parameters.
  - **sch**: schedule object to be calculated by the generated operator.
  - **config**: map configured by compilation parameters.

After building, an operator target file \*.o (the running target of the operator is AI Core) or \*.so (the running target of the operator is AI CPU) and an operator description file \*.json are generated.

### 5.7.2.5 Running an Operator

After the custom operator code is written, you can append the operator calling statement to the \*.py code of the operator as follows. Construct the input data by referring to [5.7.4 Operator Running Verification](#) and use it to check the operator execution result.

For example:

```
if __name__ == "__main__":
 reduction((2, 3, 4), "float16", 1, "SUM", coeff = 2, kernel_name = "Reduction")
```

## 5.7.3 Code Examples

Open the copied operator sample file **reduction.py**.

For details about the code description, see the following comments:

```
#coding=utf-8
import te.lang.cce
```

```

from te import tvm
from topi import generic
from topi.cce import util
def reduction(shape, dtype, axis, operation, coeff, kernel_name="Reduction",
 need_build=True, need_print=False):
 """
 Reduce a tensor on a certain axis, and scale output with coeff
 Parameters

 shape : shape of data
 dtype : source data type, only support float16, float32
 axis : the first axis to reduce, may be negative to index from the end (e.g., -1 for the last axis).
 If axis == 0, the output Blob always has the empty shape (count 1), performing reduction across the
 entire input.
 op : can only be one of "SUM, ASUM (sum of abs), SUMSQ (sum of sqr), MEAN"
 coeff : scale for output
 kernel_name : cce kernel name, default value is "cce_reductionLayer"
 need_buid : if need to build CCEC kernel, default value is False
 need_print : if need to print the ir, default value is False
 Returns

 None
 """
 # Basic parameter verification
 # shape parameter verification. The check_shape_rule() function definition is stored in ddk/ddk/site-
packages/topi-0.4.0.egg/topi/cce/util.py.
 util.check_shape_rule(shape)
 check_list = ["float16", "float32"]
 if not (dtype.lower() in check_list):
 raise RuntimeError("Reduction only support %s while dtype is %s" % (" ".join(check_list), dtype))

 reduction_op = ("SUM", "ASUM", "SUMSQ", "MEAN")

 # Axis parameter verification
 if type(axis) != int:
 raise RuntimeError("type of axis value should be int")
 if axis >= len(shape) or axis < -len(shape):
 raise RuntimeError(
 "input axis is out of range, axis value can be from %d to %d" % (-len(shape), len(shape) - 1))
 # op parameter verification
 if operation not in reduction_op:
 raise RuntimeError("op can only be one of SUM, ASUM, SUMSQ , MEAN")
 # coeff parameter verification
 if type(coeff) != int and type(coeff) != float:
 raise RuntimeError("coeff must be a value")
 # Parameter pre-processing
 if axis < 0:
 axis = len(shape) + axis
 shape = list(shape)
 shape1 = shape[:axis] + [reduce(lambda x, y: x * y, shape[axis:])]
 inp_dtype = dtype.lower()
 # Define the tensor object of the input data. The data is only used as a placeholder and no actual
 memory is allocated.
 data = tvm.placeholder(shape1, name="data_input", dtype=inp_dtype)
 # Define the operator calculation process.
 with tvm.target.cce():
 if operation == "ASUM":
 data_tmp_input = te.lang.cce.vabs(data)
 cof = coeff
 tmp = te.lang.cce.vmults(data_tmp_input, cof)
 elif operation == "SUMSQ":
 data_tmp_input = te.lang.cce.vmul(data, data)
 cof = coeff
 tmp = te.lang.cce.vmults(data_tmp_input, cof)
 elif operation == "MEAN":
 size = shape1[-1]
 cof = float(coeff) * (size ** (-0.5))
 tmp = te.lang.cce.vmults(data, cof)
 elif operation == "SUM":

```

```

 cof = coeff
 data_tmp_input = te.lang.cce.vmuls(data, cof)
 tmp = data_tmp_input

 # Sum up data by axis to reduce dimensions.
 res_tmp = te.lang.cce.sum(tmp, axis=axis)
 # Convert the data type.
 res = te.lang.cce.cast_to(res_tmp, inp_dtype, f1628IntegerFlag = True)
 if operation == "MEAN":
 size = shape1[-1]
 sqrt_size = size ** (-0.5)
 res = te.lang.cce.vmuls(res_tmp, sqrt_size)
 # Generate the schedule object to be calculated by the operator.
 sch = generic.auto_schedule(res)
 # Define compilation parameters.
 config = {"print_ir": need_print,
 "need_build": need_build,
 "name": kernel_name,
 "tensor_list": [data, res]}
 # Compile the operator and generate the target file.
 te.lang.cce.cce_build_code(sch, config)

Call the reduction operator by using the parameters: shape (2, 3, 4), datatype (float16), axis (1), op
(SUM), coeff (2), and operator name (Reduction).
if __name__ == "__main__":
 reduction((2, 3, 4), "float16", 1, "SUM", coeff = 2, kernel_name = "Reduction")

```

## 5.7.4 Operator Running Verification

This section describes how to independently run a custom operator to verify its correctness.

### 5.7.4.1 Compiling an Operator

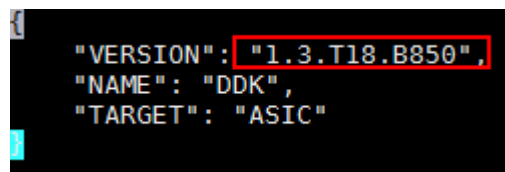
Compile the operator code to generate the operator binary file and operator description file as follows:

**Step 1** Obtain the DDK version number, which is required for compiling the operator.

Check the DDK version number in **\$HOME/tools/che/ddk/ddk/ddk\_info**.

As shown in **Figure 5-8**, the **VERSION** field indicates the current DDK version.

**Figure 5-8** Checking the DDK version number



```

"VERSION": "1.3.T18.B850",
"NAME": "DDK",
"TARGET": "ASIC"

```

**Step 2** Set the version number and compile the operator.

1. Run the following command in the **customop\_te/operator** directory to enter the python interaction mode:

```
python
```

2. In python interaction mode, run the following commands in sequence to set the DDK version number:

```

from topi.cce import te_set_version
import subprocess

```

```
te_set_version("1.3.T18.B850")
```

**Figure 5-9** shows an example.

**Figure 5-9** Example of setting the DDK version number

```
ascend@ascend-HP-ProDesk-600-G4-PCI-MT:~/tools/projects/customop_te/operator$ python
Python 2.7.12 (default, Nov 12 2018, 14:36:49)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license()" for more information.
>>> from topi.coe import te_set_version
[06:54:41] src/tbe_lang/tune/plugin/plugin_mgr.cc:86: plugin type:tiling name:ml_tuning is registered successfully!
>>> import subprocess
>>> te_set_version("1.3.T18.B850")
'success'
```

3. Run the following commands to compile the **reduction.py** file to generate the binary file and description file of the operator:

```
subprocess.call("python reduction.py",shell=True)
```

**Figure 5-10** Compiling operator files

```
>>> subprocess.call("python reduction.py",shell=True)
[06:55:46] src/tbe_lang/tune/plugin/plugin_mgr.cc:86: plugin type:tiling name:ml_tuning is registered successfully!
0
```

4. Exit the python interaction mode.

```
quit(0)
```

**Step 3** After the operator compilation is completed, the **kernel\_meta** folder is generated in the current operator directory. This folder contains the operator binary file \*.o (with operators to run on AI Core) or \*.so (with operators to run on AI CPU) and operator description file \*.json.

- The \*.o or \*.so file is the binary file of the operator.
- The .json file is the operator description file, which is used to define operator properties and resources required for running.

The .json file is parsed as follows:

```
{
 "binFileName": "Reduction", // Binary file name of the generated operator
 "binFileSuffix": ".o", // Extension of the generated operator binary file. For the operators to be
 // running on AI CPU, the generated binary file name is suffixed by .so.
 "blockDim": 1, // Number of AI Cores used for calculation
 "kernelName": "Reduction_kernel0", // Name of the kernel function of the operator
 "magic": "RT_DEV_BINARY_MAGIC_ELF", // The operation target is AI Core. If the value of magic is
 // RT_DEV_BINARY_MAGIC_ELF_AICPU, the operation target is AI CPU.
 "sha256": "d5158df2ff2e64743eec7f527ddd9078b99c1d670f5adbd8b789224657ab0f91" // Value
 // obtained after the .o file is encrypted
}
```

----End

### 5.7.4.2 Building an Input Data File

To run a single operator, you need to build input data and save it in binary format to the operator project.

**Step 1** Go to the operator code directory of the custom operator project as the DDK installation user.

```
cd $HOME/tools/projects/customop_te/operator/
```

**Step 2** Run the script compiled based on sample data to generate a sample data file for the reduction operator.

```
python data_gen.py reduction
```

The data files shown in [Figure 5-11](#) are generated in the operator project.

**Figure 5-11** Generated sample data

```
Reduction_input_2_3_4_sum_axis_1.data
Reduction_input_2_3_4_sum_axis_1.txt
Reduction_output_2_3_4_sum_axis_1.data
Reduction_output_2_3_4_sum_axis_1.txt
```

[Table 5-7](#) describes the data files.

**Table 5-7** Description of data files

| Data File                              | Description                                                                                                                                                                            |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reduction_input_2_3_4_sum_axis_1.data  | Indicates the input data file in binary format of the reduction operator.                                                                                                              |
| Reduction_input_2_3_4_sum_axis_1.txt   | Displays the binary file of the reduction operator in .txt format to facilitate result viewing.                                                                                        |
| Reduction_output_2_3_4_sum_axis_1.data | Indicates the output data verification file in binary format of the reduction operator, which is used to verify whether the output of the operator is correct after the operator runs. |
| Reduction_output_2_3_4_sum_axis_1.txt  | Displays the binary file of the reduction operator in .txt format to facilitate result viewing.                                                                                        |

----End

### 5.7.4.3 Running a Single Operator

**Step 1** Generate a single-operator compilation binary file in the DDK sample project **customop\_runner**.

1. Assign the write permission to the **customop\_runner** sample project.  
**chmod -R +w \$HOME/tools/che/ddk/ddk/sample/customop/customop\_runner/**
2. Go to the **build** directory of the DDK sample project **customop\_runner**.  
**cd \$HOME/tools/che/ddk/ddk/sample/customop/customop\_runner/build**
3. Set the environment variable of **DDK\_PATH**.  
**export DDK\_PATH=\$HOME/tools/che/ddk/ddk**
4. Generate the **Makefile** file.
  - For the Atlas 200 DK developer board, run the following command:  
**cmake -Dtarget=OI .**
  - In the ASIC environment, run the following command:  
**cmake .**
5. Run the **make** command to generate a binary file.  
**make**

The executable file **main** and dynamic library **libcustom\_engine.so** are generated in **\$HOME/tools/che/ddk/ddk/sample/customop/customop\_runner/out/**.

**Step 2** Copy the **out** directory to the directory of the custom TE operator project.

```
cp -rf $HOME/tools/che/ddk/ddk/sample/customop/customop_runner/out
$HOME/tools/projects/customop_te/
```

**Step 3** Configure the input data, output data, and verification description file.

Go to the **out** folder in the directory of the custom TE operator project.

```
cd $HOME/tools/projects/customop_te/out
```

- In the **input.txt** file, configure the path and name of the input data file.

For example:

```
dataPath=./operator/Reduction_input_2_3_4_sum_axis_1.data
```

If the custom operator has multiple inputs, you need to define multiple **.txt** files according to **input.txt**, for example, **input1.txt**, **input2.txt**, and more.

- Configure the output data in the **output.txt** file.

For example:

```
size=4
dataPath=./output/out0.data
dtype=1
```

- **size**: expected size of the output data file, in bytes

- **dataPath**: path and name of the output data file

- **dtype**: output data type. The value **1** indicates **float16**, and the value **2** indicates **float32**.

- In the **expect.txt** file, configure the path and file name of the data to be generated.

For example:

```
dataPath=./operator/Reduction_output_2_3_4_sum_axis_1.data
```

**Step 4** Copy the **operator** and **out** folders to the host of the developer board or ASIC device as the **HwHiAiUser** user. The **operator** and **out** folders must be placed in the same directory.

For example, copy **operator** and **out** folders to the **/home/HwHiAiUser/projects** directory.

**Step 5** Run the operator.

Log in to the host of the developer board or ASIC device as the **HwHiAiUser** user and go to the **out** directory, for example, **/home/HwHiAiUser/projects/out**.

1. Create the **output** folder in the current directory to store the generated data file. The file path and name are defined in the **output.txt** file.

```
mkdir output
```

2. Run the following command to assign the execution permission to the **main** file:

```
chmod +x main
```

3. Run the following command to run a single operator and compare the operators:



```
./main -i input.txt -o output.txt -e expect.txt -b ../operator/kernel_meta/
Reduction.o -p 0.2 -d 0.2 -k Reduction_kernel0 -t 0
```

In the preceding command:

- **-i**: input data configuration file, which specifies the path and file name of the input data  
If the custom operator has multiple inputs, you need to define multiple .txt files according to **input.txt**, for example, **input1.txt**, **input2.txt**, and more.
- **-o**: output data configuration file, which specifies the size, path, file name, and type of the output data
- **-e**: expected data configuration file, which specifies the path and file name of the expected output data for result comparison
- **-b**: operator binary file \*.o (operator to be running on AI Core) or \*.so (operator to be running on AI CPU)
- **-p**: allowed precision deviation. The value range is [0, 1). A smaller value indicates higher precision.
- **-d**: statistical discrepancy, that is, the percentage of the data whose precision deviation is above the threshold. The value range is (0, 1). A smaller value indicates higher precision.
- **-k**: kernel name. The first letter must be in uppercase. The kernel name of the TE operator must be the same as **kernelName** in the .json file. The kernel name of the C++ operator must be the same as the object name **opetype** registered in Framework for the operator.
- **-t**: The value can be **0** (TE\_AI Core operator), **1** (TE\_AI CPU operator), or **2** (custom C++ operator or AI CPU operator).

You can check whether the TE operator is running on AI Core or AI CPU based on the value of **magic** in the \*\*.json file generated after the operator is compiled.

After the command is executed successfully, the **out0.data** result file and the **verifyResult.txt** verification data comparison result file are generated in the **output** folder of the current directory.

The following is an example of the **verifyResult.txt** file, indicating that the actual output data is consistent with the expected output data.

```
Output file ./output/out0.data compare result true
```

----End

## 5.8 caffe.proto File Operator Definition (Optional)

If a model under the Caffe open source framework is used, define the **caffe.proto** file by referring to the description in this section. If a model under the TensorFlow open-source framework is used, skip this section.

After the operator is developed, add the definition of the custom operator to the built-in **caffe.proto** file of the DDK. If the definition of this operator already exists in the built-in **caffe.proto** file, skip this section.

**If there are multiple unsupported custom operators in the same model, add related operator definitions at a time by referring to this section.** During the

implementation of operator plug-ins, related operator parameters are read from the **caffe.proto** file based on the operator name, and then the data structures of operators are converted into those supported by the offline model supported by the Ascend AI processor.

The built-in **caffe.proto** file of the DDK is stored in **\$HOME/tools/che/ddk/ddk/include/inc/custom/proto/caffe/caffe.proto**. You can modify the file and add the definition of the custom operator.

The following describes how to add the definition of the reduction operator (the definition of the reduction operator has been added to the built-in **caffe.proto** file of the DDK):

**Step 1** Add the definition of the reduction operator to **LayerParameter**.

Add the definition of the reduction layer to **LayerParameter** and set its ID.

```
message LayerParameter {
...
 optional ReductionParameter reduction_param = 136; // The ID must be unique.
...
}
```

**Step 2** Add the parameter definition of the reduction layer to the **caffe.proto** file.

```
message ReductionParameter {
 enum ReductionOp { // Operation types supported by the operator
 SUM = 1; // Sums the values of all the axes on which the reduce operation is
 performed.
 ASUM = 2; // Sum the calculated absolute values of all axes on which the reduce
 operation is performed.
 SUMSQ = 3; // Sum the squared values of all axes on which the reduce operation is
 performed.
 MEAN = 4; // Average the values of all axes on which the reduce operation is performed.
 }
 optional ReductionOp operation = 1 [default = SUM]; // Defines the operation of the operator.
 optional int32 axis = 2 [default = 0]; // Defines the axis for which the reduce
 operation is to be performed.
 optional float coeff = 3 [default = 1.0]; // coefficient for output // Scalar, indicating the scaling multiple of
 the result
}
```

----End

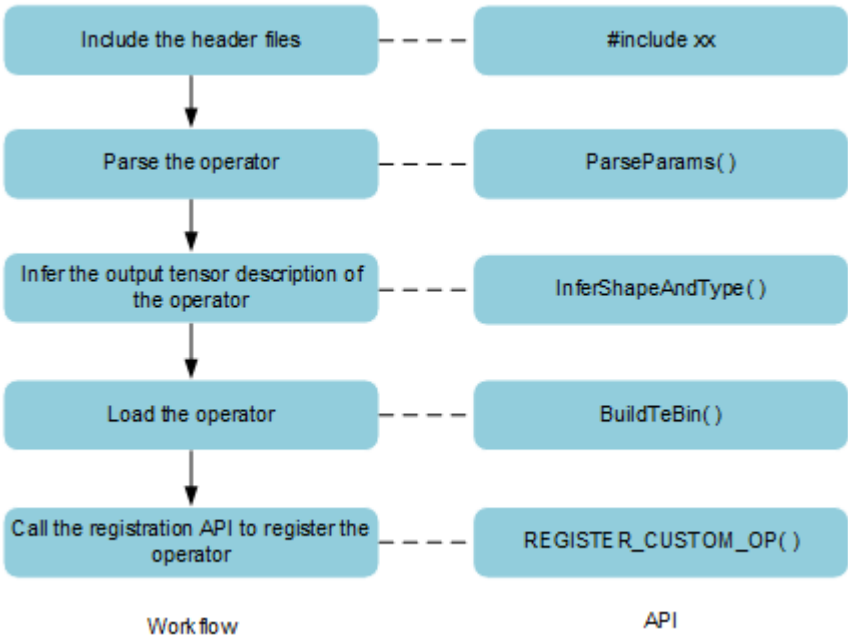
## 5.9 Developing the Plug-In of a Custom Operator

### 5.9.1 Implementing a Plug-In

#### 5.9.1.1 Implementing the Plug-In

After the custom operator is developed, OMG should be able to adapt the attribute values of the custom operator to the offline model so that the custom operator can run on the offline model. Therefore, you need to develop the custom operator plug-in for parsing operator attributes, inferring the shapes and types, and registering the custom operator through the registration mechanism provided by OMG.

Figure 5-12 Implementing a custom operator plug-in



5.9.1.2 Including Header Files

Use the **#include** command in the header of the plug-in implementation file to include the header files related to the plug-in implementation functions in the plug-in implementation source file.

```
#include "custom/custom_op.h"
#include "framework/omg/register.h"
#include "framework/omg/omg_types.h"
#include "proto/caffe/caffe.pb.h"
#include "operator.h"
#include "attr_value.h"
#include <memory>
#include <string>
#include <vector>
```

Table 5-8 Description of header files

| Header File                  | Category                                                                           | Function                                                                                                                                           |
|------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| custom/custom_op.h           | /include/inc/custom/<br>custom_op.h in the DDK<br>installation directory           | User-defined functions<br>for building, operator<br>debugging, and operator<br>verification can be called<br>once this header file is<br>included. |
| framework/omg/<br>register.h | /include/inc/<br>framework/omg/<br>register.h in the DDK<br>installation directory | Class operator<br>registration can be used<br>and APIs of class<br>operator registration can<br>be called once this<br>header file is included.    |

| Header File                   | Category                                                                                                                                   | Function                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| framework/omg/<br>omg_types.h | <b>/include/inc/<br/>framework/omg/<br/>omg_types.h</b> in the DDK<br>installation directory                                               | TBE custom operator<br>information structure<br><b>TEBinInfo</b> can be used<br>once this header file is<br>included.                                                                                                                                                                                                                                               |
| proto/caffe/caffe.pb.h        | <b>proto/caffe/caffe.pb.h</b><br>generated in the<br>directory of the operator<br>project during the<br>building of an operator<br>plug-in | When the operator plug-<br>in is built, the <b>/<br/>include/inc/custom/<br/>proto/caffe/caffe.proto</b><br>file in the DDK<br>installation directory is<br>automatically built, and<br>the <b>proto/caffe/<br/>caffe.pb.h</b> file is<br>generated in the<br>directory of the operator<br>project for the plug-in<br>code to call to parse<br>operator parameters. |
| operator.h                    | <b>include/inc/graph/<br/>operator.h</b> in the DDK<br>installation directory.                                                             | APIs for setting/<br>obtaining operator<br>attributes and setting<br>input/output can be<br>called once this header<br>file is included.                                                                                                                                                                                                                            |
| attr_value.h                  | <b>/include/inc/graph/<br/>attr_value.h</b> in the DDK<br>installation directory                                                           | Data types of class<br><b>AttrValue</b> can be used<br>once this header file is<br>included.                                                                                                                                                                                                                                                                        |
| memory                        | C++ standard library                                                                                                                       | Smart pointers, memory<br>allocators, temporary<br>functions for allocating<br>and releasing dynamic<br>memory, and functions<br>for constructing memory<br>objects in the C++<br>standard library can be<br>called once this header<br>file is included.                                                                                                           |
| string                        | C++ standard library                                                                                                                       | Class <b>string</b> can be used<br>to construct objects APIs<br>of class <b>string</b> can be<br>called once this header<br>file is included.                                                                                                                                                                                                                       |

| Header File | Category             | Function                                                                                                      |
|-------------|----------------------|---------------------------------------------------------------------------------------------------------------|
| vector      | C++ standard library | Vector templates can be used and APIs of class <b>vector</b> can be called once this header file is included. |

Before the operator plug-in is built, ignore the following message (the project does not have the **proto/caffe/caffe.pb.h** file).

**Figure 5-13** Message indicating a caffe.pb.h parsing failure

```
#include "custom/custom_op.h"
#include "framework/omg/register.h"
#include "framework/omg/omg_types.h"
#include "proto/caffe/caffe.pb.h"
#include "operator.h"
#include "attr_value.h"
#include <memory>
#include <string>
#include <vector>
```

### 5.9.1.3 Parsing an Operator

For a newly developed custom operator, you need to customize a function for parsing operator attributes and converting the operator attribute definitions in the source model to the operator attribute definitions in the offline model supported by the Ascend AI processor. If you are rewriting a built-in operator of the Ascend AI processor, skip this step. A built-in operator is automatically parsed by its plug-in.

## Function Declaration

The operator parsing function is declared as follows:

```
Status ParseParamsxx(const Message* op_origin, ge::Operator& op_dest)
```

- **ParseParamsxx**: function name, which is user-defined and must be unique
- **op\_origin**: source operator model. It is a data struct in the protobuf format. It is derived from the .proto file of the Caffe model in the **/include/inc/custom/proto/caffe/caffe.proto** directory under the DDK installation directory. If the custom operator is not defined in the **caffe.proto** file, add the operator definition by referring to [5.8 caffe.proto File Operator Definition \(Optional\)](#).

The operator plug-in reads operator attributes based on the operator name from the **proto/caffe/caffe.pb.h** file and **caffe.pb.cc** file generated after **caffe.proto** building and parses the operator attributes to convert the operator data structure to a data structure supported by the offline model.

Find the preset **caffe.proto** file in **/include/inc/custom/proto/caffe/caffe.proto** in the DDK installation path. You can modify the file and add the definition of the custom operator.

- **op\_dest**: target operator model. As the operator data struct of the offline model supported by the Ascend AI processor, it stores operator information. For details about class **Operator**, see **Class Operator** in *GE API Reference*.

## Procedure

Implement the **ParseParamsxx** function as follows:

- Step 1** Define the object that points to **LayerParameter** and obtain the handle to the current operator layer.

```
const caffe::LayerParameter* layer =dynamic_cast<const caffe::LayerParameter*>(op_origin);
const caffe::xxxParameter& param = layer->reduction_param();
```

In the preceding information:

- **xxxParameter** in **caffe::xxxParameter** of the **param** object must be the same as the type declared in the **LayerParameter** object.
- The name of the member function **xxx\_param()** of the **layer** object must be the same as the object name declared in the **LayerParameter** object.

The following uses the Reduction and convolution operators in **caffe.proto** as an example:

```
message LayerParameter {
 optional ReductionParameter reduction_param = 136;
 optional ConvolutionParameter convolution_param = 106;
 ...
}
```

The code for obtaining the handles to the **Reduction** and **Convolution** operator layers are as follows:

```
const caffe::ReductionParameter& param = layer->reduction_param()
```

```
const caffe::ConvolutionParameter& param = layer->convolution_param()
```

- Step 2** Parse the operator attributes and assign the attributes to the **op\_dest** object of the **Operator** type.

You can call the **CreateFrom<AttrValue::T>(DT&& val)** API to convert **DT** parameters to **T** parameters of class **AttrValue** and call the **SetAttr(const string& name, const AttrValue& value)** API to assign the converted values of the **AttrValue** object to the corresponding attributes of the **op\_dest** object.

Type **T** is introduced to the Ascend AI software stack to simplify the type definition. The supported data types are renamed. For the mapping between type **T** and the source data type, see [Table 5-9](#). For the prototype definitions, see the **include/inc/graph/attr\_value.h** file in the DDK installation directory.

**Table 5-9** Mapping between type **T** and source data types

| T Type | Source Data Type |
|--------|------------------|
| INT    | int64_t          |
| FLOAT  | float            |
| STR    | std::string      |

| T Type           | Source Data Type    |
|------------------|---------------------|
| TENSOR           | TensorPtr           |
| TENSOR_DESC      | TensorDesc          |
| GRAPH            | ComputeGraphPtr     |
| BYTES            | Buffer              |
| NAMED_ATTRS      | NamedAttrs          |
| BOOL             | bool                |
| LIST_INT         | vector<INT>         |
| LIST_FLOAT       | vector<FLOAT>       |
| LIST_BOOL        | vector<BOOL>        |
| LIST_STR         | vector<STR>         |
| LIST_TENSOR      | vector<TENSOR>      |
| LIST_TENSOR_DESC | vector<TENSOR_DESC> |
| LIST_GRAPH       | vector<GRAPH>       |
| LIST_BYTES       | vector<BYTES>       |
| LIST_NAMED_ATTRS | vector<NAMED_ATTRS> |

For details about the **SetAttr** API, see the *GE API Reference*.

The following are examples of parsing common parameters:

- Parameters of the int or float type

For example, the operator parameters in the **caffe.proto** file are defined as follows:

```
message ReductionParameter {

 optional int32 axis = 2 [default = 0];
 optional float coeff = 3 [default = 1.0];
}
```

Call the **SetAttr** API to assign the value of **param.axis()** to the **axis** attribute of the **op\_dest** object and convert the type to INT. Assign the value of **param.coeff()** to the **coeff** attribute of the **op\_dest** object and convert the type to FLOAT, as follows:

```
op_dest.SetAttr("axis", AttrValue::CreateFrom<AttrValue::INT>(param.axis()));
op_dest.SetAttr("coeff", AttrValue::CreateFrom<AttrValue::FLOAT>(param.coeff()));
```

In the preceding information, **CreateFrom<AttrValue::T>(DT&& val)** is used to convert **DT** parameters to **T** parameters of the **AttrValue** class.

- Parameters of the enum type

For example, the operator parameters in the **caffe.proto** file are defined as follows:

```
message ReductionParameter {
 enum ReductionOp {
```

```
SUM = 1;
ASUM = 2;
SUMSQ = 3;
MEAN = 4;
}
...}
```

- a. Convert parameters of the enum type to parameters of the **map** type.  

```
std::map<caffe::ReductionParameter_ReductionOp, std::string> operation_map = {
 { caffe::ReductionParameter_ReductionOp_SUM, "SUM" },
 { caffe::ReductionParameter_ReductionOp_ASUM, "ASUM" },
 { caffe::ReductionParameter_ReductionOp_SUMSQ, "SUMSQ" },
 { caffe::ReductionParameter_ReductionOp_MEAN, "MEAN" },
};
```

- b. Call the **SetAttr** API to assign the *operation\_map* parameter of the map type to the **operation** attribute of the **op\_dest** object.

```
op_dest.SetAttr("operation",
AttrValue::CreateFrom<AttrValue::STR>(operation_map[param.operation()]));
```

For details about the **SetAttr** API, see the *GE API Reference*.

- Parameters of the repeated type

For example, the operator parameters in the **caffe.proto** file are defined as follows:

```
message xxxParameter {
...
 repeated float min_size = 1;
 repeated uint32 offset = 2;
....
}
```

- a. Convert parameters of the repeated float type to parameters of the **vector<float>** type, convert parameters of the repeated uint32 type to parameters of the **vector<uint32>** type, and assign values to the parameters of the vector type.

```
vector<float> min_size;
vector<uint32> offset;
for(int i = 0; i < param.min_size_size(); ++i)
{
 min_size.push_back(param.min_size(i)); // Call the push_back function of the
vector type to assign a value to the min_size parameter of the repeated object.
}
for(int i = 0; i < param.offset_size(); ++i)
{
 offset.push_back(param.offset(i)); // Call the push_back function in the vector
object to assign a value to the offset parameter of the repeated object.
}
```

- b. Call the **SetAttr** API to assign the value of the **min\_size** parameter of the **vector<float>** type to the **min\_size** attribute of the **op\_dest** object. Assign the value of the **offset** parameter of the **vector<uint32>** type to the **offset** attribute of the **LIST\_INT** type in the **op\_dest** object.

```
op_dest.SetAttr("min_size",
ge::AttrValue::CreateFrom<ge::AttrValue::LIST_FLOAT>(min_size));
op_dest.SetAttr("offset", ge::AttrValue::CreateFrom<ge::AttrValue::LIST_INT>(offset));
```

For details about the **SetAttr** API, see the *GE API Reference*.

----End



### 5.9.1.4 Inferring the Output Tensor Description of an Operator

Infer the output tensor description of the operator based on the input tensor description, operator logic, and operator attributes. The output tensor description includes the tensor shape, data type, and data layout format. In this way, all tensors can be statically allocated with memory during offline model conversion, thereby avoiding overhead caused by dynamic memory allocation.

#### Function Declaration

The function is declared as follows:

```
Status InferShapeAndTypexx(const ge::Operator& op, vector<ge::TensorDesc>& v_output_desc)
```

- **InferShapeAndTypexx**: function name, which is user-defined and must be unique
- **op**: compute node definition, which stores the input tensor description and operator attributes. For details about the `ge::Operator` type, see **Class Operator** in *GE API Reference*
- **v\_output\_desc**: output tensor description of the compute node, including the shape, data layout format, and data type. For details about the **TensorDesc** type, see **Class TensorDesc** in *GE API Reference*

The following describes the implementation of the **InferShapeAndTypexx** function in different scenarios.

#### Operator with Same-Shape Output and Input Tensors

For an operator whose output and input tensors have the identical shape, you can directly insert the description of the input tensor into the vector space in which the output tensor description is located.

A code sample is provided as follows:

```
v_output_desc.push_back(op.GetInputDesc(0));
```

The **GetInputDesc** API is used to obtain the input tensor description based on the operator input name or input index in class `Operator`. For details about the API, see **Class Operator** in *GE API Reference*.

#### Operator with Reduced Dimensions

For common dimension reduction operations such as Reduction and Reduce, compute the shape of the output tensor (including the output tensor dimensions and element count of each dimension) according to information such as the operator input attribute **axis**, and then insert the shape of the output tensor into the **v\_output\_desc** vector.

A code sample is provided as follows:

**Step 1** Obtain the input tensor description and shape of the input tensor of the operator.

```
auto tensorDesc = op.GetInputDesc(0); // Obtain the input tensor description, including the
shape, data layout format, and data type.
auto shape = tensorDesc.GetShape(); // Obtain the shape of the input tensor.
```

For details about the **GetShape** API, see **Class TensorDesc** in *GE API Reference*.

- Step 2** Obtain the attribute values of the operator according to the computation logic, and compute the shape of the output tensor of the operator.

For example, for the reduction operator in the MyLeNet network, because the upper layer of Reduction is Softmax and the output from Softmax is padded to 4-dimensional from 2-dimensional in the offline model, you need to adjust **axis** so that it points to a 2-dimensional position. After the reduce operation, and assign the adjusted **Shape** to the output tensor description.

- Obtain the key-value pair of the **axis** attribute from the operator object, obtain the **axis** attribute value from the key-value pair, convert the attribute from the INT type to the int64\_t type, assign the attribute value to the variable **axis**, verify and adjust the value of **axis**, and point the value to axis 1, as follows:

```
int64_t axis = -1;
ge::AttrValue axisAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("axis", axisAttrValue)) || (ge::GRAPH_SUCCESS != axisAttrValue.GetValue<AttrValue::INT>(axis)))
{
 printf("Get axis failed!\n");
}
// In the OM model, all shape are supplemented to 4d. In this case, axis needs to be repaired to point to the original 2d.
if (axis < 0) axis -= 2;

if (axis < 0) axis += shape.GetDimNum();

if (axis < 0 || axis >= shape.GetDimNum())
{
 printf("invalid axis:%d, dim_size:%d\n", (int32_t)axis, (int32_t)shape.GetDimNum());
 return PARAM_INVALID;
}
```

- Adjust **Shape** and set the dimensions from axis 1 to 1. For example, if the input tensor is with shape (2, 3, 4, 5), adjust the shape to (2, 1, 1, 1).

```
int32_t dimsize = (int32_t)shape.GetDimNum();
int32_t idx = 0;
for(idx=axis; idx<dimsize; idx++)
{
 shape.SetDim(idx, 1);
}
```

- Set the adjusted shape to the **tensorDesc** object.

```
tensorDesc.SetShape(shape);
```

For details about APIs **GetDimNum** and **SetDim**, see **Class Shape** in *GE API Reference*.

- Step 3** Set the output tensor description of the operator.

```
v_output_desc.push_back(tensorDesc)
```

Assign **tensorDesc** to the description object **v\_output\_desc** of the output tensor.

----End

## Network with Multiple Operators of the Identical Type

A network can have multiple operator layers of the same type, such as the convolution operator. Sometimes, you need to customize the shape of an operator layer (redefine an existing operator). In this case, the output tensor description inference needs to be performed according to different situations of the network, determine the operator layers to be customized based on the attributes such as

**num\_output**, **kernel**, **stride**, and **pad**, and obtain the tensor information of the operators.

A code sample is provided as follows:

- Step 1** Assign the input tensor description to the output tensor description, and obtain the input tensor description and the shape of the input tensor of the operator.

```
v_output_desc.push_back(op.GetInputDesc(0)); // Assign the input tensor description to the
output tensor description object. Alternatively, assign the shape obtained after inference to the
tensorDesc object, and then assign the tensorDesc object to the output tensor description
object.
auto tensorDesc = op.GetInputDesc(0); // Obtain the input tensor description, including the
shape, data layout format, and data type.
auto shape = tensorDesc.GetShape(); // Obtain the shape of the input tensor.
```

For details about the **GetShape** API, see **Class TensorDesc** in *GE API Reference*.

- Step 2** Obtain the attribute values of the operator according to the computation logic, match the operator according to the attribute values of the operator and shape, and compute the shape of the output tensor of the operator.

For example, for operators of type **convolution** in a network, match the convolution operator whose **num\_outputs** is **128**, **shape.GetDim(0)** is **1**, **shape.GetDim(1)** is **128**, **shape.GetDim(2)** is **28**, and **shape.GetDim(3)** is **28**, and reassign the shape in the output tensor description of the operator.

- Obtains the value of **num\_outputs**.

```
ge::AttrValue num_outputsAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("num_output", num_outputsAttrValue)) ||
 (ge::GRAPH_SUCCESS != num_outputsAttrValue.GetValue<AttrValue::INT>(num_outputs)))
{
 printf("GetOpAttr num_outputs failed!\n");
}
```

- Match the convolution operator whose **num\_outputs** is **128**, **shape.GetDim(0)** is **1**, **shape.GetDim(1)** is **128**, **shape.GetDim(2)** is **28**, and **shape.GetDim(3)** is **28**, and reassign the shape in the output tensor description of the operator.

```
if(shape.GetDim(0) == 1 && shape.GetDim(1) == 128 &&
 shape.GetDim(2) == 28 && shape.GetDim(3) == 28 && num_outputs == 128)
{
 shape.SetDim(0, 1);
 shape.SetDim(1, 128);
 shape.SetDim(2, 28);
 shape.SetDim(3, 28);
 v_output_desc[0].SetShape(shape);
 return SUCCESS;
 return FAILED;
}
```

For details about APIs **GetDimNum** and **SetDim**, see **Class Shape** in *GE API Reference*.

#### NOTE

To use **op\_name** for operator matching, obtain **op\_name** as follows:

```
auto op_name = op.GetName();
```

The method for obtaining other attributes **kernel\_w**, **kernel\_h**, **stride\_w**, **stride\_h**, **pad\_w**, and **pad\_h** are similar. You only need to change the value of **key** in **op.GetAttr**.

----End

### 5.9.1.5 Building an Operator

#### Function Declaration

The operator building function is declared as follows:

```
Status BuildTeBinxx(const ge::Operator& op, TEBinInfo& te_bin_info)
```

In the preceding information:

- **BuildTeBinxx**: function name, which is user-defined and must be unique
- **op**: target operator model. As the operator data struct of the offline model supported by the Ascend AI processor, it stores operator information. For details about class **Operator**, see **Class Operator** in *GE API Reference*.
- **te\_bin\_info**: path of the operator binary file, operator description file path, and DDK version information. For details about the **TEBinInfo** struct, see **TEBinBuildFn** in *Framework API Reference*.

#### Implementation Procedure

The operator building function is called during model conversion by OMG as follows:

- Obtain the operator tensor description and operator attributes. During model conversion, the information must be fixed values for operator matching.

For example, in model conversion, match the reduction operator whose **axis** is **1** and **Dim** of the input tensor **Shape** is **4**.

```
// Parse the operator attribute operation.
ge::AttrValue operationAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("operation", operationAttrValue)) || (ge::GRAPH_SUCCESS !=
operationAttrValue.GetValue<AttrValue::STR>(operation)))
{
 printf("GetOpAttr operation failed!\n");
}

// Parse the operator attribute axis, and adjust axis to point to the actual output dimension of the
Softmax operator at the upper layer of the reduction operator in the MyLeNet network, that is, axis 1.
ge::AttrValue axisAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("axis", axisAttrValue)) || (ge::GRAPH_SUCCESS !=
axisAttrValue.GetValue<AttrValue::INT>(axis)))
{
 printf("GetOpAttr axis failed!\n");
}
// In the OM model, all shape are supplemented to 4d. In this case, axis needs to be repaired to
point to the original 2d.
if(axis < 0)
 axis -= 2;

// Parse the operator attribute coeff.
ge::AttrValue coeffAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("coeff", coeffAttrValue)) || (ge::GRAPH_SUCCESS !=
coeffAttrValue.GetValue<AttrValue::FLOAT>(coeff)))
{
 printf("GetOpAttr coeff failed!\n");
}

// Obtain the input tensor description of the operator.
TensorDesc input_desc = op.GetInputDesc(0);

// Parse the input shape and check whether Dim of the operator is 4.
if(input_desc.GetShape().GetDimNum() != 4)
{
 printf("The shape size is %d, which is not 4!", (int32_t)input_desc.GetShape().GetDimNum());
}
```

```
 return FAILED;
}
```

- Specify the operator implementation file, operator implementation function, and operator name in the kernel.

```
FilePath = "project_path/operator/reduction"; // Absolute path of the operator implementation file
+ name of the .py operator file
FuncName = "Reduction"; // Name of the operator implementation function in the operator
implementation file
KernelName = "Reduction"; // kernel_name defined in the operator implementation function of
the operator implementation file, that is, the name of the generated binary file
```

- Specify the path of the operator description file (\*.json) generated during operator compilation. Use the following fixed configuration.

```
te_bin_info.json_file_path = "./kernel_meta/" + KernelName + ".json";
```

During model conversion, operator information will be obtained from the operator description file in this path.

When the **omg** model conversion command is executed, the **kernel\_meta** folder generated after operator building is copied to the path where the **omg** command is executed based on the operator implementation path configured in the **FilePath** file. Therefore, the path of the \*.json file relative to the path where the **omg** command is executed is fixed to **./kernel\_meta**.

- Call the **te::BuildCustomop** function to call the Python function in the operator implementation file to build the operator.

Call the **BuildCustomop** function as follows:

```
te::BuildTeCustomOp(te_bin_info.ddk_version, op.GetName(), FilePath, FuncName, "(i,i,i), s, i, s, f, s",
input_desc.GetShape().GetDim(0),input_desc.GetShape().GetDim(1),input_desc.GetShape().GetDim(2),
input_desc.GetShape().GetDim(3), "float16", axis, operation.c_str(), coeff,KernelName.c_str());
```

In the preceding information:

- te\_bin\_info.ddk\_version**: DDK version information (unconfigurable), which will be automatically filled during model conversion
- op.GetName()**: obtaining operator name (unconfigurable),
- FilePath**: relative path of the operator file
- FuncName**: name of the operator implementation function in the operator implementation file
- (i, i, i, i), s, i, s, f, s**: parameter placeholders of the implementation functions in the operator implementation file, where, **i** indicates the integer type, **s** indicates the string type, **f** indicates the single-precision floating point number type, and **o** indicates the **PyObject\*** type. The placeholders must be consistent with the sequence and types of the succeeding parameters, and must be consistent with the definition of the operator implementation function in the operator implementation file. **BuildCustomop** calls the operator implementation function based on these parameters and generates the kernel using the TVM mechanism.

### 5.9.1.6 Registering an Operator

As the framework manager, Framework provides the **REGISTER\_CUSTOM\_OP** macro to register an operator based on the specified operator name.

The code of custom operator registration is as follows:

```
REGISTER_CUSTOM_OP("test_layer")
.FrameworkType(CAFFE)
.OriginOpType("Test")
.ParseParamsFn(ParseParamsxx)
```

```
.InferShapeAndTypeFn(InferShapeAndTypeFn)
.TEBinBuildFn(BuildTeBinxx)
.ImplyType(ImplType::TVM)
.Formats({DOMI_TENSOR_NC1HWC0}, {DOMI_TENSOR_NC1HWC0})
.WeightFormats({DOMI_TENSOR_FRAC_TAL_Z, DOMI_TENSOR_NC1HWC0});
```

In the preceding information:

- **REGISTER\_CUSTOM\_OP**: Registers a custom operator. Replace **test\_layer** with the operator name in the offline model file. The operator name can be random but must not conflict with existing operator names.
- **FrameworkType**: The operator parameter parsing logic varies depending on the framework. Therefore, models under different frameworks require different plug-ins. The plug-in registration code must specify the model framework. Set this parameter to **CAFFE**.
- **OriginOpType**: operator type, which must be the same as the operator type defined in Caffe Prototxt. Otherwise, parsing fails. Find the preset **caffe.proto** file in **/include/inc/custom/proto/caffe/caffe.proto** in the DDK installation path.
- **ParseParamsFn**: Registers the function for model parsing. **ParseParamsxx** has been implemented in [5.9.1.3 Parsing an Operator](#). This step is required for a plug-in developed for the Caffe framework. If you are rewriting a built-in operator of the Ascend AI processor, skip this step. If the custom operator is not supported by the Ascend AI processor, this step is mandatory.
- **InferShapeAndTypeFn**: Registers the function for shape and class inference. **InferShapeAndTypeFn** has been implemented in [5.9.1.4 Inferring the Output Tensor Description of an Operator](#).
- **TEBinBuildFn**: Registers the TBE operator building function. **BuildTeBinxx** has been implemented in [5.9.1.5 Building an Operator](#).
- **ImplType**: Specifies the operator implementation. **ImplType::TVM** indicates that the operator is a TE operator.
- **Formats**: Specifies the layout formats of the input data and output data of the operator. The first list is the input data format list, and the second list is the output data format list. If there are multiple inputs, list the layout format of each input data in the first list. For example, if there are two pieces of input data in the NC1HWC0 format, call the **Formats** function as follows:  

```
.Formats({DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0},
{DOMI_TENSOR_NC1HWC0})
```

For details, see **Formats** in *Framework API Reference*.

- **WeightFormats**: Sets the layout format of operator weight data. For details about the supported data formats, see **WeightFormats** in *Framework API Reference*. For example, the data layout format of the filter of convolution is **fractal\_Z**, and the data layout format of the filter of bias is **NC1HWC0**.

If quantization during model conversion is enabled, the constant formats for Framework processing need to be added to this API. Currently, Framework supports the following quantization operators: Conv, FC, and Depthwise Conv. If quantization is enabled for these operators during model conversion, you need to add six **DOMI\_TENSOR\_NC1HWC0** data formats to the end of the parameter list of the **WeightFormats** API. (During quantization, Framework adds six constants whose data layout format is NC1HWC0. The following is a code sample of the **WeightFormats** API for the convolution operator with quantization enabled:

```
.WeightFormats({DOMI_TENSOR_FRACTAL_Z, DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0,
DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0,DOMI_TENSOR_NC1HWC0,
DOMI_TENSOR_NC1HWC0, DOMI_TENSOR_NC1HWC0})
```

## 5.9.2 Plug-In Code Example

**Step 1** Modify the code file in **customop\_te/plugin/caffe\_reduction\_layer.cpp**.

- Change **FilePath** to the relative path of the current operator file+name of the operator py file.  
FilePath = "../operator/reduction";
- Change **FuncName** to the name of the operator implementation function in the **reduction.py** file.  
FuncName = "reduction"
- Change the value of **KernelName** to the **KernelName** of the operator configured in the **reduction.py** file.  
KernelName = "Reduction";
- Change the path of the binary file \*.o and description file \*.json of the operator.  
te\_bin\_info.bin\_file\_path = "../operator/kernel\_meta/" + KernelName + ".o";  
te\_bin\_info.json\_file\_path = "../operator/kernel\_meta/" + KernelName + ".json";  
The path is relative to the current project directory. For example, the **Reduction.o** file is in the **operator/kernel\_meta** directory under the current project directory.

**Step 2** Check the modified code file. The following is a code example of the modified reduction operator plug-in:

```
#include <Python.h>
#include "custom/custom_op.h"
#include "framework/omg/register.h"
#include "framework/omg/omg_types.h"
#include "proto/caffe/caffe.pb.h"
#include "operator.h"
#include "attr_value.h"
#include <memory>
#include <string>
#include <vector>
using namespace ge;

namespace domi
{
// Rewrite the ParseParams function to parse operator parameters.
Status CaffeReductionParseParams(const Message* op_origin, ge::Operator& op_dest)
{
 // Convert op_origin into the layer object.
 const caffe::LayerParameter* layer =
 dynamic_cast<const caffe::LayerParameter*>(op_origin);

 // Verify the validity of the input parameters.
 if (nullptr == layer)
 {
 printf("Dynamic cast op_src to LayerParameter failed\n");
 return FAILED;
 }
 // Convert the enumeration type of the operator parameters to the map type.
 std::map<caffe::ReductionParameter_ReductionOp, std::string> operation_map = {
 { caffe::ReductionParameter_ReductionOp_SUM, "SUM" },
 { caffe::ReductionParameter_ReductionOp_ASUM, "ASUM" },
 { caffe::ReductionParameter_ReductionOp_SUMSQ, "SUMSQ" },
 { caffe::ReductionParameter_ReductionOp_MEAN, "MEAN" },
 }
```

```

};
// Obtain the handle to the current operator layer.
const caffe::ReductionParameter& param = layer->reduction_param();
// Assign all parameter values of the operator to the object of the operator data structure op_dest of the
Da Vinci model.
if(param.has_axis())
{
 op_dest.SetAttr("axis", AttrValue::CreateFrom<AttrValue::INT>(param.axis()));
}
if(param.has_coeff())
{
 op_dest.SetAttr("coeff", AttrValue::CreateFrom<AttrValue::FLOAT>(param.coeff()));
}
if(param.has_operation())
{
 op_dest.SetAttr("operation",
AttrValue::CreateFrom<AttrValue::STR>(operation_map[param.operation()]));
}
return SUCCESS;
}

// Rewrite the InferShapeAndType function to obtain the output description of the operator.
Status CaffeReductionInferShapeAndType(const ge::Operator& op, vector<ge::TensorDesc>& v_output_desc)
{
 // Obtain the TensorDesc object input by the operator.
 auto tensorDesc = op.GetInputDesc(0);
 // Obtain the input shape.
 auto shape = tensorDesc.GetShape();
 int64_t axis = -1;

 ge::AttrValue axisAttrValue;
 if ((ge::GRAPH_SUCCESS != op.GetAttr("axis", axisAttrValue)) || (ge::GRAPH_SUCCESS !=
axisAttrValue.GetValue<AttrValue::INT>(axis)))
 {
 printf("Get axis failed!\n");
 }
 // In the OM model, all shape are supplemented to 4d. In this case, axis needs to be repaired to point to
the original 2d.
 if (axis < 0) axis += 2;

 if (axis < 0) axis += shape.GetDimNum();

 if (axis < 0 || axis >= shape.GetDimNum())
 {
 printf("invalid axis:%d, dim_size:%d\n", (int32_t)axis, (int32_t)shape.GetDimNum());
 return PARAM_INVALID;
 }
 int32_t dimsize = (int32_t)shape.GetDimNum();
 int32_t idx = 0;
 for(idx=axis; idx<dimsize; idx++)
 {
 shape.SetDim(idx, 1);
 }
 // Set the adjusted shape information to tensorDesc.
 tensorDesc.SetShape(shape);
 v_output_desc.push_back(tensorDesc);

 return SUCCESS;
}

// Compile the operator and obtain the binary file and description file of the operator.
Status CaffeReductionBuildTeBin(const ge::Operator& op, TEBinInfo& te_bin_info)
{
 // Declare operator parameters.
 std::string FilePath = "";
 std::string FuncName = "";
 std::string KernelName = "";

```



```
std::string operation = "";
int64_t axis = -1;
float coeff = 1;
// Obtain operator parameter values.
ge::AttrValue operationAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("operation", operationAttrValue)) || (ge::GRAPH_SUCCESS !=
operationAttrValue.GetValue<AttrValue::STR>(operation)))
{
 // Add exception handling and maintenance information.
 printf("GetOpAttr operation failed!\n");
}

// Parse the axis parameter.
ge::AttrValue axisAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("axis", axisAttrValue)) || (ge::GRAPH_SUCCESS !=
axisAttrValue.GetValue<AttrValue::INT>(axis)))
{
 printf("GetOpAttr axis failed!\n");
}
// In the OM model, all shape are supplemented to 4d. In this case, axis needs to be repaired to point to
the original 2d.
if(axis < 0)
 axis -= 2;

// Parse the coeff parameter.
ge::AttrValue coeffAttrValue;
if ((ge::GRAPH_SUCCESS != op.GetAttr("coeff", coeffAttrValue)) || (ge::GRAPH_SUCCESS !=
coeffAttrValue.GetValue<AttrValue::FLOAT>(coeff)))
{
 printf("GetOpAttr coeff failed!\n");
}
// Obtain the operator input description.
TensorDesc input_desc = op.GetInputDesc(0);

// Parse the input shape and check whether Dim of the operator is 4.
if(input_desc.GetShape().GetDimNum() != 4)
{
 printf("The shape size is %d, which is not 4!", (int32_t)input_desc.GetShape().GetDimNum());
 return FAILED;
}
// Set the operator file path.
FilePath = "../operator/reduction";
// Set the name of the operator implementation function.
FuncName = "reduction";
// Set the operator name defined in the operator implementation file.
KernelName = "Reduction";

// i => int; s => string; f => double; O => bool, and bool value is Py_True or Py_False. Call the compilation
interface of Tensor Engine to compile the operator.
te::BuildTeCustomOp(te_bin_info.ddk_version, op.GetName(), FilePath, FuncName,
 "(i,i,i,i), s, i, s, f, s", input_desc.GetShape().GetDim(0), input_desc.GetShape().GetDim(1),
 input_desc.GetShape().GetDim(2), input_desc.GetShape().GetDim(3), "float16", axis,
operation.c_str(), coeff,
 KernelName.c_str()); // The parameter sequence must be the same as that in the operator
implementation function.

// set te op json to te_bin_info
te_bin_info.bin_file_path = "./kernel_meta/" + KernelName + ".o";
te_bin_info.json_file_path = "./kernel_meta/" + KernelName + ".json";

return SUCCESS;
}

REGISTER_CUSTOM_OP("custom_reduction") //####test_reduction is the type name of the operator in the
OM model. It can be specified randomly and cannot be the same as an existing type name. It is case
sensitive.
.FrameworkType(CAFFE) // Enumerated type. The options are as follows: CAFFE, TENSORFLOW
.OriginOpType("Reduction") // Reduction indicates the type name of the operator in the caffe
```

```
framework.
.ParseParamsFn(CaffeReductionParseParams) // Op parameters parse function
.InferShapeAndTypeFn(CaffeReductionInferShapeAndType) // Set output description and datatype
function
.TEBinBuildFn(CaffeReductionBuildTeBin) // Build Te op binary function
.ImplyType(ImplType::TVM); // Implementation type. Enumerated type, The options are as follows:
TVM, AI_CPU.
} // namespace domi
```

----End

## 5.9.3 Compiling the Operator Plug-In

**Step 1** Log in to the DDK server as the DDK installation user.

**Step 2** Modify the **Makefile** file in **projects/customop\_te/plugin**.

```
cd $HOME/tools/projects/customop_te/plugin/
```

**vi Makefile**

- Change the name of the generated operator plug-in.

```
ll : libcaffe_reduction_layer.so lib_caffe_parser.so
.....bian
libcaffe_reduction_layer.so: $(OBS_customop)
$(CC) -c $(CC_FLAGS) -o proto/caffe/caffe.pb.o proto/caffe/caffe.pb.cc
$(CC) $^ $(LNK_FLAGS) -o $@

lib_caffe_parser.so: $(OBS_no_customop)
$(CC) -c $(CC_FLAGS) -o proto/caffe/caffe.pb.o proto/caffe/caffe.pb.cc
@if [-f $(LOCAL_DIR)/proto/caffe/caffe.proto]; then $(CC) $^ proto/caffe/caffe.pb.o $
(LNK_FLAGS) -o $@; fi;
```

**libcaffe\_reduction\_layer.so** is the name of the generated operator plug-in.  
You can change the name as required.

**lib\_caffe\_parser.so** is the library file generated during the parsing of the **caffe.proto** file, and its name cannot be changed. For Caffe operators, ensure that all unsupported custom ones in the same model have been defined in [5.8 caffe.proto File Operator Definition \(Optional\)](#).

- Set **TOPDIR** to the installation directory of the DDK.

```
ifeq ($(DDK_PATH),)
TOPDIR := $(HOME)/tools/che/ddk/ddk
else
TOPDIR := $(DDK_PATH)
endif
```

- Use the default values for other parameters in **Makefile**, which serve as the common template for the operator plug-in compilation.

**Step 3** Compile the operator plug-in.

Run the following command in the plug-in directory to compile the operator plug-in:

**make**

After the compilation is complete, the operator plug-in file **libcaffe\_reduction\_layer.so** is generated in the current directory.

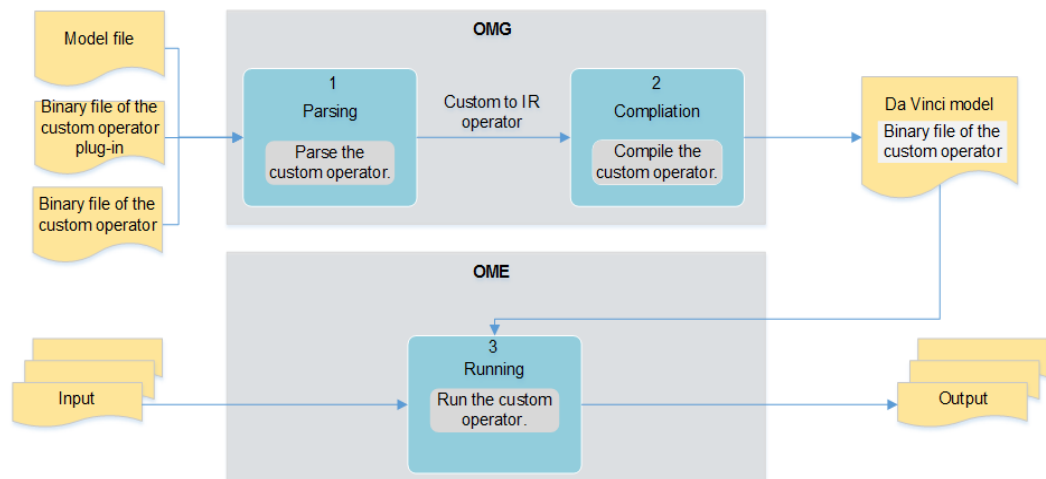
----End

## 5.10 Loading the Plug-In for Model Conversion

### Principle Description

**Figure 5-14** shows the model conversion process when the custom operator plug-in is loaded.

**Figure 5-14** Loading a plug-in for model conversion



1. The offline model generator (OMG) loads the model file and custom operator plug-in, parses the operator in the model file, and converts the custom operator into the intermediate representation (IR) operator.
2. The OMG converts the data of the custom operator based on the running environment, calculates the running memory, compiles and generates the binary file (\*.o) of the custom operator, and generates the Da Vinci offline model file (\*.om).
3. During application running, the offline model executor (OME) obtains the input data, loads the offline model file, calls the operator cyclically, and outputs the result data.

### Operating Procedure

**Step 1** Go to the root directory of the custom operator development project as the DDK installation user.

```
cd $HOME/tools/projects/customop_te/
```

**Step 2** Run the following command to convert the model:

```
omg --model=model/deploy_mylenet-1.prototxt --weight=model/
mylenet-1.caffemodel --framework=0 --plugin_path=plugin --output=mylenet
--ddk_version=1.3.T18.B850
```

- **--model**: relative path of the original model file of the MyLeNet network
- **--weight**: relative path of the pre-trained model file of the MyLeNet network

- **--framework:** original framework type
  - 0: Caffe
  - 3: TensorFlow
- **--plugin:** directory where the custom operator plug-in is located
- **--output:** name of the output model file, which can be customized
- **--ddk\_version:** version number of the matched DDK for running the custom operator. You can view the version number of the DDK in the **\$HOME/tools/che/ddk/ddk/ddk\_info** file.

 **NOTE**

For details about other parameters, see *Model Conversion Guide*.

----End

## 5.11 Function Reference for Operator Plug-In Development

### 5.11.1 ParseParamsFn

#### Function

Parses parameters.

#### Syntax

```
OpRegistrationData& ParseParamsFn(ParseParamFunc parseParamFn);
```

#### Parameter Description

| Parameter    | Input/Output | Description                                                                                           |
|--------------|--------------|-------------------------------------------------------------------------------------------------------|
| parseParamFn | Input        | Callback function ParseParamFunc. For details, see <a href="#">Callback Function ParseParamFunc</a> . |

### Callback Function ParseParamFunc

You can customize and implement the ParseParamFunc class functions to convert the parameters and weights of the Caffe model and fill the results in the Operator class.

#### Syntax

```
Status ParseParamFunc(const Message* op_origin, ge::Operator& op_dest);
```

#### Parameter Description

| Parameter | Input/<br>Output | Description                                                                                                                                                                                                           |
|-----------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| op_origin | Input            | Data structure in protobuf format (from the prototxt file of the Caffe model), including operator parameter information                                                                                               |
| op_dest   | Output           | Operator data structure of offline model supported by the Ascend AI processor, which stores operator information<br>For details about the operator class, see <b>Operator Class APIs</b> in <i>GE API Reference</i> . |

## 5.11.2 InferShapeAndTypeFn

### Function

Shape inference function

### Syntax

```
OpRegistrationData& InferShapeAndTypeFn(InferShapeFunc inferShapeFn);
```

### Parameter Description

| Parameter    | Input/<br>Output | Description                                                                                           |
|--------------|------------------|-------------------------------------------------------------------------------------------------------|
| inferShapeFn | Input            | Callback function InferShapeFunc. For details, see <a href="#">Callback Function InferShapeFunc</a> . |

## Callback Function InferShapeFunc

You can customize and implement the InferShapeFunc class function to obtain the output description of an operator, including the tensor description such as the output shape information and data type.

### Syntax

```
Status InferShapeFunc(const ge::Operator& op, vector<ge::TensorDesc>&
v_output_desc);
```

### Parameter Description

| Parameter     | Input/Output | Description                                                                                                                                                                        |
|---------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| op            | Input        | Operator data structure of offline model supported by the Ascend AI processor<br>For details about the operator class, see <b>Operator Class APIs</b> in <i>GE API Reference</i> . |
| v_output_desc | Output       | Operator output description<br>For details about the TensorDesc class, see <b>TensorDesc Class APIs</b> in <i>GE API Reference</i> .                                               |

### 5.11.3 TEBinBuildFn

#### Function

Build callback function

#### Syntax

```
OpRegistrationData& TEBinBuildFn(BuildTeBinFunc buildTeBinFn);
```

#### Parameter Description

| Parameter    | Input/Output | Description                                                                                           |
|--------------|--------------|-------------------------------------------------------------------------------------------------------|
| buildTeBinFn | Input        | Callback function BuildTeBinFunc. For details, see <a href="#">Callback Function BuildTeBinFunc</a> . |

### Callback Function BuildTeBinFunc

You can customize and implement the BuildTeBinFunc class function to construct the operator binary file.

#### Syntax

```
virtual Status BuildTeBinFunc(const ge::Operator& op, TEBinInfo& teBinInfo);
```

#### Parameter Description

| Parameter | Input/<br>Output | Description                                                                                                                                                                                                                                                                                                                                           |
|-----------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| op        | Input            | Operator data structure of offline model supported by the Ascend AI processor, which stores operator information<br>For details about the operator class, see <b>Operator Class APIs</b> in <i>GE API Reference</i> .                                                                                                                                 |
| teBinInfo | Output           | Path of the binary file of a custom operator and DDK description<br>struct TEBinInfo<br>{<br>std::string bin_file_path; // Automatically obtained from the <b>binFileName</b> field in the JSON file. To ensure compatibility with cases written by users, the field is not deleted.<br>std::string json_file_path;<br>std::string ddk_version;<br>}; |

## 5.11.4 WeightFormats

### Function

Sets the data formats supported by the operator weight.

### Syntax

```
OpRegistrationData& WeightFormats(
const std::initializer_list<domi::tagDomiTensorFormat>& weight_formats);
```

## Parameter Description

| Parameter      | Input/<br>Output | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| weight_formats | Input            | Data formats supported by the weight:<br>typedef enum tagDomiTensorFormat<br>{<br>DOMI_TENSOR_NCHW = 0, /**< NCHW */<br>DOMI_TENSOR_NHWC, /**< NHWC */<br>DOMI_TENSOR_ND, /**< Nd Tensor */<br>DOMI_TENSOR_NC1HWC0, /**< NC1HWC0<br>*/<br>DOMI_TENSOR_FRACTAL_Z, /**<<br>FRACTAL_Z */<br>DOMI_TENSOR_NC1C0HWPAD,<br>DOMI_TENSOR_NHWC1C0,<br>DOMI_TENSOR_FSR_NCHW,<br>DOMI_TENSOR_FRACTAL_DECONV,<br>DOMI_TENSOR_BN_WEIGHT,<br>DOMI_TENSOR_CHWN, /*Android NN<br>Depth CONV*/<br>DOMI_TENSOR_FILTER_HWCK, /* filter<br>input tensor format */<br>DOMI_TENSOR_RESERVED<br>} domiTensorFormat_t; |

## 5.11.5 Registration Macro of the Operator Building Function

### Function

Registers the operator by calling DOMI\_REGISTER\_OP.

### Prototype

REGISTER\_CUSTOM\_OP(name)



## Parameter Description

| Parameter | Input/Output | Description                                                                                                                 |
|-----------|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| name      | Input        | Type of an operator in the Da Vinci model, which can be specified randomly and must be unique. The value is case sensitive. |

## 5.12 Operator List

### 5.12.1 Parameter Description

| Parameter                                 | Description                                              |
|-------------------------------------------|----------------------------------------------------------|
| ni                                        | Batch size                                               |
| ci/co                                     | Channel count                                            |
| hi/ho/                                    | Height                                                   |
| wi/wo                                     | Width                                                    |
| sh/sw                                     | Stride                                                   |
| kh/kw                                     | Size of the convolution filter                           |
| window_h(window_y)/<br>window_w(window_x) | Window size                                              |
| dh(dilation_h)/<br>dw(dilation_w)         | Convolution dilation coefficient                         |
| FilterHDilation/<br>FilterWDilation       | H/W dimension of the dilated filter                      |
| FilterH/FilterW                           | H/W dimension of the convolution weight                  |
| padWHead/padHHead                         | Pad head of the H/W dimension                            |
| PadWTail/padHTail                         | Pad tail of the H/W dimension                            |
| dilationsize                              | User-defined dilation coefficient                        |
| FilterSize                                | User-defined filter count                                |
| INT32_MAX                                 | Maximum value that can be represented by data type int32 |
| ALIGN                                     | Roundup alignment                                        |
| CEIL                                      | Mapping to the least succeeding integer                  |

## 5.12.2 Operator Boundaries

### 5.12.2.1 General Restrictions

Under the Caffe framework, if the input dimension count of an operator is not 4 and the **axis** parameter is available, its value cannot be negative.

### 5.12.2.2 Caffe Operator Boundaries

| N o. | Operator | Description                               | Boundary                                                                                                                                                                                                                                                                                                                         |
|------|----------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Absval   | Computes the absolute value of the input. | [Inputs]<br>One input<br>[Arguments]<br><b>engine</b> : (optional) enum, default to <b>0</b> , CAFFE = 1, CUDNN = 2<br>[Restrictions]<br>None<br>[Quantization tool supporting]<br>Yes                                                                                                                                           |
| 2    | Argmax   | Computes the index of the maximum values. | [Inputs]<br>One input<br>[Arguments]<br><ul style="list-style-type: none"> <li>• <b>out_max_val</b>: (optional) bool, default to <b>false</b></li> <li>• <b>top_k</b>: (optional) unit32, default to <b>1</b></li> <li>• <b>axis</b>: (optional) int32</li> </ul> [Restrictions]<br>None<br>[Quantization tool supporting]<br>No |

| N<br>o. | Operator  | Description                                                    | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------|-----------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3       | BatchNorm | Normalizes the input:<br>variance of $[(x - \text{avg}(x))/x]$ | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>use_global_stats</b>: bool, must be <b>true</b></li> <li>• <b>moving_average_fraction</b>: (optional) float, default to <b>0.999</b></li> <li>• <b>eps</b>: (optional) float, default to <b>1e - 5</b></li> </ul> <p>[Restrictions]<br/>Only the C dimension can be normalized.</p> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                       |
| 4       | Concat    | Concatenates the input along the given dimension.              | <p>[Inputs]<br/>Multiple inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>concat_dim</b>: (optional) uint32, default to <b>1</b>, greater than <b>0</b></li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b>, exclusive with <b>concat_dim</b>. When axis is <b>-1</b>, four input dimensions are required. Otherwise, the result may be incorrect.</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• The number of dimensions of the input tensors must match, and all dimensions except <b>axis</b> must be equal.</li> <li>• The range of the input Tensor count is [1, 1000].</li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator             | Description           | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------|----------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5       | ConvolutionDepthwise | Convolution depthwise | <p>[Inputs]<br/>One input, with a constant filter and four dimensions</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_output</b>: (optional) uint32</li> <li>• <b>bias_term</b>: (optional) bool, default to <b>true</b></li> <li>• <b>pad</b>: uint32, default to <b>0</b>, array</li> <li>• <b>kernel_size</b>: uint32, array</li> <li>• <b>stride</b>: uint32, default to <b>1</b>, array</li> <li>• <b>dilation</b>: uint32, only dilation=1 is supported, array</li> <li>• <b>pad_h</b>: (optional) uint32, default to <b>0</b> (2D only)</li> <li>• <b>pad_w</b>: (optional) uint32, default to <b>0</b> (2D only)</li> <li>• <b>kernel_h</b>: (optional) uint32 (2D only)</li> <li>• <b>kernel_w</b>: (optional) uint32 (2D only)</li> <li>• <b>stride_h</b>: (optional) uint32 (2D only)</li> <li>• <b>stride_w</b>: (optional) uint32 (2D only)</li> <li>• <b>group</b>: (optional) uint32, default to <b>1</b></li> <li>• <b>weight_filler</b>: This parameter is not supported.</li> <li>• <b>bias_filler</b>: This parameter is not supported.</li> <li>• <b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> <li>• <b>force_nd_im2col</b>: (optional) bool, default to <b>false</b></li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b></li> </ul> <p>[Restrictions]<br/>filterN = inputC = group<br/><math>(W + 15)/16 * 16) * \text{filter.W} * 32 \leq 32 * 1024</math>,<br/>where, W is W of the operator input and filter.W is W of the filter.</p> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator    | Description         | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------|-------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6       | Convolution | Convolve the input. | <p>[Inputs]<br/>One input, with a constant filter and four dimensions</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_output</b>: (optional) uint32</li> <li>• <b>bias_term</b>: (optional) bool, default to <b>true</b></li> <li>• <b>pad</b>: uint32, default to <b>0</b>, array</li> <li>• <b>kernel_size</b>: uint32, array</li> <li>• <b>stride</b>: uint32, default to <b>1</b>, array</li> <li>• <b>dilation</b>: uint32, default to <b>1</b>, array</li> <li>• <b>pad_h</b>: (optional) uint32, default to <b>0</b> (2D only)</li> <li>• <b>pad_w</b>: (optional) uint32, default to <b>0</b> (2D only)</li> <li>• <b>kernel_h</b>: (optional) uint32 (2D only)</li> <li>• <b>kernel_w</b>: (optional) uint32 (2D only)</li> <li>• <b>stride_h</b>: (optional) uint32 (2D only)</li> <li>• <b>stride_w</b>: (optional) uint32 (2D only)</li> <li>• <b>group</b>: (optional) uint32, default to <b>1</b></li> <li>• <b>weight_filler</b>: This parameter is not supported.</li> <li>• <b>bias_filler</b>: This parameter is not supported.</li> <li>• <b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> <li>• <b>force_nd_im2col</b>: (optional) bool, default to <b>false</b></li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <math>(inputW + padWHead + padWTail) \geq (((FilterW - 1) * dilationW) + 1)</math></li> <li>• <math>(inputW + padWHead + padWTail) / StrideW + 1 \leq 2147483647</math></li> <li>• <math>(inputH + padHHead + padHTail) \geq (((FilterH - 1) * dilationH) + 1)</math></li> <li>• <math>(inputH + padHHead + padHTail) / StrideH + 1 \leq 2147483647</math></li> <li>• <math>0 \leq Pad &lt; 256, 0 &lt; FilterSize &lt; 256, 0 &lt; Stride &lt; 64, 1 \leq dilationsize &lt; 256</math></li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operato<br>r | Description      | Boundary                                                                                                                                                                                                                                                                                               |
|---------|--------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7       | Crop         | Crops the input. | [Inputs]<br>Two inputs<br>[Arguments] <ul style="list-style-type: none"><li>• <b>axis</b>: (optional) int32, default to 2. When axis is <b>-1</b>, four input dimensions are required.</li><li>• <b>offset</b>: uint32, array</li></ul> [Restrictions]<br>None<br>[Quantization tool supporting]<br>No |

| N<br>o. | Operator      | Description   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------|---------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8       | Deconvolution | Deconvolution | <p>[Inputs]</p> <p>One input, with a constant filter and four dimensions</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_output</b>: (optional) uint32</li> <li>• <b>bias_term</b>: (optional) bool, default to <b>true</b></li> <li>• <b>pad</b>: uint32, default to <b>0</b>, array</li> <li>• <b>kernel_size</b>: uint32, array</li> <li>• <b>stride</b>: uint32, default to <b>1</b>, array</li> <li>• <b>dilation</b>: uint32, default to <b>1</b>, array</li> <li>• <b>pad_h</b>: (optional) uint32, default to <b>0</b> (2D only)</li> <li>• <b>pad_w</b>: (optional) uint32, default to <b>0</b> (2D only)</li> <li>• <b>kernel_h</b>: (optional) uint32 (2D only)</li> <li>• <b>kernel_w</b>: (optional) uint32 (2D only)</li> <li>• <b>stride_h</b>: (optional) uint32 (2D only)</li> <li>• <b>stride_w</b>: (optional) uint32 (2D only)</li> <li>• <b>group</b>: (optional) uint32, default to <b>1</b></li> <li>• <b>weight_filler</b>: This parameter is not supported.</li> <li>• <b>bias_filler</b>: This parameter is not supported.</li> <li>• <b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> <li>• <b>force_nd_im2col</b>: (optional) bool, default to <b>false</b></li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• group = 1</li> <li>• dilation = 1</li> <li>• filterH – padHHead – 1 ≥ 0</li> <li>• filterW – padWHead – 1 ≥ 0</li> <li>• Restrictions involving intermediate variables:</li> </ul> <p>1. <math>a = \text{ALIGN}(\text{filter\_num}, 16) * \text{ALIGN}(\text{filter\_c}, 16) * \text{filter\_h} * \text{filter\_w} * 2</math></p> <p>If <math>\text{ALIGN}(\text{filter\_c}, 16) \% 32 = 0</math>, <math>a = a/2</math></p> <p>2. <math>\text{conv\_input\_width} = (\text{deconvolution input W} - 1) * \text{strideW} + 1</math></p> |

| No. | Operator | Description | Boundary                                                                                                                                                                       |
|-----|----------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |          |             | 3. $b = (\text{conv\_input\_width}) * \text{filter\_h} * \text{ALIGN}(\text{filter\_num}, 16) * 2 * 2$<br>4. $a + b \leq 1024 * 1024$<br>[Quantization tool supporting]<br>Yes |



| No. | Operator        | Description                                  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|-----------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9   | DetectionOutput | Generates detection results and outputs FSR. | <p>[Inputs]<br/>Three inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_classes</b>: (mandatory) int32, indicating the number of classes to be predicted</li> <li>• <b>share_location</b>: (optional) bool, default to <b>true</b>, indicating that classes share one BBox</li> <li>• <b>background_label_id</b>: (optional) int32, default to <b>0</b></li> <li>• <b>nms_param</b>: (optional) indicating non-maximum suppression (NMS)</li> <li>• <b>save_output_param</b>: (optional) indicating whether to save the detection result</li> <li>• <b>code_type</b>: (optional) default to <b>CENTER_SIZE</b></li> <li>• <b>variance_encoded_in_target</b>: (optional) bool, default to <b>true</b>. The value <b>true</b> indicates that the variance is encoded in the target, otherwise the prediction offset needs to be adjusted accordingly.</li> <li>• <b>keep_top_k</b>: (optional) int32, indicating the total number of BBoxes to be reserved for each image after NMS</li> <li>• <b>confidence_threshold</b>: (optional) float, indicating that only the detection whose confidence is above the threshold is considered. If this parameter is not set, all boxes are considered.</li> <li>• <b>nms_threshold</b>: (optional) float</li> <li>• <b>top_k</b>: (optional) int32</li> <li>• <b>boxes</b>: (optional) int32, default to <b>1</b></li> <li>• <b>relative</b>: (optional) bool, default to <b>true</b></li> <li>• <b>objectness_threshold</b>: (optional) float, default to <b>0.5</b></li> <li>• <b>class_threshold</b>: (optional) float, default to <b>0.5</b></li> <li>• <b>biases</b>: array</li> <li>• <b>general_nms_param</b>: (optional)</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• Used for Faster R-CNN</li> <li>• Non-maximum suppression (NMS) ratio <b>nmsThreshold</b> is of range (0, 1).</li> </ul> |

| N<br>o. | Operator | Description                                           | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------|----------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |          |                                                       | <ul style="list-style-type: none"> <li>Probability threshold <b>postConfThreshold</b> is of range (0, 1).</li> <li>At least two classes</li> <li>Input box count <math>\leq 1024</math></li> <li>Output <b>W</b> dimension = 16</li> </ul> [Quantization tool supporting]<br>Yes                                                                                                                                                                                                                                                                                                                |
| 10      | Eltwise  | Compute element-wise operations (PROD, MAX, and SUM). | [Inputs]<br>At least two inputs<br>[Arguments] <ul style="list-style-type: none"> <li><b>operation</b>: (optional) enum, (PROD = 0; SUM = 1; MAX = 2), default to SUM</li> <li><b>coeff</b>: array, float</li> <li><b>stable_prod_grad</b>: (optional) bool, default to <b>true</b></li> </ul> [Restrictions] <ul style="list-style-type: none"> <li>Up to four inputs</li> <li>Compared with the native operator, this operator does not support the <b>stable_prod_grad</b> parameter.</li> <li>PROD, MAX, and SUM operations are supported.</li> </ul> [Quantization tool supporting]<br>Yes |
| 11      | Elu      | Activation function                                   | [Inputs]<br>One input<br>[Arguments]<br><b>alpha</b> : (optional) float, default to <b>1</b><br>[Restrictions]<br>None<br>[Quantization tool supporting]<br>No                                                                                                                                                                                                                                                                                                                                                                                                                                  |

| N<br>o. | Operator | Description                                                         | Boundary                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------|----------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1<br>2  | Exp      | Applies <b>e</b> as the base and <b>x</b> as the exponent.          | [Inputs]<br>One input<br>[Arguments] <ul style="list-style-type: none"> <li>• <b>base</b>: (optional) float, default to <b>-1.0</b></li> <li>• <b>scale</b>: (optional) float, default to <b>1.0</b></li> <li>• <b>shift</b>: (optional) float, default to <b>0.0</b></li> </ul> [Restrictions]<br>None<br>[Quantization tool supporting]<br>No                                                             |
| 1<br>3  | Flatten  | Converts an input $n * c * h * w$ into a vector $n * (c * h * w)$ . | [Inputs]<br>One input<br>$top\_size \neq bottom\_size \neq 1$<br>When <b>axis</b> is <b>-1</b> , four input dimensions are required.<br>[Arguments] <ul style="list-style-type: none"> <li>• <b>axis</b>: (optional) int32, default to <b>1</b></li> <li>• <b>end_axis</b>: (optional) int32, default to <b>-1</b></li> </ul> [Restrictions]<br>$axis < end\_axis$<br>[Quantization tool supporting]<br>Yes |

| N<br>o. | Operator           | Description                   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------|--------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1<br>4  | FullConn<br>ection | Computes an<br>inner product. | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_output</b>: (optional) uint32</li> <li>• <b>bias_term</b>: (optional) bool, default to <b>true</b></li> <li>• <b>weight_filler</b>: This parameter is not supported.</li> <li>• <b>bias_filler</b>: This parameter is not supported.</li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b></li> <li>• <b>transpose</b>: (optional) bool, default to <b>false</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• transpose = false, axis = 1</li> <li>• In the quantization scenario, Bais_C &lt;= 59136; In non-quantified scenarios, Bais_C &lt;= 118272</li> <li>• To quantify the model, the following dimension restrictions must be satisfied:</li> <li>• When N = 1: <math>2 * \text{CEIL}(C, 16) * 16 * xH * xW \leq 1024 * 1024</math>;</li> <li>• When N &gt; 1: <math>2 * 16 * \text{CEIL}(C, 16) * 16 * xH * xW \leq 1024 * 1024</math>.</li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator | Description                                  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------|----------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1<br>5  | Interp   | Interpolation layer                          | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>height</b>: (optional) int32, default to <b>0</b></li> <li>• <b>width</b>: (optional) int32, default to <b>0</b></li> <li>• <b>zoom_factor</b>: (optional) int32, default to <b>1</b></li> <li>• <b>shrink_factor</b>: (optional) int32, default to <b>1</b></li> <li>• <b>pad_beg</b>: (optional) int32, default to <b>0</b></li> <li>• <b>pad_end</b>: (optional) int32, default to <b>0</b></li> </ul> <p>Note:</p> <ul style="list-style-type: none"> <li>• <b>zoom_factor</b> and <b>shrink_factor</b> are exclusive.</li> <li>• <b>height</b> and <b>zoom_factor</b> are exclusive.</li> <li>• <b>height</b> and <b>shrink_factor</b> are exclusive.</li> </ul> <p>[Restrictions]<br/><math>(\text{outputH} * \text{outputW}) / (\text{inputH} * \text{inputW}) &gt; 1/30</math></p> <p>[Quantization tool supporting]<br/>No</p> |
| 1<br>6  | Log      | Performs logarithmic operation on the input. | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>base</b>: (optional) float, default to <b>-1.0</b></li> <li>• <b>scale</b>: (optional) float, default to <b>1.0</b></li> <li>• <b>shift</b>: (optional) float, default to <b>0.0</b></li> </ul> <p>[Restrictions]<br/>None</p> <p>[Quantization tool supporting]<br/>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

| N<br>o. | Operator | Description                             | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------|----------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17      | LRN      | Normalizes the input in a local region. | <p>[Inputs]<br/>One non-constant input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>local_size</b>: (optional) uint32, default to <b>5</b></li> <li>• <b>alpha</b>: (optional) float, default to <b>1</b></li> <li>• <b>beta</b>: (optional) float, default to <b>0.75</b></li> <li>• <b>norm_region</b>: (optional) enum, default to <b>ACROSS_CHANNELS</b> (ACROSS_CHANNELS = 0, WITHIN_CHANNEL = 1)</li> <li>• <b>lrnk</b>: (optional) float, default to <b>1</b></li> <li>• <b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <b>local_size</b> is an odd number greater than 0.</li> <li>• Inter-channel: If <b>local_size</b> is of range [1, 15]: <math>\text{lrnk} &gt; 0.00001</math> and <math>\text{beta} &gt; 0.01</math>; Otherwise, <b>lrnk</b> and <b>beta</b> are any values. <b>lrnk</b> and <b>alpha</b> are not <b>0</b> at the same time. When the <b>C</b> dimension is greater than <b>1776</b>, <math>\text{local\_size} &lt; 1728</math>.</li> <li>• Intra-channel: <math>\text{lrnk} = 1</math>, <b>local_size</b> is of range [1, 15], <math>\text{beta} &gt; 0.01</math></li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator | Description                               | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------|----------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 18      | LSTM     | Long and short term memory network (LSTM) | <p>[Inputs]</p> <p>Two or three inputs</p> <ul style="list-style-type: none"> <li>• <b>X</b>: time sequence data (<math>T * B * X_t</math>). According to the NCHW format of 4D, ensure that the following conditions are met: <b>N</b> is the time sequence length <b>T</b>, <b>C</b> is the batch number <b>B</b>, <b>H</b> is the input data <math>X_t</math> at the <b>t</b> moment, and <b>W</b> is 1.</li> <li>• <b>Cont</b>: sequence continuity flag (<math>T * B</math>)</li> <li>• <b>Xs</b>: (optional) static data (<math>B * X_t</math>)</li> </ul> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_output</b>: (optional) uint32, default to <b>0</b></li> <li>• <b>weight_filler</b>: This parameter is not supported.</li> <li>• <b>bias_filler</b>: This parameter is not supported.</li> <li>• <b>debug_info</b>: (optional) bool, default to <b>false</b></li> <li>• <b>expose_hidden</b>: (optional) bool, default to <b>false</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• Restrictions involving intermediate variables, <b>ht</b> and <b>output</b> are the argument <b>num_output</b>:</li> </ul> $a = (\text{ALIGN}(xt, 16) + \text{ALIGN}(\text{output}, 16)) * 16 * 2 * 2$ $b = (\text{ALIGN}(xt, 16) + \text{ALIGN}(\text{output}, 16)) * 16 * 4 * 2 * 2$ $d = 16 * \text{ALIGN}(ht, 16) * 2$ $e = B * 4$ <p>That is:</p> $a + b \leq 1024 * 1024$ $d \leq 256 * 1024 / 8$ $e \leq 256 * 1024 / 32$ <ul style="list-style-type: none"> <li>• <math>B \leq 16, T \leq 768</math></li> </ul> <p>[Quantization tool supporting]</p> <p>No</p> |

| N<br>o. | Operator  | Description                                              | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------|-----------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 19      | Normalize | Normalization layer                                      | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>across_spatial</b>: (optional) bool, default to <b>true</b></li> <li>• <b>scale_filler</b>: This parameter is not supported.</li> <li>• <b>channel_shared</b>: (optional) bool, default to <b>true</b></li> <li>• <b>eps</b>: (optional) float, default to <b>1e - 10</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <math>1e - 7 &lt; \text{eps} \leq 0.1 + (1e - 6)</math></li> <li>• <b>across_spatial</b> must be <b>true</b> for Caffe, indicating normalization by channel</li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |
| 20      | Permute   | Permutes the input dimensions according to a given mode. | <p>[Inputs]<br/>One input</p> <p>[Arguments]<br/><b>order</b>: uint32, array</p> <p>[Restrictions]<br/>None</p> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |



| N<br>o. | Operator | Description      | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------|----------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 21      | Pooling  | Pools the input. | <p>[Inputs]</p> <p>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>pool</b>: (optional) enum, indicating the pooling method, MAX = 0, AVE = 1, and STOCHASTIC = 2, default to <b>MAX</b></li> <li>• <b>pad</b>: (optional) uint32, default to <b>0</b></li> <li>• <b>pad_h</b>: (optional) uint32, default to <b>0</b></li> <li>• <b>pad_w</b>: (optional) uint32, default to <b>0</b></li> <li>• <b>kernel_size</b>: (optional) uint32, exclusive with <b>kernel_h/kernel_w</b></li> <li>• <b>kernel_h</b>: (optional) uint32</li> <li>• <b>kernel_w</b>: (optional) uint32, used in pair with <b>kernel_h</b></li> <li>• <b>stride</b>: (optional) uint32, default to <b>1</b></li> <li>• <b>stride_h</b>: (optional) uint32</li> <li>• <b>stride_w</b>: (optional) uint32</li> <li>• <b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> <li>• <b>global_pooling</b>: (optional) bool, default to <b>false</b></li> <li>• <b>ceil_mode</b>: (optional) bool, default to <b>true</b></li> <li>• <b>round_mode</b>: (optional) enum, CEIL = 0, FLOOR = 1, default to <b>CEIL</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <math>\text{kernelH} \leq \text{inputH} + \text{padTop} + \text{padBottom}</math></li> <li>• <math>\text{kernelW} \leq \text{inputW} + \text{padLeft} + \text{padRight}</math></li> <li>• <math>\text{padTop} &lt; \text{windowH}</math></li> <li>• <math>\text{padBottom} &lt; \text{windowH}</math></li> <li>• <math>\text{padLeft} &lt; \text{windowW}</math></li> <li>• <math>\text{padRight} &lt; \text{windowW}</math></li> <li>• Only the global pooling mode is supported. The following restrictions must be satisfied:<br/>1) <math>\text{outputH} == 1 \ \&amp;\&amp; \ \text{outputW} == 1 \ \&amp;\&amp; \ \text{kernelH} \geq \text{inputH} \ \&amp;\&amp; \ \text{kernelW} \geq \text{inputW}</math><br/>2) <math>\text{inputH} * \text{inputW} \leq 10000</math></li> </ul> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| N<br>o. | Operator | Description                                            | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------|----------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2<br>2  | Power    | $y = (\text{scale} * x + \text{shift})^{\text{power}}$ | [Inputs]<br>One input<br>[Arguments] <ul style="list-style-type: none"> <li>• <b>power</b>: (optional) float, default to <b>1.0</b></li> <li>• <b>scale</b>: (optional) float, default to <b>1.0</b></li> <li>• <b>shift</b>: (optional) float, default to <b>0.0</b></li> </ul> [Restrictions] <ul style="list-style-type: none"> <li>• <math>\text{power} \neq 1</math></li> <li>• <math>\text{scale} * x + \text{shift} &gt; 0</math></li> </ul> [Quantization tool supporting]<br>Yes |
| 2<br>3  | Prelu    | Activation function                                    | [Inputs]<br>One input<br>[Arguments] <ul style="list-style-type: none"> <li>• <b>filler</b>: This parameter is not supported.</li> <li>• <b>channel_shared</b>: (optional) bool, indicating whether to share slope parameters across channels, default to <b>false</b></li> </ul> [Restrictions]<br>None<br>[Quantization tool supporting]<br>Yes                                                                                                                                         |

| N<br>o. | Operator | Description                                                     | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------|----------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2<br>4  | PriorBox | Obtains the real location of the target from the box proposals. | <p>[Inputs]</p> <p>Two inputs:</p> <ul style="list-style-type: none"> <li>• <b>Original input image of model</b>, data format: NCHW;</li> <li>• <b>FeatureMap</b>, data format: NCHW.</li> </ul> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>min_size</b>: (mandatory) indicating the minimum frame size (in pixels)</li> <li>• <b>max_size</b>: (mandatory) indicating the maximum frame size (in pixels)</li> <li>• <b>aspect_ratio</b>: array, float. A repeated ratio is ignored. If no aspect ratio is provided, the default ratio <b>1</b> is used.</li> <li>• <b>flip</b>: (optional) bool, default to true. The value true indicates that each aspect ratio is reversed. For example, for aspect ratio <i>r</i>, the aspect ratio <b>1.0/<i>r</i></b> is generated.</li> <li>• <b>clip</b>: (optional) bool, default to <b>false</b>. The value true indicates that the previous value is clipped to the range [0, 1].</li> <li>• <b>variance</b>: array, used to adjust the variance of the BBoxes</li> <li>• <b>img_size</b>: (optional) uint32. exclusive with <b>img_h/img_w</b></li> <li>• <b>img_h</b>: (optional) uint32</li> <li>• <b>img_w</b>: (optional) uint32</li> <li>• <b>step</b>: (optional) float. <b>step_h</b> and <b>step_w</b> are exclusive.</li> <li>• <b>step_h</b>: (optional) float</li> <li>• <b>step_w</b>: (optional) float</li> <li>• <b>offset</b>: (optional) float, default to <b>0.5</b></li> </ul> <p>[Restrictions]</p> <p>Used for the SSD network only</p> <p>Output dimensions: [n, 2, detection frame * 4, 1]</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Operator | Description                                                                                    | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|----------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 25  | Proposal | Sorts the box proposals by (proposal, score) and obtains the top N proposals by using the NMS. | <p>[Inputs]<br/>Three inputs: scores, bbox_pred, im_info</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>feat_stride</b>: (optional) float</li> <li>• <b>base_size</b>: (optional) float</li> <li>• <b>min_size</b>: (optional) float</li> <li>• <b>ratio</b>: array (optional), float</li> <li>• <b>scale</b>: array (optional), float</li> <li>• <b>pre_nms_topn</b>: (optional) int32</li> <li>• <b>post_nms_topn</b>: (optional) int32</li> <li>• <b>nms_thresh</b>: (optional) float</li> </ul> <p>[Restrictions]</p> <p>Used only for Faster R-CNN</p> <ul style="list-style-type: none"> <li>• <b>ProposalParameter</b> and <b>PythonParameter</b> are exclusive. <ol style="list-style-type: none"> <li>1. Value range of <b>preTopK</b>: 1–6144</li> <li>2. Value range of <b>postTopK</b>: 1–1024</li> <li>3. <math>scaleCnt * ratioCnt \leq 64</math></li> <li>4. <math>0 &lt; nmsTresh \leq 1</math> (threshold for box filtering)</li> <li>5. <b>minSize</b>: minimum edge length of a proposal. A box with any side smaller than <b>minSize</b> is removed.</li> <li>6. <b>featStride</b>: H/W stride between the two adjacent boxes used in default box generation</li> <li>7. <b>baseSize</b>: base box size used in default box generation</li> <li>8. <b>ratio</b> and <b>scale</b>: used in default box generation</li> <li>9. <b>imgH</b> and <b>imgW</b>: height and width of the image input to the network. The values must be greater than 0.</li> </ol> </li> <li>• Restrictions on the input dimensions: <p>clsProb: <math>C = 2 * scaleCnt * ratioCnt</math></p> <p>bboxPred: <math>C = 4 * scaleCnt * ratioCnt</math></p> <p>bboxPrior: <math>N = clsProb.N, C = 4 * scaleCnt * ratioCnt</math></p> <p><b>imInfo</b>: <math>N = clsProb.N, C = 3</math></p> </li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator      | Description                                                   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------|---------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 26      | PSROI Pooling | Position-sensitive region-of-interest pooling (PSROI Pooling) | <p>[Inputs]<br/>Two inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>spatial_scale</b>: (mandatory) float</li> <li>• <b>output_dim</b>: (mandatory) int32, indicating the number of output channels</li> <li>• <b>group_size</b>: (mandatory) int32, indicating the number of groups to encode position-sensitive score maps</li> </ul> <p>[Restrictions]</p> <p>Used for the Region-based Fully Convolutional Network (R-FCN)</p> <ul style="list-style-type: none"> <li>• ROI coordinates [roiN, roiC, roiH, roiW]: <math>1 \leq \text{roiN} \leq 65535</math>, <math>\text{roiC} == 5</math>, <math>\text{roiH} == 1</math>, <math>\text{roiW} == 1</math></li> <li>• Dimensions of the input feature map: [xN, xC, xH, xW]</li> <li>• <math>\text{pooledH} == \text{pooledW} == \text{groupSize} \leq 128</math></li> </ul> <p><b>pooledH</b> and <b>pooledW</b> indicate the length and width of the pooled ROI.</p> <p>Output format: y [yN, yC, yH, yW]</p> <ul style="list-style-type: none"> <li>• <math>\text{poolingMode} == \text{avg pooling}</math>, <math>\text{pooledH} == \text{pooledW} == \text{groupSize}</math>, <math>\text{pooledH} \leq 128</math>, <math>\text{spatialScale} &gt; 0</math>, <math>\text{groupSize} &gt; 0</math>, <math>\text{outputDim} &gt; 0</math></li> <li>• <math>1 \leq \text{xN} \leq 65535</math>, <math>\text{roisN} \% \text{xN} == 0</math></li> <li>• <b>HW_LIMIT</b> is the limit of <b>xH</b> and <b>xW</b>.</li> </ul> <p><math>\text{xHW} = \text{xH} * \text{xW}</math><br/> <math>\text{pooledHW} = \text{pooledH} * \text{pooledW}</math><br/> <math>\text{HW\_LIMIT} = (64 * 1024 - 8 * 1024) / 32</math>,<br/> <math>\text{xH} \geq \text{pooledH}</math>, <math>\text{xW} \geq \text{pooledW}</math><br/> <math>\text{xHW} \geq \text{pooledHW}</math><br/> <math>\text{xHW} / \text{pooledHW} \leq \text{HW\_LIMIT}</math></p> <ul style="list-style-type: none"> <li>• In multi-batch scenarios, the ROIs are allocated equally to the batches. In addition, the batch sequence of the ROIs is the same as the feature.</li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator | Description                                                                                     | Boundary                                                                                                                                                                                                                                                                                                                                                |
|---------|----------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 27      | Relu     | Activation function, including common ReLU and Leaky ReLU, which can be specified by parameters | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>negative_slope</b>: (optional) float, default to <b>0</b></li> <li>• <b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Quantization tool supporting]<br/>Yes</p>                     |
| 28      | Reshape  | Reshapes the input.                                                                             | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>shape</b>: constant, int64 or int</li> <li>• <b>axis</b>: (optional) int32, default to <b>0</b></li> <li>• <b>num_axes</b>: (optional) int32, default to <b>-1</b></li> </ul> <p>[Restrictions]<br/>None</p> <p>[Quantization tool supporting]<br/>Yes</p> |

| N<br>o. | Operator   | Description                          | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------|------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 29      | ROIAlign   | Aggregates features using ROIs.      | <p>[Inputs]<br/>At least two inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>pooled_h</b>: (optional) uint32, default to 0</li> <li>• <b>pooled_w</b>: (optional) uint32, default to 0</li> <li>• <b>spatial_scale</b>: (optional) float, default to 1</li> <li>• <b>sampling_ratio</b>: (optional) int32, default to -1</li> </ul> <p>[Restrictions]<br/>Mainly used for Mask R-CNN<br/>Restrictions on the feature map:</p> <ul style="list-style-type: none"> <li>• <math>H * W \leq 5248</math> or <math>W * C &lt; 40960</math></li> <li>• <math>C \leq 1280</math></li> <li>• <math>((C - 1)/128 + 1) * \text{pooledW} \leq 216</math></li> </ul> <p>Restrictions on the ROI:</p> <ul style="list-style-type: none"> <li>• <math>C = 5</math> (Caffe), <math>H = 1</math>, <math>W = 1</math></li> <li>• <math>\text{samplingRatio} * \text{pooledW} \leq 128</math>,<br/><math>\text{samplingRatio} * \text{pooledH} \leq 128</math></li> <li>• <math>H \geq \text{pooledH}</math>, <math>W \geq \text{pooledW}</math></li> </ul> <p>[Quantization tool supporting]<br/>Yes</p> |
| 30      | ROIPooling | Maps ROI proposals to a feature map. | <p>[Inputs]<br/>At least two inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>pooled_h</b>: (mandatory) uint32, default to 0</li> <li>• <b>pooled_w</b>: (mandatory) uint32, default to 0</li> <li>• <b>spatial_scale</b>: (mandatory) float, default to 1. The multiplication spatial scale factor is used to convert ROI coordinates from the input scale to the pool scale.</li> </ul> <p>[Restrictions]<br/>Mainly used for Faster R-CNN</p> <ul style="list-style-type: none"> <li>• Input dimensions: <math>H * W \leq 8160</math>, <math>H \leq 120</math>, <math>W \leq 120</math></li> <li>• Output dimensions: <math>\text{pooledH} \leq 20</math>, <math>\text{pooledW} \leq 20</math></li> </ul> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                                                                              |

| N<br>o. | Operator       | Description                                                  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------|----------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3<br>1  | Scale          | out =<br>alpha*Input<br>+beta                                | <p>[Inputs]<br/>Two inputs, each with four dimensions</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>axis</b>: (optional) int32, <b>1</b> (default) or <b>-3</b></li> <li>• <b>num_axes</b>: (optional) int32, default to <b>1</b></li> <li>• <b>filler</b>: This parameter is not supported.</li> <li>• <b>bias_term</b>: (optional) bool, default to <b>false</b>, indicating whether to learn a bias (equivalent to ScaleLayer + BiasLayer, but may be more efficient).</li> <li>• <b>bias_filler</b>: This parameter is not supported.</li> </ul> <p>[Restrictions]<br/><b>shape</b> of <b>scale</b> and <b>bias</b>: (n, c, 1, 1), with the <b>C</b> dimension equal to that of the input</p> <p>[Quantization tool supporting]<br/>Yes</p> |
| 3<br>2  | ShuffleChannel | Shuffles<br>information<br>cross the<br>feature<br>channels. | <p>[Inputs]<br/>One input</p> <p>[Arguments]<br/><b>group</b>: (optional) uint32, default to <b>1</b></p> <p>[Restrictions]<br/>None</p> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 3<br>3  | Sigmoid        | Activation<br>function                                       | <p>[Inputs]<br/>One input</p> <p>[Arguments]<br/><b>engine</b>: (optional) enum, default to <b>0</b>, CAFFE = 1, CUDNN = 2</p> <p>[Restrictions]<br/>None</p> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |



| N<br>o. | Operator | Description                            | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------|----------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3<br>4  | Slice    | Slices an input into multiple outputs. | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>slice_dim</b>: (optional) uint32, default to <b>1</b>, exclusive with <b>axis</b></li> <li>• <b>slice_point</b>: array, uint32</li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b>, indicating concatenation along the channel dimension</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>No restrictions</p> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 3<br>5  | Softmax  | Normalization logic function           | <p>[Inputs]<br/>One input</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>engine</b>: (optional) default to <b>0</b>, CAFFE = 1, CUDNN = 2</li> <li>• <b>axis</b>: (optional) int32, default to <b>1</b>, indicating the axis along which softmax is performed</li> </ul> <p>[Restrictions]<br/>If the input contains four dimensions, softmax is performed on each of them.<br/>According to <b>axis</b>:</p> <ul style="list-style-type: none"> <li>• When axis = 1: <math>C \leq ((256 * 1024/4) - 8 * 1024 - 256)/2</math></li> <li>• When axis = 0: <math>N \leq (56 * 1024 - 256)/2</math></li> <li>• When axis = 2: <math>W = 1, 0 &lt; H &lt; (1024 * 1024/32)</math></li> <li>• When axis = 3: <math>0 &lt; W &lt; (1024 * 1024/32)</math></li> </ul> <p>If the input contains fewer than four dimensions, softmax is performed only on the last dimension, with the last dimension <math>\leq 46080</math>.</p> <p>[Quantization tool supporting]<br/>Yes</p> |

| No. | Operator | Description                         | Boundary                                                                                                                                                                                                                                        |
|-----|----------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 36  | Tanh     | Activation function                 | [Inputs]<br>One input<br>[Arguments]<br><b>engine:</b> (optional) enum, default to <b>0</b> , CAFFE = 1, CUDNN = 2<br>[Restrictions]<br>The number of tensor elements cannot exceed <b>INT32_MAX</b> .<br>[Quantization tool supporting]<br>Yes |
| 37  | Upsample | Backward propagation of max pooling | [Inputs]<br>Two inputs<br>[Arguments]<br><b>scale:</b> (optional) int32, default to <b>1</b><br>[Restrictions]<br>None<br>[Quantization tool supporting]<br>Yes                                                                                 |

| No. | Operator           | Description                  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|--------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 38  | SSDDetectionOutput | SSD network detection output | <p>[Inputs]</p> <p>Three inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_classes</b>: (mandatory) int32, indicating the number of classes to be predicted</li> <li>• <b>share_location</b>: (optional) bool, default to <b>true</b>, indicating that classes share one BBox</li> <li>• <b>background_label_id</b>: (optional) int32, default to <b>0</b></li> <li>• <b>nms_param</b>: (optional) indicating non-maximum suppression (NMS)</li> <li>• <b>save_output_param</b>: (optional) indicating whether to save the detection result</li> <li>• <b>code_type</b>: (optional) default to <b>CENTER_SIZE</b></li> <li>• <b>variance_encoded_in_target</b>: (optional) bool, default to <b>true</b>. The value <b>true</b> indicates that the variance is encoded in the target, otherwise the prediction offset needs to be adjusted accordingly.</li> <li>• <b>keep_top_k</b>: (optional) int32, indicating the total number of BBoxes to be reserved for each image after NMS</li> <li>• <b>confidence_threshold</b>: (optional) float, indicating that only the detection whose confidence is above the threshold is considered. If this parameter is not set, all boxes are considered.</li> <li>• <b>nms_threshold</b>: (optional) float</li> <li>• <b>top_k</b>: (optional) int32</li> <li>• <b>boxes</b>: (optional) int32, default to <b>1</b></li> <li>• <b>relative</b>: (optional) bool, default to <b>true</b></li> <li>• <b>objectness_threshold</b>: (optional) float, default to <b>0.5</b></li> <li>• <b>class_threshold</b>: (optional) float, default to <b>0.5</b></li> <li>• <b>biases</b>: array</li> <li>• <b>general_nms_param</b>: (optional)</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• Used for the SSD network</li> <li>• Value range of <b>preTopK</b> and <b>postTopK</b>: 1–1024</li> </ul> |

| N<br>o. | Operator  | Description                   | Boundary                                                                                                                                                                                                                                                                                              |
|---------|-----------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |           |                               | <ul style="list-style-type: none"> <li>• shareLocation = true</li> <li>• nmsEta = 1</li> <li>• Value range of <b>numClasses</b>: 1–2048</li> <li>• code_type = CENTER_SIZE</li> <li>• Value range of nms_threshold and confidence_threshold: 0.0–1.0</li> </ul> [Quantization tool supporting]<br>Yes |
| 39      | Reorg     | Real-time object detection    | [Inputs]<br>One input<br>[Arguments]<br><ul style="list-style-type: none"> <li>• <b>stride</b>: (optional) uint32, default to 2</li> <li>• <b>reverse</b>: (optional) bool, default to <b>false</b></li> </ul> [Restrictions]<br>Used only for YOLOv2<br>[Quantization tool supporting]<br>No         |
| 40      | Reverse   | Reversion                     | [Inputs]<br>One input<br>[Arguments]<br><b>axis</b> : (optional) int32, default to 1. Controls the axis to be reversed. The content layout will not be reversed.<br>[Restrictions]<br>None<br>[Quantization tool supporting]<br>No                                                                    |
| 41      | LeakyRelu | LeakyRelu activation function | [Inputs]<br>One input<br>[Arguments]<br>Same as Relu<br>[Restrictions]<br>None<br>[Quantization tool supporting]<br>Yes                                                                                                                                                                               |

| No. | Operator            | Description                   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|---------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 42  | YOLODetectionOutput | YOLO network detection output | <p>[Inputs]</p> <p>Four inputs</p> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>num_classes</b>: (mandatory) int32, indicating the number of classes to be predicted</li> <li>• <b>share_location</b>: (optional) bool, default to <b>true</b>, indicating that classes share one BBox</li> <li>• <b>background_label_id</b>: (optional) int32, default to <b>0</b></li> <li>• <b>nms_param</b>: (optional) indicating non-maximum suppression (NMS)</li> <li>• <b>save_output_param</b>: (optional) indicating whether to save the detection result</li> <li>• <b>code_type</b>: (optional) default to <b>CENTER_SIZE</b></li> <li>• <b>variance_encoded_in_target</b>: (optional) bool, default to <b>true</b>. The value <b>true</b> indicates that the variance is encoded in the target, otherwise the prediction offset needs to be adjusted accordingly.</li> <li>• <b>keep_top_k</b>: (optional) int32, indicating the total number of BBoxes to be reserved for each image after NMS</li> <li>• <b>confidence_threshold</b>: (optional) float, indicating that only the detection whose confidence is above the threshold is considered. If this parameter is not set, all boxes are considered.</li> <li>• <b>nms_threshold</b>: (optional) float</li> <li>• <b>top_k</b>: (optional) int32</li> <li>• <b>boxes</b>: (optional) int32, default to <b>1</b></li> <li>• <b>relative</b>: (optional) bool, default to <b>true</b></li> <li>• <b>objectness_threshold</b>: (optional) float, default to <b>0.5</b></li> <li>• <b>class_threshold</b>: (optional) float, default to <b>0.5</b></li> <li>• <b>biases</b>: array</li> <li>• <b>general_nms_param</b>: (optional)</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• Used only for YOLOv2</li> <li>• classNUM &lt; 10240, anchorBox &lt; 5</li> <li>• <math>W \leq 1536</math></li> </ul> |

| N<br>o. | Operato<br>r | Description | Boundary                                                                                                                                                                            |
|---------|--------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |              |             | <ul style="list-style-type: none"><li>• The upper layer of <b>yolodetectionoutput</b> must be the <b>yolo</b><b>region</b> operator.</li></ul> [Quantization tool supporting]<br>No |

### 5.12.2.3 TensorFlow Operator Boundaries

| No. | Python API     | C++ API                     | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|----------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | tf.nn.avg_pool | AvgPool<br>Type:<br>Pooling | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>value</b>: 4D tensor of float32 type, with shape [batch, height, width, channels]</li> <li>• <b>ksize</b>: list or tuple of four integers, each value corresponding to the window size for each dimension of the input tensor.</li> <li>• <b>strides</b>: list or tuple of four integers, each value corresponding to the stride of the sliding window for each dimension of the input tensor</li> <li>• <b>padding</b>: string, either <b>VALID</b> or <b>SAME</b></li> <li>• <b>data_format</b>: string, either <b>NHWC</b> (default) or <b>NCHW</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <math>\text{kernelH} \leq \text{inputH} + \text{padTop} + \text{padBottom}</math></li> <li>• <math>\text{kernelW} \leq \text{inputW} + \text{padLeft} + \text{padRight}</math></li> <li>• <math>\text{padTop} &lt; \text{windowH}</math></li> <li>• <math>\text{padBottom} &lt; \text{windowH}</math></li> <li>• <math>\text{padLeft} &lt; \text{windowW}</math></li> <li>• <math>\text{padRight} &lt; \text{windowW}</math></li> <li>• Only the global pooling mode is supported. The following restrictions must be satisfied:<br/>1) <math>\text{outputH} == 1 \ \&amp;\&amp; \ \text{outputW} == 1 \ \&amp;\&amp; \ \text{kernelH} \geq \text{inputH} \ \&amp;\&amp; \ \text{kernelW} \geq \text{inputW}</math><br/>2) <math>\text{inputH} * \text{inputW} \leq 10000</math></li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API          | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|---------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   | tf.math.reduce_mean | Mean    | <p>[Arguments]<br/>Same as <b>tf.nn.avg_pool</b></p> <p>[Restrictions]<br/>The Mean operator adapts to the AvgPool operator. The restrictions are as follows:</p> <ul style="list-style-type: none"> <li>• keep_dims=true. If keep_dims=false, the subsequent operators must be Reshape.</li> <li>• The average value can be calculated only for the HW dimension, that is, when format=NCHW, axis = [2,3]. When format=NHWC, axis=[1,2].</li> </ul> <p>[Returns]<br/>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]<br/>Yes</p> |
| 3   | tf.nn.max_pool      | MaxPool | Same as <b>tf.nn.avg_pool</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |



| No. | Python API   | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|--------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4   | tf.nn.conv2d | Conv2D  | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>value</b>: 4D tensor of float32 type, with shape [batch, height, width, channels]</li> <li>• <b>filter</b>: constant Tensor, with same data type and dimensions as <b>value</b>, with shape [filter_height, filter_width, in_channels, out_channels]</li> <li>• <b>strides</b>: non-null list or tuple of four integers, each value corresponding to the stride of the sliding window for each dimension of the input tensor</li> <li>• <b>padding</b>: non-null string, either <b>VALID</b> or <b>SAME</b></li> <li>• <b>use_cudnn_on_gpu</b>: bool, default to <b>True</b></li> <li>• <b>data_format</b>: non-null, string, either <b>NHWC</b> (default) or <b>NCHW</b></li> <li>• <b>dilations</b>: (optional) list of four integers, default to [1,1,1,1], each value corresponding to a dimension. If <math>k &gt; 1</math>, <math>k - 1</math> units are skipped at the corresponding dimension in filtering. The dimension sequence is determined by <b>data_format</b>. The values of <b>batch</b> and <b>depth</b> of <b>dilations</b> must be 1.</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <math>(inputW + padWHead + padWTail) \geq (((FilterW - 1) * dilationW) + 1)</math></li> <li>• <math>(inputW + padWHead + padWTail)/StrideW + 1 \leq INT32\_MAX</math></li> <li>• <math>(inputH + padHHead + padHTail) \geq (((FilterH - 1) * dilationH) + 1)</math></li> <li>• <math>(inputH + padHHead + padHTail)/StrideH + 1 \leq INT32\_MAX</math></li> <li>• <math>0 \leq Pad &lt; 256, 0 &lt; FilterSize &lt; 256, 0 &lt; Stride &lt; 64, 1 \leq dilationsize &lt; 256</math></li> </ul> <p>[Returns]<br/>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]<br/>Yes</p> |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5   | tf.concat  | Concat  | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>values</b>: list of Tensor objects or a single Tensor. The values of dimensions must be the same except the dimensions to be concatenated.</li> <li>• <b>axis</b>: 0D Tensor of type int32, specifying the dimension to be concatenated. The value range is <math>[-\text{rank}(\text{values}), \text{rank}(\text{values})]</math>. As in Python, indexing for <b>axis</b> is 0-based. Positive <b>axis</b> in the range <math>[0, \text{rank}(\text{values}))</math> refers to <b>axis</b>-th dimension, while negative <b>axis</b> refers to <math>[\text{axis} + \text{rank}(\text{values})]</math>-th dimension.</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• The number of dimensions of the input tensors must match, and all dimensions except <b>axis</b> must be equal.</li> </ul> <p>The range of the input Tensor count is <math>[1, 1000]</math>.</p> <p>[Returns]</p> <p>Tensor, resulting from concatenation of the input Tensors</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----|------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6   | tf.matmul  | MatMul  | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>a</b>: non-constant Tensor of type float32, rank <math>\geq 2</math></li> <li>• <b>b</b>: constant Tensor with the same data type and rank as <b>a</b></li> <li>• <b>transpose_a</b>: The value <b>true</b> indicates that <b>a</b> is transposed before multiplication.</li> <li>• <b>transpose_b</b>: The value <b>true</b> indicates that <b>b</b> is transposed before multiplication. If <b>transpose_a</b> is <b>false</b>, <b>transpose_b</b> is also <b>false</b>.</li> <li>• <b>adjoint_a</b>: The value <b>true</b> indicates that <b>a</b> is conjugated and transposed before multiplication.</li> <li>• <b>adjoint_b</b>: The value <b>true</b> indicates that <b>b</b> is conjugated and transposed before multiplication.</li> <li>• <b>a_is_sparse</b>: The value <b>true</b> indicates that <b>a</b> is treated as a sparse matrix.</li> <li>• <b>b_is_sparse</b>: The value <b>true</b> indicates that <b>b</b> is treated as a sparse matrix.</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• The transposing property of weight is <b>false</b>.</li> <li>• The multiplication of two Tensors is not supported. Only one Tensor by one constant is supported.</li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>a</b> and <b>b</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API             | C++ API        | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|------------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | tf.nn.fused_batch_norm | FusedBatchNorm | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: 4D Tensor of type float32</li> <li>• <b>scale</b>: 1D Tensor for scaling</li> <li>• <b>offset</b>: 1D Tensor for bias</li> <li>• <b>mean</b>: 1D Tensor for population mean used for inference</li> <li>• <b>variance</b>: 1D Tensor for population variance used for inference</li> <li>• <b>epsilon</b>: small float number added to the variance of <b>x</b></li> <li>• <b>data_format</b>: data format for <b>x</b>, either <b>NHWC</b> (default) or <b>NCHW</b></li> <li>• <b>is_training</b>: bool, specifying whether the operation is used for training or inference</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>The shape of <b>scale</b>, <b>bias</b>, <b>mean</b>, and <b>var</b> must be (1, C, 1, 1), with the same <b>C</b> dimension as <b>input</b>.</p> <p>[Returns]</p> <ul style="list-style-type: none"> <li>• <b>y</b>: 4D Tensor for the normalized, scaled, offset <b>x</b></li> <li>• <b>batch_mean</b>: 1D Tensor for the mean of <b>x</b></li> <li>• <b>batch_var</b>: 1D Tensor for the variance of <b>x</b></li> </ul> <p>[Quantization tool supporting]</p> <p>No</p> |
| 8   | tf.abs                 | Abs            | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor or SparseTensor of type float32</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Returns the absolute value of <b>x</b>, Tensor or SparseTensor. The size and type are the same as those of <b>x</b>.</p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

| No. | Python API                       | C++ API               | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|----------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9   | tf.image.resize_nearest_neighbor | ResizeNearestNeighbor | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>images:</b> 4D Tensor of type float32, with shape [batch, height, width, channels] or 3D Tensor of type float32 with shape [height, width, channels]</li> <li>• <b>size:</b> 1D 2-element constant Tensor, indicating the new size for the images</li> <li>• <b>method:</b><br/><b>ResizeMethod.NEARESTNEIGHBOR</b></li> <li>• <b>align_corners:</b> bool, default to <b>False</b>. The value true indicates that the centers of the 4 corner pixels of the input and output tensors are aligned, preserving the values at the corner pixels.</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of type float, with the identical shape as the <b>input</b></p> <p>[Quantization tool supporting]<br/>No</p>                     |
| 10  | tf.image.resize_bilinear         | ResizeBilinear        | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>images:</b> 4D non-constant Tensor with shape [batch, height, width, channels] of type float32</li> <li>• <b>size:</b> 1D 2-element constant Tensor, indicating the new size for the images</li> <li>• <b>method:</b> <b>ResizeMethod.BILINEAR</b></li> <li>• <b>align_corners:</b> bool, default to <b>False</b>. The value true indicates that the centers of the 4 corner pixels of the input and output tensors are aligned, preserving the values at the corner pixels.</li> </ul> <p>[Restrictions]<br/><math>(\text{outputH} * \text{outputW}) / (\text{inputH} * \text{inputW}) &gt; 1/7</math></p> <p>[Returns]<br/>Tensor of type float, with the identical shape as the <b>input</b></p> <p>[Quantization tool supporting]<br/>No</p> |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11  | tf.cast    | Cast    | <p>[Inputs]</p> <ul style="list-style-type: none"> <li>Data type: float32, int32, bool, int64, int16, int8, uint8, uint16, double</li> </ul> <p>[Arguments]</p> <ul style="list-style-type: none"> <li><b>x</b>: Tensor, SparseTensor, or IndexedSlices</li> <li><b>dtype</b>: destination type, same as the data type of <b>x</b></li> <li><b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor, SparseTensor, or IndexedSlices, same <b>dtype</b> and <b>shape</b> as the input</p> <p>[Quantization tool supporting]</p> <p>N/A</p> |

| No. | Python API             | C++ API               | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12  | tf.nn.depthwise_conv2d | DepthwiseConv2dNative | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: 4D</li> <li>• <b>filter</b>: 4D constant, with shape [filter_height, filter_width, in_channels, channel_multiplier]</li> <li>• <b>strides</b>: non-null list of four integers, each value corresponding to the stride of the sliding window for each dimension of the input tensor</li> <li>• <b>padding</b>: string, either <b>VALID</b> or <b>SAME</b></li> <li>• <b>rate</b>: 1D of size 2. The dilation rate in which we sample input values across the height and width dimensions in atrous convolution. If it is greater than 1, then all values of strides must be 1.</li> <li>• <b>data_format</b>: data format for input, either <b>NHWC</b> (default) or <b>NCHW</b></li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p><math>(W + 15)/16 * 16 * \text{filter.W} * 32 \leq 32 * 1024</math>, where, W is W of the operator input and filter.W is W of the filter.</p> <p>[Returns]</p> <p>4D Tensor, with shape according to <b>data_format</b>. For example, for format NHWC, shape = [batch, out_height, out_width, in_channels * channel_multiplier] for the NHWC format</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 13  | tf.reshape             | Reshape               | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>tensor</b>: Tensor</li> <li>• <b>shape</b>: output shape, constant Tensor of type int64 or int</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

| No. | Python API     | C++ API    | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----|----------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 14  | tf.squeeze     | Squeeze    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: non-constant Tensor</li> <li>• <b>axis</b>: list of ints, specifying the dimensions to be squeezed, default to []. It is an error to squeeze a dimension that is not <b>1</b>.</li> <li>• <b>name</b>: (optional) operation name</li> <li>• <b>squeeze_dims</b>: (deprecated) exclusive with <b>axis</b></li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor, with the same data and type as <b>input</b>, but has one or more dimensions of size <b>1</b> removed.</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 15  | tf.expand_dims | ExpandDims | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor</li> <li>• <b>axis</b>: 0D (scalar), specifying the dimension index of the extended <b>input</b> shape</li> <li>• <b>name</b>: name of the output Tensor</li> <li>• <b>dim</b>: (deprecated) 0D (scalar), equivalent to <b>axis</b></li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor with the same data as <b>input</b>, but its shape has an additional dimension of size <b>1</b> added</p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                 |



| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 16  | tf.greater | Greater | <p>[Inputs]<br/>Two inputs</p> <ul style="list-style-type: none"> <li>One input is a constant tensor. The data format requirements are as follows:<br/>NC1HWC0 (Huawei-developed format 5D) and 4-dimensional input data (for example, NCHW)<br/>In the NC1HWC0 data format, the C0 is closely related to the micro architecture, and the value is equal to the size of the cube unit, for example, 16. C1 divides the C dimension by C0, <math>C1=C/C0</math>. If the result is not divided, the last data needs to be padding to C0.</li> <li>Another input is a constant scalar.</li> </ul> <p>[Arguments]<br/><b>name:</b> (optional) operation name</p> <p>[Restrictions]<br/> <ul style="list-style-type: none"> <li>Broadcasting is supported, so the shape of <b>x</b> and shape of <b>y</b> are compared. For a right-aligned dimension, if the values of <b>xdim[i]</b> and <b>ydim[i]</b> are not the same, one of them must be <b>1</b> or missing.</li> <li>The next layer operator of the tf.greater can have only two inputs, one of which is the output of the tf.greater operator and the other is a constant.</li> </ul> </p> <p>[Returns]<br/>Constant tensor of type bool</p> <p>[Quantization tool supporting]<br/>N/A</p> |
| 17  | tf.nn.relu | Relu    | <p>[Arguments]<br/> <ul style="list-style-type: none"> <li><b>features:</b> non-constant Tensor</li> <li><b>name:</b> (optional) operation name</li> </ul> </p> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>features</b></p> <p>[Quantization tool supporting]<br/>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

| No. | Python API       | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 18  | tf.nn.relu6      | Relu6   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: non-constant Tensor</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>features</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                                   |
| 19  | tf.nn.leaky_relu | /       | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: non-constant Tensor representing pre-activation values</li> <li>• <b>alpha</b>: slope of the activation function at <math>x &lt; 0</math></li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Activation value</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 20  | tf.exp           | exp     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32 or double</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                     |

| No. | Python API             | C++ API             | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 21  | tf.nn.conv2d_transpose | Conv2DBackpropInput | <p>[Inputs]</p> <ul style="list-style-type: none"> <li>• <b>value</b>: 4D Tensor with shape [batch, height, width, in_channels] for NHWC data format or [batch, in_channels, height, width] for NCHW data format</li> <li>• <b>filter</b>: 4D constant Tensor with shape [height, width, output_channels, in_channels]</li> <li>• <b>input_sizes</b>: 1D constant Tensor</li> </ul> <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>output_shape</b>: 1D Tensor, indicating the output shape</li> <li>• <b>strides</b>: non-null list of integers, each value corresponding to the stride of the sliding window for each dimension of the input tensor</li> <li>• <b>padding</b>: non-null, string, either <b>VALID</b> or <b>SAME</b></li> <li>• <b>data_format</b>: non-null string, either <b>NHWC</b> or <b>NCHW</b></li> <li>• <b>name</b>: (optional) output name</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• group = 1</li> <li>• dilation = 1</li> <li>• filterH – padHHead – 1 ≥ 0</li> <li>• filterW – padWHead – 1 ≥ 0</li> <li>• Restrictions involving intermediate variables: <ol style="list-style-type: none"> <li>1. <math>a = \text{ALIGN}(\text{filter\_num}, 16) * \text{ALIGN}(\text{filter\_c}, 16) * \text{filter\_h} * \text{filter\_w} * 2</math><br/>If <math>\text{ALIGN}(\text{filter\_c}, 16) \% 32 = 0</math>, <math>a = a/2</math></li> <li>2. <math>\text{conv\_input\_width} = (\text{deconvolution input } W - 1) * \text{strideW} + 1</math></li> <li>3. <math>b = (\text{conv\_input\_width}) * \text{filter\_h} * \text{ALIGN}(\text{filter\_num}, 16) * 2 * 2</math></li> <li>4. <math>a + b \leq 1024 * 1024</math></li> </ol> </li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 22  | tf.sigmoid | Sigmoid | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 23  | tf.add     | Add     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32 or int32</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b>. For two constant inputs, one of them is a scalar.</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>If the two inputs have inconsistent dimensions, broadcasting (that is, dimension padding) is performed.</p> <p>Broadcasting is supported only in the following scenarios:</p> <p>NHWC+ NHWC, NHWC+scalar</p> <p>NHWC + 1 1 1 1</p> <p>NHWC + W, HWC + W, HW + W (W-based broadcasting)</p> <p>NCHW + NH1C, HWC + H1C, HW + H1</p> <p>HWC + 1 WC (H-based broadcasting)</p> <p>Note: The input sequence of the two Tensors is not fixed.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>y</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 24  | tf.add_n   | -       | <p>The <b>tf.add_n</b> operator internally invokes the <b>Eltwise</b> operator to add all input tensors by element-wise. For details about the input, constraints, and attributes, see the <b>Eltwise</b> operator in the <a href="#">5.12.2.2 Caffe Operator Boundaries</a>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

| No. | Python API  | C++ API                  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|-------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 25  | tf.multiply | Multiply<br>Type:<br>Mul | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32 or int32</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b>. If two constants are input, their dimensions must be identical (scalar or 1D Tensor)</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>If the two inputs have inconsistent dimensions, broadcasting (that is, dimension padding) is performed.</p> <p>Broadcasting is supported only in the following scenarios:</p> <p>NHWC+ NHWC, NHWC+scalar</p> <p>NHWC + 1 1 1 1</p> <p>NHWC + W, HWC + W, HW + W (W-based broadcasting)</p> <p>NCHW + NH1C, HWC + H1C, HW + H1</p> <p>HWC + 1 WC (H-based broadcasting)</p> <p>Note: The input sequence of the two Tensors is not fixed.</p> <p>[Returns]</p> <p>Tensor</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API  | C++ API               | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|-------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 26  | tf.subtract | Subtract<br>Type: Sub | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b>, constant or non-constant</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>If the two inputs have inconsistent dimensions, broadcasting (that is, dimension padding) is performed.</p> <p>Broadcasting is supported only in the following scenarios:</p> <p>NHWC+ NHWC, NHWC+scalar</p> <p>NHWC + 1 1 1 1</p> <p>NHWC + W, HWC + W, HW + W (W-based broadcasting)</p> <p>NCHW + NH1C, HWC + H1C, HW + H1</p> <p>HWC + 1 WC (H-based broadcasting)</p> <p>Note: The input sequence of the two Tensors is not fixed.</p> <p>[Returns]</p> <p>Tensor</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API     | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|----------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 27  | tf.nn.bias_add | BiasAdd | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>value</b>: non-constant Tensor</li> <li>• <b>bias</b>: 1D constant Tensor, with size matching the last dimension of <b>value</b>, of the same type as <b>value</b> unless <b>value</b> is a quantized type</li> <li>• <b>data_format</b>: string, either <b>NHWC</b> or <b>NCHW</b></li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <math>C &lt; 10000</math></li> <li>• <b>input</b> and <b>bias</b> must have the same data layout.</li> <li>• When <b>bias</b> is added to the <b>C</b> dimensions, the <b>C</b> dimensions of <b>input</b> and <b>bias</b> must be the same.</li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API | C++ API                            | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 28  | tf.nn.lrn  | Local response normalization (LRN) | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: 4D Tensor of type float32</li> <li>• <b>depth_radius</b>: 0D of type int, default to <b>5</b>, indicating the half-width of the 1D normalization window</li> <li>• <b>bias</b>: (optional) float, default to <b>1</b>, indicating the offset (usually positive to avoid dividing by <b>0</b>)</li> <li>• <b>alpha</b>: (optional) float, default to <b>1</b>, indicating the scale factor, usually positive</li> <li>• <b>beta</b>: (optional) float, default to <b>0.5</b>, indicating an exponent</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• <b>depth_radius</b> is an odd number greater than 0.</li> <li>• Inter-channel:<br/>When <b>depth_radius</b> is of range [1,15], alpha &gt; 0.00001 and beta &gt; 0.01; Otherwise, <b>alpha</b> and <b>beta</b> are any values. When C &gt; 1776, depth_radius &lt; 1728.</li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 29  | tf.nn.elu  | Elu                                | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: non-constant Tensor</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>features</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |



| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                          |
|-----|------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 30  | tf.rsqrt   | Rsqrt   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p>                |
| 31  | tf.log     | Log     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 32  | tf.tanh    | Tanh    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: non-constant Tensor</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p>   |

| No. | Python API     | C++ API  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|----------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 33  | tf.slice       | Slice    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_</b>: Tensor</li> <li>• <b>begin</b>: Tensor of type int32 or int64</li> <li>• <b>size</b>: Tensor of type int32 or int64</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>The number of tensor elements cannot exceed <b>INT32_MAX</b>.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input_</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 34  | tf.split       | Split    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>value</b>: Tensor</li> <li>• <b>num_or_size_splits</b>: not supported</li> <li>• <b>axis</b>: integer, specifying the dimension along which to split</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>List of Tensor objects resulting from splitting</p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                |
| 35  | tf.nn.softplus | Softplus | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>features</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                            |

| No. | Python API     | C++ API                     | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|----------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 36  | tf.nn.softsign | Softsign                    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>features</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                                                     |
| 37  | tf.pad         | Pad/<br>MirrorPad/<br>PadV2 | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>tensor</b>: 4D Tensor of type float32 or int32</li> <li>• <b>paddings</b>: constant Tensor of type int32</li> <li>• <b>mode</b>: string, one of <b>CONSTANT</b>, <b>REFLECT</b>, or <b>SYMMETRIC</b></li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>constant_values</b>: scalar pad value to use, of the identical data type as <b>tensor</b></li> </ul> <p>[Restrictions]</p> <p>In <b>CONSTANT</b> mode: <math>0 \leq \text{PAD} \leq 128</math>, <math>0 &lt; W \leq 3000</math></p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>tensor</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |

| No. | Python API                      | C++ API                 | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|---------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 38  | tf.fake_quant_with_min_max_vars | FakeQuantWithMinMaxVars | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>inputs</b>: Tensor of type float32</li> <li>• <b>min</b>: Tensor of type float32</li> <li>• <b>max</b>: Tensor of type float32</li> <li>• <b>num_bits</b>: scalar of type int, default to <b>8</b></li> <li>• <b>narrow_range</b>: (optional) bool, default to <b>False</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p><math>-65504 \leq \min \leq +65504, -65504 \leq \max \leq +65504</math></p> <p>[Returns]</p> <p>Tensor of type float32</p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 39  | tf.reduce_max                   | Max                     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_tensor</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, uint16, int16, int8</li> <li>• <b>axis</b>: 1D list or scalar of type integer</li> <li>• <b>keepdims</b>: bool, indicating whether to retain reduced dimensions with length <b>1</b></li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>reduction_indices</b>: (deprecated) equivalent to <b>axis</b></li> <li>• <b>keep_dims</b>: (deprecated) equivalent to <b>keepdims</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• When the input Tensor has four dimensions: input axis = {3,{1,2,3}}, keepDims = true, <math>H * W * 16 * 2 \leq 16 * 1024</math></li> <li>• When the input Tensor has two dimensions: input axis = {1,{1}}, keepDims = true, <math>H * W * \text{CEIL}(C, 16) * 16 * 2 \leq 16 * 1024</math></li> </ul> <p>[Returns]</p> <p>Reduced Tensor of the identical data type as <b>input_tensor</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API       | C++ API      | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 40  | tf.strided_slice | StridedSlice | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_</b>: Tensor</li> <li>• <b>begin</b>: 1D Tensor of type int32</li> <li>• <b>end</b>: 1D Tensor of type int32</li> <li>• <b>strides</b>: 1D Tensor of type int32</li> <li>• <b>begin_mask</b>: scalar of type int32</li> <li>• <b>end_mask</b>: scalar of type int32</li> <li>• <b>ellipsis_mask</b>: scalar of type int32</li> <li>• <b>new_axis_mask</b>: scalar of type int32</li> <li>• <b>shrink_axis_mask</b>: scalar of type int32</li> <li>• <b>var</b>: variable corresponding to <b>input_</b> or <b>None</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>strides ≠ 0</p> <p>[Returns]<br/>Tensor of the identical data type as <b>input_</b></p> <p>[Quantization tool supporting]<br/>No</p> |
| 41  | tf.reverse       | Reverse      | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>tensor</b>: list of Tensor objects</li> <li>• <b>axis</b>: dimensions to reverse, of type int32 or int64</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>tensor</b></p> <p>[Quantization tool supporting]<br/>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                            |

| No. | Python API   | C++ API   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|--------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 42  | tf.realdiv   | RealDiv   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>y</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                                                                                                                                                   |
| 43  | tf.stack     | Stack     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>values</b>: list of Tensor objects with the same shape and type (float32 or int32)</li> <li>• <b>axis</b>: (mandatory) integer, indicating the axis to stack along, default to the first dimension</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Stacked Tensor of the identical data type as <b>values</b></p> <p>Properties <b>T</b>, <b>N</b>, and <b>axis</b> are mandatory.</p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 44  | tf.transpose | Transpose | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>a</b>: Tensor</li> <li>• <b>perm</b>: permutation of the dimensions of <b>a</b></li> <li>• <b>name</b>: (optional) operation name</li> <li>• <b>conjugate</b>: (optional) bool, default to <b>False</b>. Setting it to <b>True</b> is mathematically equivalent to <b>tf.conj(tf.transpose(input))</b>.</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Transposed Tensor</p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                           |

| No. | Python API           | C++ API        | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|----------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 45  | tf.space_to_batch_nd | SpaceToBatchND | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: ND Tensor with shape <code>input_shape = [batch] + spatial_shape + remaining_shape</code>, where <b>spatial_shape</b> has M dimensions. The following data types are supported: uint8, int8, int16, uint16, int32, int64, and float32</li> <li>• <b>block_shape</b>: 1D Tensor of type int32 or int64, with shape [M]. All values must be <math>\geq 1</math>.</li> <li>• <b>paddings</b>: 2D Tensor of type int32 or int64, with shape [M, 2]. All values must be <math>\geq 0</math>.</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• When the tensor rank is 4: the length of <b>blockShape</b> must be 2, and the length of <b>paddings</b> must be 4.</li> <li>• Element value of <b>blockShape</b> <math>\geq 1</math>; Element value of <b>paddings</b> <math>\geq 0</math></li> <li>• The padded <b>H</b> dimension is a multiple of <b>blockShape[0]</b>, and the padded <b>W</b> dimension is a multiple of <b>blockShape[1]</b>.</li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API           | C++ API        | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|----------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 46  | tf.batch_to_space_nd | BatchToSpaceND | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: ND Tensor with shape <code>input_shape = [batch] + spatial_shape + remaining_shape</code>, where <b>spatial_shape</b> has M dimensions. The following data types are supported: uint8, int8, int16, uint16, int32, int64, and float32</li> <li>• <b>block_shape</b>: 1D Tensor of type int32 or int64, with shape [M]. All values must be <math>\geq 1</math>.</li> <li>• <b>crops</b>: 2D Tensor of type int32 or int64, with shape [M, 2]. All values must be <math>\geq 0</math>.</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• The element data type of <b>blockShape</b> and <b>crops</b> must be int32. When the dimension count of the Tensor is 4, the length of <b>blockShape</b> must be 2, and the length of <b>crops</b> must be 4.</li> <li>• Element value of <b>blockShape</b> <math>\geq 1</math>; Element value of <b>crops</b> <math>\geq 0</math></li> </ul> <p><code>crop_start[i] + crop_end[i] &lt; block_shape[i] * input_shape[i + 1]</code></p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>images</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |



| No. | Python API               | C++ API             | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|--------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 47  | tf.extract_image_patches | ExtractImagePatches | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>images</b>: 4-D Tensor with shape [batch, in_rows, in_cols, depth], must be one of the following types: float32, int32, int64, uint8, int8, uint16, and int16;</li> <li>• <b>ksizes</b>: list of ints with length <math>\geq 4</math></li> <li>• <b>strides</b>: list of ints, must be [1, stride_rows, stride_cols, 1]</li> <li>• <b>rate</b>: list of ints, must be [1, rate_rows, rate_cols, 1]</li> <li>• <b>padding</b>: string, either <b>VALID</b> or <b>SAME</b>. <b>VALID</b> indicates that the selected patch area must be completely included in the source image. <b>SAME</b> indicates that the part that exceeds the source image is padded with 0.</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>images</b></p> <p>[Quantization tool supporting]<br/>No</p> |
| 48  | tf.floormod              | FloorMod            | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32 or int32</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]<br/>Broadcasting is supported, so the shape of <b>x</b> and shape of <b>y</b> are compared. For a right-aligned dimension, if the values of <b>xdim[i]</b> and <b>ydim[i]</b> are not the same, one of them must be 1 or missing.</p> <p>[Returns]<br/>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]<br/>No</p>                                                                                                                                                                                                                                                                                                                                      |

| No. | Python API    | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----|---------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 49  | tf.nn.softmax | Softmax | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>logits</b>: non-null Tensor of type float32</li> <li>• <b>axis</b>: dimension softmax to be performed on, default to <b>-1</b>, indicating the last dimension. The value cannot be greater than the rank of <b>logits</b>.</li> <li>• <b>name</b>: (optional) operation name</li> <li>• <b>dim</b>: (deprecated) equivalent to <b>axis</b></li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• If the input contains four dimensions, softmax is performed on each of them.</li> </ul> <p>According to <b>axis</b>:</p> <p>When axis = 1: <math>C \leq ((256 * 1024/4) - 8 * 1024 - 256)/2</math></p> <p>When axis = 0: <math>N \leq (56 * 1024 - 256)/2</math></p> <p>When axis = 2: <math>W = 1, 0 &lt; H &lt; (1024 * 1024/32)</math></p> <p>When axis = 3: <math>0 &lt; W &lt; (1024 * 1024/32)</math></p> <ul style="list-style-type: none"> <li>• If the input contains fewer than four dimensions, softmax is performed only on the last dimension, with the last dimension <math>\leq 46080</math>.</li> </ul> <p>[Returns]</p> <p>Tensor of the identical type and shape as <b>logits</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 50  | tf.math.pow   | Power   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>y</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>power! = 1</p> <p>scale * x + shift &gt; 0</p> <p>[Returns]</p> <p>Tensor</p> <p>[Quantization tool supporting]</p> <p>Yes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

| No. | Python API       | C++ API   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 51  | tf.nn.leaky_relu | LeakyRelu | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: Tensor of type float32</li> <li>• <b>alpha</b>: slope of the activation function at <math>x &lt; 0</math></li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Activation value</p> <p>[Quantization tool supporting]<br/>Yes</p>                                                    |
| 52  | tf.placeholder   | -         | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>dtype</b>: (mandatory) data type</li> <li>• <b>shape</b>: (mandatory) shape of the tensor to be fed</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor</p> <p>[Quantization tool supporting]<br/>No</p>                                                                                                                                     |
| 53  | tf.shape         | Shape     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor or SparseTensor</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>out_type</b>: data type for the output Tensor, either int32 (default) or int64</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the data type specified by <b>out_type</b></p> <p>[Quantization tool supporting]<br/>N/A</p> |

| No. | Python API     | C++ API            | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|----------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 54  | tf.math.argmax | ArgMax             | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor, must be one of the following types: int8, uint8, int16, uint16, int32, int64, float32</li> <li>• <b>axis</b>: Tensor of type int32 or int64</li> <li>• <b>out_type</b>: data type for the output Tensor, either int32 or int64 (default)</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the data type specified by <b>out_type</b></p> <p>[Quantization tool supporting]<br/>No</p>                                 |
| 55  | tf.gather      | Gather<br>GatherV2 | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>params</b>: Tensor, must be at least rank <b>axis + 1</b></li> <li>• <b>indices</b>: Tensor of type float32 or int64, must be in range [0, params.shape[axis])</li> <li>• <b>axis</b>: output Tensor of type float32 or int64, specifying the axis in params to gather indices from, rank = 0</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>params</b></p> <p>[Quantization tool supporting]<br/>No</p> |

| No. | Python API        | C++ API  | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|-------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 56  | tf.gather_nd      | GatherNd | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>params</b>: Tensor, must be at least rank <b>axis</b> + 1</li> <li>• <b>axis</b>: Tensor of type int32 or int64</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p><b>indices</b>: The last dimension of <b>indices</b> can be at most the rank of <b>params</b>.</p> <p>The elements in the last dimension of <b>indices</b> correspond to the coordinates along a dimension of <b>params</b>. Therefore, the coordinate rules must be met.</p> <p>The coordinates along the corresponding dimension of <b>indices</b> cannot exceed the dimension size.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>params</b></p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 57  | tf.math.floor_div | FloorDiv | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32 or int32</li> <li>• <b>y</b>: Tensor, denominator of type float32 or int32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported, so the shape of x and shape of y are compared. For a right-aligned dimension, if the values of xdim[i] and ydim[i] are not the same, one of them must be 1 or missing.</p> <p>[Returns]</p> <p>Tensor, floor (x/y)</p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                             |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 58  | tf.range   | Range   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>start</b>: start constant scalar of type float32 or int32</li> <li>• <b>limit</b>: end constant scalar of type float32 or int32</li> <li>• <b>delta</b>: stride constant scalar of type float32 or int32</li> <li>• <b>dtype</b>: data type of the resulting Tensor</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>1D Tensor</p> <p>[Quantization tool supporting]<br/>No</p> |
| 59  | tf.tile    | Tile    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor, must be one of the following types: int8, uint8, int16, uint16, int32, int64, float32</li> <li>• <b>multiples</b>: 1D constant Tensor of type int32. The length must be the same as that of <b>input</b>.</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor</p> <p>[Quantization tool supporting]<br/>Yes</p>                                          |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 60  | tf.size    | Size    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>out_type</b>: data type for the output Tensor, default to int32</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the data type specified by <b>out_type</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                   |
| 61  | tf.fill    | Fill    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>dims</b>: 1D Tensor of type int32</li> <li>• <b>value</b>: variable of type int32 or float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The following padding modes are supported: Constant, GivenTensor, Range, Diagonal, Gaussian, MSRA, Uniform, UniformInt, UniqueUniform, and XavierFill. When the Uniform, UniformInt, UniqueUniform, and xavier padding modes are used, the value range of the generated value is [min, max).</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>value</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----|------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 62  | tf.concat  | Concat  | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>value</b>: list of Tensor objects of type int32 or float32</li> <li>• <b>axis</b>: Tensor of type int32, specifying the dimension to be concatenated</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The number of dimensions of the input tensors must match, and all dimensions except <b>axis</b> must be equal.</p> <p>The range of the input Tensor count is [1, 1000].</p> <p>[Returns]</p> <p>Tensor</p> <p>[Quantization tool supporting]</p> <p>Yes</p> |
| 63  | tf.reverse | Reverse | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>tensor</b>: list of Tensor objects</li> <li>• <b>axis</b>: dimensions to reverse, of type int32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>tensor</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                              |



| No. | Python API      | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----|-----------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 64  | tf.reduce_sum   | sum     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_tensor</b>: Tensor</li> <li>• <b>axis</b>: dimensions to sum up, int32</li> <li>• <b>keepdims</b>: bool, indicating whether to retain reduced dimensions</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>reduction_indices</b>: (deprecated) string, equivalent to <b>axis</b></li> <li>• <b>keep_dims</b>: (deprecated) equivalent to <b>keepdims</b></li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>tensor</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                  |
| 65  | tf.math.maximum | Maximum | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: int32, int64, float32</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported, so the shape of x and shape of y are compared. For a right-aligned dimension, if the values of xdim[i] and ydim[i] are not the same, one of them must be 1 or missing.</p> <p>[Returns]</p> <p>Tensor. Returns the max of x and y (x &gt; y ? x: y). Identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API          | C++ API     | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|---------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 66  | tf.math.minimum     | Minimum     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: int32, int64, float32</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported in the following two scenarios:<br/>NHWC+scaler, NHWC+NHWC</p> <p>[Returns]</p> <p>Tensor. Returns the min of x and y (<math>x &lt; y ? x : y</math>).<br/>Identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 67  | tf.clip_by_value    | ClipByValue | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>t</b>: Tensor</li> <li>• <b>clip_value_min</b>: minimum value to clip by</li> <li>• <b>clip_value_max</b>: maximum value to clip by</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The minimum value must be less than or equal to the maximum value.</p> <p>[Returns]</p> <p>Clipped Tensor. The return value range is [clip_value_min, clip_value_max].</p> <p>[Quantization tool supporting]</p> <p>No</p>                                               |
| 68  | tf.math.logical_not | LogicalNot  | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type bool</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of type bool</p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                              |

| No. | Python API          | C++ API    | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|---------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 69  | tf.math.logical_and | LogicalAnd | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type bool</li> <li>• <b>y</b>: Tensor of type bool</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported in the following dimension scenarios: NHWC and [1,1,1,1], [N,C,H,W], [N,1,H,W], [1,C,H,W], [N,C,1,1]</p> <p>[Returns]</p> <p>Tensor of type bool</p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                             |
| 70  | tf.equal            | Equal      | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported, so the shape of <b>x</b> and shape of <b>y</b> are compared. For a right-aligned dimension, if the values of <b>xdim[i]</b> and <b>ydim[i]</b> are not the same, one of them must be 1 or missing.</p> <p>[Returns]</p> <p>Tensor of type bool</p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 71  | tf.square           | Square     | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                   |

| No. | Python API               | C++ API       | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|--------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 72  | tf.image.crop_and_resize | CropAndResize | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>image</b>: 4D Tensor, must be one of the following types: float32 and int8, int32, int64; with shape [num_boxes, 4]</li> <li>• <b>boxes</b>: 2D Tensor of type float32, with shape [num_boxes]</li> <li>• <b>box_ind</b>: 1D Tensor of type int32</li> <li>• <b>crop_size</b>: 1D 2-element Tensor of type int32</li> <li>• <b>method</b>: interpolation method string, options: <b>bilinear</b> (default) or <b>nearest</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of type float32</p> <p>[Quantization tool supporting]<br/>No</p>                                  |
| 73  | tf.math.top_k            | TopKV2        | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: 1D Tensor or higher with the last dimension at least <b>k</b>, of type float32 (<b>k</b>: scalar of type int32, <math>\geq 1</math>)</li> <li>• <b>sorted</b>: bool</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/><b>k</b>: constant</p> <p>[Returns]</p> <ul style="list-style-type: none"> <li>• <b>values</b>: Tensor, indicating k largest elements along each last dimensional slice</li> <li>• <b>indices</b>: Tensor, indicating the indices of <b>values</b> of <b>input</b></li> </ul> <p>[Quantization tool supporting]<br/> <ul style="list-style-type: none"> <li>• No</li> </ul> </p> |

| No. | Python API            | C++ API           | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|-----------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 74  | tf.invert_permutation | InvertPermutation | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: 1D Tensor of type int32 or int64</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 75  | tf.multinomial        | Multinomial       | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>logits</b>: 2D Tensor with shape [batch_size, num_classes]</li> <li>• <b>num_samples</b>: scalar, indicating the number of samples to draw</li> <li>• <b>seed</b>: int32 or int64, used to create a random seed</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>output_dtype</b>: integer, data type for the output Tensor, default to int64</li> </ul> <p>[Restrictions]</p> <p>When <b>seed</b> is <b>0</b>, the generated random is dynamic.</p> <p>The number of output data rows is the actual number of output data rows while the number of output data columns is <b>num_samples</b>.</p> <p>[Returns]</p> <p>Tensor of the data type specified by <b>output_dtype</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API          | C++ API         | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|---------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 76  | tf.reverse_sequence | ReverseSequence | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor</li> <li>• <b>seq_lengths</b>: 1D Tensor of type int32 or int64</li> <li>• <b>seq_axis</b>: scalar of type integer</li> <li>• <b>batch_axis</b>: scalar of type integer</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• The length of <b>seq_lengths</b> must be equal to the number of elements of input in <b>batchAxis</b>.</li> <li>• The maximum element in <b>seq_lengths</b> must be less than or equal to the number of elements in <b>seq_dim</b>.</li> <li>• <b>seqAxis</b>, <b>batchAxis</b>, <b>seqDim</b>, and <b>batchDim</b> must be of type int64.</li> <li>• <b>seqAxis</b> and <b>seqDim</b> are exclusive. <b>batchAxis</b> and <b>batchDim</b> are exclusive.</li> <li>• <b>batchAxis</b> and <b>batchDim</b> are optional. The default values are 0.</li> </ul> <p>Only one weight is required.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 77  | tf.math.reciprocal  | Reciprocal      | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The input data cannot contain 0.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

| No. | Python API    | C++ API | Boundary                                                                                                                                                                                                                                                                                                                       |
|-----|---------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 78  | tf.nn.selu    | Selu    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>features</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>features</b></p> <p>[Quantization tool supporting]<br/>No</p> |
| 79  | tf.math.acosh | Acosh   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]<br/>No</p>               |
| 80  | tf.math.asinh | Asinh   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]<br/>No</p>               |

| No. | Python API          | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|---------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 81  | tf.math.reduce_prod | Prod    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_tensor</b>: Tensor</li> <li>• <b>axis</b>: dimension to reduce</li> <li>• <b>keepdims</b>: bool, indicating whether to retain reduced dimensions</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>reduction_indices</b>: (deprecated) equivalent to <b>axis</b></li> <li>• <b>keep_dims</b>: (deprecated) equivalent to <b>keepdims</b></li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor and reduced Tensor</p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 82  | tf.math.sqrt        | Sqrt    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                          |



| No. | Python API         | C++ API     | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|--------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 83  | tf.math.reduce_all | All         | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_tensor</b>: Tensor of type bool</li> <li>• <b>axis</b>: dimension to reduce</li> <li>• <b>keepdims</b>: bool</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>reduction_indices</b>: (deprecated) equivalent to <b>axis</b></li> <li>• <b>keep_dims</b>: (deprecated) equivalent to <b>keepdims</b></li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input_tensor</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                               |
| 84  | tf.nn.l2_normalize | L2Normalize | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type bool</li> <li>• <b>axis</b>: dimension along which to normalize<br/>For format NCHW, <b>axis</b> must be set to <b>1</b>.<br/>For format NHWC, <b>axis</b> must be set to <b>3</b>.</li> <li>• <b>epsilon</b>: lower bound value for the norm.<br/>Value range: (1e-7, 0.1]. If norm &lt; sqrt(epsilon), <b>sqrt(epsilon)</b> is used as the divisor.</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>dim</b>: (deprecated) equivalent to <b>axis</b></li> </ul> <p>[Restrictions]</p> <p><math>H * W * 2 &lt; 256 * 1024 / 4</math></p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API                      | C++ API         | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|---------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 85  | tf.keras.backend.hard_sigmoid   | Hardsigmoid     | <p>[Arguments]<br/> <b>x</b>: Tensor<br/> [Restrictions]<br/> None<br/> [Returns]<br/> Output Tensor<br/> If <math>x &lt; -2.5</math>, <b>0</b> is returned. If <math>x &gt; 2.5</math>, <b>1</b> is returned.<br/> If <math>-2.5 \leq x \leq 2.5</math>, <math>0.2 * x + 0.5</math> is returned.<br/> [Quantization tool supporting]<br/> No</p>                                                                                                                        |
| 86  | tf.keras.layers.ThresholdedReLU | ThresholdedReLU | <p>[Arguments]<br/> <b>theta</b>: scalar <math>\geq 0</math> of type float32<br/> [Restrictions]<br/> None<br/> [Returns]<br/> Tensor<br/> [Quantization tool supporting]<br/> No</p>                                                                                                                                                                                                                                                                                    |
| 87  | tf.math.acos                    | Acos            | <p>[Arguments]<br/> <ul style="list-style-type: none"> <li><b>x</b>: Tensor, must be one of the following types: float32, int32, int64</li> <li><b>name</b>: (optional) operation name, string</li> </ul> [Restrictions]<br/> The input data range is <math>(-1 \leq x \leq +1)</math>, and the output data range is <math>(0 \leq y \leq \pi)</math>.<br/> [Returns]<br/> Tensor of the identical data type as <b>x</b><br/> [Quantization tool supporting]<br/> No</p> |

| No. | Python API    | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|---------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 88  | tf.math.atan  | Arctan  | <p>[Arguments]</p> <ul style="list-style-type: none"> <li><b>x</b>: Tensor, must be one of the following types: float32, int32, int64</li> <li><b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The input data range is <math>(-65504 \leq x \leq +65504)</math>, and the output data range is <math>(-\pi/2 \leq y \leq +\pi/2)</math>.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 89  | tf.math.asin  | Asin    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li><b>x</b>: Tensor, must be one of the following types: float32, int32, int64</li> <li><b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The input data range is <math>(-1 \leq x \leq +1)</math>, and the output data range is <math>(-\pi/2 \leq y \leq +\pi/2)</math>.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>         |
| 90  | tf.math.atanh | Atanh   | <p>[Arguments]</p> <ul style="list-style-type: none"> <li><b>x</b>: Tensor, must be one of the following types: float32, int32, int64</li> <li><b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Input data range: <b>x</b> is of range <math>(-1, +1)</math></p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                             |

| No. | Python API         | C++ API   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|--------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 91  | tf.math.tan        | Tan       | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int32, int64</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                           |
| 92  | tf.math.logical_or | LogicalOr | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type bool</li> <li>• <b>y</b>: Tensor of type bool</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported, so the shape of <b>x</b> and shape of <b>y</b> are compared. For a right-aligned dimension, if the values of <b>xdim[i]</b> and <b>ydim[i]</b> are not the same, one of them must be 1 or missing.</p> <p>[Returns]</p> <p>Tensor of type bool</p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API         | C++ API   | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----|--------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 93  | tf.math.reduce_min | ReduceMin | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input_tensor</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, uint16, int8, int16</li> <li>• <b>axis</b>: dimension to reduce</li> <li>• <b>keepdims</b>: scalar of type bool</li> <li>• <b>name</b>: (optional) operation name, string</li> <li>• <b>reduction_indices</b>: (deprecated) equivalent to axis</li> <li>• <b>keep_dims</b>: (deprecated) equivalent to keepdims</li> </ul> <p>[Restrictions]</p> <ul style="list-style-type: none"> <li>• When the input Tensor has four dimensions: input axis = {3,{1,2,3}}, keepDims = true, <math>H * W * 16 * 2 \leq 16 * 1024</math></li> <li>• When the input Tensor has two dimensions: input axis = {1,{1}}, keepDims = true, <math>H * W * \text{CEIL}(C, 16) * 16 * 2 \leq 16 * 1024</math></li> </ul> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input_tensor</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 94  | tf.math.negative   | Neg       | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int64, int32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>The input data range is <math>-65504 \leq x \leq +65504</math>, and the output data range is <math>-65504 \leq y \leq +65504</math>.</p> <p>[Returns]</p> <p>Tensor. Returns <math>-x</math>.</p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

| No. | Python API            | C++ API      | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|-----------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 95  | tf.math.greater_equal | GreaterEqual | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, uint16, int8, int16</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Input data range: <math>-65504 \leq x \leq +65504</math></p> <p>[Returns]</p> <p>Tensor of type bool</p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                        |
| 96  | tf.space_to_depth     | SpaceToDepth | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, int8</li> <li>• <b>block_size</b>: scalar of type integer, <math>\geq 2</math></li> <li>• <b>data_format</b>: string, default to <b>NHWC</b> (options: <b>NHWC</b>, <b>NCHW</b>, <b>NCHW_VECT_C</b>)</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p><math>\text{blockSize} \geq 1</math> and <b>blockSize</b> must be a divisor of both the input height and width.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

| No. | Python API        | C++ API      | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----|-------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 97  | tf.depth_to_space | DepthToSpace | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>input</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, int8</li> <li>• <b>block_size</b>: scalar of type integer, <math>\geq 2</math></li> <li>• <b>data_format</b>: string, default to <b>NHWC</b> (options: <b>NHWC</b>, <b>NCHW</b>, <b>NCHW_VECT_C</b>)</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p><b>blockSize</b> must be greater than or equal to 1, and <b>blockSize</b> * <b>blockSize</b> must be exactly divided by <b>C</b>.</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>input</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |
| 98  | tf.math.round     | Round        | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int64, int32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type and shape as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                                         |
| 99  | tf.math rint      | Rint         | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, int8</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>None</p> <p>[Returns]</p> <p>Tensor of the identical data type and shape as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p>                                                                                                                                                                                                                                                                                                                            |

| No. | Python API   | C++ API | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|--------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 100 | tf.math.less | Less    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int64, int32, uint8, uint16, int8, int16</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of type bool</p> <p>[Quantization tool supporting]<br/>No</p> |
| 101 | tf.math.sinh | Sinh    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]<br/>No</p>                                                                                                                 |
| 102 | tf.math.cosh | Cosh    | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor of type float32</li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]<br/>None</p> <p>[Returns]<br/>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]<br/>No</p>                                                                                                                 |



| No. | Python API                 | C++ API            | Boundary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|----------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 103 | tf.math.squared_difference | Squared_difference | <p>[Arguments]</p> <ul style="list-style-type: none"> <li>• <b>x</b>: Tensor, must be one of the following types: float32, int64, int32</li> <li>• <b>y</b>: Tensor of the identical data type as <b>x</b></li> <li>• <b>name</b>: (optional) operation name, string</li> </ul> <p>[Restrictions]</p> <p>Broadcasting is supported only in the following scenarios:</p> <p>One NCHW Tensor and one Tensor of the following format: dim{} = [1,1,1,1], [N,C,H,W], [N,1,H,W], [1,C,H,W], [N,C,1,1], [1,C,1,1], [1,1,H,W], or [N,1,1,1]</p> <p>[Returns]</p> <p>Tensor of the identical data type as <b>x</b></p> <p>[Quantization tool supporting]</p> <p>No</p> |

# 6 Application Software Development

---

- [6.1 Before You Start](#)
- [6.2 Software Development Description \(Important\)](#)
- [6.3 Preparations](#)
- [6.4 Development Environment](#)
- [6.5 Inference Service Flow](#)
- [6.6 Code Execution Sample \(HelloDavinci\)](#)
- [6.7 Software Code Development](#)
- [6.8 Graph Keywords](#)

## 6.1 Before You Start

This topic describes the basic knowledge, requirements, and precautions for using the Atlas 200 to develop services.

You are advised to read this section carefully before starting the development.

### 6.1.1 Application Scenarios

Based on application scenarios, the Atlas 200 can serve as:

- Main processor
- Coprocessor
- PCIe slave device

#### NOTE

- For details about the three application modes, see "Application Modes" in the *Atlas 200 AI Accelerator Module Earlier Than 1.0.0.SPC200 System Software Development Guide (Model 3000)*.
- The application software development processes of the main processor and coprocessor are basically the same, which are called the SoC mode in this document. The slave PCIe device mode is called the EP mode.

## 6.1.2 Key Concepts

| Concept     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ascend 310  | <p>The Ascend 310 is a high-performance and low-power consumption AI chip designed for scenarios such as image recognition, video processing, inference computing, and machine learning.</p> <p>The chip has two built-in AI cores, supports 128-bit LPDDR4X, and provides up to 16 TOPS (Float16/INT8) computing capability.</p>                                                                                                                                                                                                                       |
| Atlas 200   | <p>The Atlas 200 is a high-performance AI acceleration module based on the Ascend 310 processor, which implements efficient model inference, image recognition, and image processing. Featuring compact size, high performance, low energy efficiency, and simple integration, the Atlas 200 can be used as an SoC, a coprocessor, or a PCIe slave device.</p>                                                                                                                                                                                          |
| Mind Studio | <p>Mind Studio is an AI full-stack development platform based on Huawei neural-network processing units (NPUs). This platform supports chip-based operator development, commissioning, optimization, and custom operator development. It also supports network migration, optimization, and analysis at the network layer. In addition, This platform supports visualized drag-and-drop AI engine programming. For details about Mind Studio, see the <i>Ascend 310 Mind Studio Quick Start</i> and <i>Ascend 310 Mind Studio Basic Operations</i>.</p> |
| DDK         | <p>The Mind Studio solution provides the Digital Development Kit (DDK) for developers. You can install the DDK to obtain the APIs, libraries, and toolchains required for development on Mind Studio.</p>                                                                                                                                                                                                                                                                                                                                               |
| Graph       | <p>Graph is a concept in the HiAI framework instead of the computational graph in the deep learning framework. In the HiAI framework, a graph is a diagram that describes the entire service processing flow. It is a program processing flow consisting of multiple engines.</p>                                                                                                                                                                                                                                                                       |
| HiAI Engine | <p>HiAI Engine is a universal service flow execution engine. It consists of Agent that runs on the host and Manager that runs on the device. Each engine provides a function implemented by user code, that is, the engine processing program is implemented by users.</p>                                                                                                                                                                                                                                                                              |
| Host        | <p>In a server+Atlas 200 system, the host is the operating system (OS) of the server CPU.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Device      | <p>In a server+Atlas 200 system, the device is the OS of Atlas 200.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| DVPP        | <p>Digital vision pre-processing (DVPP) supports pre-processing operations such as image/video decoding and scaling. It is also capable of encoding and outputting preprocessed videos and images.</p>                                                                                                                                                                                                                                                                                                                                                  |
| AIPP        | <p>AI pre-processing (AIPP) provides functions such as format conversion and padding/cropping, CSC (YUV2RGB or RGB2YUV), scaling, and channel data exchange.</p>                                                                                                                                                                                                                                                                                                                                                                                        |

| Concept  | Description                                                                                                                                                                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OMG      | Offline model generator (OMG) converts models trained by using frameworks such as Caffe and TensorFlow into offline models supported by Huawei chips. The OMG also supports model optimization functions that are performed independent from the device, such as operator scheduling optimization, weight data rearrangement and compression, and memory usage optimization. |
| OME      | Offline model executor (OME) loads converted offline models for inference.                                                                                                                                                                                                                                                                                                   |
| Ctrl CPU | One Ascend 310 chip has four Ctrl CPUs, which are used for service logic processing.                                                                                                                                                                                                                                                                                         |
| AI CPU   | One Ascend 310 chip has four AI CPUs, which are used for operator task scheduling and implementation of some operators.                                                                                                                                                                                                                                                      |
| AI Core  | One Ascend 310 chip has two AI cores, which are used for matrix computing.                                                                                                                                                                                                                                                                                                   |
| IPC      | An IP camera (IPC) provides RTSP data streams.                                                                                                                                                                                                                                                                                                                               |

## 6.1.3 Service Development Constraints

For details about development constraints, see [Table 6-1](#).

**Table 6-1** Service development constraints

| N o. | Type          | Constraint Description                                                                                                                                                              | Importance Level |
|------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| 1    | Compatibility | Currently, the file system of the Atlas 200 supports only Ubuntu 16.04.3 - Arm64.                                                                                                   | High             |
| 2    |               | The current delivery version supports g++5.4. If the compilation chain is earlier than this version, you are advised to compile the source code again.                              | High             |
| 3    | Inference     | The Atlas 200 uses NPUs and is intended for deep learning computing.                                                                                                                | High             |
| 4    |               | Currently, the Atlas 200 supports only AI inference instead of training. The inference platform supports only the Caffe and TensorFlow deep learning frameworks.                    | High             |
| 5    |               | Most video and image inference networks and operators are supported. Operators of other fields such as voice and NLP are not supported. For details, see the operator support list. | High             |

| N<br>o. | Typ<br>e  | Constraint Description                                                                                                                                                                                                                                                                                                                                                                                                        | Import<br>ance<br>Level |
|---------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| 6       |           | The inference does not support dynamic batch sizes. You need to perform inference based on a preset batch size (the recommended batch size is 8). For scenarios that do not support dynamic batch sizes, you need to supplement the batch size data.<br><br>For example, the batch size during model conversion is 8. If data of only four batches is used, you need to add the data of the other four batches for inference. | High                    |
| 7       | Cod<br>ec | Video decoding (VDEC) supports a maximum of 16 decoding engines for each chip.                                                                                                                                                                                                                                                                                                                                                | High                    |
| 8       |           | JPEG image decoding supports only the Huffman encoding format instead of the arithmetic, progressive, or JPEG 2000 encoding formats. Currently, the YUV440 sampling format is not supported.                                                                                                                                                                                                                                  | High                    |
| 9       |           | PNG images support only the 32-bit RGBA format.                                                                                                                                                                                                                                                                                                                                                                               | High                    |
| 10      |           | Data output by the DVPP VDEC is in HFBC format (Huawei format), which cannot be used directly and has to be converted by the VPC first.<br><br>The VPC supports only YUV420SP outputs instead of YUV2RGB conversion.                                                                                                                                                                                                          | High                    |
| 11      |           | Constraints on the memory used by VPC input and output images are as follows: the image start address is 128-byte aligned, the image width is 128-byte aligned, and the image height is 16-byte aligned. Redundant image data after alignment is cropped by using the AIPP.                                                                                                                                                   | High                    |
| 12      |           | The minimum image resolution for VPC cropping is 16 x 16.                                                                                                                                                                                                                                                                                                                                                                     | Low                     |
| 13      |           | AIPP supports only YUV420SP_U8 format input (NV12).                                                                                                                                                                                                                                                                                                                                                                           | Medium                  |
| 14      |           | The JPEG decoder (JPEGD) of the DVPP supports the NV21 format. Before inference, you must use the VPC to convert data in NV21 format into data in NV12 format.                                                                                                                                                                                                                                                                | Medium                  |
| 15      |           | Due to hardware limitations, the DVPP must use addresses in the 4 GB address space. It is recommended that the DVPP be created before the model manager is initialized so that the DVPP data is reserved within the 4 GB address space.                                                                                                                                                                                       | High                    |
| 16      |           | The maximum VPC scaling ratio is 16x. You can use multiple levels of software scaling to achieve scaling ratios greater than 16x.                                                                                                                                                                                                                                                                                             | High                    |

| N<br>o. | Typ<br>e | Constraint Description                                                                                                                                                                   | Import<br>ance<br>Level |
|---------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| 17      |          | Video decoding supports a maximum of 480 fps@1080p. A single graph supports a maximum of 320 fps@1080p (you can use the multi-process mode to achieve frame rates greater than 320 fps). | High                    |

## 6.2 Software Development Description (Important)

### Software Development in EP Mode

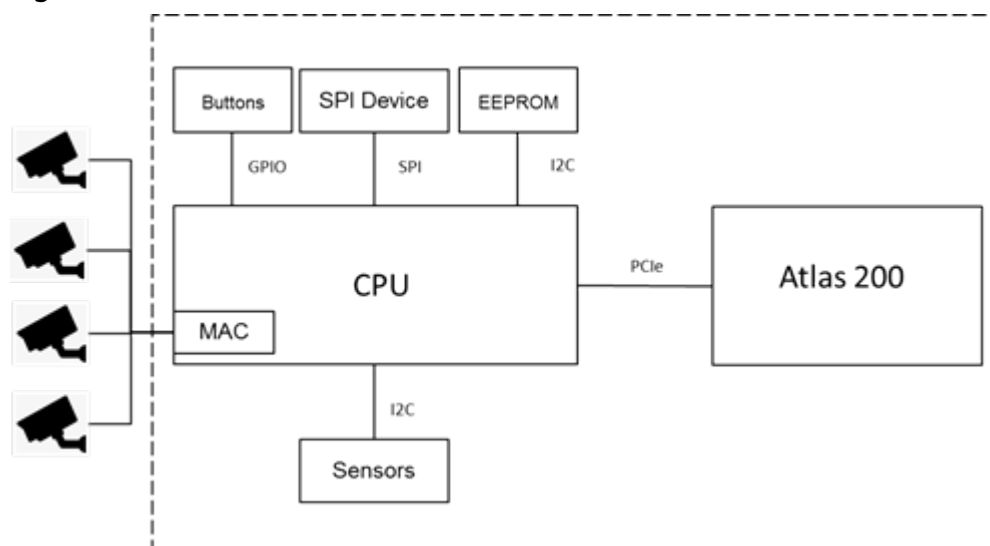
The Atlas 200 EP mode indicates that the Atlas 200 connects to the CPU system as a PCIe slave device. [Figure 6-1](#) shows the Atlas 200 EP mode. In EP mode, service software runs on the host, interacts with the Atlas 200 through the PCIe channel, and allocates AI tasks to the Ascend 310 chip for running.

In this application mode, pay attention to the CPU model and consider the compatibility between the CPU and Atlas 200.

Processors compatible with the Atlas 200 must meet the following requirements:

1. Supports PCIe MSI-X interrupts (at least 116 MSI-X interrupts can be allocated).
2. Supports three base address registers (BARs) with sizes of 128 KB, 16 MB, and 64 MB respectively.

**Figure 6-1** Atlas 200 EP mode



Different from that in SoC mode, the CPU serves as the host in EP mode, and the Atlas 200 serves as the device. The two sides communicate with each other through the PCIe. Therefore, cross-side transmission is involved in EP mode, that is, data is transmitted between the host side and the device side: The data to be

transmitted needs to be serialized into binary data. After data is transmitted by using hardware such as the PCIe or DMA, the data is deserialized into valid data. Therefore, you need to customize serialization and deserialization functions for custom data structures. The Matrix framework allows you to define serialization and deserialization functions using either a common or a high-speed interface. The common interface is applicable to data transmission at a rate below 256 kbit/s, whereas the high-speed interface is applicable to data transmission at a rate of 256 kbit/s or higher. The high-speed interface operates at a speed similar to that of a common interface when transmitting small memory blocks. For details, see "Graph::SetDataRecvFunctor" and "Engine::SetDataRecvFunctor" in the *Matrix API Reference*.

In addition, code needs to be compiled separately because some code runs on the host side while some code runs on the device side. You need to use a proper compiler based on the selected CPU model and OS type, to correctly compile code and run services.

The service development in EP mode is similar to that in Atlas 500 mode. The Atlas 500 is an example of the Atlas 200 running in EP mode. In the Atlas 500, Hi3559A serves as the main processor, that is, the CPU in [Figure 6-1](#). The Atlas 200 is used as a PCIe slave device, which is controlled by Hi3559A to complete service flows.

If you want to develop service software in EP mode, refer to the *Atlas 200 AI Accelerator Module Hardware Development Guide (Model 3000)* for the design of the Atlas 200 hardware mother board, refer to the *Atlas 200 AI Accelerator Module 1.0.0.SPC200 or Later System Software Development Guide (Model 3000)* for setting up the development environment and execution environment, and refer to the *Atlas 500 Application Software Development Guide* for the service software development.

## Software Development in SoC Mode

The Atlas 200 SoC mode indicates that the Atlas 200 is used as the main processor or coprocessor. If the Atlas 200 is used as the main processor or coprocessor, the application software development processes of the main processor and coprocessor are the same. For details about the service software development method, see the content about software code development in this document.

## 6.3 Preparations

### 6.3.1 Obtaining the Sample Program

- Sample code  
Download URL: <https://gitee.com/HuaweiAtlas/samples>  
Usage: For details, see the **README.md** file of each sample provided on gitee.
- Demo code  
Download URL: <https://gitee.com/HuaweiAtlas>  
Usage: For details, see the **README.md** file of each demo provided on gitee.

## 6.4 Development Environment

This chapter describes the development environment after installation, including the distribution of header files, dynamic libraries, compilation tools, and other tools.

### 6.4.1 DDK Directory Distribution

#### Setting Environment Variables

After installing the DDK, you need to export the installation path of the DDK to the environment variable **DDK\_HOME**. The code implementation is as follows:

```
export DDK_HOME=/home/HwHiAiUser/tools/che/ddk/ddk
```

The preceding directory is the DDK installation directory. Change it based on the site requirements.

#### Directory Structure

The following shows the directory structure in **\$DDK\_HOME**. The main directories are described as follows:

- **include**: header file directory
- **host/lib**: library directory on the host
- **device/lib**: library directory on the device side

```
├── bin
├── conf
├── ddk_info
├── device
│ └── lib -> ../lib/aarch64-linux-gcc5.4
├── host
│ └── lib -> ../lib/aarch64-linux-gcc5.4
├── include
│ ├── inc
│ ├── libc_sec
│ └── third_party
├── lib
│ ├── aarch64-linux-gcc5.4
│ └── x86_64-linux-gcc5.4
├── packages
├── python3.5
├── sample
├── scripts
├── site-packages
├── toolchains
├── uihost
│ ├── bin -> ../bin/x86_64-linux-gcc5.4
│ ├── lib -> ../lib/x86_64-linux-gcc5.4
│ └── toolchains -> ../toolchains
```

### 6.4.2 Header Files and Libraries

#### Header Files

The **include** directory contains the following three files:



```

├── include
│ ├── inc
│ ├── libc_sec
│ └── third_party

```

- **include/inc**: contains header files of main functions of the DDK.
- **include/libc\_sec**: contains header files of security functions.
- **include/third\_party**: contains header files of third-party libraries.

## Dynamic Libraries

The Atlas 200 is not divided into master and slave modes. All programs use the AArch64-Linux compiler for cross compilation. Therefore, the **host/lib** and **device/lib** directories in the Atlas 200 DDK point to the same folder **lib/aarch64-linux-gcc5.4** that stores the library files required for cross compilation, including the third-party library files such as protobuf and OpenCV.

### 6.4.3 Compilation Tool

For the Atlas 200, programs use the AArch64-based GCC compilation toolchain for cross compilation. For details about how to install the cross compilation tool **aarch64-linux-gcc5.4**, see "Installing the Toolchain" in the *Atlas 200 AI Accelerator Module 1.0.0.SPC200 or Later System Software Development Guide (Model 3000)*. (The recommended installation path of the cross compilation tool is **/opt/atlas200\_compiler/bin.**)

```

├── /opt/atlas200_compiler/aarch64-linux-gnu
│ ├── bin
│ ├── include
│ ├── lib
│ ├── lib64
│ └── libc
├── /opt/atlas200_compiler/bin
│ ├── aarch64-linux-gnu-addr2line
│ ├── aarch64-linux-gnu-ar
│ ├── aarch64-linux-gnu-as
│ ├── aarch64-linux-gnu-c++
│ ├── aarch64-linux-gnu-c++filt
│ ├── aarch64-linux-gnu-cpp
│ ├── aarch64-linux-gnu-elfedit
│ ├── aarch64-linux-gnu-g++
│ ├── aarch64-linux-gnu-gcc
│ ├── aarch64-linux-gnu-gcc-5.4.1
│ ├── aarch64-linux-gnu-gcc-ar
│ ├── aarch64-linux-gnu-gcc-nm
│ ├── aarch64-linux-gnu-gcc-ranlib
│ ├── aarch64-linux-gnu-gcov
│ ├── aarch64-linux-gnu-gcov-dump
│ ├── aarch64-linux-gnu-gcov-tool
│ ├── aarch64-linux-gnu-gprof
│ ├── aarch64-linux-gnu-ld
│ ├── aarch64-linux-gnu-ld.bfd
│ ├── aarch64-linux-gnu-nm
│ ├── aarch64-linux-gnu-objcopy
│ ├── aarch64-linux-gnu-objdump
│ ├── aarch64-linux-gnu-ranlib
│ ├── aarch64-linux-gnu-readelf
│ ├── aarch64-linux-gnu-size
│ ├── aarch64-linux-gnu-strings
│ └── aarch64-linux-gnu-strip
├── /opt/atlas200_compiler/include
├── /opt/atlas200_compiler/lib
├── /opt/atlas200_compiler/libexec
└── /opt/atlas200_compiler/share

```

## 6.4.4 DDK Tools

DDK tool directory: `$DDK_HOME/bin/x86_64-linux-gcc5.4`

| Tool              | Description                               | Remarks                                                                                                                                         |
|-------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| IDE-daemon-client | IDE daemon command toolkit                | For details, see the <i>IDE-daemon-client Command Reference</i> .                                                                               |
| omg               | Model conversion tool                     | The Caffe or TensorFlow model can be converted into a .om model file supported by the DDK. For details, see the <i>Model Conversion Guide</i> . |
| protoc            | Tool for the third-party library protobuf | This tool converts .proto files into protocols supported by each language.                                                                      |

## 6.5 Inference Service Flow

This topic describes the typical service flow of the inference service.

### 6.5.1 Software Service Flow

Figure 6-2 Service flow



Figure 6-2 shows the software service flow. For details, see Table 6-2.

Table 6-2 Service flow description

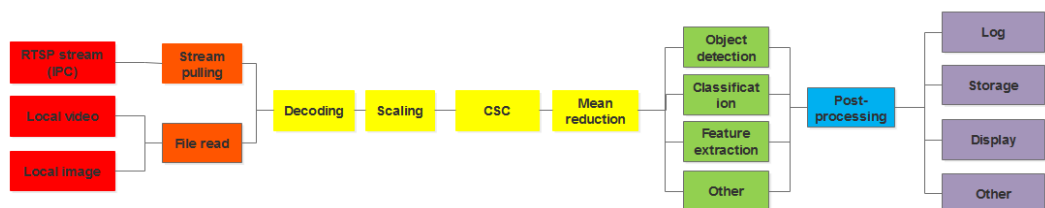
| Process        | Description                          | Remarks                                                                                                                                                      |
|----------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data source    | Indicates the video or image source. | The source can be RTSP streams from an IP camera (IPC) or offline videos or pictures in a disk.                                                              |
| Data obtaining | Implements data obtaining.           | You can pull streams from the open-source FFmpeg function library or using custom implementation code. The software for obtaining data runs on the host CPU. |

| Process             | Description                                                                   | Remarks                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data pre-processing | Implements image pre-processing functions such as decoding, scaling, and CSC. | <ul style="list-style-type: none"> <li>If software decoding is used, the open-source OpenCV function library is called. The data pre-processing software runs on the host CPU or Ctrl CPU.</li> <li>If hardware decoding is used, the API for DVPP decoding and AIPP CSC is called to implement pre-processing. Then the corresponding hardware modules are started on the device side.</li> </ul> |
| Inference           | Implements the model inference function.                                      | The function runs on the device-side AI Core or AI CPU.                                                                                                                                                                                                                                                                                                                                            |
| Post-processing     | Implements post-processing of model inference results.                        | The function runs on the device-side Ctrl CPU or host CPU.                                                                                                                                                                                                                                                                                                                                         |
| Output              | Displays the implementation result.                                           | -                                                                                                                                                                                                                                                                                                                                                                                                  |

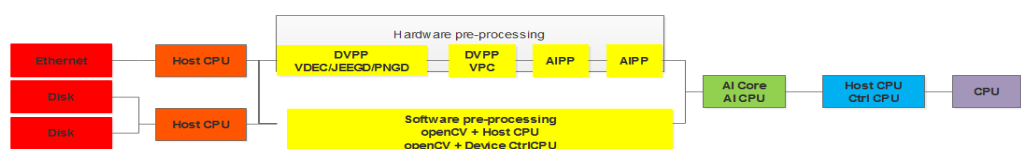
## 6.5.2 Example Mapping Between Software and Hardware Modules

The device uses an Arm processor as the Ctrl CPU. Due to its limited processing capability, you can implement common logic operations (such as pre-processing and post-processing) on the device side based on the service process, data transmission between the host and device, and Ctrl CPU load.

**Figure 6-3** Software modules



**Figure 6-4** Hardware modules



## 6.6 Code Execution Sample (HelloDavinci)

This chapter describes the entry-level sample HelloDavinci. This sample introduces the application development process based on the Atlas product and the method of verifying compilation environment and running environment configurations.

### 6.6.1 Obtaining the HelloDavinci Code

Download URL: <https://gitee.com/HuaweiAtlas/samples>

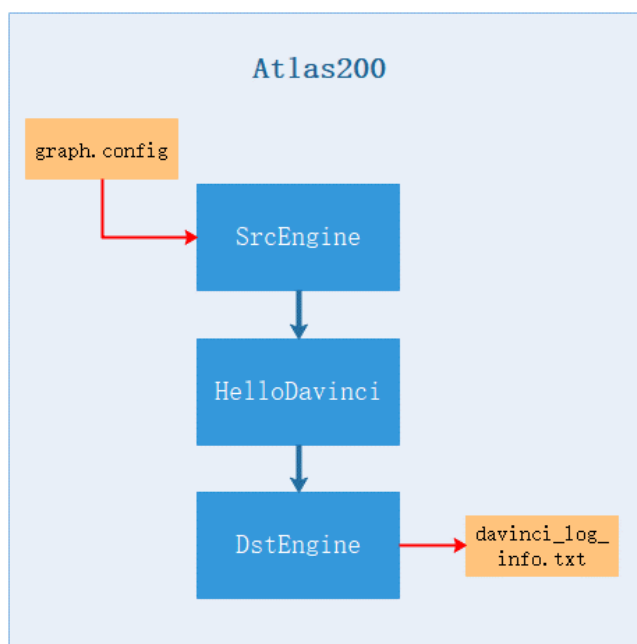
### 6.6.2 HelloDavinci Process Framework

The following uses the HelloDavinci program to demonstrate data transmission between engines.

#### NOTE

If you are familiar with this process, directly go to [6.6.3 HelloDavinci Compilation and Running](#) and check the running result.

**Figure 6-5** Running process



The execution process is as follows:

1. The **main** function is called to send data to SrcEngine.
2. After receiving the data, SrcEngine reads configurations in the **graph.config** file and sends them to HelloDavinci.
3. HelloDavinci generates a character string and sends it to DstEngine.
4. After receiving the data, DstEngine saves it to **dacvinci\_log\_info.txt** and sends a completion signal to the **main** function.

5. After the **main** function receives the completion signal, the graph is destroyed, information indicating the operation completion is printed on the terminal, and the program exits.
6. The program exits.

## 6.6.3 HelloDavinci Compilation and Running

### Prerequisite

1. The environment has been successfully installed.
2. CMake 2.8.4 or later has been installed.
3. The DDK has been installed.
4. The environment variable (**DDK\_HOME: export DDK\_HOME=<DDK install Path>/ddk/**) has been configured.

### Compilation Procedure

- Step 1** Copy the obtained samples file to a directory on the development host.
- Step 2** Go to the **Samples/HelloDavinci** directory.
- Step 3** Run the **bash build.sh A200** command to perform compilation. After the compilation is successful, the executable files **main** and **so** and the configuration file **graph.config** are generated in the **out/** directory.
- Step 4** Run the **scp -r out HwHiAiUser@A200IP:~/** command (*A200IP* indicates the IP address of the Atlas 200 system) to transfer the file to the Atlas 200 system.
- End

### Software Running

- Step 1** On the Atlas 200 system, run the following command (*ip* indicates the IP address of the Atlas 200 system):
- ```
ssh HwHiAiUser@ip
```
- Step 2** Enter the password to log in to the Atlas 200 system.
- Step 3** Run the **./main** command in the **out** directory. If the following information is displayed, the execution is successful:
- If the command is successfully executed, the following information is displayed:

```
HwHiAiUser@davinci-mini:~/out$ ./main
Hello Davinci!
The sample end!!
```
 - In the **dacvinci_log_info.txt** file in the same directory, the following information is displayed:

```
HwHiAiUser@davinci-mini:~/out$ cat dacvinci_log_info.txt
This message is from HelloDavinci
```
- End

6.7 Software Code Development

With the integrated Ascend 310 chip, the Atlas 200 provides robust computing power to accelerate the inference of neural networks.

6.7.1 Configuring the Matrix Framework

The Matrix framework enables developers to quickly migrate inference services and efficiently use the computing power of the Ascend 310 chip. The Matrix framework consists of graphs and engines.

An engine is defined as a basic functional unit of a service, and its implementation can be customized (for example, inputting image data, classifying images, and outputting the predicted class of images). By default, each engine corresponds to a thread.

A graph manages a service flow composed of multiple engines. Each graph starts a process on the Ascend 310 to implement the corresponding service. It describes data transmission relationships between engines.

Figure 6-6 Relationship between the graph and engines

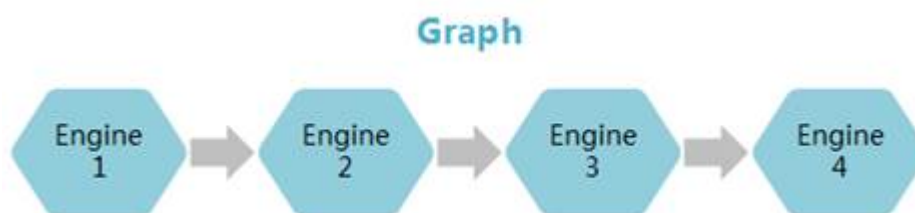
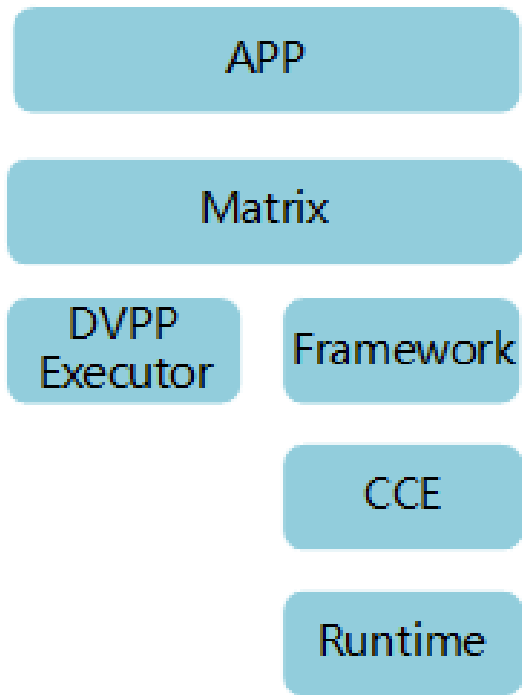


Figure 6-7 shows the logical architecture of service software.

- The Matrix is a universal service flow execution engine that provides functional APIs for the Ascend 310 chip to exchange control commands and processed data with apps. It establishes service flows based on configuration files and destroys them based on commands. Engines call the APIs of the DVPP to implement media pre-processing, and call the APIs of the model manager to implement model inference.
- The DVPP Executor provides APIs for video encoding/decoding, image encoding/decoding, and vision pre-processing (including cropping and scaling).
- The Framework is an offline model framework that provides APIs for offline model conversion, offline model execution, and custom operator implementation.
- The cube-based computing engine (CCE) acceleration library provides acceleration for upper-layer applications (Framework or Matrix) through APIs.
- The Runtime runs in the app process and provides Ascend 310-powered devices with various functions, such as memory management, device management, stream management, event management, and kernel execution.

Figure 6-7 Logic of the service software



6.7.1.1 Configuring, Creating, and Destroying a Graph

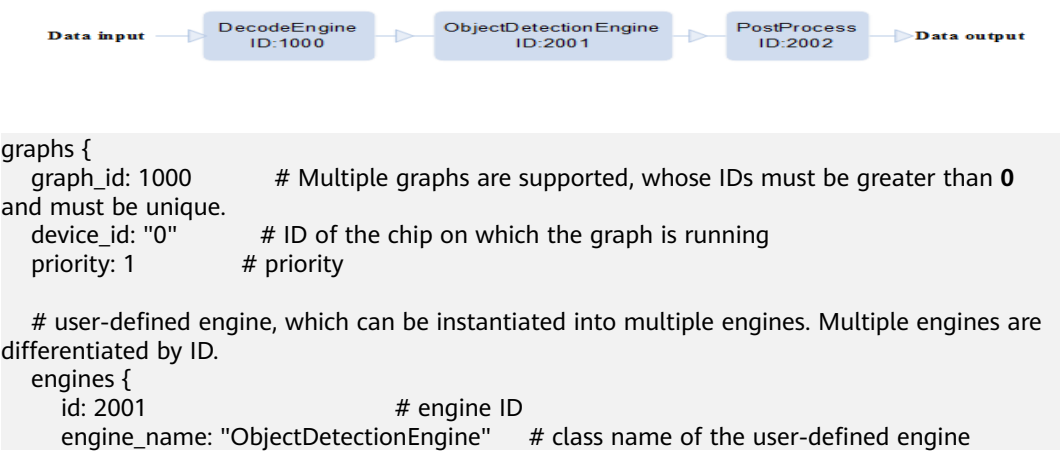
The graph describes the data transmission relationship between engines in a service. The Matrix framework defines the graph data structure in a protobuf file. You can define the graph configurations in the configuration file.

NOTE

For details about the graph keywords, see 6.8 Graph Keywords. For more information, see the *Matrix API Reference*.

The following provides a simple graph configuration file, which will create a graph service flow with the ID of 1000. The service flow contains three engines. Figure 6-8 shows the data transmission relationship between the engines.

Figure 6-8 Data transmission between engines



```

side: DEVICE                # whether the engine runs on the host or device
so_name: "./libObjectDetectionEngine.so" # name and path of the dynamic link library
# user-definable parameters
ai_config{
  items{
    name: "model"           # model name
    value: "./FaceDetection.om" # model path
  }
  items{
    name: "mode"
    value: "test"
  }
}
}
engines {
  id: 1000
  engine_name: "DecodeEngine"
  side: DEVICE
  so_name: "./libDecodeEngine.so"
}
engines {
  id: 2002
  engine_name: "PostProcess"
  side: HOST
}

# Describes interface connections between engines.
connects {
  src_engine_id: 1000          # source engine ID for data transmission
  src_port_id: 0               # source port ID for data transmission
  target_engine_id: 2001       # target engine ID for data transmission
  target_port_id: 0           # target port ID for data transmission
}
connects {
  src_engine_id: 2001
  src_port_id: 0
  target_engine_id: 2002
  target_port_id: 0
}
}

```

The Matrix framework copies the dynamic link library (specified by **so_name**) and model file (added in **ai_config**) that device-side engines depend on to the device side for engine threads. [Table 6-3](#) lists three frequently used graph APIs provided by the Matrix. For details, see the *Matrix API Reference*.

Table 6-3 API description

API	Description
HIAI_StatusT HIAI_Init(uint32_t deviceId)	Initializes an Ascend 310 chip. Note that the chip ID is the chip number of Ascend 310. The Atlas 200 packs only one Ascend 310 chip. Therefore, the chip ID must be 0, which is different from the Atlas 300 AI Accelerator Card (Model 3010) (Model 3000).

API	Description
static HIAI_StatusT Graph::CreateGraph(const std::string& configFile)	Reads the graph configuration file, initializes engines, and creates threads and channels for data transmission to initialize the service flow.
static HIAI_StatusT Graph::DestroyGraph(uint32_t graphID)	Destroys the graph and runs the destructor functions of engines.

6.7.1.2 Configuring Engine

An engine is a basic functional unit of service software defined by the Matrix framework. Users inherit the engine template class defined by the Matrix framework and create engines for each functional module in services (such as reading input files, image pre-processing, neural network inference, inference result post-processing, and host/device data transmission). For details about the functional modules, see the *Matrix API Reference*. Each engine defines the **Init()** and **Process()** functions. When the graph is initialized, the **Init()** function is automatically executed to initialize engine parameters (including memory allocation and model loading). The **Process()** function implements data transmission and service logic.

- [Table 6-4](#) describes the common APIs.

Table 6-4 API description

API	Description	Remarks
HIAI_DEFINE_PROCESS (inputPortNum, outputPortNum)	Specifies the number of interfaces for inputting and outputting engine data.	The Matrix framework creates a queue for each interface to cache data. The transmission relationship between engines is specified in the graph configuration file. For details, see 6.8 Graph Keywords .
HIAI_StatusT Engine::Init(const AIConfig &config, const vector<AIModelDescripti on>&modelDesc)	Initializes the engine.	During graph creation, this API is called to initialize engines. The ai_config value defined in the graph configuration file is transferred to the argument config of the function. You can add custom items to the configuration file and use them in the initialization function.

API	Description	Remarks
HIAI_IMPL_ENGINE_PROCESS(name, engineClass, inPortNum)	Indicates the process of an engine.	The Process() function of the engine is driven by data on the device side. That is, after the input interface receives data, the framework starts the process. If multiple input interfaces are configured, each input interface triggers a process after receiving data. Therefore, if service processing depends on multiple inputs, you need to implement the synchronization logic of multiple inputs. The framework has encapsulated the Process() function into a macro definition for implementation.

- The engine reads data from the input interface. The framework supports a maximum of 16 input interfaces (arg0–arg15) that can be directly used. From the perspective of service applications, shared pointers are transmitted. The following shows the transmission code sample:

```
// Transmission engine: transmits the user-defined data USER_DEFINE_TYPE to the target engine.
std::shared_ptr<USER_DEFINE_TYPE> streamData= std::make_shared< USER_DEFINE_TYPE>();
// After a value is assigned to streamData, call Senddata to send the value. The shared pointer needs to be converted to the void type for data transmission.
hiai::Engine::SendData(0, " USER_DEFINE_TYPE ",
std::static_pointer_cast<void>(deviceStreamData));
// Receiving engine: receives data and converts it to the user-defined type USER_DEFINE_TYPE.
std::shared_ptr< USER_DEFINE_TYPE> inputArg = std::static_pointer_cast<
USER_DEFINE_TYPE>(arg0);
```

6.7.1.3 Configuring Data Transmission

Based on service applications, data transmission defined by the Matrix framework is classified into the following types (The used APIs are similar and the transmitted objects are shared pointers, but the API usage varies according to scenarios):

- Input API for transmitting data outside a graph to engines. For details, see "Graph::SendData" in the *Matrix API Reference*.
- Output API for transmitting data of engines in a graph to the outside of the graph. For details, see "Graph::SetDataRecvFunctor" and "Engine::SetDataRecvFunctor" in the *Matrix API Reference*.
- Data is transmitted between engines in a graph. For details, see "Engine::SendData" in the *Matrix API Reference*.

Data Transmission Between Engines

The Atlas 200 involves only the intra-side data transmission. The transferred data is the address of the shared pointer and is not copied. As the **Process()** function of

engines is instantiated into threads, intra-side transmission implements data transmission in memory-sharing mode. Ensure that there is no unauthorized access. The Matrix framework recommends that the local engine does not modify the shared pointer after transmitting it to the next engine.

Input API for Transmitting Data Outside a Graph to Engines

- The service flow composed of engines is driven by data. This API allows data outside a graph to be sent to engines in the graph.
- The following APIs can be called to implement cross-side and intra-side transmission, with similar requirements to those for data transmission between engines. For details, see "Graph::SendData" in the *Matrix API Reference*.

```
HIAI_StatusT Graph::SendData(const EnginePortID& targetPortConfig, const
std::string& messageName, const std::shared_ptr<void>& dataPtr, const uint32_t
timeOut = 500)
```

Output API for Transmitting Data of Engines in a Graph to the Outside of the Graph

- The Matrix framework transmits the data of engines in a graph to the outside of the graph using either of the following callback functions:
 - Callback on the output API for an engine (node of a service flow). For details, see "Graph::SetDataRecvFunc" in the *Matrix API Reference*.
 - For details about the callback on the output API in other scenarios, see "Engine::SetDataRecvFunc" in the *Matrix API Reference*.
- The framework provides the template class **DataRecvInterface** of callback functions.

6.7.2 Using DVPP APIs

DVPP is an image pre-processing hardware acceleration module provided by Ascend 310. This module integrates the following six functions. For details about the APIs and their usage, see the *DVPP API Reference*.

- Format conversion, image cropping, and scaling (by the VPC)
- H.264/H.265 video decoding (by the VDEC)
- H.264/H.265 video encoding (by the VENC)
- JPEG image decoding (by the JPEGD)
- JPEG image encoding (by the JPEGE)
- PNG image decoding (by the PNGD)

6.7.2.1 Using DVPP APIs

The DVPP provides the following three types of APIs by using handles, namely, APIs for creating, using, and destroying handles.

The VPC, JPEGE, JPEGD, and PNGD components share the same set of APIs with varying input parameters. VDEC and VENC each have a separate set of APIs.

- VDEC uses the following three APIs for decoding, which can be called in asynchronous mode. The **VdecCtl** API is used to transfer configurations

including the callback function) and H.264/H.265 data. After hardware decoding, the Matrix framework calls the callback function to return the result.

NOTE

The data decoded by the VDEC is in HFBC format (internal format), which has to be converted to the YUV420SP format using the VPC. For details, see samples in "Implementing the VDEC Function" of the *DVPP API Reference*.

```
int CreateVdecApi(IDVPPAPI *&pIDVPPAPI, int singleton)
int VdecCtl(IDVPPAPI *&pIDVPPAPI, int CMD, dvppapi_ctl_msg *MSG, int singleton)
int DestroyVdecApi(IDVPPAPI *&pIDVPPAPI, int singleton)
```

- The VPC, JPEG, JPEGD, and PNGD components all use the following APIs. The configuration parameters to be transferred vary according to functions. For details, see "VPC/JPEG/JPEGD/PNGD Function Interfaces" in the *DVPP API Reference*.

```
int CreateDvppApi(IDVPPAPI *&pIDVPPAPI)
int DvppCtl(IDVPPAPI *&pIDVPPAPI, int CMD, dvppapi_ctl_msg *MSG)
int DestroyDvppApi(IDVPPAPI *&pIDVPPAPI)
```

- The DVPP is restricted by hardware during its usage. To speed up data read and write, an image's length and width must be aligned to the specified size without affecting the valid region. The image's length and width are aligned to the specified size by padding 0s to leftward and downward.

For example, for a 300 x 300 YUV420SP_UV image, the size must be aligned to 304 x 300 (The width is 16-pixel aligned, and the height is 2-pixel aligned). The valid region ranges from [0, 0] to [300, 300]. In this case, you need to pad 0s rightward to column 304.

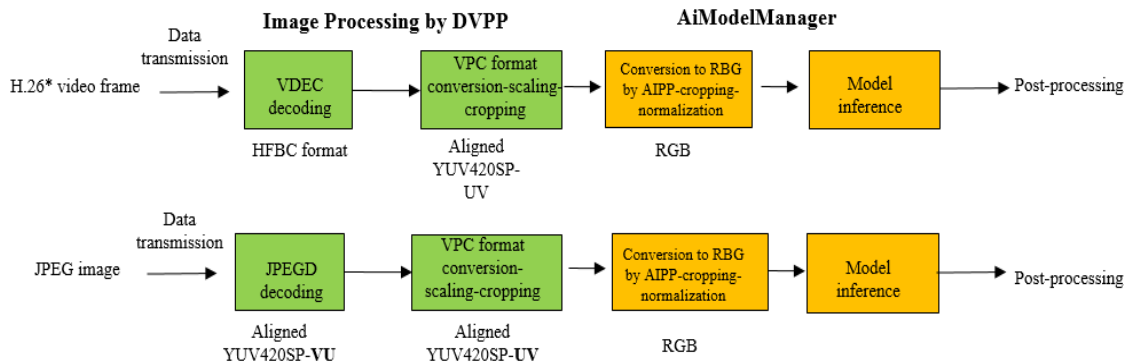
- When the JPEGD, VDEC, and PNGD components of the DVPP are used to read input images, the decoded images must meet the length and width alignment requirements. In this case, you need to apply for memory for output images based on the size of the aligned images.

For example, for a 300 x 300 YUV420SP_UV image, you need to apply for memory with the size of (304*300*3/2) bytes. Each pixel of a YUV420SP image requires a 1.5-byte storage space.

- VPC: input and output memory address aligned by 16 bytes
- VPC: output image width aligned by 16 bytes
- VPC: output image height aligned by two bytes
- VPC: input image width aligned by 16 bytes
- VPC: input image height aligned by two bytes
- JPEGD: output image width aligned by 128 bytes
- JPEGD: output image height aligned by 16 bytes
- DVPP components pose many restrictions on output images based on the processing speed and memory usage. For example, the length and width of output images must be aligned, and the output format must be **YUV420SP**. However, the model input is usually in RGB or BGR format, and the sizes of the input images are different. Therefore, the Ascend 310 chip provides AI pre-processing (AIPP) for image format conversion and image cropping. For details, see the *Model Conversion Guide*.

Figure 6-9 shows the handling process of the JPEG image input and H.26* video input.

Figure 6-9 Handling process of video and image inputs



6.7.2.2 Applying for DVPP Memory

The Matrix framework provides the memory allocation API **HIAI_DVPP_DMalloc** and memory freeing API **HIAI_DVPP_DFree**. **HIAI_DVPP_DMalloc** is used to allocate memory that meets DVPP alignment requirements. The two APIs must be used in pairs. To prevent memory leakage, you are advised to use the shared pointer to manage the allocated memory. The implementation code is as follows:

```

uint8_t* buffer = nullptr;
HIAI_StatusT ret = hiai::HIAIMemory::HIAI_DVPP_DMalloc(dataSize, (void*)& buffer);
std::shared_ptr<uint8_t> dataBuffer = std::shared_ptr<uint8_t>( buffer, \
[] (std::uint8_t* data) hiai::HIAIMemory::HIAI_DVPP_DFree(data));

```

In addition, these APIs are available only on the device side. The memory allocated by **HIAI_DVPP_DMalloc** can be used for high-speed data transmission from the device to host. In this mode, the memory is freed by the Matrix framework. For details, see the sample code `DvppDecodeResize`.

6.7.3 Offline Model Inference

The Ascend 310 chip supports inference acceleration for the Caffe and TensorFlow frameworks. After model training is complete, you need to convert the trained model to the model file (.om file) supported by Ascend 310, compile service code, and call APIs provided by the Matrix framework to implement service functions. The Matrix framework encapsulates the AIPP and model inference functions into a module. When a user calls the inference API, the Matrix framework calls the AIPP first to pre-process input images, sends the pre-processed images to the model inference module, and returns the inference result.

6.7.3.1 Configuring AIPPs

AIPP is a hardware image pre-processing function provided by Ascend 310. The pre-processing includes CSC, image normalization (by subtracting the mean value or multiplying a coefficient), image cropping (by specifying the start point of cropping and cropping the image to the size required by the neural network), and much more.

AIPP supports static and dynamic modes.

- **Static AIPP:** In this mode, parameters are set during model conversion. The model inference process uses fixed AIPP pre-processing (cannot be modified). For details about the keywords and configuration file template of static AIPP, see "Configuration File Template" in the *Model Conversion Guide*.
- **Dynamic AIPP:** During model conversion, AIPP is set to dynamic mode. Before model inference, set AIPP pre-processing parameters as required. Dynamic AIPP is used when pre-processing parameters have to be changed based on service requirements. For example, cameras use different normalization parameters, and the input image format must be compatible with YUV420 and RGB. For details about how to use dynamic AIPP, see "AIPP Configuration APIs" in the *Matrix API Reference*.

NOTICE

- The input format of AIPP is **YUV420SP_U8** (the default format is **YUV420SP_UV**). If the format is **YUV420SP_VU**, change the value of **rbuv_swap_switch**. Otherwise, the output result will be affected.
- The model input is in either **RGB_U8** or **BGR_U8** format, corresponding to different color gamut conversion matrices.

Images output by the DVPP module are the aligned YUV420SP images. The output does not support the RGB format. Therefore, the service flow needs to use converted and aligned YUV420SP images of the AIPP module, and crop the image to the size in line with the model input.

For example, the model requires the input of a 300 x 300 RGB image. After DVPP APIs are called for processing (such as JPEG decoding and scaling), the DVPP module outputs a 384 x 304 image YUV420SP_UV image (the valid region size is 300 x 300, and 0s are padded to rightward and downward).

The following shows the static AIPP configuration. The file configures the start coordinates of the image to be cropped. The length and width of the image to be cropped are set according to the model input by default. The image normalization parameters are the mean value and the reciprocal of the variance (The final value is obtained by subtracting the mean value and multiplying this coefficient).

```
aipp_op{
# Sets AIPP to static mode.
aipp_mode: static
# Enables image cropping.
crop: true
# Sets the format and size for an input image.
input_format : YUV420SP_U8
src_image_size_w : 384
src_image_size_h : 304
# Sets the start coordinates for cropping. The width and height of the cropped region are in line
with the model input by default.
load_start_pos_h : 0
load_start_pos_w : 0

# Enables format conversion. The conversion matrix converts YUV420SP_UV to RGB888.
csc_switch : true
matrix_r0c0 : 298
matrix_r0c1 : 516
```

```
matrix_r0c2 : 0
matrix_r1c0 : 298
matrix_r1c1 : -100
matrix_r1c2 : -208
matrix_r2c0 : 298
matrix_r2c1 : 0
matrix_r2c2 : 409
input_bias_0 : 16
input_bias_1 : 128
input_bias_2 : 128

# Enables data normalization and configure the mean value and the reciprocal of variance.
mean_chn_0 : 125
mean_chn_1 : 125
mean_chn_2 : 125
var_reci_chn_0 : 0.0039
var_reci_chn_1 : 0.0039
var_reci_chn_2 : 0.0039
}
```

6.7.3.2 Converting an Offline Model

There are two methods of evaluating whether trained Caffe and TensorFlow models are supported by Ascend 310:

1. Check whether the operators used by a trained model are all included in the operator list supported by Ascend 310. If yes, Ascend 310 supports this trained model.
2. Check whether a trained model can be successfully converted using the model conversion tool. If the conversion is successful, Ascend 310 supports this trained model. For unsupported operators, an error message is returned.

NOTE

1. For details about the operator list, see the *Operator List*.
2. For unsupported operators, you can customize them by referring to the *TE Custom Operator Development Guide*.

The offline model conversion tool is stored in the `<SDDK_HOME>/uihost/bin/omg` directory. The offline model generator (OMG) tool is a CLI tool (parameters can be obtained by using the `-h` command). It is used to convert models under Caffe and TensorFlow frameworks into .om files supported by Ascend 310. For details about how to use the OMG tool, see "Model Conversion Using OMG" in the *Model Conversion Guide*.

1. Caffe model conversion:

```
#omg --framework 0 --model <model.prototxt> --weight <model.caffemodel> --output <output name> --insert_op_conf <aipp.cfg>
```
2. TensorFlow model conversion:

```
#omg --framework 3 --model <model.pb> --input_shape "input_name:1,112,112,3" --output <output_name> --insert_op_conf <aipp.cfg>
```

Table 6-5 Parameter description

Parameter	Description
framework 0	Indicates a Caffe model.

Parameter	Description
framework 3	Indicates a TensorFlow model.
model	Specifies a model file.
weight	Specifies a weight file for the Caffe model.
output	Specifies a name of the .om file.
input_shape	Specifies the name and size of the input layer. The default value for TensorFlow models is input_layer_name: n, h, w, c.
insert_op_conf	Specifies an AIPP configuration file.

6.7.3.3 Performing Model Inference

The Matrix framework provides the **AIModelManager** class to implement model loading and inference. For details, see the *Matrix API Reference*.

Model Inference Initialization

- Step 1** Set the model path in the graph configuration file of the custom inference model engine (add items to **ai_config** and set the model path on the host).
- Step 2** Use the Matrix framework to transfer the model file to the device.
- Step 3** Parse the custom items in the custom engine to obtain the path of the model on the device side.
- Step 4** Call **AIModelManager::Init()** to complete the initialization. The implementation is as follows:

```
/* Define member variables of a custom engine. */
std::shared_ptr<hiiai::AIModelManager> modelManager;
/* Implement AIModelManager initialization with the Init function of the custom engine. */
std::vector<hiiai::AIModelDescription> model_desc_vec;
hiiai::AIModelDescription model_desc_;
.....
/* Parse the model path from the ai_config structure in the graph configuration file. */
model_desc_.set_path(model_path);// Set the model path.
model_desc_vec.push_back(model_desc_);
ret = modelManager->Init(config, model_desc_vec);// If configurations are meaningless, input
the arguments of Engine::Init.
```

----End

Setting the Input and Output of Model Inference

- The Matrix framework defines the **IAITensor** class for managing the input and output matrices of model inference. For ease of use, the Matrix framework derives **AISimpleTensor** and **AINeuralNetworkBuffer** based on the **IAITensor** class.

- Memory for the model inference input and output is allocated by calling **HIAI_DMAlloc**, which reduces memory copy.
- Even though the Matrix framework can automatically free the memory managed by **AI SimpleTensor**, you are advised to apply for and free the memory by yourself to prevent memory leakage or repeated freeing.
- During model conversion, if the functions such as image cropping, format conversion, and image normalization of the AIPP module are enabled, the input data must be processed by the AIPP module before it is used for model inference.

Input and Output Implementation

The code for input and output implementation is as follows:

```
/* Obtain the descriptions of input and output tensors of the inference model. */
std::vector<hiai::TensorDimension> inputTensorDims;
std::vector<hiai::TensorDimension> outputTensorDims;
ret = modelManager->GetModelIOTensorDim(modelName, inputTensorDims,
outputTensorDims);

/* Set the input. If there are multiple inputs, create and set them in sequence. */
std::shared_ptr<hiai::AISimpleTensor> inputTensor =
std::shared_ptr<hiai::AISimpleTensor>(new hiai::AISimpleTensor());
inputTensor->SetBuffer (< memory address of the input data >, < length of the input data >);
inputTensorVec.push_back(inputTensor);

/* Set the output. */
for (uint32_t index = 0; index < outputTensorDims.size(); index++) {
hiai::AITensorDescription outputTensorDesc = hiai::AINeuralNetworkBuffer::GetDescription();
uint8_t* buf = (uint8_t*)HIAI_DMAlloc(outputTensorDims[index].size);
.....
std::shared_ptr<hiai::AITensor> outputTensor = hiai::AITensorFactory::GetInstance()-
>CreateTensor(
outputTensorDesc, buf, outputTensorDims[index].size);
outputTensorVec.push_back(outputTensor);
}
```

Model Inference

- The Matrix framework supports either synchronous inference and asynchronous inference. Synchronous inference is used by default. You can set the **AIContext** configuration item and call the callback function to implement asynchronous inference.

```
/* Model inference */
hiai::AIContext aiContext;
HIAI_StatusT ret = modelManager->Process(aiContext, inputTensorVec, outputTensorVec,
0);
```

- If the **AIModelManager** object loads multiple models, you can set the **AIContext** configuration item to set model parameters (initialization phase and model name). For details, see "Offline Model Manager" in the *Matrix API Reference*.

Model Inference Post-Processing

The result matrix of model inference is stored in the **IAITensor** object as the memory+description information. You need to parse the memory information into

valid output based on the actual output format (data type and data sequence) of the model.

```
/* Parse the inference result. */
for (uint32_t index = 0; index < outputTensorVec.size(); index++) {
    shared_ptr<hiaai::AINeuralNetworkBuffer> resultTensor =
        std::static_pointer_cast<hiaai::AINeuralNetworkBuffer>(outputTensorVec[i]);
    // resultTensor->GetNumber() -- N
    // resultTensor->GetChannel() -- C
    // resultTensor->GetHeight() -- H
    // resultTensor->GetWidth() -- W
    // resultTensor->GetSize() -- memory size
    // resultTensor->GetBuffer() -- memory address
}
```

NOTE

For details about the post-processing of common classification models, see the sample code `InferClassification`. For details about the post-processing of the SSD object detection model, see the sample code `InferObjectDetection`.

6.7.4 Optimizing Software Performance

Performance optimization (memory usage, data transmission, and high-performance operator selection) must be considered during software service flow development. For details about high-performance software programming, see the *High Performance Application Programming Guide*.

6.7.5 Commissioning Software Logs

This topic describes how to write code for printing and viewing logs during Atlas software development. The code is used to print debugging logs during software development and record exceptions during software running.

6.7.5.1 Registering Log Levels

Before using log APIs, register error codes. The APIs for log registration must be defined in the header file `hiaaiengine/status.h`. The procedure is as follows:

Step 1 Define the ID of a log module.

```
#define USE_DEFINE_ERROR 0x6001
```

ID of the log module. The value **0x6001** is for reference only, which can be modified as required. The value cannot be the same as **MODID_GRAPH** in the `status.h` file.

Step 2 Customize a name for the error code.

```
enum{
    HIAI_INVALID_INPUT_MSG_CODE = 0 // error code name. The value 0 is a custom value.
};
```

Step 3 Registers error codes.

Function format:

```
HIAI_DEF_ERROR_CODE(moduleId, logLevel, codeName, codeDesc)
```

Table 6-6 Parameter description

Parameter	Description
moduleId	Component ID
logLevel	Error level
codeName	Name of an error code
codeDesc	Description of an error code. The value is a character string.

API calling example:

```
HIAI_DEF_ERROR_CODE(USE_DEFINE_ERROR, HIAI_ERROR, HIAI_INVALID_INPUT_MSG, "invalid  
input message pointer");
```

----End

6.7.5.2 Printing Logs

You can call different **HIAI_ENGINE_LOG_IMPL** functions based on the input parameters through **HIAI_ENGINE_LOG** to print the related API definitions in **log.h**. There are eight log formats. This section describes one of them based on the log registration content.

- Function format:
#define HIAI_ENGINE_LOG(...) \
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##_VA_ARGS_)
void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int lineNumber,
const uint32_t errorCode, const char* format, ...);

Table 6-7 Parameter description

Parameter	Description
errorCode	Error code
format	Log description
...	This is a variable parameter in the format and is added based on the log content.

- Calling sample
HIAI_ENGINE_LOG (HIAI_INVALID_INPUT_MSG, "RUNNING OK");

6.7.5.3 Viewing Logs

Changing the Log Level

The framework provides five log levels: error > warning > info > debug > event.

Event logs record the most critical logs of the system and must be set separately. For the other four log levels, logs of the specified level and higher can all be printed.

1. Log in to the Atlas 200.
2. Change the value of **global_level** in the **/etc/slog.conf** file to change the log level. [Table 6-8](#) describes the value of **global_level** corresponding to each log level.

For example, if the value of **global_level** is **3**, the log level is **error**. In this case, only logs at the error level are recorded. You need to configure the log level on the host and device separately.

Table 6-8 Log levels

Value	Level
0	DEBUG
1	INFO
2	WARNING
3	ERROR

The following is a code sample:

```
# Global log level
global_level=3
# User
user=HwHiAiUser
# Share memory size of node 512k * 32 biggest support
maxNodeSize=524272
# Share memory count of queue
maxQueueCount=40
# log-agent-host #
logAgentMaxFileNum=8
# set host one log file max size, range is (0, 104857600]
logAgentMaxFileSize=10485760
# set host log dir
logAgentFileDir=/var/dlog
...
```

Viewing Logs

- Log path: **/var/dlog**
- Log file name format: host-*x**.log (*x* indicates the chip ID.)
- Log compression: Log files are compressed by default and cannot be viewed directly. To view log files, set the **slog.conf** file or contact support engineers for a dedicated decompression tool first. Check whether the value of **zip_switch** is **0** in the **/etc/slog.conf** file. If the value is not **0**, set it to **0** and restart the slogd process to make the settings take effect. For details about how to restart the slogd process, see [6.7.5.4 Restarting Log-related Processes](#).

The **/etc/slog.conf** file content is as follows:

```
...
#log server sends logs to IDE
#IP_address to bind: if you want to set ip_address manually, please open the item and add
your ip.
#log_server_ip = 127.0.0.1
# Port: Please select a port number between 18000 and 18080 ([18000, 18080]).
daemon_socket_port=18080
#enable print event log, 0: disable, 1: enable
enableEvent=0
#zip switch, default: not zip
#0: not zip
#1: zip
zip_switch = 0
#zip level: 0-9, others err and exit, default: 0
#zip_level = 0
...
```

6.7.5.4 Restarting Log-related Processes

- Step 1** Run the following commands to query the IDs of the slogd and IDE-daemon-host processes:

```
HwHiAiUser@davinci-mini:/var/dlog$ ps aux |grep slogd
HwHiAiU+ 667 0.0 0.0 237652 4952 ? Ssl 15:37 0:00 /var/slogd
syslog 1732 0.0 0.0 256400 4320 ? Ssl Aug14 0:11 /usr/sbin/rsyslogd -n
HwHiAiU+ 44956 0.0 0.0 21292 944 pts/14 S+ 15:51 0:00 grep --color=auto slogd
HwHiAiUser@davinci-mini:/var/dlog$ ps aux |grep IDE
HwHiAiU+ 34640 0.0 0.0 39864 6304 ? Ss 15:48 0:00 /var/IDE-daemon-device
HwHiAiU+ 61918 0.0 0.0 21292 1024 pts/14 S+ 15:56 0:00 grep --color=auto IDE
HwHiAiU+ 215378 0.2 0.0 1401252 18340 ? Ssl 15:33 0:03 /var/IDE-daemon-host
```

- Step 2** Run the following command to switch to the **HwHiAiUser** user of the process:

```
su HwHiAiUser
```

- Step 3** Run the following commands to end the slogd and IDE-daemon-host processes:

```
kill 667
```

```
kill 215378
```

- Step 4** Run the following commands to restart the two processes for the modification to take effect:

```
cd /var
```

```
./slogd
```

```
./IDE-daemon-host
```

```
----End
```

6.7.6 Service Software Compilation

Service software requires the AArch64-based compilation tool and dynamic link libraries (DLLs) for cross compilation. You are advised to compile the main function file as an executable file and all engines as a DLL. Therefore, you can divide a service into one executable file and multiple DLLs. That is, one engine corresponds to one DLL.

For example, compile a .main file as an executable file and each engine as a DLL.

NOTE

It is recommended that the compilation system of service software be set up by using CMake. For details about setting up a compilation system using the CMake, see the HelloDavinci sample.

6.8 Graph Keywords

Parameter		Description	Mandatory (M) or Optional (O)
graph_id	-	Graph ID, which is a positive integer	O
priority	-	Priority, which does not need to be adjusted	O
device_id	-	Device ID. Different graphs can run on one or more chips or on the chips of different PCIe cards. If they run on different chips, the device_id field has to be added to the graph configuration file to specify the device ID on which the graphs are running. If device_id is not specified, the graphs are running on the chips of device 0 by default. The value of device id ranges from 0 to N-1 (N indicates the number of devices).	O
engines	id	Engine ID	M
	engine_name	Engine name	M
	side	Target for the engine to run. The value can be HOST or DEVICE , which is determined based on service requirements.	M
	so_name	During engine running, you need to copy the dynamic library file in .so format from the host to the device. If FrameworkerEngine running depends on a third-party library file or user-defined library file, the dependent .so file must be configured in the graph file. (Replace xxx in the following example with the actual name of the dependent library.) so_name: "/libFrameworkerEngine.so" so_name: "/libxxx.so" so_name: "/libxxx.so"	O

Parameter		Description	Mandatory (M) or Optional (O)
NOTE You are advised to configure multiple engines for multi-channel decoding. One engine corresponds to one thread. If one engine corresponds to multiple threads, the sequence cannot be ensured during decoding.	thread_num	Number of threads. You are advised to set this parameter to 1 for multi-channel decoding. If the value of thread_num is greater than 1 , the decoding sequence of threads cannot be ensured.	M
	thread_priority	Thread priority. The value ranges from 1 to 99 . This parameter is used to set the priority of the data processing thread corresponding to the engine. A higher priority is configured for the corresponding engine according to the SCHED_RR scheduling policy.	O
	queue_size	Queue size. The default value is 200 . The parameter value needs to be adjusted based on the service load fluctuation, size of data received by the engine, and system memory.	O
	ai_config	Configuration example: <pre>ai_config{ items{ name: "model_path" value: "./test_data/model/resnet18.om" } }</pre> name does not need to be set. - Set the value to the path of the model file, including the file name. The file name can contain only digits, letters, underscores (_), and dots (.). Alternatively, you can set the value to the path of a single model file, for example, ./test_data/model/resnet18.om. You can also compress the model file into a .tar package and set the parameter to the path ./test_data/model/resnet18.tar where the .tar package is stored. If there are multiple AI configuration items, ensure that files with the same name but in different formats (for example, ./test_data/model/test.zip and ./test_data/model/test.tar) do not exist in the same directory.	O

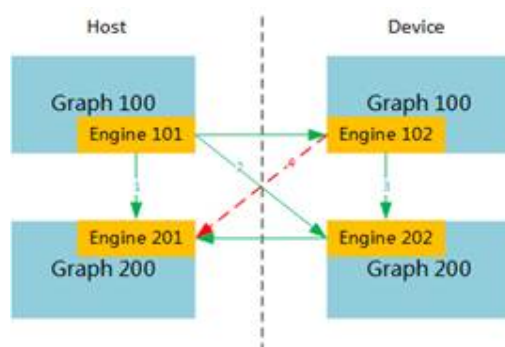
Parameter		Description	Mandatory (M) or Optional (O)
	ai_model	Configuration example: <pre>ai_model{ name: "" // model name type: "" // model type version: "" // model version size: "" // model size path: "" // model path sub_path: "" // auxiliary model path key: "" // model key sub_key: "" // auxiliary model key frequency: "UNSET" // device frequency. Value UNSET or 0 indicates that the setting is canceled. Value LOW or 1 indicates low frequency. Value MEDIUM or 2 indicates medium frequency. Value HIGH or 3 indicates high frequency. device_frameworktype: "NPU" // type of the device on which a model is running. The value is NPU, IPU, MLU, or CPU. framework: "OFFLINE" // execution framework. The value is OFFLINE, CAFFE, or TENSORFLOW. }</pre>	O
	oam_config	Mode of dumping algorithm data. If the inference is inaccurate, you can view the algorithm execution results of some or all layers. Configuration example: <pre>oam_config{ items{ model_name: "" // relative path+model name +suffix is_dump_all: "" // whether to dump all data. The value is true or false. layer: "" // layer } dump_path: "" // dump path }</pre>	O
	internal_so_name	Dynamic library file embedded on the device. You can directly use the file without copying it from the host to the device.	O
	wait_in_putdata_max_time	Maximum timeout of waiting for the next piece of data after the current data is received - Unit: millisecond - The default value is 0, indicating that the wait does not time out.	O

Parameter		Description	Mandatory (M) or Optional (O)
	is_repeat_timeout_flag	Whether to perform cyclic timeout processing (wakeup) when the engine does not receive data. This parameter is used together with wait_inputdata_max_time . The options are as follows: 0: Cyclic timeout processing is not performed. The default value is 0. 1: Cyclic timeout processing is performed.	O
	holdModelFileFlag	Whether to retain the model file of the engine. The options are as follows: 0: no - Other values: yes	O
connects	src_engine_id	ID of the source engine	M
	src_port_id	Sending port number of the source engine. The port number starts from 0. The number of ports is defined by the HIAI_DEFINE_PROCESS macro in the corresponding header file. The implementation code is as follows: <pre>#define SOURCE_ENGINE_INPUT_SIZE 1 #define SOURCE_ENGINE_OUTPUT_SIZE 1 class SrcEngine : public Engine { HIAI_DEFINE_PROCESS(SOURCE_ENGINE_INPUT_SIZE, SOURCE_ENGINE_OUTPUT_SIZE) }</pre>	M
	target_engine_id	ID of the target engine	M
	target_port_id	Receive port number of the target engine	M

Parameter		Description	Mandatory (M) or Optional (O)
	target_graph_id	ID of the target graph. Engines can be connected across graphs. In this scenario, the target_graph_id field needs to be added to the connection configuration file to indicate the graph ID at the receive end. If target_graph_id is not specified, engines are connected within the same graph by default. When multiple Ascend 310 chips are available, you can enable models to run on different Ascend 310 chips by connecting engines across graphs. Therefore, you are advised to place the same type of models on the same chip for inference, which avoids unnecessary memory consumption. Scenario description: - As shown in Figure 6-10, engines support inter-graph serial connection in the following scenarios: 1: The host engine of graph A directly sends data to the host engine of graph B. 2: The host engine of graph A directly sends data to the device engine of graph B. 3: The device engine of graph A directly sends data to the device engine of graph B. - Engines do not support inter-graph serial connection in the following scenarios: 4: The device engine of graph A directly sends data to the host engine of graph B, as shown in Figure 6-10.	O

Parameter		Description	Mandatory (M) or Optional (O)
	receive_memory_without_dvpp	- The default value is 0, indicating that the RX memory of the target engines running on the device side must meet the address space limit of 4 GB. - If this parameter is set to 1, the target engines running on the device side does not need to meet the address space limit of 4 GB. - In the graph configuration file, you can set the receive_memory_without_dvpp parameter (under the connects property) to 1 for all target engines running on the device. Alternatively, you can set the receive_memory_without_dvpp parameter (under the connects property) to 1 for all target engines, because this parameter setting does not affect the target engines running on the host. In this way, the Matrix RX memory pool is not limited by the 4 GB address space, which improves the memory utilization. NOTE In the current DVPP, the input memory for the VPC, JPEGE, JPEGD, and PNGD must meet the address space limit of 4 GB, which is not mandatory for the input memory of the VDEC and VENC.	O

Figure 6-10 Engine connection across graphs



7 High Performance Application Programming

[7.1 Overview](#)

[7.2 Key Points](#)

[7.3 Operator Usage Suggestions](#)

[7.4 Samples](#)

[7.5 FAQs](#)

7.1 Overview

Purpose

This document describes the constraints and suggestions for building high-performance applications on the Ascend 310, helping users understand the samples and quickly build high-performance applications.

Scope

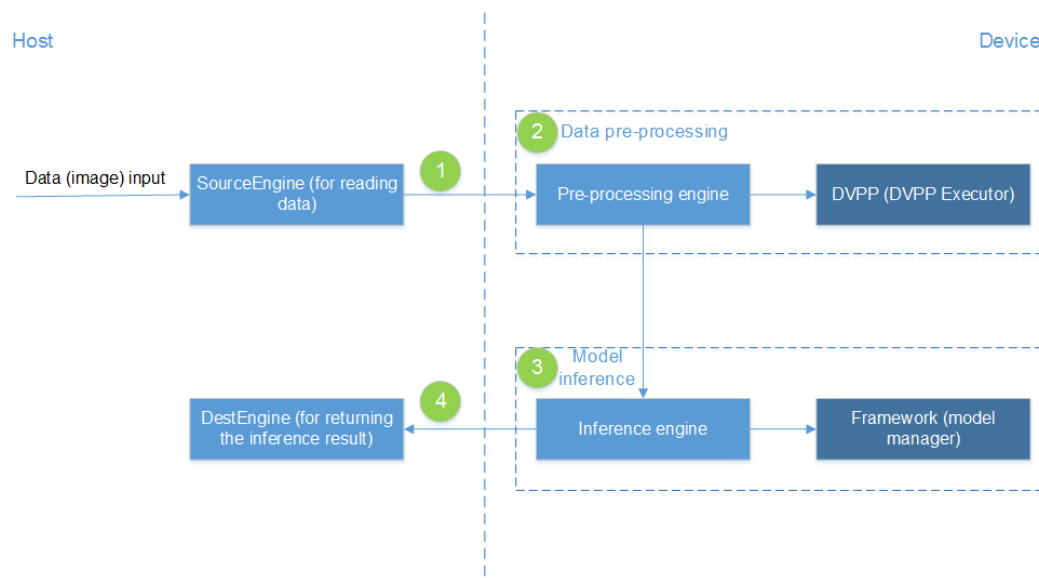
This document describes the key knowledge points for high-performance programming on the Ascend 310. It is intended for users who use chip hardware decoding, inference, and general image processing on the Ascend 310.

7.2 Key Points

During application development, the Matrix module processes data in the following phases:

1. Calls an API to transfer data (images) from the host to the device.
2. Calls DVPP APIs to encode, decode, and resize data.
3. Calls the model manager APIs provided by Framework to infer data.
4. Calls an API to send the inference result from the device to the host.

Figure 7-1 Data processing



During Matrix module processing, data transfer, DVPP processing, and model inference are most time-consuming. This document mainly describes how to streamline these phases and boost the performance.

For details about each API, see *Matrix API Reference* and *DVPP API Reference*

7.2.1 Memory Management

The Ascend 310 supports two types of APIs for memory management: memory management APIs provided by the native language and memory management APIs provided by the Matrix module.

7.2.1.1 Memory Management APIs Provided by the Native Language

The native language (C/C++) provides the **malloc**, **free**, **memcpy**, **memset**, **new**, and **delete** APIs for memory management. You can manage and control the lifecycle of memory allocated by using these APIs. If the memory to be allocated is less than 256 KB, memory management APIs provided by the native language and those provided by the Matrix module show similar performance. Therefore, you are advised to use a memory management API provided by the native language to simplify programming.

The following code shows how to use memory management APIs provided by the native language:

```
// Use malloc to alloc buffer
unsigned char* inbuf = (unsigned char*)malloc( fileLen );
// free buffer
free(inbuf);
inbuf = nullptr;
```

7.2.1.2 Memory Management APIs Provided by the Matrix Module

The Matrix module provides a set of C/C++ APIs for allocating and freeing memory, including **HIAI_DMAlloc/HIAI_DFree** and **HIAI_DVPP_DMAlloc/HIAI_DVPP_DFree**. Among these APIs, **HIAI_DMAlloc** and **HIAI_DFree** are used to

apply for memory and transfer data from the host to the device by working with **SendData**. While **HIAI_DVPP_DMAlloc** and **HIAI_DVPP_DFree** are used to allocate memory for the DVPP on the device. You can call the **HIAI_DMAlloc/HIAI_DFree** and **HIAI_DVPP_DMAlloc/HIAI_DVPP_DFree** APIs to allocate memory to reduce copy operations and save time.

API Description

Table 7-1 describes the functions of the **HIAI_DMAlloc/HIAI_DFree** and **HIAI_DVPP_DMAlloc/HIAI_DVPP_DFree** APIs.

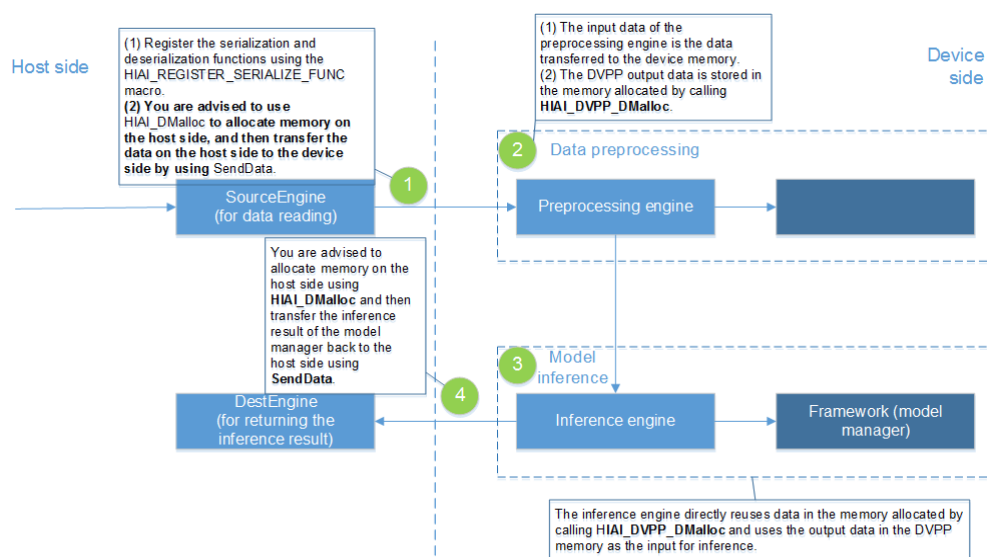
Table 7-1 API description

API Name	Function
HIAIMemory::HIAI_DMAlloc (for C++ only)	Allocates memory. The memory is similar to common memory but offers better performance in cross-side transmission (host-device/device-host) and model inference.
HIAIMemory::HIAI_DFree (for C++ only)	Frees the memory allocated by HIAIMemory::HIAI_DMAlloc . This API is used together with HIAIMemory::HIAI_DMAlloc . When calling the HIAIMemory::HIAI_DMAlloc API, you can set flag to MEMORY_ATTR_AUTO_FREE . In this case, if data is sent to the peer end by calling the SendData API, the allocated memory is automatically freed without calling the HIAIMemory::HIAI_DFree API after the program is complete. However, if the SendData API is not called to send data to the peer end after memory is allocated, you need to call HIAIMemory::HIAI_DFree to free the memory.
HIAI_DMAlloc (for C/C++)	Allocates memory. The memory is similar to common memory but offers better performance in cross-side transmission (host-device/device-host) and model inference.

API Name	Function
HIAI_DFree (for C/C++)	Frees the memory allocated by HIAI_DMAlloc . This API is used together with HIAI_DMAlloc . When calling the HIAI_DMAlloc API, you can set flag to MEMORY_ATTR_AUTO_FREE . In this case, if data is sent to the peer end by calling the SendData API, the allocated memory is automatically freed without calling the HIAI_DFree API after the program is complete. However, if the SendData API is not called to send data to the peer end after memory is allocated, you need to call HIAI_DFree to free the memory.
HIAIMemory::HIAI_DVPP_DMAlloc (for C++ only)	Allocates memory for the DVPP on the device.
HIAIMemory::HIAI_DVPP_DFree (for C++ only)	Frees the memory allocated by the HIAIMemory::HIAI_DVPP_DMAlloc API.
HIAI_DVPP_DMAlloc (for C/C++)	Allocates memory for the DVPP on the device.
HIAI_DVPP_DFree (for C/C++)	Frees the memory allocated by the HIAI_DVPP_DMAlloc API.

API Calling Process

Figure 7-2 API Calling Process



The usage of APIs in [Figure 7-2](#) is described as follows:

- The memory allocated by using the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API can be used in end-to-end data transmission and model inference. The data transmission efficiency and performance can be improved by calling the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API and using the **HIAI_REGISTER_SERIALIZE_FUNC** macro that serializes or deserializes user-defined data types.

Allocating memory by using the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API has the following advantages:

- The allocated memory can be directly used by host-device communication (HDC) module for data transmission to avoid data copy between the Matrix module and HDC.
- You can use the allocated memory for zero-copy inference to reduce data copy time.
- The memory allocated by using the **HIAI_DVPP_DMAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API can be used by the DVPP. After being used by the DVPP, data in the memory can be transparently transmitted to the inference model. If model inference is not required, data in the memory allocated by using the **HIAI_DVPP_DMAlloc** API can be directly sent back to the host.
- The memory allocated by using the **HIAI_DMAlloc**, **HIAIMemory::HIAI_DMAlloc**, **HIAI_DVPP_DMAlloc**, and **HIAIMemory::HIAI_DVPP_DMAlloc** APIs is compatible with memory management APIs provided by the native language. It can be used as common memory, but cannot be freed by using APIs such as **free** and **delete**. Generally, memory allocated by using the **HIAI_DMAlloc**, **HIAIMemory::HIAI_DMAlloc**, **HIAI_DVPP_DMAlloc**, and **HIAIMemory::HIAI_DVPP_DMAlloc** APIs needs to be freed by calling **HIAI_DFree**, **HIAIMemory::HIAI_DFree**, **HIAI_DVPP_DFree**, and **HIAIMemory::HIAI_DVPP_DFree**, respectively.

When calling the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API, you can set **flag** to **MEMORY_ATTR_AUTO_FREE**. In this case, if data is sent to the peer end by calling the **SendData** API, the allocated memory is automatically freed without calling the **HIAIMemory::HIAI_DFree** API after the program is complete. However, if the **SendData** API is not called to send data to the peer end after memory is allocated, you need to call **HIAIMemory::HIAI_DFree** to free the memory.

- The memory allocated by using the **HIAI_DVPP_DMAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API meets the requirements of the DVPP. Therefore, when the resources are limited, you are advised to use these APIs only for the DVPP.

Precautions for API Usage

When allocating memory by using **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc**, pay attention to the following issues about memory management:

- When allocating memory to be automatically freed for host-device or device-host data transmission, if a smart pointer is used, the Matrix module automatically frees the memory. Therefore, the destructor specified by the

smart pointer must be empty. If the pointer is not a smart pointer, the Matrix module automatically frees the memory.

- When allocating memory to be manually freed for host-device or device-host data transmission, if a smart pointer is used, you need to set the destructor to **HIAI_DFree** or **HIAIMemory::HIAI_DFree**. If the pointer is not a smart pointer, you need to call **HIAI_DFree** or **HIAIMemory::HIAI_DFree** to free the memory after data transmission is complete.
- When memory to be manually freed is allocated, the **SendData** API cannot be called repeatedly to send data in the memory.
- When allocating memory to be manually freed, if the memory is used for data transmission between the host and device, do not reuse the data in the memory before the memory is freed. If the memory is used for host-host or device-device data transmission, the data in the memory can be reused before the memory is freed.
- When allocating memory to be manually freed, if the **SendData** API is called to asynchronously send data, data in the memory cannot be modified after data is sent.

If the **HIAI_DVPP_MAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API is called to allocate memory for device-host data transmission, you need to call the **HIAI_DVPP_DFree** or **HIAIMemory::HIAI_DVPP_DFree** API to manually free the memory, because the **HIAI_DVPP_MAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API does not automatically free the memory. If a smart pointer is used to store the allocated memory address, the destructor must be set to **HIAI_DVPP_DFree** or **HIAIMemory::HIAI_DVPP_DFree**.

API Calling Example

```
(1) When the performance optimization solution is used to transmit data, the data transmit API must be manually serialized and deserialized.
// Note: The serialization function is used at the transmit end and the deserialization function is used at the receive end. Therefore, you are advised to register this function with both transmit and receive ends.
// Data structure
typedef struct
{
    uint32_t left_offset = 0;
    uint32_t right_offset = 0;
    uint32_t top_offset = 0;
    uint32_t bottom_offset = 0;
    // The serialize function is used to serialize a structure.
    template <class Archive>
    void serialize(Archive & ar)
    {
        ar(left_offset, right_offset, top_offset, bottom_offset);
    }
} crop_rect;

// Registers the structure to be transferred between engines.
typedef struct EngineTransNew
{
    std::shared_ptr<uint8_t> trans_buff = nullptr; // Transfer buffer
    uint32_t buffer_size = 0; // Transfer buffer size
    std::shared_ptr<uint8_t> trans_buff_extend = nullptr;
    uint32_t buffer_size_extend = 0;
    std::vector<crop_rect> crop_list;
    // The serialize function is used to serialize a structure.
    template <class Archive>
    void serialize(Archive & ar)
```

```

    {
        ar(buffer_size, buffer_size_extend, crop_list);
    }
}EngineTransNewT;
// Serialization function
/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Serializes the Trans data.
 * @param [in]: data_ptr // Structure pointer
 * @param [out]: struct_str // Structure buffer
 * @param [out]: data_ptr // Structure data pointer buffer
 * @param [out]: struct_size // Structure size
 * @param [out]: data_size // Structure data size
 */
void GetTransSearPtr(void* data_ptr, std::string& struct_str,
    uint8_t*& buffer, uint32_t& buffer_size)
{
    EngineTransNewT* engine_trans = (EngineTransNewT*)data_ptr;
    uint32_t dataLen = engine_trans->buffer_size;
    uint32_t dataLen_extend = engine_trans->buffer_size_extend;
    // Obtains the structure buffer and size.
    buffer_size = dataLen + dataLen_extend;
    buffer = (uint8_t*)engine_trans->trans_buff.get();

    // Serialization
    std::ostream outputStr;
    cereal::PortableBinaryOutputArchive archive(outputStr);
    archive((*engine_trans));
    struct_str = outputStr.str();
}
// Deserialization function
/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Deserializes the Trans data.
 * @param [in]: ctrl_ptr // Structure pointer
 * @param [in]: data_ptr // Structure data pointer
 * @param [out]: std::shared_ptr<void> // Structure pointer assigned to the engine
 */
std::shared_ptr<void> GetTransDearPtr(
    const char* ctrlPtr, const uint32_t& ctrlLen,
    const uint8_t* dataPtr, const uint32_t& dataLen)
{
    if(ctrlPtr == nullptr) {
        return nullptr;
    }
    std::shared_ptr<EngineTransNewT> engine_trans_ptr = std::make_shared<EngineTransNewT>();
    // Assigns a value to engine_trans_ptr.
    std::istream inputStream(std::string(ctrlPtr, ctrlLen));
    cereal::PortableBinaryInputArchive archive(inputStream);
    archive((*engine_trans_ptr));
    uint32_t offsetLen = engine_trans_ptr->buffer_size;
    if(dataPtr != nullptr) {
        (engine_trans_ptr->trans_buff).reset((const_cast<uint8_t*>(dataPtr)), ReleaseDataBuffer);
        // trans_buff and trans_buff_extend point to a contiguous memory space whose address starts with
dataPtr;
        // therefore, you only need to bind trans_buff to the destructor, and then the destructor will free the
        contiguous memory space after being used.
        (engine_trans_ptr->trans_buff_extend).reset((const_cast<uint8_t*>(dataPtr + offsetLen)),
SearDeleteNothing);
    }
    return std::static_pointer_cast<void>(engine_trans_ptr);
}
// Registers EngineTransNewT
HIAI_REGISTER_SERIALIZE_FUNC("EngineTransNewT", EngineTransNewT, GetTransSearPtr, GetTransDearPtr);

(2) When sending data, you can use only the registered data types. Use HIAI_DMAlloc to allocate memory
to optimize performance.
Note: When transferring data from the host to the device, you are advised to use HIAI_DMAlloc to
optimize transmission efficiency. The data size supported by the HIAI_DMAlloc API ranges from 0 bytes to

```

(256 MB – 96 bytes). If the data size exceeds this range, use the **malloc** API to allocate memory.

// Allocates the data memory by calling the **HIAI_DMAlloc** API. The value **10000** indicates the delay in microseconds, that is, if the memory space is insufficient, the program waits 10000 ms.

HIAI_StatusT get_ret = HIAIMemory::HIAI_DMAlloc(width*align_height*3/2,(void*)&align_buffer, 10000);
// Sends data. After the **SendData** API is called, the **HIAI_DFree** API does not need to be called. The value **10000** indicates the delay.

graph->SendData(engine_id_0, "TEST_STR", std::static_pointer_cast<void>(align_buffer), 10000);

7.2.2 Host-Device Data Transmission

```
// EngineTransNewT structure
typedef struct EngineTransNew
{
    std::shared_ptr<uint8_t> trans_buff = nullptr; // Transfer buffer
    uint32_t buffer_size = 0; // Transfer buffer size
    std::shared_ptr<uint8_t> trans_buff_extend = nullptr;
    uint32_t buffer_size_extend = 0;
    std::vector<crop_rect> crop_list;
    // The serialize function is used to serialize a structure.
    template <class Archive>
    void serialize(Archive & ar)
    {
        ar(buffer_size, buffer_size_extend, crop_list);
    }
}EngineTransNewT;

/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Serializes the Trans data.
 * @param [in]: dataPtr // Structure pointer
 * @param [out]: structStr // Structure buffer
 * @param [out]: buffer // Structure data pointer buffer
 * @param [out]: buffSize // Structure data size
 */
void GetTransSearPtr(void* data_ptr, std::string& struct_str,
    uint8_t*& buffer, uint32_t& buffer_size)
{
    EngineTransNewT* engine_trans = (EngineTransNewT*)data_ptr;
    uint32_t dataLen = engine_trans->buffer_size;
    uint32_t dataLen_extend = engine_trans->buffer_size_extend;
    // Obtains the structure buffer and size.
    buffer_size = dataLen + dataLen_extend;
    buffer = (uint8_t*)engine_trans->trans_buff.get();

    // Serialization
    std::ostringstream outputStr;
    cereal::PortableBinaryOutputArchive archive(outputStr);
    archive(*engine_trans);
    struct_str = outputStr.str();
}

/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Deserializes the Trans data.
 * @param [in] : ctrlPtr // Structure pointer
 * @param [in]: ctrlLen // Control information size of the data structure
 * @param [in] : dataPtr // Structure data pointer
 * @param [in]: dataLen // Data storage size of the structure, which is used only for verification
and does not represent the original data size
 * @param [out]: std::shared_ptr<void> // Structure pointer assigned to the engine
 */
std::shared_ptr<void> GetTransDearPtr(
    const char* ctrlPtr, const uint32_t& ctrlLen,
    const uint8_t* dataPtr, const uint32_t& dataLen)
{
    if(ctrlPtr == nullptr) {
        return nullptr;
    }
    std::shared_ptr<EngineTransNewT> engine_trans_ptr = std::make_shared<EngineTransNewT>();
```

```
// Assigns a value to engine_trans_ptr.
std::istringstream inputStream(std::string(ctrlPtr, ctrlLen));
cereal::PortableBinaryInputArchive archive(inputStream);
archive((*engine_trans_ptr));
uint32_t offsetLen = engine_trans_ptr->buffer_size;
if(dataPtr != nullptr) {
    (engine_trans_ptr->trans_buff).reset((const_cast<uint8_t*>(dataPtr)), ReleaseDataBuffer);
    // trans_buff and trans_buff_extend point to a contiguous memory space whose address starts with
dataPtr;
    // therefore, you only need to bind trans_buff to the destructor, and then the destructor will free the
    contiguous memory space after being used.
    (engine_trans_ptr->trans_buff_extend).reset((const_cast<uint8_t*>(dataPtr + offsetLen)),
    SearDeleteNothing);
}
return std::static_pointer_cast<void>(engine_trans_ptr);
}
// Registers EngineTransNewT
HIAI_REGISTER_SERIALIZE_FUNC("EngineTransNewT", EngineTransNewT, GetTransSearPtr, GetTransDearPtr);
```

In the case of a large amount of image data or code stream transmission, **HIAI_REGISTER_SERIALIZE_FUNC** is used to serialize or deserialize the user-defined data types, which can implement high-performance data transmission and save the transmission time.

The Matrix module describes the data to be transmitted in the form of "control information+data information". The control information refers to a user-defined data type, and the data information refers to content to be transmitted. The Matrix module provides the following mechanism to ensure data transmission between the host and device.

1. Before data transmission, you can call the **HIAI_REGISTER_SERIALIZE_FUNC** macro to register user-defined data types, user-defined serialization functions, and user-defined deserialization functions.
2. After you call the **SendData** API to send data at the local end, the Matrix module performs the operations. [Figure 7-3](#) shows the processing process.

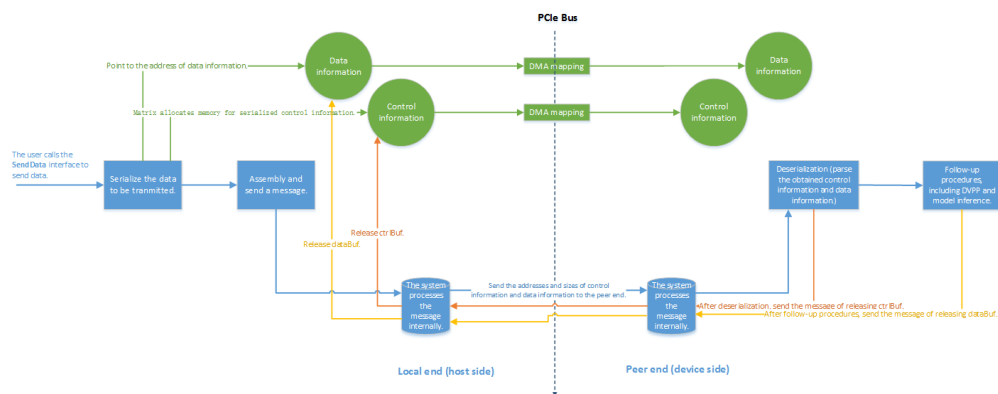
- a. Call the user-defined serialization function to serialize the control information and save the serialized control information to the memory (**ctrlBuf**).
- b. Copy the control information and store it in the memory of the peer end through the Direct Memory Access (DMA) mapping, and maintain the mapping between the control information of the local end and that of the peer end.

The pointer to the memory (**dataBuf**) has been transferred through the input parameter of the **SendData** API. **dataBuf** is allocated by calling the **HIAI_DMAlloc/HIAI_DVPP_DMAlloc** API. After the memory is allocated, the system copies the local data information through DMA mapping and stores it to the memory of the peer end, and maintains the mapping between the local end and the peer end.

- c. A message including the addresses and sizes of **ctrlBuf** and **dataBuf** is sent to the peer end.
- d. After the peer end receives the message, the Matrix module calls the user-defined deserialization function to parse the control information and data information obtained by the peer end, and sends the parsed data to the corresponding receiving engine for processing.
- e. After the peer end parses the data, the control information is used out. Therefore, the memory (**ctrlBuf**) for storing the control information can

- be freed. This memory is allocated at the local end, so the peer end needs to send the local end a message for freeing **ctrlBuf**.
- f. After receiving the message, the local end frees **ctrlBuf**.
- g. After the engine receives and processes the data, **dataBuf** can be freed. The Matrix module does not know when **dataBuf** is used up. Therefore, when the deserialization function is implemented, **dataBuf** needs to return a smart pointer and bound to the destructor **hiai::Graph::ReleaseDataBuffer**. When the smart pointer ends the lifecycle destruction, the destructor is automatically called to send a message to the local end for freeing **dataBuf**.
- h. After receiving the message, the local end frees **dataBuf**.

Figure 7-3 Data processing process



Example scenario: The following code declares the serialization/deserialization function, defines the data type (struct), and registers the user-defined data type, serialization function, and deserialization function using the **HIAI_REGISTER_SERIALIZE_FUNC** macro. Before data transmission, the Matrix module calls the registered serialization function. After data transmission, the Matrix module calls the registered deserialization function.

```
// EngineTransNewT structure
struct EngineTransNewT
{
    std::shared_ptr<uint8_t> transBuff;
    uint32_t bufferSize; // Buffer size
    std::string url;
    // The serialize function is used to serialize a structure.
    template <class Archive>
    void serialize(Archive & ar)
    {
        ar(url);
    }
}

/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Serializes the Trans data.
 * @param [in]: dataPtr // Structure pointer
 * @param [out]: structStr // Structure buffer
 * @param [out]: buffer // Structure data pointer buffer
 * @param [out]: buffSize // Structure data size
 */
void GetTransSearPtr(void* dataPtr, std::string& structStr, uint8_t*& buffer, uint32_t& bufferSize)
{
    EngineTransNewT* engineTrans = (EngineTransNewT *)dataPtr;
```

```
std::shared_ptr<uint8_t> transBuff = ((EngineTransNewT *)dataPtr)->transBuff;
buffer = (uint8_t*)engineTrans->transBuff.get();
bufferSize = engineTrans->bufferSize;

engineTrans->transBuff = nullptr;
engineTrans->buffSize = 0;

std::ostringstream outputStr;
cereal::PortableBinaryOutputArchive archive(outputStr);
archive((*engineTrans));
struct_str = outputStr.str();

((EngineTransNewT*)dataPtr)->transBuff = transBuff;
engineTrans->buffSize = bufferSize;
}

/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr,          // Deserializes the Trans data.
 * @param [in] : ctrlPtr           // Structure pointer
 * @param [in]: ctrlLen            // Control information size of the data structure
 * @param [in] : dataPtr           // Structure data pointer
 * @param [in]: dataLen            // Data storage size of the structure, which is used only for verification
 *                                // and does not represent the original data size
 * @param [out]: std::shared_ptr<void> // Structure pointer assigned to the engine
 */
std::shared_ptr<void> GetTransDearPtr(char* ctrlPtr, const uint32_t& ctrlLen, uint8_t* dataPtr, const
uint32_t& dataLen)
{
    std::shared_ptr<EngineTransNewT> engineTransPtr = std::make_shared<EngineTransNewT>();
    std::istringstream inputStream(std::string(ctrlPtr, ctrlLen));
    cereal::PortableBinaryInputArchive archive(inputStream);
    archive((*engineTransPtr));
    engineTransPtr->bufferSize = dataLen;
    engineTransPtr->transBuff.reset(dataPtr, hiai::Graph::ReleaseDataBuffer);
    return std::static_pointer_cast<void>(engineTransPtr);
}
// Registers EngineTransNewT
HIAI_REGISTER_SERIALIZE_FUNC("EngineTransNewT", EngineTransNewT, GetTransSearPtr, GetTransDearPtr);
```

7.2.3 Usage of DVPP

7.2.3.1 Image and Video Encoding and Decoding

The Matrix module provides APIs for image processing and video encoding/decoding. You can perform image/video decoding on the device to reduce the data traffic between the host and device, the data transmission time, and the bandwidth pressure.

On the host, the **HIAI_DMalloc** API provided by the Matrix module is called to allocate memory on the device. The allocated memory is used as the input of image and video encoding and decoding. It is recommended that the start address of the memory for data storage be 128-byte aligned. On the device, after image or video pre-processing, the DVPP calls the **HIAI_DVPP_DMalloc** API provided by the Matrix module to allocate memory. The memory is used to store the image/video pre-processing output.

7.2.3.2 Image Cropping and Resizing

When programming with Ascend 310, you are advised to use digital vision pre-processing (DVPP) for image cropping and resizing.

Figure 7-4 Cropping and resizing process

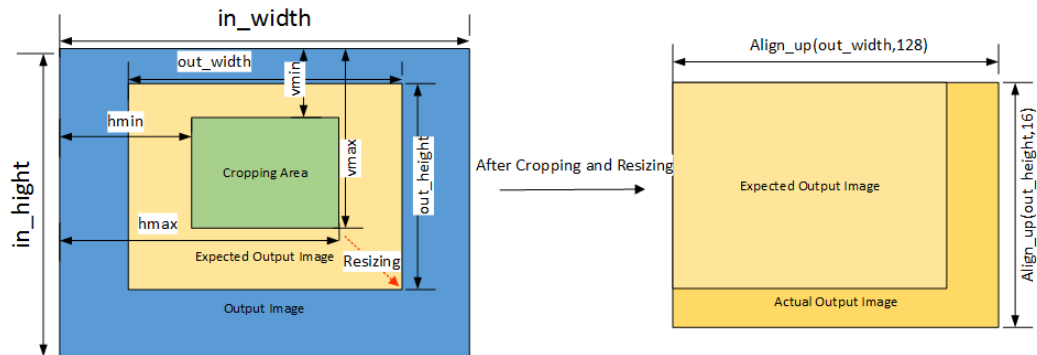


Figure 7-4 shows the cropping/resizing process. This process crops the region of interest (ROI) and uses the cropped area for re-sampling. The process of cutting out the ROI is called cropping, and the process of re-sampling is called resizing. When the resizing coefficient is 1, only cropping is performed. When the original image is the cropping output, only resizing is performed.

The restrictions on cropping/resizing input are as follows:

- Input data address (OS virtual address): 16-byte aligned
- Buffer restriction for the input image width: 16-byte aligned
- Buffer restriction for the input image height: 2-byte aligned

To achieve zero-copy during high performance programming, the memory addresses assigned to users on the device (receive end) must meet the preceding restrictions when the memory is allocated. Generally, you can use either of the following methods depending on different input types:

- Method 1: If the app uses the host or other hardware for decoding, perform data cropping or padding on the host (transmit end) to meet the 16 x 2 alignment requirement. In this way, the Matrix module automatically allocates data memory that meets the preceding restrictions at the data receive end.

NOTE

The input data in the sample is a YUV image. Therefore, the size of the image is calculated by using the following formula: $(Image\ width \times Image\ height) \times 3/2$. You need to change the formula according to the image format.

```
static const uint32_t ALIGN_W = 16;
static const uint32_t ALIGN_H = 2;
uint32_t imageWidth = 500;
uint32_t imageHeight = 333;
uint32_t imageSize = imageWidth * imageHeight * 3/2;
uint32_t align_width = image_width, align_height = image_height;

// Performs width and height alignment.
alignWidth = (alignWidth % ALIGN_W) ? alignWidth : (imageWidth + ALIGN_W)/ALIGN_W *
ALIGN_W;
alignHeight = (alignHeight % ALIGN_H) ? alignHeight : (imageHeight + ALIGN_H)/ALIGN_H *
ALIGN_H;
uint32_t alignSize = alignWidth * alignHeight * 3/2;

// Reads the file data and copies the aligned memory.
FILE *fpIn = fopen(filePath.data(), "rb");
Uint8_t* imageBuffer = (uint8_t*) malloc(imageSize);
```

```
HIAI_StatusT getRet = hiai::HIAIMemory::HIAI_DMAlloc(fileLen, (void*)&alignBuffer, 10000);

size_t size = fread(imageBuffer, 1, imageSize, fpIn);
// Copies data.
uint32_t tempLen = imageWidth * imageHeight;
if (alignWidth == imageWidth)
{
    if (alignHeight > imageHeight)
    {
        memcpy_s(alignBuffer, tempLen, imageBuffer, tempLen);
        memcpy_s(alignBuffer + alignHeight * alignWidth,
            tempLen/2, imageBuffer + tempLen, tempLen/2);
    }
} else {
    for (int32_t n=0; n<imageHeight; n++)
    {
        memcpy_s(alignBuffer+n*alignWidth, imageWidth,
            imageBuffer+n*imageWidth, imageWidth);
    }
    for (int32_t n=0; n<imageHeight/2; n++)
    {
        memcpy_s(alignBuffer+n*alignWidth+alignHeight* alignWidth,
            imageWidth, imageBuffer+n* imageWidth + imageHeight*imageWidth, imageWidth); //UV
    }
}
```

- Method 2: Decode the image or video on the device to generate 16 x 2 aligned output that can be used as the input for DVPP VPC cropping and resizing.

7.2.4 Model Conversion Pre-Processing Configuration

As shown in [Figure 7-4](#), the cropped or resized image are aligned. As a result, some areas are padded and are not the input required by the original model. To obtain the required image, you can copy the data to a new buffer and input the data to the model inference module. However, this process causes overheads. To reduce such overheads, the Matrix module provides a mechanism that allows padded areas in input images to the model manager (**modelManager**). The AIPP module of the model manager crops the image based on the user-defined width and height to generate an image that meets the model input requirements. When the image generated by the AIPP module is used, the inference module does not need to copy data, greatly improving the performance.

In the following example, the input image size for module inference is 224 pixels x 224 pixels, and the data obtained from DVPP is 128 pixels x 16 pixels aligned (the aligned image size is 256 pixels x 224 pixels).

7.2.5 Batch and Timeout

For most models, especially small models, chip inference using batch input brings performance improvement. Batch inference greatly improves the data throughput and chip utilization. Although a certain amount of latency is generated, the overall performance of the system is improved. Therefore, to build a high-performance application, the input batch needs to be the largest batch allowed by the latency.

To make batch operations more simple and flexible, the Matrix module supports the timeout mechanism. Users can configure **is_repeat_timeout_flag** in the **config** file to set whether to enable timeout waiting, and configure **wait_inputdata_max_time** to set the timeout duration. If timeout parameters are set, the system transfers null pointers when calling the **Process** function after timeout. You need to compile the code logic for timeout processing, which stores

the received data in a queue and performs inference until the received data is sufficient to form a batch. You can use the **hiai::MultiTypeQueue** queue provided by the Matrix module. To prevent data starvation, a timeout duration must be specified by using the timeout setting API based on the latency requirement of the application. When the timeout duration expires, the Matrix module calls the main processing flow of the engine. Users can obtain and process the data in the queue to avoid starvation.

If the model input is multiple batches and you want to send data of each batch to the model manager (inference engine) for inference, you need to add the following code logic:

1. You need to apply for a buffer space on the device side to store the data of each batch.
2. When the inference engine on the device side receives data of all batches, you need to combine the data and store it in the buffer space allocated in [1](#).
3. You can use the data in batches stored in the buffer space for inference only when the number of received batches on the device side is the same as that required for model inference.

7.2.6 Processing the Input and Output Data of Algorithm Inference

To avoid memory copy during algorithm inference, you are advised to use the **HIAI_DMalloc** API to allocate memory for the input and output data when calling the process API of the model manager. In this way, zero-copy can be achieved and the processing time is optimized. If DVPP processing is required before inference, use the memory transferred by the Matrix module as the DVPP input memory, use the **HIAI_DVPP_DMalloc** API to allocate the DVPP output memory, and use the DVPP output memory as the input memory of the inference engine.

7.2.7 Optimization of Data Backhaul

After the inference is complete, the inference result or inference end signal needs to be sent to the host. If the inference engine calls **SendData** to send data back to the host, the time of the inference engine is consumed. It is recommended that an independent engine (for example, DataOptEngine) be deployed to return data. After the inference is completed, the inference engine transparently transmits the processed data to DataOptEngine, which forwards the data to an engine (for example, DstEngine) on the host side.

```
// DataOptEngine on the device side sends data to the host side.
HIAI_IMPL_ENGINE_PROCESS("DataOptEngine", DataOptEngine, 1)
{
    HIAI_StatusT hiaiRet = HIAI_OK;
    if (arg0 == nullptr) {
        HIAI_ENGINE_LOG(HIAI_INVALID_INPUT_MSG, "get inference result timeout");
        return HIAI_INVALID_INPUT_MSG;
    }
    hiaiRet = SendData(0, "EngineTransNewT", arg0);
}
// DataOptEngine on the device side sends data to the host side.
HIAI_IMPL_ENGINE_PROCESS("DstEngine", DstEngine, 1)
{
    HIAI_StatusT ret = HIAI_OK;
    if (nullptr != arg0) {
        printf("dest engine had receive data already\n");
    }
}
```

```
std::shared_ptr<std::string> result =  
    std::static_pointer_cast<std::string>(arg0);  
ret = SendData(0, "string", result);  
if (HIAI_OK != ret) {  
    HIAI_ENGINE_LOG(ret, "DstEngine SendData to recv failed");  
    return ret;  
}  
}  
else {  
    HIAI_ENGINE_LOG(HIAI_INVALID_INPUT_MSG, "DestEngine Fail to receive data");  
    printf("destengine do not receive data arg0 is null\n");  
    return HIAI_INVALID_INPUT_MSG;  
}  
return HIAI_OK;  
}
```

7.3 Operator Usage Suggestions

Principles

On the Ascend 310 chip, you can improve the efficiency of the Cube to optimize the algorithm performance. In this case, you need to reduce data transmission and vector calculation. The general principles are as follows:

1. Network structure
 - Mainstream network topologies such as ResNet and MobileNet are recommended because their performance has been optimized.
 - Earlier network topologies such as VGG and AlexNet use large network models and bring high bandwidth pressure.
 - In matrix multiplication, set the values of **M**, **K**, **N** to multiples of 16. Increase the number of channels in the algorithm if possible and do not reduce the number of channels by creating groups.
 - Increasing the parameter reuse rate can reduce the bandwidth pressure. Therefore, you can increase the filter reuse rate to improve the performance. For example, use greater feature map sizes, smaller stride values, and smaller dilation values.
2. Conv operator
 - In non-quantization mode, it is recommended that the number of input and output channels of a Conv operator be integral multiples of 16.
 - In quantization mode, it is recommended that the number of input and output channels of a Conv operator be integral multiples of 32.
 - In quantization mode, it is recommended that fewer pooling operators be inserted between multiple Conv operators.
3. Full connection (FC) operator

If FC operators exist on the network, use multiple batches to perform inference at the same time.
4. Concat operator
 - In non-quantization mode, it is recommended that the number of input channels of the Concat operator be integral multiples of 16.
 - In quantization mode, it is recommended that the input channel of the Concat operator be integral multiples of 32.

5. Conv fusion operator
The Conv+BatchNorm+Scale+Relu/Relu6 combination is recommended. The performance has been optimized.
6. Norm operator
 - The BatchNorm operator is recommended, which uses the pre-trained **Norm** parameter.
 - Operators (such as LRN) that require the online calculation of the **Norm** parameter are not recommended.
7. Detection operator
Mainstream detection network topologies such as Faster R-CNN and SSD are recommended because their performance has been optimized.

Tips

1. The performance of Conv+(BatchNorm+Scale)+Relu is better than that of Conv+(BatchNorm+Scale)+Tanh. Avoid using complex activation functions.
2. When Concat operators are assembled in the C dimension, the performance is better if the values of **Tensor** and **Channel** are multiples of 16.
3. When the value of **Batch** is a multiple of 16, the performance is better.
4. The continuous convolution structure shows better performance. If multiple vector operators (such as pooling) are inserted between the convolution layers, the performance is poor. This is obvious in the INT8 model.
5. In early versions of AlexNet and GoogleNet, LRN is used as the normalization operator. The calculation of this operator is complex. During the evolution of the algorithm, the operator is replaced with other operators such as BatchNorm. The LRN operator is no longer used in mainstream network structures such as ResNet and Inception. For the Ascend310 platform, it is recommended BatchNorm be used on the network.

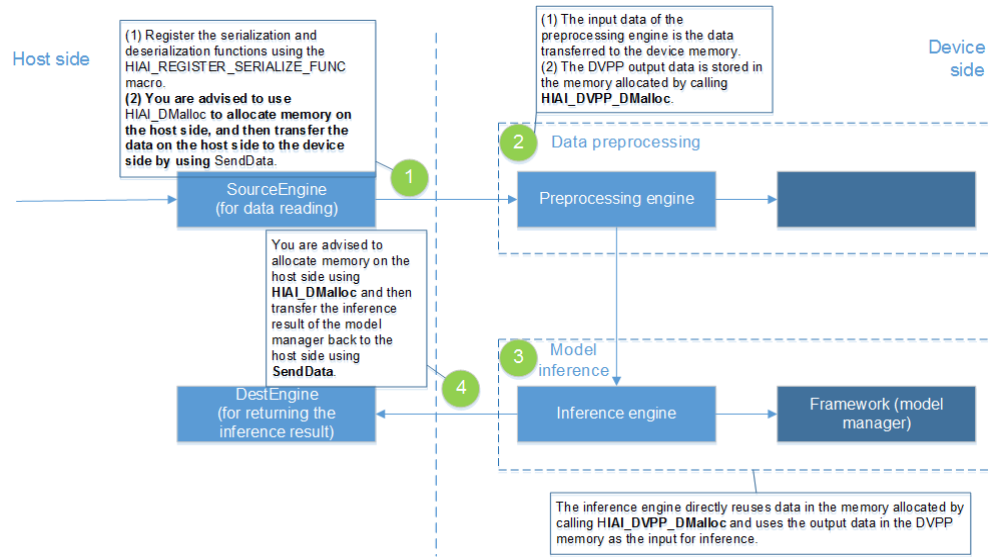
7.4 Samples

If multiple inference models are run at the same time, multiple graphs are not recommended. You are advised to use single graph multiple batches or single graph multiple engines.

7.4.1 Data Flow

The sample code uses the classification network (ResNet-18) to process images. [Figure 7-5](#) shows the data flow.

Figure 7-5 API Calling Process



7.4.2 Zero-Copy

The sample code shows the idea of zero-copy. Zero-copy means that the user does not perform any explicit copy operation on the image data in the entire process. Zero-copy requires the following operations:

1. Use **HIAI_REGISTER_SERIALIZE_FUNC** to register the serialization function (**GetSerializeFunc**) and deserialization function (**GetDeserializeFunc**) to implement serialization and deserialization of data types, respectively.
2. Call the **HIAI_DMalloc** API provided by the Matrix module to allocate memory and then call the **SendData** API to transfer data from the host to the device.
3. The memory allocated by calling the **HIAI_DMalloc** API is used as the input of image/video encoding and decoding without data copying.
4. The memory allocated by calling the **HIAI_DVPP_DMalloc** API provided by the Matrix module meets the input/output address requirements of the DVPP and can be used directly as the image/video output. After the **HIAI_DVPP_DMalloc** API is called to allocate memory, the **HIAIMemory::HIAI_DVPP_DFree** API must be called to free the memory.
In the DVPP, the input of the VPC module can directly reuse the output data of the JPEGD module in the memory.
5. The memory allocated by calling the **HIAI_DVPP_DMalloc** API can be used directly as the input of the first-layer model inference without data copying.

7.5 FAQs

7.5.1 What Do I Do If an Exception Occurs Because a Smart Pointer Is Used to Automatically Free Memory but the Destructor Is Not Empty?

Symptom

HIAI_DMalloc or **HIAIMemory::HIAI_DMalloc** is called to allocate memory for host-device or device-host data transmission. **flag** is set to **MEMORY_ATTR_AUTO_FREE** to automatically free memory. A smart pointer is used to store the address of the allocated memory, but no destructor is defined (the default destructor is called) or another destructor is called to free the memory (such as **HIAI_DFree**, which causes repeated memory freeing). As a result, the program becomes abnormal and memory leakage occurs.

Solution

The Matrix module automatically frees the memory. Therefore, set the destructor of the smart pointer to empty to resolve the problem.

The reference code is as follows:

```
* @ Allocates memory to be automatically freed. If a smart pointer is used, the destructor must be empty
  (the Matrix module automatically frees the memory).
**/
int main()
{
    // Uses DMalloc to allocate memory to be automatically freed.
    unsigned char* inbuf = nullptr;
    uint32_t bufferLen = 1080*1920;
    // The allocated memory is automatically freed. In this case, set flag to
    hiai::HIAI_MEMORY_ATTR::MEMORY_ATTR_AUTO_FREE.
    HIAI_StatusT getRet = haii::HIAIMemory::HIAI_DMalloc(bufferLen, (void*)&inbuf, 10000);
    // getRet exception judgment
    // Stores the data to be transmitted in inbuf. The size of the data is bufferLen.
    // Assembles the engineTranData structure for sending data
    EngineTransNewT engineTranData;
    engineTranData->bufferSize = bufferLen;
    engineTranData->transBuff.reset(std::static_pointer_cast<uint8_t*>(inbuf), [](uint8_t*)(addr)
    {
        // The destructor does not perform any operation. The Matrix module automatically frees the
        memory. If the destructor is not defined, the default destructor is called, causing an exception.
        // If another destructor such as HIAI_DFree is called, repeated memory freeing occurs, causing
        an exception.
    });
    // Sends data to the peer end by calling SendData.
    HIAI_StatusT sendRet =
    SendData(DEFAULT_PORT,"EngineTransNewT",std::static_pointer_cast<void*>(engineTranData));
    // sendRet exception judgment
}
```

7.5.2 What Do I Do If an Exception Occurs Because a Smart Pointer Is Used to Manually Free Memory but the Destructor Is Not **HIAI_DFree**?

Symptom

HIAI_DMalloc or **HIAIMemory::HIAI_DMalloc** is called to allocate memory for host-device or device-host data transmission. **flag** is set to

MEMORY_ATTR_MANUAL_FREE to manually free memory. A smart pointer is used to store the address of the allocated memory. If the destructor of the smart pointer is incorrect, the program becomes abnormal and memory leakage occurs.

Solution

Set the destructor of the smart pointer to **HIAI_DFree** or **HIAIMemory::HIAI_DFree**.

An example of the code is as follows:

```
/**
 * @ Allocates memory to be manually freed. If a smart pointer is used, use HIAI_DFree as the destructor.
 */
int main()
{
    // Uses DMalloc to manually free memory.
    unsigned char* inbuf = nullptr;
    uint32_t bufferLen = 1080*1920;
    HIAI_StatusT getRet = hiai::HIAIMemory::HIAI_DMalloc(bufferLen, (void*)&inbuf,
10000,hiai::HIAI_MEMORY_ATTR::MEMORY_ATTR_MANUAL_FREE);
    // getRet exception judgment

    // Stores the data to be transmitted in inbuf. The size of the data is bufferLen.
    // Assembles the engineTranData structure for sending data
    EngineTransNewT engineTranData;
    engineTranData->bufferSize = bufferLen;
    engineTranData->transBuff.reset(std::static_pointer_cast<uint8_t*>(inbuf), [](uint8_t*)(addr)
    {
        // Uses HIAI_DFree as the destructor. Otherwise, memory leakage occurs.
        hiai::HIAIMemory::HIAI_DFree(void* addr)
    });
    // Sends data to the peer end by calling SendData.
    HIAI_StatusT sendRet =
SendData(DEFAULT_PORT,"EngineTransNewT",std::static_pointer_cast<void*>(engineTranData));
    // sendRet exception judgment
}
```

8 API Reference

[8.1 Framework API Reference](#)

[8.2 Matrix API Reference](#)

[8.3 DVPP API Reference](#)

[8.4 GE API Reference](#)

[8.5 TE API Reference](#)

8.1 Framework API Reference

8.1.1 Introduction

This document describes the internal APIs (no need to be called by the user) of the Framework on which the compilation of custom operators depends and the APIs involved in the development of custom operator plug-ins. The APIs include non-class member functions and class member functions. In addition, this document describes the macros related to the APIs.

For details about how to use these APIs to develop custom operator plug-ins and compile and run code, see *TE Custom Operator Development Guide*.

You can view the API definition files in **ddk/include/inc** in the installation directory of the device development kit (DDK). If you install Mind Studio and the DDK together in boot installation mode, you can log in to the Mind Studio server as the Mind Studio installation user and view the API definition files in **~/tools/che/ddk/ddk/include/inc**. For details about the mapping between the definition files and the APIs, see the API description.

8.1.2 Non-Class Member Functions

8.1.2.1 custom_op_run

Function

Entry function for the commissioning of a single custom operator. This function is defined in **custom\custom_op.h**.

The return value of **custom_op_run** is of the **ErrorInfo** type, including the error code and error description. If the error code is **0**, the operation is successful. Other values indicate failure. The **ErrorInfo** type is defined as follows:

```
struct ErrorInfo
{
    uint32_t error_code;
    std::string error_msg;
};
```

Syntax

```
ErrorInfo custom_op_run(const std::string& name, int32_t type, const std::string&
bin_file,
const std::vector< std::string >& in_files,
const std::vector< std::string >& out_files,
const std::vector< uint32_t >& out_buf_sizes,
const std::vector< uint32_t >& workspace_sizes = std::vector<uint32_t>(),
const std::string& op_cfg_file = "", void *param = nullptr, int param_len = 0);
```

Parameter Description

Parameter	Input/ Output	Description
name	Input	Operator kernel name
type	Input	Operator type: • 0 - TE_AICORE • 1 -TE_AICPU • 2 - AI CPU
bin_file	Input	Binary file path of an operator, including the file name. A relative path is supported. In this case, the binary file must be stored in the directory where the program is located, for example ./*.bin .

Parameter	Input/ Output	Description
in_files	Input	Path of the input data file, including the file name. A relative path is supported. In this case, the data file must be stored in the directory where the program is located.
out_files	Input	Path of the output data file, including the file name. A relative path is supported. In this case, the data file must be stored in the directory where the program is located.
out_buf_sizes	Input	Size of the output data, in bytes
workspace_sizes	Input	Size of the workspace, in bytes
op_cfg_file	Input	Input and output description of the operator specified by the user
param	Input	Address information of the parameter structure of the operator specified by the user
param_len	Input	Length of the parameter structure of the operator specified by the user, in bytes. The structure length can be obtained using sizeof(param) .

8.1.2.2 custom_op_compare

Function

Verification function of a single custom operator, which is used to verify data precision. This function is defined in **custom\custom_op.h**.

The return value of **custom_op_compare** is of the **ErrorInfo** type, including the error code and error description. If the error code is **0**, the operation is successful. Other values indicate failure. The **ErrorInfo** type is defined as follows:

```
struct ErrorInfo
{
    uint32_t error_code;
    std::string error_msg;
};
```

Syntax

```
ErrorInfo custom_op_compare(const std::string& expect_file, const std::string&
actual_file,
int32_t data_type, float precision_deviation,
```

```
float statistical_discrepancy, bool& compare_result);
```

Parameter Description

Parameter	Input/ Output	Description
expect_file	Input	Expected data file, including the file name
actual_file	Input	Actual data file, including the file name
data_type	Input	Operator type. Currently, FP32 (0) and FP16 (1) are supported.
precision_deviation	Input	Precision deviation of single data. The value range is (0, 1). A smaller deviation value indicates higher precision.
statistical_discrepancy	Input	Statistical deviation of the entire dataset. The value range is (0, 1). A smaller deviation value indicates higher precision.
compare_result	Output	Comparison result. true : The comparison is successful. false : The comparison failed.

8.1.2.3 GetContext

Function

Obtains the OMG context and returns it. This function is defined in **framework\omg\omg_types.h**.

The return value of the **GetContext** function is of the **OmgContext** type. For details, see **framework\omg\omg_types.h**.

Syntax

```
OmgContext& GetContext();
```

Parameter Description

None

8.1.2.4 AutoMappingFn

Function

Automatic mapping callback function. This function is defined in **framework\omg\register.h**.

Syntax

```
Status AutoMappingFn(const google::protobuf::Message* op_src, ge::Operator&  
op);
```

Parameter Description

Parameter	Input/ Output	Description
op_src	Input	Original operator before conversion
op	Input	Mapped operator

8.1.2.5 aicpu_run_func

Function

Pointer to the function of an AI CPU custom operator. The function name can be customized. This function is defined in **cce\customize.h**.

Syntax

```
void (*aicpu_run_func)(opTensor_t **, void **, int32_t,  
opTensor_t **, void **, int32_t, void *, rtStream_t);
```

Parameter Description

Parameter	Input/ Output	Description
opTensor_t **	Input	Input descriptor tensor. typedef struct tagOpTensor { // real dim info opTensorFormat_t format; //char align1[4]; opDataType_t data_type; //char align2[4]; int32_t dim_cnt; int32_t mm; //lint !e148 int32_t dim[CC_DEVICE_DIM_MAX]; //lint ! e148 } opTensor_t;
void **	Input	Address of the input descriptor tensor

Parameter	Input/ Output	Description
int32_t	Input	Number of input descriptors
opTensor_t **	Input	Output descriptor tensor
void **	Input	Address of the output descriptor tensor
int32_t	Input	Number of output descriptors
void *	Input	Attribute handle address
rtStream_t	Input	Stream identifier

8.1.3 Class Member Functions

8.1.3.1 Class StatusFactory

This class of functions is defined in **custom\common\fmk_error_codes.h**.

8.1.3.1.1 Instance

Function

Returns the instance object of the status factory class.

Syntax

```
static StatusFactory* Instance();
```

Parameter Description

None

8.1.3.1.2 RegisterErrorNo

Function

Registers error codes.

Syntax

```
void RegisterErrorNo(uint32_t err, const std::string& desc);
```

Parameter Description

Parameter	Input/ Output	Description
err	Input	Error code
desc	Input	Description of an error code, which is a character string

8.1.3.1.3 GetErrDesc

Function

Obtains the error code description.

Syntax

```
std::string GetErrDesc(uint32_t err);
```

Parameter Description

Parameter	Input/ Output	Description
err	Input	Error code

8.1.3.1.4 StatusFactory

Function

Constructor function

Syntax

```
StatusFactory();
```

Parameter Description

None

8.1.3.1.5 ~StatusFactory

Function

Destructor function

Syntax

```
~StatusFactory();
```

Parameter Description

None

8.1.3.2 Class ErrorNoRegisterar

This class of functions is defined in **custom\common\fmk_error_codes.h**.

8.1.3.2.1 ErrorNoRegisterar

Function

Constructor function

Syntax

```
ErrorNoRegisterar(uint32_t err, const std::string& desc);
```

Parameter Description

Parameter	Input/ Output	Description
err	Input	Error code
desc	Input	Description of an error code, which is a character string

8.1.3.2.2 ~ErrorNoRegisterar

Function

Destructor function

Syntax

```
~ErrorNoRegisterar();
```

Parameter Description

None

8.1.3.3 Class OpTypeContainer

This class of functions is defined in **custom\common\op_types.h**.

8.1.3.3.1 Instance

Function

Obtains the instance object of the **OpTypeContainer** factory class.

Syntax

```
static OpTypeContainer * Instance();
```

Parameter Description

None

8.1.3.3.2 Register

Function

Saves the operator type to a custom set.

Syntax

```
void Register(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type to be saved

8.1.3.3.3 IsExisting

Function

Checks whether an operator type exists.

Syntax

```
bool IsExisting(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type to be queried

8.1.3.3.4 OpTypeContainer

Function

Constructor function

Syntax

```
OpTypeContainer();
```

Parameter Description

None

8.1.3.4 Class OpTypeRegistrar

This class of functions is defined in `custom\common\op_types.h`.

8.1.3.4.1 OpTypeRegistrar

Function

Constructor function, which registers the type of an operator with the container

Syntax

```
OpTypeRegistrar(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Type of an operator to be registered

8.1.3.4.2 ~OpTypeRegistrar

Function

Destructor function

Syntax

```
~OpTypeRegistrar();
```

Parameter Description

None

8.1.3.5 Class OpRegistrationData

This class of functions is defined in **framework\omg\register.h**.

8.1.3.5.1 OpRegistrationData

Function

Constructor function

Syntax

```
OpRegistrationData(const std::string& om_optype);
```

Parameter Description

Parameter	Input/ Output	Description
om_optype	Input	Operator type

8.1.3.5.2 ~OpRegistrationData

Function

Destructor function

Syntax

```
~OpRegistrationData();
```

Parameter Description

None

8.1.3.5.3 FrameworkType

Function

Sets the framework type.

Syntax

```
OpRegistrationData& FrameworkType(domi::FrameworkType fmk_type);
```

Parameter Description

Parameter	Input/ Output	Description
fmk_type	Input	Framework type 0: Caffe 3: TensorFlow

8.1.3.5.4 OriginOpType

Function

Sets the operator type or operator type list of the original model.

Syntax

- Setting the operator type list:
`OpRegistrationData& OriginOpType (const std::initializer_list<std::string> & ori_optype_list);`
- Setting the operator type:
`OpRegistrationData& OriginOpType (const std::string& ori_optype);`

Parameter Description

Parameter	Input/ Output	Description
ori_optype_list	Input	Operator type list of the original model
ori_optype	Input	Operator type of the original model

8.1.3.5.5 ParseParamsFn

Function

Parses parameters.

Syntax

`OpRegistrationData& ParseParamsFn(ParseParamFunc parseParamFn);`

Parameter Description

Parameter	Input/ Output	Description
parseParamFn	Input	Callback function ParseParamFunc. For details, see Callback Function ParseParamFunc .

Callback Function ParseParamFunc

You can customize and implement the ParseParamFunc class functions to convert the parameters and weights of the Caffe model and fill the results in the Operator class.

Syntax

```
Status ParseParamFunc(const Message* op_origin, ge::Operator& op_dest);
```

Parameter Description

Parameter	Input/ Output	Description
op_origin	Input	Data structure in protobuf format (from the prototxt file of the Caffe model), including operator parameter information
op_dest	Output	Operator data structure of offline model supported by the Ascend AI processor, which stores operator information For details about the operator class, see Operator Class APIs in <i>GE API Reference</i> .

8.1.3.5.6 InferShapeAndTypeFn

Function

Shape inference function

Syntax

```
OpRegistrationData& InferShapeAndTypeFn(InferShapeFunc inferShapeFn);
```

Parameter Description

Parameter	Input/Output	Description
inferShapeFn	Input	Callback function InferShapeFunc. For details, see Callback Function InferShapeFunc .

Callback Function InferShapeFunc

You can customize and implement the InferShapeFunc class function to obtain the output description of an operator, including the tensor description such as the output shape information and data type.

Syntax

```
Status InferShapeFunc(const ge::Operator& op, vector<ge::TensorDesc>& v_output_desc);
```

Parameter Description

Parameter	Input/Output	Description
op	Input	Operator data structure of offline model supported by the Ascend AI processor For details about the operator class, see Operator Class APIs in <i>GE API Reference</i> .
v_output_desc	Output	Operator output description For details about the TensorDesc class, see TensorDesc Class APIs in <i>GE API Reference</i> .

8.1.3.5.7 UpdateOpDescFn

Function

Operator description update function

Syntax

```
OpRegistrationData& UpdateOpDescFn(UpdateOpDescFunc updateOpDescFn);
```

Parameter Description

Parameter	Input/ Output	Description
updateOpDescFn	Input	Callback function UpdateOpDescFunc. For details, see Callback Function UpdateOpDescFunc .

Callback Function UpdateOpDescFunc

You can customize and implement the UpdateOpDescFunc class function to update operator information.

Syntax

```
static Status UpdateOpDescFunc(ge::Operator& op);
```

Parameter Description

Parameter	Input/ Output	Description
op	Input/ Output	Operator data structure of offline model supported by the Ascend AI processor, which stores operator information For details about the operator class, see Operator Class APIs in <i>GE API Reference</i> .

8.1.3.5.8 GetWorkspaceSizeFn

Function

Obtains the size of workspace.

Syntax

```
OpRegistrationData& GetWorkspaceSizeFn(GetWorkspaceSizeFunc  
getWorkspaceSizeFn);
```

Parameter Description

Parameter	Input/ Output	Description
getWorkspaceSizeFn	Input	Callback function GetWorkspaceSizeFunc. For details, see Callback Function GetWorkspaceSizeFunc .

Callback Function GetWorkspaceSizeFunc

You can customize and implement GetWorkspaceSizeFunc class function to calculate the workspace size of an operator and export the calculation result.

Syntax

```
Status GetWorkspaceSizeFn(domi::Status (const ge::Operator& op,  
std::vector<int64_t>& size));
```

Parameter Description

Parameter	Input/ Output	Description
op	Input	Operator object whose workspace needs to be calculated For details about the operator class, see Operator Class APIs in <i>GE API Reference</i> .
size	Output	Size of the workspace. The value needs to be calculated and provided by the user.

8.1.3.5.9 TEBinBuildFn

Function

Build callback function

Syntax

```
OpRegistrationData& TEBinBuildFn(BuildTeBinFunc buildTeBinFn);
```

Parameter Description

Parameter	Input/ Output	Description
buildTeBinFn	Input	Callback function BuildTeBinFunc. For details, see Callback Function BuildTeBinFunc .

Callback Function BuildTeBinFunc

You can customize and implement the BuildTeBinFunc class function to construct the operator binary file.

Syntax

```
virtual Status BuildTeBinFunc(const ge::Operator& op, TEBinInfo& teBinInfo);
```

Parameter Description

Parameter	Input/ Output	Description
op	Input	Operator data structure of offline model supported by the Ascend AI processor, which stores operator information For details about the operator class, see Operator Class APIs in <i>GE API Reference</i> .
teBinInfo	Output	Path of the binary file of a custom operator and DDK description struct TEBinInfo { std::string bin_file_path; // Automatically obtained from the binFileName field in the JSON file. To ensure compatibility with cases written by users, the field is not deleted. std::string json_file_path; std::string ddk_version; };

8.1.3.5.10 ImpliedType

Function

Sets the operator execution mode.

Syntax

```
OpRegistrationData& ImpliedType(domi::ImpliedType implied_type);
```

Parameter Description

Parameter	Input/ Output	Description
imply_type	Input	Operator execution mode. enum class ImplyType : unsigned int { BUILDIN = 0, // Built-in operator, which is normally executed by the OME TVM, // Executed after being compiled into a TVM binary file CUSTOM, // The calculation logic is customized by the user and executed by the CPU. AI_CPU, // AI CPU custom operator type INVALID = 0xFFFFFFFF, };

8.1.3.5.11 Formats

Function

Sets the input/output data formats supported by an operator.

Syntax

```
OpRegistrationData& Formats(  
const std::initializer_list<domi::tagDomiTensorFormat>& input_formats,  
const std::initializer_list<domi::tagDomiTensorFormat>& output_formats);  
OpRegistrationData& Formats(  
const domi::tagDomiTensorFormat& input_format,  
const domi::tagDomiTensorFormat& output_format);
```

Parameter Description

Parameter	Input/ Output	Description
input_format	Input	Data format supported by the input
output_format	Input	Data format supported by the output

8.1.3.5.12 WeightFormats

Function

Sets the data formats supported by the operator weight.

Syntax

```
OpRegistrationData& WeightFormats(  
    const std::initializer_list<domi::tagDomiTensorFormat>& weight_formats);
```

Parameter Description

Parameter	Input/ Output	Description
weight_formats	Input	Data formats supported by the weight: typedef enum tagDomiTensorFormat { DOMI_TENSOR_NCHW = 0, /**< NCHW */ DOMI_TENSOR_NHWC, /**< NHWC */ DOMI_TENSOR_ND, /**< Nd Tensor */ DOMI_TENSOR_NC1HWC0, /**< NC1HWC0 */ DOMI_TENSOR_FRACTAL_Z, /**< FRACTAL_Z */ DOMI_TENSOR_NC1C0HWPAD, DOMI_TENSOR_NHWC1C0, DOMI_TENSOR_FSR_NCHW, DOMI_TENSOR_FRACTAL_DECONV, DOMI_TENSOR_BN_WEIGHT, DOMI_TENSOR_CHWN, /*Android NN Depth CONV*/ DOMI_TENSOR_FILTER_HWCK, /* filter input tensor format */ DOMI_TENSOR_RESERVED } domiTensorFormat_t;

8.1.3.5.13 Finalize

Function

Creates the parser and builder functions of all supported operators. If the creation is successful, **true** is returned. Otherwise, **false** is returned.

Syntax

```
bool Finalize();
```

Parameter Description

None

8.1.3.6 Class OpRegistry

This class of functions is defined in **framework\omg\register.h**.

8.1.3.6.1 Instance

Function

Obtains the instance of an object of the **OpRegistry** class.

Syntax

```
static OpRegistry* Instance();
```

Parameter Description

None

8.1.3.6.2 Register

Function

Registers **OpRegistrationData**. If the registration is successful, **true** is returned. Otherwise, **false** is returned.

Syntax

```
bool Register(const OpRegistrationData& reg_data);
```

Parameter Description

Parameter	Input/ Output	Description
reg_data	Input	Operator registration information

8.1.3.6.3 GetImPLYType

Function

Obtains the operator execution type.

Syntax

```
domi::ImPLYType GetImPLYType(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type to be queried

8.1.3.6.4 GetOpTypeByImPLYType

Function

Queries the names of all operators of a specified execution type.

Syntax

```
void GetOpTypeByImPLYType(std::vector<std::string>& vec_op_type,const  
domi::ImPLYType& imPLY_type);
```

Parameter Description

Parameter	Input/ Output	Description
vec_op_type	Output	Operator name information to be saved and queried
imPLY_type	Input	Operator execution type

8.1.3.6.5 GetFormats

Function

Queries the input and output data types supported by the **op_type** operator.

Syntax

```
void GetFormats(const std::string& op_type,  
std::vector<domi::tagDomiTensorFormat>&  
input_format_vector,std::vector<domi::tagDomiTensorFormat>&  
output_format_vector);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Name of an operator type
input_format_vector	Output	Supported input data types
output_format_vector	Output	Supported output data types

8.1.3.6.6 GetWeightFormats

Function

Queries the weight data type supported by the **op_type** operator.

Syntax

```
void GetWeightFormats(const std::string& op_type,  
std::vector<domi::tagDomiTensorFormat>& format_vector);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Name of an operator type
format_vector	Output	Supported weight data types

8.1.3.6.7 GetParseParamFunc

Function

Obtains the callback function for parsing parameters.

Syntax

```
domi::ParseParamFunc GetParseParamFunc(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type

8.1.3.6.8 GetInferShapeFunc

Function

Obtains the inference shape callback function.

Syntax

```
domi::InferShapeFunc GetInferShapeFunc(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type

8.1.3.6.9 GetGetWorkspaceSizeFunc

Function

Obtains the callback function for the workspace size.

Syntax

```
domi::GetWorkspaceSizeFunc GetGetWorkspaceSizeFunc(const std::string&  
op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type

8.1.3.6.10 GetUpdateOpDescFunc

Function

Obtains the callback function for updating the operator description.

Syntax

```
domi::UpdateOpDescFunc GetUpdateOpDescFunc(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type

8.1.3.6.11 GetBuildTeBinFunc

Function

Obtains the **build** callback function.

Syntax

```
domi::BuildTeBinFunc GetBuildTeBinFunc(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type

8.1.3.6.12 GetTransWeightFunc

Function

Obtains the callback function for transferring the weight.

Syntax

```
domi::TransWeightFunc GetTransWeightFunc(const std::string& op_type);
```

Parameter Description

Parameter	Input/ Output	Description
op_type	Input	Operator type

8.1.3.7 Class OpReceiver

This class of functions is defined in **framework\omg\register.h**.

8.1.3.7.1 OpReceiver

Function

Constructor function

Syntax

```
OpReceiver(OpRegistrationData& reg_data);
```

Parameter Description

Parameter	Input/ Output	Description
reg_data	Input	Operator information to be registered

8.1.3.7.2 ~OpReceiver

Function

Destructor function

Syntax

```
~OpReceiver();
```

Parameter Description

None

8.1.4 Macro Definitions

8.1.4.1 Macro for Generating Error Codes

Function

Generates the error codes of a specified module by calling **DEF_ERRORNO**.

Syntax

```
DEF_ERRORNO(sysid, modid, name, value, desc)
```

Parameter Description

Parameter	Input/ Output	Description
sysid	Input	System ID

Parameter	Input/ Output	Description
modid	Input	Module ID
name	Input	Name of an error code
value	Input	Actual value of an error code
desc	Input	Description of an error code, which is a character string

8.1.4.2 Macro for Generating Error Codes of the Common Module

Function

Generates the error codes of the common module by calling **DEF_ERRORNO_COMMON**.

Syntax

DEF_ERRORNO_COMMON(name, value, desc)

Parameter Description

Parameter	Input/ Output	Description
name	Input	Name of an error code
value	Input	Actual value of an error code
desc	Input	Description of an error code, which is a character string

8.1.4.3 Macro for Generating Error Codes of the OMG Module

Function

Generates the error codes of the OMG module by calling **DEF_ERRORNO_OMG**.

Syntax

DEF_ERRORNO_OMG(name, value,desc)

Parameter Description

Parameter	Input/ Output	Description
name	Input	Name of an error code
value	Input	Actual value of an error code
desc	Input	Description of an error code, which is a character string

8.1.4.4 Macro for Generating Error Codes of the OME Module

Function

Generates the error codes of the OME module by calling **DEF_ERRORNO_OME**.

Syntax

DEF_ERRORNO_OME(name, value, desc)

Parameter Description

Parameter	Input/ Output	Description
name	Input	Name of an error code
value	Input	Actual value of an error code
desc	Input	Description of an error code, which is a character string

8.1.4.5 Macro for Generating Error Codes of the CALIBRATION Module

Function

Generates the error codes of the CALIBRATION module by calling **DEF_ERRORNO_CALIBRATION**.

Syntax

DEF_ERRORNO_CALIBRATION(name, value, desc)

Parameter Description

Parameter	Input/ Output	Description
name	Input	Name of an error code
value	Input	Actual value of an error code
desc	Input	Description of an error code, which is a character string

8.1.4.6 Macro for Obtaining Error Code Description.

Function

Obtains the error code description by calling **GET_ERRORNO_STR**.

Syntax

GET_ERRORNO_STR(value)

Parameter Description

Parameter	Input/ Output	Description
value	Input	Value of an error code

8.1.4.7 Macro for Registering the Operator Type

Function

Register an operator type, mainly to declare a constant string, which is the name of the operator type, and then save the string to a set.

Syntax

REGISTER_OPTYPE_DECLARE(var_name, str_name)

REGISTER_OPTYPE_DEFINE(var_name, str_name)

Parameter Description

Parameter	Input/ Output	Description
var_name	Input	Name of a constant string

Parameter	Input/ Output	Description
str_name	Input	Name of an operator type

8.1.4.8 Macro Indicating Whether an Operator Type Exists

Function

Checks whether an operator type exists.

Syntax

IS_OPTYPE_EXISTING(str_name)

Parameter Description

Parameter	Input/ Output	Description
str_name	Input	Name of an operator type

8.1.4.9 Macro for Registering an Operator

8.1.4.9.1 REGISTER_CUSTOM_OP

Function

Registers an operator with a specified name.

Syntax

REGISTER_CUSTOM_OP(name)

Parameter Description

Parameter	Input/ Output	Description
name	Input	Operator name

8.1.4.10 Macros for Marking External APIs

8.1.4.10.1 FMK_FUNC_HOST_VISIBILITY

Function

Marks APIs exposed to external use on the host, so that the APIs are visible and can be called in the dynamic link library.

Syntax

FMK_FUNC_HOST_VISIBILITY

Parameter Description

None

8.1.4.10.2 FMK_FUNC_DEV_VISIBILITY

Function

Marks APIs exposed to external use on the device, so that the APIs are visible and can be called in the dynamic link library.

Syntax

FMK_FUNC_DEV_VISIBILITY

Parameter Description

None

8.2 Matrix API Reference

8.2.1 Overview

Matrix runs above the operating system (OS) and below service applications. It provides unified standard application programming interfaces (APIs) for applications, shielding differences between OSs. It also provides multi-node scheduling and multi-process management. In addition, Matrix can set up and run service processes based on configuration files and summarize statistics.

The Matrix logic consists of three parts: Matrix Agent (running on the host side), Matrix Daemon (running on the device side), and Matrix Service (running on the device side).

NOTE

- The host side refers to the x86 server, ARM server, or Windows PC connected to the device side. The host side uses the neural-network (NN) computing capability provided by the device side to implement services.
- The device side refers to an Ascend 310-powered hardware device that provides the host side with the NN computing capability over the PCIe interface.
- The digital vision pre-processing (DVPP) implements video encoding and decoding, JPEG encoding and decoding, PNG decoding, and vision pre-processing.
- Framework is a deep learning framework that can convert models under open source frameworks such as Caffe into models supported by Mind Studio.
- The cube-based computing engine (CCE) acceleration library accelerates upper-layer applications (frameworks or applications provided for machine learning) through APIs.
- Runtime runs in the application process space and provides Ascend 310-powered devices with various functions, such as memory management, device management, stream management, event management, and kernel execution.

Matrix is a general-purpose service process execution engine that consists of the following parts:

- **Matrix Agent:** It runs on the host side and provides the following functions:
 - Interacts with the host application for controlling commands and processing data.
 - Implements interprocess communication (IPC) with the device side.
- **Matrix Daemon:** It runs on the device side and provides the following functions:
 - Set up a service process based on the configuration file.
 - Destroys the service process and reclaims resources according to commands.
 - Serves as the Daemon process, which is used to start the Matrix process.
- **Matrix Service:** It runs on the device side and provides the following functions:
 - Implements media preprocessing by using the DVPP APIs called by engines.
 - Implements model inference by using the AIModelManger APIs called by engines.

You can view the API definition files in **ddk/include/inc/hiaiengine** under the installation directory of the device development kit (DDK). If you install both Mind Studio and the DDK by following the installation wizard, you can log in to the Mind Studio server as the Mind Studio installation user and view the API definition file in **~/tools/che/ddk/ddk/include/inc/hiaiengine**. For details about the mapping between the definition files and the APIs, see the API description.

8.2.2 Process Orchestration APIs

8.2.2.1 Process Connection APIs (C Language)

8.2.2.1.1 HIAI_Init (Common with C++)

Initializes the host device communication (HDC) module of the Matrix. The device ID specified in the graph configuration file is preferred. Otherwise, use the value of **deviceId** configured below. This API is defined in **init.h**.

Syntax

HIAI_StatusT HIAI_Init(uint32_t deviceId)

NOTE

For details about the definition of the **HIAI_StatusT** structure, see [8.2.7.2.2 Matrix Custom Data Types](#).

Parameter Description

Parameter	Description	Value Range
deviceId	Device ID. The default value is 0 .	0–63

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_INIT_FAIL	HiAI initialization fails.

8.2.2.1.2 HIAI_CreateGraph

Creates and starts the entire graph. This API supports the connection between graphs in a single configuration file rather than in different configuration files. It can be called only on the host or Ascend 310 RC. This API is defined in **c_graph.h**.

Syntax

HIAI_StatusT HIAI_CreateGraph(const char* configFile, size_t len)

Parameter Description

Parameter	Description	Value Range
configFile	Configuration file path. Ensure that the transferred file path is correct.	A single configuration file supports a maximum of 2048 graphs. However, restricted by system resources, only several or dozens of graphs are supported based on different hardware configurations and graph sizes.
len	Name length of the configuration file	1-255

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_PROTO_FILE_PARSE_FAILED	ParseFromString return failed.

8.2.2.1.3 HIAI_DestroyGraph

Destroys a graph. It can be called only on the host or Ascend 310 RC. This API is defined in **c_graph.h**.

Syntax

HIAI_StatusT HIAI_DestroyGraph (uint32_t graphID)

Parameter Description

Parameter	Description	Value Range
graphID	Specified graph ID	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_DESTROY_TIMEO UT	Destroying a graph timed out.
3	HIAI_GRAPH_ENGINE_DESTROY Y_FAILED	Destroying an engine failed.

8.2.2.2 Process Connection APIs (C++ Language)

8.2.2.2.1 Graph::GetInstance

Obtains a graph instance. After a graph is created, this API can be used. This API is defined in **graph.h**.

Syntax

```
static std::shared_ptr<Graph> Graph::GetInstance(uint32_t graphID)
```

Parameter Description

Parameter	Description	Value Range
graphID	ID of the target graph	Graph ID configured in the configuration file

Return Value

Smart pointer to the graph

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_PROTO_FILE_PARSE_FAILED	ParseFromString return failed.

8.2.2.2.2 Graph::CreateGraph (Creating a Graph Based on the Configuration File)

Creates and starts the entire graph. This API supports the connection between graphs in a single configuration file rather than in different configuration files. This API is defined in **graph.h**.

It can be called only on the host or Ascend 310 RC.

Syntax

```
static HIAI_StatusT Graph::CreateGraph(const std::string& configFile)
```

Parameter Description

Parameter	Description	Value Range
configFile	Path of the graph configuration file. Ensure that the transferred file path is correct.	A single configuration file supports a maximum of 2048 graphs. However, restricted by system resources, only several or dozens of graphs are supported based on different hardware configurations and graph sizes.

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_GET_INSTANCE_NULL	A null instance is obtained.

8.2.2.2.3 Graph::CreateGraph (Creating a Graph Based on the Configuration File and Writing the Generated Graph Back to the List)

Overloads [8.2.2.2.2 Graph::CreateGraph \(Creating a Graph Based on the Configuration File\)](#). The Matrix creates a graph based on the configuration file and writes the generated graph back to the list. This API is defined in **graph.h**.

It can be called only on the host or Ascend 310 RC.

Syntax

```
static HIAI_StatusT Graph::CreateGraph(const std::string& configFile,  
std::list<std::shared_ptr<Graph>>& graphList)
```

Parameter Description

Parameter	Description	Value Range
configFile	Path of the graph configuration file. Ensure that the transferred file path is correct.	A single configuration file supports a maximum of 2048 graphs. However, restricted by system resources, only several or dozens of graphs are supported based on different hardware configurations and graph sizes.
graphList	User-defined list. The Matrix writes the generated graph back to the list.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_GET_INSTANCE_NULL	A null instance is obtained.

8.2.2.2.4 Graph::CreateGraph (Creating a Graph Based on the Protobuf Data Format)

Creates and starts the entire graph based on the Protobuf data format. This API is defined in **graph.h**.

It can be called only on the host or Ascend 310 RC.

Syntax

```
static HIAI_StatusT Graph::CreateGraph(const GraphConfigList& graphConfig)
```

Parameter Description

Parameter	Description	Value Range
graphConfig	Protobuf data format. When writing code, you can define the graphConfig parameter as follows: - Define a parameter of the GraphConfigList type as the input parameter graphConfig of the Graph::CreateGraph(const GraphConfigList& graphConfig) API. - After the API in 8.2.5.1.12 Parsing the Matrix Configuration File is called, obtain the graphConfigList parameter and use it as the input parameter graphConfig of the Graph::CreateGraph(const GraphConfigList& graphConfig) API.	The number of engines in a graph must be less than or equal to 512.

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_GET_INSTANCE_NULL	A null instance is obtained.

8.2.2.2.5 Graph::DestroyGraph

Destroys the entire graph. This API is defined in **graph.h**.

It can be called only on the host or Ascend 310 RC.

Syntax

```
static HIAI_StatusT Graph::DestroyGraph(uint32_t graphID)
```

Parameter Description

Parameter	Description	Value Range
graphID	ID of the graph to be destroyed	Graph ID configured in the configuration file

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_DESTROY_TIMEO UT	Destroying a graph timed out.
3	HIAI_GRAPH_ENGINE_DESTROY_FAILED	Destroying a graph failed.

8.2.2.2.6 Graph::UpdateEngineConfig

Updates the parameters of a specified engine. This API is defined in **graph.h**.

Application scenario:

This API allows you to update the specified running parameters in the running process.

For example, in the video snapshot scenario, the running process needs to be updated based on the change of day and night. Therefore, you need to update the values for day and night to the engine by using the **UpdateEngineConfig** function.

Syntax

```
static HIAI_StatusT Graph::UpdateEngineConfig(const uint32_t& graphId,  
const uint32_t& engineId, const AIConfig& aiConfig, const bool& syncFlag =  
false)
```

Parameter Description

Parameter	Description	Value Range
graphId	Graph ID	-
engineId	engine ID	-
aiConfig	Configuration parameter	-
syncFlag	Synchronous or asynchronous execution. For synchronous execution, wait for the returned result.	true/false

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_MEMORY_POOL_NOT_EXISTED	The memory pool does not exist.
4	HIAI_GRAPH_MALLOC_LARGER	The buffer fails to be allocated because the size is larger than 256 MB.
5	HIAI_MEMORY_POOL_UPDATE_FAILED	The memory pool fails to be updated.
6	HIAI_GRAPH_SENDMSG_FAILED	The HDC module fails to send a message.
7	HIAI_GRAPH_MEMORY_POOL_INITED	The memory pool has been initialized.
8	HIAI_GRAPH_NO_MEMORY	There is no memory.

8.2.2.3 Data Sending APIs (C Language)

8.2.2.3.1 HIAI_SetDataRecvFunctor

Sets the callback function for an engine in the graph to receive messages. This API is defined in **c_graph.h**.

 NOTE

Callback function type: **typedef HIAI_StatusT (*HIAI_RecvDataCallbackT)(void*)**

Syntax

**HIAI_StatusT HIAI_SetDataRecvFunc(HIAI_PortID_t targetPortConfig,
HIAI_RecvDataCallbackT recvCallback)**

Parameter Description

Parameter	Description	Value Range
targetPortConfig	Graph ID, engine ID, and port ID of the data receiver	-
recvCallback	Callback function for setting the mode of receiving the user-defined result	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_PORT_ID_ERROR	The port ID has an error.

8.2.2.3.2 HIAI_C_SendData

Sends data or messages to the Matrix from an external system. This API is defined in **c_graph.h**.

 NOTE

- The user-defined message type needs to be registered by calling the API in [8.2.5.2 Data Type Serialization and Deserialization \(C++ Language\)](#).
- The message pointer **dataPtr** is controlled and released by the caller. The Matrix does not manage this pointer.
- The **SendData** interface uses the DMA transfer mode, which may affect the CPU's timely response to and processing of interrupt requests, for example, memory allocation by calling the **new** or **malloc** function.

Syntax

```
HIAI_StatusT HIAI_C_SendData(HIAI_PortID_t targetPortConfig, const char*  
messageName, size_t len, void* dataPtr)
```

Parameter Description

Parameter	Description	Value Range
targetPortConfig	Graph ID, engine ID, and port ID of the data receiver	-
messageName	Name of the message that is being sent	-
len	Length of a message name	-
dataPtr	Pointer to a specific message. The message pointer is controlled and released by the caller. The Matrix does not manage this pointer.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_ENGINE_NOT_EXIST	The engine does not exist.
3	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
4	HIAI_GRAPH_NO_USEFUL_MEMORY	There is no useful memory.
5	HIAI_GRAPH_SENDMSG_FAILED	The HDC module fails to send a message.
6	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.

8.2.2.4 Data Sending APIs (C++ Language)

8.2.2.4.1 Graph::SetDataRecvFunc

Sets the callback function for an engine in the graph to receive messages. The running environment of the code that calls the API must be the same as that of the engine. For example, if the engine runs on the device side, the code that calls the function must also run in the device side. This API is defined in **graph.h**.

This API, which must be used together with the [DataRecvInterface::RecvData](#) API, is generally used to return the inference result to the user. The detailed principles are as follows:

1. Define a subclass (for example, **DdkDataRecvInterface**) of class **DataRecvInterface** and initialize the object of the subclass.
2. Set the object of class **DdkDataRecvInterface** to a callback function by calling **Graph::SetDataRecvFunctor**.
3. Send the input data to the corresponding output port by calling **Engine::SendData**.
4. Return the inference result by calling the **RecvData** function of class **DdkDataRecvInterface**.

Syntax

```
static HIAI_StatusT Graph::SetDataRecvFunctor(const EnginePortID&  
targetPortConfig, const std::shared_ptr<DataRecvInterface>& dataRecv);
```

Parameter Description

Parameter	Description	Value Range
targetPortConfig	Graph ID, engine ID, and port ID of the data receiver	-
dataRecv	Callback function for receiving user-defined data	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_PORT_ID_ERROR	The port ID has an error.

8.2.2.4.2 Graph::SendData

Sends **void** data from an external system to a specified port on the Matrix. This API is defined in **graph.h**.

NOTE

The **SendData** interface uses the DMA transfer mode, which may affect the CPU's timely response to and processing of interrupt requests, for example, memory allocation by calling the **new** or **malloc** function.

Syntax

```
HIAI_StatusT Graph::SendData(const EnginePortID& targetPortConfig, const  
std::string& messageName, const std::shared_ptr<void>& dataPtr, const  
uint32_t timeout = 500)
```

Parameter Description

Parameter	Description	Value Range
targetPortConfig	Graph ID, engine ID, and port ID of the data receiver	-
messageName	Name of a message	-
dataPtr	Pointer to a message	-
timeOut	Timeout for calling this API to send data. The default timeout is 500 ms. If data sending times out, the system background attempts to send data again. A maximum of 16 attempts are allowed.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_ENGINE_NOT_EXIST	The engine does not exist.
3	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
4	HIAI_GRAPH_NO_USEFUL_MEMORY	There is no useful memory.

No.	Error Code	Description
5	HIAI_GRAPH_SENDMSG_FAIL	The HDC module fails to send a message.
6	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.

8.2.2.4.3 HIAI_SendData

Sends **void** data from an external system to a specified port on the Matrix. This API is defined in **c_graph.h**.

NOTE

The **SendData** interface uses the DMA transfer mode, which may affect the CPU's timely response to and processing of interrupt requests, for example, memory allocation by calling the **new** or **malloc** function.

Syntax

HIAI_StatusT HIAI_SendData (HIAI_PortID_t targetPortConfig, const std::string& messageName, std::shared_ptr<void>& dataPtr, uint32_t timeOut=0)

Parameter Description

Parameter	Description	Value Range
targetPortConfig	Graph ID, engine ID, and port ID of the data receiver	-
messageName	Name of a message	-
dataPtr	Pointer to a message	-
timeOut	Timeout for data sending, in ms. If the TX queue is full or the memory pool is insufficient, blocking is enabled based on the timeout configuration. The default value is 0 , indicating no blocking.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_ENGINE_NOT_EXIST	The engine does not exist.
3	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
4	HIAI_GRAPH_NO_USEFUL_MEMORY	There is no useful memory.
5	HIAI_GRAPH_SENDMSG_FAILED	The HDC module fails to send a message.
6	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.

8.2.2.5 API Usage Example

8.2.2.5.1 Example of Using the C APIs

```
/**
 * @file graph_c_api_example.cpp
 *
 * Copyright(C), 2017 - 2017, Huawei Tech. Co., Ltd. ALL RIGHTS RESERVED.
 *
 * @Source Files for HIAI Graph Orchestration
 *
 * @version 1.0
 */
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#include "hiaengine/c_api.h"
#include "user_def_data_type.h"

/*****
 * Normally the mandatory steps to set up the whole HIAI graph include:
 * 1) Call the HIAI_Init() API to perform global system initialization;
 * 2) Call HIAI_CreateGraph to create and start a graph.
 * 3) Set profile config if necessary.
 * 4) Call HIAI_DestroyGraph to destroy the graph finally.
 */
*****/

// Just define variables which are used in only this example
// and are not required in real user case.
#define MATRIX_USER_SPECIFY_ENGINE_PRIORITY (10)

static uint32_t g_graph_id = 100;
static uint32_t src_engine_id = 1000; // Defined in graph.prototxt
static uint32_t dest_engine_id = 1002; // Defined in graph.prototxt
static const char graph_config_proto_file[] = "llt/hiaengine/st/examples/config/graph.prototxt";
static const char graph_config_len = sizeof(graph_config_proto_file);

// Define End
```

```
// Receive user-defined data (recv_data is automatically released by the framework).
HIAI_StatusT UserDefDataRecvCallBack(void* recv_data)
{
    // Process the received data.
    return HIAI_OK;
}

/*****
* HIAIEngine Example
* Graph:
* SrcEngine<-->Engine<-->DestEngine
*
*****/
HIAI_StatusT HIAI_InitAndStartGraph()
{
    // Step1: Global System Initialization before using Matrix
    HIAI_StatusT status = HIAI_Init(0);

    // Step2: Create and Start the Graph
    status = HIAI_CreateGraph(graph_config_proto_file, graph_config_len);
    if (status != HIAI_OK)
    {
        return status;
    }

    // Step3: Set the callback function to receive data.
    HIAI_PortID_t engine_id;
    engine_id.graph_id = g_graph_id;
    engine_id.engine_id = dest_engine_id;
    engine_id.port_id = 0;
    HIAI_SetDataRecvFunc(engine_id, UserDefDataRecvCallBack);

    return HIAI_OK;
}

// User Application Main Example
int main()
{
    HIAI_StatusT ret = HIAI_OK;

    // Creation process
    ret = HIAI_InitAndStartGraph();

    if (HIAI_OK != ret)
    {
        return -1;
    }

    // Read data and inject data into the process.
    HIAI_PortID_t engine_id;
    engine_id.graph_id = g_graph_id;
    engine_id.engine_id = src_engine_id;
    engine_id.port_id = 0;

    int is_end_of_data = 0;
    while (!is_end_of_data)
    {
        UserDefDataType* user_def_msg = (UserDefDataType*)malloc(sizeof(UserDefDataType));

        // Read data and pad UserDefDataType. For details about the data type registration, see the
        // corresponding section in this document.

        const char * message_name = "UserDefDataType";
        size_t msg_len = strlen(message_name);
        if (HIAI_OK != HIAI_C_SendData(engine_id, message_name, msg_len, user_def_msg))
        {
            // When the queue is full, a message is returned indicating that sending data fails. The service logic

```

```
determines whether to resend or discard the data.
    break;
}

    free(user_def_msg);
    user_def_msg = nullptr;
}

// After the processing is complete, delete the entire graph.
HIAI_DestroyGraph(g_graph_id);
return 0;
}
```

8.2.2.5.2 Example of Using C++ APIs (Creating a Graph Based on the Configuration File)

```
/**
 * @file graph_cplusplus_api_example.cpp
 *
 * Copyright(C), 2017 - 2017, Huawei Tech. Co., Ltd. ALL RIGHTS RESERVED.
 *
 * @Source Files for HIAI Graph Orchestration
 *
 * @version 1.0
 */
#include <unistd.h>
#include <thread>
#include <fstream>
#include <algorithm>

#include "hiaengine/api.h"
#include "user_def_data_type.h"

/*****
 * Normally the mandatory steps to set up the whole HIAI graph include:
 * 1) Call the HIAI_Init() API to perform global system initialization;
 * 2) Call HIAI_CreateGraph or Graph::CreateGraph to create and start a graph.
 * 3) Set profile config if necessary.
 * 4) Call HIAI_DestroyGraph or Graph::DestroyGraph to destroy the graph finally.
 *****/

// Just define variables which are used in only this example
// and are not required in real user case.
#define MATRIX_USER_SPECIFY_ENGINE_PRIORITY (10)

static uint32_t g_graph_id = 100;
static uint32_t src_engine_id = 1000; // Defined in graph.prototxt
static uint32_t dest_engine_id = 1002; // Defined in graph.prototxt
static std::string graph_config_proto_file = "./config/graph.prototxt";
// Define End

namespace hiai {

class GraphDataRecvInterface: public DataRecvInterface
{
public:
    GraphDataRecvInterface()
    {
    }
}

/**
 * @ingroup hiaiengine
 * @brief Read and save the data.
 * @param [in] in_data Input data
 * @return HIAI Status
 */
}
```

```
HIAI_StatusT RecvData(const std::shared_ptr<void>& message)
{
    // Convert the data to a specific message type and process the related message. For example:
    // shared_ptr<std::string> data =
    // std::static_pointer_cast<std::string>(message);
    return HIAI_OK;
}
private:
};
}

/*****
* HIAIEngine Example
* Graph:
* SrcEngine<-->Engine<-->DestEngine
*
*****/
HIAI_StatusT HIAI_InitAndStartGraph()
{
    // Step1: Global System Initialization before using Matrix
    HIAI_StatusT status = HIAI_Init(0);

    // Step2: Create and Start the Graph
    status = hiiai::Graph::CreateGraph(graph_config_proto_file);
    if (status != HIAI_OK)
    {
        HIAI_ENGINE_LOG(status, "Fail to start graph");
        return status;
    }

    // Step3: Set the callback function to receive data.
    std::shared_ptr<hiiai::Graph> graph = hiiai::Graph::GetInstance(g_graph_id);
    if (nullptr == graph)
    {
        HIAI_ENGINE_LOG("Fail to get the graph-%u", g_graph_id);
        return status;
    }

    // Specify the port id (default to zero)
    hiiai::EnginePortID target_port_config;
    target_port_config.graph_id = g_graph_id;
    target_port_config.engine_id = dest_engine_id;
    target_port_config.port_id = 0;

    graph->SetDataRecvFunctor(target_port_config,
        std::shared_ptr<hiiai::GraphDataRecvInterface>(
            new hiiai::GraphDataRecvInterface()));

    return HIAI_OK;
}

// User Application Main Example
int main()
{
    HIAI_StatusT ret = HIAI_OK;

    // Creation process
    ret = HIAI_InitAndStartGraph();

    if(HIAI_OK != ret)
    {
        HIAI_ENGINE_LOG("Fail to start graph");;
        return -1;
    }

    // Read data and inject data into the process.
    std::shared_ptr<hiiai::Graph> graph = hiiai::Graph::GetInstance(g_graph_id);
    if (nullptr == graph)
```

```
{
    HIAI_ENGINE_LOG("Fail to get the graph-%u", g_graph_id);
    return -1;
}

hiai::EnginePortID engine_id;
engine_id.graph_id = g_graph_id;
engine_id.engine_id = src_engine_id;
engine_id.port_id = 0;

bool is_end_of_data = false;
while (!is_end_of_data)
{
    std::shared_ptr<UserDefDataType> user_def_msg(new UserDefDataType);

    // Read data and pad UserDefDataType. For details about the data type registration, see the
    corresponding section in this document.
    // Inject data to the graph.
    if (HIAI_OK != graph->SendData(engine_id, "UserDefDataType",
        std::static_pointer_cast<void>(user_def_msg)))
    {
        // When the queue is full, a message is returned indicating that sending data fails. The service logic
        determines whether to resend or discard the data.
        HIAI_ENGINE_LOG("Fail to send data to the graph-%u", g_graph_id);
        break;
    }
}

// After the processing is complete, delete the entire graph.
hiai::Graph::DestroyGraph(g_graph_id);
return 0;
}
```

8.2.2.5.3 Usage Example for C++ APIs (Creating a Graph Based on the Configuration File and Writing the Generated Graph to the List)

```
/**
 * @file graph_cplusplus_api_example.cpp
 *
 * Copyright(C), 2017 - 2017, Huawei Tech. Co., Ltd. ALL RIGHTS RESERVED.
 *
 * @Source Files for HIAI Graph Orchestration
 *
 * @version 1.0
 */
#include <unistd.h>
#include <thread>
#include <fstream>
#include <algorithm>
#include "hiaiengine/api.h"
#include "user_def_data_type.h"
/*****
 * Normally the mandatory steps to setup the whole HIAI graph include:
 * 1) Call the HIAI_Init() API to perform global system initialization;
 * 2) Call HIAI_CreateGraph or Graph::CreateGraph to create and start a graph.
 * 3) Set profile config if necessary.
 * 4) Call HIAI_DestroyGraph or Graph::DestroyGraph to destroy the graph finally.
 */
*****
// Just define variables which are used in only this example
// and are not required in real user case.
#define MATRIX_USER_SPECIFY_ENGINE_PRIORITY (10)
static uint32_t src_engine_id = 1000; // Defined in graph.prototxt
static uint32_t dest_engine_id = 1002; // Defined in graph.prototxt
static std::string graph_config_proto_file = "./config/graph.prototxt";
// Define End
namespace hiai {
```

```
class GraphDataRecvInterface: public DataRecvInterface
{
public:
    GraphDataRecvInterface()
    {
    }
    /**
     * @ingroup hiaiengine
     * @brief Read and save the data.
     * @param [in] in_data Input data
     * @return HIAI Status
     */
    HIAI_StatusT RecvData(const std::shared_ptr<void>& message)
    {
        // Convert the data to a specific message type and process the related message. For example:
        // shared_ptr<std::string> data =
        // std::static_pointer_cast<std::string>(message);
        return HIAI_OK;
    }
private:
};
}
/*****
 * Matrix Example
 * Graph:
 * SrcEngine<-->Engine<-->DestEngine
 *
 *****/
HIAI_StatusT HIAI_InitAndStartGraph(std::list<std::shared_ptr<hiai::Graph>>& graphList)
{
    // Step1: Global System Initialization before using Matrix
    HIAI_StatusT status = HIAI_Init(0);
    // Step2: Create and Start the Graph
    status = hiai::Graph::CreateGraph(graph_config_proto_file, graphList);
    if (status != HIAI_OK)
    {
        HIAI_ENGINE_LOG(status, "Fail to start graph");
        return status;
    }
    // Step3: Set the callback function to receive data.
    // If multiple graphs are configured in a configuration file, you can traverse the graph list to obtain the
    corresponding graph and perform the expected operation.
    std::list<std::shared_ptr<hiai::Graph>>::iterator iter = graphList.begin();
    std::shared_ptr<hiai::Graph> graph = hiai::Graph::GetInstance((*iter)->GetGraphId());
    if (nullptr == graph)
    {
        HIAI_ENGINE_LOG("Fail to get the graph");
        return status;
    }
    // Specify the port id (default to zero)
    hiai::EnginePortID target_port_config;
    target_port_config.graph_id = (*iter)->GetGraphId();
    target_port_config.engine_id = dest_engine_id;
    target_port_config.port_id = 0;
    graph->SetDataRecvFuncion(target_port_config,
        std::shared_ptr<hiai::GraphDataRecvInterface>(
            new hiai::GraphDataRecvInterface()));
    return HIAI_OK;
}
// User Application Main Example
int main()
{
    HIAI_StatusT ret = HIAI_OK;
    // Store the graphs created by the user.
    std::list<std::shared_ptr<hiai::Graph>> graphList;
    // Creation process
    ret = HIAI_InitAndStartGraph(graphList);
    if (HIAI_OK != ret)
    {
    }
```



```
HIAI_ENGINE_LOG("Fail to start graph");  
return -1;  
}  
// Read data and inject data into the process.  
std::list<std::shared_ptr<hiaai::Graph>>::iterator iter = graphList.begin();  
std::shared_ptr<hiaai::Graph> graph = hiaai::Graph::GetInstance((*iter)->GetGraphId());  
if (nullptr == graph)  
{  
    HIAI_ENGINE_LOG("Fail to get the graph");  
    return -1;  
}  
hiaai::EnginePortID engine_id;  
engine_id.graph_id = (*iter)->GetGraphId();  
engine_id.engine_id = src_engine_id;  
engine_id.port_id = 0;  
bool is_end_of_data = false;  
while (!is_end_of_data)  
{  
    std::shared_ptr<UserDefDataType> user_def_msg(new UserDefDataType);  
    // Read data and pad UserDefDataType. For details about the data type registration, see the  
corresponding section in this document.  
    // Inject data to the graph.  
    if (HIAI_OK != graph->SendData(engine_id, "UserDefDataType",  
        std::static_pointer_cast<void>(user_def_msg)))  
    {  
        // When the queue is full, a message is returned indicating that sending data fails. The service logic  
determines whether to resend or discard the data.  
        HIAI_ENGINE_LOG("Fail to send data to the graph-%u", (*iter)->GetGraphId());  
        break;  
    }  
}  
// After the processing is complete, delete all graphs.  
std::list<std::shared_ptr<hiaai::Graph>>::iterator graphIter;  
for (graphIter = graphList.begin(); graphIter != graphList.end(); ++graphIter) {  
    hiaai::Graph::DestroyGraph((*graphIter)->GetGraphId());  
}  
return 0;  
}
```

8.2.3 Engine Implementation APIs (C++ Language)

8.2.3.1 Constructor and Destructor Functions of an Engine

This API is defined in **engine.h**.

Syntax

Engine()

virtual ~Engine()

NOTE

The constructor and destructor functions are optional. You can determine whether to overload and implement them based on the actual situation.

8.2.3.2 Engine::Init

Initializes the configuration of an engine instance. This API is defined in **engine.h**.

 NOTE

The **Init** API is optional. You can determine whether to overload and implement it based on the actual situation.

Syntax

**HIAI_StatusT Engine::Init(const AIConfig &config, const
vector<AIModelDescription> &modelDesc)**

Parameter Description

Parameter	Description	Value Range
config	Engine configuration item	-
modelDesc	Model description	-

Return Value

The returned error codes are registered by the user in advance. For details about the registration method, see [8.2.5.5.1 Error Code Registration](#).

Error Codes

The error codes of this API are registered by the user.

Example of Overload and Implementation

Initialize the inference engine. During the initialization, load the model through the API of the model manager (AIModelManager), as shown in [8.2.4.1.1 AIModelManager::Init](#).

```
HIAI_StatusT FrameworkerEngine::Init(const hiai::AIConfig& config,
const std::vector<hiai::AIModelDescription>& model_desc)
{
    hiai::AIStatus ret = hiai::SUCCESS;
    // init ai_model_manager_
    if (nullptr == ai_model_manager_)
    {
        ai_model_manager_ = std::make_shared<hiai::AIModelManager>();
    }
    std::cout<<"FrameworkerEngine Init"<<std::endl;
    HIAI_ENGINE_LOG("FrameworkerEngine Init");

    for (int index = 0; index < config.items_size(); ++index)
    {
        const ::hiai::AIConfigItem& item = config.items(index);
        // loading model
        if(item.name() == "model_path")
        {
            const char* model_path = item.value().data();
            std::vector<hiai::AIModelDescription> model_desc_vec;
            hiai::AIModelDescription model_desc_;
            model_desc_.set_path(model_path);
            model_desc_vec.push_back(model_desc_);
        }
    }
}
```

```
ret = ai_model_manager_>Init(config, model_desc_vec);

if (hiai::SUCCESS != ret)
{
    HIAI_ENGINE_LOG(this, HIAI_AI_MODEL_MANAGER_INIT_FAIL, "[DEBUG] fail to init ai_model");
    return HIAI_AI_MODEL_MANAGER_INIT_FAIL;
}
}
}
HIAI_ENGINE_LOG("FrameworkerEngine Init success");
return HIAI_OK;
}
```

8.2.3.3 Macro: HIAI_DEFINE_PROCESS

You can directly call the number of input and output ports of the engine in the macro definition. This macro is defined in **engine.h**.

This macro encapsulates the following function:

```
HIAI_StatusT Engine::InitQueue(const uint32_t& in_port_num, const uint32_t& out_port_num);
```

Related macro:

This macro is called before **HIAI_IMPL_ENGINE_PROCESS(name, engineClass, inPortNum)**.

Syntax

HIAI_DEFINE_PROCESS (inputPortNum, outputPortNum)

Parameter Description

Parameter	Description	Value Range
inputPortNum	Number of input ports of an engine	-
outputPortNum	Number of output ports of an engine	-

Calling Example

```
#define FRAMEWORK_ENGINE_INPUT_SIZE 1
#define FRAMEWORK_ENGINE_OUTPUT_SIZE 1

* @[in]: Defines an input port and an output port. */
HIAI_DEFINE_PROCESS(FRAMEWORK_ENGINE_INPUT_SIZE, FRAMEWORK_ENGINE_OUTPUT_SIZE)
```

8.2.3.4 Macro: HIAI_IMPL_ENGINE_PROCESS

You need to overload and implement this macro to define the implementation of the engine. This macro is defined in **engine.h**.

This macro encapsulates the following functions:

```
static HIAIEngineFactory* GetInstance();
HIAI_StatusT HIAIEngineFactory::RegisterEngineCreator(const std::string&
engine_name, HIAI_ENGINE_FUNCTOR_CREATOR engineCreatorFunc);
HIAI_StatusT HIAIEngineFactory::UnRegisterEngineCreator(const std::string& engine_name);
```

Related macro:

This macro is called after **HIAI_DEFINE_PROCESS** (inputPortNum, outputPortNum).

Syntax

HIAI_IMPL_ENGINE_PROCESS(name, engineClass, inPortNum)

Parameter Description

Parameter	Description	Value Range
name	Engine name in the configuration	-
engineClass	Name of the engine implementation class	-
inPortNum	Number of input ports	-

Return Value

The returned error codes are registered by the user in advance.

Error Codes

The error codes of this API are registered by the user.

Example of Overload and Implementation

Define the implementation of the inference engine. During implementation, the model inference is executed through the API of the model manager (AIModelManager), as shown in [8.2.4.1.3 AIModelManager::Process](#).

```
HIAI_IMPL_ENGINE_PROCESS("FrameworkerEngine", FrameworkerEngine,
FRAMEWORK_ENGINE_INPUT_SIZE)
{
    hiai::AIStatus ret = hiai::SUCCESS;
    HIAI_StatusT hiai_ret = HIAI_OK;
    // receive data
    // arg0 indicates the input port (numbered 0) of the engine. If there are multiple input ports, you can use
    // arg1 (numbered 1) and arg2 (numbered 2) to match the input ports. The data sent by the previous engine
    // is obtained through the input port.
    std::shared_ptr<std::string> input_arg =
        std::static_pointer_cast<std::string>(arg0);
    if (nullptr == input_arg)
    {
        HIAI_ENGINE_LOG(this, HIAI_INVALID_INPUT_MSG, "[DEBUG] input arg is invalid");
        return HIAI_INVALID_INPUT_MSG;
    }
    std::cout<<"FrameworkerEngine Process"<<std::endl;

    // prepare for calling the process of ai_model_manager_
    std::vector<std::shared_ptr<hiai::IAITensor>> input_data_vec;

    uint32_t len = 75264;
```

```
HIAI_ENGINE_LOG("FrameworkerEngine:Go to Process");
std::cout << "HIAIAippOp:Go to process" << std::endl;
std::shared_ptr<hiaai::AINeuralNetworkBuffer> neural_buffer =
std::shared_ptr<hiaai::AINeuralNetworkBuffer>(new hiaai::AINeuralNetworkBuffer());//
std::static_pointer_cast<hiaai::AINeuralNetworkBuffer>(input_data);
neural_buffer->SetBuffer((void*)(input_arg->c_str()), (uint32_t)(len));
std::shared_ptr<hiaai::IAITensor> input_data = std::static_pointer_cast<hiaai::IAITensor>(neural_buffer);
input_data_vec.push_back(input_data);

// call Process and inference
hiaai::AIContext ai_context;
std::vector<std::shared_ptr<hiaai::IAITensor>> output_data_vec;
ret = ai_model_manager->CreateOutputTensor(input_data_vec, output_data_vec);
if (hiaai::SUCCESS != ret)
{
    HIAI_ENGINE_LOG(this, HIAI_AI_MODEL_MANAGER_PROCESS_FAIL, "[DEBUG] fail to process
ai_model");
    return HIAI_AI_MODEL_MANAGER_PROCESS_FAIL;
}

ret = ai_model_manager->Process(ai_context, input_data_vec, output_data_vec, 0);

if (hiaai::SUCCESS != ret)
{
    HIAI_ENGINE_LOG(this, HIAI_AI_MODEL_MANAGER_PROCESS_FAIL, "[DEBUG] fail to process
ai_model");
    return HIAI_AI_MODEL_MANAGER_PROCESS_FAIL;
}
std::cout<<"[DEBUG] output_data_vec size is "<< output_data_vec.size()<<std::endl;
for (uint32_t index = 0; index < output_data_vec.size(); index++)
{
    // send data of inference to destEngine
    std::shared_ptr<hiaai::AINeuralNetworkBuffer> output_data =
std::static_pointer_cast<hiaai::AINeuralNetworkBuffer>(output_data_vec[index]);
    std::shared_ptr<std::string> output_string_ptr = std::shared_ptr<std::string>(new
std::string((char*)output_data->GetBuffer(), output_data->GetSize()));

    hiai_ret = SendData(0, "string", std::static_pointer_cast<void>(output_string_ptr));
    if (HIAI_OK != hiai_ret)
    {
        HIAI_ENGINE_LOG(this, HIAI_SEND_DATA_FAIL, "fail to send data");
        return HIAI_SEND_DATA_FAIL;
    }
}
return HIAI_OK;
}
```

8.2.3.5 Engine::SetDataRecvFunctor

Sets the callback function for an engine to receive messages. This API is defined in **engine.h**.

It must be used together with the [DataRecvInterface::RecvData](#) API. The detailed principles are as follows:

1. Define a subclass (for example, **DdkDataRecvInterface**) of class **DataRecvInterface** and initialize the object of the subclass.
2. Set the object of class **DdkDataRecvInterface** to a callback function by calling **Engine::SetDataRecvFunctor**.
3. Send the input data to the corresponding output port by calling **Engine::SendData**.
4. Return data by calling the **RecvData** function of class **DdkDataRecvInterface**.

Syntax

```
HIAI_StatusT Engine::SetDataRecvFuncor(const uint32_t portId, const  
shared_ptr<DataRecvInterface>& userDefineDataRecv)
```

Parameter Description

Parameter	Description	Value Range
portId	Port ID	-
userDefineDataRecv	Callback function for receiving user-defined data	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_PORT_ID_ERROR	The port ID has an error.

8.2.3.6 Engine::SendData

Sends data from the engine to the specified port_id. This API is defined in **engine.h**.

NOTE

The **SendData** interface uses the DMA transfer mode, which may affect the CPU's timely response to and processing of interrupt requests, for example, memory allocation by calling the **new** or **malloc** function.

Syntax

```
HIAI_StatusT Engine::SendData(uint32_t portId, const std::string&  
messageName, const shared_ptr<void>& dataPtr, uint32_t timeOut =  
TIME_OUT_VALUE)
```

Parameter Description

Parameter	Description	Value Range
portId	ID of the output port of the engine	-

Parameter	Description	Value Range
messageName	Name of the message that is being sent. The message must be registered in advance by calling the macro provided by HiAI.	-
dataPtr	Pointer that points to a specific message	-
timeOut	Timeout for calling this API to send data. The default timeout is 500 ms. If data sending times out, the system background attempts to send data again. A maximum of 16 attempts are allowed.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_ENGINE_NULL_POINTER	The pointer is null.
3	HIAI_GRAPH_ENGINE_NOT_EXIST	The engine does not exist.
4	HIAI_GRAPH_SRC_PORT_NOT_EXIST	The source port does not exist.

8.2.3.7 Calling Example

```
/**
 * @file multi_input_output_engine_example.h
 *
 * Copyright(c)<2018>, <Huawei Technologies Co.,Ltd>
 *
 * @version 1.0
 *
 * @date 2018-4-25
 */
```

```
#ifndef MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_H_
#define MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_H_

#include "hiaengine/engine.h"
#include "hiaengine/data_type.h"
#include "hiaengine/multitype_queue.h"

#define MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_INPUT_SIZE 3
#define MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_OUTPUT_SIZE 2

namespace hiai {
// Define New Engine
class HIAIMultiEngineExample : public Engine {
public:
    HIAIMultiEngineExample() :
        input_que_(MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_INPUT_SIZE) {}

    // Overload the Init of the parent engine.
    HIAI_StatusT Init(const AIConfig& config,
        const std::vector<AIModelDescription>& model_desc)
    {
        return HIAI_OK;
    }

    /**
     * @ingroup hiaiengine
     * @brief HIAI_DEFINE_PROCESS
     * @[in]: Defines an input port and an output port.
     */
    HIAI_DEFINE_PROCESS(MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_INPUT_SIZE,
MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_OUTPUT_SIZE)
private:
    // Implement a member variable in a private manner to cache the input queue.
    hiai::MultiTypeQueue input_que_;
};

}

#endif //MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_H_

/**
 * @file multi_input_output_engine_example.h
 *
 * Copyright(c)<2018>, <Huawei Technologies Co.,Ltd>
 *
 * @version 1.0
 *
 * @date 2018-4-25
 */

#include "multi_input_output_engine_example.h"
#include "use_def_data_type.h"
#include "use_def_errorcode.h"

namespace hiai {

/**
 * @ingroup hiaiengine
 * @brief HIAI_DEFINE_PROCESS: implements the input and output processing flow of multiple ports.
 * @[in]: Defines an input port and an output port,
 * The engine is registered and named "HIAIMultiEngineExample".
 */
HIAI_IMPL_ENGINE_PROCESS("HIAIMultiEngineExample", HIAIMultiEngineExample,
MULTI_INPUT_OUTPUT_ENGINE_EXAMPLE_INPUT_SIZE)
{
    // This Engine has three input args and two output

    // Each port may receive data at different time points. Therefore, some input ports may be empty.
}
```



```
input_que_.PushData(0,arg0);
input_que_.PushData(1,arg1);
input_que_.PushData(2,arg2);

std::shared_ptr<void> input_arg1;
std::shared_ptr<void> input_arg2;
std::shared_ptr<void> input_arg3;

// The subsequent processing is performed only when all the three ports have input data. Method 1:
if (!input_que_.PopAllData(input_arg1, input_arg2, input_arg3))
{
    HIAI_ENGINE_LOG(this, HIAI_INVALID_INPUT_MSG, "fail to process");
    return HIAI_INVALID_INPUT_MSG;
}

// The subsequent processing is performed only when all the three ports have input data. Method 2:
/*
if (!(input_que_.FrontData(0, input_arg1) && input_que_.FrontData(1, input_arg2) &&
input_que_.FrontData(2, input_arg3)))
{
    HIAI_ENGINE_LOG(this, HIAI_INVALID_INPUT_MSG, "fail to process");
    return HIAI_INVALID_INPUT_MSG;
} else {
    input_que_.PopData(0, input_arg1);
    input_que_.PopData(1, input_arg2);
    input_que_.PopData(2, input_arg3);
}
*/
//
// Intermediate service logic can directly call the DVPP API or AIModelManger API for processing.
//

// Allocate memory to save results.
std::shared_ptr<UseDefDataTypeT> output1 =
    std::make_shared<UseDefDataTypeT>();

std::shared_ptr<UseDefTemplateDataType<uint64_t, uint64_t, uint64_t>> output2 =
    std::make_shared<UseDefTemplateDataType<uint64_t, uint64_t, uint64_t>>();

// Fill in the output data structure, for example, assign a value.
*output1 = *input_arg2;
*output2 = *input_arg3;

// Send data to output port 0.
hiai::Engine::SendData(0, "UseDefDataTypeT", std::static_pointer_cast<void>(output1));

// Send data to output port 1.
hiai::Engine::SendData(1, "UseDefTemplateDataType_uint64_t_uint64_t_uint64_t",
    std::static_pointer_cast<void>(output2));

return HIAI_OK;
}
}
```

8.2.4 Model Manager APIs (C++ Language)

8.2.4.1 Offline Model Manager

The APIs marked by the following macros are not implemented in the current version and cannot be directly called: `__ANDROID__`, `ANDROID`, and `__LITE__`.

8.2.4.1.1 AIModelManager::Init

Initializes the model manager to load a model.

Syntax

```
virtual AIStatus AIModelManager::Init(const AIConfig &config, const  
std::vector<AIModelDescription> &model_descs = {}) override;
```

Parameter Description

Parameter	Description	Value Range
config	Configuration information For details about the syntax of the AIConfig data type, see 8.2.4.3.2 AIConfig .	-
model_descs	Model description list. A model can be loaded from the memory (prior) or file. For details about the syntax of the AIModelDescription data type, see 8.2.4.3.6 AIModelDescription .	-

Return Value

SUCCESS indicates that initialization succeeds, while **FAILED** indicates that initialization fails.

Calling Example

Note that the value of **MODEL_NAME** can contain only uppercase letters, lowercase letters, digits, underscores (_), and dots (.). The value of **MODEL_PATH** can contain only uppercase letters, lowercase letters, digits, and underscores (_).

- Load a model from a file:

```
AIModelManager model_mgr;  
AIModelDescription model_desc;  
AIConfig config;  
/* Check whether the input file path is valid.  
   Path: The value can contain uppercase letters, lowercase letters, digits, and underscores (_).  
   File name: The value can contain uppercase letters, lowercase letters, digits, underscores (_), and  
   dots (.). */  
model_desc.set_path(MODEL_PATH);  
model_desc.set_name(MODEL_NAME);  
model_desc.set_type(0);  
vector<AIModelDescription> model_descs;  
model_descs.push_back(model_desc);  
// AIModelManager Init  
AIStatus ret = model_mgr.Init(config, model_descs);  
if (SUCCESS != ret)  
{  
    printf("AIModelManager Init failed. ret = %d\n", ret);
```

```
    return -1;
}
```

- Load a model from memory:

```
AIModelManager model_mgr;
AIModelDescription model_desc;
vector<AIModelDescription> model_descs;
AIConfig config;
model_desc.set_name(MODEL_NAME);
model_desc.set_type(0);
char *model_data = nullptr;
uint32_t model_size = 0;
ASSERT_EQ(true, Utils::ReadFile(MODEL_PATH.c_str(), model_data, model_size));
model_desc.set_data(model_data, model_size);
model_desc.set_size(model_size);
AIStatus ret = model_mgr.Init(config, model_descs);
if (SUCCESS != ret)
{
    printf("AIModelManager Init failed. ret = %d\n", ret);
    return -1;
}
}
```

 NOTE

The value of **model_size** must be the same as the actual size of the model.

8.2.4.1.2 AIModelManager::SetListener

Sets the model manager.

 NOTE

If this API is not called or **listener** is set to **nullptr** when this API is called, the **Process** interface is called synchronously. Otherwise, the **Process** interface is called asynchronously.

Syntax

```
virtual AIStatus AIModelManager::SetListener(std::shared_ptr<IAIListener>
listener) override;
```

Parameter Description

Parameter	Description	Value Range
listener	Callback function For details about the syntax of the IAIListener data type, see 8.2.4.3.19 IAIListener .	-

Return Value

SUCCESS indicates that initialization succeeds, while **FAILED** indicates that initialization fails.

8.2.4.1.3 AIModelManager::Process

Processes input data with a model. When inference is performed on a single model, the buffer queue length of the process is limited to 2048 bytes. If the buffer queue length exceeds 2048 bytes, a failure message is returned.

Syntax

```
virtual AIStatus AIModelManager::Process(AIContext &context, const  
std::vector<std::shared_ptr<IAITensor>> &in_data,  
std::vector<std::shared_ptr<IAITensor>> &out_data, uint32_t timeout);
```

Parameter Description

Parameter	Description	Value Range
context	Context information, including the configurations of variable parameters when the engine is running For details about the syntax of the AIContext data type, see 8.2.4.3.17 AIContext .	-
in_data	List of input tensors for a model For details about the syntax of the IAITensor data type, see 8.2.4.3.20 IAITensor .	-
out_data	List of output tensors for a model For details about the syntax of the IAITensor data type, see 8.2.4.3.20 IAITensor .	-
timeout	Calculation timeout. This parameter is reserved. The default value is 0 , indicating that the configuration is invalid.	-

Return Value

SUCCESS indicates that initialization succeeds, while **FAILED** indicates that initialization fails.

8.2.4.1.4 AIModelManager::CreateOutputTensor

Creates a list of output tensors.

NOTE

If you use this API to create tensors, the memory start address returned by this API must be 512-byte aligned. You are advised to use the `AITensorFactory::CreateTensor` interface to create a tensor. For details, see [8.2.4.1.10 AITensorFactory::CreateTensor](#).

Syntax

```
virtual AIStatus AIModelManager::CreateOutputTensor(const  
std::vector<std::shared_ptr<IAITensor>> &in_data,  
std::vector<std::shared_ptr<IAITensor>> &out_data) override;
```

Parameter Description

Parameter	Description	Value Range
in_data	List of input tensors For details about the syntax of the <code>IAITensor</code> data type, see 8.2.4.3.20 IAITensor .	-
out_data	List of output tensors For details about the syntax of the <code>IAITensor</code> data type, see 8.2.4.3.20 IAITensor .	-

Return Value

SUCCESS indicates that initialization succeeds, while **FAILED** indicates that initialization fails.

8.2.4.1.5 AIModelManager::CreateInputTensor

Creates a list of input tensors.

NOTE

If you use this API to create tensors, the memory start address returned by this API must be 512-byte aligned. You are advised to use the `AITensorFactory::CreateTensor` interface to create a tensor. For details, see [8.2.4.1.10 AITensorFactory::CreateTensor](#).

Syntax

```
virtual AIStatus  
AIModelManager::CreateInputTensor(std::vector<std::shared_ptr<IAITensor>>  
&in_data);
```

Parameter Description

Parameter	Description	Value Range
in_data	List of input tensors For details about the syntax of the IAITensor data type, see 8.2.4.3.20 IAITensor .	-

Return Value

SUCCESS indicates that initialization succeeds, while **FAILED** indicates that initialization fails.

8.2.4.1.6 AIModelManager::IsPreAllocateOutputMem

Determines whether the output memory can be pre-allocated.

Related function:

```
AIStatus AIModelManager::CreateOutputTensor(const  
std::vector<std::shared_ptr<IAITensor>> &in_data,  
std::vector<std::shared_ptr<IAITensor>> &out_data);
```

Syntax

```
virtual bool AIModelManager::IsPreAllocateOutputMem() override ;
```

Parameter Description

None

Return Value

- **True:** You can call CreateOutputTensor to apply for output memory.
- **False:** You cannot call CreateOutputTensor.

NOTE

True is returned only when the model manager loads a model.

8.2.4.1.7 AIModelManager::GetModelIOTensorDim

Obtains the input and output dimensions of the loaded model.

Syntax

```
virtual AIStatus AIModelManager::GetModelIOTensorDim(const std::string&  
model_name, std::vector<TensorDimension>& input_tensor,  
std::vector<TensorDimension>& output_tensor) ;
```

Parameter Description

Parameter	Description	Value Range
model_name	Model name	-
input_tensor	List of input dimensions for a model For details about the syntax of the TensorDimension data type, see 8.2.4.3.18 TensorDimension .	-
output_tensor	List of output dimensions for a model For details about the syntax of the TensorDimension data type, see 8.2.4.3.18 TensorDimension .	-

Return Value

SUCCESS indicates that initialization succeeds, while **FAILED** indicates that initialization fails.

Example

The input and output tensors of the ResNet-50 model are described as follows.

If **size** of **output_tensor** is not **0**, the obtained output_tensor data is appended to the original output_tensor data.

```
input_tensor
{
  name = "data"           # Name of the input layer
  data_type = 0           # Reserved data type, which is not used currently
  size = 20               # Memory size, in bytes
  format = 0             # Reserved tensor format, which is not used currently
  dims = {1,3,224,224}
}
output_tensor
{
  name = "output_0_prob_0" # Name of the output tensor. The format is as follows: output_{digit}_{name
of the output node}_{index of the output node}
  data_type = 0           # Reserved data type, which is not used currently
  size = 20               # Memory size, in bytes
  format = 0             # Reserved tensor format, which is not used currently
  dims = {1,1000,1,1}
}
```

8.2.4.1.8 AIModelManager::GetMaxUsedMemory

Queries the size of the memory used by the model based on the model name.

Syntax

```
int32_t AIModelManager::GetMaxUsedMemory(std::string model_name);
```

Parameter Description

Parameter	Description	Value Range
model_name	Model name	-

Return Value

Size of the memory used by the model

8.2.4.1.9 AISimpleTensor::SetBuffer

Sets the tensor data address.

Syntax

```
void SetBuffer(void *data, const int32_t size, bool isown=false);
```

Parameter Description

Parameter	Description	Value Range
data	Data address	-
size	Data length NOTE The unit is byte. The value of size must be the same as the actual data size.	-
isown	Whether the tensor frees the memory of the data address after the tensor life cycle expires. • false (default): You can perform the free operation after the tensor life cycle expires. • true: After the tensor life cycle expires, the tensor frees the memory. Do not perform the free operation manually in this case. Otherwise, the memory is repeatedly freed.	-

Return Value

None

8.2.4.1.10 AITensorFactory::CreateTensor

Creates a model tensor. This API is defined in **ai_tensor.h**.

Syntax

```
std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription  
&tensor_desc, void *buffer, int32_t size);
```

```
std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription  
&tensor_desc);
```

```
std::shared_ptr<IAITensor> CreateTensor(const std::string &type);
```

Parameter Description

Parameter	Description	Value Range
tensor_desc	Tensor description For details about the syntax of the AITensorDescription data type, see 8.2.4.3.4 AITensorDescription .	-
buffer	Data address NOTE - You are advised to allocate the memory for the input data and output data through the HIAI_DMalloc interface. This enables a zero-copy mechanism for algorithm inference to optimize the processing time. - The memory address is applied for and released by the customer.	-
size	Data length NOTE The unit is byte. The value of size must be the same as the actual data size.	-
type	Type of registering a tensor Note: This API should be used only when no memory is pre-allocated. For details, see the description of type in 8.2.4.3.4 AITensorDescription .	-

Return Value

If a model tensor is created successfully, the tensor pointer is returned. If a model tensor fails to be created, a null pointer is returned.

8.2.4.1.11 Calling Example

Example 1: Synchronous Calling

The current example is the implementation code of the inference engine in the case of a single model.

In the case of multiple models, pay attention to the following points if you need to refer to this example:

- The **preOutBuffer** variable cannot contain the **static** keyword. You need to define the variable as follows:

```
bool preOutBuffer = false;
```
- Before calling the **Process** function, call the **AddPara** function to set the name of each model.

```
ai_context.AddPara("model_name", modelName);// Multiple models. You must set the separate model name for each model.  
ret = ai_model_manager->Process(ai_context,  
    inDataVec, outDataVec_, 0);
```

The sample code is as follows:

```
// Process function implementation of the inference engine  
HIAI_IMPL_ENGINE_PROCESS("ClassifyNetEngine", ClassifyNetEngine, CLASSIFYNET_ENGINE_INPUT_SIZE)  
{  
    HIAI_ENGINE_LOG(this, HIAI_OK, "ClassifyNetEngine Process");  
    HIAI_StatusT ret = HIAI_OK;  
    static bool preOutBuffer = false;  
    std::vector<std::shared_ptr<hiiai::IAITensor>> inDataVec;  
  
    // Obtain the data input by the previous engine.  
    std::shared_ptr<EngineTransNewT> input_arg =  
        std::static_pointer_cast<EngineTransNewT>(arg0);  
    // If the input data is a null pointer, an error code is returned  
    if (nullptr == input_arg)  
    {  
        HIAI_ENGINE_LOG(this, HIAI_INVALID_INPUT_MSG,  
            "fail to process invalid message");  
        return HIAI_INVALID_INPUT_MSG;  
    }  
  
    // Prepare output data. If the synchronization mechanism is used, use the HIAI_DMAlloc interface to  
    // allocate memory for the output data, and use the CreateTensor interface to provide memory for algorithm  
    // inference.  
    //If the Process function for inference is called, the output memory needs to be allocated for only once.  
    if (preOutBuffer == false) {  
        std::vector<hiiai::TensorDimension> inputTensorVec;  
        std::vector<hiiai::TensorDimension> outputTensorVec;  
        ret = ai_model_manager->GetModelIOTensorDim(modelName, inputTensorVec, outputTensorVec);  
        if (ret != hiiai::SUCCESS)  
        {  
            HIAI_ENGINE_LOG(this, HIAI_AI_MODEL_MANAGER_INIT_FAIL,  
                "hiiai ai model manager init fail");  
            return HIAI_AI_MODEL_MANAGER_INIT_FAIL;  
        }  
        // allocate OutData in advance  
        HIAI_StatusT hiai_ret = HIAI_OK;  
        for (uint32_t index = 0; index < outputTensorVec.size(); index++) {  
            hiiai::AITensorDescription outputTensorDesc = hiiai::AINeuralNetworkBuffer::GetDescription();  
            uint8_t* buffer = nullptr;  
            //Allocate the memory by calling the HIAI_Dmalloc interface. The memory is used for algorithm  
            //inference and can be released by calling the HIAI_DFree interface.  
            //Release the memory during engine destruction.  
            hiai_ret = hiiai::HIAIMemory::HIAI_DMAlloc(outputTensorVec[index].size, (void*)&buffer, 1000);  
            if (hiai_ret != HIAI_OK || buffer == nullptr) {  
                std::cout<<"HIAI_DMAlloc failed"<< std::endl;
```

```
        continue;
    }
    outData_.push_back(buffer);
    shared_ptr<hiAi::AITensor> outputTensor =
        hiAi::AITensorFactory::GetInstance()->CreateTensor(outputTensorDesc, buffer,
outputTensorVec[index].size);
    outDataVec_.push_back(outputTensor);
    }
    preOutBuffer = true;
}

// Transfer buffer to Framework directly, only one inputsz
hiAi::AITensorDescription inputTensorDesc =
    hiAi::AINeuralNetworkBuffer::GetDescription();
shared_ptr<hiAi::AITensor> inputTensor =
    hiAi::AITensorFactory::GetInstance()->CreateTensor(inputTensorDesc,
    input_arg->trans_buff.get(), input_arg->buffer_size);
// AIModelManager: fill in the input data.
inDataVec_.push_back(inputTensor);

hiAi::AIContext ai_context;
// Process work
ret = ai_model_manager_->Process(ai_context,
    inDataVec, outDataVec_, 0);
if (hiAi::SUCCESS != ret)
{
    HIAI_ENGINE_LOG(this, HIAI_AI_MODEL_MANAGER_PROCESS_FAIL,
        "Fail to process ai model manager");
    return HIAI_AI_MODEL_MANAGER_PROCESS_FAIL;
}

// Convert the generated data to the buffer of the string type and send the data.
for (uint32_t index = 0; index < outDataVec_.size(); index++)
{
    HIAI_ENGINE_LOG(this, HIAI_OK, "ClassifyNetEngine SendData");
    std::shared_ptr<hiAi::AINeuralNetworkBuffer> output_data =
std::static_pointer_cast<hiAi::AINeuralNetworkBuffer>(outDataVec_[index]);
    std::shared_ptr<std::string> output_string_ptr =
        std::shared_ptr<std::string>(new std::string((char*)output_data->GetBuffer(), output_data-
>GetSize()));
    hiAi::Engine::SendData(0, "string",
        std::static_pointer_cast<void>(output_string_ptr));
    }
    inDataVec_.clear();
    return HIAI_OK;
}

ClassifyNetEngine::~ClassifyNetEngine() {
    //Release the pre-allocated memory of outData.
    HIAI_StatusT ret = HIAI_OK;
    for (auto buffer : outData_) {
        if (buffer != nullptr) {
            ret = hiAi::HIAIMemory::HIAI_DFree(buffer);
            buffer = nullptr;
        }
    }
}
```

Example 2: Asynchronous Calling

For details, see `ddk/sample/customop/customop_app/main.cpp` under the DDK installation directory in the DDK sample.

8.2.4.2 AIPP Configuration APIs

8.2.4.2.1 Overview

Artificial intelligence pre-processing (AIPP) involves image resizing (including cropping and padding), color space conversion (CSC), and subtracting the mean value and multiplying a factor (pixel changing). All these functions are implemented by the AI Core.

For dynamic AIPP, the AIPP mode is only set to dynamic during model conversion, when different AIPP parameters can be used. Before model inference, however, the dynamic AIPP parameter values must be set in the code of the inference engine. To set the dynamic AIPP parameter values in the code of the inference engine, the AIPP configuration APIs provided in this section must be called. These APIs are defined in **ai_tensor.h**.

8.2.4.2.2 SetDynamicInputIndex

Syntax

```
void SetDynamicInputIndex(uint32_t dynamicInputIndex = 0);
```

Function Description

Specifies the sequence number of the original input of a model to implement AIPP.

Parameter Description

Parameter	Input / Output	Type	Description
dynamicInputIndex	Input	uint32_t	Subscript of the original input of a model, starting from 0. The default value is 0 . For example, if the model has two inputs. To have AIPP start from the second input, set dynamicInputIndex to 1 .

Return Value

None

Exception Handling

None

Restriction

None

8.2.4.2.3 SetDynamicInputEdgeIndex

Syntax

```
void SetDynamicInputEdgeIndex(uint32_t dynamicInputEdgeIndex = 0);
```

Function Description

If a model input is shared by multiple operators, that is, the **Data** operator is followed by more operators, set this parameter to perform AIPP differently for different output edges of the **Data** operator.

Parameter Description

Parameter	Input / Output	Type	Description
dynamicInputEdgeIndex	Input	uint32_t	Subscript of the output edge corresponding to the input of a model, starting from 0. The default value is 0 .

Return Value

None

Exception Handling

None

Restriction

None

8.2.4.2.4 SetInputFormat

Syntax

```
AIStatus SetInputFormat(AippInputFormat inputFormat);
```

Function Description

Sets the original input type of a model.

Parameter Description

Parameter	Input/Output	Type	Description
inputFormat	Input	AippInputFormat	Original input type of a model. enum AippInputFormat { YUV420SP_U8 = 1, XRGB8888_U8, RGB888_U8, YUV400_U8, RESERVED };

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.5 SetCscParams (Setting the Default Parameter Value)

Syntax

```
AIStatus SetCscParams(AippInputFormat srcFormat,  
AippModelFormat dstFormat,  
ImageFormat imageFormat = BT_601NARROW);
```

Function Description

You can call this API to implement the CSC function. The default values of CSC parameters in AIPP are automatically generated based on the original input type,

target input type, and image type specified in the input parameters. In addition, the RB/UV channel switching function is enabled or disabled based on the original input type and target input type.

Parameter Description

Parameter	Input/Output	Type	Description
srcFormat	Input	AippInputFormat	Original input type of a model. enum AippInputFormat { YUV420SP_U8 = 1, XRGB8888_U8, RGB888_U8, YUV400_U8, RESERVED };
dstFormat	Input	AippModelFormat	Target input type after CSC in AIPP. enum AippModelFormat { MODEL_RGB888_U8 = 1, MODEL_BGR888_U8, MODEL_GRAY, MODEL_YUV444SP_U8, MODEL_YVU444SP_U8 };
imageFormat	Input	ImageFormat	Image type. Currently, only JPEG and BT_601NARROW are supported. enum ImageFormat { BITMAP, PNG, JPEG, BT_601NARROW };

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

This API is used to quickly set CSC parameters. The system provides a group of default CSC parameters. For details, see "AIPP Configuration" in *Model Conversion Guide*.

The supported input formats of images before and after CSC in AIPP are as follows:

AippInputFormat::YUV420SP_U8 to AippModelFormat::MODEL_YVU444SP_U8

AippInputFormat::YUV420SP_U8 to AippModelFormat::MODEL_RGB888_U8

AippInputFormat::YUV420SP_U8 to AippModelFormat::MODEL_BGR888_U8

AippInputFormat::YUV420SP_U8 to AippModelFormat::MODEL_GRAY

AippInputFormat::XRGB8888_U8 to AippModelFormat::MODEL_YUV444SP_U8

AippInputFormat::XRGB8888_U8 to AippModelFormat::MODEL_YVU444SP_U8

AippInputFormat::XRGB8888_U8 to AippModelFormat::MODEL_GRAY

AippInputFormat::RGB888_U8 to AippModelFormat::MODEL_BGR888_U8

AippInputFormat::RGB888_U8 to AippModelFormat::MODEL_YUV444SP_U8

AippInputFormat::RGB888_U8 to AippModelFormat::MODEL_YVU444SP_U8

AippInputFormat::RGB888_U8 to AippModelFormat::MODEL_GRAY

If the supported image types or image processing formats do not meet the requirements, call another API in [8.2.4.2.5 SetCscParams \(Setting the Default Parameter Value\)](#) to set CSC parameters.

8.2.4.2.6 SetCscParams (Setting a Parameter Value as Required)

Syntax

```
void SetCscParams(bool csc_switch = false,  
int16_t cscMatrixR0C0 = 0,
```



```
int16_t cscMatrixR0C1 = 0,  
int16_t cscMatrixR0C2 = 0,  
int16_t cscMatrixR1C0 = 0,  
int16_t cscMatrixR1C1 = 0,  
int16_t cscMatrixR1C2 = 0,  
int16_t cscMatrixR2C0 = 0,  
int16_t cscMatrixR2C1 = 0,  
int16_t cscMatrixR2C2 = 0,  
uint8_t cscOutputBiasR0 = 0,  
uint8_t cscOutputBiasR1 = 0,  
uint8_t cscOutputBiasR2 = 0,  
uint8_t cscInputBiasR0 = 0,  
uint8_t cscInputBiasR1 = 0,  
uint8_t cscInputBiasR2 = 0);
```

Function Description

You can call this API to flexibly configure CSC parameters in AIPP as required.

Parameter Description

Parameter	Input/ Output	Type	Description
csc_switch	Input	bool	CSC switch. It is set to false by default.
cscMatrixR0C0	Input	int16_t	CSC matrix parameter.
cscMatrixR0C1	Input	int16_t	CSC matrix parameter.
cscMatrixR0C2	Input	int16_t	CSC matrix parameter.
cscMatrixR1C0	Input	int16_t	CSC matrix parameter.
cscMatrixR1C1	Input	int16_t	CSC matrix parameter.
cscMatrixR1C2	Input	int16_t	CSC matrix parameter.
cscMatrixR2C0	Input	int16_t	CSC matrix parameter.
cscMatrixR2C1	Input	int16_t	CSC matrix parameter.
cscMatrixR2C2	Input	int16_t	CSC matrix parameter.

Parameter	Input/Output	Type	Description
cscOutputBiasR0	Input	uint8_t	Output bias for RGB-to-YUV conversion. The default value is 0 . You can set only some configuration items.
cscOutputBiasR1	Input	uint8_t	Output bias for RGB-to-YUV conversion. The default value is 0 . You can set only some configuration items.
cscOutputBiasR2	Input	uint8_t	Output bias for RGB-to-YUV conversion. The default value is 0 . You can set only some configuration items.
cscInputBiasR0	Input	uint8_t	Input bias for YUV-to-RGB conversion. The default value is 0 . You can set only some configuration items.
cscInputBiasR1	Input	uint8_t	Input bias for YUV-to-RGB conversion. The default value is 0 . You can set only some configuration items.
cscInputBiasR2	Input	uint8_t	Input bias for YUV-to-RGB conversion. The default value is 0 . You can set only some configuration items.

Return Value

None

Exception Handling

None

Restriction

None

8.2.4.2.7 SetRbuvSwapSwitch

Syntax

```
void SetRbuvSwapSwitch(bool rbuvSwapSwitch = false);
```

Function Description

Sets whether to enable R/B swap or U/V channel swap before CSC.

Parameter Description

Parameter	Input/Output	Type	Description
rbuvSwapSwitch	Input	bool	true : Swap is supported. false : Swap is not supported.

Return Value

None

Exception Handling

None

Restriction

None

8.2.4.2.8 SetAxSwapSwitch

Syntax

```
void SetAxSwapSwitch(bool axSwapSwitch = false);
```

Function Description

Sets whether to enable GBA->ARGB swap or YUVA->AYUV swap before CSC.

Parameter Description

Parameter	Input/Output	Type	Description
axSwapSwitch	Input	bool	true : Swap is supported. false : Swap is not supported.

Return Value

None

Exception Handling

None

Restriction

None

8.2.4.2.9 SetSrcImageSize

Syntax

```
void SetSrcImageSize(int32_t srcImageSizeW = 0, int32_t srcImageSizeH = 0);
```

Function Description

Sets the width and height of the source image before AIPP.

Parameter Description

Parameter	Input/ Output	Type	Description
srcImageSizeW	Input	int32_t	Width of the source image.
srcImageSizeH	Input	int32_t	Height of the source image.

Return Value

None

Exception Handling

None

Restriction

None

8.2.4.2.10 SetCropParams

Syntax

```
AIStatus SetCropParams(bool cropSwitch,  
int32_t cropStartPosW, int32_t cropStartPosH,  
int32_t cropSizeW, int32_t cropSizeH,  
uint32_t batch_index = 0);
```

Function Description

Sets the crop parameters. Dynamic AIPP supports the configuration of different crop parameters for each batch. **batchIndex** indicates the sequence number of the

batch for which the crop parameters are set. The value range of **batchIndex** is [0, batchNum). If the value exceeds the range, a failure message will be returned.

Parameter Description

Parameter	Input/Output	Type	Description
cropSwitch	Input	Bool	true : Crop is supported. false : Crop is not supported.
cropStartPosW	Input	int32_t	Horizontal coordinate of the start point in the image during cropping
cropStartPosH	Input	int32_t	Vertical coordinate of the start point in the image during cropping
cropSizeW	Input	int32_t	Crop width
cropSizeH	Input	int32_t	Crop height
batchIndex	Input	uint32_t	Sequence number of the batch for which cropping is performed. The default value is 0 . The value range is [0, batchNum).

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.11 SetPaddingParams

Syntax

```
AIStatus SetPaddingParams(int8_t paddingSwitch,  
int32_t paddingSizeTop, int32_t paddingSizeBottom,
```

```
int32_t paddingSizeLeft, int32_t paddingSizeRight,  
uint32_t batch_index = 0);
```

Function Description

Sets the padding parameters. The **paddingSizeTop**, **paddingSizeBottom**, **paddingSizeLeft**, and **paddingSizeRight** parameters indicate that padding is added to the top, bottom, left, and right of an image, respectively.

Parameter Description

Parameter	Input/Output	Type	Description
paddingSwitch	Input	Bool	true : Padding is supported. false : Padding is not supported.
paddingSizeTop	Input	int32_t	Padding to the top of an image
paddingSizeBottom	Input	int32_t	Padding to the bottom of an image
paddingSizeLeft	Input	int32_t	Padding to the left of an image
paddingSizeRight	Input	int32_t	Padding to the right of an image
batchIndex	Input	uint32_t	Sequence number of the batch for which padding is performed. The default value is 0 . The value range is [0, batchSize).

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.12 SetDtcPixelMean

Syntax

```
AIStatus SetDtcPixelMean(int16_t dtcPixelMeanChn0 = 0,  
int16_t dtcPixelMeanChn1 = 0,  
int16_t dtcPixelMeanChn2 = 0,  
int16_t dtcPixelMeanChn3 = 0,  
uint32_t batch_index = 0);
```

Function Description

Sets the parameters for the mean values of DTC channels.

Parameter Description

Parameter	Input/ Output	Type	Description
dtcPixelMeanChn0	Input	int16_t	Mean value of channel 0
dtcPixelMeanChn1	Input	int16_t	Mean value of channel 1
dtcPixelMeanChn2	Input	int16_t	Mean value of channel 2
dtcPixelMeanChn3	Input	int16_t	Mean value of channel 3
batchIndex	Input	uint32_t	Sequence number of the batch for which the mean value of a DTC channel is set. The default value is 0 . The value range is [0, batchNum).

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.13 SetDtcPixelMin

Syntax

```
AIStatus SetDtcPixelMin(float dtcPixelMinChn0 = 0,  
float dtcPixelMinChn1 = 0,  
float dtcPixelMinChn2 = 0,  
float dtcPixelMinChn3 = 0,  
uint32_t batch_index = 0);
```

Function Description

Sets the parameters for the minimum values of DTC channels.

Parameter Description

Parameter	Input/ Output	Type	Description
dtcPixelMinChn0	Input	float	Minimum value of DTC channel 0
dtcPixelMinChn1	Input	float	Minimum value of DTC channel 1
dtcPixelMinChn2	Input	float	Minimum value of DTC channel 2
dtcPixelMinChn3	Input	float	Minimum value of DTC channel 3
batchIndex	Input	uint32_t	Sequence number of the batch for which the minimum value of a DTC channel is set. The default value is 0. The value range is [0, batchSize).

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.14 SetPixelVarReci

Syntax

```
AIStatus SetPixelVarReci(float dtcPixelVarReciChn0 = 0,  
float dtcPixelVarReciChn1 = 0,  
float dtcPixelVarReciChn2 = 0,  
float dtcPixelVarReciChn3 = 0,  
uint32_t batch_index = 0);
```

Function Description

Sets the parameters for the variance values or (max-min) reciprocal values of DTC channels.

Parameter Description

Parameter	Input / Output	Type	Description
dtcPixelVarReciChn0	Input	float	Variance of DTC channel 0
dtcPixelVarReciChn1	Input	float	Variance of DTC channel 1
dtcPixelVarReciChn2	Input	float	Variance of DTC channel 2
dtcPixelVarReciChn3	Input	float	Variance of DTC channel 3

Parameter	Input / Output	Type	Description
batchIndex	Input	uint32_t	Sequence number of the batch for which the variance of a DTC channel is set. The default value is 0 . The value range is [0, batchNum).

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.15 SetInputDynamicAIPP

Syntax

```
AIStatus SetInputDynamicAIPP(std::vector<std::shared_ptr<IAITensor>>& inData,
std::shared_ptr<AippDynamicParaTensor> aippParms);
```

Function Description

Inserts the generated dynamic AIPP parameter **AippDynamicParaTensor** into the input tensor list. The insertion position is determined by the **dynamicInputIndex** and **dynamicInputEdgeIndex2** attributes in the AIPP tensor. The two attributes are set by calling APIs in [8.2.4.2.2 SetDynamicInputIndex](#) and [8.2.4.2.15 SetInputDynamicAIPP](#).

The AIPP tensor will be inserted after the tensor of the original input.

If multiple AippDynamicParaTensors need to be inserted, this API needs to be called for multiple times. In this case, if the **dynamicInputIndex** value of the newly inserted AippDynamicParaTensor is smaller than that of the existing AippDynamicParaTensor in the tensor list, or if the **dynamicInputIndex** value of

the newly inserted AippDynamicParaTensor is equal to that of the existing AippDynamicParaTensor but the **dynamicInputEdgeIndex** value is smaller, the AIPP tensor will be inserted before the existing AippDynamicParaTensor. If the values of **dynamicInputIndex** and **dynamicInputEdgeIndex** are equal to those of the existing AippDynamicParaTensor in the tensor list, the newly inserted AippDynamicParaTensor overwrites the existing AippDynamicParaTensor.

When this API is called, the validity of **dynamicInputIndex** and **dynamicInputEdgeIndex** is not checked. That is, if the value of **dynamicInputIndex** is greater than the number of outputs of the model or the value of **dynamicInputEdgeIndex** is greater than the number of output edges corresponding to the input, no error is returned.

Parameter Description

Parameter	Input/Output	Type	Description
inData	Input	std::vector<std::shared_ptr<IAITensor>>&	List of input tensors
aippParms	Input	std::shared_ptr<AippDynamicParaTensor>	Dynamic AIPP tensor

Return Value

Parameter	Type	Description
-	AIStatus	If the parameter is set successfully, 0 is returned. If the input is invalid, other values are returned. The AIStatus type is defined as follows: AIStatus = uint32_t

Exception Handling

None

Restriction

None

8.2.4.2.16 GetDynamicInputIndex

Syntax

uint32_t GetDynamicInputIndex() const;

Function Description

Obtains the subscript (specified by **SetDynamicInputIndex**) of the original input of a model.

Parameter Description

None

Return Value

Parameter	Type	Description
-	uint32_t	Subscript of the original input of a model

Exception Handling

None

Restriction

None

8.2.4.2.17 GetDynamicInputEdgeIndex

Syntax

```
uint32_t GetDynamicInputEdgeIndex() const;
```

Function Description

Obtains the subscript (specified by **SetDynamicInputEdgeIndex**) of the output edge corresponding to the input of a model.

Parameter Description

None

Return Value

Parameter	Type	Description
-	uint32_t	Subscript of the output edge corresponding to the input of a model

Exception Handling

None

Restriction

None

8.2.4.2.18 Calling Example

```
/**
 * @file mngr_sample.h
 *
 * Copyright(c)<2018>, <Huawei Technologies Co.,Ltd>
 *
 * @version 1.0
 *
 * @date 2018-4-25
 */
#include <stdio.h>
#include <string>
#include <vector>
#include <iostream>
#include <fstream>
#include <assert.h>
#include "hiaengine/ai_model_manager.h"
using namespace std;
using namespace hiai;
// Image data path
static const std::string IMAGE_FILE_PATH = "/data/input_zebra.bin";
static const char* MODEL_PATH = "/data/ResNet.davincimodel";
static const char* MODEL_NAME = "resnet18";
/**
 *** brief: Read input data.
 */
char* ReadBinFile(const char *file_name, uint32_t *fileSize)
{
    std::filebuf *pbuf;
    std::ifstream filestr;
    size_t size;
    filestr.open(file_name, std::ios::binary);
    if (!filestr)
    {
        return nullptr;
    }
    pbuf = filestr.rdbuf();
    size = pbuf->pubseekoff(0, std::ios::end, std::ios::in);
    pbuf->pubseekpos(0, std::ios::in);
    char * buffer = (char*)malloc(size);
    if (nullptr == buffer)
    {
        return nullptr;
    }
    pbuf->sgetn(buffer, size);
    *fileSize = size;
    filestr.close();
    return buffer;
}

int main(int argc, char* argv[])
{
    vector<shared_ptr<IAITensor>> model_input;
    vector<shared_ptr<IAITensor>> model_output;
    AIModelManager model_mgr;
    AIModelDescription model_desc;
    AIConfig config;
    AIContext context;
    model_desc.set_path(MODEL_PATH);
    model_desc.set_name(MODEL_NAME);
    model_desc.set_type(0);
    vector<AIModelDescription> model_descs;
    model_descs.push_back(model_desc);
    // AIModelManager Init
    AIStatus ret = model_mgr.Init(config, model_descs);
```

```
if (SUCCESS != ret)
{
    printf("AIModelManager Init failed. ret = %d\n", ret);
    return -1;
}
// Input tensor
// The input tensor will be reset after the image data is read. The function here is only for initialization.
AITensorDescription tensor_desc = AINeuralNetworkBuffer::GetDescription();
shared_ptr<IAITensor> input_tensor = AITensorFactory::GetInstance()->CreateTensor(tensor_desc);
if (nullptr == input_tensor)
{
    printf("Create input_tensor failed.\n");
    return -1;
}
// Read image data.
uint32_t image_data_size = 0;
float* image_data = (float*)ReadBinFile(IMAGE_FILE_PATH.c_str(), &image_data_size);
if (nullptr == image_data)
{
    printf("ReadBinFile failed bin file path= %s \n", IMAGE_FILE_PATH.c_str());
    return -1;
}
// Set the pointer and length of the image data address to input_simple_tensor.
shared_ptr<AISimpleTensor> input_simple_tensor = static_pointer_cast<AISimpleTensor>(input_tensor);
input_simple_tensor->SetBuffer((void*)image_data, image_data_size);
model_input.push_back(input_tensor);
// Create an output tensor.
if(model_mgr.IsPreAllocateOutputMem())
{
    ret = model_mgr.CreateOutputTensor(model_input, model_output);
    if (SUCCESS != ret)
    {
        printf("CreateOutputTensor failed.ret = %d\n", ret);
        delete image_data;
        return -1;
    }
}
else
{
    // Create a tensor.
    ret = model_mgr.GetModelIOTensorDim(MODEL_NAME, input_tensor_dims, output_tensor_dims);
    std::vector<TensorDimension> input_tensor_dims;
    std::vector<TensorDimension> output_tensor_dims;
    for(TensorDimension & dims : output_tensor_dims)
    {
        shared_ptr<IAITensor> output_tensor = AITensorFactory::GetInstance()->CreateTensor(tensor_desc);
        shared_ptr<AISimpleTensor> output_simple_tensor =
static_pointer_cast<AISimpleTensor>(output_tensor);
        output_simple_tensor->setBuffer((void*) new char[dims.size], dims.size);
        model_output.push_back(output_tensor)
    }
}

bool dynamic_aipp_flag = true;
if (dynamic_aipp_flag)
{
    const int batch_number = 2;
    AITensorDescription desc = AippDynamicParaTensor::GetDescription(std::to_string(batch_number));
    shared_ptr<IAITensor> tensor = AITensorFactory::GetInstance()->CreateTensor(desc);
    shared_ptr<AippDynamicParaTensor> aipp_params_tensor =
static_pointer_cast<AippDynamicParaTensor>(tensor);
    // Set the original input type of a model.
    aipp_params_tensor->SetInputFormat(hiai::YUV420SP_U8);
    // Set the CSC matrix parameters.
    aipp_params_tensor->SetCscParams(hiai::YUV420SP_U8, haii::MODEL_BGR888_U8, haii::JPEG);
    // Set the width and height of the source image.
    aipp_params_tensor->SetSrcImageSize(256, 224);
    // If there are multiple batches, set the AIPP parameters for each batch.
    for (int i = 0; i < batch_number; i++) {
```

```
// Set the crop parameters.
aipp_params_tensor->SetCropParams(true, 0, 0, 200, 200, i);
// Set the padding parameters.
aipp_params_tensor->SetPaddingParams(true, 12, 12, 12, 12, i);
// Set the parameters for the mean values of DTC channels.
aipp_params_tensor->SetDtcPixelMean(104, 117, 123, 0, i);
// Set the parameters for the variance values or (max-min) reciprocal values of DTC channels.
aipp_params_tensor->SetPixelVarReci(1.0, 1.0, 1.0, 1.0, i);
}

model_mgr.SetInputDynamicAIPP(model_input, aipp_params_tensor);
}

// Start model inference.
printf("Start process.\n");
ret = model_mgr.Process(context, model_input, model_output, 0);
if (SUCCESS != ret)
{
    printf("Process failed.ret = %d\n", ret);
    return -1;
}
// Because the listener is not set, the model inference is synchronously processed. The model inference
output data can be obtained directly.
shared_ptr<AISimpleTensor> result_tensor = static_pointer_cast<AISimpleTensor>(model_output[0]);
printf("Get Result, bufsize is %d", result_tensor->GetSize());
for(TensorDimension & dims : output_tensor_dims)
{
}
printf("predict ok.\n");
return 0;
}
```

8.2.4.3 Data Types

8.2.4.3.1 AIConfigItem

Provides the AIConfig item description. For details, see [ai_type.proto](#).

```
message AIConfigItem
{
    string name = 1; // Configuration item name
    string value = 2; // Configuration item value
    repeated AIConfigItem sub_items = 3; // Configuration subitem
};
```

8.2.4.3.2 AIConfig

Serves as the input parameter when the Init interface of the model manager is called. For details, see [ai_types.proto](#).

```
message AIConfig
{
    repeated AIConfigItem items = 1; // Configuration item list
};
```

8.2.4.3.3 AITensorParaDescription

Provides the tensor parameter description. For details, see [ai_types.proto](#).

```
message AITensorParaDescription
{
    string name = 1; // Parameter name
    string type = 2; // Parameter type
    string value = 3; // Parameter value
    string desc = 4; // Parameter description
}
```

```
repeated AITensorParaDescription sub_paras = 5; // Sub-parameter list  
};
```

8.2.4.3.4 AITensorDescription

Provides the tensor description, mainly about the input and output information of a model. For details, see [ai_types.proto](#).

```
// Tensor description  
message AITensorDescription  
{  
    string name = 1; // Tensor name  
    string type = 2; // Tensor type  
    repeated string compatible_type = 3; // Specifies all types of parent classes that can be compatible.  
    repeated AITensorParaDescription paras = 4; // Parameter list  
};
```

8.2.4.3.5 AITensorDescriptionList

Provides the tensor description list, mainly used for describing the input and output information list of a model. For details, see [ai_types.proto](#).

```
message AITensorDescriptionList  
{  
    repeated AITensorDescription tensor_descs = 1; // Tensor list  
}
```

8.2.4.3.6 AIModelDescription

Serves as the input when the **Init** interface is called. For details, see [ai_types.proto](#).

```
message AIModelDescription  
{  
    string name = 1; // Model name, which can contain uppercase letters, lowercase letters, digits,  
underscores (_), and dots (.)  
    int32 type = 2; // Model type. Currently, only the DAVINCI_OFFLINE_MODEL type is supported. The value  
is 0.  
    // The model parsing capability has been added to the model manager. Therefore, it does not  
matter if this field is not set. To ensure forward compatibility, this field is reserved.  
    string version = 3; // Model version  
    int32 size = 4; // Model size  
    string path = 5; // Model path, which can contain uppercase letters, lowercase letters, digits, and  
underscores (_).  
    repeated string sub_path = 6; // Auxiliary model path, which is used when there are multiple model files,  
for example, Caffe online model  
    // To be forward compatible, the "path" field is not changed to "repeated". Instead,  
the "sub_path" field is added.  
    string key = 7; // Model key  
    repeated string sub_key = 8; // Auxiliary model key, which is used when there are multiple model keys,  
for example, Caffe online model  
    // To be forward compatible, the "key" field is not changed to "repeated". Instead,  
the "sub_key" field is added.  
    enum Frequency  
    {  
        UNSET = 0;  
        LOW = 1;  
        MEDIUM = 2;  
        HIGH = 3;  
    }  
    Frequency frequency = 9;  
    enum DeviceType  
    {  
        NPU = 0;  
        IPU = 1;  
        MLU = 2;  
        CPU = 3;  
    }  
}
```



```
NONE = 255;
}
DeviceType device_type = 10;
enum Framework
{
    OFFLINE    =0;
    CAFFE      =1;
    TENSORFLOW =2;
}
Framework framework = 11;
bytes data = 100; // Model data
repeated AITensorDescription inputs = 12; // Description of input tensors
repeated AITensorDescription outputs = 13; // Description of output tensors
};
```

8.2.4.3.7 AINNNodeDescription

NN node description, which is used for describing the model, input tensor, and output tensor required by an NN node. For details, see [ai_types.proto](#).

```
message AINNNodeDescription
{
    string name = 1; // NN node name
    string desc = 2; // NN node description
    bool isPreAllocateOutputMem = 3; // Whether to pre-allocate the output memory
    AIConfig config = 4; // Configuration parameter
    repeated AIModelDescription model_list = 5; // List of models required by an NN node
    repeated AITensorDescription inputs = 6; // Description of input tensors
    repeated AITensorDescription outputs = 7; // Description of output tensors
    bool need_verify = 8; // Whether to verify the tensor match during serial connection
    repeated string ignored_check_aitensor = 9; // Specifies the list of tensors for which the mechanism for
    verifying whether they match the tensors of the inputs is ignored during serial connection.
};
```

8.2.4.3.8 AINNNodeDescriptionList

NN node description list, which is used for describing the model, input tensor, and output tensor required by an NN node. For details, see [ai_types.proto](#).

```
message AINNNodeDescriptionList
{
    repeated AINNNodeDescription nnnode_descs = 1; // NN node list
}
```

8.2.4.3.9 AIAPIDescription

API description, which is used for describing the name, input tensor, and output tensor of an API. For details, see [ai_types.proto](#).

```
message AIAPIDescription
{
    string name = 1; // API name
    string desc = 2; // API description
    bool isPreAllocateOutputMem = 3; // Whether to pre-allocate the output memory
    AIConfig config = 4; // Configuration parameter
    repeated AITensorDescription inputs = 5; // Description of input tensors
    repeated AITensorDescription outputs = 6; // Description of output tensors
    bool need_verify = 7; // Whether to verify the tensor match during serial connection
    repeated string ignored_check_aitensor = 8; // Specifies the list of tensors for which the mechanism for
    verifying whether they match the tensors of the inputs is ignored during serial connection.
};
```

8.2.4.3.10 AIAPIDescriptionList

API description list, which is used for describing the name, input tensor, and output tensor of an API. For details, see [ai_types.proto](#).

```
message AIAPIDescriptionList
{
    repeated AIAPIDescription api_descs = 1; // API list
}
```

8.2.4.3.11 AIOPDescription

This data type is defined in **ai_types.proto**.

```
// AI operation description
message AIOPDescription
{
    enum OP_Type
    {
        AI_API = 0;
        AI_NNNode = 1;
    }

    OP_Type type = 1;
    AINNNodeDescription nnode_desc = 2;
    AIAPIDescription api_desc = 3;
};
```

8.2.4.3.12 AIOPDescriptionList

This data type is defined in **ai_types.proto**.

```
// AI operation description list
message AIOPDescriptionList
{
    repeated AIOPDescription op_descs = 1; // AI operation list
}
```

8.2.4.3.13 NodeDesc

This data type is defined in **ai_types.proto**.

```
message NodeDesc
{
    string name=1; // IAINNNode or ALG_API name
    AIConfig config=2; // Initialization parameter required by IAINNNode or ALG_API
    repeated AIModelDescription mode_desc=3; // Initialization parameter required by IAINNNode
}
```

8.2.4.3.14 EngineDesc

This data type is defined in **ai_types.proto**.

```
message EngineDesc
{
    enum RunSide
    {
        DEVICE=0;
        HOST=1;
    }
    enum EngineType
    {
        NORMAL=0;
        SOURCE=1;
        DEST=2;
    }
    uint32 id=1; // Engine ID (node)
    EngineType type=2;
    string name=3; // Engine node name
    repeated string so_name=4; // List of all required DLL .so files
    RunSide side=5; // Run on the host or device.
    int32 priority=6; // Node priority
}
```

```
uint32 instance_cnt=7; // Number of instances (equivalent to the number of threads)
repeated uint32 next_node_id=8; // Next node list
bool user_input_cb=9; // IDE can be ignored.
bool user_output_cb=10; // IDE can be ignored.
repeated NodeDesc oper=11; // HIAEngine node list
}
```

8.2.4.3.15 GraphInitDesc

This data type is defined in **ai_types.proto**.

```
message GraphInitDesc
{
    int32 priority=1; // Priority of the entire process for a graph
}
```

8.2.4.3.16 GeneralFileBuffer

This data type is defined in **ai_types.proto**.

```
message GeneralFileBuffer
{
    bytes raw_data = 1;
    string file_name = 2;
}
```

8.2.4.3.17 AIContext

Saves the process context when the **Process** interface of the model manager is called asynchronously, including the key-value pair of the string type. This data type is defined in **ai_types.h**.

```
class AIContext
{
public:
    /*
     * @brief Obtain a parameter.
     * @param [in] key Key corresponding to the parameter
     * @return string Value corresponding to the key. If the value does not exist, a null string is returned.
     */
    const std::string GetPara(const std::string &key) const;

    /*
     * @brief Set a parameter.
     * @param [in] key Key corresponding to the parameter
     * @param [in] value Value corresponding to the parameter
     */
    void AddPara(const std::string &key, const std::string &value);

    /*
     * @brief Delete a parameter.
     * @param [in] key Key corresponding to the parameter to be deleted
     */
    void DeletePara(const std::string &key);

    /*
     * @brief Obtain all parameters.
     * @param [out] keys Key corresponding to all parameters that have been set
     */
    void GetAllKeys(std::vector<std::string> &keys);

#if defined(__ANDROID__) || defined(ANDROID)
    std::string Serialize();

    AIStatus Deserialize(std::string str);
#endif
private:
```

```
std::map<std::string, std::string> paras_; /** Definition of the name-value pairs of parameters */  
};
```

8.2.4.3.18 TensorDimension

The [AIModelManager::GetModelIOTensorDim](#) API can be called to obtain the information about the model input/output size, including the tensor dimension information, data type, buffer size, and tensor name. This data type is defined in [ai_types.h](#).

```
/*  
 * Describe the dimensions of a tensor.  
 */  
struct TensorDimension  
{  
    uint32_t n; // Batch size  
    uint32_t c; // Feature map channels  
    uint32_t h; // Feature map height  
    uint32_t w; // Feature map width  
    uint32_t data_type;  
    uint32_t size;  
    std::string name;  
};
```

- If the parameters related to AIPP are not configured during model conversion, the information about the model input/output dimensions obtained by calling [AIModelManager::GetModelIOTensorDim](#) is as follows:

- **n**, **c**, **h**, and **w** are those before model conversion.
- The enumerated value of **data_type** is defined as follows:

```
enum DataType  
{  
    DT_UNDEFINED = 0; // Used to indicate a DataType field has not been set.  
    DT_FLOAT = 1; // float type, 4 bytes  
    DT_FLOAT16 = 2; // fp16 type, 2 bytes  
    DT_INT8 = 3; // int8 type, 1 byte  
    DT_UINT8 = 4; // uint8 type, 1 byte  
    DT_INT16 = 5; // int16 type, 2 bytes  
    DT_UINT16 = 6; // uint16 type, 2 bytes  
    DT_INT32 = 7; // int32 type, 4 bytes  
    DT_INT64 = 8; // int64 type, 8 bytes  
    DT_UINT32 = 9; // unsigned int32, 4 bytes  
    DT_UINT64 = 10; // unsigned int64, 8 bytes  
    DT_BOOL = 11; // boolean type, 1 byte  
    DT_DOUBLE = 12; // double type, 8 bytes  
}
```

- $\text{size} = n * c * h * w * \text{Number of bytes of each data type}$

- If AIPP parameters are set during model conversion, the information about the model input/output size obtained by [AIModelManager::GetModelIOTensorDim](#) is as follows:

- **n**, **c**, **h**, and **w** are those before model conversion, instead of **h** (corresponding to the **src_image_size_h** parameter) and **w** (corresponding to the **src_image_size_w** parameter) configured in AIPP.
- The value of **data_type** is fixed at **3**, indicating the DT_INT8 type.
- The parameter **size** indicates the size of the picture data processed by AIPP. The size varies according to the input format. For details, see [Table 8-1](#).

Table 8-1 Size formula

input_format	size
YUV400_U8	$n * \text{src_image_size_w} * \text{src_image_size_h}$
YUV420SP_U8	$n * \text{src_image_size_w} * \text{src_image_size_h} * 1.5$
XRGB8888_U8	$n * \text{src_image_size_w} * \text{src_image_size_h} * 4$
RGB888_U8	$n * \text{src_image_size_w} * \text{src_image_size_h} * 3$

NOTE

For details about model conversion or AI PP, see *Model Conversion Guide*.

8.2.4.3.19 IAIListener

Provides IAIListener when the Process interface is asynchronously called, used for callback notification after model execution is complete. The details are as follows. This data type is defined in **ai_nn_node.h**.

```
/*
 * Asynchronous callback interface, which is implemented by the caller
 */
class IAIListener
{
public:
    virtual ~IAIListener(){}
    /*
     * @brief Asynchronous callback interface
     * @param [in] context Context information, including variable parameter configurations when the
     NNNode is running
     * @param [in] result Task status when the execution is complete
     * @param [in] out_data Output data when the execution is complete
     */
    virtual void OnProcessDone(const AIContext &context, int result,
        const std::vector<std::shared_ptr<IAITensor>> &out_data) = 0;
    /*
     * @brief Service death callback interface. When the client is suspended from the server, the
     application is notified.
     */
    virtual void OnServiceDied() {};
};
```

8.2.4.3.20 IAITensor

Serves as the input and output of a model when the process function is used. This data type is defined in **ai_tensor.h**.

```
/**
 * Tensor Input and output data
 */
class IAITensor
{
public:
    IAITensor() {}
    virtual ~IAITensor() {};
    /*
     * @brief Set parameters through AITensorDescription.
     * @param [in] tensor_desc Tensor description
     * @return true: Initialization success
     * @return false: Initialization failure. The possible cause is that tensor_desc is inconsistent with the
     current tensor.
     */
};
```

```
*/
virtual bool Init(const AITensorDescription &tensor_desc) = 0;
/*
 * @brief Obtain the type name.
 */
virtual const char* const GetTypeName() = 0;
/*
 * @brief Obtain the byte length after serialization.
 */
virtual uint32_t ByteSizeLong() = 0;
virtual void SetBufferAttr(void *data, int32_t size, bool isowner, bool is_support_mem_share) = 0;

virtual bool IsSupportZeroCopy() = 0;

#ifdef __ANDROID__ || defined(ANDROID)
/*
 * @brief Serialize data to the buffer for cross-process data interaction.
 * @param [in] buffer Address of the buffer to which the data is serialized, allocated by the caller
 * @param [in] size Size of the buffer to which the data is output
 * @return SUCCESS Success
 *         FAILED: failure. If the tensor does not support cross-process, this API does not need to be
implemented and a failure message is returned.
 */
virtual AStatus SerializeTo(void* buffer, const uint32_t size) = 0;

/*
 * @brief Deserialize data from the buffer to a tensor for cross-process data interaction.
 * @param [in] buffer Size of the buffer to which the data is input
 * @param [in] size Size of the buffer to which the data is input
 * @return SUCCESS Success
 *         FAILED: failure. If the tensor does not support cross-process, this API does not need to be
implemented and a failure message is returned.
 */
virtual AStatus DeserializeFrom(const void* buffer, const uint32_t size) = 0;
#endif
};
```

8.2.4.4 Other Compilation Dependent APIs

This section describes the compilation dependent APIs. It recommended that these APIs not be directly called.

8.2.4.4.1 AIAlgAPIFactory

ALG API registration factory class. This class is defined in **ai_alg_api.h**.

```
class AIAlgAPIFactory
{
public:
    static AIAlgAPIFactory* GetInstance();

    /*
     * @brief Obtain the API.
     * @param [in] name API name
     * @return API prototype pointer
     */
    AI_ALG_API GetAPI(const std::string &name);

    /*
     * @brief Register an API.
     * @param [in] desc API description
     * @param [in] func API definition
     * @return SUCCESS Success
     * @return Other: failure
     */
    AStatus RegisterAPI(const AIAPIDescription &desc, AI_ALG_API func);
};
```

```

    * @brief Obtain all API descriptions.
    * @param [in] api_desc_list API description list
    */
    void GetAllAPIDescription(AIAPIDescriptionList &api_desc_list);

    /*
    * @brief Deregister an API.
    * @param [in] api_desc API description
    */
    AIStatus UnRegisterApi(const AIAPIDescription &api_desc);

    /*
    * @brief Obtain the API description.
    * @param [in] name API name
    * @param [in] api_desc API description
    * @return SUCCESS Success
    * @return Other: failure
    */
    AIStatus GetAPIDescription(const std::string &name, AIAPIDescription &api_desc);

private:
    std::map<std::string, AI_ALG_API> func_map_;
    std::map<std::string, AIAPIDescription> desc_map_;
    std::mutex api_reg_lock_;
};

```

8.2.4.4.2 AIAlgAPIRegistrar

Macro for registering and encapsulating the **AIAlgAPIFactory** class. This class is defined in **ai_alg_api.h**.

```

class AIAlgAPIRegistrar
{
public:
    AIAlgAPIRegistrar(const AIAPIDescription &desc, AI_ALG_API func)
    {
        AIAlgAPIFactory::GetInstance()->RegisterAPI(desc, func);
        api_desc_ = desc;
    }

    ~AIAlgAPIRegistrar()
    {
        AIAlgAPIFactory::GetInstance()->UnRegisterApi(api_desc_);
    }
private:
    AIAPIDescription api_desc_;
};

```

8.2.4.4.3 REGISTER_ALG_API_UNIQUE

Macro for API registration, which is used in API implementation. This class is defined in **ai_alg_api.h**.

```

#define REGISTER_ALG_API_UNIQUE(desc, ctr, func) \
    AIAlgAPIRegistrar g_##ctr##_api_registerar(desc, func)

```

8.2.4.4.4 REGISTER_ALG_API

Macro for encapsulating **REGISTER_ALG_API_UNIQUE**. This class is defined in **ai_alg_api.h**.

```

#define REGISTER_ALG_API(name, desc, func) \
    REGISTER_ALG_API_UNIQUE(desc, name, func)

```

8.2.4.4.5 AIModelManager

Model manager class. This class is defined in **ai_model_manager.h**.

```
class AIModelManager : public IAINNNode
{
public:
    AIModelManager();

    /*
    * @brief Set the dynamic batch feature. This API does not need to be called by users.
    * @param [in] inputDim Input dimension of the model
    * @param [in] input Input data
    * @param [out] inputIndex Set the sequence number of a dynamic batch for input, starting from 0.
    * @param [in] batchSize Dynamic batch size
    * @return SUCCESS Success
    * @return Other: failure
    */
    AIStatus SetInputDynamicBatch(const vector<TensorDimension>& inputDim, std::vector
<std::shared_ptr<IAITensor>> &input,
    uint32_t inputIndex, uint32_t batchSize);

    /*
    * @brief Whether to pre-allocate the output memory. This API is implemented by the NNNode
    service. The default value is true.
    */
    virtual bool IsPreAllocateOutputMem() override;

    /*
    * @brief Obtain the AINNNodeDescription object.
    */
    static AINNNodeDescription GetDescription();

    /*
    * @brief Obtain the input and output dimensions of the loaded model.
    * @param [in] model_name Model name
    * @param [out] input_tensor Input dimension of the model
    * @param [out] output_tensor Output dimension of the model
    * @return SUCCESS Success
    * @return Other: failure
    */
    AIStatus GetModelIOTensorDim(const std::string& model_name,
    std::vector<TensorDimension>& input_tensor, std::vector<TensorDimension>& output_tensor);

    /*
    * @brief Set the thread inference request ID.
    * @param [in] request_id Model name
    * @return None
    */
    static void SetRequestId(uint64_t request_id);

    ~AIModelManager();

#ifdef __LITE__
    /*
    * @brief Release resources in the case of idle state and timeout. UnloadModels is called for
    implementation.
    */
    virtual AIStatus IdleRelease();
    /*
    * @brief recovers Resume services after timeout, including enabling a device and loading a model.
    LoadModels is called for implementation.
    */
    virtual AIStatus IdleResume();
#endif // __LITE__

private:
    AIModelManagerImpl* impl_;
};
```


8.2.4.4.6 getModelInfo

Macro for parsing a model file to obtain model information. This API is defined in **ai_model_parser.h**.

```
/**
 * @ingroup hiai
 * @brief Parse a model file to obtain model information.
 * @param [in] model_path Model file path
 * @param [in] key Model decryption key
 * @param [out] model_desc Model information
 * @return Status Running result
 */
hiai::AIStatus getModelInfo(const std::string & model_path,
                           const std::string & key, AIModelDescription & model_desc);
```

8.2.4.4.7 IAINNNode

NN node API, which is implemented by the service provider. The IAINNNode class is defined in **ai_nn_node.h**.

```
class IAINNNode
{
public:
    virtual ~IAINNNode(){}
    /**
     * @brief initialization interface, in which the service implements model loading or other initialization
     actions
     * @param [in] model_desc Model information. If the model is not required, a null vector is transferred.
     * @param [in] config Configuration parameters
     * @return SUCCESS Success
     * @return Other: failure
     */
    virtual AIStatus Init(const AIConfig &config,
                        const std::vector<AIModelDescription> &model_descs = {}) = 0;

    /**
     * @brief Set listener.
     * @param [in] If the listener is set to a null pointer, the process interface is called synchronously.
     Otherwise, the process interface is called asynchronously.
     * @return SUCCESS Success
     * @return Other: failure
     */
    virtual AIStatus SetListener(std::shared_ptr<IAIListener> listener) = 0;

    /**
     * @brief Compute interface
     * @param [in] context Context information, including variable parameter configurations when the
     NNNode is running
     * @param [in] in_data Input data
     * @param [out] out_data Output data
     * @param [in] timeout Timeout interval, which is invalid during calling in sync mode
     * @return SUCCESS Success
     * @return Other: failure
     */
    virtual AIStatus Process(AIContext &context,
                        const std::vector<std::shared_ptr<IAITensor>> &in_data,
                        std::vector<std::shared_ptr<IAITensor>> &out_data, uint32_t timeout) = 0;

    /**
     * @brief Create a list of output tensors.
     * @param [in] in_data List of input tensors, which may be used for calculating output
     * @param [out] out_data List of output tensors
     * @return SUCCESS Success
     * @return Other: failure
     */
    virtual AIStatus CreateOutputTensor(
```

```
const std::vector<std::shared_ptr<IAITensor>> &in_data,
std::vector<std::shared_ptr<IAITensor>> &out_data) { (void)in_data;(void)out_data;return SUCCESS; }

/*
 * @brief Whether to pre-allocate the output memory. This interface is implemented by the NNNode
service. The default value is true.
 */
virtual bool IsPreAllocateOutputMem() { return true; }

/*
 * @brief Check whether the NNNode is valid.
 */
virtual AIStatus IsValid() { return SUCCESS; }

/*
 * @brief Query the synchronization mode supported by the node.
 * @return BOTH Synchronous and asynchronous modes are both supported.
 * @return ASYNC Only the asynchronous mode is supported.
 * @return SYNC Only the synchronous mode is supported.
 */
virtual AI_NODE_EXEC_MODE GetSupportedExecMode() { return AI_NODE_EXEC_MODE::BOTH; }

#ifdef __LITE__
/*
 * @brief Release resources when an NN node is in the idle state and times out. This API is user-
defined and applies to the lite scenario. If the model manger is used, the corresponding IdleRelease
function should be called.
 */
virtual AIStatus IdleRelease() { return SUCCESS; }
/*
 * @brief Resume services after an NN node times out, including enabling a device and loading a
model. This API is user-defined and applies to the lite scenario. If the model manger is used, the
corresponding IdleResume function should be called.
 */
virtual AIStatus IdleResume() { return SUCCESS; }
/*
 * @brief Set the maximum idle time. If the maximum idle time is exceeded, the system automatically
destroys and releases resources. The default idle time is 60s.
 * Called in the service constructor function
 * Implemented in AIServiceBase
 * @param [in] time Maximum idle time, in ms
 */
virtual void SetMaxIdleTime(const int32_t time) { (void)time;}
#endif // __LITE__

/*
 * @brief Obtain the maximum memory used by the service.
 * @return Maximum size of the memory used by the service
 */
virtual uint32_t GetMaxUsedMemory() { return 0; }
};
```

8.2.4.4.8 AINNNodeFactory

Macro that supports service engine self-registration, provides the API for creating an NN node, and supports the query of registered NN node description information (by name or all). The AINNNodeFactory class is defined in **ai_nn_node.h**.

```
class AINNNodeFactory
{
public:
    static AINNNodeFactory* GetInstance();

    /*
     * @brief Create an NN node based on the NN node name.
     * @param [in] name NN node name
     * @return std::shared_ptr<IAINNNode> Pointer to the NN node object corresponding to the name. If
```

```
nullptr is returned, the corresponding NN node cannot be found.
*/
std::shared_ptr<AINNNode> CreateNNNode(const std::string &name);

/*
 * @brief Obtain the descriptions of all registered NN nodes.
 * @param [out] nnnode_desc Descriptions of all registered NN nodes
 * @return SUCCESS Success
 * @return Other: failure
 */
void GetAllNNNodeDescription(AINNNodeDescriptionList &nnnode_desc);

/*
 * @brief Obtain the description of the NN node based on the NN node name.
 * @param [in] name NN node name
 * @param [out] engin_desc NN node description
 * @return SUCCESS Success
 * @return Other: failure
 */
AIStatus GetNNNodeDescription(const std::string &name, AINNNodeDescription &engin_desc);

/*
 * @brief Register the NN node creation function.
 * @param [in] nnnode_desc NN node description
 * @param [in] create_func Function for creating an NN node
 * @return SUCCESS Success
 * @return Other: failure
 */
AIStatus RegisterNNNodeCreator(const AINNNodeDescription &nnnode_desc,
    AINNNODE_CREATE_FUN create_func);
AIStatus RegisterNNNodeCreator(const string nnnode_str,
    AINNNODE_CREATE_FUN create_func);

/*
 * @brief Deregister the NN node.
 * @param [in] name NN node name
 * @return SUCCESS Success
 */
AIStatus UnRegisterNNNode(const AINNNodeDescription &nnnode_desc);
AIStatus UnRegisterNNNode(const string nnnode_str);

private:
    std::map<std::string, AINNNODE_CREATE_FUN> create_func_map_;
    std::map<std::string, AINNNodeDescription> nnnode_desc_map_;
    std::mutex map_lock_;
};
```

8.2.4.4.9 AINNNodeRegistrar

NN node registration class. This class is defined in **ai_nn_node.h**.

```
class AINNNodeRegistrar
{
public:
    AINNNodeRegistrar(const AINNNodeDescription &nnnode_desc, AINNNODE_CREATE_FUN
create_func)
    {
        AINNNodeFactory::GetInstance()->RegisterNNNodeCreator(nnnode_desc, create_func);
        nnnode_desc_ = nnnode_desc;
        nnnode_str_erase(nnnode_str_.begin(), nnnode_str_.end());
    }

    ~AINNNodeRegistrar(){
        AINNNodeFactory::GetInstance()->UnRegisterNNNode(nnnode_desc_);
    }
private:
    AINNNodeDescription nnnode_desc_;
    string nnnode_str_;
};
```

8.2.4.4.10 REGISTER_NN_NODE

NN node registration macro.

```
/*
 * @brief NNNode Registration macro. The service NN node is used in the implementation class.
 * REGISTER_ENGINE(desc, clazz) is used directly.
 * @param [in] desc Object of the NN node description information
 * @param [in] clazz Class name of the NN node
 */
#define REGISTER_NN_NODE(desc, name) \
    std::shared_ptr<IAINNNode> NNNode_##name##_Creator() \
    { \
        return std::make_shared<name>(); \
    } \
    AINNNodeRegistrar g_nnnode_##name##_creator(desc, NNNode_##name##_Creator)
```

8.2.4.4.11 AITensorGetBytes

Macro for obtaining the byte size of a tensor. This API is defined in **ai_tensor.h**.

```
extern uint32_t AITensorGetBytes(int32_t data_type);
```

8.2.4.4.12 AITensorFactory

Tensor factory class, which is used to create tensors. The **AITensorFactory** class is defined in **ai_tensor.h**.

```
class AITensorFactory
{
public:
    static AITensorFactory* GetInstance();

    /*
     * @brief Create a tensor by using parameters, including memory allocation
     * @param [in] tensor_desc Description that contains the tensor parameters
     * @return shared_ptr<IAITensor> Pointer to the created tensor. If the creation fails, a null pointer is
     returned.
     */
    std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription &tensor_desc);

    /*
     * @brief Create a tensor by using the type, not involving memory allocation. It is used in non-pre-
     allocation mode.
     * @param [in] type Type during tensor registration
     * @return shared_ptr<IAITensor> Pointer to the created tensor. If the creation fails, a null pointer is
     returned.
     */
    std::shared_ptr<IAITensor> CreateTensor(const std::string &type);

#if defined( __ANDROID__ ) || defined(ANDROID)
    /*
     * @brief Creates a tensor by using parameters and buffers. The content is obtained from buffer
     deserialization.
     * @param [in] tensor_desc Description that contains the tensor parameters
     * @param [in] buffer Existing data buffer
     * @param [in] size Buffer size
     * @return shared_ptr<IAITensor> Pointer to the created tensor. If the creation fails, a null pointer is
     returned.
     */
    std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription &tensor_desc,
        const void *buffer, const int32_t size);
#else
    std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription &tensor_desc,
        void *buffer, int32_t size);
#endif
};
```

```
/*
 * @brief Register a tensor.
 * @param [in] tensor_desc Tensor description
 * @param [in] create_func Function for creating a tensor
 */
AIStatus RegisterTensor(const AITensorDescription &tensor_desc,
    CREATE_TENSOR_FUN create_func);
AIStatus RegisterTensor(const string tensor_str, CREATE_TENSOR_FUN create_func)
{
    // Add a lock to prevent multi-thread concurrency.
    AITensorDescription tensor_desc;
    tensor_desc.set_type(tensor_str);
    return RegisterTensor(tensor_desc, create_func);
}

/*
 * @brief Unregister a tensor.
 * @param [in] tensor_desc Tensor description
 */
AIStatus UnRegisterTensor(const AITensorDescription &tensor_desc);
AIStatus UnRegisterTensor(const string tensor_str)
{
    AITensorDescription tensor_desc;
    tensor_desc.set_type(tensor_str);
    return UnRegisterTensor(tensor_desc);
}

/*
 * @brief Obtain the list of all tensors.
 * @param [out] tensor_desc_list Description list of output tensors
 */
void GetAllTensorDescription(AITensorDescriptionList &tensor_desc_list);

/*
 * @brief Obtain the tensor description.
 */
AIStatus GetTensorDescription(const std::string& tensor_type_name,
    AITensorDescription &tensor_desc);

private:
    std::map<std::string, AITensorDescription> tensor_desc_map_;
    std::map<std::string, CREATE_TENSOR_FUN> create_func_map_;
    std::mutex tensor_reg_lock_;
};
```

8.2.4.4.13 REGISTER_TENSOR_CREATER_UNIQUE

Tensor registration macro. This macro is defined in **ai_tensor.h**.

```
#define REGISTER_TENSOR_CREATER_UNIQUE(desc, name, func) \
    AITensorRegistrar g_##name##_tensor_registerar(desc, func)
```

8.2.4.4.14 REGISTER_TENSOR_CREATER

Macro for encapsulating the tensor registration macro. This macro is defined in **ai_tensor.h**.

```
#define REGISTER_TENSOR_CREATER(name, desc, func) \
    REGISTER_TENSOR_CREATER_UNIQUE(desc, name, func)
```

8.2.4.4.15 AISimpleTensor

Tensor of a simple data type. This class is defined in **ai_tensor.h**.

```
class AISimpleTensor : public IAITensor
{
public:
```

```
    AISimpleTensor();

    ~AISimpleTensor();

    /*
    * @brief Obtain the type name.
    */
    virtual const char* const GetTypeName() override;

    /* Obtain the data address. */
    void* GetBuffer();

    /*
    @brief Set the data address.
    @param [in] data Data pointer
    @param [in] size Size
    @param [in] isown Whether the tensor releases the data address memory after the tensor life cycle
ends.
    */
    void SetBuffer(void *data, const int32_t size, bool isown=false);

    /*
    @brief Obtain the space occupied by data.
    */
    uint32_t GetSize();

    /*
    @brief Allocate data space.
    */
    void* MallocDataBuffer(uint32_t size);

    /*
    * @brief Obtain the byte length after serialization.
    */
    virtual uint32_t ByteSizeLong() override;

    /*
    * @brief Set parameters through AITensorDescription.
    * @param [in] tensor_desc Tensor description
    * @return true: Initialization success
    * @return false: Initialization failure. The possible cause is that tensor_desc is inconsistent with the
current tensor.
    */
    virtual bool Init(const AITensorDescription &tensor_desc) override;

#if defined( __ANDROID__ ) || defined(ANDROID)
    /*
    * @brief Serialize data to the buffer for cross-process data interaction.
    * @param [in] buffer Address of the buffer to which the data is serialized, allocated by the caller
    * @param [in] size Size of the buffer to which the data is output
    * @return SUCCESS Success
    * FAILED: failure. If the tensor does not support cross-process, this API does not need to be
implemented and a failure message is returned.
    */
    virtual AIStatus SerializeTo(void* buffer, const uint32_t size) override;

    /*
    * @brief Deserialize data from the buffer to a tensor for cross-process data interaction.
    * @param [in] buffer Size of the buffer to which the data is input
    * @param [in] size Size of the buffer to which the data is input
    * @return SUCCESS Success
    * FAILED: failure. If the tensor does not support cross-process, this API does not need to be
implemented and a failure message is returned.
    */
    virtual AIStatus DeserializeFrom(const void* buffer, const uint32_t size) override;
#endif

    /*
    @brief Build tensor description.
```

```
*/
static AITensorDescription BuildDescription(const std::string& size = "");

/*
@brief Create a tensor according to the description.
*/
static std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription& tensor_desc);

bool IsSupportZeroCopy();
/*
@brief Set the data address.
@param [in] data Data pointer
@param [in] size Size
@param [in] isown Whether the tensor releases the data address memory after the tensor life cycle
ends.
@param [in] is_data_support_mem_share_ Whether the tensor releases the data address memory
after the tensor life cycle ends.
*/
void SetBufferAttr(void *data, int32_t size, bool isowner, bool is_support_mem_share) override;

private:
void* data_;
uint32_t size_;
bool isowner_;

static std::string type_name_;
}
```

8.2.4.4.16 AINeuralNetworkBuffer

Defines the general buffer of the NN. This class is defined in **ai_tensor.h**.

```
class AINeuralNetworkBuffer : public AISimpleTensor
{
public:
    AINeuralNetworkBuffer()
    {
        data_type_ = 0;
        number_ = 1;
        channel_ = 1;
        height_ = 1;
        width_ = 1;
        name_ = "";
    };

    ~AINeuralNetworkBuffer() {};
    /*
    * @brief Obtain the type name.
    */
    const char* const GetTypeName();

    /*
    * @brief Obtain the byte size.
    */
    uint32_t ByteSizeLong();

    /*
    * @brief Initialization
    * @param [in] tensor_desc Tensor description
    */
    bool Init(const AITensorDescription &tensor_desc);

#ifdef __ANDROID__ || defined(ANDROID)
    /*
    * @brief Serialize data to the buffer for cross-process data interaction.
    * @param [in] buffer Address of the buffer to which the data is serialized, allocated by the caller
    * @param [in] size Size of the buffer to which the data is output
    * @return SUCCESS Success
    *         FAILED: failure. If the tensor does not support cross-process, this API does not need to be
    */

```

```

implemented and a failure message is returned.
    */
    AIStatus SerializeTo(void* buffer, uint32_t size_);

    /*
    * @brief Deserialize data from the buffer to a tensor for cross-process data interaction.
    * @param [in] buffer Size of the buffer to which the data is input
    * @param [in] size Size of the buffer to which the data is input
    * @return SUCCESS Success
    * FAILED: failure. If the tensor does not support cross-process, this API does not need to be
    implemented and a failure message is returned.
    */
    AIStatus DeserializeFrom(const void* buffer, uint32_t size_);
#endif
    /*
    * @brief Obtain the description.
    */
    static AITensorDescription GetDescription(
        const std::string &size = "0", const std::string &data_type="0",
        const std::string &number="0", const std::string &channel="0",
        const std::string &height="0", const std::string &width="0");

    /*
    @brief Create a tensor.
    */
    static std::shared_ptr<AITensor> CreateTensor(const AITensorDescription& tensor_desc);

    /*
    @brief Obtain the number.
    */
    int32_t GetNumber();

    /*
    @brief Set the number.
    */
    void SetNumber(int32_t number);

    /*
    @brief Obtain the number of channels.
    */
    int32_t GetChannel();

    /*
    @brief Set the channel.
    */
    void SetChannel(int32_t channel);

    /*
    @brief Obtain the height.
    */
    int32_t GetHeight();

    /*
    @brief Set the height.
    */
    void SetHeight(int32_t height);

    /*
    @brief Obtain the width.
    */
    int32_t GetWidth();

    /*
    @brief Set the width.
    */
    void SetWidth(int32_t width);

    /*
    @brief Obtain the data type.

```



```
*/
int32_t GetData_type();

/*
@brief Set the data type.
*/
void SetData_type(int32_t data_type);

/*
@brief Obtain the data type.
*/
const std::string& GetName() const;

/*
@brief Set the data type.
*/
void SetName(const std::string& value);

private:
int32_t data_type_;
int32_t number_;
int32_t channel_;
int32_t height_;
int32_t width_;
std::string name_;
};
```

8.2.4.4.17 AllImageBuffer

Defines the general buffer of images. This class is defined in **ai_tensor.h**.

```
class AllImageBuffer : public AISimpleTensor
{
public:
    AllImageBuffer()
    {
        format_ = JPEG;
        width_ = 0;
        height_ = 0;
    };

    /*
    * @brief Obtain the type name.
    */
    const char* const GetTypeName();

    /*
    @brief Obtain the byte size.
    */
    uint32_t ByteSizeLong();

    /*
    @brief Initialization
    @param [in] tensor_desc Tensor description
    */
    bool Init(const AITensorDescription &tensor_desc);

#if defined( __ANDROID__ ) || defined(ANDROID)
    /*
    * @brief Serialize data to the buffer for cross-process data interaction.
    * @param [in] buffer Address of the buffer to which the data is serialized, allocated by the caller
    * @param [in] size Size of the buffer to which the data is output
    * @return SUCCESS Success
    * FAILED: failure. If the tensor does not support cross-process, this API does not need to be
    implemented and a failure message is returned.
    */
    AStatus SerializeTo(void* buffer, uint32_t size_);
    /*
    */
#endif
};
```

```

    * @brief Deserialize data from the buffer to a tensor for cross-process data interaction.
    * @param [in] buffer Size of the buffer to which the data is input
    * @param [in] size Size of the buffer to which the data is input
    * @return SUCCESS Success
    * FAILED: failure. If the tensor does not support cross-process, this API does not need to be
    implemented and a failure message is returned.
    */
    AIStatus DeserializeFrom(const void* buffer, uint32_t size_);
#endif
    /*
    * @brief Obtain the description.
    */
    static AITensorDescription GetDescription(
        const std::string &size = "0", const std::string &height="0",
        const std::string &width="0", const std::string format="JPEG");

    /*
    @brief Create a tensor.
    */
    static std::shared_ptr<IAITensor> CreateTensor(const AITensorDescription& tensor_desc);

    /*
    @brief Obtain the image format.
    */
    ImageFormat GetFormat();
    /*
    @brief Set the image format.
    */
    void SetFormat(ImageFormat format);

    /*
    @brief Obtain the height.
    */
    int32_t GetHeight();

    /*
    @brief Set the height.
    */
    void SetHeight(int32_t height);

    /*
    @brief Obtain the width.
    */
    int32_t GetWidth();

    /*
    @brief Set the width.
    */
    void SetWidth(int32_t width);
private:
    ImageFormat format_;
    int32_t width_;
    int32_t height_;
};

```

8.2.4.4.18 HIAILog

Log class of HiAI. This class is defined in **log.h**.

```

class HIAILog {
public:
    /**
    * @ingroup hiaiengine
    * @brief Obtain the HIAILog pointer.
    */
    HIAI_LIB_INTERNAL static HIAILog* GetInstance();

    /**
    * @ingroup hiaiengine

```

```
* @brief: Whether to initialize HiAILog or not
* @return: Whether to initialize HiAILog or not. "true" indicates yes.
*/
HIAI_LIB_INTERNAL bool IsInit() { return isInitFlag; }

/**
* @ingroup hiaiengine
* @brief Obtain the log output level.
* @return: Log output level
*/
HIAI_LIB_INTERNAL uint32_t HIAIGetCurLogLevel();

/**
* @ingroup hiaiengine
* @brief Check whether the log needs to be output.
* @param [in]errorCode: Error code of a message
* @return Whether a log can be output
*/
HIAI_LIB_INTERNAL bool HIAILogOutputEnable(const uint32_t errorCode);

/**
* @ingroup hiaiengine
* @brief Level corresponding to the log
* @param [in]logLevel: Log level in the macro definition
*/
HIAI_LIB_INTERNAL std::string HIAILevelName(const uint32_t logLevel);

/**
* @ingroup hiaiengine
* @brief Output a log.
* @param [in]moudleID: Component ID defined by enum
* @param [in]logLevel: Log level in the macro definition
* @param [in]strLog: Output log content
*/
HIAI_LIB_INTERNAL void HIAISaveLog(const int32_t moudleID,
    const uint32_t logLevel, const char* strLog);
protected:
/**
* @ingroup hiaiengine
* @brief HIAILog constructor
*/
HIAI_LIB_INTERNAL HIAILog();
HIAI_LIB_INTERNAL ~HIAILog() {}
private:
/**
* @ingroup hiaiengine
* @brief HIAILog initialization function
*/
HIAI_LIB_INTERNAL void Init();
HIAI_LIB_INTERNAL HIAILog(const HIAILog&) = delete;
HIAI_LIB_INTERNAL HIAILog(HIAILog&&) = delete;
HIAI_LIB_INTERNAL HIAILog& operator=(const HIAILog&) = delete;
HIAI_LIB_INTERNAL HIAILog& operator=(HIAILog&&) = delete;
private:
HIAI_LIB_INTERNAL static std::mutex mutexHandle;
uint32_t outputLogLevel;
std::map<uint32_t, std::string> levelName;
std::map<std::string, uint32_t> levelNum;
static bool isInitFlag;
};
```

8.2.4.4.19 HIAI_ENGINE_LOG

Log encapsulation macro, which encapsulates the **HIAI_ENGINE_LOG_IMPL** macro. This macro is defined in **log.h**.

```
#define HIAI_ENGINE_LOG(...) \
    HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##__VA_ARGS__)
```

8.2.4.5 Exception Handling

If the offline model manager API fails to be called and the return value is **FAILED**, you can view logs in the **Log** window on Mind Studio. View the latest logs based on the time and rectify the error based on the log information.

Example: When the **Init** API is called, the model to be loaded does not exist.

```
Init:hiaiengine/node/ai_model_manager_impl.cpp:163:"Load models failed!"
```

NOTE

For details about how to view logs, see section 2.3.1 "Viewing a Log" in *Ascend 310 Mind Studio Auxiliary Tools*.

8.2.5 Auxiliary APIs

8.2.5.1 Data Receiving APIs (C++ Language)

You can obtain related parameters by using the following auxiliary APIs. Most APIs for obtaining data are common between the C language and the C++ language. The APIs dedicated to C++ are described in the syntax.

8.2.5.1.1 Obtaining the Number of Devices

Obtains the number of devices. This API is defined in **c_graph.h**.

Syntax

```
HIAI_StatusT HIAI_GetDeviceNum(uint32_t *devCount);
```

Parameter Description

Parameter	Description	Value Range
devCount	Pointer to the number of output devices	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Level	Error Code	Description
1	HIAI_INFO	HIAI_OK	The running is OK.
2	HIAI_ERROR	HIAI_GRAPH_GET_DEVNUM_ERROR	There is an error in obtaining the number of devices.

Calling Example

```
uint32_t dev_count;  
HIAI_GetDeviceNum(&dev_count);
```

8.2.5.1.2 Obtaining the ID of the First Device

Obtains the ID of the first device. This API is defined in **c_graph.h**.

Syntax

```
HIAI_StatusT HIAI_GetFirstDeviceId(uint32_t *firstDevID);
```

Parameter Description

Parameter	Description	Value Range
firstDevID	Pointer to the ID of the first device	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Level	Error Code	Description
1	HIAI_INFO	HIAI_OK	The running is OK.
2	HIAI_ERROR	HIAI_GRAPH_GET_DEVID_ERROR	There is an error in obtaining the device ID.

Calling Example

```
uint32_t first_dev_id;  
HIAI_GetFirstDeviceId(&first_dev_id);
```

8.2.5.1.3 Obtaining the ID of the First Graph

Obtains the graph ID. This API is defined in **c_graph.h**.

Syntax

Common API between C and C++: **HIAI_StatusT HIAI_GetFirstGraphId(uint32_t *firstGraphID);**

Parameter Description

Parameter	Description	Value Range
firstGraphID	Pointer to the ID of the first graph	-

Return Value

Common API between C and C++: Error code

Error Codes

No.	Error Level	Error Code	Description
1	HIAI_INFO	HIAI_OK	The running is OK.
2	HIAI_ERROR	HIAI_GRAPH_GET_GRAPHID_ERROR	There is an error in obtaining the graph ID.

Calling Example

```
uint32_t first_graph_id;  
HIAI_GetFirstGraphId(&first_graph_id);
```

8.2.5.1.4 Obtaining the ID of the Next Device

Obtains the ID of the next device. This API is defined in **c_graph.h**.

Syntax

```
HIAI_StatusT HIAI_GetNextDeviceId(const uint32_t curDevID, uint32_t  
*nextDevID);
```

Parameter Description

Parameter	Description	Value Range
curDevID	ID of the current device	-
nextDevID	Pointer to the ID of the next device	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Level	Error Code	Description
1	HIAI_INFO	HIAI_OK	The running is OK.
2	HIAI_ERROR	HIAI_GRAPH_GET_DE VID_ERROR	There is an error in obtaining the device ID.

Calling Example

```
uint32_t cur_dev_id = 1;  
uint32_t next_dev_id;  
HIAI_GetNextDevicId(cur_dev_id, &next_dev_id);
```

8.2.5.1.5 Obtaining the ID of the Next Graph

Obtains the ID of the next graph. This API is defined in **c_graph.h**.

Syntax

Common API between C and C++: **HIAI_StatusT HIAI_GetNextGraphId(const uint32_t curGraphID, uint32_t *nextGraphID);**

Parameter Description

Parameter	Description	Value Range
curGraphID	ID of the current graph	-
nextGraphID	Pointer to the ID of the next graph	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Level	Error Code	Description
1	HIAI_INFO	HIAI_OK	The running is OK.
2	HIAI_ERROR	HIAI_GRAPH_GET_GR APHID_ERROR	There is an error in obtaining the graph ID.

Calling Example

```
uint32_t cur_graph_id = 1;  
uint32_t next_graph_id;  
HIAI_GetNextGraphId(cur_graph_id, &next_graph_id);
```

8.2.5.1.6 Obtaining the Engine Pointer

Queries the pointer to the engine object based on the engine ID. This API is defined in **graph.h**.

Syntax

```
std::shared_ptr<Engine> Graph::GetEngine(uint32_t engineID);
```

Parameter Description

Parameter	Description	Value Range
engineID	ID of the target engine	-

Return Value

Smart pointer to the engine

Calling Example

```
uint32_t engineID= 1000;  
auto graphPtr = Graph::GetInstance(100);  
auto enginePtr = graphPtr->GetEngine(engineID);
```

8.2.5.1.7 Obtaining the Graph ID

Obtains the ID of the current graph. This API is defined in **graph.h**.

Syntax

```
uint32_t Graph::GetGraphId();
```

Return Value

Graph ID

8.2.5.1.8 Obtaining the Device ID of a Graph

Obtains the ID of the current graph on the device side. This API is defined in **graph.h**.

Syntax

```
uint32_t Graph::GetDeviceID();
```

Return Value

ID of the current graph on the device side

Calling Example

```
auto graphPtr = Graph::GetInstance(100);  
auto deviceId = graphPtr->GetDeviceID();
```


8.2.5.1.9 Obtaining the Graph ID of an Engine

Obtains the graph ID of an engine. This API is defined in **engine.h**.

Syntax

```
uint32_t Engine::GetGraphId();
```

Return Value

Graph ID of an engine

Calling Example

```
uint32_t engineID= 1000;  
auto graphPtr = Graph::GetInstance(100);  
auto enginePtr = graphPtr->GetEngine(engineID);  
auto graphID= enginePtr->GetGraphId();
```

8.2.5.1.10 Obtaining the Maximum Queue Size of an Engine

Obtains the maximum queue size of an engine. This API is defined in **engine.h**.

Syntax

```
const uint32_t Engine::GetQueueMaxSize();
```

Return Value

Maximum queue size of the engine

Calling Example

```
uint32_t engineID= 1000;  
auto graphPtr = Graph::GetInstance(100);  
auto enginePtr = graphPtr->GetEngine(engineID);  
auto engineQueueSize = enginePtr->GetQueueMaxSize();
```

8.2.5.1.11 Obtaining the Current Queue Size of a Specified Port of the Engine

Obtains the queue size of a specified port of the engine. This API is defined in **engine.h**.

Syntax

```
const uint32_t Engine::GetQueueCurrentSize(const uint32_t portID);
```

Parameter Description

Parameter	Description	Value Range
portID	Port ID	-

Return Value

Queue size corresponding to the port ID of the engine

Calling Example

```
uint32_t engineID= 1000;  
uint32_t portID = 0;  
auto graphPtr = Graph::GetInstance(100);  
auto enginePtr = graphPtr->GetEngine(engineID);  
auto engineQueueSize = enginePtr->GetQueueCurrentSize(portID);
```

8.2.5.1.12 Parsing the Matrix Configuration File

Parses the configuration file. The Matrix parses the configuration file and writes the generated graph back to the list for the user. This API is defined in **graph.h**.

Syntax

```
static HIAI_StatusT Graph::ParseConfigFile(const std::string& configFile,  
GraphConfigList& graphConfigList)
```

Parameter Description

Parameter	Description	Value Range
configFile	Configuration file path. Ensure that the transferred file path is correct.	-
graphConfigList	Protobuf data format.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Level	Error Code	Description
1	-	HIAI_GRAPH_GET_INSTANCE_NULL	-

8.2.5.1.13 Obtaining the PCIe Information

Obtains the Peripheral Component Interconnect Express (PCIe) information. This API is defined in **c_graph.h**.

Syntax

```
static HIAI_StatusT HIAI_GetPCIeInfo(const uint32_t devId, int32_t* bus,  
int32_t* dev, int32_t* func);
```

Parameter Description

Parameter	Description	Value Range
devId	Device ID to be queried	-
bus	Returned PCIe bus ID	-
dev	Returned PCIe device ID	-
func	Returned PCIe function ID	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Level	Error Code	Description
1	HIAI_INFO	HIAI_OK	The running is OK.
2	HIAI_ERROR	HIAI_GRAPH_GET_PCI EINFO_ERROR	The bus , dev , or func parameter has a null pointer or there is an error in obtaining the PCIe information.

Calling Example

```
uint32_t devId= 1;  
uint32_t bus;  
uint32_t dev;  
uint32_t func;  
HIAI_GetPCleInfo(devId, &bus, &dev, &func);
```

8.2.5.1.14 Obtaining the Version Number

Obtains the version number. This API is defined in **c_graph.h**.

Syntax

```
HIAI_API_VERSION HIAI_GetAPIVersion();
```

Return Value

Enumerated information of the version number

Calling Example

```
HIAI_API_VERSION HIAI_GetAPIVersion();
```

8.2.5.1.15 Obtaining the OamConfig Smart Pointer

Obtains the OamConfig smart pointer, that is, to obtain OAM configuration information, including the model name, whether to enable the dump function, and layers at which operators need to be dumped in the model. This API is defined in **engine.h**.

Syntax

```
OAMConfigDef* GetOamConfig();
```

Parameter Description

None

Return Value

OamConfig smart pointer. For details about the definition of the **OAMConfigDef** type, see [8.2.7.2.1 Protobuf Data Types](#).

8.2.5.2 Data Type Serialization and Deserialization (C++ Language)

Automatic serialization and deserialization mechanisms are provided for various user-defined data types.

The following API is used for encapsulating related macros:

```
static HIAIDataTypeFactory* HIAIDataTypeFactory::GetInstance();
```

8.2.5.2.1 Macro: HIAI_REGISTER_DATA_TYPE

Provides automatic serialization and deserialization mechanisms for various user-defined data types. This macro is defined in **data_type_reg.h**.

 NOTE

- This API must be registered on both the host side and the device side.
- Due to the restrictions of Cereal, do not use the **long double** type in the struct body. To ensure running across platforms (for example, from Windows to Linux), the portable type is recommended for the struct members, for example, **int32_t**, in the struct body.
- Due to the restrictions of Cereal, the struct body needs to use the shared pointer.

Syntax

```
HIAI_REGISTER_DATA_TYPE(name, type)
```

Parameter Description

Parameter	Description	Value Range
name	Name of the user-defined data type. (The name must be unique for each data type.)	-

Parameter	Description	Value Range
type	User-defined data type	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_ENGINE_FUNCTOR_NULL	The HiAI Engine function is null.
3	HIAI_ENGINE_FUNCTOR_EXIST	The HiAI Engine function exists.

8.2.5.2.2 Macro: HIAI_REGISTER_TEMPLATE_DATA_TYPE

Provides automatic serialization and deserialization mechanisms for various data types of a user-defined template. This macro is defined in **data_type_reg.h**.

NOTE

This API needs to be registered on both the host side and the device side.

Syntax

HIAI_REGISTER_TEMPLATE_DATA_TYPE (name, type, basictype1, basictype2, ...)

Parameter Description

Parameter	Description	Value Range
name	Name of the user-defined data type. (The name must be unique for each data type.)	-
type	Data type of a user-defined template	-
basicType1	User-defined data type	-
basicType2	User-defined data type	-
...	...	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_ENGINE_FUNCTOR_NULL	The HiAI Engine function is null.
3	HIAI_ENGINE_FUNCTOR_EXIST	The HiAI Engine function exists.

8.2.5.2.3 Macro: HIAI_REGISTER_SERIALIZE_FUNC

Provides user-defined serialization and deserialization mechanisms for user-defined data types. This macro is defined in **data_type_reg.h**.

That is, during data sending, the struct pointers transferred by the user are converted into the struct buffer and data buffer. During data receiving, the struct buffer and data buffer obtained from the framework are converted into the structs.

Application scenario:

This API is used to quickly transfer data from the host side to the device side. If you want to improve the transfer performance, you must call this API.

NOTE

- This API is used to quickly transfer data from the host side to the device side.
- If you want to use a high-performance transfer API, you must call this API to register serialization and deserialization functions.

Syntax

HIAI_REGISTER_SERIALIZE_FUNC(name, type, hiaiSerializeFunc, hiaiDeSerializeFunc)

Parameter Description

Parameter	Description	Value Range
name	Name of a registered message	-
type	Type of a user-defined type	-

Parameter	Description	Value Range
hiaSerializeFunc	Serialization function. The function format is as follows: <pre>typedef void(*hiaSerializeFunc) (void* inputPtr, std::string& ctrlStr, uint8_t*& dataPtr, uint32_t& dataLen);</pre> The parameters are described as follows: • inputPtr: struct pointer • ctrlStr: struct buffer • dataPtr: buffer of the struct data pointer • dataLen: struct data size	-
hiaDeSerializeFunc	Deserialization function. The function format is as follows: <pre>typedef std::shared_ptr<void>(*hi aiDeSerializeFunc)(const char* ctrlPtr, const uint32_t& ctrlLen, const uint8_t* dataPtr, const uint32_t& dataLen);</pre> The parameters are described as follows: • ctrlPtr: struct pointer • ctrlLen: control information size in the data structure • dataPtr: struct data pointer • dataLen: size of the data storage space in the struct. This parameter is used only for verification and does not indicate the size of the original data.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_ENGINE_FUNCTOR_NULL	The HiAI Engine function is null.
3	HIAI_ENGINE_FUNCTOR_EXIST	The HiAI Engine function exists.

8.2.5.2.4 Graph::ReleaseDataBuffer

During data deserialization, this function is registered as a deleter when a value is assigned to the smart pointer in the data memory. This API is defined in **graph.h**.

Syntax

```
static void Graph::ReleaseDataBuffer(void* ptr)
```

Parameter Description

Parameter	Description	Value Range
ptr	Memory pointer	-

8.2.5.2.5 API Usage Example

```
/******  
*           Copyright (C) HiSilicon Co., Ltd.  
*  
*   Filename: user_def_datatype.h  
*   Description: User defined data type  
*  
*   Version: 1.0  
*   Created: 2018-01-08 10:15:18  
*   Author:  
*  
*   Revision: initial draft;  
*****/  
#ifndef USE_DEF_DATA_TYPE_H_  
#define USE_DEF_DATA_TYPE_H_  
  
#ifdef __cplusplus  
#include <vector>  
#include <map>  
#endif  
  
// User-defined data type (C language)  
typedef struct UseDefDataType
```



```
{
    int data1;
    float data2;
}UseDefDataTypeT;

#ifdef __cplusplus
// User-defined data type (with template)
template<typename T1, typename T2, typename T3>
class UseDefTemplateDataType
{
public:
    std::vector<T1> data_vec_;
    std::map<T2, T3> data_map_;
};
#endif

#ifdef
/*****
*           Copyright (C) HiSilicon Co., Ltd.
*
*   Filename: use_def_data_type_reg.cpp
*   Description: User defined Data Type Register
*
*   Version: 1.0
*   Created: 2018-01-08 10:15:18
*   Author: h00384043
*
*   Revision: initial draft;
*****/

#include "use_def_data_type.h"
#include "hiaiengine/data_type_reg.h"

// The user-defined data type must be registered with the Matrix before being used as the communication
// interface between engines.
template<class Archive>
void serialize(Archive& ar, UseDefDataTypeT& data)
{
    ar(data.data1, data.data2);
}

// Register UseDefDataTypeT.
HIAI_REGISTER_DATA_TYPE("UseDefDataTypeT", UseDefDataTypeT)

// Register the data of the template type.
// All the required template types must be registered.
template<class Archive, typename T1, typename T2, typename T3>
void serialize(Archive& ar, UseDefTemplateDataType<T1, T2, T3>& data)
{
    ar(data.data_vec_, data.data_map_);
}

// Register UseDefTemplateDataType<int, float>.
HIAI_REGISTER_TEMPLATE_DATA_TYPE("UseDefTemplateDataType_uint64_t_float_string",
    UseDefTemplateDataType, uint64_t, float, std::string);

//... Register other types.

// Register UseDefTemplateDataType<int, int>.
HIAI_REGISTER_TEMPLATE_DATA_TYPE("UseDefTemplateDataType_uint64_t_uint64_t_uint64_t",
    UseDefTemplateDataType, uint64_t, uint64_t, uint64_t);
// Serialization and deserialization of registered structs
typedef struct tagST_SC_IMAGE
{
    INT32 iWidth;           // image width
    INT32 iHeight;          // image height
    INT32 iWidthStep;       // Size of aligned image row in bytes
    INT32 iChannel;          // channels
}
```

```
    INT32 iDepth;          // depth in bits
    UINT32 uiTimeStampH;
    UINT32 uiTimeStampL;
    UINT32 iSize;
    UINT64 uiID;
    EN_SC_IMAGE_TYPE eType;    // image type
    ST_SC_IMAGE_ROI *roi;      // Image ROI. If NULL, the whole image is selected
    UINT8* puclImageData;      // Image data
    UINT64 uiPhyAddr;
}ST_SC_IMAGE;

std::shared_ptr<ST_SC_IMAGE> st_image = std::make_shared<ST_SC_IMAGE>();
st_image->iWidth = 1080;
st_image->iHeight = 1080;
st_image->iChannel = 1;
st_image->iWidthStep = 1080;
st_image->iSize = 1080*1080*3/2 ;
st_image->eType = SC_IMAGE_YUV420SP;
st_image->roi = (ST_SC_IMAGE_ROI*)malloc(sizeof(ST_SC_IMAGE_ROI));
st_image->puclImageData = nullptr;
uint8_t* align_buffer = nullptr;
HIAI_StatusT get_ret = HIAIMemory::HIAI_DMAlloc(st_image->iSize,(void*)&align_buffer,10000);
hiai::EnginePortID engine_id;
engine_id.graph_id = 1;
engine_id.engine_id = 1;
engine_id.port_id = 0;
HIAI_SendData(engine_id, "ST_SC_IMAGE", std::static_pointer_cast<void>(st_image),100000);

void GetStScImageSearPtr(void* inputPtr, std::string& ctrlStr, uint8_t*& dataPtr, uint32_t& dataLen)
{
    // If the struct body contains multiple pointers in addition to image, the pointers are concatenated.
    ST_SC_IMAGE* st_image = (ST_SC_IMAGE*)inputPtr;

    ctrlStr = std::string((char*)inputPtr, sizeof(ST_SC_IMAGE));
    if(nullptr != st_image->roi)
    {
        std::string image_roi_str = std::string((char*)st_image->roi, sizeof(ST_SC_IMAGE_ROI));
        ctrlStr += image_roi_str;
    }

    dataPtr = (UINT8*)st_image->puclImageData;
    dataLen = st_image->iSize;
}

std::shared_ptr<void> GetStScImageDearPtr(const char* ctrlPtr, const uint32_t& ctrlLen, const uint8_t*
dataPtr, const uint32_t& dataLen)
{
    ST_SC_IMAGE* st_sc_image = (ST_SC_IMAGE*)ctrlPtr;
    std::shared_ptr<hiai::BatchImagePara<uint8_t>> ImageInfo(new haii::BatchImagePara<uint8_t>);
    haii::ImageData<uint8_t> image_data;
    image_data.width = st_sc_image->iWidth;
    image_data.height = st_sc_image->iHeight;
    image_data.channel = st_sc_image->iChannel;
    image_data.width_step = st_sc_image->iWidthStep;
    if (st_sc_image->eType == SC_IMAGE_U8C3PLANAR)
    {
        image_data.size = st_sc_image->iWidth * st_sc_image->iHeight * 3;
        image_data.format = haii::BGR888;
    }
    else if (SC_IMAGE_U8C3PACKAGE == st_sc_image->eType)
    {
        image_data.size = st_sc_image->iWidth * st_sc_image->iHeight * 3;
        image_data.format = haii::BGR888;
    }
    else if (st_sc_image->eType == SC_IMAGE_YUV420SP)
    {
        image_data.size = st_sc_image->iSize;//st_sc_image->iWidth * st_sc_image->iHeight * 3 / 2;
        image_data.format = haii::YUV420SP;
    }
}
```

```
}  
image_data.data.reset(dataPtr, hiai::Graph::ReleaseDataBuffer);  
ImageInfo->v_img.push_back(image_data);  
ImageInfo->b_info.frame_ID.push_back(0);  
ImageInfo->b_info.batch_size = ImageInfo->b_info.frame_ID.size();  
  
return std::static_pointer_cast<void>(ImageInfo);  
}  
  
HIAI_REGISTER_SERIALIZE_FUNC("ST_SC_IMAGE", ST_SC_IMAGE, GetStScImageSearPtr,  
GetStScImageDearPtr);
```

8.2.5.3 Memory Management (C Language)

Precautions for API Usage

When memory is allocated by using **HIAI_DMAlloc**, pay attention to the following issues about memory management:

- When memory to be automatically freed is allocated for host-device or device-host data sending, if a smart pointer is used, Matrix automatically frees the memory, so the destructor specified by the smart pointer must be null. If the pointer is not a smart pointer, Matrix automatically frees the memory.
- When memory to be manually freed is allocated for host-device or device-host data sending, if a smart pointer is used, you need to set the destructor to **HIAI_DFree**. If the pointer is not a smart pointer, **HIAI_DFree** must be called to free the memory after data sending is complete.
- When memory to be manually freed is allocated, the **SendData** API cannot be called repeatedly to send data in the memory.
- When memory to be manually freed is allocated for host-device or device-host data sending, do not reuse the data in the memory before the memory is freed. If the memory is used for host-host or device-device data sending, the data in the memory can be reused before the memory is freed.
- When memory to be manually freed is allocated, if the **SendData** API is called to asynchronously send data, data in the memory cannot be modified after being sent.

If the **HIAI_DVPP_MAlloc** API is called to allocate memory for device-host data sending, the **HIAI_DVPP_DFree** API must be called to manually free the memory, because the **HIAI_DVPP_MAlloc** API does not automatically free the memory. If a smart pointer is used to store the allocated memory address, the destructor must be set to **HIAI_DVPP_DFree**.

8.2.5.3.1 HIAI_DMAlloc

Allocates memory blocks on the host side or device side to support efficient data transmission. This API is defined in **c_graph.h**.

To allocate memory for DVPP on the device side, see [8.2.5.3.3 HIAI_DVPP_MAlloc](#).

Application scenario:

With Matrix, this API is used to send large data from the host side to the device side. It must be used together with [8.2.5.2.3 Macro: HIAI_REGISTER_SERIALIZE_FUNC](#).

For example, a 1080p or 4K image needs to be sent to the device side. To improve the data sending performance, you must register the structure serialization and deserialization functions through **HIAI_REGISTER_SERIALIZE_FUNC**. In addition, the large data block memory is allocated through **HIAI_DMAlloc**. This greatly improves the data sending performance.

NOTE

This API is used to send large data between the host side and the device side. If there is no performance requirement or no data sending is involved, this API is not recommended.

Syntax

```
void* HIAI_DMAlloc (const uint32_t dataSize, const uint32_t timeOut, uint32_t flag)
```

Parameter Description

Parameter	Description	Value Range
dataSize	Size of a memory block	• If this API is called to allocate memory on the host side, the value range is 0–256 MB, excluding 0. The recommended range is 256 KB–256 MB. • If this API is called to allocate memory on the device side, the value range is 0 to (256 MB – 96 bytes), excluding 0. The recommended range is (256 KB – 96 bytes) to (256 MB – 96 bytes). The model manager on the device side occupies 96 bytes.
timeOut	Timeout when memory fails to be allocated. The default value is 500 ms.	-

Parameter	Description	Value Range
flag	- If flag is set to MEMORY_ATTR_AUTO_FREE (default value), it indicates that if the memory is allocated and data is sent to the peer end using SendData, the HIAI_DFree API does not need to be called. After the programming is complete, the memory is automatically freed. If the memory is allocated but the data is not sent to the peer end using SendData, the HIAI_DFree API needs to be called to free the memory. - If flag is set to MEMORY_ATTR_MANUAL_FREE, the HIAI_DFree API must be called to free the memory in any case. MEMORY_ATTR_NONE and MEMORY_ATTR_MAX are not used currently.	<pre>typedef enum { MEMORY_ATTR_NONE = 0, // The framework // automatically frees the // memory allocated by // using DMalloc. MEMORY_ATTR_AUTO_FREE = (0x1 << 1), // DFree needs to be // called manually to free // the memory. MEMORY_ATTR_MANUAL_FREE = (0x1 << 2), MEMORY_ATTR_MAX } HIAI_MEMORY_ATTR;</pre>

Return Value

Address of the memory allocated by using **HIAI_DMalloc**. If the memory fails to be allocated, a null pointer is returned.

8.2.5.3.2 HIAI_DFree

Releases the memory that is allocated through the HiAI interface. This API is defined in **c_graph.h**.

Application scenario:

When memory is allocated using [8.2.5.3.1 HIAI_DMalloc](#) but the **SendData** interface is not called to send data, the memory must be released using the **HIAI_DFree** interface, unless the **SendData** interface has been called.

Syntax

HIAI_StatusT HIAI_DFree (void* dataBuffer)

Parameter Description

Parameter	Description	Value Range
dataBuffer	Pointer to the memory to be released	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.
4	HIAI_GRAPH_NOT_FIND_MEMORY	The memory cannot be found.

8.2.5.3.3 HIAI_DVPP_DMalloc

Allocates memory for DVPP on the device side. This API is defined in **c_graph.h**. After this API is called to allocate memory, **HIAI_DVPP_DFree** must be called to free the memory.

Syntax

void* HIAI_DVPP_DMalloc(const uint32_t dataSize)

Parameter Description

Parameter	Description	Value Range
dataSize	Size of a memory block	0 to (256 MB – 96 bytes), excluding 0. The recommended range is (256 KB – 96 bytes) to (256 MB – 96 bytes). The model manager on the device side occupies 96 bytes.

Return Value

Address of the memory allocated by using **HIAI_DVPP_DMAlloc**. If the memory fails to be allocated, a null pointer is returned.

Remarks

HIAI_DVPP_DMAlloc is a new API for allocating memory to DVPP. The memory allocated by using this API can meet the various memory requirements of DVPP.

If **HIAI_DMAlloc(1000, buffer, 1000, HIAI_MEMORY_HUGE_PAGE)** is called to allocate memory for DVPP in the earlier version, use **HIAI_DVPP_DMAlloc** instead. For details, see [8.2.5.3.1 HIAI_DMAlloc](#).

Usage Example:

```
// Allocate memory by using HIAI_DVPP_DMAlloc.
uint32_t allocSize = 1000;
char* allocBuffer = (char*)HIAI_DVPP_DMAlloc(allocSize);
if (allocBuffer == nullptr) {
    // If the allocation fails, handle the exception.
}
```

8.2.5.3.4 HIAI_DVPP_DFree

Releases the memory allocated by the **HIAI_DVPP_DMAlloc** interface. This API is defined in **c_graph.h**.

Syntax

HIAI_StatusT HIAI_DVPP_DFree(void* dataBuffer)

Parameter Description

Parameter	Description	Value Range
dataBuffer	Address of the memory to be released	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.

No.	Error Code	Description
4	HIAI_GRAPH_NOT_FIND_MEMORY	The memory cannot be found.

8.2.5.4 Memory Management (C++ Language)

Precautions for API Usage

When memory is allocated by using **HIAIMemory::HIAI_DMAlloc**, pay attention to the following issues about memory management:

- When memory to be automatically freed is allocated for host-device or device-host data sending, if a smart pointer is used, Matrix automatically frees the memory, so the destructor specified by the smart pointer must be null. If the pointer is not a smart pointer, Matrix automatically frees the memory.
- When memory to be manually freed is allocated for host-device or device-host data sending, if a smart pointer is used, you need to set the destructor to **HIAIMemory::HIAI_DFree**. If the pointer is not a smart pointer, you need to call **HIAIMemory::HIAI_DFree** to free the memory after data sending is complete.
- When memory to be manually freed is allocated, the **SendData** API cannot be called repeatedly to send data in the memory.
- When memory to be manually freed is allocated for host-device or device-host data sending, do not reuse the data in the memory before the memory is freed. If the memory is used for host-host or device-device data sending, the data in the memory can be reused before the memory is freed.
- When memory to be manually freed is allocated, if the **SendData** API is called to asynchronously send data, data in the memory cannot be modified after being sent.

If the **HIAIMemory::HIAI_DVPP_DMAlloc** API is called to allocate memory for device-host data sending, the **HIAIMemory::HIAI_DVPP_DFree** API must be called to manually free the memory, because the **HIAIMemory::HIAI_DVPP_DMAlloc** API does not automatically free the memory. If a smart pointer is used to store the allocated memory address, the destructor must be set to **HIAIMemory::HIAI_DVPP_DFree**.

8.2.5.4.1 HIAIMemory::HIAI_DMAlloc

Allocates memory blocks on the host side or device side to support efficient data transmission. This API is defined in **ai_memory.h**.

To allocate memory for DVPP on the device side, see [8.2.5.4.4 HIAIMemory::HIAI_DVPP_DMAlloc](#).

Application scenario:

With Matrix, this API is used to send large data from the host side to the device side. It must be used together with [8.2.5.2.3 Macro: HIAI_REGISTER_SERIALIZE_FUNC](#).

For example, a 1080p or 4K image needs to be sent to the device side. To improve the data sending performance, you must register the structure serialization and deserialization functions through **HIAI_REGISTER_SERIALIZE_FUNC**. In addition, the large data block memory is allocated through **HIAI_DMalloc**. This greatly improves the data sending performance.

NOTE

- This API deals with sending of large data between the host side and device side. You are advised not to use it as a common **malloc** API.
- For the sake of performance, this API pre-allocates memory. The actual memory size is different from the allocated memory size. The running time of this API may also change.

Syntax

```
static HIAI_StatusT HIAIMemory::HIAI_DMalloc (const uint32_t dataSize,  
void*& dataBuffer, const uint32_t timeOut = MALLOC_DEFAULT_TIME_OUT,  
uint32_t flag = MEMORY_ATTR_AUTO_FREE)
```

Parameter Description

Parameter	Description	Value Range
dataSize	Size of a memory block	• If this API is called to allocate memory on the host side, the value range is 0–256 MB, excluding 0. The recommended range is 256 KB–256 MB. • If this API is called to allocate memory on the device side, the value range is 0 to (256 MB – 96 bytes), excluding 0. The recommended range is (256 KB – 96 bytes) to (256 MB – 96 bytes). The model manager on the device side occupies 96 bytes.
dataBuffer	Memory pointer	-

Parameter	Description	Value Range
timeOut	Timeout when memory fails to be allocated. The default value is MALLOC_DEFAULT_TIME_OUT (indicating 500 ms).	-
flag	- If flag is set to MEMORY_ATTR_AUTO_FREE (default value), it indicates that if the memory is allocated and data is sent to the peer end using SendData, the HIAIMemory::HIAI_DFree API does not need to be called. After the programming is complete, the memory is automatically freed. If the memory is allocated but the data is not sent to the peer end using SendData, the HIAIMemory::HIAI_DFree API needs to be called to free the memory. - If flag is set to MEMORY_ATTR_MANUAL_FREE, the HIAIMemory::HIAI_DFree API must be called to free the memory in any case. MEMORY_ATTR_NONE and MEMORY_ATTR_MAX are not used currently.	<pre>typedef enum { MEMORY_ATTR_NONE = 0, // The framework automatically frees the memory allocated by using DMalloc. MEMORY_ATTR_AUTO_FREE = (0x1 << 1), // DFree needs to be called manually to free the memory. MEMORY_ATTR_MANUAL_FREE = (0x1 << 2), MEMORY_ATTR_MAX } HIAI_MEMORY_ATTR;</pre>

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_MEMORY_POOL_NOT_EXISTED	The memory pool does not exist.
4	HIAI_GRAPH_MALLOC_LARGER	The buffer fails to be allocated because the size is larger than 256 MB.
5	HIAI_MEMORY_POOL_UPDATE_FAILED	The memory pool fails to be updated.
6	HIAI_GRAPH_SENDMSG_FAILED	The HDC module fails to send a message.
7	HIAI_GRAPH_MEMORY_POOL_INITED	The memory pool has been initialized.
8	HIAI_GRAPH_NO_MEMORY	There is no memory.

8.2.5.4.2 HIAIMemory::HIAI_DFree

Releases the memory that is allocated through the HiAI interface. This API is defined in **ai_memory.h**.

Application scenario:

When memory is allocated using [8.2.5.4.1 HIAIMemory::HIAI_DMalloc](#) but the **SendData** interface is not called to send data, the memory must be released using the **HIAI_DFree** interface, unless the **SendData** interface has been called.

Syntax

```
static HIAI_StatusT HIAIMemory::HIAI_DFree (void* dataBuffer)
```

Parameter Description

Parameter	Description	Value Range
dataBuffer	Pointer to the memory to be released	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.
4	HIAI_GRAPH_NOT_FIND_MEMORY	The memory cannot be found.

8.2.5.4.3 HIAIMemory::IsDMalloc

Checks whether the memory is allocated by **DMalloc**. This API is defined in **ai_memory.h**.

Syntax

```
static bool IsDMalloc(const void* dataBuffer, const uint32_t& dataSize)
```

Parameter Description

Parameter	Description	Value Range
dataBuffer	Pointer to the memory to be determined	-
dataSize	Size of the memory address, in bytes	-

Return Value

true: The memory is applied for by using **DMalloc**.

false: The memory is not applied for by using **DMalloc**.

8.2.5.4.4 HIAIMemory::HIAI_DVPP_DMalloc

Allocates memory for DVPP on the device side. This API is defined in **ai_memory.h**. After this API is called to allocate memory, **HIAIMemory::HIAI_DVPP_DFree** must be called to free the memory.

Syntax

```
HIAI_StatusT HIAI_DVPP_DMalloc(const uint32_t dataSize, void*& dataBuffer)
```

Parameter Description

Parameter	Description	Value Range
dataSize	Size of a memory block	0 to (256 MB – 96 bytes), excluding 0. The recommended range is (256 KB – 96 bytes) to (256 MB – 96 bytes). The model manager on the device side occupies 96 bytes.
dataBuffer	Allocated memory address. If the allocation fails, a null pointer is returned.	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_MEMORY_POOL_NOT_EXISTED	The memory pool does not exist.
4	HIAI_GRAPH_MALLOC_LARGER	The buffer fails to be allocated because the size is larger than 256 MB.
5	HIAI_MEMORY_POOL_UPDATE_FAILED	The memory pool fails to be updated.
6	HIAI_GRAPH_SENDMSG_FAILED	The HDC module fails to send a message.
7	HIAI_GRAPH_MEMORY_POOL_INITED	The memory pool has been initialized.
8	HIAI_GRAPH_NO_MEMORY	There is no memory.

Remarks

HIAIMemory::HIAI_DVPP_DMAlloc is a new API for allocating memory to DVPP. The memory allocated by using this API can meet the various memory requirements of DVPP.

If **HIAIMemory::HIAI_DMAlloc(1000, buffer, 1000, HIAI_MEMORY_HUGE_PAGE)** is called to allocate memory for DVPP in the earlier version, use **HIAIMemory::HIAI_DVPP_DMAlloc** instead. For details, see [8.2.5.4.1 HIAIMemory::HIAI_DMAlloc](#).

Usage Example:

```
// Allocate memory by using HIAI_DVPP_DMAlloc.
uint32_t allocSize = 1000;
char* allocBuffer = nullptr;
HIAI_StatusT ret = hiai::HIAIMemory::HIAI_DVPP_DMAlloc(allocSize, (void*)&allocBuffer);
if (ret != HIAI_OK || allocBuffer == nullptr) {
    // If the allocation fails, handle the exception.
}
```

8.2.5.4.5 HIAIMemory::HIAI_DVPP_DFree

Releases the memory allocated by the **HIAIMemory::HIAI_DVPP_DMAlloc** interface. This API is defined in **ai_memory.h**.

Syntax

HIAI_StatusT HIAI_DVPP_DFree(void* dataBuffer)

Parameter Description

Parameter	Description	Value Range
dataBuffer	Address of the memory to be released	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_GRAPH_NOT_EXIST	The graph does not exist.
3	HIAI_GRAPH_INVALID_VALUE	The graph has an invalid value.
4	HIAI_GRAPH_NOT_FIND_MEMORY	The memory cannot be found.

8.2.5.5 Logs (C++)

8.2.5.5.1 Error Code Registration

?1. HIAI_DEF_ERROR_CODE

Registers error codes. This macro is defined in **status.h**.

This macro encapsulates the following functions:

```
static StatusFactory* StatusFactory::StatusFactory::GetInstance();  
void RegisterErrorNo(const uint32_t err, const std::string& desc);
```

Syntax

HIAI_DEF_ERROR_CODE(moduleId, logLevel, codeName, codeDesc)

Parameter Description

Parameter	Description	Value Range
moduleId	module ID	-
logLevel	Error level	-
codeName	Error code name	-
codeDesc	Description of an error code, which is a character string	-

Return Value

None

?2. API Usage Example

```
/*  
*****  
*          Copyright (C) HiSilicon Co., Ltd.  
*  
*          Filename: user_def_errorcode.h  
*          Description: User defined Error Code  
*  
*          Version: 1.0  
*          Created: 2018-01-08 10:15:18  
*          Author:  
*  
*          Revision: initial draft;  
*****  
*/  
#ifndef USE_DEF_ERROR_CODE_H_  
#define USE_DEF_ERROR_CODE_H_  
  
#include "hiaiengine/status.h"  
#define USE_DEFINE_ERROR 0x6001 // ID of the custom module. The value 0x6001 is for reference only.  
You can set the value as required. The value cannot be the same as MODID_GRAPH in the status.h file.  
enum  
{  
    HIAI_INVALID_INPUT_MSG_CODE = 0 // error code name. The value 0 is for reference only. You can set
```

```
the value as required.
};
HIAI_DEF_ERROR_CODE(USE_DEFINE_ERROR, HIAI_ERROR, HIAI_INVALID_INPUT_MSG, "invalid input
message pointer");
#endif
```

8.2.5.5.2 Log Printing

Use HIAI_ENGINE_LOG to invoke different HIAI_ENGINE_LOG_IMPL functions based on the input parameters. When the formatting function is invoked, the type and number of parameters in the format must be the same as the actual parameter type and number. The APIs associated with log printing are defined in **log.h**.

After **HIAI_ENGINE_LOG** is called, the system records logs in the **/var** directory on the host side or developer board. Device logs are recorded in log files whose names start with **device-id**. Host logs are recorded in log files whose names start with **host-0**.

?1. Log Printing Format 1

Syntax

```
#define HIAI_ENGINE_LOG(...) \
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##_VA_ARGS_)

void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int
lineNumber,
hiaii::Graph* graph, const uint32_t errorCode, const char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
graph	Pointer to the graph object	-
errorCode	Error code	-
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
auto graph = Graph::Getinstance(1);
HIAI_ENGINE_LOG (graph.get(), HIAI_OK, "RUNNING OK");
```


?2. Log Printing Format 2

Syntax

```
#define HIAI_ENGINE_LOG(...) \  
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##_VA_ARGS_)  
  
void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int  
lineNumber,  
hiai::Engine* engine, const uint32_t errorCode, const char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
engine	Pointer to the engine object	-
errorCode	Error code	-
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
// This API is called in HIAI_IMPL_ENGINE_PROCESS.  
HIAI_ENGINE_LOG(this, HIAI_OK, "RUNNING OK");
```

?3. Log Printing Format 3

Syntax

```
#define HIAI_ENGINE_LOG(...) \  
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##_VA_ARGS_)  
  
void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int  
lineNumber,  
const uint32_t errorCode, const char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
errorCode	Error code	-
format	Log description	-

Parameter	Description	Value Range
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
HIAI_ENGINE_LOG(HIAI_OK, "RUNNING OK");
```

4. Log Printing Format 4

Syntax

```
#define HIAI_ENGINE_LOG(...) \
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##_VA_ARGS_)

void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int
lineNumber,
const char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
HIAI_ENGINE_LOG ("RUNNING OK");
```

5. Log Printing Format 5

Syntax

```
#define HIAI_ENGINE_LOG(...) \
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##_VA_ARGS_)

void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int
lineNumber,
const int32_t moudleID, hiai::Graph* graph, const uint32_t errorCode, const
char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
moudleID	Enumeration ID of the module name	-
graph	Pointer to the graph object	-
errorCode	Error code	-
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
auto graph = Graph::GetInstance(1);  
HIAI_ENGINE_LOG(MODID_OTHER, graph.get(), HIAI_OK, "RUNNING OK");
```

6. Log Printing Format 6

Syntax

```
#define HIAI_ENGINE_LOG(...) \  
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##__VA_ARGS__)  
  
void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int  
lineNumber,  
const int32_t moudleID, hiai::Engine* engine, const uint32_t errorCode, const  
char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
moudleID	Enumeration ID of the module name	-
engine	Pointer to the engine object	-
errorCode	Error code	-
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
// This API is called in HIAI_IMPL_ENGINE_PROCESS.  
HIAI_ENGINE_LOG(MODID_OTHER, this, HIAI_OK, "RUNNING OK");
```

?7. Log Printing Format 7

Syntax

```
#define HIAI_ENGINE_LOG(...) \  
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##__VA_ARGS__)  
  
void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int  
lineNumber,  
const int32_t moudleID, const char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
moudleID	Enumeration ID of the module name	-
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
HIAI_ENGINE_LOG(MODID_OTHER, "RUNNING OK");
```

?8. Log Printing Format 8

Syntax

```
#define HIAI_ENGINE_LOG(...) \  
HIAI_ENGINE_LOG_IMPL(__FUNCTION__, __FILE__, __LINE__, ##__VA_ARGS__)  
  
void HIAI_ENGINE_LOG_IMPL(const char* funcPointer, const char* filePath, int  
lineNumber,  
const int32_t moudleID, const uint32_t errorCode, const char* format, ...);
```

Parameter Description

Parameter	Description	Value Range
moudleID	Enumeration ID of the module name	-
errorCode	Error code	-
format	Log description	-
...	Variable parameter in format , which is added based on the log content	-

Calling Example

```
HIAI_ENGINE_LOG(MODID_OTHER, HIAI_OK, "RUNNING OK");
```

8.2.5.6 MultiTypeQueue API for Queue Management (C++ Language)

Supports storing multiple types of message queues. The queue management APIs are defined in **multitype_queue.h**.

8.2.5.6.1 MultiTypeQueue Constructor

MultiTypeQueue constructor

Syntax

```
MultiTypeQueue(uint32_t queNum, uint32_t maxQueLen = 1024, uint32_t  
durationMs = 0);
```

Parameter Description

Parameter	Description	Value Range
queNum	Number of queues	-
maxQueLen	Maximum length of each queue	-

Parameter	Description	Value Range
durationMs	Maximum interval between the time when a queue member enters a queue and the time when the member leaves the queue. If a member does not enter the queue (Pop) within duration_ms after entering the queue (Push), the member is automatically deleted.	-

Return Value

None

8.2.5.6.2 PushData

Inserts data into a specified queue.

Syntax

```
bool MultiTypeQueue::PushData(uint32_t qIndex, const  
std::shared_ptr<void>& dataPtr);
```

Parameter Description

Parameter	Description	Value Range
qIndex	Queue ID	0 to queNum - 1
dataPtr	Pointer to the queue function	-

Return Value

If the operation is successful, **true** is returned. Otherwise, **false** is returned (for example, when the queue is full).

8.2.5.6.3 FrontData

Reads the header data of a specified queue (not deleting it from the queue).

Syntax

```
bool MultiTypeQueue::FrontData(uint32_t qIndex, std::shared_ptr<void>&  
dataPtr);
```

Parameter Description

Parameter	Description	Value Range
qIndex	Queue ID	0 to queNum – 1
dataPtr	Pointer to the queue function	-

Return Value

If the operation is successful, **true** is returned. Otherwise, **false** is returned (for example, when the queue is empty).

8.2.5.6.4 PopData

Reads the header data of a specified queue (deleting it from the queue).

Syntax

```
bool MultiTypeQueue::PopData(uint32_t qIndex, std::shared_ptr<void>& dataPtr);
```

Parameter Description

Parameter	Description	Value Range
qIndex	Queue ID	0 to que_num – 1
dataPtr	Pointer to the queue function	-

Return Value

If the operation is successful, **true** is returned. Otherwise, **false** is returned (for example, when the queue is empty).

8.2.5.6.5 PopAllData

Reads the header data of all queues. The read operation is successful only when all queue headers have data, and the header data of the queues are deleted. Otherwise, a failure message is returned, but no data is deleted.

Syntax

```
template<typename T1>
bool MultiTypeQueue::PopAllData(std::shared_ptr<T1>& arg1);

template<typename T1, typename T2>
```

```
bool MultiTypeQueue::PopAllData(std::shared_ptr<T1>& arg1,
std::shared_ptr<T2>& arg2);

.....

template<typename T1, typename T2, typename T3, typename T4, typename
T5, typename T6, typename T7, typename T8, typename T9, typename T10,
typename T11, typename T12, typename T13, typename T14, typename T15,
typename T16>

bool MultiTypeQueue::PopAllData(std::shared_ptr<T1>& arg1,
std::shared_ptr<T2>& arg2, std::shared_ptr<T3>& arg3,
std::shared_ptr<T4>& arg4, std::shared_ptr<T5>& arg5,
std::shared_ptr<T6>& arg6, std::shared_ptr<T7>& arg7,
std::shared_ptr<T8>& arg8, std::shared_ptr<T9>& arg9,
std::shared_ptr<T16>& arg16)
```

Parameter Description

Parameter	Description	Value Range
arg1	Pointer to the queue function	-
...	Pointer to the queue function	-
arg16	Pointer to the queue function	-

Return Value

The read operation is successful and **true** is returned only when all queue headers have data. Otherwise, **false** is returned.

8.2.5.6.6 API Usage Example

For details, see [8.2.3.7 Calling Example](#).

8.2.5.7 Event Registration API (C++ Language)

8.2.5.7.1 Graph::RegisterEventHandle

Subscribes to interested events. Currently, the host-device disconnection event is supported. When disconnection occurs, the user can receive this event and handle the disconnection logic (for example, waiting for stopping the main program) in the registered callback function. This API is defined in **graph.h**.

Syntax

```
HIAI_StatusT Graph::RegisterEventHandle(const HIAIEvent& event, const  
std::function<HIAI_StatusT(void)>&callback)
```

Parameter Description

Parameter	Description	Value Range
event	Subscribed event	typedef enum { HIAI_DEVICE_DISCONNECT_EVENT // Disconnection message }HIAIEvent;
callback	Event callback function	Callback function for user subscription

Return Value

If the operation is successful, **HIAI_OK** is returned. Otherwise, a failure message is returned.

8.2.5.8 Others (C++ Language)

8.2.5.8.1 DataRecvInterface::RecvData

Defines a subclass of class **DataRecvInterface** to initialize the object of the subclass. It works as the callback function for receiving data and transfers the API in [8.2.2.4.1 Graph::SetDataRecvFunctor](#) or [8.2.3.5 Engine::SetDataRecvFunctor](#). In addition, the **RecvData** API needs to be overloaded and implemented in the subclass. This API is defined in **data_recv_interface.h**.

Syntax

```
virtual HIAI_StatusT DataRecvInterface::RecvData(const  
std::shared_ptr<void>& message)
```

Parameter Description

Parameter	Description	Value Range
message	Callback message	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.

8.2.5.8.2 Graph::SetPublicKeyForSignatureEncryption

Set a public key which is used to verify the digital signature of the *.so file to be loaded by the Matrix on the device side. This API must be called before the creation of a graph. This API is defined in **graph.h**.

The **SHA256withRSA** algorithm is used for digital signature. This component provides only the digital signature verification function. You need to sign the required SO file based on this algorithm. The signature file must be stored in the same directory as the SO file on the host side. The signature file name is "SO file name+signature". For example, if the SO file name is **libhosttodevice.so**, the signature file name is **libhosttodevice.so.signature**.

CAUTION

If **Graph::SetPublicKeyForSignatureEncryption** is not called or its return value is not **HIAI_OK**, the public key cannot be set in the Matrix, and the signature verification mechanism will not be enabled on the device side. As a result, the *.so file that may be tampered cannot be identified by using the digital signature.

Syntax

```
static HIAI_StatusT Graph::SetPublicKeyForSignatureEncryption(const  
std::string& publicKey)
```

Parameter Description

Parameter	Description	Value Range
publicKey	Public key required for digital signature verification	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.

```
std::string publicKey =
"-----BEGIN RSA PUBLIC KEY-----\n"
"MIIBCAKCAQEAt+kdmu8CdVw2xHlh/JWXOL/0AitDgYGd+9RadULGZmj0tt/UQLv\n"
"EYPZaXC8E0a9e97kqfg/ZHinu04XG5RhXv2J0kwkhuBAeCCllefvrQy5OBRqdLYq\n"
"EZe8wUO0adSMDsLbPL52b+NdO9/zX+MUzvsBzitWJbiH96s4xCiEERX87/uQfr6F\n"
"lHLtrNtooeF2VmxOm3n0yzh4kcSouLgb/O0+v0I+fnR+CTNG95lvzDNOYLWD6ZkL\n"
"c7JyIYFZA0CNx9SlPhqbrfjOV5XSG3g3CW0TopUDfHyhAgZt5vACMpeDDx+89tg2\n"
"RfT4M9DH/qKzkLIOURvsMShRMD6/PwzsPwIBAw==\n"
"-----END RSA PUBLIC KEY-----";
auto ret = Graph::SetPublicKeyForSignatureEncryption(publicKey);
```

8.2.6 Python APIs

You are not advised to use the following APIs. If you need to use them, contact Huawei technical support.

Table 8-2 Basic process connection APIs

Class	API
Class Graph	Graph.__init__
	Graph.__del__
	Graph.get_id
	Graph.set_device_id
	Graph.get_graph
	Graph.destroy
	Graph.destroy_graph
	Graph.as_default
	Graph.create_graph
Class Engine	Engine.__init__
	Engine.update_config
	Engine.as_default
Class EngineConfig	EngineConfig.__init__
	EngineConfig.engine_id
	EngineConfig.engine_name
	EngineConfig.so_names

Class	API
	EngineConfig.side
	EngineConfig.thread_num
	EngineConfig.thread_priority
	EngineConfig.queue_size
	EngineConfig.ai_config
	EngineConfig.internel_so_names
	EngineConfig.wait_inputdata_max_time
	EngineConfig.hold_model_file_flag
Class GraphConfig	GraphConfig.__init__
	GraphConfig.graph_id
	GraphConfig.priority
	GraphConfig.device_id
Function APIs	get_default_graph
	get_default_engine
	crop
	resize
	inference

Table 8-3 Basic data types

Class	API
Class NNTensor	NNTensor.__init__
	NNTensor.height
	NNTensor.width
	NNTensor.channel
	NNTensor.batch_size
Class NNTensorList	NNTensorList.__init__
	NNTensorList.__getitem__
	NNTensorList.get_tensor_num
	NNTensorList.next

Class	API
	NNTensorList.__iter__

Table 8-4 Data pre-processing

Class	API
Class Engine	Engine.crop
	Engine.resize
	Engine.padding
Class CropConfig	CropConfig.__init__
Class ResizeConfig	ResizeConfig.__init__

Table 8-5 Offline model inference

Class	API
Class Engine	Engine.inference
Class AIModelDescription	AIModelDescription.__init__
	AIModelDescription.name
	AIModelDescription.model_type
	AIModelDescription.version
	AIModelDescription.size
	AIModelDescription.path
Class AIConfig	AIConfig.__init__
Class AIConfigItem	AIConfigItem.__init__

Table 8-6 Data TX/RX

Class	API
Class Graph	Graph.proc

8.2.7 Appendix

8.2.7.1 APIs to Be Deleted in Later Versions

The APIs to be deleted in later versions are not called by users. The subsequent compatibility cannot be ensured after the APIs are called.

8.2.7.1.1 Low-Power APIs (C++ Language)

?1. PowerState::SubscribePowerStateEvent

Registers the low-power callback function. This API is defined in **power_state.h**.

Syntax

```
HIAI_StatusT SubscribePowerStateEvent(PowerStateNotifyCallbackT  
notifyCallback)
```

Parameter Description

Parameter	Description	Value Range
notifyCallback	Callback function that the user expects to register	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_POWER_STATE_CALLBACK_ERROR	A null pointer is returned when the app callback function is called.

?2. PowerState::UnsubscribePowerStateEvent

Deregisters the previously registered low-power callback function. This API is defined in **power_state.h**.

Syntax

```
HIAI_StatusT UnsubscribePowerStateEvent(PowerStateNotifyCallbackT  
notifyCallback)
```

Parameter Description

Parameter	Description	Value Range
notifyCallback	Callback function that the user expects to deregister	-

Return Value

For details about the returned error codes, see "Error Codes."

Error Codes

No.	Error Code	Description
1	HIAI_OK	The running is OK.
2	HIAI_POWER_STATE_CALLBACK_ERROR	A null pointer is returned when the app callback function is called.

8.2.7.1.2 Data Types

?1. Enumeration: PowerState::DEVICE_POWER_STATE

```
enum DEVICE_POWER_STATE {  
    DEVICE_POWER_STATE_S0 = 0,    // The system enters the S0 state, the device is available, and the app  
    is woken up.  
    DEVICE_POWER_STATE_SUSPEND,    // The system is about to enter the S3/S4 state, and the app  
    releases resources such as channels and memory.  
    DEVICE_POWER_STATE_D0,    // The NPU device enters the D0 state, the device is available, and the  
    app is woken up. (No processing is required for the Matrix services.)  
    DEVICE_POWER_STATE_D3,    // The NPU device is about to enter the D3 state, and the app stops  
    communicating with the device. (No processing is required for the Matrix services.)  
    DEVICE_POWER_STATE_ENABLE,    // The NPU device driver is installed or enabled, and the NPU is  
    available.  
    DEVICE_POWER_STATE_DISABLE,    // The NPU device driver is uninstalled or disabled, and the NPU is  
    unavailable.  
    DEVICE_POWER_STATE_MAX    // Indicates the maximum number of states. It does not indicate the  
    actual state.  
};
```

?2. PowerState::PowerStateNotifyCallbackT

```
typedef HIAI_StatusT (*PowerStateNotifyCallbackT)(uint32_t, DEVICE_POWER_STATE);
```

8.2.7.2 Matrix Data Types

8.2.7.2.1 Protobuf Data Types

For details about how to use Protobuf, see <https://developers.google.com/protocol-buffers/docs/reference/proto3-spec>.

The Protobuf data types used by Matrix are as follows:

ai_types.proto

```
syntax = "proto3";
package hiai;

// Tensor parameter description
message AITensorParaDescription
{
    string name = 1; // Parameter name
    string type = 2; // Parameter type
    string value = 3; // Parameter value
    string desc = 4; // Parameter description
    repeated AITensorParaDescription sub paras = 5; // Sub-parameter list
};

// Tensor description
message AITensorDescription
{
    string name = 1; // Tensor name
    string type = 2; // Tensor type
    repeated string compatible_type = 3; // Specifies all types of parent classes that are compatible.
    repeated AITensorParaDescription paras = 4; // Parameter list
};

// Tensor description list
message AITensorDescriptionList
{
    repeated AITensorDescription tensor_descs = 1; // Tensor list
};

// Common configuration items
// If sub_items exists, the current node is the parent node and the value is invalid.
message AIConfigItem
{
    string name = 1; // Configuration item name
    string value = 2; // Configuration item value. When this parameter is set to the path of a model file, it
    contains the file name, supporting only digits, letters, underscores (_), and dots (.). You can set this
    parameter to the path of a single model file, for example, ./test_data/model/resnet18.om. You can also
    compress the model files into a tar package and set the parameter to the path of the tar package, for
    example, ./test_data/model/resnet18.tar. If there are multiple AIConfigItem, the files with the same name
    but in different formats cannot be in the same directory when you set the path of the tar package, for
    example, ./test_data/model/test and ./test_data/model/test.tar. The APIs for creating graphs include
    Graph::CreateGraph or HIAI_CreateGraph.
    repeated AIConfigItem sub_items = 3; // Configuration subitem
};

// Configuration parameter for running an NN node or API
message AIConfig
{
    repeated AIConfigItem items = 1; // Configuration item list
};

// Model description
message AIModelDescription
{
    string name = 1; // Model name
    int32 type = 2; // Model type. Currently, only the DAVINCI_OFFLINE_MODEL type is supported. The value
    is 0.
    string version = 3; // Model version
    int32 size = 4; // Model size
    string path = 5; // Model path
    string key = 8; // Model key
    bytes data = 100; // Model data
    repeated AITensorDescription inputs = 6; // Description of input tensors
    repeated AITensorDescription outputs = 7; // Description of output tensors
};
```



```
// NNNode description
message AINNNodeDescription
{
    string name = 1; // NN node name
    string desc = 2; // NN node description
    bool isPreAllocateOutputMem = 3; // Whether to pre-allocate the output memory
    AIConfig config = 4; // Configuration parameter
    repeated AIModelDescription model_list = 5; // List of models required by an NN node
    repeated AITensorDescription inputs = 6; // Description of input tensors
    repeated AITensorDescription outputs = 7; // Description of output tensors
    bool need_verify = 8; // Whether to verify the tensor match during connection
    repeated string ignored_check_aitensor = 9; // Specifies the list of tensors for which the mechanism for
    verifying whether they match the tensors of the inputs is ignored during connection.
};

// NNNode description list
message AINNNodeDescriptionList
{
    repeated AINNNodeDescription nnnode_descs = 1; // NN node list
}

// API description
message AIAPIDescription
{
    string name = 1; // API name
    string desc = 2; // API description
    bool isPreAllocateOutputMem = 3; // Whether to pre-allocate the output memory
    AIConfig config = 4; // Configuration parameter
    repeated AITensorDescription inputs = 5; // Description of input tensors
    repeated AITensorDescription outputs = 6; // Description of output tensors
    bool need_verify = 7; // Whether to verify the tensor match during connection
    repeated string ignored_check_aitensor = 8; // Specifies the list of tensors for which the mechanism for
    verifying whether they match the tensors of the inputs is ignored during connection.
};

// API description list
message AIAPIDescriptionList
{
    repeated AIAPIDescription api_descs = 1; // API list
}

// AI operation description
message AIOPDescription
{
    enum OP_Type
    {
        AI_API = 0;
        AI_NNNODE = 1;
    }

    OP_Type type = 1;
    AINNNodeDescription nnnode_desc = 2;
    AIAPIDescription api_desc = 3;
};

// AI operation description list
message AIOPDescriptionList
{
    repeated AIOPDescription op_descs = 1; // AI operation list
}

// IDE passes parameters to Matrix APIs.

message NodeDesc
{
    string name=1; // IAINNNode or ALG_API name
    AIConfig config=2; // Initialization parameter required by IAINNNode or ALG_API
    repeated AIModelDescription mode_desc=3; // Initialization parameter required by IAINNNode
}
```

```
message EngineDesc
{
    enum RunSide
    {
        DEVICE=0;
        HOST=1;
    }
    enum EngineType
    {
        NORMAL=0;
        SOURCE=1;
        DEST=2;
    }
    uint32 id =1; // Engine ID (node)
    EngineType type=2; //
    string name =3; // Engine node name
    repeated string so_name=4; // The list of all DLL .so file names that need to be copied is loaded first
    according to the configuration sequence. If the loading fails, the system tries to load the list of files
    according to another loading sequence.
    RunSide side=5; // Run on the host side or device side.
    int32 priority=6; // Node priority
    uint32 instance_cnt=7; // Number of instances (equivalent to the number of threads)
    repeated uint32 next_node_id=8; // Next node list
    bool user_input_cb=9; // IDE can be ignored.
    bool user_output_cb=10; // IDE can be ignored.
    repeated NodeDesc oper=11; // Matrix node list
}

message GraphInitDesc
{
    int32 priority=1; // Priority of the entire process for a graph
    //Runside side = 2; // Assume that the graph is deployed on the host and cannot be configured.
}

message GeneralFileBuffer
{
    bytes raw_data = 1;
    string file_name = 2;
}
```

graph_config.proto

```
syntax = "proto3";
import "ai_types.proto";
package hiai;

message DumpDef
{
    bytes model_name = 1; // Model name
    bytes is_dump_all = 2; // Whether to enable the dump function. This parameter is used to dump
    information about operators at all layers in the model.
    repeated bytes layer = 3; // Determines the layers on which the operators need to be dumped in the
    model.
}

message OAMConfigDef
{
    repeated DumpDef dump_list = 1;
    bytes dump_path = 2; // Path for storing dump files
}

message EngineConfig
{
    enum RunSide
    {
        DEVICE=0; // Run on the device side.
        HOST=1; // Run on the host side.
    }

    uint32 id =1; // Engine ID (node)
    string engine_name =2; // Engine node name
}
```

```
repeated string so_name=3; // The list of all DLL .so file names that need to be copied is loaded first
according to the configuration sequence. If the loading fails, the system tries to load the list of files
according to another loading sequence.
RunSide side=4; // Run on the host side or device side.
uint32 thread_num = 5; // Number of threads. If multi-channel decoding is used, you are advised to set
this parameter to 1. If the value of "thread_num" is greater than 1, the decoding sequence between threads
cannot be ensured.
uint32 thread_priority = 6; // Thread priority
uint32 queue_size = 7; // Queue size
AIConfig ai_config = 8; // Aiconfig configuration file
repeated AIModelDescription ai_model = 9; // AIModelDescription
repeated string internal_so_name=10; // List of all DLL .so files that do not need to be copied
uint32 wait_inputdata_max_time = 12; // Maximum timeout for waiting to receive the next data (in ms)
uint32 holdModelFileFlag = 13; // Whether to retain the model file of this engine. 0: Not reserved; Other
values: Reserved.
OAMConfigDef oam_config = 14; // OAM configuration
bool is_repeat_timeout_flag = 15; // Whether to perform timeout processing repeatedly when no data is
received by an engine. 0: no; 1: yes. For example, if there is a large amount of multi-channel data to be
analyzed, you are advised to set this parameter to 1. This parameter is used together with the
wait_inputdata_max_time parameter. If "is_repeat_timeout_flag" is set to 0, the wait_inputdata_max_time
parameter is used to set the timeout duration for a single time. If "is_repeat_timeout_flag" is set to 1, the
wait_inputdata_max_time parameter is used to set the timeout duration of the entire period.
}
message ConnectConfig
{
    uint32 src_engine_id=1; // Engine ID of the transmit end
    uint32 src_port_id = 2; // Port ID of the transmit end
    uint32 target_graph_id=3; // Graph ID of the receive end
    uint32 target_engine_id=4; // Port ID of the receive end
    uint32 target_port_id=5; // Port ID of the receive end
    bool receive_memory_without_dvpp=6; // The default value is 0, indicating that the RX memory of the
target engines running on the device side must meet the 4 GB address space limit. If this parameter is set
to 1, the 4 GB address space limit does not need to be met. You can set the receive_memory_without_dvpp
parameter (under the "connects" property) to 1 in the graph configuration file for all target engines
running on the device side. You can also set the receive_memory_without_dvpp parameter under the
"connects" property to 1 because the target engines running on the host side are not affected even if the
parameter is set. In this way, the Matrix RX memory pool is not limited by the 4 GB address space, which
improves the memory utilization. In the current DVPP, the input memory for the VPC, JPEGG, JPEGD, and
PNGD must meet the 4 GB address space requirements, while that for the VDEC and VENC can either or
not meet the 4 GB address space requirements.
}
message GraphConfig
{
    uint32 graph_id = 1; //Graph ID
    int32 priority = 2; // Priority
    string device_id = 3; // Device ID, for example, "0"
    repeated EngineConfig engines = 3; // All engines. You are advised to configure multiple engines for multi-
channel decoding. One engine corresponds to one thread. If one engine corresponds to multiple threads,
the decoding sequence cannot be ensured.
    repeated ConnectConfig connects = 4; // Connection mode
}
message GraphConfigList
{
    repeated GraphConfig graphs = 1; // Configuration parameter for a single graph
}

message ProfileConfig
{
    string matrix_profiling = 1; // Whether to enable profiling for Matrix. The value "on" indicates that the
function is enabled.
    string ome_profiling = 2; // Whether to enable the performance statistics function of OME. The value
"on" indicates that the function is enabled.
    string cce_profiling = 3; // Whether to enable the performance statistics function of CCE. The value "on"
indicates that the function is enabled.
    string runtime_profiling = 4; // Whether to enable the performance statistics function of Runtime. The
value "on" indicates that the function is enabled.
    string PROFILER_TARGET = 5; // Parameter for transparent transmission profiling
    string PROFILER_JOBCTX = 6;
    string src_path = 7;
```

```
string dest_path = 8;
string runtime_config = 9;
string RTS_PATH = 10;
string profiler_jobctx_path = 11;
string profiler_target_path = 12;
}

message GraphUpdateConfig
{
    GraphConfig updataGraphConfig = 1;
    repeated uint32 del_engine_ids = 2;
    repeated ConnectConfig del_connects = 3;
}
```

8.2.7.2.2 Matrix Custom Data Types

- **HIAI_StatusT**: Specifies the data type of the result code as **uint32_t**.
- **EnginePortID**: Specifies the port ID in the Matrix.

```
struct EnginePortID {
    uint32_t graph_id;
    uint32_t engine_id;
    uint32_t port_id;
};
```

8.2.7.3 Data Structures Registered in the Matrix

The data structures transferred in the Matrix need to be registered first. The following data structures have been registered and can be directly used.

```
namespace hiai {
// message name <=====> message type
// name:"BatchInfo" type:BatchInfo
// name:"FrameInfo" type:FrameInfo
// name:"IMAGEFORMAT" type:IMAGEFORMAT
// name:"Angle" type:Angle
// name:"Point2D" type:Point2D
// name:"Point3D" type:Point3D
// name:"ROICube" type:ROICube
// name:"RLECode" type:RLECode
// name:"IDTreeNode" type:IDTreeNode
// name:"IDTreePara" type:IDTreePara
// name:"BatchIDTreePara" type:BatchIDTreePara
// name:"RetrievalResult" type:RetrievalResult
// name:"RetrievalResultTopN" type:RetrievalResultTopN
// name:"RetrievalResultPara" type:RetrievalResultPara
// name:"RawDataBuffer" type:RawDataBuffer
// name:"BatchRawDataBuffer" type:BatchRawDataBuffer
// name:"string" type:string

// name:"vector_uint8_t" type:vector<uint8_t>
// name:"vector_float" type:vector<float>

// name:"ImageData_uint8_t" type:ImageData<uint8_t>
// name:"ImageData_float" type:ImageData<float>

// name:"ImagePara_uint8_t" type:ImagePara<uint8_t>
// name:"ImagePara_float" type:ImagePara<float>

// name:"BatchImagePara_uint8_t" type:BatchImagePara<uint8_t>
// name:"BatchImagePara_float" type:BatchImagePara<float>
}
```

```
// name:"Line_uint8_t"           type:Line<uint8_t>
// name:"Line_float"            type:Line<float>

// name:"Rectangle_uint8_t"      type:Rectangle<uint8_t>
// name:"Rectangle_float"        type:Rectangle<float>

// name:"Polygon_uint8_t"        type:Polygon<uint8_t>
// name:"Polygon_float"          type:Polygon<float>

// name:"MaskMatrix_uint8_t"     type:MaskMatrix<uint8_t>
// name:"MaskMatrix_float"       type:MaskMatrix<float>

// name:"ObjectLocation_uint8_t_uint8_t" type:ObjectLocation<uint8_t, uint8_t>
// name:"ObjectLocation_float_float" type:ObjectLocation<float, uint8_t>

// name:"DetectedObjectPara_uint8_t_uint8_t" type:DetectedObjectPara<uint8_t, uint8_t>
// name:"DetectedObjectPara_float_float" type:DetectedObjectPara<float, float>

// name:"BatchDetectedObjectPara_uint8_t_uint8_t"
type:BatchDetectedObjectPara<uint8_t, uint8_t>
// name:"BatchDetectedObjectPara_float_float" type:BatchDetectedObjectPara<float, float>
// name:"BatchDetectedObjectPara_Rectangle_Point2D_int32_t"
type:BatchDetectedObjectPara<Rectangle<Point2D>, int32_t>
// name:"BatchDetectedObjectPara_Rectangle_Point2D_float"
type:BatchDetectedObjectPara<Rectangle<Point2D>, float>

// name:"RegionImage_uint8_t"     type:RegionImage<uint8_t>
// name:"RegionImage_float"       type:RegionImage<float>

// name:"RegionImagePara_uint8_t" type:RegionImagePara<uint8_t>
// name:"RegionImagePara_float"   type:RegionImagePara<float>

// name:"BatchRegionImagePara_uint8_t" type:BatchRegionImagePara<uint8_t>
// name:"BatchRegionImagePara_float" type:BatchRegionImagePara<float>

// name:"Attribute_uint8_t"        type:Attribute<uint8_t>
// name:"Attribute_float"          type:Attribute<float>

// name:"AttributeTopN_uint8_t"    type:AttributeTopN<uint8_t>
// name:"AttributeTopN_float"      type:AttributeTopN<float>

// name:"AttributeVec_uint8_t"     type:AttributeVec<uint8_t>
// name:"AttributeVec_float"       type:AttributeVec<float>

// name:"AttributeResultPara_uint8_t" type:AttributeVec<uint8_t>
// name:"AttributeResultPara_float" type:AttributeVec<float>

// name:"BatchAttributeResultPara_uint8_t" type:AttributeVec<uint8_t>
// name:"BatchAttributeResultPara_float" type:AttributeVec<float>

// name:"Feature_uint8_t"          type:AttributeVec<uint8_t>
// name:"Feature_float"            type:AttributeVec<float>
```

```
// name:"FeatureList_uint8_t"      type:FeatureList<uint8_t>
// name:"FeatureList_float"        type:FeatureList<float>

// name:"FeatureVec_uint8_t"        type:FeatureVec<uint8_t>
// name:"FeatureVec_float"          type:FeatureVec<float>

// name:"FeatureResultPara_uint8_t" type:FeatureResultPara<uint8_t>
// name:"FeatureResultPara_float"   type:FeatureResultPara<float>

// name:"BatchFeatureResultPara_uint8_t" type:BatchFeatureResultPara<uint8_t>
// name:"BatchFeatureResultPara_float"   type:BatchFeatureResultPara<float>

// name:"FeatureRecord_uint8_t"      type:FeatureRecord<uint8_t>
// name:"FeatureRecord_float"        type:FeatureRecord<float>

// name:"RetrievalSet_uint8_t_uint8_t" type:RetrievalSet<uint8_t, uint8_t>
// name:"RetrievalSet_float_float"    type:RetrievalSet<float, float>

// name:"EvaluationResult_uint8_t"    type:EvaluationResult<uint8_t>
// name:"EvaluationResult_float"      type:EvaluationResult<float>

// name:"EvaluationResultVec_uint8_t" type:EvaluationResultVec<uint8_t>
// name:"EvaluationResultVec_float"   type:EvaluationResultVec<float>

// name:"EvaluationResultPara_uint8_t" type:EvaluationResultPara<uint8_t>
// name:"EvaluationResultPara_float"   type:EvaluationResultPara<float>

// name:"BatchEvaluationResultPara_uint8_t" type:BatchEvaluationResultPara<uint8_t>
// name:"BatchEvaluationResultPara_float"   type:BatchEvaluationResultPara<float>

// name:"Classification_uint8_t"       type:Classification<uint8_t>
// name:"Classification_float"         type:Classification<float>

// name:"ClassificationTopN_uint8_t"   type:ClassificationTopN<uint8_t>
// name:"ClassificationTopN_float"     type:ClassificationTopN<float>

// name:"ClassificationVec_uint8_t"    type:ClassificationVec<uint8_t>
// name:"ClassificationVec_float"      type:ClassificationVec<float>

// name:"ClassificationResultPara_uint8_t" type:ClassificationResultPara<uint8_t>
// name:"ClassificationResultPara_float"   type:ClassificationResultPara<float>

// name:"BatchClassificationResultPara_uint8_t" type:BatchClassificationResultPara<uint8_t>
// name:"BatchClassificationResultPara_float"   type:BatchClassificationResultPara<float>

////////////////////////////////////
//////////////////////////////////// batch information //////////////////////////////////////
////////////////////////////////////
struct BatchInfo {
    bool is_first = false;      // Whether it is the first batch
    bool is_last = false;       // Whether it is the last batch
    uint32_t batch_size = 0;     // Actual size of the current batch
    uint32_t max_batch_size = 0; // Preset size of a batch (maximum capacity)
    uint32_t batch_ID = 0;      // Current batch ID
```

```
uint32_t channel_ID = 0;           // ID of the channel that processes the current batch
uint32_t processor_stream_ID = 0;  // ID of the processor computing stream

std::vector<uint32_t> frame_ID;     // ID of the image frame in the batch
std::vector<uint32_t> source_ID;    // ID of the image source in the batch

std::vector<uint64_t> timestamp;    // Image timestamp in the batch
};

template<class Archive>
void serialize(Archive& ar, BatchInfo& data) {
    ar(data.is_first, data.is_last, data.batch_size,
        data.max_batch_size, data.batch_ID, data.channel_ID,
        data.processor_stream_ID, data.frame_ID, data.source_ID,
        data.timestamp);
}

////////////////////////////////////
//////////////////////////////////// frame information //////////////////////////////////
////////////////////////////////////
struct FrameInfo {
    bool is_first = false;          // Whether it is the first frame
    bool is_last = false;           // Whether it is the last frame
    uint32_t channel_ID = 0;         // ID of the channel that processes the current frame
    uint32_t processor_stream_ID = 0; // ID of the processor computing stream
    uint32_t frame_ID = 0;           // Image frame ID
    uint32_t source_ID = 0;          // Image source ID
    uint64_t timestamp = 0;          // Image timestamp
};

template<class Archive>
void serialize(Archive& ar, FrameInfo& data) {
    ar(data.is_first, data.is_last, data.channel_ID,
        data.processor_stream_ID, data.frame_ID, data.source_ID,
        data.timestamp);
}

////////////////////////////////////
//////////////////////////////////// 1. image tensor //////////////////////////////////
////////////////////////////////////

// Image format
enum IMAGEFORMAT {
    RGB565,          // Red 15:11, Green 10:5, Blue 4:0
    BGR565,          // Blue 15:11, Green 10:5, Red 4:0
    RGB888,          // Red 24:16, Green 15:8, Blue 7:0
    BGR888,          // Blue 24:16, Green 15:8, Red 7:0
    BGRA8888,        // Blue 31:24, Green 23:16, Red 15:8, Alpha 7:0
    ARGB8888,        // Alpha 31:24, Red 23:16, Green 15:8, Blue 7:0
    RGBX8888,
    XRGB8888,
    YUV420Planar,    // I420
    YVU420Planar,    // YV12
    YUV420SP,        // NV12
    YVU420SP,        // NV21
    YUV420Packed,    // YUV420 Interleaved
    YVU420Packed,    // YVU420 Interleaved
    YUV422Planar,    // Three arrays Y,U,V.
    YVU422Planar,
    YUYVPacked,      // 422 packed per payload in planar slices
    YVYUPacked,      // 422 packed
    UYVYPacket,      // 422 packed
}
```

```
VYUYPacket,    // 422 packed
YUV422SP,      // 422 packed
YVU422SP,
YUV444Interleaved, // Each pixel contains equal parts YUV
Y8,
Y16,
RAW
};

enum OBJECTTYPE {
    OT_VEHICLE,
    OT_HUMAN,
    OT_NON_MOTOR,
    OT_FACE,
    OT_FACE_BODY,
    OT_PLATE
};

template<class T> // T: uint8_t float
struct ImageData {
    IMAGEFORMAT format; // Image format

    uint32_t width = 0; // Image width
    uint32_t height = 0; // Image height
    uint32_t channel = 0; // Number of image channels
    uint32_t depth = 0; // Bit depth
    uint32_t height_step = 0; // Alignment height
    uint32_t width_step = 0; // Alignment width
    uint32_t size = 0; // Data size (in bytes)
    std::shared_ptr<T> data; // Data pointer
};

template<class Archive, class T>
void serialize(Archive& ar, ImageData<T>& data) {
    ar(data.format, data.width, data.height, data.channel,
        data.depth, data.height_step, data.width_step, data.size);
    if (data.size > 0 && data.data.get() == nullptr) {
        data.data.reset(new(std::nothrow) T[data.size]);
    }
    ar(cereal::binary_data(data.data.get(), data.size * sizeof(T)));
}

// Parameter
template<class T>
struct ImagePara {
    FrameInfo f_info; // Frame information
    ImageData<T> img; // Image
};

template<class Archive, class T>
void serialize(Archive& ar, ImagePara<T>& data) {
    ar(data.f_info, data.img);
}

// Parameter
template<class T>
struct BatchImagePara {
    BatchInfo b_info; // Batch information
    std::vector<ImageData<T>> v_img; // Image in the batch
};
```



```
template<class Archive, class T>
void serialize(Archive& ar, BatchImagePara<T>& data) {
    ar(data.b_info, data.v_img);
}

// Angle
typedef struct {
    float x;
    float y;
    float z;
}Angle;

template<class Archive>
void serialize(Archive& ar, Angle& data) {
    ar(data.x, data.y, data.z);
}

// 2D point
struct Point2D {
    int32_t x;
    int32_t y;
};

template<class Archive>
void serialize(Archive& ar, Point2D& data) {
    ar(data.x, data.y);
}

////////////////////////////////////
////////// ROI information //////////
////////////////////////////////////

struct RoiPolygon {
    uint32_t    uiPtNum;    // Number of polygon vertices
    std::vector<Point2D> astPts;    // Multiple retrieval results (TopN)
};

template<class Archive>
void serialize(Archive &ar, RoiPolygon &data) {
    ar(data.uiPtNum, data.astPts);
}

struct ArgsRoiPolygon {
    RoiPolygon stRoiPolygon; // Configuration item ROI
    uint32_t iMinFace; // Minimum face size of 26 x 26
    uint32_t iMaxFace; // Maximum face size of 300 x 300
    uint32_t imgWidth; // Video frame image width
    uint32_t imgHeight; // Video frame image height
};

template<class Archive>
void serialize(Archive &ar, ArgsRoiPolygon &data) {
    ar(data.stRoiPolygon, data.iMinFace, data.iMaxFace, data.imgWidth, data.imgHeight, data.iMinFace);
}

// 3D point
struct Point3D {
    int32_t x;
    int32_t y;
```

```

        int32_t z;
    };

    template<class Archive>
    void serialize(Archive& ar, Point3D& data) {
        ar(data.x, data.y, data.z);
    }

    // Lines in the 2D or 3D space
    template<class T> // T: Point2D Point3D
    struct Line {
        T start;
        T end;
    };

    template<class Archive, class T>
    void serialize(Archive& ar, Line<T>& data) {
        ar(data.start, data.end);
    }

    // Rectangular plane in the 2D or 3D space
    template<class T> // T: Point2D Point3D
    struct Rectangle {
        T anchor_lt;
        T anchor_rb;
    };

    template<class Archive, class T>
    void serialize(Archive& ar, Rectangle<T>& data) {
        ar(data.anchor_lt, data.anchor_rb);
    }

    // Polygon plane in the 2D or 3D space
    template<class T> // T: Point2D Point3D
    struct Polygon {
        std::vector<T> anchors;
    };

    template<class Archive, class T>
    void serialize(Archive& ar, Polygon<T>& data) {
        ar(data.anchors);
    }

    // 3D cube
    struct ROICube {
        Point3D anchor;
        uint32_t length;
        uint32_t width;
        uint32_t height;
    };

    template<class Archive>
    void serialize(Archive& ar, ROICube& data) {
        ar(data.anchor, data.length, data.width, data.height);
    }

    // Mask matrix
    template<class T> // T: int32_t float etc.
    struct MaskMatrix {

```

```
uint32_t cols;
uint32_t rows;
std::shared_ptr<T> data;
};

template<class Archive, class T>
void serialize(Archive& ar, MaskMatrix<T>& data) {
    ar(data.cols, data.rows);
    if (data.cols*data.rows > 0 && data.data.get() == nullptr) {
        data.data.reset(new(std::nothrow) T[data.cols*data.rows]);
    }
    ar(cereal::binary_data(data.data.get(),
        data.cols * data.rows * sizeof(T)));
}

// RLE coding
struct RLECode {
    uint32_t len;
    uint32_t cols;
    uint32_t rows;
    std::shared_ptr<uint32_t> data;
};

template<class Archive>
void serialize(Archive& ar, RLECode& data) {
    ar(data.len, data.cols, data.rows);
    if (data.len > 0 && data.data.get() == nullptr) {
        data.data.reset(new(std::nothrow) uint32_t[data.len]);
    }
    ar(cereal::binary_data(data.data.get(),
        data.len * sizeof(uint32_t)));
}

template<class T1, class T2> // T1: Point2D Rectangle RLECode etc.
struct ObjectLocation { // T2: int8_t float etc.
    std::vector<uint32_t> v_obj_id;
    std::vector<T1> range; // Target scope description
    std::vector<Angle> angle; // Angle information
    std::vector<T2> confidence; // Confidence
    std::vector<uint32_t> label; // Target type tag
};

template<class Archive, class T1, class T2>
void serialize(Archive& ar, ObjectLocation<T1, T2>& data) {
    ar(data.v_obj_id, data.range, data.angle,
        data.confidence, data.label);
}

// Parameter
template<class T1, class T2>
struct DetectedObjectPara {
    FrameInfo f_info; // Frame information
    ObjectLocation<T1, T2> location; // Multiple target positions
};

template<class Archive, class T1, class T2>
void serialize(Archive& ar, DetectedObjectPara<T1, T2>& data) {
    ar(data.f_info, data.location);
}

// Parameter
```

```
template<class T1, class T2>
struct BatchDetectedObjectPara {
    // Batch information
    BatchInfo b_info;
    // Multiple target positions corresponding to each frame of image
    std::vector<ObjectLocation<T1, T2> > v_location;
};

template<class Archive, class T1, class T2>
void serialize(Archive& ar, BatchDetectedObjectPara<T1, T2>& data) {
    ar(data.b_info, data.v_location);
}

////////////////////////////////////
////////// 3. region image tensor //////////
////////////////////////////////////

template<class T>
struct RegionImage {
    std::vector<ImageData<T> > region;
    std::vector<uint32_t> v_obj_id;
};

template<class Archive, class T>
void serialize(Archive& ar, RegionImage<T>& data) {
    ar(data.v_obj_id, data.region);
}

// Parameter
template<class T>
struct RegionImagePara {
    FrameInfo f_info;        // Frame information
    RegionImage<T> region;    // Small image corresponding to each frame
};

template<class Archive, class T>
void serialize(Archive& ar, RegionImagePara<T>& data) {
    ar(data.f_info, data.region);
}

// Parameter
template<class T>
struct BatchRegionImagePara {
    // Batch information
    BatchInfo b_info;
    // Each frame has several region images, which are referenced by a pointer pointing to the image array.
    std::vector<RegionImage<T>> v_region;
};

template<class Archive, class T>
void serialize(Archive& ar, BatchRegionImagePara<T>& data) {
    ar(data.b_info, data.v_region);
}
```

```
// Recursive ID tree node
struct IDTreeNode {
    std::vector<uint32_t> nodeID;    // Node ID
    std::vector<bool> is_leaf;      // Whether it is a leaf node
    std::vector<IDTreeNode> node;   // Pointer to the subnode structure
};

template <class Archive>
void serialize(Archive& ar, IDTreeNode& data) {
    ar(data.nodeID, data.is_leaf, data.node);
}

// Parameter
struct IDTreePara {
    FrameInfo f_info;    // Frame information
    IDTreeNode tree;     // ID tree in the image
};

template <class Archive>
void serialize(Archive& ar, IDTreePara& data) {
    ar(data.f_info, data.tree);
}

// Parameter
struct BatchIDTreePara {
    BatchInfo b_info;    // Batch information
    std::vector<IDTreeNode> v_tree; // ID tree in each frame of image
};

template <class Archive>
void serialize(Archive& ar, BatchIDTreePara& data) {
    ar(data.b_info, data.v_tree);
}

template<class T>
struct Attribute {
    std::string attr_value;
    T score;
};

template <class Archive, class T>
void serialize(Archive& ar, Attribute<T>& data) {
    ar(data.attr_value, data.score);
}

template<class T>
struct AttributeTopN {
    uint32_t obj_ID;
    std::map<std::string, std::vector<Attribute<T>>> attr_map;
};

template <class Archive, class T>
void serialize(Archive& ar, AttributeTopN<T>& data) {
    ar(data.obj_ID, data.attr_map);
}

template<class T>
```

```
struct AttributeVec {
    std::vector<AttributeTopN<T>> obj_attr;
};

template <class Archive, class T>
void serialize(Archive& ar, AttributeVec<T>& data) {
    ar(data.obj_attr);
}

// 序列化
template<class T>
struct AttributeResultPara {
    FrameInfo f_info;
    AttributeVec<T> attr;
};

template <class Archive, class T>
void serialize(Archive& ar, AttributeResultPara<T>& data) {
    ar(data.f_info, data.attr);
}

// 序列化
template<class T>
struct BatchAttributeResultPara {
    BatchInfo b_info;
    std::vector<AttributeVec<T>> v_attr;
};

template <class Archive, class T>
void serialize(Archive& ar, BatchAttributeResultPara<T>& data) {
    ar(data.b_info, data.v_attr);
}

template<class T>
struct Feature {
    int32_t len;           // Feature vector length
    std::shared_ptr<T> feature; // Pointer to the feature vector
};

template<class Archive, class T>
void serialize(Archive& ar, Feature<T>& data) {
    ar(data.len);
    if (data.len > 0 && data.feature.get() == nullptr) {
        data.feature.reset(new(std::nothrow) T[data.len]);
    }
    ar(cereal::binary_data(data.feature.get(), data.len * sizeof(T)));
}

template<class T>
struct FeatureList {
    uint32_t obj_ID;       // Target ID
    std::vector<Feature<T>> feature_list; // Multiple features of the target
};

template<class Archive, class T>
void serialize(Archive& ar, FeatureList<T>& data) {
```

```
        ar(data.obj_ID, data.feature_list);
    }

    template<class T>
    struct FeatureVec {
        std::vector<FeatureList<T>> obj_feature; // Attributes of multiple targets
    };

    template<class Archive, class T>
    void serialize(Archive& ar, FeatureVec<T>& data) {
        ar(data.obj_feature);
    }

    // Parameter
    template<class T>
    struct FeatureResultPara {
        FrameInfo f_info; // Frame information
        FeatureVec<T> feature; // Features of multiple targets in an image
    };

    template<class Archive, class T>
    void serialize(Archive& ar, FeatureResultPara<T>& data) {
        ar(data.f_info, data.feature);
    }

    // Parameter
    template<class T>
    struct BatchFeatureResultPara {
        // Batch information
        BatchInfo b_info;
        // Features of multiple targets corresponding to each frame of image
        std::vector<FeatureVec<T>> v_feature;
    };

    template<class Archive, class T>
    void serialize(Archive& ar, BatchFeatureResultPara<T>& data) {
        ar(data.b_info, data.v_feature);
    }

    template<class T>
    struct FeatureRecord {
        uint32_t ID; // Feature map ID
        uint32_t len; // Feature vector length
        std::shared_ptr<T> feature; // Pointer to the feature vector
    };

    template<class Archive, class T>
    void serialize(Archive& ar, FeatureRecord<T>& data) {
        ar(data.ID, data.len);
        if (data.len > 0 && data.feature.get() == nullptr) {
            data.feature.reset(new(std::nothrow) T[data.len]);
        }
        ar(cereal::binary_data(data.feature.get(), data.len * sizeof(T)));
    }

    template<class T1, class T2>
    struct RetrievalSet {
        uint32_t TopN; // TopN result setting
    };

```

```

        T1 threshold;    // Similarity threshold
        std::vector<FeatureRecord<T2>> record; // All records in the feature set
    };

    template<class Archive, class T1, class T2>
    void serialize(Archive& ar, RetrievalSet<T1, T2>& data) {
        ar(data.TopN, data.threshold, data.record);
    }

    // Parameter
    template<class T1, class T2>
    struct RetrievalSetPara {
        RetrievalSet<T1, T2> set; // Parameter and feature set
    };

    template<class Archive, class T1, class T2>
    void serialize(Archive& ar, RetrievalSetPara<T1, T2>& data) {
        ar(data.set);
    }

    struct RetrievalResult {
        uint32_t ID;    // Feature map ID
        float similarity; // Similarity
    };

    template<class Archive>
    void serialize(Archive& ar, RetrievalResult& data) {
        ar(data.ID, data.similarity);
    }

    struct RetrievalResultTopN {
        std::vector<RetrievalResult> result_list; // Multiple retrieval results (TopN)
    };

    template<class Archive>
    void serialize(Archive& ar, RetrievalResultTopN& data) {
        ar(data.result_list);
    }

    // Parameter
    struct RetrievalResultPara {
        FrameInfo f_info;    // Frame information
        RetrievalResultTopN result; // Retrieval result
    };

    template<class Archive>
    void serialize(Archive& ar, RetrievalResultPara& data) {
        ar(data.f_info, data.result);
    }

    // Parameter
    struct BatchRetrievalResultPara {
        // Batch information
        BatchInfo b_info;
        // Retrieval result corresponding to each frame of image
        std::vector<RetrievalResultTopN> v_result;
    };

```



```
template<class Archive>
void serialize(Archive& ar, BatchRetrievalResultPara& data) {
    ar(data.b_info, data.v_result);
}

template<class T>
struct EvaluationResult {
    uint32_t obj_ID;           // Target ID
    std::string description;    // Description
    T score;                   // Evaluation score
};

template <class Archive, class T>
void serialize(Archive& ar, EvaluationResult<T>& data) {
    ar(data.obj_ID, data.description, data.score);
}

template<class T>
struct EvaluationResultVec {
    std::vector<EvaluationResult<T>> result; // Evaluation results of multiple targets
};

template <class Archive, class T>
void serialize(Archive& ar, EvaluationResultVec<T>& data) {
    ar(data.result);
}

// Parameter
template<class T>
struct EvaluationResultPara {
    FrameInfo f_info;          // Frame information
    EvaluationResultVec<T> result; // Evaluation result
};

template <class Archive, class T>
void serialize(Archive& ar, EvaluationResultPara<T>& data) {
    ar(data.f_info, data.result);
}

// Parameter
template<class T>
struct BatchEvaluationResultPara {
    // Batch information
    BatchInfo b_info;
    // Evaluation result of each frame of image
    std::vector<EvaluationResultVec<T>> v_result;
};

template <class Archive, class T>
void serialize(Archive& ar, BatchEvaluationResultPara<T>& data) {
    ar(data.b_info, data.v_result);
}

template<class T>
struct Classification {
    std::string class_value; // Type value
    T score;                 // Confidence
};
```

```
template<class Archive, class T>
void serialize(Archive& ar, Classification<T>& data) {
    ar(data.class_value, data.score);
}

template<class T>
struct ClassificationTopN {
    uint32_t obj_ID;           // Target ID
    std::vector<Classification<T>> class_result; // Classification result (TopN)
};

template<class Archive, class T>
void serialize(Archive& ar, ClassificationTopN<T>& data) {
    ar(data.obj_ID, data.class_result);
}

template<class T>
struct ClassificationVec {
    std::vector<ClassificationTopN<T>> class_list; // Classification of multiple targets
};

template<class Archive, class T>
void serialize(Archive& ar, ClassificationVec<T>& data) {
    ar(data.class_list);
}

// Parameter
template<class T>
struct ClassificationResultPara {
    FrameInfo f_info;           // Frame information
    ClassificationVec<T> classification; // Classification of multiple targets
};

template<class Archive, class T>
void serialize(Archive& ar, ClassificationResultPara<T>& data) {
    ar(data.f_info, data.classification);
}

// Parameter
template<class T>
struct BatchClassificationResultPara {
    // Batch information
    BatchInfo b_info;
    // Classification of multiple targets corresponding to each frame of image
    std::vector<ClassificationVec<T>> v_class;
};

template<class Archive, class T>
void serialize(Archive& ar, BatchClassificationResultPara<T>& data) {
    ar(data.b_info, data.v_class);
}

struct RawDataBuffer {
    uint32_t len_of_byte; // size length
    std::shared_ptr<uint8_t> data; // One buffer, which can be converted to a user-defined type
};
```

```
template<class Archive>
void serialize(Archive& ar, RawDataBuffer& data) {
    ar(data.len_of_byte);
    if (data.len_of_byte > 0 && data.data.get() == nullptr) {
        data.data.reset(new(std::nothrow) uint8_t[data.len_of_byte]);
    }
    ar(cereal::binary_data(data.data.get(), data.len_of_byte *
        sizeof(uint8_t)));
}

// common raw databuffer struct
struct BatchRawDataBuffer {
    BatchInfo b_info; // batch info
    std::vector<RawDataBuffer> v_info;
};

template<class Archive>
void serialize(Archive& ar, BatchRawDataBuffer& data) {
    ar(data.b_info, data.v_info);
}

template<typename T> const char* TypeName(void);

#define REGISTER_TYPE_DEFINITION(type) \
    template<> inline const char* TypeName<type>(void) { return #type; }

REGISTER_TYPE_DEFINITION(int8_t);
REGISTER_TYPE_DEFINITION(uint8_t);
REGISTER_TYPE_DEFINITION(int16_t);
REGISTER_TYPE_DEFINITION(uint16_t);
REGISTER_TYPE_DEFINITION(int32_t);
REGISTER_TYPE_DEFINITION(uint32_t);
REGISTER_TYPE_DEFINITION(int64_t);
REGISTER_TYPE_DEFINITION(uint64_t);
REGISTER_TYPE_DEFINITION(float);
REGISTER_TYPE_DEFINITION(double);
REGISTER_TYPE_DEFINITION(Point2D);
REGISTER_TYPE_DEFINITION(Point3D);
REGISTER_TYPE_DEFINITION(Line<Point2D>);
REGISTER_TYPE_DEFINITION(Line<Point3D>);
REGISTER_TYPE_DEFINITION(Rectangle<Point2D>);
REGISTER_TYPE_DEFINITION(Rectangle<Point3D>);
REGISTER_TYPE_DEFINITION(Polygon<Point2D>);
REGISTER_TYPE_DEFINITION(Polygon<Point3D>);
REGISTER_TYPE_DEFINITION(ROICube);
REGISTER_TYPE_DEFINITION(MaskMatrix<int8_t>);
REGISTER_TYPE_DEFINITION(MaskMatrix<int32_t>);
REGISTER_TYPE_DEFINITION(MaskMatrix<float>);
REGISTER_TYPE_DEFINITION(RLECode);
REGISTER_TYPE_DEFINITION(RoiPolygon);
REGISTER_TYPE_DEFINITION(ArgsRoiPolygon);
}
```

8.2.7.4 Example

8.2.7.4.1 Orchestration Configuration Example

The configuration file for creating a graph (**graph.prototxt**) is in proto format. In the following example, two graphs are created at the same time, multiple engines are created for each graph, and the mapping is configured.

NOTICE

To transfer a file, the following conditions must be met. Otherwise, the system transfers a string or number by default.

- In the Linux operating system, the file must contain a relative path or an absolute path, for example, **/home/1.txt** or **../test/2.txt**. Note: The string may be mistakenly considered as a file if it contains the "\" or "/" symbol. Therefore, do not use the symbols.
- In Windows, the file must contain a relative path or an absolute path, for example: **c:\1.txt** or **..\test\2.txt**. Note: The string may be mistakenly considered as a file if it contains the "\" or "/" symbol. Therefore, do not use the symbols.

```
graphs {
  graph_id: 100
  device_id: "0"
  priority: 1
  engines {
    id: 1000
    engine_name: "SrcEngine"
    side: HOST
    thread_num: 1
  }
  engines {
    id: 1001
    engine_name: "HelloWorldEngine"
    so_name: "./libhelloworld.so"
    side: DEVICE
    thread_num: 1
  }
  engines {
    id: 1002
    engine_name: "DestEngine"
    side: HOST
    thread_num: 1
  }
}
connects {
  src_engine_id: 1000
  src_port_id: 0
  target_engine_id: 1001
  target_port_id: 0
}
connects {
  src_engine_id: 1001
  src_port_id: 0
  target_engine_id: 1002
  target_port_id: 0
}
}
graphs {
  graph_id: 200
  device_id: "1"
  priority: 1
  engines {
    id: 1000
    engine_name: "SrcEngine"
```

```
    side: HOST
    thread_num: 1
  }
  engines {
    id: 1001
    engine_name: "HelloWorldEngine"
    internal_so_name: "/lib64/libhelloworld.so"
    side: DEVICE
    thread_num: 1
  }
  engines {
    id: 1002
    engine_name: "DestEngine"
    side: HOST
    thread_num: 1
  }
  connects {
    src_engine_id: 1000
    src_port_id: 0
    target_engine_id: 1001
    target_port_id: 0
  }
  connects {
    src_engine_id: 1001
    src_port_id: 0
    target_engine_id: 1002
    target_port_id: 0
  }
}
```

8.2.7.4.2 Example of Data Transmission Based on Optimized Performance

(1) When the performance optimization solution is used to send data, the data must be manually serialized and deserialized.

// Note: The serialization function is used at the transmit end, while the deserialization function is used at the receive end. Therefore, the registration function must be registered at both the receive and transmit ends.

Data structure

typedef struct

```
{
    uint32_t frameId;
    uint8_t  bufferId;
    uint8_t* image_data;
    uint8_t  image_size;
}TEST_STR;
```

// Serialize the TEST_STR structure. This function needs to be registered only by calling the registration function. The parameters are described as follows:

// Input: inputPtr, pointer to the TEST_STR structure

// Output: ctrlStr, control information address

imageData, data information pointer

imageLen, data information size

void GetTestStrSearPtr(void* inputPtr, std::string& ctrlStr, uint8_t*& imageData, uint32_t& imageLen)

```
{
    // Obtain the structure buffer.
    TEST_STR* test_str = (TEST_STR*)inputPtr;
    ctrlStr = std::string((char*)inputPtr, sizeof(TEST_STR));
```

// Obtain data information and assign a value to the data pointer of the structure.

imageData = (uint8_t*)test_str->image_data;

imageLen = test_str->image_size;

```
}
```

// Deserialization structure. The returned data is the structure buffer and data block buffer.

// Input: ctrlPtr, control information address

ctrlLen, control information size

imageData, data information pointer

imageLen, data information size

```
// Output: std::shared_ptr<void>, smart pointer to the structure
std::shared_ptr<void> GetTestStrDearPtr(const char* ctrlPtr, const uint32_t& ctrlLen, const uint8_t*
imageData, const uint32_t& imageLen)
{
    // Obtain the structure.
    TEST_STR* test_str = (TEST_STR*)ctrlPtr;

    // Obtain the transferred large memory data.
    // Note: For large memory data, assign a value to the smart pointer and register the deleter. If the smart
    pointer is not used or the deleter is not registered, manually invoke hiai::Graph::ReleaseDataBuffer(void*
ptr) when releasing the memory.
    std::shared_ptr<TEST_STR<uint8_t>> shared_data = std::make_shared<TEST_STR<uint8_t>>();
    shared_data->frameId = test_str->frame_ID;
    shared_data->bufferId = test_str->bufferId;
    shared_data->image_size = imageLen;
    shared_data->image_data.reset(imageData, hiai::Graph::ReleaseDataBuffer);

    // The smart pointer is returned to the engine on the device.
    return std::static_pointer_cast<void>(shared_data);
}
// Register the serialiation and deserialization functions.
HIAI_REGISTER_SERIALIZE_FUNC("TEST_STR", TEST_STR, GetTestStrSearPtr, GetTestStrDearPtr);

(2) When sending data, you must use a registered data type together with HIAI_DMAlloc which is used to
allocate data memory to improve the performance.
    Note: During data transfer from the host to the device, you are advised to use HIAI_DMAlloc, which can
greatly improve the transmission efficiency. The HIAI_DMAlloc interface supports the size of 0 to (256 MB –
96 bytes). If the data exceeds this range, use the malloc interface to allocate memory.
    // Use the Dmalloc interface to allocate the data memory. 10000 indicates the delay (in milliseconds),
that is, if the memory is insufficient, wait for 10000 milliseconds.
    HIAI_StatusT get_ret = HIAIMemory::HIAI_DMAlloc(width*align_height*3/2, (void*)&align_buffer, 10000);

    // Send data. After this interface is called, the HIAI_Dfree interface does not need to be called. The dealy
10000 indicates the delay.
    graph->SendData(engine_id_0, "TEST_STR", std::static_pointer_cast<void>(align_buffer), 10000);
```

8.3 DVPP API Reference

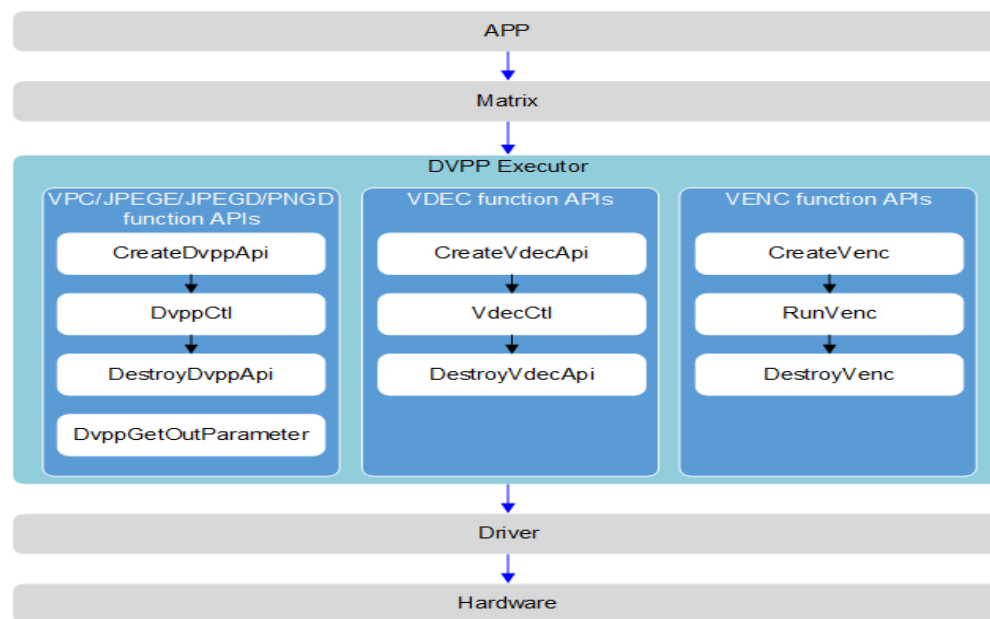
8.3.1 Overview

8.3.1.1 API Introduction

This document describes the external APIs of the digital vision pre-processing (DVPP) executor. The API functions, API calling guidance, and API usage examples are provided for developers and test engineers.

The process of calling the DVPP APIs during application development is as follows:

Figure 8-1 Process flowchart



- In the current version, the VPC function can be implemented by **CreateDvppApi**, **DvppCtl**, and **DestroyDvppApi** in either of the following methods:
 - Method 1: Pass input parameters to **DvppCtl**, that is, the **DVPP_CTL_VPC_PROC** command word and **VpcUserImageConfigure** structure parameters. **This method is recommended.**
 - Method 2: Pass input parameters to **DvppCtl**, that is, the **DVPP_CTL_VPC_PROC** command word and **resize_param_in_msg** structure parameters. In scenarios that have low requirements on latency and process a large number of low-resolution images, pass input parameters to **DvppCtl**, that is, the **DVPP_CTL_CMDLIST_PROC** command word and **IMAGE_CONFIG** structure parameters. **This method is not recommended.** It is used to be compatible with the functions in earlier versions and will be deleted in later versions.
- In the current version, the VENC function can be implemented in either of the following methods:
 - Method 1: Call **CreateVenc**, **RunVenc**, and **DestroyVenc** to implement the VENC function. **This method is recommended.**
 - Method 2: Call **CreateDvppApi**, **DvppCtl**, and **DestroyDvppApi** to implement the VENC function. Pass input parameters to **DvppCtl**, that is, the **DVPP_CTL_VENC_PROC** command word and **venc_in_msg** structure parameters. **This method is not recommended.** It is used to be compatible with the functions in earlier versions and will be deleted in later versions.

8.3.1.2 API List

You can view the header files of the APIs in the **ddk/include/inc/dvpp/** installation directory of the device development kit (DDK). If the APIs provided by the DVPP need to be called, the code can contain **idvppapi.h**, **Venc.h**, and **Vpc.h**.

For details about the header files that define data types, see [8.3.7.1 Structures in VpcUserImageConfigure](#).

Table 8-7 API list

Category	API	Function	Header File
Implementing the VPC, JPEGG, JPEGD, and PNGD functions	CreateDvppApi	Creates a DVPP API instance, which is equivalent to creating the handle to the DVPP executor. The caller can use the applied DVPP API instance to call DvppCtl to process an image, either across functions or across threads.	idvppapi.h
	DvppCtl	Controls the execution of DVPP modules, such as the VPC, JPEGG, JPEGD and PNGD. The DvppCtl API is called by using the instance created by CreateDvppApi .	
	DestroyDvppApi	Destroys the DVPP API instance created by calling CreateDvppApi and closes the DVPP executor.	
	DvppGetOutParameter	Obtains the output buffer size of the JPEGD/JPEGG/PNGD module.	
Implementing the VDEC function	CreateVdecApi	Obtains a VDEC API instance, which is equivalent to the handle to the VDEC executor. The caller can use the obtained VDEC API instance to call CreateVdecApi for video decoding. Cross-function calling and cross-thread calling are supported.	idvppapi.h

Category	API	Function	Header File
	VdecCtl	Controls the DVPP executor to implement video decoding. The VdecCtl API is called by using the instance created by CreateVdecApi .	
	DestroyVdecApi	Releases the VDEC API instance created by CreateVdecApi and closes the VDEC executor.	
Implementing the VENC function	CreateVenc	Obtains the VENC instance, which is equivalent to obtaining the handle of the VENC executor. The caller can call RunVenc to encode images by using the obtained VENC instance.	Venc.h
	RunVenc	Controls the DVPP executor to implement video encoding. The RunVenc API is called by using the instance created by CreateVenc .	
	DestroyVenc	Releases the VENC instance created by calling CreateVenc and closes the VENC executor.	

8.3.1.3 VPC Function

Function Description

The VPC module provides the following functions:

- **Cropping:** Crops a required area out of the input image.
- **Resizing**
 - In the VPC, image resizing can be classified into 8K image resizing and non-8K image resizing modes based on resolution:

- 8K image resizing: used to process the input image whose width or height is within the range of 4096–8192 pixels
- Non-8K image resizing: used to process the input image whose resolution is within the range of 32 x 6 pixels to 4096 x 4096 pixels
- Single-image cropping/resizing (supporting uncompressed format and HFBC compressed format) and single-image multi-ROI cropping/resizing (supporting uncompressed format and HFBC compressed format):
HFBC is a compressed image format for the VDEC output. In this format, the VDEC has better processing performance.
- Other resizing modes: for example, original image resizing and proportional image resizing.
- **Overlaying:** Crops an image out of an input image, resizes the cropped image, and places it in a specified area of the output image. The output image may be a blank image (when the output buffer allocated by the user is empty) or an existing image (when an image has been read into the output buffer allocated by the user). Note that the overlaying concept here refers only to the case when the output image is an existing image.
- **Stitching:** Crops multiple images out of an input image, resizes the cropped images, and place them in a specified area of the output image.
- **Format conversion**
 - Converts an RGB, YUV422, or YUV444 image to a YUV420 image.
 - Converts a color image to a grayscale image. For the output image, only the data of the Y component is used.

Restriction

- The following table describes the 8K image resizing and non-8K image resizing modes in the VPC based on resolution.

Input Image Resolution	Input Image Format	Width Stride x Height Stride Alignment for the Input Image	VPC Function	Output Image Resolution	Output Image Format	Width Stride x Height Stride Alignment for the Output Image
Width or height within the range of 4096–8192 pixels (excluding 4096)	YUV420 SP (NV12 and NV21)	2 x 2 alignment	8K image resizing	16 x 16 to 4096 x 4096	For details, see the outputFormat parameter in Table 8-10 .	2 x 2 alignment

Input Image Resolution	Input Image Format	Width Stride x Height Stride Alignment for the Input Image	VPC Function	Output Image Resolution	Output Image Format	Width Stride x Height Stride Alignment for the Output Image
32 x 6 to 4096 x 4096 pixels (including 4096)	For details, see the inputFormat parameter in Table 8-10 .	- For details about the width stride alignment, see the widthStride parameter in Table 8-10. - The height stride is 2-pixel aligned.	Non-8K image resizing	32 x 6 to 4096 x 4096	For details, see the outputFormat parameter in Table 8-10 .	16 x 2 alignment

- Resizing ratio of the width or height: [1/32, 16]
- 8K image resizing: Resizing is supported, the format conversion between YUV420SP NV12 and YUV420SP NV21 is supported, but cropping is not supported.
- Buffer restrictions:
 - The start address of the buffer must be 16-byte aligned.
 - In the same task, the virtual addresses of the input and output buffers must be in the same 4 GB space.

VPC Function Diagram

Figure 8-2 VPC function diagram (cropping+resizing+overlying)

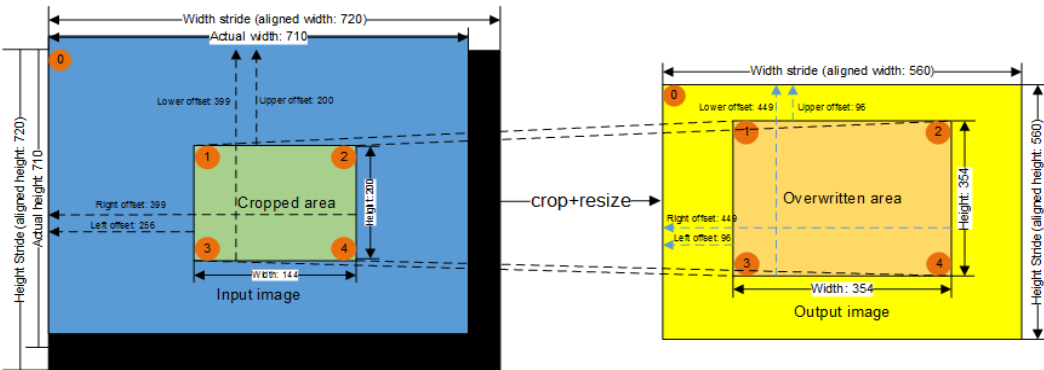


Figure 8-3 VPC function diagram (stitching)

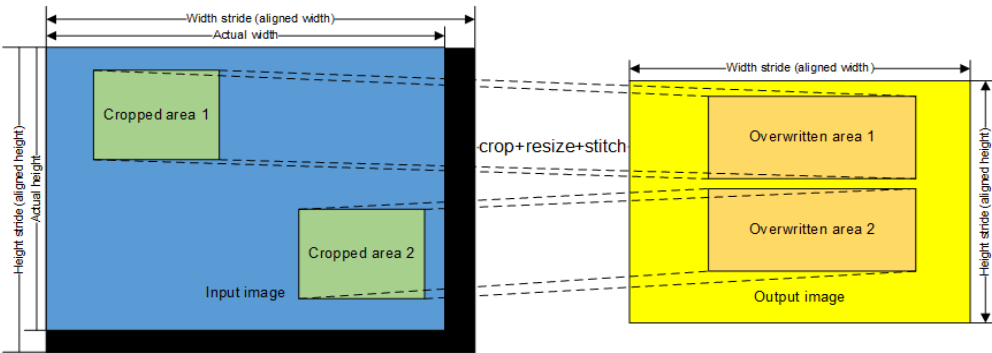


Table 8-8 Concepts

Concept	Description
Width stride	Step for a row of pixels, indicating the width of the input image after alignment. The width stride calculation of an RGB image is different from that of a YUV image. The minimum width stride is 32, and the maximum width stride is 4096 x 4 (the image width is 4096 and the image format is ARGB). - YUV400SP, YUV420SP, YUV422SP, and YUV444SP: Align the width of the input image to a 16-pixel boundary. - YUV422 packed: Multiply the width of the input image by 2 and align the result to a 16-pixel boundary. - YUV444 packed and RGB888: Multiply the width of the input image by 3 and align the result to a 16-pixel boundary. - XRGB8888: Multiply the width of the input image by 4 and align the result to a 16-pixel boundary. - HFBC format: The wide stride equals the width of the input image.

Concept	Description
Height stride	Number of lines in the buffer of an image, indicating the height of the input image after alignment. Value: The height of the input image must be 2-pixel aligned. The minimum height stride is 6, and the maximum height stride is 4096.
Top/ Bottom/ Left/ Right offset	You can configure the top offset, bottom offset, left offset, and right offset to implement the following functions: (1) Specify the position of the cropped area or overwritten area. (2) Control the width and height of the cropped area or overwritten area by using the following formulas: Right offset – Left offset + 1 = Width; Bottom offset – Top offset + 1 = Height. • Left offset: horizontal offset of points 1 and 3 in the cropped/overwritten area relative to point 0 in the input/output image • Right offset: horizontal offset of points 2 and 4 in the cropped/overwritten area relative to point 0 in the input/output image • Top offset: vertical offset of points 1 and 2 in the cropped/overwritten area relative to point 0 in the input/output image • Bottom offset: vertical offset of points 3 and 4 in the cropped/overwritten area relative to point 0 in the input/output image
Cropped area	Image area to be cropped. The minimum resolution is 10 x 6, and the maximum resolution is 4096 x 4096.
Overwritten area	Area specified by the user in the output image. The minimum resolution is 10 x 6, and the maximum resolution is 4096 x 4096. Restrictions: • For the overwritten area, the left offset and the top offset must be even numbers, and the right offset and the bottom offset must be odd numbers. • The cropped area cannot be larger than the input image, and the overwritten area cannot be larger than the output image. • The overwritten area can be directly mapped on the leftmost side of the output image, that is, the left offset of the output image is 0. • The maximum number of overwritten areas is 256. • The left offset of the overwritten area relative to the output image is 16-pixel aligned. • It is recommended that the width of the output overwritten area be 16-pixel aligned. If the width is not 16-pixel aligned, invalid extra data is written to make the width 16-pixel aligned.

Performance Specifications

- For **non-8K image resizing**, the VPC performance involves different scenarios such as image cropping and resizing. When the resolution changes during image processing, the performance is calculated based on the larger

resolution. For example, if the resolution of the image after resizing is greater than that before resizing, the former is used to calculate the performance specifications. If the resolution of the overwritten area is greater than that of the cropped area, the former is used to calculate the performance specifications. For YUV420 SP images, the performance specifications in typical scenarios are as follows:

Scenario	Total Frame Rate
1080p x n channels ($n < 4$, one channel corresponds to one thread)	$n \times 360$ fps
1080p x n channels ($n \geq 4$, one channel corresponds to one thread)	1440 fps
4K x n channels ($n < 4$, one channel corresponds to one thread)	$n \times 90$ fps
4K x n channels ($n \geq 4$, one channel corresponds to one thread)	360 fps

- For **8K image resizing**, the VPC performance is closely related to the output resolution. A higher output resolution indicates longer processing time and lower performance. The following table lists the performance specifications in typical scenarios (output resolution of 1080p or 4K and image format of YUV420 SP).

Scenario	Total Frame Rate
1080p x n channels ($n = 1$, one channel corresponds to one thread)	4 fps
1080p x n channels ($n \geq 4$, one channel corresponds to one thread)	16 fps
4K x n channels ($n = 1$, one channel corresponds to one thread)	1 fps
4K x n channels ($n \geq 4$, one channel corresponds to one thread)	4 fps

Reference

The following figure shows the component layout of the RBG and YUV images. Two YUV420SP images are used as examples for SP format images and an ARGB image is used as an example for packed and RGB images.

Format	Resolution	Width Stride	Height Stride	Buffer Size					
YUV420SP	4 x 4	4	4	24					
Buffer Layout									
y11	y12	y13	y14						
y21	y22	y23	y24						
y31	y32	y33	y34						
y41	y42	y43	y44						
u11	v11	u13	v13						
u31	v31	u33	v33						
Format	Resolution	Width Stride	Height Stride	Buffer Size					
YUV420SP	4 x 4	6	6	54					
Buffer Layout The letter x indicates invalid data.									
y11	y12	y13	y14	x	x				
y21	y22	y23	y24	x	x				
y31	y32	y33	y34	x	x				
y41	y42	y43	y44	x	x				
x	x	x	x	x	x				
x	x	x	x	x	x				
u11	v11	u13	v13	x	x				
u31	v31	u33	v33	x	x				
x	x	x	x	x	x				
Format	Resolution	Width Stride	Height Stride	Buffer Size					
ARGB	2 x 2	10	4	40					
Buffer Layout The letter x indicates invalid data.									
a11	r11	g11	b11	a12	r12	g12	b12	x	x
a21	b21	g21	b21	a22	r22	g22	b22	x	x
x	x	x	x	x	x	x	x	x	x
x	x	x	x	x	x	x	x	x	x

8.3.1.4 JPEG Function

Function and Restriction

The JPEG module is used to encode JPG images.

- The JPEG supports the following input formats:
 - YUV422 packed (YUYV, YVYU, UYVY, and VYUY)
 - YUV420 semi-planar (NV12 and NV21)
- The JPEG supports the following resolution range:

Maximum resolution: 8192 x 8192; minimum resolution: 32 x 32
- The JPEG output format is JPEG. Only Huffman encoding is supported. Arithmetic encoding and progressive encoding are not supported.
- Buffer restrictions:
 - The start address of the buffer must be 128-byte aligned.
 - In the same task, the virtual addresses of the input and output buffers must respectively be in the same 4 GB space.

Performance Specifications

Scenario	Total Frame Rate
1080p x n channels ($n \geq 1$)	64 fps
4K x n channels ($n \geq 1$)	16 fps

8.3.1.5 JPEGD Function

Function Description

The JPEGD module is used to decode .jpg, .jpeg, .JPG, and .JPEG images. For the image formats not supported by the hardware, software decoding is used.

After decoding, the images are output in the following formats:

JPEG(444) -> YUV444 / YUV420 semi-planar with the V component before the U component

JPEG(422) -> YUV422 / YUV420 semi-planar with the V component before the U component

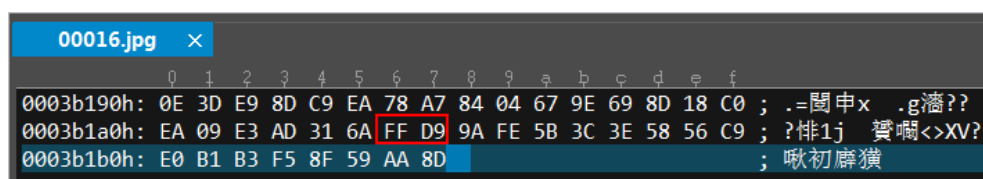
JPEG(420) -> YUV420 / YUV420 semi-planar with the V component before the U component

JPEG(400) -> YUV420/ UV data is padded by 0x80.

NOTICE

- The JPEGD uses the camel-case naming style due to the requirements of the programming standards. However, the original kernel-style input parameters and output parameters are also supported. Therefore, the JPEGD supports two sets of parameters. The camel-case naming style is recommended.
- If user-defined data exists after the end of image (EOI) flag **0xFFD9**, the JPEGD clears the 8-byte data after the EOI during decoding. If you want to retain the user-defined data, read it to the buffer and back it up before sending it to the JPEGD.

To check whether user-defined data exists after the EOI flag in an image, use the binary viewing tool to open the image. For example, user-defined data exists after the EOI flag **FFD9** in the following figure.



Restriction

- Restrictions on the input image:
 - Maximum resolution: 8192 x 8192; minimum resolution: 32 x 32
 - Only Huffman encoding is supported. The color space of the stream is YUV, and the subsample of the stream is 444, 422, 420, or 400.
 - Arithmetic coding is not supported.
 - The progressive JPEG format is not supported.
 - The JPEG2000 format is not supported.
- Hardware restrictions:

- Only a maximum of four Huffman tables are supported, including two direct coefficient (DC) tables and two alternating coefficient (AC) tables.
- Only a maximum of three quantization tables are supported.
- Only 8-bit sampling precision is supported.
- Only images after sequential encoding can be decoded.
- Only JPEG decoding based on discrete cosine transform (DCT) is supported.
- Only one start of scan (SOS) flag is supported for image decoding.
- Software restrictions:
 - A maximum of three SOS flags are supported for image decoding.
 - Abnormal image decoding with insufficient minimum coded unit (MCU) data is supported.
- Buffer restrictions:
 - The start address of the buffer must be 128-byte aligned.
 - In the same task, the virtual addresses of the input and output buffers must respectively be in the same 4 GB space.

Performance Specifications

The performance specifications of the JPEGD are based on the hardware decoding performance. The JPEGD hardware decoding does not support the image decoding with three SOS flags. For the image formats that are not supported by the hardware, software decoding is used. The software decoding performance for reference is 1080p x 1 channel at 15 fps.

Scenario	Total Frame Rate
1080p x 1 channel	128 fps
1080p x n channels ($n \geq 2$)	256 fps
4K x 1 channel	32 fps
4K x n channels ($n \geq 2$)	64 fps

8.3.1.6 PNGD Function

Function and Restriction

The PNGD module is used to implement hardware decoding for PNG images.

- The PNGD supports the following input formats:
RGBA and RGB
- The PNGD supports the following output formats:
RGBA and RGB
- The PNGD supports the following resolution range:
Maximum resolution: 4096 x 4096; minimum resolution: 32 x 32

Performance Specifications

Scenario	Total Frame Rate
1080p x 1 channel	4 fps
1080p x 2 channels	8 fps
1080p x 3 channels	12 fps
1080p x 4 channels	16 fps
1080p x 5 channels	20 fps
1080p x n channels ($n \geq 6$)	24 fps
4K x 1 channel	1 fps
4K x 2 channels	2 fps
4K x 3 channels	3 fps
4K x 4 channels	4 fps
4K x 5 channels	5 fps
4K x n channels ($n \geq 6$)	6 fps

8.3.1.7 VDEC Function

Function and Restriction

The VDEC module is used to decode videos.

- The VDEC supports the following input formats:
H.264 BP/MP/HP Level 5.1 YUV420SP encoded streams
H.265 8-Bit/10-Bit Level 5.1 YUV420SP encoded streams
- The VDEC supports the following resolution range:
Maximum resolution: 4096 x 4096; minimum resolution: 128 x 128
- The VDEC output is HFBC compressed YUV420SP data.
HFBC is a compressed image format for the VDEC output. In this format, the VDEC has better processing performance.
- Bad frames or frame loss in the streams may cause VDEC frame loss.
- The VDEC cannot decode the streams encoded in interlaced scanning mode.
- The VDEC module does not use the singleton pattern. Therefore, the C APIs provided by the VDEC module are different for those provided by other modules.

Performance Specifications

Scenario	Total Frame Rate
1080p x n channels ($2 \leq n \leq 16$)	480 fps
1080p x 1 channel	240 fps
4K x n channels ($2 \leq n \leq 16$)	120 fps
4K x 1 channel	60 fps

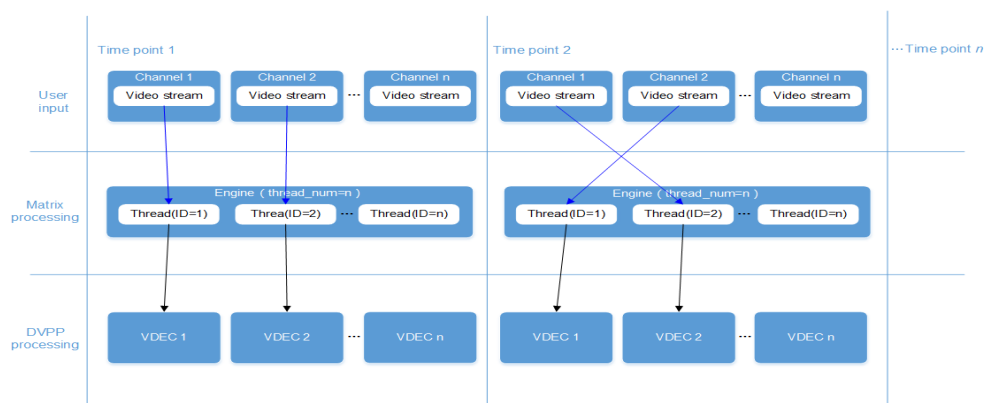
If Matrix is used to orchestrate the application process, when DVPP is used to decode multiple channels of input video streams, because data of each frame in the video streams is associated, each VDEC is required to fixedly correspond to one video stream to ensure the data sequence of each frame in a same video stream. Otherwise, the VDEC in DVPP cannot decode video streams.

- When multiple video streams are decoded, multiple implementation modes may be available to ensure the data sequence of each frame in the video streams. The following configurations are recommended:
 - In the graph configuration file, configure multiple VDEC engines in the graphs segment. One video stream corresponds to one engine.
 - In the graph configuration file, set **thread_num** to **1** in the VDEC engine segment. One engine corresponds to one thread.

NOTE

- For details about the functions of Matrix and the configuration of the graph configuration file, see *Matrix API Reference*.
- One graph (one thread) can correspond to a maximum of 16 video streams.
- If only one VDEC engine is configured in the graph configuration file and **thread_num** is set to **n** (the value of **n** is the number of video stream channels) during multi-channel video stream decoding, video streams in Matrix may be processed in different threads at different time points. However, in DVPP, one VDEC requires one fixed channel of video stream data. One channel of video stream data corresponds to one thread. In this case, the data sequence of each frame in the video streams may not be ensured during video decoding, as shown in the figure.

Figure 8-4 VDEC process



8.3.1.8 VENC Function

The VENC module implements encoding in three steps: instantiation, encoding, and resource release.

Function and Restriction

The VENC module is used to encode YUV420 and YVU420 image data.

- The VENC supports the following input formats:
YUV420 semi-planar NV12/NV21-8bit
- The VENC supports the following resolution range:
Maximum resolution: 1920 x 1920; minimum resolution: 128 x 128
- The VENC supports the following output formats:
H.264 BP/MP/HP
H.265 MP (Only slice streams are supported.)

Performance Specifications

Scenario	Total Frame Rate
1080p x n channels (one process corresponds to one channel)	30 fps

Note: The performance at other resolutions can be estimated in the same way.

8.3.1.9 Input and Output Memory

For details about the **HIAI_DMAlloc/HIAI_DFree** and **HIAI_DVPP_DMAlloc/HIAI_DVPP_DFree** APIs as well as the graph configuration file (**receive_memory_without_dvpp** parameter), see *Matrix API Reference*.

Table 8-9 Memory requirements

Module	Input Memory	Output Memory
VPC	Use the APIs provided by Matrix to allocate or free memory. • Use HIAI_DVPP_DMAlloc to allocate memory, which must meet the DVPP requirement (the start address of the memory must be 16-byte aligned). • Use HIAI_DVPP_DFree to free memory.	Use the APIs provided by Matrix to allocate or free memory. • Use HIAI_DVPP_DMAlloc to allocate memory, which must meet the DVPP requirement (the start address of the memory must be 16-byte aligned). • Use HIAI_DVPP_DFree to free memory.

Module	Input Memory	Output Memory
JPEGE and JPEG D	Use the APIs provided by Matrix to allocate or free memory. • Use HIAI_DVPP_DMAlloc to allocate memory, which must meet the DVPP requirement (the start address of the memory must be 128-byte aligned). • Use HIAI_DVPP_DFree to free memory.	• If the output memory is specified by the user, it should also be freed by the user. Use the APIs provided by Matrix to allocate or free memory. – Use HIAI_DVPP_DMAlloc to allocate memory, which must meet the DVPP requirement (the start address of the memory must be 128-byte aligned). Before memory allocation, call DvppGetOutParameter to obtain the output memory size. – Use HIAI_DVPP_DFree to free memory. • If the output memory is not specified by the user, DVPP allocates the memory internally. The cbFree() callback function in the JPEGE or JPEGD output parameter structure should be called to free the memory and set the memory address pointer to null.
PNG D	Use the APIs provided by Matrix to allocate or free memory. • Use HIAI_DVPP_DMAlloc to allocate memory, which must meet the DVPP requirement (the start address of the memory must be 128-byte aligned). • Use HIAI_DVPP_DFree to free memory.	If the output memory is specified by the user, it should also be freed by the user. Use the APIs provided by Matrix to allocate or free memory. • Use HIAI_DVPP_DMAlloc to allocate memory, which must meet the DVPP requirement (the start address of the memory must be 128-byte aligned). Before memory allocation, call DvppGetOutParameter to obtain the output memory size. • Use HIAI_DVPP_DFree to free memory.

Module	Input Memory	Output Memory
VDEC and VENC	There is no requirement on the memory. Native APIs such as malloc/free and new/delete can be called to allocate or free the memory. The HIAI_DMAlloc/HIAI_DFree and HIAI_DVPP_DMAlloc/HIAI_DVPP_DFree APIs provided by Matrix can also be called to allocate or free the memory. The receive_memory_without_dvpp parameter provided by Matrix can be used to control whether the memory is used by DVPP.	The HFBC data output by the VDEC is directly used as the input of the VPC. The VENC output memory is internally managed by the DVPP. You can copy the data in the output memory when using the VENC.

8.3.2 VPC/JPEGE/JPEGD/PNGD Function Interfaces

8.3.2.1 CreateDvppApi

Syntax	int CreateDvppApi(IDVPPAPI*& pIDVPPAPI)
Function	Creates a DVPP API instance, which is equivalent to creating the handle to the DVPP executor. The caller can use the applied DVPP API instance to call DvppCtl to process an image, either across functions or across threads.
Input	IDVPPAPI pointer reference. The input pointer must be NULL .
Output	IDVPPAPI pointer reference. If a DVPP API instance fails to be obtained, the output pointer is NULL . If a DVPP API instance is successfully obtained, the output pointer is not NULL .
Return Value	• 0: success • -1: failure
Instructions	The caller creates the IDVPPAPI object pointer, which is initialized to NULL . The IDVPPAPI object pointer is passed by calling the CreateDvppApi function. If the application is successful, the CreateDvppApi function returns the DVPP API instance. Otherwise, NULL is returned. The caller needs to verify the return value.
Restriction	The caller is responsible for the life cycle of the DVPP API instance, including the application and release, which are implemented by using CreateDvppApi and DestroyDvppApi , respectively.

Calling Example

```
IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);
```

8.3.2.2 DvppCtl

8.3.2.2.1 API Description

Syntax	int DvppCtl(IDVPPAPI*& pIDVPPAPI, int CMD, dvppapi_ctl_msg* MSG)
Function	Controls the execution of DVPP modules, such as the VPC, JPEG, JPEGD and PNGD. The DvppCtl API is called by using the instance created by CreateDvppApi .
Input	• IDVPPAPI pointer reference • Command word (CMD) • DVPP executor configuration information (dvppapi_ctl_msg type). Different DVPP modules have different configuration information. Therefore, the corresponding configuration information needs to be passed when the functions of each module are invoked.
Output	The output parameters vary according to the functions of the DVPP submodules. For details, see dvppapi_ctl_msg .
Return Value	• 0: success • -1: failure
Instructions	The caller calls the DvppCtl function to pass the IDVPPAPI object pointer, the correct command word (CMD) , and the configured dvppapi_ctl_msg .
Restriction	None

Command Words (CMD)

The CMDs are defined in the **ddk/include/inc/dvpp/DvppCommon.h** file in the DDK installation directory.

Member Variable	Description	Value Range
DVPP_CTL_VPC_PROC	VPC function command word	0
DVPP_CTL_PNGD_PROC	PNGD function command word	1
DVPP_CTL_JPEGE_PROC	JPEGE function command word	2

Member Variable	Description	Value Range
DVPP_CTL_JPEGD_PROC	JPEGD function command word	3
DVPP_CTL_VENC_PROC	Reserved	5
DVPP_CTL_DVPP_CAPABILITY	Command word for querying the DVPP capability	6
DVPP_CTL_CMDLIST_PROC	Reserved	7
DVPP_CTL_TOOL_CASE_GET_RESIZE_PARAM	Reserved	8

dvppapi_ctl_msg

This type is defined in the **ddk/include/inc/dvpp/DvppCommon.h** file in the DDK installation directory.

Member Variable	Description	Value Range
int32_t in_size	Input parameter size	-
int32_t out_size	Output parameter size	-
void *in	Input parameter	- Input parameter of the VPC function: VpcUserImageConfigure - Input parameter of the JPEGE function: SJpegIn - Input parameter of the JPEGD function: JpegdIn (camel-case-style JPEGD input structure, which is recommended) - Input parameter of the JPEGD function: jpegd_raw_data_info (kernel-style JPEGD input structure, which is not recommended) - Input parameter of the PNGD function: PngInputInfoAPI - Input parameter for querying the DVPP engine: device_query_req_stru

Member Variable	Description	Value Range
void *out	Output parameter	- Output parameter of the VPC function: none - Output parameter of the JPEGE function: sJpegeOut - Output parameter of the JPEGD function: JpegdOut (camel-case-style JPEGD output structure, which is recommended) - Output parameter of the JPEGD function: jpegd_yuv_data_info (kernel-style JPEGD output structure, which is not recommended) - Output parameter of the PNGD function: PngOutputInfoAPI - Output parameter for querying the DVPP engine: dvpp_engine_capability_stru

8.3.2.2.2 VPC Parameters

Input Parameter: VpcUserImageConfigure

Table 8-10 Input parameter VpcUserImageConfigure

Member Variable	Description	Value Range
uint8_t* bareDataAddr	Address of an uncompressed input image. If the input image is compressed, this parameter does not need to be set and the default value is null. You are advised to allocate the buffer by calling HIAI_DVPP_DMAlloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 16-pixel aligned. For details about HIAI_DVPP_DMAlloc, see <i>Matrix API Reference</i>.	-

Member Variable	Description	Value Range
uint32_t bareDataBuf ferSize	Buffer size of an uncompressed input image. The buffer size is calculated based on the width stride and height stride of the image. The buffer size pointed by bareDataAddr must be the same as the value of bareDataBufferSize . When the input image is not compressed, this parameter does not need to be set and the default value is 0.	For the uncompressed format, the buffer size is calculated as follows based on the image format: • YUV400SP and YUV420SP: widthStride x heightStride x 3/2 • YUV422SP: widthStride x heightStride x 2 • YUV444SP: widthStride x heightStride x 3 • YUV422 packed: widthStride x heightStride • YUV444 packed and RGB888: widthStride x heightStride • XRGB8888: widthStride x heightStride

Member Variable	Description	Value Range
uint32_t widthStride	Width stride of the image	The minimum width stride is 32, and the maximum width stride is 4096 x 4 (the image width is 4096 and the image format is ARGB). For 8K image resizing, the width stride must be 2-pixel aligned. For non-8K image resizing, the formula for calculating widthStride varies according to the image format. - YUV400SP, YUV420SP, YUV422SP, and YUV444SP: Align the width of the input image to a 16-pixel boundary. - YUV422 packed: Align the width of the input image to a 16-pixel boundary and multiply the result by 2 (the width is 16-pixel aligned and each pixel occupies 2 bytes). - YUV444 packed and RGB888: Align the width of the input image to a 16-pixel boundary and multiply the result by 3 (the width is 16-pixel aligned and each pixel occupies 3 bytes). - XRGB8888: Align the width of the input image to a 16-pixel boundary and multiply the result by 4 (the width is 16-pixel aligned and each pixel occupies 4 bytes). - HFBC format: The wide stride equals the width of the input image.
uint32_t heightStride	Height stride. You can calculate the UV data start addresses of YUV SP images based on this parameter.	Value: The height of the input image must be 2-pixel aligned. The minimum height stride is 6, and the maximum height stride is 4096.

Member Variable	Description	Value Range
enum VpcInputFormat inputFormat	Input image format	enum VpcInputFormat { INPUT_YUV400, // 0 INPUT_YUV420_SEMI_PLANNER_UV, // 1 INPUT_YUV420_SEMI_PLANNER_VU, // 2 INPUT_YUV422_SEMI_PLANNER_UV, // 3 INPUT_YUV422_SEMI_PLANNER_VU, // 4 INPUT_YUV444_SEMI_PLANNER_UV, // 5 INPUT_YUV444_SEMI_PLANNER_VU, // 6 INPUT_YUV422_PACKED_YUYV, // 7 INPUT_YUV422_PACKED_UYVY, // 8 INPUT_YUV422_PACKED_YVYU, // 9 INPUT_YUV422_PACKED_VYUY, // 10 INPUT_YUV444_PACKED_YUV, // 11 INPUT_RGB, // 12, input image format: RGB888 INPUT_BGR, // 13, input image format: BGR888 INPUT_ARGB, // 14. The component sequence of an input image in this format during storage is similar to RGB888. A indicates transparency. INPUT_ABGR, // 15. The component sequence of an input image in this format during storage is similar to BGR888. A indicates transparency. INPUT_RGBA, // 16. The component sequence of an input image in this format during storage is similar to

Member Variable	Description	Value Range
		RGB888. A indicates transparency. INPUT_BGRA, // 17. The component sequence of an input image in this format during storage is similar to BGR888. A indicates transparency. INPUT_YUV420_SEMI_PLANNER_UV_10BIT, // 18 INPUT_YUV420_SEMI_PLANNER_VU_10BIT, // 19 };
enum VpcOutputFormat outputFormat	Output image format	enum VpcOutputFormat { OUTPUT_YUV420SP_UV, OUTPUT_YUV420SP_VU };
VpcUserRoiConfigure* roiConfigure	Cropped area configuration. For details, see VpcUserRoiConfigure structure .	-
bool isCompressData	Whether the HFBC compressed image format of the VDEC output is used	When the image comes from the VDEC, set this parameter to true. For other image formats, set this parameter to false. The default value is false. The HFBC compression format supports single-image cropping/resizing and single-image multi-ROI cropping/resizing.
VpcCompressDataConfigure compressDataConfigure	Data configuration of the HFBC compressed image output by the VDEC. For details, see VpcCompressDataConfigure structure .	-

Member Variable	Description	Value Range
bool yuvSumEnable	Whether to calculate the yuvSum value. Calculation of the yuvSum value supports only the single-image single-ROI mode.	When the yuvSum value needs to be calculated, set this parameter to true . Otherwise, set this parameter to false . The default value is false . The restrictions for calculating the yuvSum value are as follows: 1. The input image resolution is less than or equal to 4096 x 4096. 2. The start point of the overwritten area is (0, 0). 3. The input is a single image with a single ROI.
VpcUserYuvSum yuvSum	yuvSum calculation configuration. For details, see VpcUserYuvSum structure .	-
VpcUserPerformanceTuningParameter tuningParameter	Reserved. This parameter is used for parameter tuning. For details, see VpcUserPerformanceTuningParameter .	-
uint32_t* cmdListBufferAddr	Reserved	-
uint32_t cmdListBufferSize	Reserved	-
uint64_t yuvScalerParaSetAddr	Reserved. This parameter indicates the file path and file name array of the filtering parameter set. NOTE - The parameter set must be on the device. - Ensure that the input file path is correct.	A parameter set file path consists of an absolute file path on the device side and the file name, for example, "/root/share/vpc/YUVScaler_pra.h" .

Member Variable	Description	Value Range
uint16_t yuvScalerParaSetSize	Reserved. This parameter indicates the number of elements in the parameter set array to which the parameter set address points. The default value is 1 .	$1 \leq \text{yuvScaler_paraset_size} \leq 10$
uint16_t yuvScalerParaSetIndex	Reserved. This parameter indicates the index of yuvScalerParaSetAddr corresponding to yuvScalerParaSetIndex (The default value is 0 .)	$0 \leq \text{yuvScalerParaSetIndex} < 10$
uint8_t isUseMultiCoreAccelerate	Reserved	-
uint8_t reserve1	Reserved	-
uint16_t reserve2;	Reserved	-

Output Parameters

None

8.3.2.2.3 JPEG Parameters

Input Parameter: sJpegIn

Table 8-11 Input parameter sJpegIn

Member Variable	Description
eEncodeFormat format	Format of the YUV input data. YUV422 packed (YUYV, YVYU, UYVY, and VYUY) and YUV420 semi-planar (NV12 and NV21) are supported. <pre>typedef enum { JPGENC_FORMAT_UYVY = 0x0, JPGENC_FORMAT_VYUY = 0x1, JPGENC_FORMAT_YVYU = 0x2, JPGENC_FORMAT_YUYV = 0x3, JPGENC_FORMAT_NV12 = 0x10, JPGENC_FORMAT_NV21 = 0x11, } eEncodeFormat;</pre>
unsigned char* buf	The YUV input data needs to be allocated by the caller and must be properly aligned. For details, see the information about the stride and heightAligned parameters. You are advised to allocate the buffer by calling HIAI_DVPP_DMalloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMalloc, see <i>Matrix API Reference</i>. NOTICE When HIAI_DVPP_DMalloc is used to allocate a buffer, ensure that the size of the allocated buffer is the same as the value of the input parameter bufSize.
uint32_t bufSize	Length of the input buffer, that is, the length of the data after the width and height are aligned
uint32_t width	Width of the input image. The value range is [32, 8192].

Member Variable	Description
uint32_t height	Height of the input image. The value range is [32, 8192].
uint32_t stride	Width of the input image after alignment. The width alignment boundary can be 16 pixels or a multiple of 16 pixels, for example, 128 pixels. For YUV422 packed data, the stride must be the result after the width is multiplied by 2 and then aligned to a 16-pixel boundary.
uint32_t heightAligned	1. The value can be the same as the height. 2. The value can be the aligned image height after Matrix cross-side transmission. 3. The value supports 16-pixel alignment.
uint32_t level	Encoding quality in a range of [0, 100]. The encoding quality of level 0 is similar to that of level 100. For the value range of [1, 100], a smaller value indicates poorer output image quality.

Output Parameter: sJpegeOut

Table 8-12 Output parameter sJpegeOut

Member Variable	Description
unsigned char* jpgData	Start address of the JPG data in the output buffer You are advised to allocate the buffer by calling HIAI_DVPP_DMAlloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMAlloc, see <i>Matrix API Reference</i>. NOTICE When HIAI_DVPP_DMAlloc is used to allocate a buffer, ensure that the size of the allocated buffer is the same as the value of the input parameter jpgSize.
uint32_t jpgSize	Data length of the encoded JPG image

Member Variable	Description
JpegCalBackFree cbFree	Callback function for freeing the output buffer • If the output buffer is specified by the user, it should also be freed by the user. • If the output buffer is not specified by the user, DVPP allocates the buffer internally. The cbFree() callback function should be called to free the buffer, and jpgData needs to be set to a null pointer. For details about the calling example, see 8.3.6.2 Implementing the JPEG Function.

8.3.2.2.4 JPEGD Parameters

Input Parameter: JpegdIn

Table 8-13 Input parameter JpegdIn

Member Variable	Description
unsigned char* jpegData	JPG input image data. The start address is 128-byte aligned and the buffer is allocated by using the 2 MB huge page table. In the same task, the virtual addresses of the input and output buffers must be in the same 4 GB space. You are advised to allocate the buffer by calling HIAI_DVPP_DMAlloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMAlloc, see <i>Matrix API Reference</i>. NOTICE When HIAI_DVPP_DMAlloc is used to allocate memory, ensure that the size of the allocated memory is the same as the value of the input parameter jpegDataSize.

Member Variable	Description
uint32_t jpegDataSize	Length of the input buffer. Use HIAI_DVPP_DMalloc provided by Matrix to allocate the buffer. The size of the allocated buffer is (Actual data size + 8 bytes). 8 bytes is the hardware restriction requirement.
bool isYUV420Need	Whether to output data in YUV420 semi-planar format. • true: Yes • false: No. The output is in original format. The JPEGD supports raw format output (including YUV420SP, YUV422SP, and YUV444SP) and down-sampling YUV420 semi-planar output. YUV420 grayscale image output is in fake420 format.
bool isVBeforeU	Reserved. This parameter must be set to true . The V component is before the U component.

Output Parameter: JpegdOut

Table 8-14 Output parameter JpegdOut

Member Variable	Description
unsigned char* yuvData	Buffer for the output YUV data. The width and height of the image are values after alignment based on 128 x 16. You are advised to allocate the buffer by calling HIAI_DVPP_DMalloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMalloc , see <i>Matrix API Reference</i> . NOTICE When HIAI_DVPP_DMalloc is used to allocate a buffer, ensure that the size of the allocated buffer is the same as the value of the input parameter yuvDataSize .
uint32_t yuvDataSize	Length of the output YUV data. The data length is calculated based on the aligned width and height or can be obtained by calling DvppGetOutParameter .

Member Variable	Description
uint32_t imgWidth	Width of the output YUV image
uint32_t imgHeight	Height of the output YUV image
uint32_t imgWidthAligned	Width of the output image, which is 128-pixel aligned
uint32_t imgHeightAligned	Height of the output image, which is 16-pixel aligned
JpegCalBackFree cbFree	Callback function for freeing the output buffer • If the output buffer is specified by the user, it should also be freed by the user. • If the output buffer is not specified by the user, DVPP allocates the buffer internally. The cbFree() callback function should be called to free the buffer, and yuvData needs to be set to a null pointer. For details about the calling example, see 8.3.6.3 Implementing the JPEGD Function.
jpegd_color_space outFormat	Format of the output YUV data. <pre>enum jpegd_color_space{ DVPP_JPEG_DECODE_OUT_UNKNOWN = -1, DVPP_JPEG_DECODE_OUT_YUV444 = 0, DVPP_JPEG_DECODE_OUT_YUV422_H2V1 = 1, /* YUV422 */ DVPP_JPEG_DECODE_OUT_YUV422_H1V2 = 2, /* YUV440 */ DVPP_JPEG_DECODE_OUT_YUV420 = 3, DVPP_JPEG_DECODE_OUT_YUV400 = 4, DVPP_JPEG_DECODE_OUT_FORMAT_MAX, };</pre>

Input Parameter: jpegd_raw_data_info

Table 8-15 Input parameter jpegd_raw_data_info

Member Variable	Description
unsigned char* jpeg_data	Data of the input JPG image. The start address is 128-byte aligned, and the 2 MB huge page table is used for allocation. You are advised to allocate the buffer by calling HIAI_DVPP_DMalloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMalloc, see <i>Matrix API Reference</i>. NOTICE When HIAI_DVPP_DMalloc is used to allocate a buffer, ensure that the size of the allocated buffer is the same as the value of the input parameter jpeg_data_size.
uint32_t jpeg_data_size	Length of the input buffer. Use HIAI_DVPP_DMalloc provided by Matrix to allocate the buffer. The size of the allocated buffer is (Actual data size + 8 bytes). 8 bytes is the hardware restriction requirement.
jpegd_raw_format in_format	YUV sampling format of the input image. Retain the default value. <pre>enum jpegd_raw_format{ DVPP_JPEG_DECODE_RAW_YUV_UNSupport = -1, DVPP_JPEG_DECODE_RAW_YUV444 = 0, DVPP_JPEG_DECODE_RAW_YUV422_H2V1 = 1, // 422 // YUV440 is not supported anymore. This field is reserved. DVPP_JPEG_DECODE_RAW_YUV422_H1V2 = 2, // 440 DVPP_JPEG_DECODE_RAW_YUV420 = 3, DVPP_JPEG_DECODE_RAW_MAX, };</pre>

Member Variable	Description
bool IsYUV420Need	Whether to output data in YUV420 semi-planar format. • true: Yes • false: No. The output is in original format. The JPEGD supports raw format output (including YUV420SP, YUV422SP, and YUV444SP) and down-sampling YUV420 semi-planar output. YUV420 grayscale image output is in fake420 format.
bool isVBeforeU	Reserved. This parameter must be set to true . The V component is before the U component.

Output Parameter: jpegd_yuv_data_info

Table 8-16 Output parameter jpegd_yuv_data_info

Member Variable	Description
unsigned char* yuv_data	Buffer for the output YUV data. The width and height of the image are aligned values. NOTICE The buffer is allocated and managed by DVPP and cannot be specified by users.
uint32_t yuv_data_size	Length of the output YUV data. The data length is calculated based on the aligned width and height.
uint32_t img_width	Width of the output YUV image
uint32_t img_height	Height of the output YUV image
uint32_t img_width_aligned	Width of the output image, which is 128-pixel aligned
uint32_t img_height_aligned	Height of the output image, which is 16-pixel aligned
JpegCalBackFree cbFree	Callback function for freeing the output buffer DVPP internally allocates buffers. You need to call the cbFree() callback function to free buffers and set yuv_data to a null pointer. For details about the calling example, see 8.3.6.3 Implementing the JPEGD Function .

Member Variable	Description
enum jpegd_color_space out_format	Format of the output YUV data. enum jpegd_color_space { DVPP_JPEG_DECODE_OUT_UNKNOWN = -1, DVPP_JPEG_DECODE_OUT_YUV444 = 0, DVPP_JPEG_DECODE_OUT_YUV422_H2V1 = 1, // 422 // YUV440 is not supported anymore. This field is reserved. DVPP_JPEG_DECODE_OUT_YUV422_H1V2 = 2, // 440 DVPP_JPEG_DECODE_OUT_YUV420 = 3, DVPP_JPEG_DECODE_OUT_YUV400 = 4, DVPP_JPEG_DECODE_OUT_FORMAT_MAX, };

8.3.2.2.5 PNGD Parameters

Input Parameter: PngInputInfoAPI

Table 8-17 Input parameter PngInputInfoAPI

Member Variable	Description
void* inputData	Address of the input image data. You are advised to allocate the buffer by calling HIAI_DVPP_DMalloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMalloc , see <i>Matrix API Reference</i> . NOTICE When HIAI_DVPP_DMalloc is used to allocate a buffer, ensure that the size of the allocated buffer is the same as the value of the input parameter inputSize .
uint64_t inputSize	Input buffer length, which is used to verify the input data

Member Variable	Description
void* address	Address of the input image data. The current version does not support this parameter.
uint64_t size	Length of the input buffer. The current version does not support this parameter.
int32_t transformFlag	Transform flag. The value 1 indicates that RGBA is converted into RGB, whereas the value 0 indicates that the original format is retained.

Output Parameter: PngOutputInfoAPI

Table 8-18 Output parameter PngOutputInfoAPI

Member Variable	Description
void* outputData	Address of the output image data. The current version does not support this parameter.
uint64_t outputSize	Address of the output image data. The current version does not support this parameter.
void* address	Address of the output buffer. You are advised to allocate the buffer by calling HIAI_DVPP_DMalloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 128-pixel aligned. For details about HIAI_DVPP_DMalloc, see <i>Matrix API Reference</i>. NOTICE When HIAI_DVPP_DMalloc is used to allocate a buffer, ensure that the size of the allocated buffer is the same as the value of the input parameter outputSize.
uint64_t size	Buffer size
int32_t format	Output image format • 2: RGB output • 6: RGBA output

Member Variable	Description
uint32_t width	Output image width
uint32_t high	Output image height
uint32_t widthAlign	Width alignment. Align the size of the buffer occupied by a line of image data to a 128-byte boundary. If the output format is RGB, multiply the width by 3 and align the result to a 128-byte boundary. If the output format is RGBA, multiply the width by 4 and align the result to a 128-byte boundary.
uint32_t highAlign	Height alignment. Currently, 16-byte alignment is used.

8.3.2.2.6 Parameters for Querying the DVPP Engine

Function

The parameters are used to query the DVPP engine capabilities, including the capabilities, resolution restrictions, and performance parameters of all modules.

Input Parameter: `device_query_req_stru`

Table 8-19 Input parameter `device_query_req_stru`

Member Variable	Description	Value Range
uint32_t module_id	Module ID	The value is fixed at 1.
uint32_t engine_type	DVPP engine type	VDEC: 0 JPEGD: 1 PNGD: 2 JPEGE: 3 VPC: 4 VENC: 5

Output Parameter: dvpp_engine_capability_stru

Table 8-20 Output parameter dvpp_engine_capability_stru

Member Variable	Description	Value Range
int32_t engine_type	DVPP engine type	VDEC: 0 JPEGD: 1 PNGD: 2 JPEGE: 3 VPC: 4 VENC: 5
struct dvpp_resolution_stru max_resolution	Maximum resolution	For details, see 8.3.7.4 Structures in dvpp_engine_capability_s tru.
struct dvpp_resolution_stru min_resolution;	Minimum resolution	For details, see 8.3.7.4 Structures in dvpp_engine_capability_s tru.
uint32_t protocol_num;	Number of standard protocol types supported by the engine	VDEC: 5 JPEGD: 1 PNGD: 1 JPEGE: 1 VPC: 0 VENC: 4

Member Variable	Description	Value Range
uint32_t protocol_type[DVPP_PROTOCOL_TYPE_MAX];	Table of the standard protocol types supported by the engine	<i>enum</i> dvpp_proto_type { dvpp_proto_unsupport = -1, dvpp_itu_t81, iso_iec_15948_2003, h265_main_profile_level_5_1_hightier, h265_main_10_profile_level_5_1_hightier, h264_main_profile_level_5_1, h264_baseline_profile_level_5_1, h264_high_profile_level_5_1, h264_high_profile_level_4_1, h264_main_profile_level_4_1, h264_baseline_profile_level_4_1, h265_main_profile_level_4_1 };
uint32_t input_format_num;	Number of supported input formats	VDEC: 2 JPEGD: 1 PNGD: 1 JPEGE: 6 VPC: 51 VENC: 2
struct dvpp_format_unit_struct engine_input_format_table[DVPP_VADIO_FORMAT_MAX];	Table of the input formats supported by the engine	For details, see 8.3.7.4 Structures in dvpp_engine_capability_struct .
uint32_t output_format_num;	Number of supported output formats	VDEC: 4 JPEGD: 4 PNGD: 2 JPEGE: 1 VPC: 2 VENC: 2

Member Variable	Description	Value Range
struct dvpp_format_unit_stru engine_output_format_t able[DVPP_VADIO_FOR MAT_MAX];	Table of the output formats supported by the engine	For details, see 8.3.7.4 Structures in dvpp_engine_capability_s tru.
uint32_t performance_mode_num ;	Number of performance modes	The value is fixed at 1.
struct dvpp_performance_unit_s tru performance_mode_tabl e[DVPP_PERFOMANCE_ MODE_MAX];	Structure of the performance in the module	For details, see 8.3.7.4 Structures in dvpp_engine_capability_s tru.
struct dvpp_pre_contraction_str u pre_contraction;	Structure of pre- contraction information	For details, see 8.3.7.4 Structures in dvpp_engine_capability_s tru.
struct dvpp_pos_scale_stru pos_scale;	Structure of post- scaling information	For details, see 8.3.7.4 Structures in dvpp_engine_capability_s tru.
uint32_t spec_input_num	Number of types of the input formats. Currently, the following two types are supported: • HFBC • YUV or RGB	-
struct dvpp_vpc_data_spec_stru spec_input[DVPP_DATA_ INPUT_SPEC_TYPE_MAX]	Description of the types of the input formats	For details, see dvpp_vpc_data_spec_stru. ...

8.3.2.3 DestroyDvppApi

Syntax	int DestroyDvppApi(IDVPPAPI*& pIDVPPAPI)
Function	Destroys the DVPP API instance created by calling CreateDvppApi and closes the DVPP executor.
Input	IDVPPAPI pointer reference
Output	None

Return Value	• 0: success • -1: failure
Instructions	None
Restriction	To continue to invoke DVPP after DestroyDvppApi is called, call CreateDvppApi to create a DVPP API instance.

Calling Example

```
DestroyDvppApi(pidvppapi);
```

8.3.2.4 DvppGetOutParameter

Syntax	int32_t DvppGetOutParameter(void* in, void* out, int32_t cmd)
Function	Obtains the output buffer size of the JPEGD/JPEGE/PNGD module.
Input	• in pointer, which varies according to the module. For details, see Table 8-21. • out pointer, which varies according to the module. For details, see Table 8-21. • Command word (CMD)
Output	Output buffer size of each module: • For the JPEGE function, the output buffer size is the value of jpgSize in the sJpegOut struct. • For the JPEGD function, the output buffer size is the value of yuvDataSize in the JpegdOut struct. • For the PNGD function, the output buffer size is the value of outputSize in the PngOutputInfoAPI struct.
Return Value	• 0: success • -1: failure
Instructions	The caller calls the DvppGetOutParameter function, passes the in and out pointers and the correct CMD command word. The calling example is as follows: • For details about the JPEGD module, see 8.3.6.3 Implementing the JPEGD Function. • For details about the JPEGE module, see 8.3.6.2 Implementing the JPEGE Function. • For details about the PNGD module, see 8.3.6.4 Implementing the PNGD Function.
Restriction	None

Table 8-21 Input parameter description

Input Parameter	Description	Value Range
void* in	Pointer to the input structure	- Input parameter of the JPEG function: sJpegIn - Input parameter of the JPEGD function: JpegdIn - Input parameter of the PNGD function: PngInputInfoAPI NOTICE This function does not support the jpegd_raw_data_info structure of the JPEGD.
void* out	Pointer to the output structure	- Output parameter of the JPEG function: sJpegOut - Output parameter of the JPEGD function: JpegdOut - Output parameter of the PNGD function: PngOutputInfoAPI NOTICE This function does not support the jpegd_yuv_data_info structure of the JPEGD.
int32_t cmd	Command word for obtaining output parameters	- Command word of the JPEG function: GET_JPEG_OUT_PARAMETER - Command word of the JPEGD function: GET_JPEGD_OUT_PARAMETER - Command word of the PNGD function: GET_PNGD_OUT_PARAMETER

8.3.3 VDEC Function Interfaces

8.3.3.1 General Description

For a video stream, after you create an instance by calling [CreateVdecApi](#), you must use the same instance to call [VdecCtl](#) for video decoding, and then call [DestroyVdecApi](#) to release the instance.

During video decoding, if you need to switch from a video stream to another video stream, you must release the instance of the previous stream by calling [DestroyVdecApi](#), and then create an instance by calling [CreateVdecApi](#) to process the new video stream.

8.3.3.2 CreateVdecApi

Syntax	int CreateVdecApi(IDVPPAPI*& pIDVPPAPI, int singleton)
--------	---

Function	Obtains a VDEC API instance, which is equivalent to the handle to the VDEC executor. The caller can use the obtained VDEC API instance to call CreateVdecApi for video decoding. Cross-function calling and cross-thread calling are supported.
Input	• IDVPPAPI pointer reference. The input pointer must be NULL. • The singleton is reserved for internal use to implement a single pidVPPAPI instance in the future. It is recommended that the caller set the parameter to 0 currently.
Output	IDVPPAPI pointer reference. The output pointer may be or may not be NULL . If a DVPP API instance fails to be obtained, the output pointer is NULL . If a DVPP API instance is successfully obtained, the output pointer is not NULL .
Return Value	• 0: success • -1: failure
Instructions	The caller creates the IDVPPAPI object pointer, which is initialized to NULL . The IDVPPAPI object pointer is passed by calling the CreateVdecApi function. If the application is successful, CreateVdecApi returns the DvppApi instance. Otherwise, NULL is returned. The caller needs to verify the return value.
Restriction	The caller is responsible for the life cycle of the VDEC API instance, including the application and release, which are implemented by using CreateVdecApi and DestroyVdecApi , respectively.

8.3.3.3 VdecCtl

Syntax	int VdecCtl(IDVPPAPI*& pIDVPPAPI, int CMD, dvppapi_ctl_msg* MSG, int singleton)
Function	Controls the DVPP executor to implement video decoding. The VdecCtl API is called by using the instance created by CreateVdecApi .
Input	• IDVPPAPI pointer reference • For the VDEC module, the command word (CMD) is DVPP_CTL_VDEC_PROC. • Configuration information (MSG) of the VDEC executor in the dvppapi_ctl_msg type. For details about the in pointer in this struct, see Input Parameter: vdec_in_msg. • The singleton is reserved for internal use to implement a single pidVPPAPI instance in the future. It is recommended that the caller set the parameter to 0 currently.

Output	Output buffer in the configuration information (MSG) and output status information of the VDEC (which is all stored in MSG)
Return Value	• 0: This API is called successfully, but it does not mean that the decoding is successful (because the VDEC is asynchronous). • -1: This API fails to be called.
Instructions	The caller calls the VdecCtl function to pass the IDVPPAPI object pointer, the configured dvppapi_ctl_msg , and the correct command word (CMD). The VDEC is asynchronous. When VdecCtl is called, the data in the user stream buffer is copied to the internal input buffer of the VDEC and then returned. Therefore, the decoding is not necessarily successful (only the data is copied successfully) after VdecCtl is called. In addition, after VdecCtl is called, the user stream buffer is released.
Restriction	For a video stream, after you create an instance by calling CreateVdecApi , you must use the same instance to call VdecCtl for video decoding, and then call DestroyVdecApi to release the instance.

Input Parameter: vdec_in_msg

Table 8-22 Input parameter vdec_in_msg

Member Variable	Description
char video_format[10]	Input video format, for example, h264 or h265 . The default value is h264 . Only H.264 and H.265 streams after YUV420SP (NV12 and NV21) encoding are supported.
char image_format[10]	Output frame format, for example, nv12 or nv21 . The default value is nv12 .

Member Variable	Description
void (*call_back)(FRAME* frame,void * hiai_data)	Callback function. FRAME is the output structure after VDEC decoding. For details, see FRAME structure in 8.3.7.2 Structures and Classes in vdec_in_msg. You can obtain the output result based on this pointer. It is recommended that only DvppCtl be called in the callback function to output YUV image data. Do not implement other functionality in the callback function. Otherwise, it may take a long time and block the waiting resources during decoding. The maximum consumption allowed by the callback function is related to the frame rate. The calculation formula is as follows: Maximum time consumption = 1/Frame rate. For example, if the frame rate is 30 fps, the maximum time consumption is 0.033s (1/30 fps). If the frame rate is 25 fps, the maximum consumption is 0.04s (1/25 fps).
char* in_buffer	Buffer of input H.264 or H.265 raw video streams. The buffer for storing the streams (allocated by the user) before decoding is assigned to in_buffer. After VdecCtl is called and a return value is received, this buffer can be freed.
int32_t in_buffer_size	Buffer size of input video streams
void * hiai_data	Pointer to the formal parameter of the output frame callback function after decoding. The object that the pointer points to is defined by the caller. NOTICE Only hiai_data that is passed to the VDEC for the first time is called. To use hiai_data to pass different objects for multiple times, use the following smart pointer object.

Member Variable	Description
std::shared_ptr<HIAI_DATA_SP> hiai_data_sp	If the frame sequence number is not required, this parameter does not need to be specified and the default value is NULL. The frame sequence number is described as follows. Pointer to the formal parameter of the callback function. HIAI_DATA_SP is the parent class defined in the VDEC. For details, see HIAI_DATA_SP class in 8.3.7.2 Structures and Classes in vdec_in_msg. You can inherit the class to define subclasses. For details, see the sample code in 8.3.6.5 Implementing the VDEC Function. The class object to which this pointer points must be configured with the frame sequence number. Each frame (I-frame, P-frame, or B-frame) maps to a unique sequence number. The frame sequence numbers start from 1 with an increment of 1. Notes: 1. The objects of the user-defined subclass hiai_data_sp cannot contain member variables that require large memory. For the decoding of each video stream, the VDEC supports a maximum of 100 hiai_data_sp objects. If the memory requested by the member variables is too large, the memory usage may be too high. 2. If the objects of the user-defined subclass hiai_data_sp contain member variables that request allocation of memory, free the memory in the destructor function after the memory is allocated to avoid memory leaks. 3. During video stream decoding, if a hiai_data_sp object is used but an abnormal frame exists in the video stream, the VDEC directly discards the hiai_data_sp object of the abnormal frame. Therefore, you are advised to handle the abnormal frame in the implementation code of the destructor function. 4. During video stream decoding, if a hiai_data_sp object is used, the reference frame interval of the stream cannot be greater than 30 frames. For example, frame 1 can be referenced for frame 30 rather than frame 31. 5. The VDEC caches the hiai_data_sp objects passed by the user to a queue. A maximum number of 100 objects are supported. If a frame fails to be decoded, the corresponding hiai_data_sp object is discarded at the earliest

Member Variable	Description
	time when all the subsequent 30 frames are decoded. For example, if frame 1 fails to be decoded, the corresponding hiai_data_sp object is discarded after frame 31 is decoded. If all frames in the stream are abnormal, the hiai_data_sp object corresponding to frame 1 is discarded when frame 101 starts to be decoded. 6. Either hiai_data_sp or hiai_data can be used. hiai_data_sp must be used in conjunction with channelId. 7. If the stream contains B-frames, the frames are numbered according to the display sequence instead of the decoding sequence.
int32_t channelId	ID of the VDEC channel corresponding to the input stream. Different values must be set when different streams are decoded at the same time. The value range is 0–15.
void (*err_report) (VDECERR* vdecErr)	Error reporting callback function, which is used to notify users of decoding exceptions, such as stream errors, hardware faults, and decoder status errors, for subsequent troubleshooting. The VDECERR formal parameter is a structure defined in the VDEC. For details, see VDECERR structure in 8.3.7.2 Structures and Classes in vdec_in_msg .
bool isEOS	End of stream (EOS) flag indicating that the decoding of the current channel is complete. You can ignore this flag. By default, isEOS is set to true in the DestroyVdecApi API so that resources are released when the decoding of the channel ends. If isEOS is set to true , the user-specified configuration is preferentially used in the calling of the VdecCtl API so that resources are released when the decoding of the channel ends. For details, see 8.3.6.5 Implementing the VDEC Function .
int32_t isOneInOneOutMode	Reserved. Use the default value 0 .

8.3.3.4 DestroyVdecApi

Syntax	int DestroyVdecApi(IDVPPAPI*& pIDVPPAPI, int singleton)
Function	Releases the VDEC API instance created by CreateVdecApi and closes the VDEC executor.

Input	IDVPPAPI pointer reference. The singleton is reserved for internal use to implement a single pIDVPPAPI instance in the future. It is recommended that the caller set the parameter to 0 currently.
Output	None
Return Value	• 0: success • -1: failure
Instructions	None
Restriction	To continue to invoke the VDEC after DestroyVdecApi is called, create a VDEC API instance.

8.3.4 VENC Function Interfaces

8.3.4.1 General Description

To encode multiple images into a video, call **RunVenc** to encode the video by using the same instance after creating an instance by calling **CreateVenc**, and then call **DestroyVenc** to release the instance.

8.3.4.2 CreateVenc

Syntax	int32_t CreateVenc(struct VencConfig* vencConfig)
Function	Obtains the VENC instance, which is equivalent to obtaining the handle of the VENC executor. The caller can call RunVenc to encode images by using the obtained VENC instance.
Input	Pointer to the VencConfig structure. For details about the VencConfig structure, see Input parameter: VencConfig . You need to configure a callback function to process the encoding result.
Output	None
Return Value	• If the return value is a non-negative number, a VENC instance is successfully created. • The return value -1 indicates that a VENC instance fails to be created.
Instructions	The caller creates the VencConfig object pointer. The VencConfig object pointer is passed by calling the CreateVenc function. If the application is successful, CreateVenc returns the handle to the VENC instance. Otherwise, -1 is returned. The caller needs to verify the return value.

Restriction	The caller is responsible for the life cycle of the VENC instance, including the application and release, which are implemented by using CreateVenc and DestroyVenc , respectively.
--------------------	---

Input parameter: VencConfig

This input parameter is used when the VENC module is initialized. All member variables of the data structure must be initialized before being used.

Table 8-23 Input parameter VencConfig

Member Variable	Description	Value Range
uint32_t width	Image width	The value is an even number ranging from 128 to 1920.
uint32_t height	Image height	The value is an even number ranging from 128 to 1920.
uint32_t codingType	Video encoding protocols	0–3 • 0: H.265 Main Profile Level (Only slice streams are supported.) • 1: H.264 Baseline Profile Level • 2: H.264 Main Profile Level • 3: H.264 High Profile Level
uint32_t yuvStoreType	YUV image storage format	The value is 0 or 1 . • 0: YUV420 semi-planar (NV12) • 1: YVU420 semi-planar (NV21)
uint32_t keyFrameInterval	I-frame interval	The value range is (0, 65535).

Member Variable	Description	Value Range
VencOutMsgCallBack vencOutMsgCallBack	Callback function, which is used to process the encoding result. When encoding each channel of image data, DVPP (the VENC module) calls the callback function twice for the first frame of image data. The first call is used to process the file header information, and the second call is used to process the image data of the current frame.	typedef void (*VencOutMsgCallBack) (struct VencOutMsg* vencOutMsg, void* userData). The value cannot be null.
void* userData	The user records the information to be passed. The data is returned to the user in the callback function.	The value can be NULL .

8.3.4.3 RunVenc

Syntax	int32_t RunVenc(int32_t vencHandle, struct VencInMsg* vencInMsg)
Function	Controls the DVPP executor to implement video encoding. The RunVenc API is called by using the instance created by CreateVenc .
Input	int32_t handle and VencInMsg pointer. vencHandle is the return value of CreateVenc . For details about VencInMsg , see Input Parameter: VencInMsg . VencInMsg indicates the VENC executor configuration information. This structure is used to transfer the video information to be encoded to the executor for encoding.
Output	None
Return Value	● 0: success ● -1: failure
Instructions	The caller calls the RunVenc function to transfer the encoding instance, the VencInMsg object pointer, and the configured vencInMsg .

Restriction	To encode multiple images into a video, call RunVenc to encode the video by using the same instance after creating an instance by calling CreateVenc , and then call DestroyVenc to release the instance.
--------------------	--

Input Parameter: VencInMsg

This input parameter is used when the VENC module is called to perform encoding. All member variables of the structure must be initialized before being used.

Table 8-24 Input parameter VencInMsg

Member Variable	Description	Value Range
void* inputData	Address of the input data	The value cannot be null.
uint32_t inputDataSize	Input data size	The value cannot be greater than the size of the input data buffer. The recommended value is the same as the size of the input data buffer.
uint32_t keyFrameInterval	I-frame interval	The value range is [0, 65535). The value 0 indicates that the parameter is invalid.
uint32_t forceIframe	Whether to forcibly restart the I-frame interval. 0 : No. 1 : Yes.	The value is 0 or 1 .
uint32_t eos	Whether the frame is an end frame. 0 : No. 1 : Yes.	The value is 0 or 1 .

Output Parameter: VencOutMsg

Table 8-25 Output parameter VencOutMsg

Member Variable	Description	Value Range
void* outputData	Output data address	Applied by the VENC internally
uint32_t outputDataSize	Output data size	Configured by the VENC internally

Member Variable	Description	Value Range
uint32_t timeStamp	Timestamp of calling the callback function	Configured by the VENC internally

8.3.4.4 DestroyVenc

Syntax	int32_t DestroyVenc(int32_t vencHandle)
Function	Releases the VENC instance created by calling CreateVenc and closes the VENC executor.
Input	Handle to the int32_t VENC instance, which is the return value of the CreateVenc function
Output	None
Return Value	• 0: success • -1: failure
Instructions	None
Restriction	To continue to invoke the VENC after DestroyVenc is called, create a VENC instance.

8.3.5 Compatibility with the Functions of Earlier Versions

8.3.5.1 Compatibility

To be compatible with the functions in earlier versions, the current version supports the VPC and VENC functions through the following methods:

- Implementing the VPC function:
 - Pass input parameters to [DvppCtl](#), that is, the **DVPP_CTL_VPC_PROC** command word and [resize_param_in_msg](#) structure parameters.
 - In scenarios that have low requirements on latency and process a large number of low-resolution images, pass input parameters to [DvppCtl](#), that is, the **DVPP_CTL_CMDLIST_PROC** command word and [IMAGE_CONFIG](#) structure parameters.

This method is not recommended. It is used to be compatible with the functions in earlier versions and will be deleted in later versions. If you use functions that are not recommended, you are advised to migrate them to the recommended VPC function method described in [8.3.2 VPC/JPEGE/JPEGD/PNGD Function Interfaces](#).

- Implementing the VENC function: Call [CreateDvppApi](#), [DvppCtl](#), and [DestroyDvppApi](#) and pass input parameters to [DvppCtl](#), that is, the **DVPP_CTL_VENC_PROC** command word and [venc_in_msg](#) structure parameters.

This method is not recommended. It is used to be compatible with the functions in earlier versions and will be deleted in later versions. If you use functions that are not recommended, you are advised to migrate them to the recommended VENC function method described in [8.3.4 VENC Function Interfaces](#).

8.3.5.2 VPC Functions and Parameters

You are not advised to use the VPC function of the earlier version, which will be removed in later versions. The VPC and CMDLIST functions in the earlier version have been combined into the VPC function. For details, see [8.3.2.2.2 VPC Parameters](#).

The VPC APIs of the earlier version are used for media image conversion including scaling, color space conversion (CSC), bit depth reduction, format conversion, block partitioning, overlay, collage, and CMDLIST calling.

- There is no limit on the number of VPC channels.

The cropping and resizing operations affect the VPC performance. If the image resolution is changed during image processing, the performance is affected. The following table shows the reference performance specifications in typical scenarios when the image resolution during image processing does not exceed the original resolution.

Scenario	Total Frame Rate
1080p x 32 channels	1440 fps
4K x 4 channels	360 fps

When image resolution during image processing is greater than the original resolution, use the following formula to calculate the frame rate:

$$\text{Total frame rate} = (3840 \times 2160 \times 360) / (W \times H) \text{ fps}$$

Where, **W** and **H** are the maximum width and height during the VPC processing. For example, if a 1080p image is cropped to 960 x 540 and then resized to 3840 x 2160, **W** is 3840 and **H** is 2160.

During the processing, the VPC performance slightly decreases when the image scaling-down ratio is increased.

- Restrictions on the width and height of the input and output images of the VPC:
 - The width and height of the input image must be 2-pixel aligned, that is, the width and height must be an even number.
 - The width and height of the output image must be 2-pixel aligned, that is, the width and height must be an even number.

NOTICE

The image width and height mentioned here refer to the valid area of an image. The actual buffer sent to the VPC must be 128 x 16 aligned.

- Restrictions on the buffers of the VPC input and output images:
 - Restrictions on the VPC input buffer:
 - Input data address (OS virtual address): 128-byte aligned
 - Input image buffer width restriction: 128-byte aligned
 - Input image buffer height restriction: 16-byte aligned
 - Restrictions on the VPC output buffer:
 - Output data address (OS virtual address): 128-byte aligned
 - Output image buffer width restriction: 128-byte aligned
 - Output image buffer height restriction: 16-byte aligned
 - In the same task, the virtual addresses of the input and output buffers must be in the same 4 GB space.

 NOTE

A YUV400SP image requires a buffer size that matches the YUV420SP format, that is, the required buffer size is (width_align_128 x high_align_16 x 1.5).

 NOTE

When the VPC is used to implement image cropping and scaling, **DvppCtl** needs to be called twice. In the first calling, the input parameter is **resize_param_in_msg**, and the output parameter is **resize_param_out_msg**. In the second calling, the input parameter is **vpc_in_msg**. For details, see [Input Parameter: **resize_param_in_msg**](#) to [Input Parameter: **vpc_in_msg**](#).

Input Parameter: **resize_param_in_msg**

Member Variable	Description	Value Range
int src_width	Width of the original image (2-pixel aligned) NOTICE The original image mentioned here refer to the valid area of an image. The actual buffer sent to the VPC must be 128 x 16 aligned.	Minimum resolution: • 128 (from VDEC) • 16 (not from VDEC) Maximum resolution: 4096 This parameter is mandatory.
int src_high	Height of the original image (2-pixel aligned) NOTICE The original image mentioned here refer to the valid area of an image. The actual buffer sent to the VPC must be 128 x 16 aligned.	Minimum resolution: • 128 (from VDEC) • 16 (not from VDEC) Maximum resolution: 4096 This parameter is mandatory.

Member Variable	Description	Value Range
int hmax	Maximum horizontal offset from the origin	The value is an odd number. For details, see Input Parameter: vpc_in_msg .
int hmin	Minimum horizontal offset from the origin	The value is an even number. For details, see Input Parameter: vpc_in_msg .
int vmax	Maximum vertical offset from the origin	The value is an odd number. For details, see Input Parameter: vpc_in_msg .
int vmin	Minimum vertical offset from the origin	The value is an even number. For details, see Input Parameter: vpc_in_msg .
int dest_width	Width of the output image	The value is an even number
int dest_high	Height of the output image	The value is an even number

Output Parameter: resize_param_out_msg

Member Variable	Description	Value Range
int dest_width_stride	Width alignment value of the output image	128
int dest_high_stride	Height alignment value of the output image	16
int hmax	Maximum horizontal offset from the origin	The value is an odd number. For details, see Input Parameter: vpc_in_msg .
int hmin	Minimum horizontal offset from the origin	The value is an even number. For details, see Input Parameter: vpc_in_msg .

Member Variable	Description	Value Range
int vmax	Maximum vertical offset from the origin	The value is an odd number. For details, see Input Parameter: vpc_in_msg .
int vmin	Minimum vertical offset from the origin	The value is an even number. For details, see Input Parameter: vpc_in_msg .
double hinc	Horizontal resizing coefficient	For details, see Input Parameter: vpc_in_msg .
double vinc	Vertical resizing coefficient	For details, see Input Parameter: vpc_in_msg .

Input Parameter: vpc_in_msg

Member Variable	Description	Value Range
int format	Image format defined by the caller. Use a correct number. For example, the number corresponding to yuv420_semi_plannar is 0 . If YUV420SP is declared, set this parameter to 0 .	yuv420_semi_plannar = 0,//0 yuv400_semi_plannar = 0,//0 yuv422_semi_plannar,//1 yuv444_semi_plannar,//2 yuv422_packed,//3 yuv444_packed,//4 rgb888_packed,//5 xrgb8888_packed,//6 This parameter is mandatory.
int rank	Image rank mode defined by the caller. Use a correct number.	For details, see Table 8-26 . This parameter is mandatory.
int bitwidth	Bit depth. The default value is 8 bits. The 10-bit depth is used only when the image format is YUV/ YVU420 Semi-Planar and HFBC compression is used.	8 or 10 This parameter is optional.

Member Variable	Description	Value Range
int cvdr_or_rdma	Whether the input image is transmitted through the CVDR or RDMA channel. The CVDR channel is used by default. The input of the RDMA channel can only be HFBC data output by the VDEC.	1: CVDR. 0: RDMA. This parameter is optional.
int width	Width of the original input image, which is 2-pixel aligned (even number) NOTICE The original image mentioned here refer to the valid area of an image. The actual buffer sent to the VPC must be 128 x 16 aligned.	Minimum resolution: 128 (from VDEC) 16 (not from VDEC) Maximum resolution: 4096 This parameter is mandatory.
int high	Height of the original input image, which is 2-pixel aligned (even number) NOTICE The original image mentioned here refer to the valid area of an image. The actual buffer sent to the VPC must be 128 x 16 aligned.	Minimum resolution: 128 (from VDEC) 16 (not from VDEC) Maximum resolution: 4096 This parameter is mandatory.
int stride	Image stride NOTICE The stride is different for different input formats.	YUV400SP, YUV420SP, YUV422SP, and YUV444SP: Align the width to a 128-pixel boundary. YUV422 Packed: Align (width x 2) to a 128-pixel boundary. YUV444 Packed and RGB888: Align (width x 3) to a 128-pixel boundary. XRGB8888: Align (width x 4) to a 128-pixel boundary. This parameter is mandatory.
double hinc	Horizontal magnification for the AI core output channel	[0.03125, 1) or (1, 4] If a resizing coefficient exceeds the value range, the VPC reports an error. If resizing is not required, set the value to 1. This parameter is mandatory.

Member Variable	Description	Value Range
double vinc	Vertical magnification for the AI core output channel	[0.03125, 1) or (1, 4] If a resizing coefficient exceeds the value range, the VPC reports an error. If resizing is not required, set the value to 1. This parameter is mandatory.
double jpeg_hinc	Horizontal magnification for the JPEG output channel	[0.03125, 1) or (1, 4] If a resizing coefficient exceeds the value range, the VPC reports an error. This parameter is optional and is not applicable currently.
double jpeg_vinc	Vertical magnification for the JPEG output channel	[0.03125, 1) or (1, 4] If a resizing coefficient exceeds the value range, the VPC reports an error. This parameter is optional and is not applicable currently.
int hmax	Maximum horizontal offset from the origin	The value must be an odd number. If cropping is not required, set the value to (Input image width - 1). This parameter is mandatory.
int hmin	Minimum horizontal offset from the origin	The value must be an even number. If cropping is not required, set the value to 0. This parameter is mandatory.
int vmax	Maximum vertical offset from the origin	The value must be an odd number. If cropping is not required, set the value to (Input image width - 1). This parameter is mandatory.
int vmin	Minimum vertical offset from the origin	The value must be an even number. If cropping is not required, set the value to 0.

Member Variable	Description	Value Range
int out_width	Width of the output image	The value must be an even number (2-pixel aligned). When a smart pointer is used to allocate the output buffer, this parameter is not required. When a buffer allocated by the user is used as the output buffer, this parameter is mandatory.
int out_high	Height of the output image	The value must be an even number (2-pixel aligned). When a smart pointer is used to allocate the output buffer, this parameter is not required. When a buffer allocated by the user is used as the output buffer, this parameter is mandatory.
int h_stride	Image height stride, which is the same as that described in int stride . You do not need to set this parameter.	The default value is 16 . That is, the default input image height is 16-pixel aligned. This parameter is optional.
char* in_buffer	Input buffer	Ensure that the input buffer size is the same as the length and width configured in in_msg . If the input image is in YUV420 Semi-Planar NV12 format, the image size is (Width x Height x 1.5). If the input buffer size is inconsistent with this calculation result, the VPC fails to be called. This parameter is mandatory.
int in_buffer_size	Size of the input buffer	This value is used to verify the input buffer. This parameter is mandatory.

Member Variable	Description	Value Range
shared_ptr<AutoBuffer> auto_out_buffer_1	Output buffer 1	This buffer needs to be configured when the basic VPC functions (cropping and resizing) are used. This buffer is used for the smart pointer allocated by the caller and is used as an input parameter of the DVPP. The output buffer is allocated inside the DVPP. You can read this buffer to obtain the output image. You only need to set either this parameter or the subsequent parameter out_buffer .
char* out_buffer	Output buffer	This buffer needs to be configured when the basic VPC functions (cropping and resizing) are used. This buffer is allocated by the caller. The start address of the buffer must be 128-byte aligned, and the allocated memory must be in the 4 GB space. You can read this buffer to obtain the output image. You only need to set either this parameter or the previous parameter auto_out_buffer_1 .
int out_buffer_1_size	Size of output buffer 1	This value is used to verify the output buffer. If shared_ptr<AutoBuffer> auto_out_buffer_1 is used to allocate the output buffer, this parameter is optional. If char* out_buffer is used to allocate the output buffer, this parameter is mandatory.
shared_ptr<AutoBuffer> auto_out_buffer_2	Output of the other VPC channel (reserved interface)	--
int out_buffer_2_size	Size of output buffer 2 of the auto_out_buffer2 smart pointer. This is a reserved interface.	--

Member Variable	Description	Value Range
int use_flag	Flag indicating whether the VPC function is used	0: basic functions of the VPC 1: raw data 8K resizing 2: raw data overlay (reserved) 3: raw data stitching (reserved) The default value is 0. This parameter is optional.
VpcOverLayReverse overlay	Overlay structure, which is reserved	Reserved
VpcCollageReverse collage	Collage structure, which is reserved	Reserved
RDMACHANNEL rdma	Configuration structure dedicated to the HFBC data	For details, see 8.3.7.5.1 RDMACHANNEL Structure .
VpcTurningReverse turningReverse1	Reserved interface for VPC tuning	For details, see 8.3.7.5.2 VpcTurningReverse Structure .
bool isVpcUseTurning1	Flag indicating whether VPC tuning is used. This is a reserved interface and is used together with turningReverse1 .	--
VpcTurningReverse turningReverse2	Reserved interface for VPC tuning	For details, see 8.3.7.5.2 VpcTurningReverse Structure .
bool isVpcUseTurning2	Flag indicating whether VPC tuning is used. This is a reserved interface and is used together with turningReverse2 .	--
string *yuvscaler_parameter	Pointer to the file path and file name array of the VPC filtering parameter set. NOTE - The parameter set must be on the device. - Ensure that the input file path is correct.	A parameter set file path consists of an absolute file path on the device and the file name, for example, {"/home/HwHiAiUser/matrix/YUVScaler_pra.h"}.

Member Variable	Description	Value Range
unsigned int yuvscaler_paraset_size	Number of elements in the string array to which yuvscaler_paraset points. The default value is 1.	yuvscaler_paraset_size<=10
unsigned int index	Index of the yuvscaler_paraset array	0<=index<10

Table 8-26 Rank configuration table of the input and output image formats

Input Image Format	Rank Parameter Configuration of the VPC	Output Image Format
YUV420sp NV12	NV12 = 0	YUV420sp NV21
YUV420sp NV21	NV21 = 1	YUV420sp NV21
YUV420sp NV12	NV12 = 1	YUV420sp NV12
YUV420sp NV21	NV21 = 0	YUV420sp NV12
YUV422sp NV16	NV16 = 1	YUV420sp NV12
YUV422sp NV61	NV61 = 0	YUV420sp NV12
YUV422sp NV16	NV16 = 0	YUV420sp NV21
YUV422sp NV61	NV61 = 1	YUV420sp NV21
YUV444sp 444spUV	444spUV = 1	YUV420sp NV12
YUV444sp 444spVU	444spVU= 0	YUV420sp NV12
YUV444sp 444spUV	444spUV = 0	YUV420sp NV21
YUV444sp 444spVU	444spVU= 1	YUV420sp NV21
YUV422packet YUYV	YUYV = 2	YUV420sp NV12
YUV422packet YVYU	YVYU = 3	YUV420sp NV21
YUV422packet UYVY	UYVY = 4	YUV420sp NV12
YUV444packet YUV	YUV = 5	YUV420sp NV12
RGB888 RGB	RGB = 6	YUV420sp NV12
RGB888 BGR	BGR = 7	YUV420sp NV21
RGB888 RGB	BGR = 7	YUV420sp NV21
RGB888 BGR	RGB = 6	YUV420sp NV12

Input Image Format	Rank Parameter Configuration of the VPC	Output Image Format
XRGB8888 RGBA	RGBA = 8	YUV420sp NV12
XRGB8888 RGBA	BGRA = 9	YUV420sp NV21

Calling Example

- Example of calling basic VPC functions:

gXxxx is a configuration parameter. Configuring the parameter and calling the VPC are two steps.

Step 1: Use the **DVPP_CTL_TOOL_CASE_GET_RESIZE_PARAM** command word to call **DvppCtl** to obtain the configuration parameters. The input parameter of **DvppCtl** is **resize_param_in_msg**, and the output parameter is **resize_param_out_msg**. Use **resize_param_in_msg** to send the width and height of the original image, cropped area (hmax,hmin,vmax,vmin), and output width and height to the DVPP. The DVPP returns the regenerated cropped area (hmax,hmin,vmax,vmin) and resizing ratio (hinc,vinc) to the caller after calculation.

Step 2: The caller transfers the parameters returned in step 1 to the DVPP by using **vpc_in_msg**, changes the **DvppCtl** command word to **DVPP_CTL_VPC_PROC** to obtain the processing result.

```
dvppapi_ctl_msg dvppApiCtlMsg;
vpc_in_msg vpcInMsg;
resize_param_in_msg resize_in_param;
resize_param_out_msg resize_out_param;
resize_in_param.src_width = gWidth;
resize_in_param.src_high = gHigh;
resize_in_param.hmax = gHmax;
resize_in_param.hmin = gHmin;
resize_in_param.vmax = gVmax;
resize_in_param.vmin = gVmin;
resize_in_param.dest_width = floor(gHinc * (gHmax - gHmin + 1) + 0.5);
resize_in_param.dest_high = floor(gVinc * (gVmax - gVmin + 1) + 0.5);
printf("width  =%d\n", gWidth);
printf("high  =%d\n", gHigh);
printf("hmax  =%d\n", gHmax);
printf("hmin  =%d\n", gHmin);
printf("vmax  =%d\n", gVmax);
printf("vmin  =%d\n", gVmin);
printf("out width  =%d\n", resize_in_param.dest_width);
printf("out high  =%d\n", resize_in_param.dest_high);

dvppApiCtlMsg.in = (void *)&resize_in_param;
dvppApiCtlMsg.out = (void *)&resize_out_param;
IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);
if (DvppCtl(pidvppapi, DVPP_CTL_TOOL_CASE_GET_RESIZE_PARAM, &dvppApiCtlMsg) != 0) {
    printf("call DVPP_CTL_TOOL_CASE_GET_RESIZE_PARAM process failed!\n");
    DestroyDvppApi(pidvppapi);
    return;
}

int bufferSize    = 0;
vpcInMsg.format   = gFormat;
vpcInMsg.cvdr_or_rdma = gcvdr_or_rdma;
vpcInMsg.bitwidth = gBitwidth;
```

```

vpclnMsg.rank = gRank;
vpclnMsg.width = gWidth;
vpclnMsg.high = gHigh;
vpclnMsg.stride = gStride;
vpclnMsg.hmax = resize_out_param.hmax;
vpclnMsg.hmin = resize_out_param.hmin;
vpclnMsg.vmax = resize_out_param.vmax;
vpclnMsg.vmin = resize_out_param.vmin;
vpclnMsg.vinc = resize_out_param.vinc;
vpclnMsg.hinc = resize_out_param.hinc;
printf("resize_out_param.hinc = %lf\n", resize_out_param.hinc);
printf("resize_out_param.vinc = %lf\n", resize_out_param.vinc);
printf("resize_out_param.hmax = %d\n", resize_out_param.hmax);
printf("resize_out_param.hmin = %d\n", resize_out_param.hmin);
printf("resize_out_param.vmax = %d\n", resize_out_param.vmax);
printf("resize_out_param.vmin = %d\n", resize_out_param.vmin);

printf("format = %d\n", vpclnMsg.format);
printf("rank   = %d\n", vpclnMsg.rank);
printf("width  = %d\n", vpclnMsg.width);
printf("high   = %d\n", vpclnMsg.high);
printf("stride = %d\n", vpclnMsg.stride);
printf("hmax   = %d\n", vpclnMsg.hmax);
printf("hmin   = %d\n", vpclnMsg.hmin);
printf("vmax   = %d\n", vpclnMsg.vmax);
printf("vmin   = %d\n", vpclnMsg.vmin);
printf("vinc   = %F\n", vpclnMsg.vinc);
printf("hinc   = %F\n", vpclnMsg.hinc);
if (vpclnMsg.cvdr_or_rdma == 0) {
    printf("rdma channel\n");
    int buffersize ;
    char * payloadY ;
    char * payloadC ;
    char * headY ;
    char * headC ;
    FILE * rdmaimage = fopen(in_file_name, "rb");

    if (vpclnMsg.bitwidth == 10) {
        buffersize      = vpclnMsg.width * vpclnMsg.high * 2 + 512 * 1024;
        payloadY        = (char*)memalign(128, buffersize);
        headY           = payloadY + vpclnMsg.width * vpclnMsg.high * 2;
        headC           = headY + 256 * 1024;
        vpclnMsg.rdma.luma_payload_addr = (long)payloadY;
        vpclnMsg.rdma.chroma_payload_addr = (long)payloadY + 640;
    } else {
        buffersize      = vpclnMsg.width * vpclnMsg.high * 1.5 + 512 * 1024;
        payloadY        = (char*)memalign(128, buffersize);
        payloadC        = payloadY + vpclnMsg.width * vpclnMsg.high;
        headY           = payloadC + vpclnMsg.width * vpclnMsg.high / 2;
        headC           = headY + 256 * 1024;
        vpclnMsg.rdma.luma_payload_addr = (long)payloadY;
        vpclnMsg.rdma.chroma_payload_addr = (long)payloadC;
    }

    fread(payloadY, 1, buffersize, rdmaimage);
    vpclnMsg.rdma.luma_head_addr = (long)headY;
    vpclnMsg.rdma.chroma_head_addr = (long)headC;
    vpclnMsg.rdma.luma_head_stride = vpclnMsg.stride;
    vpclnMsg.rdma.chroma_head_stride = vpclnMsg.stride;
    vpclnMsg.rdma.luma_payload_stride = gpYStride;
    vpclnMsg.rdma.chroma_payload_stride = gpUVStride;
    fclose(rdmaimage);
    printf("luma payload stride= %d\n", vpclnMsg.rdma.luma_payload_stride );
    printf("chroma payload stride= %d\n", vpclnMsg.rdma.chroma_payload_stride );
} else {
    alloc_buffer(vpclnMsg.width, vpclnMsg.high, vpclnMsg.stride, bufferSize);
    if (open_image(in_file_name, vpclnMsg.width, vpclnMsg.high, vpclnMsg.stride) != 0) {
        printf("open file failed~\n");
        free(in_buffer);
    }
}

```

```

        return;
    }
    vpcInMsg.in_buffer = in_buffer;
    vpcInMsg.in_buffer_size = bufferSize;
}
//alloc in and out buffer
shared_ptr<AutoBuffer> auto_out_buffer = make_shared<AutoBuffer>();
vpcInMsg.auto_out_buffer_1 = auto_out_buffer;
dvppApiCtlMsg.in = (void *)&vpcInMsg;
dvppApiCtlMsg.in_size = sizeof(vpcInMsg);

if (pidvppapi != NULL) {

    for (int i = 0; i < gLoop; i++) {
        if (DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg) != 0) {
            printf("call dvppctl process failed!\n");
            DestroyDvppApi(pidvppapi);
            free(in_buffer);
            return;
        }
    }

} else {
    printf("pidvppapi is null!\n");
}

free(in_buffer);

FILE *fp = NULL;
fp = fopen(out_file_name, "wb+");
if (fp == NULL) {
    printf("open file %s failed~\n", out_file_name);
    free(in_buffer);
    return;
}

fwrite(vpcInMsg.auto_out_buffer_1->getBuffer(), 1, vpcInMsg.auto_out_buffer_1->getBufferSize(),
fp);
fflush(fp);
fclose(fp);
DestroyDvppApi(pidvppapi);
return;

```

- Example of calling basic raw data 8K functions:

Parameter settings:

```

dvppapi_ctl_msg dvppApiCtlMsg;
vpc_in_msg vpcInMsg;
vpcInMsg.width = gWidth;
vpcInMsg.high = gHigh;
vpcInMsg.stride = gStride;
vpcInMsg.out_width = gOutWidth;
vpcInMsg.out_high = gOutHigh;
printf("width    =%d\n", vpcInMsg.width);
printf("high     =%d\n", vpcInMsg.high);
printf("stride   =%d\n", vpcInMsg.stride);
printf("out_width =%d\n", vpcInMsg.out_width);
printf("out_high  =%d\n", vpcInMsg.out_high);
//alloc in and out buffer
char *in_buffer = (char *)malloc(vpcInMsg.width * vpcInMsg.high * 1.5);
if (in_buffer == NULL) {
    printf("malloc fail\n");
    return;
}

shared_ptr<AutoBuffer> auto_out_buffer = make_shared<AutoBuffer>();
FILE *yuvimage = fopen(in_file_name, "rb");
if (yuvimage == NULL) {
    printf("no such file!,file_name=[%s]\n", in_file_name);
    free(in_buffer);
}

```

```

        return;
    } else {
        fread(in_buffer, 1, vpcInMsg.width * vpcInMsg.high * 3 / 2, yuvimage);
    }

    vpcInMsg.rank = gRank;
    vpcInMsg.in_buffer = in_buffer;
    vpcInMsg.in_buffer_size = vpcInMsg.width * vpcInMsg.high * 1.5;
    vpcInMsg.auto_out_buffer_1 = auto_out_buffer;
    vpcInMsg.use_flag = 1;
    dvppApiCtlMsg.in = (void *)&vpcInMsg;
    dvppApiCtlMsg.in_size = sizeof(vpc_in_msg);
    IDVPPAPI *pidvppapi = NULL;
    CreateDvppApi(pidvppapi);
    if (pidvppapi != NULL) {
        if (DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg) != 0) {
            printf("call dvppctl process failed!\n");
            DestroyDvppApi(pidvppapi);
            free(in_buffer);
            fclose(yuvimage);
            return;
        }
    } else {
        printf("pidvppapi is null!\n");
    }

    DestroyDvppApi(pidvppapi);
    FILE *fp = NULL;
    fp = fopen(out_file_name, "wb+");
    if (fp != NULL) {
        fwrite(vpcInMsg.auto_out_buffer_1->getBuffer(), 1, vpcInMsg.auto_out_buffer_1-
>getBufferSize(), fp);
        fflush(fp);
        fclose(fp);
    } else {
        fclose(yuvimage);
        return;
    }

    free(in_buffer);
    fclose(yuvimage);
    return;

```

- Example of calling basic raw data overlay functions:

```

dvppapi_ctl_msg dvppApiCtlMsg;
vpc_in_msg vpcInMsg;
// Configures raw data overlay parameters.
vpcInMsg.use_flag = 2;
vpcInMsg.overlay.image_type = 0;
vpcInMsg.overlay.image_rank_type = 0;
vpcInMsg.overlay.bit_width = 8;
vpcInMsg.overlay.in_width_image = 1000;
vpcInMsg.overlay.in_high_image = 1000;
vpcInMsg.overlay.in_width_text = 500;
vpcInMsg.overlay.in_high_text = 390;
vpcInMsg.overlay.image_width_stride = 2;
vpcInMsg.overlay.image_high_stride = 2;
vpcInMsg.overlay.text_width_stride = 2;
vpcInMsg.overlay.text_high_stride = 2;
vpcInMsg.overlay.in_buffer_image = NULL;
vpcInMsg.overlay.in_buffer_text = NULL;
vpcInMsg.overlay.auto_overlay_out_buffer = make_shared<AutoBuffer>();
// Initializes the image buffer.
if(NULL == (vpcInMsg.overlay.in_buffer_image =
(char*)malloc(vpcInMsg.overlay.in_width_image*vpcInMsg.overlay.in_high_image*3/2))) {
    printf("in_buffer_image alloc failed!\n");
    return ;
}
// Opens the image and sends the data to the allocated buffer.
char image_file[50] = "yuv_data_0";

```

```
FILE * yuvimage = fopen(image_file,"rb");
if(NULL == yuvimage) {
    printf("VPC_TEST: yuv image open failed!");
    return ;
} else {
    fread(vpcInMsg.overlay.in_buffer_image,
1,vpcInMsg.overlay.in_width_image*vpcInMsg.overlay.in_high_image*3/2,yuvimage);
    fclose(yuvimage);
    yuvimage = NULL;
}

// Initializes the text buffer.
if(NULL == (vpcInMsg.overlay.in_buffer_text =
(char*)malloc(vpcInMsg.overlay.in_width_text*vpcInMsg.overlay.in_high_text*3/2))) {
    printf("in_buffer_text alloc failed!");
    free(vpcInMsg.overlay.in_buffer_image);
    return ;
}

// Opens the text-based image and sends the data to the allocated memory.
char text_file[50] = "text_0";
FILE * yuvtext = fopen(text_file,"rb");
if(NULL == yuvtext) {
    printf("VPC_TEST: yuv text image open failed!");
    free(vpcInMsg.overlay.in_buffer_image);
    return ;
} else {
    fread(vpcInMsg.overlay.in_buffer_text,
1,vpcInMsg.overlay.in_width_text*vpcInMsg.overlay.in_high_text*3/2, yuvtext);
    fclose(yuvtext);
    yuvtext = NULL;
}

dvppApiCtlMsg.in = (void*)&vpcInMsg;
dvppApiCtlMsg.in_size = sizeof(vpc_in_msg);
IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);
if(pidvppapi!=NULL) {
    if(DvppCtl(pidvppapi,DVPP_CTL_VPC_PROC,&dvppApiCtlMsg)!= 0) {
        printf("call dvppctl process failed!\n");
        DestroyDvppApi(pidvppapi);
        free(vpcInMsg.overlay.in_buffer_image);
        free(vpcInMsg.overlay.in_buffer_text);
        return ;
    }
    DestroyDvppApi(pidvppapi);
    FILE * fp = fopen("./out_overlay_image_share","wb+");
    fwrite(vpcInMsg.overlay.auto_overlay_out_buffer->getBuffer(),
1,vpcInMsg.overlay.in_width_image*vpcInMsg.overlay.in_high_image*3/2,fp);
    fflush(fp);
    fclose(fp);
    fp=NULL;
    return ;
} else {
    printf("pidvppapi is null!\n");
    return ;
}
if(NULL != vpcInMsg.overlay.in_buffer_image) {
    free(vpcInMsg.overlay.in_buffer_image);
    vpcInMsg.overlay.in_buffer_image = NULL;
}
if(NULL != vpcInMsg.overlay.in_buffer_text) {
    free(vpcInMsg.overlay.in_buffer_text);
    vpcInMsg.overlay.in_buffer_text = NULL;
}
}
```

- Example of calling basic raw data stitching functions:

```
dvppapi_ctl_msg dvppApiCtlMsg;
vpc_in_msg vpcInMsg;
// Configures raw data stitching parameters.
```

```

vpcInMsg.use_flag = 3;
vpcInMsg.collage.image_type = 0;
vpcInMsg.collage.image_rank_type = 0;
vpcInMsg.collage.bit_width = 8;
vpcInMsg.collage.in_width = 1000;
vpcInMsg.collage.in_high = 1000;
vpcInMsg.collage.width_stride = 2;
vpcInMsg.collage.high_stride = 2;
vpcInMsg.collage.collage_type = 0;
vpcInMsg.collage.auto_out_buffer = make_shared<AutoBuffer>();
char image_file[4][50] = {"yuv_data_0", "yuv_data_1", "yuv_data_2", "yuv_data_3"};
// Initializes the image buffer (without stride data)*****begin*****
for(int i=0; i<4; i++) {
    if(NULL == (vpcInMsg.collage.in_buffer[i] =
(char*)malloc(vpcInMsg.collage.in_width*                                vpcInMsg.collage.in_high*3/2)))
    {
        printf("in_buffer_image alloc failed!\n");
        for(int j=0; j<i; j++) {
            free(vpcInMsg.collage.in_buffer[j]);
        }
        return ;
    }
    // Opens the image and sends the data to the allocated buffer.
    FILE * yuvimage = fopen(image_file[i], "rb");
    if(NULL == yuvimage) {
        printf("PVC_TEST: yuv image open faild!");
        return ;
    } else {
        fread(vpcInMsg.collage.in_buffer[i], 1, vpcInMsg.collage.in_width*
            vpcInMsg.collage.in_high*3/2, yuvimage);
        fclose(yuvimage);
        yuvimage = NULL;
    }
}

dvppApiCtlMsg.in = (void*)&vpcInMsg;
dvppApiCtlMsg.in_size = sizeof(vpc_in_msg);
IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);
if(pidvppapi!=NULL) {
    if(DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg) != 0) {
        printf("call dvppctl process faild!\n");
        DestroyDvppApi(pidvppapi);
        for(int i=0; i<4; i++) {
            free(vpcInMsg.collage.in_buffer[i]);
        }
        return ;
    }
    DestroyDvppApi(pidvppapi);
    FILE * fp = fopen("./out_collage_image_share", "wb+");
    fwrite(vpcInMsg.collage.auto_out_buffer->getBuffer(),
1, vpcInMsg.collage.in_width*vpcInMsg.collage.in_high*6, fp);
    fflush(fp);
    fclose(fp);
    fp=NULL;
    return ;
} else {
    printf("pidvppapi is null!\n");
    return ;
}
for(int i=0; i<4; i++) {
    if(NULL != vpcInMsg.collage.in_buffer[i]) {
        free(vpcInMsg.collage.in_buffer[i]);
        vpcInMsg.collage.in_buffer[i] = NULL;
    }
}
}

```

- Example of calling the VPC API:

```

IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);

```



```
if(pidvppapi!=NULL)
{
if(0 != DvppCtl(pidvppapi,DVPP_CTL_VPC_PROC,&dvppApiCtlMsg))
{
printf("call dvppctl process failed!\n");
DestroyDvppApi(pidvppapi);
return -1;
}
}
else
printf("pidvppapi is null!\n");
DestroyDvppApi(pidvppapi);
```

- Example of calling functions for configuring and obtaining VPC resizing parameters:

gXxxx is a configuration parameter.

```
dvppapi_ctl_msg dvppApiCtlMsg;
vpc_in_msg vpcInMsg;
resize_param_in_msg resize_in_param;
resize_param_out_msg resize_out_param;
resize_in_param.src_width = gWidth;
resize_in_param.src_high = gHigh;
resize_in_param.hmax = gHmax;
resize_in_param.hmin = gHmin;
resize_in_param.vmax = gVmax;
resize_in_param.vmin = gVmin;
resize_in_param.dest_width = floor(gHinc * (gHmax - gHmin + 1) + 0.5);
resize_in_param.dest_high = floor(gVinc * (gVmax - gVmin + 1) + 0.5);
dvppApiCtlMsg.in = (void *)&resize_in_param;
dvppApiCtlMsg.out = (void *)&resize_out_param;
IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);
if (0 != DvppCtl(pidvppapi, DVPP_CTL_TOOL_CASE_GET_RESIZE_PARAM, &dvppApiCtlMsg))
{
printf("call dvppctl process failed!\n");
DestroyDvppApi(pidvppapi);
return;
}
int bufferSize = 0;
vpcInMsg.format = gFormat;
vpcInMsg.cvdr_or_rdma = gcvdr_or_rdma;
vpcInMsg.bitwidth = gBitwidth;
vpcInMsg.rank = gRank;
vpcInMsg.width = gWidth;
vpcInMsg.high = gHigh;
vpcInMsg.stride = gStride;
vpcInMsg.hmax = resize_out_param.hmax;
vpcInMsg.hmin = resize_out_param.hmin;
vpcInMsg.vmax = resize_out_param.vmax;
vpcInMsg.vmin = resize_out_param.vmin;
vpcInMsg.vinc = resize_out_param.vinc;
vpcInMsg.hinc = resize_out_param.hinc;
```

8.3.5.3 CMDLIST Functions and Parameters

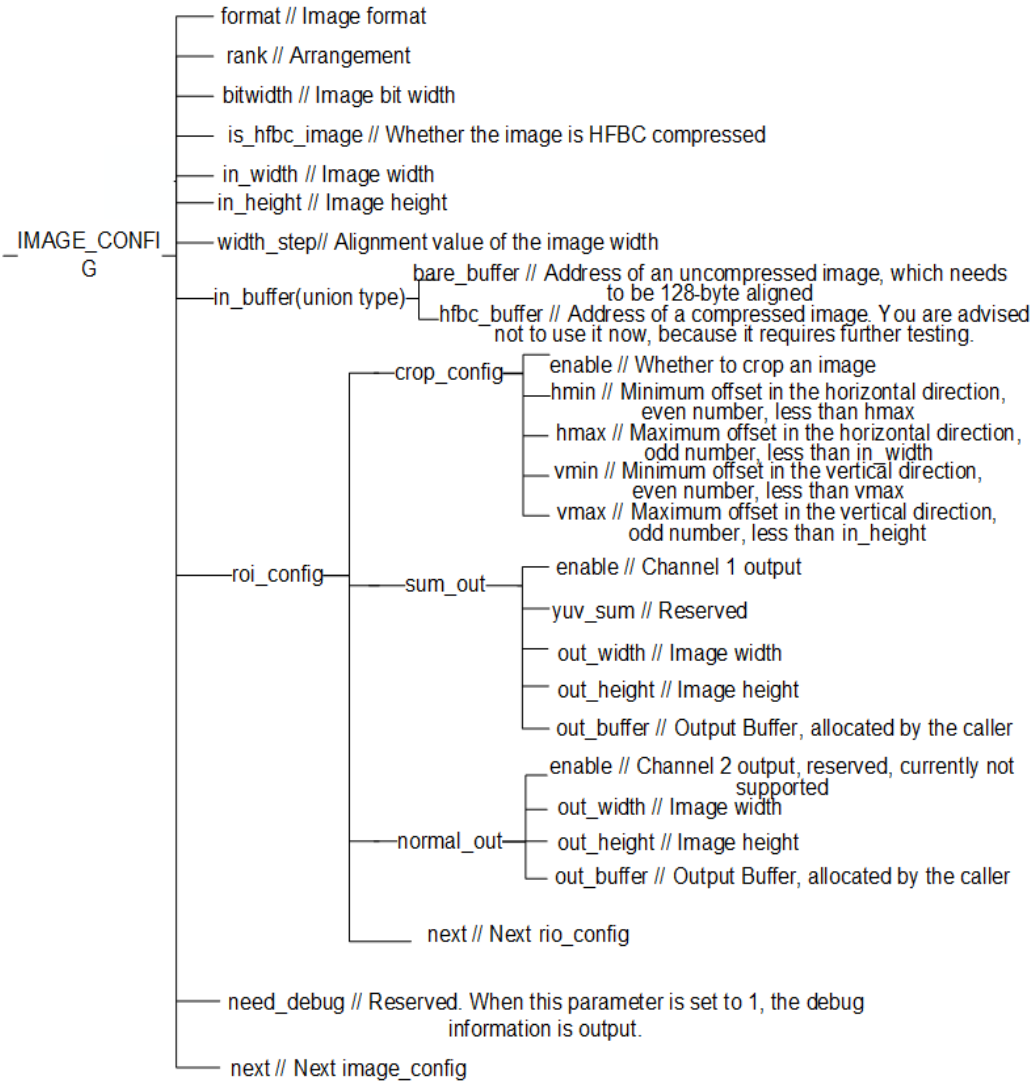
You are not advised to use the CMDLIST function, which will be removed in later versions. The VPC and CMDLIST functions in the earlier version have been combined into the VPC function. For details, see [8.3.2.2 VPC Parameters](#).

The CMDLIST is an extended function of the VPC. It combines functions that need to start the VPC for multiple times into one startup-complete-interrupt-return process. CMDLIST applies to scenarios that do not require low latency and process a large number of low-resolution images.

Input Parameter

The CMDLIST input is a structure. For details about the parameters, see [Figure 8-5](#) and [Table 8-27](#).

Figure 8-5 CMDLIST structures



For details about the description and value ranges, see [Table 8-27](#).

Table 8-27 Structure description

Member Variable	Description	Value Range
unsigned int format	Input image type	typedef enum { yuv420_semi_plannar =0,//0 yuv422_semi_plannar,//1 yuv444_semi_plannar,//2 yuv422_packed,//3 yuv444_packed,//4 rgb888_packed,//5 xrgb8888_packed,//6 yuv400_semi_plannar,//7 invalid_image_type,//20 }Imge_Type;
unsigned int rank	Output image format (NV12 or NV21)	For details, see Table 8-26 .
unsigned int bitwidth	Bit depth, which is usually 8 bits. The 10-bit depth is used only when the image format is YUV/YVU420 Semi-Planar and HFBC compression is used.	8: 8 bits 10: 10 bits
int is_hfbc_image	Input image channel. Generally, this parameter is set to 1 and the CVDR channel is used. The RDMA channel is used only when the HFBC data output by the VDEC is used as the input.	0: RDMA 1: CVDR
unsigned int in_width	Width of the input image, which must be 128-pixel aligned	128~4096
unsigned int in_height	Height of the input image, which must be 16-pixel aligned	16~4096

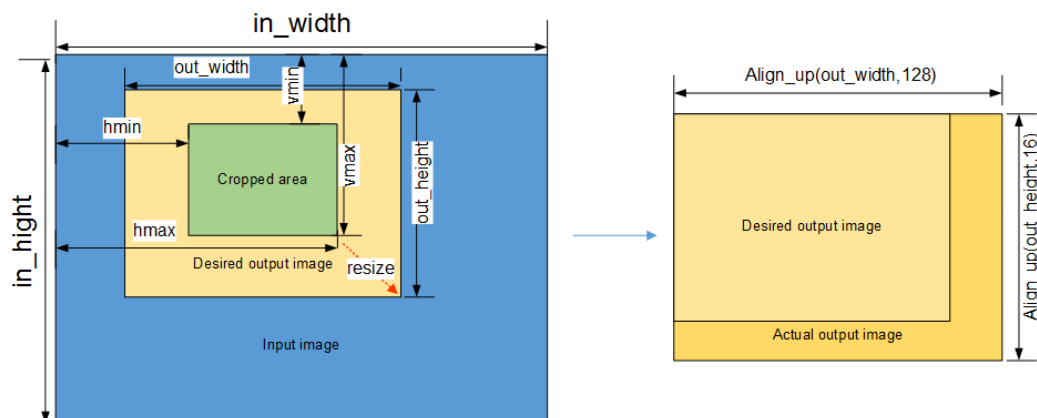
Member Variable	Description	Value Range
unsigned int width_step	Image stride	YUV400SP, YUV420SP, YUV422SP, and YUV444SP: Align the width to a 128-pixel boundary. YUV422 Packed: Align (width x 2) to a 128-pixel boundary. YUV444 Packed and RGB888: Align (width x 3) to a 128-pixel boundary. XRGB8888: Align (width x 4) to a 128-pixel boundary.
unsigned int need_debug	Reserved for internal debugging. Generally, this parameter is set to 0 .	0 : normal mode 1 : debug mode
CMDLIST_IN_BUFFER in_buffer	Pointer to the address of the input image buffer, which is a union. For common uncompressed images, use bare_buffer , which must be 128-byte aligned. For HFBC-compressed images, use hfbc_buffer .	union CMDLIST_IN_BUFFER { char* bare_buffer; RDMACHANNEL* hfbc_buffer; }; RDMACHANNEL in this syntax is the same as that in vpc_in_msg . For details, see 8.3.7.5.1 RDMACHANNEL Structure .
ROI_CONFIG roi_config	Structure of output parameter configuration	For details, see ROI_CONFIG structure .
IMAGE_CONFIG* next	Pointer to the next IMAGE_CONFIG structure. Set this parameter when multiple images are used. Otherwise, set this parameter to NULL .	--

NOTE

For details about the structure syntax, see **include/inc/dvpp/dvpp_config.h** in the DDK page.

Figure 8-6 shows the information about the input parameters

Figure 8-6 Example of input parameters



This figure shows the normalization of the cropping and resizing operations.

- When only cropping is performed, the resizing coefficient is 1.
- When only resizing is performed, the cropping size is the original image.

NOTE

- The width of the input image is 128-pixel aligned, the height of the input image is 16-pixel aligned, and the maximum resolution of the input image and that of the output image are both 4 KB.
- The following interfaces are required for buffer address alignment:
The buffer addresses of the input and output images must be both 128-byte aligned. The addresses must be in the same 4 GB space. **HIAI_DVPP_DMalloc** is required for applying for huge page memory.
The memory application API is **HIAI_DVPP_DMalloc(size)**
- The resizing ratio calculated based on the output image size and the input image size must be within the range of [0.03125, 4].
- The output buffer size must be calculated based on the output image resolution after the width is 128-pixel aligned and the height is 16-pixel aligned.
- The CMDLIST API supports a maximum of 32 images and each image supports a maximum of 256 ROIs.

Output Parameter

None. The output image is in **out_buffer** of **sum_out**. You can obtain the image by reading the buffer.

Table 8-27 shows the rank configuration table of the input and output image formats.

Calling Example

```
/****** Example description: *****/
This example uses a 1920 x 1088 YUV420 NV12 image as the input.
File name of the input image: "file1_1920x1088_nv12.yuv"
Operations on the input image: Crop five sub-images and resize them to 224 x 224.
/******
int main()
{
```

```
int ret = 0;
// Reads the input file.
char image_file[128] = "file1_1920x1088_nv12.yuv";
ifstream in_stream(image_file);
if (!in_stream.is_open()) {
    printf("can not open %s.\n", image_file);
    return -1;
}
in_stream.seekg(0, ios::end);
int file_len = in_stream.tellg();
char* in_buffer = (char *)HIAI_DVPP_DMAlloc(file_len);
in_stream.seekg(0, ios::beg);
in_stream.read(in_buffer, file_len);
in_stream.close();
// Starts to add the configuration of the first image.
IMAGE_CONFIG* image_config = (IMAGE_CONFIG*)malloc(sizeof(IMAGE_CONFIG));
image_config->in_buffer.bare_buffer = in_buffer; // Currently, all used images are uncompressed.
image_config->format = 0;
image_config->rank = 1; // When the input is in NV12 format, the output is also in NV12 format. Set
rank to 1.
image_config->bitwidth = 8; // Currently, only 8-bit units are used.
image_config->in_width = 1920;
image_config->in_height = 1088;
image_config->width_step = 1920; // The value is the same as the width.
// Start to add the configuration of the first cropped image. In this example, the cropped area is enclosed
by (0,0) and (511,511).
ROI_CONFIG* roi_config = &image_config->roi_config;
// Starts to configure the cropping parameters.
roi_config->crop_config.enable = 1; // Note: If only resizing is required, set this parameter to 0.
roi_config->crop_config.hmin = 0;
roi_config->crop_config.hmax = 511;
roi_config->crop_config.vmin = 0;
roi_config->crop_config.vmax = 511;
// Starts to configure output channel parameters.
roi_config->sum_out.enable = 1; // Enables the output of the first channel.
roi_config->sum_out.out_width = 224;
roi_config->sum_out.out_height = 224;
int out_buffer_size = AlignUp(224,128)*AlignUp(224,16)*3/2;
roi_config->sum_out.out_buffer = (char*)HIAI_DVPP_DMAlloc(out_buffer_size);
ROI_CONFIG* last_roi = roi_config;
// Starts to add the configurations of the second to fifth cropped images.
for (int i = 0 ; i < 4; i++) {
    ROI_CONFIG* roi_config = (ROI_CONFIG*)malloc(sizeof(ROI_CONFIG));
    // Starts to configure the cropping parameters.
    roi_config->crop_config.enable = 1;
    roi_config->crop_config.hmin = 100*i;
    roi_config->crop_config.hmax = 299 + 100*i;
    roi_config->crop_config.vmin = 100*i;
    roi_config->crop_config.vmax = 299 + 100*i;
    // Starts to configure output channel parameters.
    roi_config->sum_out.enable = 1;
    roi_config->sum_out.out_width = 224;
    roi_config->sum_out.out_height = 224;
    out_buffer_size = AlignUp(224,128)*AlignUp(224,16)*3/2;
    roi_config->sum_out.out_buffer = (char*)HIAI_DVPP_DMAlloc(out_buffer_size);
    roi_config->next = nullptr;
    last_roi->next = roi_config;
    last_roi = roi_config;
}
// Starts to call the CMDLIST API of the DVPP.
IDVPPAPI *pidvppapi = NULL;
// Calls the createdvppapi API only once, regardless of the subsequent callings.
ret = CreateDvppApi(pidvppapi);
if (ret != 0) {
    printf("creat dvpp api failed!\n");
    return -1;
}
dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = (void *) (image_config);
```

```
dvppApiCtlMsg.in_size = sizeof(IMAGE_CONFIG);  
ret = DvppCtl(pidvppapi, DVPP_CTL_CMDLIST_PROC, &dvppApiCtlMsg);  
if (0 != ret) {  
    printf("call cmdlist dvppctl process failed!\n");  
} else {  
    printf("cmdlist success.\n");  
}  
ret += DestroyDvppApi(pidvppapi);  
roi_config = image_config->roi_config.next;  
while (roi_config != nullptr) {  
    ROI_CONFIG* next_roi = roi_config->next;  
    UNMAP(roi_config->sum_out.out_buffer, out_buffer_size);  
    free(roi_config);  
    roi_config = next_roi;  
}  
UNMAP(image_config->roi_config.sum_out.out_buffer, out_buffer_size);  
free(image_config);  
UNMAP(in_buffer, file_len);  
return ret;  
}
```

8.3.5.4 VENC Functions and Parameters

You are not advised to use the VENC function of the earlier version, which will be removed in later versions. The VENC function in the earlier version has been migrated to new VENC APIs. For details, see [8.3.4 VENC Function Interfaces](#).

Function

The VENC module is used to encode YUV420 and YVU420 image data.

- The VENC supports the following input formats:
YUV420 semi-planner NV12/NV21-8bit
- The VENC supports the following output formats:
H264 BP/MP/HP
H265 MP

VENC Performance Specifications

Scenario	Total Frame Rate
1080p, 1 channel (multi-channel is not supported)	30fps

Input Parameter: venc_in_msg

Member Variable	Description	Value Range
Int width	Image width	The value is an even number ranging from 128 to 1920.
Int height	Image height	The value is an even number ranging from 128 to 1920.

Member Variable	Description	Value Range
Int coding_type	Video encoding protocol	0~3 0: H.265 MP 1: H.264 BP 2: H.264 MP 3: H.264 HP
Int YUV_store_type	YUV image storage format	The value is 0 or 1. 0: YUV420 Semi-Planar 1: YVU420 Semi-Planar
char* input_data	Address of the input image data	The value cannot be null.
Int input_data_size	Size of the input image data	The value is a positive number.
shared_ptr<AutoBuffer> output_data_queue	Address for output encoded stream	The buffer needs to be configured. This parameter is a smart pointer transferred into the DVPP, used for the caller to allocate the output buffer. You can read this buffer to obtain the output image. This parameter is mandatory.

Output Parameter

None

Calling Example

```
void TEST_3() //venc demo
{
    int read_file_size;
    int unit_file_size;
    FILE *fp = fopen(in_file_name, "rb");
    if (fp == NULL) {
        printf("open file: %s failed.\n", in_file_name);
        return;
    }
    printf("open yuv success \n");
    fseek(fp, 0L, SEEK_END);
    int file_size = ftell(fp);
    fseek(fp, 0L, SEEK_SET);

    venc_in_msg venc_msg;
    venc_msg.width = gWidth;
    venc_msg.height = gHigh;
```



```
venc_msg.coding_type = gFormat;
venc_msg.YUV_store_type = gBitwidth;
venc_msg.output_data_queue = make_shared<AutoBuffer>();
unit_file_size = gWidth * gHigh * 3 / 2 * MAX_FRAME_NUM_VENC; // The size of a single file is 16 frames.
dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = (void *)&venc_msg;
dvppApiCtlMsg.in_size = sizeof(venc_in_msg);
IDVPPAPI *pidvppapi = NULL;
CreateDvppApi(pidvppapi);
char out_filename[] = "venc.bin";
FILE *outputBufferFile;
outputBufferFile = fopen (out_filename, "wb+");

do{
    read_file_size = file_size>unit_file_size? unit_file_size:file_size; // The size of the file to be read each
time cannot exceed 16 frames.
    venc_msg.input_data = (char *)malloc(read_file_size);
    int read_len = fread(venc_msg.input_data, 1, read_file_size, fp);
    printf("file size is %d,read len is %d.\n", read_file_size, read_len);
    venc_msg.input_data_size = read_len;

    if (pidvppapi != NULL) {
        if (DvppCtl(pidvppapi, DVPP_CTL_VENC_PROC, &dvppApiCtlMsg) != 0) {
            printf("call dvppctl process failed!\n");
            DestroyDvppApi(pidvppapi);
            fclose(fp);
            fclose(outputBufferFile);
            return;
        }
        if (venc_msg.output_data_queue->getBufferSize() > 100) { // This is used to ensure that the
encoding result does not contain only the header information.
            char* out_buf = venc_msg.output_data_queue->getBuffer();
            int out_buf_size = venc_msg.output_data_queue->getBufferSize();
            int write_size = fwrite(out_buf, 1, out_buf_size, outputBufferFile);
            fflush(outputBufferFile);
        } else {
            printf("venc output data is too small : %d \n", venc_msg.output_data_queue->getBufferSize());
        }
    } else {
        printf("pidvppapi is null!\n");
    }

    if (venc_msg.input_data != NULL) {
        free(venc_msg.input_data);
        venc_msg.input_data = NULL;
    }

    file_size = file_size - unit_file_size;
} while (file_size > 0);

DestroyDvppApi(pidvppapi);
fclose(outputBufferFile);
fclose(fp);
return;
}
```

8.3.6 Calling Example

8.3.6.1 Implementing the VPC Function

Concepts

For details about the concepts such as cropping, resizing, overlaying, top offset, bottom offset, left offset, and right offset, see [8.3.1.3 VPC Function](#).

Example 1: Resizing of the Original Image

Set key parameters as follows:

- The width and height of the cropping area are the same as the actual width and height of the input image. The offset values of the cropped area are set as follows:
 - `leftOffset = 0`
 - `upOffset = 0`
 - `rightOffset - leftOffset + 1 = Actual width of the input image`
 - `downOffset - upOffset + 1 = Actual height of the input image`
- The width and height of the overwritten area are the width and height of the resized image. You can specify the position of the overwritten area. For example:
If the specified overwritten area is in the upper left corner of the output image, the offset values of the overwritten area are set as follows:
 - `leftOffset = 0`
 - `upOffset = 0`
 - `rightOffset - leftOffset + 1 = Width of the resized image`
 - `downOffset - upOffset + 1 = Height of the resized image`
- For original 8K image resizing, **inputFormat** and **outputFormat** must be set to **INPUT_YUV420_SEMI_PLANNER_UV** and **OUTPUT_YUV420SP_UV**, respectively, or **INPUT_YUV420_SEMI_PLANNER_VU** and **OUTPUT_YUV420SP_VU**, respectively. For original non-8K image resizing, set **inputFormat** and **outputFormat** by referring to [Table 8-10](#).

Sample Code:

In this sample, a YUV420SP image is scaled from 1080p to 720p.

```
void NewVpcTest1()
{
    uint32_t inWidthStride = 1920;
    uint32_t inHeightStride = 1080;
    uint32_t outWidthStride = 1280;
    uint32_t outHeightStride = 720;
    uint32_t inBufferSize = inWidthStride * inHeightStride * 3 / 2; // 1080P yuv420sp Image
    uint32_t outBufferSize = outWidthStride * outHeightStride * 3 / 2; // 720P yuv420sp blmage
    uint8_t* inBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMAlloc(inBufferSize)); // Construct an input picture.
    if (inBuffer == nullptr) {
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc input buffer");
        return;
    }
    uint8_t* outBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMAlloc(outBufferSize)); // Construct an output
    picture.
    if (outBuffer == nullptr) {
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc output buffer");
        HIAI_DVPP_DFree(inBuffer);
        inBuffer = nullptr;
        return;
    }

    FILE* fp = fopen("dvpp_vpc_1920x1080_nv12.yuv", "rb+");
    if (fp == nullptr) {
        HIAI_ENGINE_LOG(HIAI_ERROR, "fopen file failed");
        DFreeInOutBuffer(inBuffer, outBuffer);
        return;
    }
}
```

```
fread(inBuffer, 1, inBufferSize, fp);
fclose(fp);
fp = nullptr;
// Construct the input picture configuration.
std::shared_ptr<VpcUserImageConfigure> imageConfigure(new VpcUserImageConfigure);
imageConfigure->bareDataAddr = inBuffer;
imageConfigure->bareDataBufferSize = inBufferSize;
imageConfigure->widthStride = inWidthStride;
imageConfigure->heightStride = inHeightStride;
imageConfigure->inputFormat = INPUT_YUV420_SEMI_PLANNER_UV;
imageConfigure->outputFormat = OUTPUT_YUV420SP_UV;
imageConfigure->yuvSumEnable = false;
imageConfigure->cmdListBufferAddr = nullptr;
imageConfigure->cmdListBufferSize = 0;
std::shared_ptr<VpcUserRoiConfigure> roiConfigure(new VpcUserRoiConfigure);
roiConfigure->next = nullptr;
VpcUserRoiInputConfigure* inputConfigure = &roiConfigure->inputConfigure;
// Set the drawing area, [0,0] in the upper left corner of the area,
// and [1919,1079] in the lower right corner.
inputConfigure->cropArea.leftOffset = 0;
inputConfigure->cropArea.rightOffset = inWidthStride - 1;
inputConfigure->cropArea.upOffset = 0;
inputConfigure->cropArea.downOffset = inHeightStride - 1;
VpcUserRoiOutputConfigure* outputConfigure = &roiConfigure->outputConfigure;
outputConfigure->addr = outBuffer;
outputConfigure->bufferSize = outBufferSize;
outputConfigure->widthStride = outWidthStride;
outputConfigure->heightStride = outHeightStride;
// Set the map area, coordinate [0,0] in the upper left corner of the map area,
// and [1279,719] in the lower right corner.
outputConfigure->outputArea.leftOffset = 0;
outputConfigure->outputArea.rightOffset = outWidthStride - 1;
outputConfigure->outputArea.upOffset = 0;
outputConfigure->outputArea.downOffset = outHeightStride - 1;

imageConfigure->roiConfigure = roiConfigure.get();

IDVPPAPI *pidvppapi = nullptr;
int32_t ret = CreateDvppApi(pidvppapi);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "create dvpp api fail.");
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}
dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = static_cast<void*>(imageConfigure.get());
dvppApiCtlMsg.in_size = sizeof(VpcUserImageConfigure);
ret = DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "call vpc dvppctl process failed!");
    ret = DestroyDvppApi(pidvppapi);
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
} else {
    HIAI_ENGINE_LOG(HIAI_OK, "NewVpcTest1::call vpc dvppctl process success!");
}

FILE* outImageFp = fopen("NewVpcTest1Out.yuv", "wb+");
if (outImageFp == nullptr) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "open NewVpcTest1Out.yuv failed!");
    ret = DestroyDvppApi(pidvppapi);
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}

fwrite(outBuffer, 1, outBufferSize, outImageFp);

ret = DestroyDvppApi(pidvppapi);
```

```
DFreeInOutBuffer(inBuffer, outBuffer);  
fclose(outImageFp);  
outImageFp = nullptr;  
return;  
}
```

Example 2: Cropping and Resizing of a Single Image

Set key parameters as follows:

- The width and height of the cropped area are the width and height of the image before resizing. You can specify the position of the cropped area. For example:
If the specified cropped area is in the middle of the input image, the offset values of the cropped area are set as follows:
 - leftOffset = 100
 - rightOffset = 499
 - upOffset = 100
 - downOffset = 399
 - rightOffset – leftOffset + 1 = Width of the cropped area
 - downOffset – upOffset + 1 = Height of the cropped area
- The width and height of the overwritten area are the width and height of the resized image. You can specify the position of the overwritten area. For example:
If the specified overwritten area is in the middle of the output image, the offset values of the overwritten area are set as follows:
 - leftOffset = 256
 - rightOffset = 399
 - upOffset = 200
 - downOffset = 399
 - rightOffset – leftOffset + 1 = Width of the resized image
 - downOffset – upOffset + 1 = Height of the resized image

Sample Code:

In this sample, an image is cropped out of a YUV420SP 1080p image and resized. Then, it overwrites a 720p image (which is a blank image generated by an empty output buffer allocated by the user.) If you want to use an existing image as the output image, read the image to the output buffer after it is allocated. For details about the code example, see [Example 5: Overlaying](#).

```
void NewVpcTest2()  
{  
    uint32_t inWidthStride = 1920;  
    uint32_t inHeightStride = 1080;  
    uint32_t outWidthStride = 1280;  
    uint32_t outHeightStride = 720;  
    uint32_t inBufferSize = inWidthStride * inHeightStride * 3 / 2; // 1080P yuv420sp Image  
    uint32_t outBufferSize = outWidthStride * outHeightStride * 3 / 2; // 720P yuv420sp Image  
    uint8_t* inBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMalloc(inBufferSize)); // Construct an input  
    picture.  
    if (inBuffer == nullptr) {  
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc input buffer");  
        return;  
    }  
}
```

```
}
uint8_t* outBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMAlloc(outBufferSize)); // Construct an output
picture.
if (outBuffer == nullptr) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc output buffer");
    HIAI_DVPP_DFree(inBuffer);
    inBuffer = nullptr;
    return;
}
(void)memset_s(outBuffer, outBufferSize, 0x80, outBufferSize);

FILE* fp = fopen("dvpp_vpc_1920x1080_nv12.yuv", "rb+");
if (fp == nullptr) {
    HIAI_ENGINE_LOG(HIAI_ERROR, "fopen file failed");
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}

fread(inBuffer, 1, inBufferSize, fp);
fclose(fp);
fp = nullptr;
// Construct the input picture configuration
std::shared_ptr<VpcUserImageConfigure> imageConfigure(new VpcUserImageConfigure);
imageConfigure->bareDataAddr = inBuffer;
imageConfigure->bareDataBufferSize = inBufferSize;
imageConfigure->widthStride = inWidthStride;
imageConfigure->heightStride = inHeightStride;
imageConfigure->inputFormat = INPUT_YUV420_SEMI_PLANNER_UV;
imageConfigure->outputFormat = OUTPUT_YUV420SP_UV;
imageConfigure->yuvSumEnable = false;
imageConfigure->cmdListBufferAddr = nullptr;
imageConfigure->cmdListBufferSize = 0;
std::shared_ptr<VpcUserRoiConfigure> roiConfigure(new VpcUserRoiConfigure);
roiConfigure->next = nullptr;
VpcUserRoiInputConfigure* inputConfigure = &roiConfigure->inputConfigure;
// Set the drawing area, [100,100] in the upper left corner of the area, and [499,499] in the lower right
corner.
inputConfigure->cropArea.leftOffset = 100;
inputConfigure->cropArea.rightOffset = 499;
inputConfigure->cropArea.upOffset = 100;
inputConfigure->cropArea.downOffset = 499;
VpcUserRoiOutputConfigure* outputConfigure = &roiConfigure->outputConfigure;
outputConfigure->addr = outBuffer;
outputConfigure->bufferSize = outBufferSize;
outputConfigure->widthStride = outWidthStride;
outputConfigure->heightStride = outHeightStride;
// Set the map area, [256,200] in the upper left corner of the map area, and [399,399] in the lower right
corner.
outputConfigure->outputArea.leftOffset = 256; // The offset value must be 16-pixel-aligned.
outputConfigure->outputArea.rightOffset = 399;
outputConfigure->outputArea.upOffset = 200;
outputConfigure->outputArea.downOffset = 399;

imageConfigure->roiConfigure = roiConfigure.get();

IDVPPAPI *pidvppapi = nullptr;
int32_t ret = CreateDvppApi(pidvppapi);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "create dvpp api fail.");
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}
dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = static_cast<void*>(imageConfigure.get());
dvppApiCtlMsg.in_size = sizeof(VpcUserImageConfigure);
ret = DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "call vpc dvppctl process fail!");
    ret = DestroyDvppApi(pidvppapi);
}
```

```
    DFreeInOutBuffer(inBuffer, outBuffer);  
    return;  
} else {  
    HIAI_ENGINE_LOG(HIAI_OK, "NewVpcTest2::call vpc dvppctl process success!");  
}  
  
FILE* outImageFp = fopen("NewVpcTest2Out.yuv", "wb+");  
if (outImageFp == nullptr) {  
    HIAI_ENGINE_LOG(HIAI_ERROR, "open NewVpcTest2Out.yuv failed");  
    DestroyDvppApi(pidvppapi);  
    DFreeInOutBuffer(inBuffer, outBuffer);  
    return;  
}  
fwrite(outBuffer, 1, outBufferSize, outImageFp);  
  
ret = DestroyDvppApi(pidvppapi);  
DFreeInOutBuffer(inBuffer, outBuffer);  
fclose(outImageFp);  
outImageFp = nullptr;  
return;  
}
```

Example 3: Cropping, Resizing, and Stitching of Multiple Images

Set key parameters as follows: For details about how to configure the top offset, bottom offset, left offset, and right offset of the cropped area or overwritten area, see [Example 2: Cropping and Resizing of a Single Image](#).

Sample code: In this sample, multiple images are cropped out of a YUV420SP 1080p image. Then, the images are resized and stitched onto a 720p output image (which is a blank image generated by the empty output buffer allocated by the user). The position of the overwritten area is determined by the top offset, bottom offset, left offset, and right offset. If you want to use an existing image as the output image, read the image to the output buffer after it is allocated. For details about the code example, see [Example 5: Overlaying](#).

```
void NewVpcTest3()  
{  
    uint32_t inWidthStride = 1920;  
    uint32_t inHeightStride = 1080;  
    uint32_t outWidthStride = 1280;  
    uint32_t outHeightStride = 720;  
    uint32_t inBufferSize = inWidthStride * inHeightStride * 3 / 2; // 1080P yuv420sp Image  
    uint32_t outBufferSize = outWidthStride * outHeightStride * 3 / 2; // 720P yuv420sp Image  
    uint8_t* inBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMalloc(inBufferSize)); // Construct an input picture.  
    if (inBuffer == nullptr) {  
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc input buffer");  
        return;  
    }  
  
    FILE* fp = fopen("dvpp_vpc_1920x1080_nv12.yuv", "rb+");  
    if (fp == nullptr) {  
        HIAI_ENGINE_LOG(HIAI_ERROR, "fopen file failed");  
        HIAI_DVPP_DFree(inBuffer);  
        inBuffer = nullptr;  
        return;  
    }  
  
    fread(inBuffer, 1, inBufferSize, fp);  
    fclose(fp);  
    fp = nullptr;  
    // Construct the input picture configuration.  
    std::shared_ptr<VpcUserImageConfigure> imageConfigure(new VpcUserImageConfigure);  
    imageConfigure->bareDataAddr = inBuffer;  
    imageConfigure->bareDataBufferSize = inBufferSize;  
    imageConfigure->widthStride = inWidthStride;
```

```

imageConfigure->heightStride = inHeightStride;
imageConfigure->inputFormat = INPUT_YUV420_SEMI_PLANNER_UV;
imageConfigure->outputFormat = OUTPUT_YUV420SP_UV;
imageConfigure->yuvSumEnable = false;
imageConfigure->cmdListBufferAddr = nullptr;
imageConfigure->cmdListBufferSize = 0;
std::shared_ptr<VpcUserRoiConfigure> lastRoi;
std::vector<std::shared_ptr<VpcUserRoiConfigure>> roiVector;
for (uint32_t i = 0; i < 5; i++) {
    std::shared_ptr<VpcUserRoiConfigure> roiConfigure(new VpcUserRoiConfigure);
    roiVector.push_back(roiConfigure);
    roiConfigure->next = nullptr;
    VpcUserRoiInputConfigure* inputConfigure = &roiConfigure->inputConfigure;
    // Set the drawing area.
    inputConfigure->cropArea.leftOffset = 100 + i * 16;
    inputConfigure->cropArea.rightOffset = 499 + i * 16;
    inputConfigure->cropArea.upOffset = 100 + i * 16;
    inputConfigure->cropArea.downOffset = 499 + i * 16;
    VpcUserRoiOutputConfigure* outputConfigure = &roiConfigure->outputConfigure;
    uint8_t* outBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMAlloc(outBufferSize)); // Construct an input
picture
    if (outBuffer == nullptr) {
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc output buffer");
        HIAI_DVPP_DFree(inBuffer);
        inBuffer = nullptr;
        FreeMultipleRio(imageConfigure->roiConfigure);
        return;
    }
    (void)memset_s(outBuffer, outBufferSize, 0x80, outBufferSize);
    outputConfigure->addr = outBuffer;
    outputConfigure->bufferSize = outBufferSize;
    outputConfigure->widthStride = outWidthStride;
    outputConfigure->heightStride = outHeightStride;
    // Set the map area.
    outputConfigure->outputArea.leftOffset = 256 + i * 16; // The offset value must be 16-pixel-aligned.
    outputConfigure->outputArea.rightOffset = 399 + i * 16;
    outputConfigure->outputArea.upOffset = 256 + i * 16;
    outputConfigure->outputArea.downOffset = 399 + i * 16;
    if (i == 0) {
        imageConfigure->roiConfigure = roiConfigure.get();
        lastRoi = roiConfigure;
    } else {
        lastRoi->next = roiConfigure.get();
        lastRoi = roiConfigure;
    }
}

IDVPPAPI *pidvppapi = nullptr;
int32_t ret = CreateDvppApi(pidvppapi);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "create dvpp api fail.");
    HIAI_DVPP_DFree(inBuffer);
    inBuffer = nullptr;
    FreeMultipleRio(imageConfigure->roiConfigure);
    return;
}
dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = static_cast<void*>(imageConfigure.get());
dvppApiCtlMsg.in_size = sizeof(VpcUserImageConfigure);
ret = DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "call vpc dvppctl process fail!");
    ret = DestroyDvppApi(pidvppapi);
    HIAI_DVPP_DFree(inBuffer);
    inBuffer = nullptr;
    FreeMultipleRio(imageConfigure->roiConfigure);
    return;
} else {
    HIAI_ENGINE_LOG(HIAI_OK, "NewVpcTest3::call vpc dvppctl process success!");
}

```

```
}
FILE* outImageFp = nullptr;
uint32_t imageCount = 0;
char fileName[50] = {0};

while (imageConfigure->roiConfigure != nullptr) {
    int32_t safeFuncRet = sprintf_s(fileName, sizeof(fileName), "NewVpcTest3_%dOut.yuv", imageCount);
    if (safeFuncRet == -1) {
        HIAI_ENGINE_LOG(HIAI_ERROR, "sprintf_s fail, ret = %d", safeFuncRet);
        DestroyDvppApi(pidvppapi);
        HIAI_DVPP_DFree(inBuffer);
        inBuffer = nullptr;
        FreeMultipleRio(imageConfigure->roiConfigure);
        return;
    }
    outImageFp = fopen(fileName, "wb+");
    if (outImageFp == nullptr) {
        HIAI_ENGINE_LOG(HIAI_ERROR, "open %s failed!", fileName);
        DestroyDvppApi(pidvppapi);
        HIAI_DVPP_DFree(inBuffer);
        inBuffer = nullptr;
        FreeMultipleRio(imageConfigure->roiConfigure);
        return;
    }
    fwrite(imageConfigure->roiConfigure->outputConfigure.addr, 1,
        imageConfigure->roiConfigure->outputConfigure.bufferSize, outImageFp);
    fclose(outImageFp);
    outImageFp = nullptr;
    imageConfigure->roiConfigure = imageConfigure->roiConfigure->next;
    imageCount++;
}
ret = DestroyDvppApi(pidvppapi);
HIAI_DVPP_DFree(inBuffer);
inBuffer = nullptr;
FreeMultipleRio(imageConfigure->roiConfigure);
return;
}
```

Example 4: 8K Image Resizing

8K image resizing: Resizing is supported, the format conversion between YUV420SP NV12 and YUV420SP NV21 is supported, but cropping is not supported.

Sample code: In this sample, a YUV420SP image is scaled from 8129 x 8192 to 4000 x 4000.

```
void NewVpcTest4()
{
    uint32_t inWidthStride = 8192; // No need for 128 byte alignment
    uint32_t inHeightStride = 8192; // No need for 16 byte alignment
    uint32_t outWidthStride = 4000; // No need for 128 byte alignment
    uint32_t outHeightStride = 4000; // No need for 16 byte alignment
    uint32_t inBufferSize = inWidthStride * inHeightStride * 3 / 2;
    uint32_t outBufferSize = outWidthStride * outHeightStride * 3 / 2; // Construct dummy data
    uint8_t* inBuffer = (uint8_t*)HIAI_DVPP_DMAlloc(inBufferSize); // Construct input image
    if (inBuffer == nullptr) {
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc roiOutput buffer");
        return;
    }
    uint8_t* outBuffer = (uint8_t*)HIAI_DVPP_DMAlloc(outBufferSize); // Construct output image
    if (outBuffer == nullptr) {
        HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc roiOutput buffer");
        HIAI_DVPP_DFree(inBuffer);
        inBuffer = nullptr;
        return;
    }
    FILE* fp = fopen("dvpp_vpc_8192x8192_nv12.yuv", "rb+");
```



```

if (fp == nullptr) {
    HIAI_ENGINE_LOG(HIAI_ERROR, "fopen file failed");
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}
fread(inBuffer, 1, inBufferSize, fp);
fclose(fp);
fp = nullptr;
std::shared_ptr<VpcUserImageConfigure> imageConfigure(new VpcUserImageConfigure);
imageConfigure->bareDataAddr = inBuffer;
imageConfigure->bareDataBufferSize = inBufferSize;
imageConfigure->isCompressData = false;
imageConfigure->widthStride = inWidthStride;
imageConfigure->heightStride = inHeightStride;
imageConfigure->inputFormat = INPUT_YUV420_SEMI_PLANNER_UV;
imageConfigure->outputFormat = OUTPUT_YUV420SP_UV;
imageConfigure->yuvSumEnable = false;
imageConfigure->cmdListBufferAddr = nullptr;
imageConfigure->cmdListBufferSize = 0;
std::shared_ptr<VpcUserRoiConfigure> roiConfigure(new VpcUserRoiConfigure);
roiConfigure->next = nullptr;
VpcUserRoiInputConfigure* inputConfigure = &roiConfigure->inputConfigure; // Set the roi area
inputConfigure->cropArea.leftOffset = 0;
inputConfigure->cropArea.rightOffset = inWidthStride - 1;
inputConfigure->cropArea.upOffset = 0;
inputConfigure->cropArea.downOffset = inHeightStride - 1;
VpcUserRoiOutputConfigure* outputConfigure = &roiConfigure->outputConfigure;
outputConfigure->addr = outBuffer;
outputConfigure->bufferSize = outBufferSize;
outputConfigure->widthStride = outWidthStride;
outputConfigure->heightStride = outHeightStride; // Set the map area
outputConfigure->outputArea.leftOffset = 0;
outputConfigure->outputArea.rightOffset = outWidthStride - 1;
outputConfigure->outputArea.upOffset = 0;
outputConfigure->outputArea.downOffset = outHeightStride - 1;
imageConfigure->roiConfigure = roiConfigure.get();
IDVPPAPI *pidvppapi = nullptr;
int32_t ret = CreateDvppApi(pidvppapi);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "create dvpp api fail.");
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}
dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = static_cast<void*>(imageConfigure.get());
dvppApiCtlMsg.in_size = sizeof(VpcUserImageConfigure);
ret = DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "call vpc dvppctl process fail!");
    ret = DestroyDvppApi(pidvppapi);
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
} else {
    HIAI_ENGINE_LOG(HIAI_OK, "NewVpcTest4::call vpc dvppctl process success!");
}

FILE* outImageFp = fopen("NewVpcTest4Out.yuv", "wb+");
if (outImageFp == nullptr) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "open NewVpcTest4Out.yuv fail!");
    ret = DestroyDvppApi(pidvppapi);
    DFreeInOutBuffer(inBuffer, outBuffer);
    return;
}
fwrite(outBuffer, 1, outBufferSize, outImageFp);
fclose(outImageFp);
outImageFp = nullptr;
ret = DestroyDvppApi(pidvppapi);
DFreeInOutBuffer(inBuffer, outBuffer);

```

```
    return;  
}
```

Example 5: Overlaying

To read an existing image into the buffer for storing the output image, overlay the overwritten area on the output image. This requires you to write the code logic to read the image to the buffer. After allocating the output buffer by using code **uint8_t* outBuffer = static_cast<uint8_t*>(HIAI_DVPP_DMalloc(outBufferSize))**, add the following code:

```
FILE* fpOut = fopen("vpcOut.yuv", "rb+");  
if (fpOut == nullptr) {  
    HIAI_ENGINE_LOG(HIAI_ERROR, "fopen file failed.");  
    fclose(fpOut);  
    return;  
}  
fread(outBuffer, 1, outBufferSize, fpOut);  
fclose(fpOut);
```

8.3.6.2 Implementing the JPEG Function

- The memory size can be obtained by calling [DvppGetOutParameter](#). The output memory can be specified and freed by the user. The calling example is as follows:

```
void TEST_JPEGE_CUSTOM_MEMORY()  
{  
    sjpegIn inData;  
    sjpegOut outData;  
  
    inData.width      = g_width;  
    inData.height     = g_high;  
    inData.heightAligned = g_high; // no need to align  
    inData.format      = (eEncodeFormat)g_format;  
    inData.level       = 100;  
  
    inData.stride = ALIGN_UP(inData.width * 2, 16);  
    inData.bufSize = inData.stride * inData.heightAligned;  
    if (JPGENC_FORMAT_YUV420 == (inData.format & JPGENC_FORMAT_BIT)) {  
        inData.stride = ALIGN_UP(inData.width, 16);  
        inData.bufSize = inData.stride * inData.heightAligned * 3 / 2;  
    }  
  
    void* addrOrig = HIAI_DVPP_DMalloc(inData.bufSize);  
    if (addrOrig == nullptr) {  
        HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "can not alloc input buffer");  
        return;  
    }  
    inData.buf = reinterpret_cast<unsigned char*>(addrOrig);  
  
    unsigned char* tmpAddr = nullptr;  
  
    do {  
        // load img file  
        FILE* fpIn = fopen(g_inFileName, "rb");  
        if (nullptr == fpIn) {  
            HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "can not open input file");  
            break;  
        }  
        // only copy valid image data, other part is pending  
        if (JPGENC_FORMAT_YUV420 == (inData.format & JPGENC_FORMAT_BIT)) {  
            // for yuv420semi-planar format, like nv12 / nv21  
            HIAI_ENGINE_LOG("input yuv 420 data");  
            // for y data  
            for (uint32_t j = 0; j < inData.height; j++) {  
                fread(inData.buf + j * inData.stride, 1, inData.width, fpIn);  
            }  
        }  
    } while (0);  
}
```

```
    }  
    // for uv data  
    for (uint32_t j = inData.heightAligned; j < inData.heightAligned + inData.height / 2; j++) {  
        fread(inData.buf + j * inData.stride, 1, inData.width, fpln);  
    }  
} else {  
    // for yuv422packed format, like uyvy / vyuy / yuyv / yvyu  
    HIAI_ENGINE_LOG("input yuv 422 data");  
    for (uint32_t j = 0; j < inData.height; j++) {  
        fread(inData.buf + j * inData.stride, 1, inData.width * 2, fpln);  
    }  
}  
fclose(fpln);  
fpln = nullptr;  
  
int32_t ret = DvppGetOutParameter((void*)&inData, (void*)&outData,  
GET_JPEGE_OUT_PARAMETER);  
if (ret != 0) {  
    HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "call DvppGetOutParameter process failed");  
    break;  
}  
// The obtained jpgSize is an estimated value. The actual data length may be less than the  
estimated value.  
// After the DvppCtl API is called, the jpgSize value is the actual value after encoding.  
HIAI_ENGINE_LOG("outdata size is %d", outData.jpgSize);  
tmpAddr = (unsigned char*)HIAI_DVPP_DMalloc(outData.jpgSize);  
if (tmpAddr == nullptr) {  
    HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "can not alloc output buffer");  
    break;  
}  
uint32_t size = outData.jpgSize;  
  
// call jpeg process  
dvppapi_ctl_msg dvppApiCtlMsg;  
dvppApiCtlMsg.in = (void *)&inData;  
dvppApiCtlMsg.in_size = sizeof(inData);  
dvppApiCtlMsg.out = (void *)&outData;  
dvppApiCtlMsg.out_size = sizeof(outData);  
  
if(!IsHandleJpegeCtlSucc(dvppApiCtlMsg, tmpAddr, size, &outData)) {  
    break;  
}  
} while (0); // for resource inData.buf  
if (addrOrig != nullptr) {  
    HIAI_DVPP_DFree(addrOrig);  
    addrOrig = nullptr;  
}  
if (tmpAddr != nullptr) { // Note: tmpAddr is used to free the buffer, because address offset occurs  
after outData.jpgData is processed by the JPEG.  
    HIAI_DVPP_DFree(tmpAddr);  
    tmpAddr = nullptr;  
}  
}
```

- If the output buffer is not specified by the user, DVPP allocates the buffer internally. The **cbFree()** callback function should be called to free the buffer. The calling example is as follows:

```
void JpegeProcess()  
{  
    sJpegIn inData;  
    sJpegOut outData;  
  
    inData.width      = g_width;  
    inData.height     = g_high;  
    inData.heightAligned = g_high; // no need to align  
    inData.format      = (eEncodeFormat)g_format;  
    inData.level       = 100;  
  
    if (JPGENC_FORMAT_YUV420 == (inData.format & JPGENC_FORMAT_BIT)) {
```

```

        inData.stride = ALIGN_UP(inData.width, 16);
        inData.bufSize = inData.stride * inData.heightAligned * 3 / 2;
    } else {
        inData.stride = ALIGN_UP(inData.width * 2, 16);
        inData.bufSize = inData.stride * inData.heightAligned;
    }

    void* addrOrig = HIAI_DVPP_DMAlloc(inData.bufSize);

    if (addrOrig == nullptr) {
        HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "can not alloc input buffer");
        return;
    }
    inData.buf = reinterpret_cast<unsigned char*>(addrOrig);

    do {
        // load img file
        FILE* fpln = fopen(g_inFileName, "rb");
        if (fpln == nullptr) {
            HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "can not open input file");
            break;
        }
        // only copy valid image data, other part is pending
        if (JPGENC_FORMAT_YUV420 == (inData.format & JPGENC_FORMAT_BIT)) {
            // for yuv420semi-planar format, like nv12 / nv21
            HIAI_ENGINE_LOG("input yuv 420 data");
            // for y data
            for (uint32_t j = 0; j < inData.height; j++) {
                fread(inData.buf + j * inData.stride, 1, inData.width, fpln);
            }
            // for uv data
            for (uint32_t j = inData.heightAligned; j < inData.heightAligned + inData.height / 2; j++) {
                fread(inData.buf + j * inData.stride, 1, inData.width, fpln);
            }
        } else {
            // for yuv422packed format, like uyvy / vyuy / yuyv / yvyu
            HIAI_ENGINE_LOG("input yuv 422 data");
            for (uint32_t j = 0; j < inData.height; j++) {
                fread(inData.buf + j * inData.stride, 1, inData.width * 2, fpln);
            }
        }
    }
    fclose(fpln);
    fpln = nullptr;

    // call jpege process
    dvppapi_ctl_msg dvppApiCtlMsg;
    dvppApiCtlMsg.in = (void*)&inData;
    dvppApiCtlMsg.in_size = sizeof(inData);
    dvppApiCtlMsg.out = (void*)&outData;
    dvppApiCtlMsg.out_size = sizeof(outData);

    IDVPPAPI *pidvppapi = nullptr;
    CreateDvppApi(pidvppapi);
    if (pidvppapi == nullptr) {
        HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "can not open dvppapi engine");
        break;
    }

    JpegeProcessBranch(pidvppapi, dvppApiCtlMsg, outData);

    DestroyDvppApi(pidvppapi);

} while (0); // for resource inData.buf
if (addrOrig != nullptr) {
    HIAI_DVPP_DFree(addrOrig);
    addrOrig = nullptr;
}
}

```

```
/*
 * only handle jpeg process.
 */
void JpegeProcessBranch(IDVPPAPI*& pidvppapi, dvppapi_ctl_msg& dvppApiCtlMsg, sjpegeOut&
outData)
{
    do {
        for (int i = 0; i < g_loop; i++) { // same picture loop test
            if (DvppCtl(pidvppapi, DVPP_CTL_JPEGE_PROC, &dvppApiCtlMsg)) {
                HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "call jpeg encoder fail");
                break;
            }
            if (i < g_loop - 1) {
                outData.cbFree();
                outData.jpgData = nullptr;
            }
        }
        stringstream outFile;
        outFile << g_outFileName << "_t" << std::this_thread::get_id() << ".jpg";

        FILE* fpOut = fopen(outFile.str().c_str(), "wb");
        if (fpOut != nullptr) {
            fwrite(outData.jpgData, 1, outData.jpgSize, fpOut);
            fflush(fpOut);
            fclose(fpOut);
            fpOut = nullptr;
        } else {
            HIAI_ENGINE_LOG(HIAI_JPEGE_CTL_ERROR, "call not save result file %s ", outFile.str().c_str());
        }

        outData.cbFree();
        outData.jpgData = nullptr;
        HIAI_ENGINE_LOG(HIAI_OK, "jpeg encode process completed");
    } while (0); // for resource pddvppapi
}
```

8.3.6.3 Implementing the JPEGD Function

- The memory size can be obtained by calling [DvppGetOutParameter](#). The output memory can be specified and freed by the user. The calling example is as follows:

```
void TEST_JPEGD_CUSTOM_MEMORY()
{
    struct JpegdIn jpegdInData;
    struct JpegdOut jpegdOutData;

    if (g_rank) {
        jpegdInData.isYUV420Need = false;
    }

    FILE *fpIn = fopen(g_inFileName, "rb");
    if (fpIn == nullptr) {
        HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "can not open file %s.", g_inFileName);
        return;
    }

    do { // for resource fpIn
        fseek(fpIn, 0, SEEK_END);
        // the buf len should 8 byte larger, the driver asked
        uint32_t fileLen = ftell(fpIn);
        jpegdInData.jpegDataSize = fileLen + 8;
        fseek(fpIn, 0, SEEK_SET);

        void* addrOrig = HIAI_DVPP_DMAlloc(jpegdInData.jpegDataSize);
        if (addrOrig == nullptr) {
            HIAI_ENGINE_LOG(HIAI_JPEGD_CTL_ERROR, "can not alloc input buffer");
            fclose(fpIn);
        }
    }
```

```

        fpln = nullptr;
        break;
    }

    jpegdInData.jpegData = reinterpret_cast<unsigned char*>(addrOrig);
    do { // for resource inBuf
        fread(jpegdInData.jpegData, 1, fileLen, fpln);
        fclose(fpln);
        fpln = nullptr;
        int32_t ret = DvppGetOutParameter((void*)&jpegdInData, (void*)&jpegdOutData,
GET_JPEGD_OUT_PARAMETER);
        if (ret != 0) {
            HIAI_ENGINE_LOG(HIAI_JPEGD_CTL_ERROR, "call DvppGetOutParameter process failed");
            break;
        }
        HIAI_ENGINE_LOG("jpegd out size is %d", jpegdOutData.yuvDataSize);
        jpegdOutData.yuvData = (unsigned char*)HIAI_DVPP_DMAlloc(jpegdOutData.yuvDataSize);
        if (jpegdOutData.yuvData == nullptr) {
            HIAI_ENGINE_LOG(HIAI_JPEGD_CTL_ERROR, "can not alloc output buffer");
            break;
        }
        HIAI_ENGINE_LOG("jpegdOutData.yuvData is %p", jpegdOutData.yuvData);

        dvppapi_ctl_msg dvppApiCtlMsg;
        dvppApiCtlMsg.in = (void *)&jpegdInData;
        dvppApiCtlMsg.in_size = sizeof(jpegdInData);
        dvppApiCtlMsg.out = (void *)&jpegdOutData;
        dvppApiCtlMsg.out_size = sizeof(jpegdOutData);

        IDVPPAPI *pidvppapi = nullptr;
        CreateDvppApi(pidvppapi);

        if (pidvppapi != nullptr) {
            for (int i = 0; i < g_loop; i++) {
                if (0 != DvppCtl(pidvppapi, DVPP_CTL_JPEGD_PROC, &dvppApiCtlMsg)) {
                    HIAI_ENGINE_LOG(HIAI_JPEGD_CTL_ERROR, "call dvppctl process failed");
                    break;
                }
            }
            DestroyDvppApi(pidvppapi);
        } else {
            HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "can not create dvpp api");
            break;
        }
        WriteTestJpegdResultToFile(&jpegdOutData);
    } while (0); // for resource inBuf

    if (addrOrig != nullptr) {
        HIAI_DVPP_DFree(addrOrig);
        addrOrig = nullptr;
    }
    if (jpegdOutData.yuvData != nullptr) {
        HIAI_DVPP_DFree(jpegdOutData.yuvData);
        jpegdOutData.yuvData = nullptr;
    }
} while (0); // for resource fpln
}

```

- If the output buffer is not specified by the user, DVPP allocates the buffer internally. The **cbFree()** callback function should be called to free the buffer. The calling example is as follows:

```

void JpegdProcess()
{
    struct jpegd_raw_data_info jpegdInData;
    struct jpegd_yuv_data_info jpegdOutData;

    if (g_rank) {
        jpegdInData.IsYUV420Need = false;
    }
}

```

```

FILE *fpIn = fopen(g_inFileName, "rb");
if (fpIn == nullptr) {
    HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "can not open file %s.", g_inFileName);
    return;
}

do { // for resource fpIn
    fseek(fpIn, 0, SEEK_END);
    // the buf len should 8 byte larger, the driver asked
    uint32_t fileLen = ftell(fpIn);
    jpegInData.jpeg_data_size = fileLen + 8;
    fseek(fpIn, 0, SEEK_SET);

    void* addrOrig = HIAI_DVPP_DMAlloc(jpegInData.jpeg_data_size);
    if (addrOrig == nullptr) {
        HIAI_ENGINE_LOG(HIAI_JPEGD_CTL_ERROR, "can not alloc input buffer");
        break;
    }

    jpegInData.jpeg_data = reinterpret_cast<unsigned char*>(addrOrig);

    do { // for resource inBuf
        fread(jpegInData.jpeg_data, 1, fileLen, fpIn);
        dvppapi_ctl_msg dvppApiCtlMsg;
        dvppApiCtlMsg.in = (void*)&jpegInData;
        dvppApiCtlMsg.in_size = sizeof(jpegInData);
        dvppApiCtlMsg.out = (void*)&jpegOutData;
        dvppApiCtlMsg.out_size = sizeof(jpegOutData);

        IDVPPAPI *pidvppapi = nullptr;
        CreateDvppApi(pidvppapi);

        if (pidvppapi != nullptr) {
            for (int i = 0; i < g_loop; i++) { // same picture loop test
                if (DvppCtl(pidvppapi, DVPP_CTL_JPEGD_PROC, &dvppApiCtlMsg) != 0) {
                    HIAI_ENGINE_LOG(HIAI_JPEGD_CTL_ERROR, "call dvppctl process failed");
                    break;
                }
                if (i < g_loop - 1) {
                    jpegOutData.cbFree();
                    jpegOutData.yuv_data = nullptr;
                }
            }
            DestroyDvppApi(pidvppapi);
        } else {
            HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "can not create dvpp api");
            break;
        }

        WriteJpegdProcessResultToFile(&jpegOutData);
        jpegOutData.cbFree();
        jpegOutData.yuv_data = nullptr;
    } while (0); // for resource inBuf

    if (addrOrig != nullptr) {
        HIAI_DVPP_DFree(addrOrig);
        addrOrig = nullptr;
    }

} while (0); // for resource fpIn
fclose(fpIn);
fpIn = nullptr;
}
    
```

8.3.6.4 Implementing the PNGD Function

The memory size can be obtained by calling [DvppGetOutParameter](#). The output memory can be specified and freed by the user. The calling example is as follows:

```
void TEST_PNGD_CUSTOM_MEMORY()
{
    FILE* fpIn = fopen(g_inFileName, "rb");
    if (nullptr == fpIn) {
        HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "can not open file %s.", g_inFileName);
        return;
    }

    fseek(fpIn, 0, SEEK_END);
    uint32_t fileLen = ftell(fpIn);
    fseek(fpIn, 0, SEEK_SET);
    void* inbuf = HIAI_DVPP_DMAlloc(fileLen);
    if (inbuf == nullptr) {
        HIAI_ENGINE_LOG(HIAI_PNGD_CTL_ERROR, "can not alloc input buffer");
        fclose(fpIn);
        fpIn = nullptr;
        return;
    }

    // prepare msg
    struct PngInputInfoAPI inputPngData; // input data
    inputPngData.inputData = inbuf; // input png data
    inputPngData.inputSize = fileLen; // the size of png data

    HIAI_ENGINE_LOG("inputPngData.address: %p", inputPngData.address);
    HIAI_ENGINE_LOG("inputPngData.size: %u", inputPngData.size);
    HIAI_ENGINE_LOG("inputPngData.inputData: %p", inputPngData.inputData);
    HIAI_ENGINE_LOG("inputPngData.inputSize: %u", inputPngData.inputSize);

    fread(inputPngData.inputData, 1, fileLen, fpIn);
    fclose(fpIn);
    fpIn = nullptr;

    if (g_transform == 1) { // Whether format conversion
        inputPngData.transformFlag = 1; // RGBA -> RGB
    } else {
        inputPngData.transformFlag = 0;
    }

    struct PngOutputInfoAPI outputPngData; // output data
    int32_t ret = DvppGetOutParameter((void*)&inputPngData, (void*)&outputPngData,
    GET_PNGD_OUT_PARAMETER);
    if (ret != 0) {
        HIAI_ENGINE_LOG(HIAI_PNGD_CTL_ERROR, "call DvppGetOutParameter process failed");
        HIAI_DVPP_DFree(inbuf);
        inbuf = nullptr;
        return;
    }
    HIAI_ENGINE_LOG("pngd out size is %d", outputPngData.size);
    outputPngData.address = HIAI_DVPP_DMAlloc(outputPngData.size);
    if (outputPngData.address == nullptr) {
        HIAI_ENGINE_LOG(HIAI_PNGD_CTL_ERROR, "can not alloc output buffer");
        HIAI_DVPP_DFree(inbuf);
        inbuf = nullptr;
        return;
    }

    dvppapi_ctl_msg dvppApiCtlMsg; // call the interface msg
    dvppApiCtlMsg.in = (void*)&inputPngData;
    dvppApiCtlMsg.in_size = sizeof(struct PngInputInfoAPI);
    dvppApiCtlMsg.out = (void*)&outputPngData;
    dvppApiCtlMsg.out_size = sizeof(struct PngOutputInfoAPI);

    // use interface
```



```
IDVPPAPI *pidvppapi = nullptr;
CreateDvppApi(pidvppapi); // create dvppapi

if (pidvppapi != nullptr) { // use DvppCtl interface to handle DVPP_CTL_PNGD_PROC
    for (int i = 0; i < g_loop; i++) {
        if (-1 == DvppCtl(pidvppapi, DVPP_CTL_PNGD_PROC, &dvppApiCtlMsg)) {
            HIAI_ENGINE_LOG(HIAI_PNGD_CTL_ERROR, "call dvppctl process failed");
            HIAI_DVPP_DFree(inbuf);
            inbuf = nullptr;
            HIAI_DVPP_DFree(outputPngData.address);
            outputPngData.address = nullptr;
            DestroyDvppApi(pidvppapi); // destroy dvppapi
            return;
        }
    }
} else {
    HIAI_ENGINE_LOG(HIAI_PNGD_CTL_ERROR, "can not get dvpp api");
}

DestroyDvppApi(pidvppapi);

HIAI_ENGINE_LOG("output result");

char* pAddr = (char*)outputPngData.outputData;
FILE* fpOut = fopen(g_outFileName, "wb");
if (fpOut == nullptr) {
    HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "can not open file %s.", g_outFileName);
    HIAI_DVPP_DFree(inbuf);
    inbuf = nullptr;
    HIAI_DVPP_DFree(outputPngData.address);
    outputPngData.address = nullptr;
    return;
}

int size = outputPngData.width * 4;
if (outputPngData.format == PNGD_RGB_FORMAT_NUM) {
    size = outputPngData.width * 3;
} else if (outputPngData.format == PNGD_RGBA_FORMAT_NUM) {
    size = outputPngData.width * 4;
}
// copy valid image data from every line.
for (int i = 0; i < outputPngData.high; i++) {
    fwrite(pAddr + (int)(i * outputPngData.widthAlign), size, 1, fpOut);
}

fclose(fpOut);
fpOut = nullptr;
HIAI_DVPP_DFree(inbuf);
inbuf = nullptr;
HIAI_DVPP_DFree(outputPngData.address);
outputPngData.address = nullptr;
return;
}
```

8.3.6.5 Implementing the VDEC Function

For a video stream, after you create an instance by calling [CreateVdecApi](#), you must use the same instance to call [VdecCtl](#) for video decoding, and then call [DestroyVdecApi](#) to release the instance.

During video decoding, if you need to switch from a video stream to another video stream, you must release the instance of the previous stream by calling [DestroyVdecApi](#), and then create an instance by calling [CreateVdecApi](#) to process the new video stream.

This example reads the H.264 stream file named **test_file**, calls the VDEC function, and saves the decoding result to the **output_dir** directory.

```
// User-defined sub-class
class HIAI_DATA_SP_SON: public HIAI_DATA_SP {
public:
    ~ HIAI_DATA_SP_SON ()
    {
        //destruct here;
    }
    // User-defined member functions. The following is only an example.
    uint8_t GetTotalFrameNum()
    {
        return info_.totalFrameNum;
    }

private:
    // User-defined member variables. The INFO structure is only an example.
    struct INFO {
        uint8_t totalFrameNum;
        uint8_t frameRate;
    }info_;
};

IDVPPAPI * pidvppapi_vpc = NULL;
IDVPPAPI * pidvppapi_vdec = NULL;

// The callback function is used as an example. The caller needs to redefine the callback function as
// required.
void FrameReturn(FRAME* frame, void* hiaiData)
{
    static int32_t imageCount = 0;
    imageCount++;
    HIAI_ENGINE_LOG("call save frame number:[%d], width:[%d], height:[%d]", imageCount, frame->width,
frame->height);
    // The image directly output by vdec is an image of hfbc compression format, which cannot be directly
    // displayed.
    // It is necessary to call vpc to convert hfbc to an image of uncompressed format to display.
    HIAI_ENGINE_LOG("start call vpc interface to translate hfbc.");
    IDVPPAPI* dvppHandle = nullptr;
    int32_t ret = CreateDvppApi(dvppHandle);
    if (ret != 0) {
        HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "creat dvpp api fail.");
        return;
    }

    // Construct vpc input configuration.
    std::shared_ptr<VpcUserImageConfigure> userImage(new VpcUserImageConfigure);
    // bareDataAddr should be null which the image is hfbc.
    userImage->bareDataAddr = nullptr;
    userImage->bareDataBufferSize = 0;
    userImage->widthStride = frame->width;
    userImage->heightStride = frame->height;
    // Configuration input format
    string imageFormat(frame->image_format);
    if (frame->bitdepth == 8) {
        if (imageFormat == "nv12") {
            userImage->inputFormat = INPUT_YUV420_SEMI_PLANNER_UV;
        } else {
            userImage->inputFormat = INPUT_YUV420_SEMI_PLANNER_VU;
        }
    } else {
        if (imageFormat == "nv12") {
            userImage->inputFormat = INPUT_YUV420_SEMI_PLANNER_UV_10BIT;
        } else {
            userImage->inputFormat = INPUT_YUV420_SEMI_PLANNER_VU_10BIT;
        }
    }
    userImage->outputFormat = OUTPUT_YUV420SP_UV;
    userImage->isCompressData = true;
    // Configure hfbc input address
    VpcCompressDataConfigure* compressDataConfigure = &userImage->compressDataConfigure;
```

```
uint64_t baseAddr = (uint64_t)frame->buffer;
compressDataConfigure->lumaHeadAddr = baseAddr + frame->offset_head_y;
compressDataConfigure->chromaHeadAddr = baseAddr + frame->offset_head_c;
compressDataConfigure->lumaPayloadAddr = baseAddr + frame->offset_payload_y;
compressDataConfigure->chromaPayloadAddr = baseAddr + frame->offset_payload_c;
compressDataConfigure->lumaHeadStride = frame->stride_head;
compressDataConfigure->chromaHeadStride = frame->stride_head;
compressDataConfigure->lumaPayloadStride = frame->stride_payload;
compressDataConfigure->chromaPayloadStride = frame->stride_payload;

userImage->yuvSumEnable = false;
userImage->cmdListBufferAddr = nullptr;
userImage->cmdListBufferSize = 0;
// Configure the roi area and output area
std::shared_ptr<VpcUserRoiConfigure> roiConfigure(new VpcUserRoiConfigure);
roiConfigure->next = nullptr;
userImage->roiConfigure = roiConfigure.get();
VpcUserRoiInputConfigure* roiInput = &roiConfigure->inputConfigure;
roiInput->cropArea.leftOffset = 0;
roiInput->cropArea.rightOffset = frame->width - 1;
roiInput->cropArea.upOffset = 0;
roiInput->cropArea.downOffset = frame->height - 1;
VpcUserRoiOutputConfigure* roiOutput = &roiConfigure->outputConfigure;
roiOutput->outputArea.leftOffset = 0;
roiOutput->outputArea.rightOffset = frame->width - 1;
roiOutput->outputArea.upOffset = 0;
roiOutput->outputArea.downOffset = frame->height - 1;
roiOutput->bufferSize = ALIGN_UP(frame->width, 16) * ALIGN_UP(frame->height, 2) * 3 / 2;
roiOutput->addr = (uint8_t*)HIAI_DVPP_DMAlloc(roiOutput->bufferSize);
if (roiOutput->addr == nullptr) {
    HIAI_ENGINE_LOG(HIAI_VPC_CTL_ERROR, "can not alloc roiOutput buffer");
    DestroyDvppApi(dvppHandle);
    return;
}
roiOutput->widthStride = ALIGN_UP(frame->width, 16);
roiOutput->heightStride = ALIGN_UP(frame->height, 2);

dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = (void*)(userImage.get());
dvppApiCtlMsg.in_size = sizeof(VpcUserImageConfigure);
ret = DvppCtl(dvppHandle, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_ERROR, "call dvppctl fail");
    HIAI_DVPP_DFree(roiOutput->addr);
    roiOutput->addr = nullptr;
    DestroyDvppApi(dvppHandle);
    return;
}
ret = FrameReturnSaveVpcResult(frame, roiOutput->addr, imageCount);
if (ret != 0) {
    HIAI_ENGINE_LOG(HIAI_ERROR, "save vpc result fail");
}
HIAI_DVPP_DFree(roiOutput->addr);
roiOutput->addr = nullptr;
DestroyDvppApi(dvppHandle);
return;
}

/*
*Error reporting callback function. If you do not need error information, you do not need to define this
function.
*/
void ErrReport(VDECERR* vdecErr)
{
    if (vdecErr != nullptr) {
        HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "vdec error code is %d, channelId = %u\n", vdecErr-
>errType, vdecErr->channelId);
    }
}
```

```
// Main entry of the test function
void TEST_VDEC()
{
    char outputDir[20] = {0};
    int32_t safeFuncRet = memset_s(outputDir, sizeof(outputDir), 0, 20);
    if (safeFuncRet != EOK) {
        HIAI_ENGINE_LOG(HIAI_ERROR, "memset_s fail");
        return;
    }
    safeFuncRet = strncpy_s(outputDir, sizeof(outputDir), "output_dir", strlen("output_dir"));
    if (safeFuncRet != EOK) {
        HIAI_ENGINE_LOG(HIAI_ERROR, "strncpy_s fail");
        return;
    }
    if (access(outputDir, F_OK) == -1) {
        if (mkdir(outputDir, 0700) < 0) { // directory authority: 0700
            HIAI_ENGINE_LOG(HIAI_VDEC_CTL_ERROR, "create dir failed.");
            return;
        }
    }
    else if (access(outputDir, R_OK | W_OK | X_OK) == -1) {
        HIAI_ENGINE_LOG(HIAI_VDEC_CTL_ERROR, "output directory authority is not correct, use 700
instead.");
        return;
    }
    IDVPPAPI *pidvppapi = nullptr;
    int32_t ret = CreateVdecApi(pidvppapi, 0);
    if (ret != 0) {
        HIAI_ENGINE_LOG(HIAI_CREATE_DVPP_ERROR, "create dvpp api fail.");
        return;
    }
    if (pidvppapi != nullptr) {
        FILE* fp = fopen(g_inFileName, "rb");
        if (fp == nullptr) {
            HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "open file: %s failed.", g_inFileName);
            DestroyVdecApi(pidvppapi, 0);
            return;
        }
        fseek(fp, 0L, SEEK_END);
        int file_size = ftell(fp);
        fseek(fp, 0L, SEEK_SET);
        int rest_len = file_size;
        int len = file_size;

        vdec_in_msg vdec_msg;
        vdec_msg.call_back = FrameReturn;
        vdec_msg.err_report = ErrReport; // can be unassigned, if user does not need error info
        vdec_msg.hiai_data = nullptr;
        int32_t safeFuncRet = 0;
        if (g_format == 0) {
            safeFuncRet = memcpy_s(vdec_msg.video_format, sizeof(vdec_msg.video_format), "h264", 4);
            if (safeFuncRet != EOK) {
                HIAI_ENGINE_LOG(HIAI_ERROR, "memcpy_s fail");
                DestroyVdecApi(pidvppapi, 0);
                fclose(fp);
                fp = nullptr;
                return;
            }
        }
        else {
            safeFuncRet = memcpy_s(vdec_msg.video_format, sizeof(vdec_msg.video_format), "h265", 4);
            if (safeFuncRet != EOK) {
                HIAI_ENGINE_LOG(HIAI_ERROR, "memcpy_s fail");
                DestroyVdecApi(pidvppapi, 0);
                fclose(fp);
                fp = nullptr;
                return;
            }
        }
        vdec_msg.in_buffer = (char*)malloc(len);
    }
}
```

```
if (vdec_msg.in_buffer == nullptr) {
    HIAI_ENGINE_LOG(HIAI_ERROR, "malloc fail");
    fclose(fp);
    fp = nullptr;
    DestroyVdecApi(pidvppapi, 0);
    return;
}

dvppapi_ctl_msg dvppApiCtlMsg;
dvppApiCtlMsg.in = (void*)&vdec_msg;
dvppApiCtlMsg.in_size = sizeof(vdec_in_msg);

while (rest_len > 0) {
    int read_len = fread(vdec_msg.in_buffer, 1, len, fp);
    HIAI_ENGINE_LOG("rest_len is %d,read len is %d.", rest_len, read_len);
    vdec_msg.in_buffer_size = read_len;
    fclose(fp);
    fp = nullptr;
    if (VdecCtl(pidvppapi, DVPP_CTL_VDEC_PROC, &dvppApiCtlMsg, 0) != 0) {
        HIAI_ENGINE_LOG(HIAI_VDEC_CTL_ERROR, "call dvppctl process failed!");
        free(vdec_msg.in_buffer);
        vdec_msg.in_buffer = nullptr;
        DestroyVdecApi(pidvppapi, 0);
        return;
    }
    HIAI_ENGINE_LOG(HIAI_OK, "call vdec process success!");
    rest_len = rest_len - len;
}
free(vdec_msg.in_buffer);
vdec_msg.in_buffer = nullptr;
} else {
    HIAI_ENGINE_LOG(HIAI_VDEC_CTL_ERROR, "pidvppapi is null!");
}
DestroyVdecApi(pidvppapi, 0);
return;
}
```

8.3.6.6 Implementing the VENC Function

To encode multiple images into a video, call [RunVenc](#) to encode the video by using the same instance after creating an instance by calling [CreateVenc](#), and then call [DestroyVenc](#) to release the instance.

In this example, the VENC API is called to encode YUV images into H.265 or H.264 streams.

```
std::string vencOutFileName("venc.bin");
std::shared_ptr<FILE> vencOutFile(nullptr);
void VencCallBackDumpFile(struct VencOutMsg* vencOutMsg, void* userData)
{
    if (vencOutFile.get() == nullptr) {
        HIAI_ENGINE_LOG(HIAI_VENC_CTL_ERROR, "get venc out file fail!");
        return;
    }
    fwrite(vencOutMsg->outputData, 1, vencOutMsg->outputDataSize, vencOutFile.get());
    fflush(vencOutFile.get());
}

/*
 * venc new interface to achieve venc basic functions.
 */
void TEST_VENC()
{
    std::shared_ptr<FILE> fpIn(fopen(g_inFileName, "rb"), fclose);
    vencOutFile.reset(fopen(vencOutFileName.c_str(), "wb"), fclose);

    if (fpIn.get() == nullptr || vencOutFile.get() == nullptr) {
        HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "open open venc in/out file failed.");
    }
}
```

```
    return;
}

fseek(fpIn.get(), 0, SEEK_END);
uint32_t fileLen = ftell(fpIn.get());
fseek(fpIn.get(), 0, SEEK_SET);

struct VencConfig vencConfig;
vencConfig.width = g_width;
vencConfig.height = g_high;
vencConfig.codingType = g_format;
vencConfig.yuvStoreType = g_yuvStoreType;
vencConfig.keyFrameInterval = 16;
vencConfig.vencOutMsgCallBack = VencCallBackDumpFile;
vencConfig.userData = nullptr;

int32_t vencHandle = CreateVenc(&vencConfig);
if (vencHandle == -1) {
    HIAI_ENGINE_LOG(HIAI_VENC_CTL_ERROR, "CreateVenc fail!");
    return;
}

// input 16 frames once
uint32_t inDataLenMaxOnce = g_width * g_high * 3 / 2;
std::shared_ptr<char> inBuffer(static_cast<char*>(malloc(inDataLenMaxOnce)), free);

if (inBuffer.get() == nullptr) {
    HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "alloc input buffer failed");
    DestroyVenc(vencHandle);
    return;
}

uint32_t inDataUnhandledLen = fileLen;

auto start = std::chrono::system_clock::now();
uint32_t frameCount = 0;

while (inDataUnhandledLen > 0) {
    uint32_t inDataLen = inDataUnhandledLen;
    if (inDataUnhandledLen > inDataLenMaxOnce) {
        inDataLen = inDataLenMaxOnce;
    }
    inDataUnhandledLen -= inDataLen;
    uint32_t readLen = fread(inBuffer.get(), 1, inDataLen, fpIn.get());
    if (readLen != inDataLen) {
        HIAI_ENGINE_LOG(HIAI_OPEN_FILE_ERROR, "error in read input file");
        DestroyVenc(vencHandle);
        return;
    }

    struct VencInMsg vencInMsg;
    vencInMsg.inputData = inBuffer.get();
    vencInMsg.inputDataSize = inDataLen;
    vencInMsg.keyFrameInterval = 16;
    vencInMsg.forcelFrame = 0;
    vencInMsg.eos = 0;

    if (RunVenc(vencHandle, &vencInMsg) == -1) {
        HIAI_ENGINE_LOG(HIAI_VENC_CTL_ERROR, "call video encode fail");
        break;
    }
    ++frameCount;
}

auto end = std::chrono::system_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
size_t timeCount = static_cast<size_t>(duration.count());
HIAI_ENGINE_LOG("Total frame count: %u", frameCount);
HIAI_ENGINE_LOG("Time cost: %lu us", timeCount);
```

```
HIAI_ENGINE_LOG("FPS: %lf", frameCount * 1.0 / (timeCount / 1000000.0));  
  
DestroyVenc(vencHandle);  
return;  
}
```

8.3.7 Data Types

8.3.7.1 Structures in VpcUserImageConfigure

The structures are defined in the **ddk/include/inc/dvpp/Vpc.h** file in the DDK installation directory.

- **VpcUserRoiConfigure** structure

Member Variable	Description
VpcUserRoiInputConfigure inputConfigure	User ROI input configuration. For details, see VpcUserRoiInputConfigure structure . If the 8K image scaling function is implemented, this parameter does not need to be set.
VpcUserRoiOutputConfigure outputConfigure	User ROI output configuration. For details, see VpcUserRoiOutputConfigure structure .
VpcUserRoiConfigure* next	Next user ROI configuration. If the single-image multi-ROI mode is used, configure this parameter. Otherwise, set this parameter to NULL . The default value is NULL .
uint64_t reserve1	Reserved

- **VpcCompressDataConfigure** structure

Member Variable	Description
uint64_t lumaHeadAddr	Header address of the Y component
uint64_t chromaHeadAddr	Header address of the U and V components
uint32_t lumaHeadStride	Header stride of the Y component, which must be the same as the value of stride_head in the FRAME structure .
uint32_t chromaHeadStride	Header stride of the U and V components, which must be the same as the value of stride_head in the FRAME structure .
uint64_t lumaPayloadAddr	Payload address of the Y component

Member Variable	Description
uint64_t chromaPayloadAddr	Payload address of the U and V components
uint32_t lumaPayloadStride	Payload stride of the Y component, which must be the same as the value of stride_payload in the FRAME structure .
uint32_t chromaPayloadStride	Payload stride of the U and V components, which must be the same as the value of stride_payload in the FRAME structure .

- **VpcUserYuvSum** structure

Member Variable	Description
uint32_t ySum	Y component sum
uint32_t uSum	U component sum
uint32_t vSum	V component sum
uint64_t reserve1	Reserved

- **VpcUserPerformanceTuningParameter** structure

Member Variable	Description
uint64_t reserve1	Reserved parameter 1
uint64_t reserve2	Reserved parameter 2
uint64_t reserve3	Reserved parameter 3
uint64_t reserve4	Reserved parameter 4
uint64_t reserve5	Reserved parameter 5

- **VpcUserRoiInputConfigure** structure

Member Variable	Description
VpcUserCropConfigure cropArea	Input data configuration of the cropped area. For details, see VpcUserCropConfigure structure .
uint64_t reserve1	Reserved

- **VpcUserRoiOutputConfigure** structure

Member Variable	Description
uint8_t* addr	Start address of the output image You are advised to allocate the buffer by calling HIAI_DVPP_DMAlloc provided by Matrix. The allocated buffer must meet the DVPP requirements, that is, the buffer addresses of the input and output images must be in the same 4 GB space, and the start addresses must be 16-pixel aligned. For details about HIAI_DVPP_DMAlloc , see <i>Matrix API Reference</i> .
uint32_t bufferSize	Output buffer size calculated based on the YUV420SP format
uint32_t widthStride	Width stride of the output image, which must be 16-pixel aligned. The minimum width stride is 32, and the maximum width stride is 4096.
uint32_t heightStride	Height stride of the output image, which must be 2-pixel aligned. The minimum height stride is 6, and the maximum height stride is 4096. If the output is a YUV420SP image, you need to calculate the start addresses of the U and V data based on the value of heightStride .
VpcUserCropConfigure outputArea	Coordinates of the output area specified by the user. For details, see VpcUserCropConfigure structure . If the 8K image scaling function is implemented, this parameter does not need to be set.
uint64_t reserve1	Reserved

- **VpcUserCropConfigure** structure

For details about the top offset, bottom offset, left offset, and right offset, see [Table 8-8](#).

Member Variable	Description
uint32_t leftOffset	Left offset. The value must be an even number. The left offset of the overwritten area relative to the output image is 16-pixel aligned.
uint32_t rightOffset	Right offset. The value must be an odd number.
uint32_t upOffset	Top offset. The value must be an even number.
uint32_t downOffset	Bottom offset. The value must be an odd number.
uint64_t reserve1	Reserved

8.3.7.2 Structures and Classes in vdec_in_msg

The structures are defined in the **ddk/include/inc/dvpp/Vdec.h** file in the DDK installation directory.

- **FRAME** structure

Member Variable	Description
int height	Height of the output image after alignment. For the H.264 format, the value is 16-pixel aligned. For the H.265 format, the value is 64-pixel aligned.
int width	Width of the output image after alignment. For the H.264 format, the value is 16-pixel aligned. For the H.265 format, the value is 64-pixel aligned.
int realHeight	Actual image height
int realWidth	Actual image width
unsigned char* buffer	Buffer address of the output image
int buffer_size	Buffer size of the output image
unsigned int offset_payload_y	Y component offset of the output image payload. Y component address of the payload = Buffer + offset_payload_y
unsigned int offset_payload_c	C component offset of the output image payload. C component address of the payload = Buffer + offset_payload_c

Member Variable	Description
unsigned int offset_head_y	Y component offset of the output image header. Y component address of the header = Buffer + offset_head_y
unsigned int offset_head_c	C component offset of the output image header. C component address of the header = Buffer + offset_head_c
unsigned int stride_payload	Payload stride of the output image
unsigned int stride_head	Header stride of the output image
unsigned short bitdepth	Bit depth of the output image
char video_format[10]	Video format. The format can be h264 or h265 .
char image_format[10]	Output image format. The format can be nv12 or nv21 .

- **HIAI_DATA_SP** class

Member Variable or Function	Description
unsigned long long frameIndex	Frame sequence number
void * frameBuffer	Buffer for storing output frames
unsigned int frameSize	Size of the buffer for storing output frames
void setFrameIndex(unsigned long long index)	Function for configuring the frame sequence number
unsigned long long getFrameIndex()	Function for obtaining the frame sequence number
void setFrameBuffer(void * frameBuff)	Function for setting the frame buffer address
void * getFrameBuffer()	Function for obtaining the frame buffer address
void setFrameSize(unsigned int size)	Function for setting the frame size
unsigned int getFrameSize()	Function for obtaining the frame size

- **VDECERR** structure

Member Variable	Description
ERRTYPE errType	Error type <pre>enum ERRTYPE{ // An error occurs when the VDEC state is abnormal. You need to destroy the VDEC instance and then create a new one. ERR_INVALID_STATE = 0x10001, // A hardware error occurs. For example, the VDEC is started, executed, or stopped abnormally. You need to destroy the VDEC instance and then create a new one. ERR_HARDWARE, // An error occurs when the video stream is divided into multiple frames of images. You need to check whether the input video stream data is correct. ERR_SCD_CUT_FAIL, // An error occurs when a frame is decoded. You need to check whether the input video stream data is correct. ERR_VDM_DECODE_FAIL, // Reserved ERR_ALLOC_MEM_FAIL, // An error occurs when the input video resolution exceeds the range or the internal dynamic memory allocation fails. You need to check the resolution of the input video or available memory of the system as required. ERR_ALLOC_DYNAMIC_MEM_FAIL, // An error occurs when the input and output buffers of the VDEC are allocated. You need to check whether the system has available memory. ERR_ALLOC_IN_OR_OUT_PORT_MEM_FAIL, // A stream error occurs. This structure is reserved. ERR_BITSTREAM, // A video format error occurs. This structure is reserved. ERR_VIDEO_FORMAT, // An error occurs in the output format configuration. This structure is reserved. ERR_IMAGE_FORMAT, // A null error occurs in the callback function. This structure is reserved. ERR_CALLBACK, // A null error occurs in the input buffer. This structure is reserved.</pre>

Member Variable	Description
	ERR_INPUT_BUFFER, // An error occurs when the size of the input buffer is less than or equal to 0. This structure is reserved. ERR_INBUF_SIZE, // A thread error occurs when the system returns the decoding result through the callback function. You need to check whether the resources (such as threads and memory) in the system are available. ERR_THREAD_CREATE_FBD_FAIL, // An error occurs when the VDEC instance fails to be created. You need to create a VDEC instance again. ERR_CREATE_INSTANCE_FAIL, // An error occurs when the VDEC fails to be initialized. For example, the number of VDEC instances exceeds 16, which is the upper limit. In this case, you need to release some VDEC instances and create new ones. ERR_INIT_DECODER_FAIL, // An error occurs when the VDEC handle to a video stream fails to be obtained. You need to create a VDEC instance again. ERR_GET_CHANNEL_HANDLE_FAIL, // An error occurs when a VENC instance is set abnormally in the system. You need to check whether the input parameter values of the VENC are correct, such as the input video format (video_format) and output frame format (image_format). ERR_COMPONENT_SET_FAIL, // An error occurs when a VENC instance name is set abnormally in the system. You need to check whether the input parameter values of the VENC are correct, such as the input video format (video_format) and output frame format (image_format). ERR_COMPARE_NAME_FAIL, // Other errors occur. ERR_OTHER };
unsigned short channelId	Channel for a decoding error

8.3.7.3 Structures in IMAGE_CONFIG

- ROI_CONFIG structure

Member Variable	Description	Value Range
CROP_CONFIG crop_config	Structure for cropping parameter configuration	For details, see CROP_CONFIG structure .
YUV_SUM_OUT_CONFIG sum_out	Structure 1 of output parameter configuration	For details, see YUV_SUM_OUT_CONFIG structure .
NORMAL_OUT_CONFIG normal_out	Structure 2 of output parameter configuration, which is reserved and not supported currently	For details, see NORMAL_OUT_CONFIG structure .
ROI_CONFIG* next	Pointer to the next ROI_CONFIG structure	Set this parameter when multiple ROIs are used. Otherwise, set this parameter to NULL .

- **CROP_CONFIG** structure

Member Variable	Description	Value Range
int enable	Whether to perform cropping	0 : disabled 1 : enabled
unsigned int hmin	Minimum horizontal offset	The value is an even number and less than hmax .
unsigned int hmax	Maximum horizontal offset	The value is an odd number and less than the input width (in_width).
unsigned int vmin	Minimum vertical offset	The value is an even number and less than vmax .
unsigned int vmax	Maximum vertical offset	The value is an odd number and less than the input height (in_height).

- **YUV_SUM_OUT_CONFIG** structure

Member Variable	Description	Value Range
int enable	Whether to enable the channel for output	0: disabled 1: enabled
unsigned int out_width	Output image width	The value is an even number. The value range is [128, 4096] and the resizing ratio range is [0.03125, 4].
unsigned int out_height	Output image height	The value is an even number. The value range is [16, 4096] and the resizing ratio range is [0.03125, 4].
char* out_buffer	Pointer to the address of the output image buffer	The output buffer size must be calculated based on the output image resolution after the width is 128-pixel aligned and the height is 16-pixel aligned.
unsigned int yuv_sum	Sum of all YUV values of the output image	Reserved (not supported yet)

- **NORMAL_OUT_CONFIG** structure

Member Variable	Description	Value Range
int enable	Whether to enable the channel for output	0: disabled 1: enabled
unsigned int out_width	Output image width	The value is an even number. The value range is [128, 4096] and the resizing ratio range is [0.03125, 4].
unsigned int out_height	Output image height	The value is an even number. The value range is [16, 4096] and the resizing ratio range is [0.03125, 4].
char* out_buffer	Pointer to the address of the output image buffer	The output buffer size must be calculated based on the output image resolution after the width is 128-pixel aligned and the height is 16-pixel aligned.

8.3.7.4 Structures in dvpp_engine_capability_stru

The structures are defined in the **ddk/include/inc/dvpp/DvppCapability.h** file in the DDK installation directory.

- **dvpp_resolution_stru** structure

Member Variable	Description	Value Range
uint32_t resolution_high;	Height resolution	Maximum value: VDEC: 4096 JPEGD: 8192 PNGD: 4096 JPEGE: 8192 VPC: 4096 VENC: 1920 Minimum value: VDEC: 128 JPEGD: 32 PNGD: 32 JPEGE: 32 VPC: 16 VENC: 128
uint32_t resolution_width;	Width resolution	Maximum value: VDEC: 4096 JPEGD: 8192 PNGD: 4096 JPEGE: 8192 VPC: 4096 VENC: 1920 Minimum value: VDEC: 128 JPEGD: 32 PNGD: 32 JPEGE: 32 VPC: 16 VENC: 128

- **dvpp_format_unit_stru** structure

Member Variable	Description	Value Range
enum dvpp_color_format color_format;	Supported image format	<i>enum</i> dvpp_color_format { //YUV444 in different ordering of YUV Semi-Planar/Packed, 8-Bit, and Linear. DVPP_COLOR_YUV444_YUV_P _8BIT_LIN, DVPP_COLOR_YUV444_YVU_P _8BIT_LIN, DVPP_COLOR_YUV444_UYV_P _8BIT_LIN, DVPP_COLOR_YUV444_UVY_P _8BIT_LIN, DVPP_COLOR_YUV444_VYU_P _8BIT_LIN, DVPP_COLOR_YUV444_VUY_P _8BIT_LIN, DVPP_COLOR_YUV444_UV_SP _8BIT_LIN, DVPP_COLOR_YUV444_VU_SP _8BIT_LIN, /*422*/ DVPP_COLOR_YUYV422_YUYV _P_8BIT_LIN, DVPP_COLOR_YUYV422_YVYU _P_8BIT_LIN, DVPP_COLOR_YUYV422_UYVY _P_8BIT_LIN, DVPP_COLOR_YUYV422_VYUY _P_8BIT_LIN, DVPP_COLOR_YUV422_UV_SP _8BIT_LIN, DVPP_COLOR_YUV422_VU_SP _8BIT_LIN, /*420*/ DVPP_COLOR_YUV420_SP_8BI T_LIN, DVPP_COLOR_YVU420_SP_8BI T_LIN, DVPP_COLOR_YUV420_SP_8BI T_HFBC, DVPP_COLOR_YVU420_SP_8BI T_HFBC, DVPP_COLOR_YUV420_SP_10 BIT_HFBC, DVPP_COLOR_YVU420_SP_10 BIT_HFBC, DVPP_COLOR_YUV420_P_8BIT _LIN,

Member Variable	Description	Value Range
		/*400*/ DVPP_COLOR_YVU400_SP_8BIT, /*rgb888*/ DVPP_COLOR_RGB888_RGB_P_8BIT_LIN, DVPP_COLOR_RGB888_RBG_P_8BIT_LIN, DVPP_COLOR_RGB888_GBR_P_8BIT_LIN, DVPP_COLOR_RGB888_GRB_P_8BIT_LIN, DVPP_COLOR_RGB888_BRG_P_8BIT_LIN, DVPP_COLOR_RGB888_BGR_P_8BIT_LIN, /*argb8888*/ DVPP_COLOR_ARGB8888_ARGB_P_8BIT_LIN, DVPP_COLOR_ARGB8888_ARGB_P_8BIT_LIN, DVPP_COLOR_ARGB8888_AGBR_P_8BIT_LIN, DVPP_COLOR_ARGB8888_AGRB_P_8BIT_LIN, DVPP_COLOR_ARGB8888_ABRG_P_8BIT_LIN, DVPP_COLOR_ARGB8888_ABGR_P_8BIT_LIN, DVPP_COLOR_ARGB8888_RAGB_P_8BIT_LIN, DVPP_COLOR_ARGB8888_RABG_P_8BIT_LIN, DVPP_COLOR_ARGB8888_RGBA_P_8BIT_LIN, DVPP_COLOR_ARGB8888_RGAB_P_8BIT_LIN, DVPP_COLOR_ARGB8888_RBAG_P_8BIT_LIN, DVPP_COLOR_ARGB8888_BRGA_P_8BIT_LIN, DVPP_COLOR_ARGB8888_BRAG_P_8BIT_LIN, DVPP_COLOR_ARGB8888_BGAR_P_8BIT_LIN, DVPP_COLOR_ARGB8888_BGRAP_8BIT_LIN, DVPP_COLOR_ARGB8888_BAR

Member Variable	Description	Value Range
		G_P_8BIT_LIN, DVPP_COLOR_ARGB8888_BAG R_P_8BIT_LIN, DVPP_COLOR_ARGB8888_GRA G_P_8BIT_LIN, DVPP_COLOR_ARGB8888_GRB A_P_8BIT_LIN, DVPP_COLOR_ARGB8888_GAB R_P_8BIT_LIN, DVPP_COLOR_ARGB8888_GAR B_P_8BIT_LIN, DVPP_COLOR_ARGB8888_GBR A_P_8BIT_LIN, DVPP_COLOR_ARGB8888_GBA R_P_8BIT_LIN, PIC_JPEG, PIC_PNG, VIO_H265, VIO_H264 };
uint32_t compress_type;	Compression type	enum dvpp_compress_type { arithmetic_code = 0, huffman_code };
uint32_t stride_size;	Stride size	VDEC: 128 JPEGD: 128 PNGD: 128 JPEGE: 0 VPC: 128 VENC: 0
enum dvpp_high_align_type high_alignment;	Height alignment type	enum dvpp_high_align_type { pix_random = 0, two_pix_alignment = 2, four_pix_alignment = 4, eight_pix_alignment = 8, sixteen_pix_alignmnet = 16 };

Member Variable	Description	Value Range
enum dvpp_high_align_type width_alignment;	Width alignment type	enum dvpp_high_align_type { pix_random = 0, two_pix_alignment = 2, four_pix_alignment = 4, eight_pix_alignment = 8, sixteen_pix_alignment = 16 };
uint32_t out_mem_alignment;	Output buffer alignment	-

- **dvpp_performace_unit_stru** structure

Member Variable	Description	Value Range
uint32_t resolution_high;	Height resolution	VDEC: 1920 JPEGD: 1920 PNGD: 1920 JPEGE: 1920 VPC: 3840 VENC: 1920
uint32_t resolution_width;	Width resolution	VDEC: 1080 JPEGD: 1080 PNGD: 1080 JPEGE: 1080 VPC: 2160 VENC: 1080
uint32_t stream_num;	Stream size	VDEC: 16 JPEGD: 0 PNGD: 0 JPEGE: 0 VPC: 0 VENC: 1
unsigned long fps;	Frame rate	VDEC: 30 JPEGD: 256 PNGD: 24 JPEGE: 64 VPC: 90 VENC: 30

- **dvpp_pre_contraction_stru** structure

Member Variable	Description	Value Range
enum dvpp_support_type is_support;	Whether pre-contraction is supported	VPC: supported Others: not supported The options are as follows: enum dvpp_support_type { no_support =0, //no support do_support //support };
uint32_t contraction_types;	Contraction type	VPC: 3 Others: 0
uint32_t contraction_size[DVPP_PRE_CONTRACTION_TYPE_MAX];	Fixed ratio of pre-contraction	VPC: 2, 4, or 8 Others: 0
enum dvpp_support_type is_horizontal_support;	Whether horizontal pre-contraction is supported	VPC: supported Others: not supported
enum dvpp_support_type is_vertical_support;	Whether vertical pre-contraction is supported	VPC: supported Others: not supported

- **dvpp_pos_scale_stru** structure

Member Variable	Description	Value Range
enum dvpp_support_type is_support;	Whether post-scaling is supported	VPC: supported Others: not supported
uint32_t min_scale;	Minimum scaling coefficient	VPC: 1 Others: 1
uint32_t max_scale;	Maximum scaling coefficient	VPC: 4 Others: 1
enum dvpp_support_type is_horizontal_support;	Whether horizontal post-scaling is supported	VPC: supported Others: not supported
enum dvpp_support_type is_vertical_support;	Whether vertical post-scaling is supported	VPC: supported Others: not supported

• **dvpp_vpc_data_spec_stru** structure

Member Variable	Description	Value Range
uint32_t input_type;	Type of the input format	The options are as follows: 0: HFBC 1: YUV or RGB
struct dvpp_resolution_stru min_resolution;	Minimum resolution	For details, see struct dvpp_resolution_stru
struct dvpp_resolution_stru max_resolution;	Maximum resolution	For details, see struct dvpp_resolution_stru
enum dvpp_align_type high_alignment;	Height alignment type	enum dvpp_high_align_type { //1-pixel alignment pix_random = 0, //2-pixel alignment two_pix_alignment = 2, //4-pixel alignment four_pix_alignment = 4, //8-pixel alignment eight_pix_alignment = 8, //16-pixel alignment sixteen_pix_alignment = 16 };
enum dvpp_align_type width_alignment;	Width alignment type	enum dvpp_high_align_type { //1-pixel alignment pix_random = 0, //2-pixel alignment two_pix_alignment = 2, //4-pixel alignment four_pix_alignment = 4, //8-pixel alignment eight_pix_alignment = 8, //16-pixel alignment sixteen_pix_alignment = 16 };

8.3.7.5 Structures in vpc_in_msg

8.3.7.5.1 RDMACHANNEL Structure

Member Variable	Description	Value Range
Long luma_head_addr	Header address of the Y component	-
Long chroma_head_addr	Header address of the U and V components	-
unsigned int luma_head_stride	Header stride of the Y component	-
unsigned int chroma_head_stride;	Header stride of the U and V components	-
long luma_payload_addr	Address of the Y component data	-
long chroma_payload_addr	Address of the U and V component data	-
unsigned int luma_payload_stride	Header stride of the Y component data	-
unsigned int chroma_payload_stride;	Header stride of the U and V component data	-

8.3.7.5.2 VpcTurningReverse Structure

Member Variable	Description	Value Range
unsigned int reverse1	Reserved interface 1 for internal tuning	--
unsigned int reverse2	Reserved interface 2 for internal tuning	--
unsigned int reverse3	Reserved interface 3 for internal tuning	--
unsigned int reverse4	Reserved interface 4 for internal tuning	--

8.3.8 Appendix

8.3.8.1 Auxiliary Function APIs

- JpegCalBackFree class auxiliary API:

class JpegCalBackFree:

JpegCalBackFree() :go

~JpegCalBackFree()

JpegCalBackFree(JpegCalBackFree& others)

void setBuf(void* addr4Free)

void setBuf(void* addr4Free, size_t size4Free)

void operator() ()

const JpegCalBackFree& operator= (JpegCalBackFree& others)

- AutoBuffer class auxiliary API:

class AutoBuffer:

AutoBuffer()

~AutoBuffer()

void Reset()

char* allocBuffer(int size)

char* getBuffer()

int getBufferSize()

AutoBuffer(const AutoBuffer&)

const AutoBuffer& operator= (const AutoBuffer&)

- HIAI_DATA_SP class auxiliary API:

class HIAI_DATA_SP:

HIAI_DATA_SP()

virtual ~HIAI_DATA_SP()

void setFrameIndex(unsigned long long index)

unsigned long long setFrameIndex()

void setFrameBuffer(void * frameBuff)

void * setFrameBuffer()

void setFrameSize(unsigned int size)

unsigned int setFrameSize()

8.3.8.2 List of APIs that Are Not Recommended

- class IDVPPCAPABILITY:
virtual ~IDVPPCAPABILITY(void)
virtual int process(dvppapi_ctl_msg *MSG) = 0
virtual int init() = 0
virtual int start() = 0
virtual int stop() = 0
- class IJPEGDAPI:
virtual ~IJPEGDAPI(void)
virtual int process(dvppapi_ctl_msg* MSG) = 0
virtual int init() = 0
virtual int start() = 0
virtual int stop() = 0
- class IJPEGEAPI:
virtual ~IJPEGEAPI(){}
virtual int process(dvppapi_ctl_msg* MSG) = 0
virtual int init() = 0
virtual int start() = 0
virtual int stop() = 0
- class IVDECAPI:
virtual ~IVDECAPI(void)
virtual int process(dvppapi_ctl_msg* MSG) = 0
virtual int init() = 0
virtual int start() = 0
virtual int stop() = 0
- class IVENCAPAPI:
virtual ~IVENCAPAPI(void)
virtual int process(dvppapi_ctl_msg* MSG) = 0
virtual int init() = 0
virtual int start() = 0
- class IVPCAPI:

```
virtual ~IVPCAPI(void)
virtual int process(dvppapi_ctl_msg* MSG) = 0
virtual int init() = 0
virtual int start() = 0
virtual int stop() = 0
```

8.3.8.3 Sample Usage of the DVPP Executor

The DVPP executor integrates the basic functions of six DVPP modules: VPC, JPEGE, JPEGD, VENC, VDEC, and PNGD.

8.3.8.3.1 Environment Preparation

- Step 1** Copy the **sample_dvpp_hlt_ddk** executable file generated after compilation to a directory on the device side.

NOTE

The dependent .so files are as follows: **lib/device/libc_sec.so**, **libDvpp_api.so**, **libDvpp_jpeg_decoder.so**, **libDvpp_jpeg_encoder.so**, **libDvpp_png_decoder.so**, **libDvpp_vpc.so**, **libOMX_hisi_video_decoder.so**, **libOMX_hisi_video_encoder.so**, and **libslog.so**

By default, the .so files are in the **/usr/lib64** directory on the device side. You do not need to copy them separately.

- Step 2** Save the data file to the directory where the executable file is located, and then run the following command. In the command, **paramList** is the parameter described in [8.3.8.3.2 Sample Input Parameters of the DVPP Executor](#).

```
./sample_dvpp_hlt_ddk paramList
```

```
----End
```

8.3.8.3.2 Sample Input Parameters of the DVPP Executor

Parameter	Description
img_width	Input image width
img_height	Input image height

Parameter	Description
in_format	- For the VPC, the input image format is as follows: INPUT_YUV400, // 0 INPUT_YUV420_SEMI_PLANNER_UV, // 1 INPUT_YUV420_SEMI_PLANNER_VU, // 2 INPUT_YUV422_SEMI_PLANNER_UV, // 3 INPUT_YUV422_SEMI_PLANNER_VU, // 4 INPUT_YUV444_SEMI_PLANNER_UV, // 5 INPUT_YUV444_SEMI_PLANNER_VU, // 6 INPUT_YUV422_PACKED_YUYV, // 7 INPUT_YUV422_PACKED_UYVY, // 8 INPUT_YUV422_PACKED_VYUY, // 9 INPUT_YUV422_PACKED_VYUY, // 10 INPUT_YUV444_PACKED_YUV, // 11 INPUT_RGB, // 12, input image format: RGB888 INPUT_BGR, // 13, input image format: BGR888 INPUT_ARGB, // 14. The component sequence of an input image in this format during storage is similar to RGB888. A indicates transparency. INPUT_ABGR, // 15. The component sequence of an input image in this format during storage is similar to BGR888. A indicates transparency. INPUT_RGBA, // 16. The component sequence of an input image in this format during storage is similar to RGB888. A indicates transparency. INPUT_BGRA, // 17. The component sequence of an input image in this format during storage is similar to BGR888. A indicates transparency. INPUT_YUV420_SEMI_PLANNER_UV_10BIT, // 18 INPUT_YUV420_SEMI_PLANNER_VU_10BIT, // 19 - For the VDEC, the input stream format is as follows: 0: H.264 stream 1: H.265 stream - For the VENC, the output stream format is as follows: 0: H.265 Main Profile Level (Only slice streams are supported.) 1: H.264 Baseline Profile Level 2: H.264 Main Profile Level 3: H.264 High Profile Level - For the JPEGE, the input image formats are as follows: 0: JPGENC_FORMAT_UYVY 1: JPGENC_FORMAT_VYUY

Parameter	Description
	– 2: JPGENC_FORMAT_YVYU – 3: JPGENC_FORMAT_YUYV – 16: JPGENC_FORMAT_NV12 – 17: JPGENC_FORMAT_NV21
out_format	Output image format of the VPC • 0: OUTPUT_YUV420SP_UV • 1: OUTPUT_YUV420SP_VU
yuvStoreType	Storage format of the input YUV data of the VENC • 0: YUV420 semi-planar • 1: YVU420 semi-planar
inWidthStride	Width stride of the VPC input image. For details, see Input Parameter: VpcUserImageConfigure .
inHighStride	Height stride of the VPC input image. For details, see Input Parameter: VpcUserImageConfigure .
outWidthStride	Width stride of the VPC output image, which is 16-pixel aligned
outHighStride	Height stride of the VPC output image, which is 2-pixel aligned
hmin	Left offset of the cropped area specified by the VPC user. The value must be an even number.
hmax	Right offset of the cropped area specified by the VPC user. The value must be an odd number.
vmin	Top offset of the cropped area specified by the VPC user. The value must be an even number.
vmax	Bottom offset of the cropped area specified by the VPC user. The value must be an odd number.
outLeftOffset	Left offset of the output area specified by the VPC user. The value must be an even number.
outRightOffset	Right offset of the output area specified by the VPC user. The value must be an odd number.
outUpOffset	Top offset of the output area specified by the VPC user. The value must be an even number.
outDownOffset	Bottom offset of the output area specified by the VPC user. The value must be an odd number.
out_width	Width of the output image
out_high	Height of the output image

Parameter	Description
in_image_file	Path of the input image file, which includes the file name
out_image_file	Path of the output image file, which includes the file name
rank	• 0: The JPEGD outputs YUV420 semi-planar data. • 1: The JPEGD outputs original YUV format data.
transform	Transform flag when the PNGD processes images • 1: RGBA is converted to RGB. • 0: The original format is retained.
test_type	The options are as follows: • 1: VPC basic function 1 (image cropping and resizing) • 2: VPC basic function 2 (original image resizing, cropped image resizing, single-image multi-ROI cropping, and 8K image resizing) • 3: VDEC • 4: VENC • 5: JPEGE • 6: The JPEGE user specifies the output buffer. • 7: JPEGD • 8: The JPEGD user specifies the output buffer. • 9: PNGD • 10: The PNGD user specifies the output buffer. • 12: JPEGD+VPC+JPEGE

8.3.8.3.3 VPC Instructions

?1. VPC Basic Function 1

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_vpc_1920x1080_nv12.yuv --out_image_file vpc_out.yuv --
inWidthStride 1920 --inHighStride 1080 --in_format 1 --out_format 0 --hmax 1919 --hmin 0 --vmax 999 --
vmin 0 --outLeftOffset 0 --outRightOffset 719 --outUpOffset 0 --outDownOffset 719 --outWidthStride 720
--outHighStride 720 --test_type 1
```

Call the VPC to crop and resize an image.

- Input image:
 - An image whose width is 1920 pixels, height is 1080 pixels, format is YUV420SP, and file name is **dvpp_vpc_1920x1080_nv12.yuv**.
 - The input image is in the same directory as the executable file.
- Output image:
 - An image whose width is 720 pixels, height is 720 pixels, format is YUV420SP (NV12), and file name is **vpc_out.yuv**.

- The output image is in the same directory as the executable file.
- You can open the **vpc_out.yuv** image by using image display software to verify the correctness.

8.2. VPC Basic Function 2

```
./sample_dvpp_hlt_ddk --test_type 2
```

This command calls the VPC to resize the original image, resize the cropped image, perform single-image multi-ROI cropping, and resize an 8K image.

- Input images:
 - An image whose width is 1920 pixels, height is 1080 pixels, format is YUV420SP NV12, and file name is **dvpp_vpc_1920x1080_nv12.yuv**.
 - An image whose width is 8192 pixels, height is 8192 pixels, format is YUV420SP NV12, and file name is **dvpp_vpc_8192x8192_nv12.yuv**.
 - The input images are in the same directory as the executable file.
- Output images:
 - Call the **NewVpcTest1** function to resize the 1080p YUV420SP image to 1280 x 720 YUV420SP (NV12). The output file name is **NewVpcTest1Out.yuv**.
 - Call the **NewVpcTest2** function to crop a sub-image from the 1080p YUV420SP image, and attach the sub-images to a specified location in the output image. The output file name is **NewVpcTest2Out.yuv**, the output image size is 1280 x 720, and the output image format is YUV420SP.
 - Call the **NewVpcTest3** function to crop five sub-images from the 1080p YUV420SP image, and attach the sub-images to specified locations in the output image. The output file name is **NewVpcTest3_*Out.yuv**. The output image size is 1280 x 720 and the output image format is YUV420SP.
 - Call the **NewVpcTest4** function to resize the 8192 x 8192 YUV420SP image to a 4000 x 4000 YUV420SP image. The output file name is **NewVpcTest4Out.yuv**.
 - The generated images and the executable file are in the same directory.
- You can open the images by using image display software to verify the correctness.

8.3.8.3.4 VDEC Usage

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_vdec_h264_1frame_bp_51_1920x1080.h264 --out_image_file h264_image --in_format 0 --test_type 3  
./sample_dvpp_hlt_ddk --in_image_file dvpp_vdec_h265_1frame_mp_51_1920x1080.h265 --out_image_file h265_image --in_format 1 --test_type 3
```

Sample description: The preceding commands call the VDEC to decode H.264 and H.265 streams and call the VPC to generate YUV images.

- Input video:
 - The stream has the width of 1920 pixels, height of 1080 pixels, and file name of **dvpp_vdec_h264_1frame_bp_51_1920x1080.h264**.
 - The stream has the width of 1920 pixels, height of 1080 pixels, and file name of **dvpp_vdec_h265_1frame_mp_51_1920x1080.h265**.

- The video files are in the same directory as the executable file.
- Output image:
 - The output image is in YUV420SP format and has the width of 1920 pixels and height of 1080 pixels.
 - For the decoding of H.264 streams, an image named **h264_image*** is generated in the **output_dir** directory. Only one image is output because the current video contains only one frame. If the video contains multiple frames, multiple images will be output.
 - For the decoding of H.265 streams, an image named **h265_image*** is generated in the **output_dir** directory. Only one image is output because the current video contains only one frame. If the video contains multiple frames, multiple images will be output.
- You can open the **h264_image*** and **h265_image*** files by using image display software to verify the correctness.

8.3.8.3.5 Usage of the VENC

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_venc_128x128_nv12.yuv --img_width 128 --img_height 128 --in_format 0 --yuvStoreType 0 --test_type 4
```

This command calls the VENC to encode a YUV image into a H.265 stream.

- Input image:
 - An image whose width is 128 pixels, height is 128 pixels, format is YUV420SP NV12, and file name is **dvpp_venc_128x128_nv12.yuv**.
 - The input image file is in the same directory as the executable file.
 - **in_format** corresponds to **coding_type** in **Input parameter: VencConfig**.
 - **yuvStoreType** corresponds to **YUV_store_type** in **Input parameter: VencConfig**.
- Output video:
 - The output stream file is **venc.bin**.
 - The output stream file is in the same directory as the executable file.
- You can run the **ffmpeg.exe -i venc.bin venc.jpg** command to decode the **venc.bin** file by using FFmpeg, and then open the image by using image display software to check the correctness. The official FFmpeg website is <http://ffmpeg.org/>.

8.3.8.3.6 Usage of the JPEG

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_jpege_444_1920x1080_1920_1080_nv12.yuv --img_width 1920 --img_height 1080 --in_format 16 --test_type 5  
./sample_dvpp_hlt_ddk --in_image_file dvpp_jpege_444_1920x1080_1920_1080_nv12.yuv --img_width 1920 --img_height 1080 --in_format 16 --test_type 6
```

Sample description: In the first command (**test_type 5**), the DVPP allocates memory. In the second command (**test_type 6**), the user allocates memory. Both commands can be used to encode a YUV image into a JPG image.

- Input image:
 - The input image has the width of 1920 pixels, height of 1080 pixels, format of YUV420SP NV12, and file name of **dvpp_jpege_444_1920x1080_1920_1080_nv12.yuv**.

- The input image is in the same directory as the executable file.
- Output image:
 - The output image has the width of 1920 pixels, height of 1080 pixels, and file name of **outfile_tX.jpg** (*X* indicates the thread ID).
 - The output image is in the same directory as the executable file.

8.3.8.3.7 Usage of the JPEGD

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_jpegd_decode_1920x1080.jpg --test_type 7 --rank 0  
./sample_dvpp_hlt_ddk --in_image_file dvpp_jpegd_decode_1920x1080.jpg --test_type 8 --rank 0
```

In the first command (**test_type 7**), the DVPP allocates memory. In the second command (**test_type 8**), the user allocates memory. Both commands decode a JPG image into a YUV image.

- Input image:
 - An image whose width is 1920 pixels, height is 1080 pixels, and file name is **dvpp_jpegd_decode_1920x1080.jpg**.
 - The input image is in the same directory as the executable file.
 - If **--rank** is set to **0** or not set, the output data format is **NV21**. If **--rank** is not **0**, the output data is the semi-planar data of the original format.
- Output image:
 - An image whose width is 1920 pixels, height is 1080 pixels, and file name is **dvppapi_jpegd_wxx_hxx_fx_tx.yuv**.
 - The output image is in the same directory as the executable file.
- You can open the output YUV image by using image display software to verify the correctness.

8.3.8.3.8 Usage of PNGD

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_pngd_1920x1080_RGBA.png --out_image_file  
pngd_out_RGBAtoRGB_1.rgb --test_type 9 --transform 1  
./sample_dvpp_hlt_ddk --in_image_file dvpp_pngd_1920x1080_RGBA.png --out_image_file  
pngd_out_RGBAtoRGB_2.rgb --test_type 10 --transform 1
```

In the first command (**test_type 9**), the DVPP allocates memory. In the second command (**test_type 10**), the user allocates memory. Both commands decode a PNG image into an RGB888 image.

- Input picture:
 - An image whose width is 1920 pixels, height is 1080 pixels, and file name is **dvpp_pngd_1920x1080_RGBA.png**.
 - The input image is in the same directory as the executable file.
 - If **--transform** is set to **1**, the RGBA PNG image is converted to the RGB888 format.
- Output image:
 - Two 1920 x 1080 images. The file name of the first output image (**test_type 9**) is **pngd_out_RGBAtoRGB_1.rgb**, and the file name of the second output image (**test_type 10**) is **pngd_out_RGBAtoRGB_2.rgb**.
 - The output images are in the same directory as the executable file.

- You can open the **pngd_out_RGBAtoRGB_1.rgb** and **pngd_out_RGBAtoRGB_2.rgb** files by using image display software to verify the correctness.

8.3.8.3.9 JPEGD+VPC+JPEG Cascading

```
./sample_dvpp_hlt_ddk --in_image_file dvpp_jpegd_vpc_jpege.jpg --out_image_file
out_jpegd_vpc_jpege_new.jpg --hmax 1919 --hmin 0 --vmax 999 --vmin 0 --outLeftOffset 0 --
outRightOffset 719 --outUpOffset 0 --outDownOffset 719 --outWidthStride 720 --outHighStride 720 --
test_type 12
```

Sample description: This command calls the JPEGD, VPC, and JPEGG in sequence. **test_type 12** indicates that the user allocates memory. This command calls the JPEGD to decode a JPG image as a YUV420SP image, sends the image to the VPC for cropping and resizing, and calls the JPEGG to encode the VPC output to generate a JPG image.

- Input image:
 - The input image has the width of 1920 pixels, height of 1080 pixels, and file name of **dvpp_jpegd_vpc_jpege.jpg**.
 - The input image is in the same directory as the executable file.
- Output image:
 - The output image has the width of 720 pixels, height of 720 pixels, and file name of **out_jpegd_vpc_jpege_new.jpg**.
 - The output image is in the same directory as the executable file.

8.3.8.4 Exception Handling

If the caller fails to call **DvppCtl** or **DvppGetOutParameter** (the return value is **-1**), you can view the log in the **Log** window of Mind Studio. Select **DVPP** from the **Module Name** drop-down list box, and click **Search** to query the log. You can view the latest log based on the **Time** column and rectify the calling error accordingly.

For example, when the caller uses the VPC function, if the width and height of the input image are not 128 x 16 aligned, the following information is displayed in the **Content** column:

The input image width should be divided by 128, high should be divided by 16, now width 512, high 456.

NOTE

For details about how to view a log, see section 2.3.1 "Viewing a Log" in *Ascend 310 Mind Studio Auxiliary Tools*.

8.3.8.5 Acronyms

Table 8-28 Acronyms

Acronym	Full Name
VDEC	video decoder
VENC	video encoder

Acronym	Full Name
JPEGD	JPEG decoder
JPEGE	JPEG encoder
PNGD	PNG decoder
VPC	vision preprocessing core
DVPP	digital vision pre-processing
HFBC	HiSilicon frame buffer compression

8.4 GE API Reference

8.4.1 Introduction

The Graph Engine (GE) provides a collection of secure and easy-to-use APIs for image composition. These APIs can be called to build a network model, and set the graph in a model, operators in a graph, and attributes of the model and operators. This API collection consists of the following modules:

- API types
 - Operator
The Operator module represents the basic operator type and provides APIs for the operator attribute access and input/output configuration.
 - Graph
The Graph module represents a diagram constructed by connected operators. This module provides APIs for input/output configuration and graph verification.
 - Model
The Model module represents the model corresponding to a graph. This module provides the model version and APIs for the model attribute access and model graph access.
- Auxiliary (data) types
 - AttrValue
 - NamedAttr
 - Attribute instance in the **key-value** format
 - AttrHolder
 - Stores attribute values of operators.
 - Tensor
 - TensorDesc: Describes the tensor information, including:
 - **Shape**: tensor shape
 - **DataType**: tensor data type

- **Format:** tensor format

You can view the API definition files in **ddk/include/inc/graph** in the installation directory of the DDK. If you install Mind Studio and the DDK together in boot installation mode, you can log in to the Mind Studio server as the Mind Studio installation user and view the API definition files in **~/tools/che/ddk/ddk/include/inc/graph**. For details about the mapping between the definition files and the APIs, see the API description.

8.4.2 Attribute APIs

The Attribute APIs are defined in **attr_value.h** and **detail/attributes_holder.h**.

8.4.2.1 SetAttr

Function Prototype

```
graphStatus SetAttr(const string& name, const AttrValue& value);
```

Function Description

Sets the value of an attribute.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Attribute name
value	Input	const AttrValue&	Attribute value

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.2.2 GetAttr

Function Prototype

```
graphStatus GetAttr(const string& name, AttrValue& value) const;
```

Function Description

Obtains the value of an attribute.

Parameter Description

Parameter	Input/ Output	Type	Description
name	Input	const string&	Attribute name
value	Output	AttrValue&	Attribute value

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.2.3 HasAttr

Function Prototype

```
bool HasAttr(const string& name) const;
```

Function Description

Checks whether an attribute exists by name.

Parameter Description

Parameter	Input/ Output	Type	Description
name	Input	const string&	Attribute name

Return Value

Parameter	Type	Description
-	bool	true: An attribute named name is found. false: No attribute named name is found.

Exception Handling

None

Restriction

None

8.4.2.4 DelAttr

Function Prototype

```
graphStatus DelAttr(const string& name);
```

Function Description

Deletes an attribute.

Parameter Description

Parameter	Input/ Output	Type	Description
name	Input	const string&	Attribute name

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.2.5 SetName

Function Prototype

```
void SetName(const std::string& name);
```

Function Description

Sets the name of an attribute.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Attribute name

Return Value

None

Exception Handling

None

Restriction

None

8.4.2.6 GetName

Function Prototype

```
string GetName() const;
```

Function Description

Obtains the name of an attribute.

Parameter Description

None

Return Value

Parameter	Type	Description
-	string	Attribute name

Exception Handling

None

Restriction

None

8.4.2.7 GetItem

Function Prototype

```
AttrValue GetItem(const string& key) const;
```

Function Description

Obtains the attribute value by name.

Parameter Description

Parameter	Input/ Output	Type	Description
key	Input	const string&	Attribute name

Return Value

Parameter	Type	Description
-	AttrValue	Obtained attribute value.

Exception Handling

None

Restriction

None

8.4.2.8 GetValueType

Function Prototype

```
ValueType GetValueType() const;
```

Function Description

Obtains the type of an attribute value.

Parameter Description

None

Return Value

Parameter	Type	Description
-	ValueType	Attribute type

Exception Handling

None

Restriction

None

8.4.2.9 IsEmpty

Function Prototype

```
bool IsEmpty() const;
```


Function Description

Checks whether the value of an attribute is set.

Parameter Description

None

Return Value

Parameter	Type	Description
-	bool	false : The attribute value is not set. true : The attribute value is set.

Exception Handling

None

Restriction

None

8.4.2.10 Copy

Function Prototype

```
AttrValue Copy() const;
```

Function Description

Copies an attribute value and returns the attribute object.

Parameter Description

None

Return Value

Parameter	Type	Description
-	AttrValue	Attribute object

Exception Handling

None

Restriction

None

8.4.2.11 MutableTensor

Function Prototype

```
graphStatus MutableTensor(TensorPtr& tensor);
```

Function Description

Obtains the tensor reference (mutable).

Parameter Description

Parameter	Input/Output	Type	Description
tensor	Output	TensorPtr&	Tensor reference

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.2.12 MutableListTensor

Function Prototype

```
graphStatus MutableListTensor(vector<TensorPtr>& list_tensor);
```

Function Description

Obtains the tensor list reference (mutable).

Parameter Description

Parameter	Input/Output	Type	Description
list_tensor	Output	vector<TensorPtr>&	Tensor list reference

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.2.13 InitDefault

Function Prototype

```
void InitDefault();
```

Function Description

Initializes the default values.

Parameter Description

None

Return Value

None

Exception Handling

None

Restriction

None

8.4.3 Buffer APIs

The Buffer APIs are defined in **buffer.h**.

8.4.3.1 ClearBuffer

Function Prototype

```
void ClearBuffer();
```

Function Description

Clears a buffer by setting the pointer to the data buffer to null.

Parameter Description

None

Return Value

None

Exception Handling

None

Restriction

None

8.4.3.2 GetData

Function Prototype

```
std::uint8_t* GetData();  
const std::uint8_t* GetData() const;
```

Function Description

Obtains the pointer to the buffered data.

Parameter Description

None

Return Value

Parameter	Type	Description
-	uint8_t*	Pointer to the buffered data

Exception Handling

Exception	Description
Empty buffer	Returns a null pointer.

Restriction

None

8.4.3.3 GetSize

Function Prototype

```
std::size_t GetSize() const;
```

Function Description

Obtains the size of the buffered data.

Parameter Description

None

Return Value

Parameter	Type	Description
-	size_t	Size of the buffered data

Exception Handling

Exception	Description
Empty buffer	Returns data size 0 .

Restriction

None

8.4.3.4 CopyFrom

Function Prototype

```
static Buffer CopyFrom(std::uint8_t* data, std::size_t bufferSize);
```

Function Description

Copies **bufferSize** based on the address to which the pointer in data points, and saves the data address to buffer.

Parameter Description

Parameter	Input/Output	Type	Description
data	Input	uint8_t*	Pointer to the data to be copied
bufferSize	Input	size_t	Size of the data to be copied

Return Value

Parameter	Type	Description
-	Buffer	Buffer object for storing the copied data

Exception Handling

None

Restriction

None

8.4.4 Graph APIs

The Graph APIs are defined in **graph.h**.

8.4.4.1 SetInputs

Function Prototype

```
Graph& SetInputs(std::vector<Operator>& inputs);
```

Function Description

Sets the input operator in a graph.

Parameter Description

Parameter	Input/Output	Type	Description
inputs	Input	std::vector<Operator>&	Input operator in the Graph

Return Value

Parameter	Type	Description
-	Graph&	Caller itself

Exception Handling

None

Restriction

None

8.4.4.2 SetOutputs

Function Prototype

```
Graph& SetOutputs(std::vector<Operator>& outputs);
```

Function Description

Sets the output operator linked with a graph.

Parameter Description

Parameter	Input/Output	Type	Description
outputs	Input	std::vector<Operator>&	Output operator linked with the graph

Return Value

Parameter	Type	Description
-	Graph&	Caller itself

Exception Handling

None

Restriction

None

8.4.4.3 IsValid

Function Prototype

```
bool IsValid() const;
```

Function Description

Checks whether a graph object is valid.

Parameter Description

None

Return Value

Parameter	Type	Description
-	bool	• true: The constructed graph object is valid (non-null pointer). • false: The constructed graph object is invalid (null pointer).

Exception Handling

None

Restriction

None

8.4.4.4 AddOp

Function Prototype

```
graphStatus AddOp(ge::Operator& op);
```


Function Description

Adds an operator to a graph.

Parameter Description

Parameter	Input/Output	Type	Description
op	Input	ge::Operator&	Operator to be added

Return Value

Parameter	Type	Description
-	graphStatus	GRAPH_SUCCESS : success GRAPH_FAILED : failure

Exception Handling

None

Restriction

None

8.4.4.5 FindOpByName

Function Prototype

```
ge::Operator FindOpByName(const string& name) const;
```

Function Description

Returns the operator instance in a graph based on the operator name.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Operator name

Return Value

Parameter	Type	Description
-	ge::Operator	If the operator of the corresponding name is found in the graph, the operator in the graph is returned. Otherwise, an operator of the NULL type is returned.

Exception Handling

None

Restriction

None

8.4.4.6 CheckOpByName

Function Prototype

```
graphStatus CheckOpByName(const string& name) const;
```

Function Description

Checks whether an operator exists in a graph based on the operator name.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Operator name

Return Value

Parameter	Type	Description
-	graphStatus	GRAPH_SUCCESS : The operator is found. GRAPH_FAILED : The operator is not found.

Exception Handling

None

Restriction

None

8.4.4.7 GetAllOpName

Function Prototype

```
graphStatus GetAllOpName(std::vector<string>& opName) const ;
```

Function Description

Returns the names of all operators in a graph.

Parameter Description

Parameter	Input/Output	Type	Description
opName	Output	std::vector<string>&	Returns the names of all operators in a graph.

Return Value

Parameter	Type	Description
-	graphStatus	GRAPH_SUCCESS : success GRAPH_FAILED : failure

Exception Handling

None

Restriction

None

8.4.5 Model APIs

8.4.5.1 SetName

Function Prototype

```
void SetName(const string& name);
```

Function Description

Sets the name of a model.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Model name

Return Value

None

Exception Handling

None

Restriction

None

8.4.5.2 GetName

Function Prototype

```
string GetName() const;
```

Function Description

Obtains the name of a model.

Parameter Description

None

Return Value

Parameter	Type	Description
-	string	Model name

Exception Handling

None

Restriction

None

8.4.5.3 SetVersion

Function Prototype

```
void SetVersion(uint32_t version)
```

Function Description

Sets the version of a model.

Parameter Description

Parameter	Type	Description
version	uint32_t	Model version

Return Value

None

Exception Handling

None

Restriction

None

8.4.5.4 GetVersion

Function Prototype

```
uint32_t GetVersion() const;
```

Function Description

Obtains the version of an model.

Parameter Description

None

Return Value

Parameter	Type	Description
-	uint32_t	Model version

Exception Handling

None

Restriction

None

8.4.5.5 SetPlatformVersion

Function Prototype

```
void SetPlatformVersion(string version)
```

Function Description

Sets the version of a user-defined model.

Parameter Description

Parameter	Data Type	Description
version	string	Version of the user-defined model

Return Value

None

Exception Handling

None

Restriction

None

8.4.5.6 GetPlatformVersion

Function Prototype

```
std::string GetPlatformVersion() const;
```

Function Description

Obtains the version of a user-defined model. The version value is a constant.

Parameter Description

None

Return Value

Parameter	Type	Description
-	string	Model version

Exception Handling

None

Restriction

None

8.4.5.7 GetGraph

Function Prototype

Graph GetGraph() const;

Function Description

Obtains the graph object in a model.

Parameter Description

None

Return Value

Parameter	Type	Description
-	Graph	Graph object in the model

Exception Handling

None

Restriction

None

8.4.5.8 SetGraph

Function Prototype

```
void SetGraph(const Graph& graph);
```

Function Description

Sets the graph object of a model.

Parameter Description

Parameter	Input/Output	Type	Description
graph	Input	const Graph&	Graph object

Return Value

None

Exception Handling

None

Restriction

None

8.4.5.9 Save

Function Prototype

```
graphStatus Save(Buffer& buffer) const;
```

Function Description

Serializes a model object.

Parameter Description

Parameter	Input/Output	Type	Description
buffer	Input	Buffer&	Reference to the serialized output object

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.5.10 Load

Function Prototype

```
static graphStatus Load(const uint8_t* data, size_t len, Model& model);
```

Function Description

Loads serialized data and deserializes a model object.

Parameter Description

Parameter	Input/Output	Type	Description
data	Input	const uint8_t *	Pointer to serialized data
len	Input	size_t	Length of serialized data
model	Output	Model &	Deserialized model object

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.5.11 IsValid

Function Prototype

```
bool IsValid() const;
```

Function Description

Checks whether a graph object in a model is valid. A null pointer indicates an invalid graph object.

Parameter Description

None

Return Value

Parameter	Type	Description
-	bool	The value true indicates a valid object, and the value false indicates an invalid object.

Exception Handling

None

Restriction

None

8.4.6 Operator APIs

The Operator APIs are defined in **operator.h**.

8.4.6.1 Operator Constructor

Function Prototype

```
Operator();
```

Function Description

Operator constructor

Parameter Description

None

Return Value

Parameter	Type	Description
-	Operator	Operator object

Exception Handling

None

Restriction

None

8.4.6.2 GetName

Function Prototype

string GetName() const;

Function Description

Obtains the name of an operator.

Parameter Description

None

Return Value

Parameter	Type	Description
-	string	Operator name

Exception Handling

None

Restriction

None

8.4.6.3 SetInput (srcOprt Has Only One Output)

Function Prototype

```
Operator& SetInput(const string& dstName, const Operator& srcOprt);
```

Function Description

Sets the input of an operator.

Parameter Description

Parameter	Input/Output	Type	Description
dstName	Input	const string&	Input name of the operator
srcOprt	Input	const Operator&	Input operator object whose input name is dstName .

Return Value

Parameter	Type	Description
-	Operator&	Scheduler itself

Exception Handling

None

Restriction

Operator& **srcOprt** can have only one output.

8.4.6.4 SetInput (srcOprt Has Multiple Outputs)

Function Prototype

```
Operator& SetInput(const string& dstName, const Operator& srcOprt, const string  
&name);
```

Function Description

Sets the input of an operator based on the output name of **srcOprt**, when **srcOprt** has multiple outputs.

Parameter Description

Parameter	Input/Output	Type	Description
dstName	Input	const string&	Input name of the operator
srcOprt	Input	const Operator&	Input operator object whose input name is dstName .
name	Input	const string&	Output name of srcOprt

Return Value

Parameter	Type	Description
-	Operator&	Scheduler itself

Exception Handling

None

Restriction

None

8.4.6.5 GetInputDesc

Function Prototype

```
TensorDesc GetInputDesc(const string& name) const;
```

```
TensorDesc GetInputDesc(uint32_t index) const;
```

Function Description

Obtains the input **TensorDesc** of an operator based on its input name or input index.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Input name of the operator If no operator input name is available, the default object constructed by TensorDesc is returned. Set DataType to DT_FLOAT (indicating the FLOAT type) and Format to FORMAT_NCHW (indicating the NCHW format).
index	Input	uint32_t	Input index of the operator If no operator input index is available, the default object constructed by TensorDesc is returned. Set DataType to DT_FLOAT (indicating the FLOAT type) and Format to FORMAT_NCHW (indicating the NCHW format).

Return Value

Parameter	Type	Description
TensorDesc	TensorDesc	TensorDesc of the operator input

Exception Handling

None

Restriction

None

8.4.6.6 TryGetInputDesc

Function Prototype

```
bool TryGetInputDesc(const string& name, TensorDesc& tensorDesc) const;
```

Function Description

Obtains the input **TensorDesc** of an operator based on its input name.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Input Name
tensorDesc	Output	TensorDesc&	TensorDesc object

Return Value

Parameter	Type	Description
-	bool	If the TensorDesc is obtained successfully, true is returned. Otherwise, false is returned.

Exception Handling

None

Restriction

None

8.4.6.7 UpdateInputDesc

Function Prototype

```
graphStatus UpdateInputDesc(const string& name, const TensorDesc& tensorDesc);
```

Function Description

Updates the input **TensorDesc** of an operator based on its input name.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Operator Input Name

Parameter	Input/Output	Type	Description
tensorDesc	Input	const TensorDesc&	TensorDesc object

Return Value

Parameter	Type	Description
-	graphStatus	If TensorDesc is updated successfully, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.6.8 GetOutputDesc

Function Prototype

TensorDesc GetOutputDesc(const string& name) const;

TensorDesc GetOutputDesc(uint32_t index) const;

Function Description

Obtains the output **TensorDesc** of an operator based on its output name or output index.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Output name of the operator

Parameter	Input/Output	Type	Description
index	Input	uint32_t	Output index of the operator If no operator output index is available, the default object constructed by TensorDesc is returned. Set DataType to DT_FLOAT (indicating the FLOAT type) and Format to FORMAT_NCHW (indicating the NCHW format).

Return Value

Parameter	Type	Description
-	TensorDesc	If TensorDesc is obtained successfully, the required TensorDesc object is returned. Otherwise, the default TensorDesc object is returned. Set DataType to DT_FLOAT (indicating the FLOAT type) and Format to FORMAT_NCHW (indicating the NCHW format).

Exception Handling

None

Restriction

None

8.4.6.9 UpdateOutputDesc

Function Prototype

```
graphStatus UpdateOutputDesc (const string& name, const TensorDesc& tensorDesc);
```

Function Description

Updates the output **TensorDesc** of an operator based on its output name.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Output name of the operator
tensorDesc	Input	const TensorDesc&	TensorDesc object

Return Value

Parameter	Type	Description
-	graphStatus	If TensorDesc is updated successfully, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.6.10 GetDynamicInputDesc

Function Prototype

```
TensorDesc GetDynamicInputDesc(const string& name, const unsigned int index)  
const;
```

Function Description

Obtains the dynamic input **TensorDesc** of an operator based on the combination of its input name and index.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Dynamic input name of the operator

Parameter	Input/Output	Type	Description
index	Input	const unsigned int	Dynamic input index of the operator, which starts from 1

Return Value

Parameter	Type	Description
TensorDesc	TensorDesc	If TensorDesc is obtained successfully, the required TensorDesc object is returned. Otherwise, the default TensorDesc object is returned. Set DataType to DT_FLOAT (indicating the FLOAT type) and Format to FORMAT_NCHW (indicating the NCHW format).

Exception Handling

None

Restriction

None

8.4.6.11 UpdateDynamicInputDesc

Function Prototype

```
graphStatus UpdateDynamicInputDesc(const string& name, const unsigned int  
index, const TensorDesc& tensorDesc);
```

Function Description

Updates the dynamic input **TensorDesc** of an operator based on the combination of its input name and index.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Dynamic input name of the operator
index	Input	const unsigned int	Dynamic input index of the operator, which starts from 1
tensorDesc	Input	const TensorDesc&	TensorDesc object

Return Value

Parameter	Type	Description
-	graphStatus	If the dynamic input TensorDesc is updated successfully, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.6.12 GetDynamicOutputDesc

Function Prototype

TensorDesc GetDynamicOutputDesc (const string& name, const unsigned int index) const;

Function Description

Obtains the dynamic output **TensorDesc** of an operator based on the combination of its output name and index.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Dynamic output name of the operator
index	Input	const unsigned int	Dynamic output index of the operator, which starts from 1

Return Value

Parameter	Type	Description
TensorDesc	TensorDesc	If TensorDesc is obtained successfully, the required TensorDesc object is returned. Otherwise, the default TensorDesc object is returned. Set DataType to DT_FLOAT (indicating the FLOAT type) and Format to FORMAT_NCHW (indicating the NCHW format).

Exception Handling

None

Restriction

None

8.4.6.13 UpdateDynamicOutputDesc

Function Prototype

```
graphStatus UpdateDynamicOutputDesc (const string& name, const unsigned int  
index, const TensorDesc& tensorDesc);
```

Function Description

Updates the dynamic output **TensorDesc** of an operator based on the combination of its output name and index.

Parameter Description

Parameter	Input/Output	Type	Description
name	Input	const string&	Dynamic output name of the operator
index	Input	const unsigned int	Dynamic output index of the operator
tensorDesc	Input	const TensorDesc&	TensorDesc object

Return Value

Parameter	Type	Description
-	graphStatus	If the dynamic output TensorDesc is updated successfully, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.6.14 SetAttr

Function Prototype

```
Operator& SetAttr(const string& name, AttrValue&& attrValue);
```

```
Operator& SetAttr(const string& name, const UsrQuantizeFactorParams&  
attrValue);
```

Function Description

Sets the attribute value of an operator.

Parameter Description

Parameter	Input/ Output	Type	Description
name	Input	const string&	Attribute name
attrValue	Input	AttrValue&&	Attribute value to be set
attrValue	Input	const UsrQuantizeFactorParams&	Quantization factor For details about UsrQuantizeFactorParams , see Data Type Description .

Return Value

Parameter	Type	Description
-	Operator&	Object itself

Exception Handling

None

Restriction

None

Data Type Description

- Struct **UsrQuantizeFactorParams**

```
struct UsrQuantizeFactorParams
{
    uint32_t quantize_algo{0};
    uint32_t scale_type{0};
    UsrQuantizeFactor quantize_param;
    UsrQuantizeFactor dequantize_param;
    UsrQuantizeFactor requantize_param;
    UsrQuantizeCalcFactor quantizecalc_param;
};
```
- Struct **UsrQuantizeFactor**

```
struct UsrQuantizeFactor
{
    //QuantizeScaleMode scale_mode;
    uint32_t scale_mode{0};
    std::vector<uint8_t> scale_value;
    int64_t scale_offset{0};
    std::vector<uint8_t> offset_data_value;
    int64_t offset_data_offset{0};
    std::vector<uint8_t> offset_weight_value;
    int64_t offset_weight_offset{0};
};
```

```
std::vector<uint8_t> offset_pad_value;  
int64_t offset_pad_offset{0};  
};
```

- Struct **UsrQuantizeCalcFactor**

```
struct UsrQuantizeCalcFactor  
{  
    std::vector<uint8_t> offsetw;  
    int64_t offsetw_offset{0};  
    std::vector<uint8_t> offsetd;  
    int64_t offsetd_offset{0};  
    std::vector<uint8_t> scalereq;  
    int64_t scaledreq_offset{0};  
    std::vector<uint8_t> offsetdnext;  
    int64_t offsetdnext_offset{0};  
};
```

8.4.6.15 GetAttr

Function Prototype

graphStatus GetAttr(const string& name, AttrValue& attrValue) const;

graphStatus GetAttr(const string& name, UsrQuantizeFactorParams& attrValue)
const;

Function Description

Obtains the attribute value based on an attribute name.

Parameter Description

Parameter	Input/ Output	Type	Description
name	Input	const string&	Attribute name
attrValue	Input	AttrValue&	Attribute value
attrValue	Input	UsrQuantizeFactorParams&	Quantization factor For details about UsrQuantizeFactorParams , see Data Type Description .

Return Value

Parameter	Type	Description
-	graphStatus	If the attribute value is obtained successfully, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.7 Shape APIs

The Shape APIs are defined in **tensor.h**.

8.4.7.1 GetDimNum

Function Prototype

```
size_t GetDimNum() const;
```

Function Description

Obtains the dimension count of **Shape**.

Parameter Description

None

Return Value

Parameter	Type	Description
-	size_t	Shape dimension count of the tensor

Exception Handling

None

Restriction

None

8.4.7.2 GetDim

Function Prototype

```
int64_t GetDim(size_t idx) const;
```

Function Description

Obtains the length of dimension **idx** of **Shape**.

Parameter Description

Parameter	Input/Output	Type	Description
idx	Input	size_t	Dimension index, which starts from 0

Return Value

Parameter	Type	Description
-	int64_t	Length of dimension idx

Exception Handling

None

Restriction

None

8.4.7.3 GetDims

Function Prototype

```
std::vector<int64_t> GetDims() const;
```

Function Description

Obtains the vector composed of all **Shape** dimensions.

Parameter Description

None

Return Value

Parameter	Type	Description
-	std::vector<int64_t>	Vector composed of all Shape dimensions

Exception Handling

None

Restriction

None

8.4.7.4 SetDim

Function Prototype

```
graphStatus SetDim(size_t idx, int64_t value);
```

Function Description

Sets the value of dimension **idx** in **Shape**.

Parameter Description

Parameter	Input/Output	Type	Description
idx	Input	size_t	Dimension index of Shape , which starts from 0
value	Input	int64_t	Value to be set

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.7.5 GetShapeSize

Function Prototype

```
int64_t GetShapeSize() const;
```

Function Description

Obtains the product of all dimensions in **Shape** .

Parameter Description

None

Return Value

Parameter	Type	Description
-	int64_t	Product of all dimensions

Exception Handling

None

Restriction

None

8.4.8 Tensor APIs

The Tensor APIs are defined in **tensor.h**.

8.4.8.1 GetTensorDesc

Function Prototype

```
TensorDesc GetTensorDesc() const;
```

Function Description

Obtains the descriptor (**TensorDesc**) of a tensor.

Parameter Description

None

Return Value

Parameter	Type	Description
-	TensorDesc	Tensor descriptor

Exception Handling

None

Restriction

None

8.4.8.2 MutableTensorDesc

Function Prototype

```
TensorDesc& MutableTensorDesc();
```

Function Description

Obtains the descriptor of a tensor (mutable).

Parameter Description

None

Return Value

Parameter	Type	Description
-	TensorDesc&	Tensor descriptor (mutable)

Exception Handling

None

Restriction

None

8.4.8.3 SetTensorDesc

Function Prototype

```
graphStatus SetTensorDesc(const TensorDesc &tensorDesc);
```

Function Description

Sets the descriptor of a tensor.

Parameter Description

Parameter	Input/Output	Type	Description
tensorDesc	Input	const TensorDesc &	Tensor descriptor to be set

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.8.4 GetData

Function Prototype

```
const Buffer GetData() const;
```

Function Description

Obtains the data of a tensor.

Parameter Description

None

Return Value

Parameter	Type	Description
-	const Buffer	Tensor data

Exception Handling

None

Restriction

None

8.4.8.5 MutableData

Function Prototype

```
Buffer MutableData();
```

Function Description

Obtains the data of a tensor.

Parameter Description

None

Return Value

Parameter	Type	Description
-	Buffer	Tensor data

Exception Handling

None

Restriction

None

8.4.8.6 SetData

Function Prototype

```
graphStatus SetData(std::vector<uint8_t> &&data);  
graphStatus SetData(const std::vector<uint8_t> &data);  
graphStatus SetData(const Buffer &data);  
graphStatus SetData(const uint8_t *data, size_t size);
```

Function Description

Sets data to a tensor.

Parameter Description

Parameter	Input / Output	Type	Description
data	Input	std::vector<uint8_t> && const std::vector<uint8_t> & const Buffer & or const uint8_t *	Data to be set
size	Input	size_t	Data length, in bytes

Return Value

Parameter	Type	Description
-	graphStatus	If the operation is successful, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.8.7 Clone

Function Prototype

Tensor Clone() const;

Function Description

Clones a tensor.

Parameter Description

None

Return Value

Parameter	Type	Description
-	Tensor	Cloned tensor object

Exception Handling

None

Restriction

None

8.4.9 TensorDesc APIs

The TensorDesc APIs are defined in **tensor.h**.

8.4.9.1 Update

Function Prototype

```
void Update(Shape shape, Format format = FORMAT_NCHW, DataType dt = DT_FLOAT);
```

Function Description

Updates **shape**, **format**, and **datatype** of a **TensorDesc** object.

Parameter Description

Parameter	Input/Output	Type	Description
shape	Input	Shape	shape object to be updated
format	Input	Format	format object to be updated. The default value is FORMAT_NCHW . For the definition of the Format data types, see Format .
dt	Input	DataType	datatype object to be updated. The default value is DT_FLOAT . For the definition of the DataType data types, see DataType .

Return Value

None

Exception Handling

None

Restriction

None

Data Type Description

- Format

```
enum Format {  
    FORMAT_NCHW = 0,      /**< NCHW */  
    FORMAT_NHWC,          /**< NHWC */  
    FORMAT_ND,            /**< Nd Tensor */  
    FORMAT_NC1HWC0,       /**< NC1HWC0 */  
    FORMAT_FRACTAL_Z,     /**< FRACTAL_Z */  
    FORMAT_NC1C0HWPAD,  
    FORMAT_NHWC1C0,  
    FORMAT_FSR_NCHW,  
    FORMAT_FRACTAL_DECONV,  
    FORMAT_C1HWNCO,  
    FORMAT_FRACTAL_DECONV_TRANSPOSE,  
    FORMAT_FRACTAL_DECONV_SP_STRIDE_TRANS,  
    FORMAT_NC1HWC0_C04,  /**< NC1HWC0, C0 =4*/  
    FORMAT_FRACTAL_Z_C04,/**< FRACZ, C0 = 4 */  
    FORMAT_CHWN,  
    FORMAT_FRACTAL_DECONV_SP_STRIDE8_TRANS,  
    FORMAT_HWCN,  
    FORMAT_NC1KHKWHWCO, /**< KH,KW kernel h& kernel w maxpooling max output format*/  
    FORMAT_BN_WEIGHT,  
    FORMAT_FILTER_HWCK,  /* filter input tensor format */  
    FORMAT_HASHTABLE_LOOKUP_LOOKUPS=20,  
    FORMAT_HASHTABLE_LOOKUP_KEYS,  
    FORMAT_HASHTABLE_LOOKUP_VALUE,  
    FORMAT_HASHTABLE_LOOKUP_OUTPUT,  
    FORMAT_HASHTABLE_LOOKUP_HITS=24,  
    FORMAT_RESERVED  
};
```

- DataType

```
enum DataType {  
    DT_UNDEFINED = 16, // Used to indicate a DataType field has not been set.  
    DT_FLOAT = 0,      // float type  
    DT_FLOAT16 = 1,    // fp16 type  
    DT_INT8 = 2,        // int8 type  
    DT_INT16 = 6,       // int16 type  
    DT_UINT16 = 7,      // uint16 type  
    DT_UINT8 = 4,       // uint8 type  
    DT_INT32 = 3,       //  
    DT_INT64 = 9,       // int64 type  
    DT_UINT32 = 8,      // unsigned int32  
    DT_UINT64 = 10,     // unsigned int64  
    DT_BOOL = 12,       // bool type  
    DT_DOUBLE = 11,     // double type  
    DT_DUAL = 13,       /**< dual output type */  
    DT_DUAL_SUB_INT8 = 14, /**< dual output int8 type */  
    DT_DUAL_SUB_UINT8 = 15, /**< dual output uint8 type */  
};
```

8.4.9.2 GetShape

Function Prototype

Shape GetShape() const;

Function Description

Obtains **shape** described by **TensorDesc**.

Parameter Description

None

Return Value

Parameter	Type	Description
-	Shape	shape described by TensorDesc

Exception Handling

None

Restriction

shape returned is constant and is immutable.

8.4.9.3 MutableShape

Function Prototype

Shape& MutableShape();

Function Description

Obtains the **shape** reference (mutable) in **TensorDesc**.

Parameter Description

None

Return Value

Parameter	Type	Description
-	Shape &	shape reference (mutable) in TensorDesc

Exception Handling

None

Restriction

None

8.4.9.4 SetShape

Function Prototype

```
void SetShape(Shape shape);
```

Function Description

Sets **shape** described by **TensorDesc**.

Parameter Description

Parameter	Input/ Output	Type	Description
shape	Input	Shape	shape object to be set to TensorDesc

Return Value

None

Exception Handling

None

Restriction

None

8.4.9.5 GetFormat

Function Prototype

```
Format GetFormat() const;
```

Function Description

Obtains the **format** information of a tensor described by **TensorDesc**.

Parameter Description

None

Return Value

Parameter	Type	Description
-	Format	format information of the tensor described by TensorDesc

Exception Handling

None

Restriction

None

8.4.9.6 SetFormat

Function Prototype

```
void SetFormat(Format format);
```

Function Description

Sets the **format** information of a tensor described by **TensorDesc**.

Parameter Description

Parameter	Input/Output	Type	Description
format	Input	Format	format information to be set

Return Value

None

Exception Handling

None

Restriction

None

8.4.9.7 GetDataType

Function Prototype

```
DataType GetDataType() const;
```

Function Description

Obtains the data type of a tensor described by **TensorDesc**.

Parameter Description

None

Return Value

Parameter	Type	Description
-	DataType	Data type of the tensor described by TensorDesc

Exception Handling

None

Restriction

None

8.4.9.8 SetDataType

Function Prototype

```
void SetDataType(DataType dt);
```

Function Description

Sets the data type of a tensor described by **TensorDesc**

Parameter Description

Parameter	Input/Output	Type	Description
dt	Input	DataType	dt information to be set For details, see DataType .

Return Value

None

Exception Handling

None

Restriction

None

8.4.9.9 Clone

Function Prototype

```
TensorDesc Clone() const;
```

Function Description

Clones **TensorDesc**.

Parameter Description

None

Return Value

Parameter	Type	Description
-	TensorDesc	Cloned TensorDesc object

Exception Handling

None

Restriction

None

8.4.9.10 IsValid

Function Prototype

```
graphStatus IsValid();
```

Function Description

Checks whether a tensor object is valid.

Parameter Description

None

Return Value

Parameter	Type	Description
-	graphStatus	If the tensor object is valid, GRAPH_SUCCESS is returned. Otherwise, GRAPH_FAILED is returned.

Exception Handling

None

Restriction

None

8.4.10 Operator Registration APIs

Operator type registration starts with the **REG_OP** API and ends with the **OP_END** API. The input, output, and attribute information (**INPUT**, **OUTPUT**, and **ATTR**) to be registered is linked by periods (.). After an operator type is registered, a class named after the operator type is automatically generated.

Instance

REG_OP(FullConnection)

.INPUT(x, TensorType::ALL())

.INPUT(w, TensorType::ALL())

.INPUT(b, TensorType::ALL())

.OUTPUT(y, TensorType::ALL())

.ATTR(num_output, AttrValue::INT{0})

.INFER_SHAPE_AND_TYPE(FullConnectionInfer)

.ATTR_ALL_VERIFY(FullConnectionVerify)

.OP_END()

The operator registration APIs are defined in **operator_reg.h**. For details about registered operators and their header files, see [8.4.10.12 List of Built-in Operators](#).

8.4.10.1 REG_OP

Function Prototype

REG_OP(x)

Function Description

Registers an operator type. Two constructors corresponding to the operator type are automatically generated.

For example, register an operator type **Conv2D** by calling the **REG_OP(Conv2D)** API. Two **Conv2D** constructors are generated. The operator name needs to be specified for **Conv2D(const string& name)** for example, **Conv2D *Unique index***. If the operator name is left blank, that is, **Conv2D()**, the default operator name is used.

```
class Conv2D : public Operator {
    typedef Conv2D _THIS_TYPE;
public:
    explicit Conv2D(const string& name);
    explicit Conv2D();
}
```

Parameter Description

Parameter	Input/Output	Type	Description
x	Input	-	Macro parameter, operator type name to be registered

Return Value

None

Exception Handling

None

Restriction

The operator type name must be unique.

8.4.10.2 ATTR

Function Prototype

ATTR(x, default_value)

Function Description

Registers an operator attribute. The default value must be specified, so that the default value can be used if the attribute value of an operator object is not set.

After the operator attributes are successfully registered, three external APIs (for obtaining the attribute name, obtaining the attribute value, and setting the attribute value, respectively) are automatically generated.

The following describes the operator attribute APIs generated in `int64_t` and `int64_t` list scenarios:

- Register the attribute **mode** by calling **ATTR(mode, AttrValue::INT{1})**. The attribute type is **int64_t** and the default value is **1**.

After the attribute is successfully registered, the following APIs are automatically generated:

```
static const string name_attr_mode(); // Returns the attribute name, that is, mode.  
int64_t get_attr_mode() const; // Returns the value of the mode attribute.  
_THIS_TYPE& set_attr_mode(int64_t v); // Sets the value of the mode attribute. The operator object is  
returned.
```

- Register the **pad** attribute by calling **ATTR(pad, AttrValue::LIST_INT{0, 0, 0, 0})**. The attribute type is **int64_t list**. The default value is **{0,0,0,0}**.

After the attribute is successfully registered, the following APIs are automatically generated:

```
static const string name_attr_pad(); // Returns the attribute name, that is, pad.  
vector<int64_t> get_attr_pad() const; // Returns the value of the pad attribute.  
_THIS_TYPE& set_attr_pad(vector<int64_t> v); // Sets the value of the pad attribute. The operator  
object is returned.
```

Parameter Description

Parameter	Input/ Output t	Type	Description
x	Input	-	Macro parameter, attribute name of the operator

Parameter	Input/Output	Type	Description
default_value	Input	-	Value of an operator attribute. The default value varies depending on the attribute type. The following attribute types are supported: • AttrValue::INT: The attribute type is int64_t. • AttrValue::FLOAT: The attribute type is float. • AttrValue::STR: The attribute type is string. • AttrValue::BOOL: The attribute type is bool. • AttrValue::TENSOR: The attribute type is tensor. • AttrValue::LIST_INT: The attribute type is vector<int64_t> (int64_t list). • AttrValue::LIST_FLOAT: The attribute type is vector<float> (float list). • AttrValue::LIST_STR: The attribute type is vector<string> (string list). • AttrValue::LIST_BOOL: The attribute type is vector<bool> (bool list). • AttrValue::LIST_TENSOR: The attribute type is vector<Tensor> (tensor list).

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered attribute name must be unique.

8.4.10.3 REQUIRED_ATTR

Function Prototype

REQUIRED_ATTR (x, type)

Function Description

Registers an operator attribute. The default value must be specified.

After the operator attributes are successfully registered, three external APIs (for obtaining the attribute name, obtaining the attribute value, and setting the attribute value, respectively) are automatically generated.

For example, register the attribute **mode** of the `int64_t` type by calling the **REQUIRED_ATTR (mode, Int)** API. After the operator attribute is successfully registered, the following APIs are automatically generated:

```
static const string name_attr_mode(); // Returns the attribute name, that is, mode.  
OpInt get_attr_mode() const; // Returns the value of the mode attribute. OpInt indicates int64_t.  
_THIS_TYPE& set_attr_mode(const OpInt& v); // Sets the value of the mode attribute. A this object is  
returned.
```

Parameter Description

Parameter	Input/ Output	Type	Description
x	Input	-	Macro parameter, attribute name of the operator

Parameter	Input/Output	Type	Description
type	Input	-	The following attribute types are supported: • AttrValue::INT: The attribute type is int64_t. • AttrValue::FLOAT: The attribute type is float. • AttrValue::STR: The attribute type is string. • AttrValue::BOOL: The attribute type is bool. • AttrValue::TENSOR: The attribute type is tensor. • AttrValue::LIST_INT: The attribute type is vector<int64_t> (int64_t list). • AttrValue::LIST_FLOAT: The attribute type is vector<float> (float list). • AttrValue::LIST_STR: The attribute type is vector<string> (string list). • AttrValue::LIST_BOOL: The attribute type is vector<bool> (bool list). • AttrValue::LIST_TENSOR: The attribute type is vector<Tensor> (tensor list).

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered attribute name must be unique.

8.4.10.4 INPUT

Function Prototype

INPUT (x, t)

Function Description

Registers the input information of an operator.

After the operator input information is successfully registered, APIs (for obtaining the operator input name and setting the operator input description) are generated automatically.

For example, if the operator input is **x** and the data type supported by the operator input is **TensorType{DT_FLOAT}**, call the **INPUT(x, TensorType{DT_FLOAT})** API. After the operator input is successfully registered, the following APIs are automatically generated:

```
static const string name_in_x(); // Returns the input name, that is, x.  
_THIS_TYPE& set_input_x(Operator& v, const string& srcName); // Specifies the link between input x and  
output srcName of the operator object v. The operator object itself is returned.  
_THIS_TYPE& set_input_x(Operator& v); // Specifies the link between input x and output 0 of the operator  
object v. The operator object is returned.  
TensorDesc get_input_desc_x(); // Returns the description of input x.  
graphStatus update_input_desc_x(const TensorDesc& tensorDesc); // Sets the description of input x,  
including Shape, DataType, and Format. graphStatus indicates the uint32_t type. If a non-zero value is  
returned, an error occurs.
```

Parameter Description

Parameter	Input/ Output t	Type	Description
x	Input	-	Macro parameter, input name of the operator
t	Input	-	Data type supported by the operator input. One or more data types defined by TensorType are supported. Separate data types with commas (.). For example: TensorType{DT_FLOAT} TensorType({DT_FLOAT, DT_INT8}) For details about class TensorType , see Description of Class TensorType .

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered input name must be unique.

Description of Class `TensorType`

Class `TensorType` defines the data types supported by the input or output. The following APIs are provided:

- **`TensorType(DataType dt)`**: Specifies that only one data type is supported.
- **`TensorType(std::initializer_list<DataType> types)`**: Specifies that multiple data types are supported.
- **`static TensorType ALL()`**: Specifies that all data types are supported.
- **`static TensorType FLOAT()`**: Specifies that the `DT_FLOAT` and `DT_FLOAT16` data types are supported.

8.4.10.5 OPTIONAL_INPUT

Function Prototype

`OPTIONAL_INPUT(x, t)`

Function Description

Registers the optional input information of an operator.

After the optional input information is successfully registered, APIs (for obtaining the operator input name and setting the operator input description) are generated automatically.

For example, if the operator input is **b** and the data type supported by the operator input is **`TensorType{DT_FLOAT}`**, call the **`OPTIONAL_INPUT(b, TensorType{DT_FLOAT})`** API. After the operator input is successfully registered, the following APIs are automatically generated:

```
static const string name_in_b(); // Return the input name, that is, b.
_THIS_TYPE& set_input_b(Operator& v, const string& srcName); // Specifies the link between input b and
output srcName of the operator object v. The operator object itself is returned.
_THIS_TYPE& set_input_b(Operator& v); // Specifies the link between input x and output 0 of the operator
object v. The operator object is returned.
TensorDesc get_input_desc_b(); // Returns the description of input b.
graphStatus update_input_desc_b(const TensorDesc& tensorDesc); // Sets the description of input b,
including Shape, DataType, and Format.
```

Parameter Description

Parameter	Input/Output	Type	Description
x	Input	-	Macro parameter, input name of the operator
t	Input	-	Data type supported by the operator input. One or more data types defined by TensorType are supported. Separate data types with commas (.). For example: TensorType{DT_FLOAT} TensorType({DT_FLOAT, DT_INT8})

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered input name must be unique.

8.4.10.6 DYNAMIC_INPUT

Function Prototype

DYNAMIC_INPUT (x, t)

Function Description

Registers the dynamic input information of an operator.

After the dynamic input information is successfully registered, APIs (for creating dynamic input and setting the input description) are generated automatically.

For example, if the dynamic input is **d** and the data type supported by the operator input is **TensorType{DT_FLOAT}**, call the **DYNAMIC_INPUT(d, TensorType{DT_FLOAT})** API. After the dynamic input is successfully registered, the following APIs are automatically generated:

```
_THIS_TYPE& create_dynamic_input_d(unsigned int num); // Creates dynamic input d, including a number of num inputs.
TensorDesc get_dynamic_input_desc_d(unsigned int index); // Returns description indexth of dynamic input d, including Shape, DataType, and Format.
graphStatus update_dynamic_input_desc_d(unsigned int index, const TensorDesc& tensorDesc); // Updates
```


description **indexth** of dynamic input **d**.

`_THIS_TYPE& set_dynamic_input_d(unsigned int dstIndex, Operator &v);` // Specifies the link between input **dstIndex** of **d** and index **0** of the operator object **v**. The operator object itself is returned.

`_THIS_TYPE& set_dynamic_input_d(unsigned int dstIndex, Operator &v, const string &srcName);` // Specifies the link between input **dstIndex** of **d** and output **srcName** of the operator object **v**. The operator object itself is returned.

Parameter Description

Parameter	Input/Output	Type	Description
x	Input	-	Macro parameter, input name of the operator
t	Input	-	Data type supported by the operator input. One or more data types defined by TensorType are supported. Separate data types with commas (,). For example: TensorType{DT_FLOAT} TensorType({DT_FLOAT, DT_INT8})

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered input name must be unique.

8.4.10.7 OUTPUT

Function Prototype

`OUTPUT (x, t)`

Function Description

Registers the output information of an operator.

After the operator output information is successfully registered, APIs (for obtaining the operator output name and setting the operator output description) are generated automatically.

For example, if the operator output is **y** and the data type supported by the operator input is **TensorType{DT_FLOAT}**, call the **OUTPUT(y,**

TensorType{DT_FLOAT} API. After the operator output is successfully registered, the following APIs are automatically generated:

```
static const string name_out_y();// Returns the output name, that is, y.  
TensorDesc get_output_desc_y();// Returns the description of output y.  
graphStatus update_output_desc_y(const TensorDesc& tensorDesc);// Sets the description of output y,  
including Shape, DataType, and Format.
```

Parameter Description

Parameter	Input/Output	Type	Description
x	Input	-	Macro parameter, output name of the operator
t	Input	-	Data type supported by the operator output. One or more data types defined by TensorType are supported. Separate data types with commas (.). For example: TensorType{DT_FLOAT} TensorType({DT_FLOAT, DT_INT8}

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered output name must be unique.

8.4.10.8 DYNAMIC_OUTPUT

Function Prototype

DYNAMIC_OUTPUT (x, t)

Function Description

Registers the dynamic output information of an operator.

After the dynamic output information is successfully registered, APIs (for creating dynamic output and setting the output description) are generated automatically.

For example, if the dynamic output is **d** and the data type supported by the operator output is **TensorType{DT_FLOAT}**, call the **DYNAMIC_OUTPUT (d,**

TensorType{DT_FLOAT}) API. After the dynamic output is successfully registered, the following APIs are automatically generated:

```
_THIS_TYPE& create_dynamic_output_d(unsigned int num); // Creates dynamic output d, including a  
number of num inputs.  
TensorDesc get_dynamic_output_desc_d(unsigned int index); // Returns description indexth of dynamic  
output d, including Shape, DataType, and Format.  
graphStatus update_dynamic_output_desc_d(unsigned int index, const TensorDesc& tensorDesc); // Updates  
description indexth of dynamic output d.
```

Parameter Description

Parameter	Input/ Output t	Type	Description
x	Input	-	Macro parameter, output name of the operator
t	Input	-	Data type supported by the operator output. One or more data types defined by TensorType are supported. Separate data types with commas (.). For example: TensorType{DT_FLOAT} TensorType({DT_FLOAT, DT_INT8}

Return Value

None

Exception Handling

None

Restriction

For an operator, the registered output name must be unique.

8.4.10.9 INFER_SHAPE_AND_TYPE

Function Prototype

INFER_SHAPE_AND_TYPE (x)

Function Description

Registers the **Shape** and **DataType** inference function.

Parameter Description

Parameter	Input/Output	Type	Description
x	Input	-	Macro parameter, Shape and DataType inference function For example, INFER_SHAPE_AND_TYPE(FullConnectionInfer) is used to register the FullConnectionInfer function for inferring operator Shape and DataType . FullConnectionInfer is declared in DECLARE_INFERFUNC and defined in IMPLEMT_INFERFUNC . For details, see Description of Macros DECLARE_INFERFUNC and IMPLEMT_INFERFUNC .

Return Value

None

Exception Handling

None

Restriction

None

Description of Macros DECLARE_INFERFUNC and IMPLEMT_INFERFUNC

Before registering the function for **Shape** and **DataType** inference, you need to declare the function in the **DECLARE_INFERFUNC** macro and define the function in the **IMPLEMT_INFERFUNC** macro.

- Declaring the function
`DECLARE_INFERFUNC(FullConnection, FullConnectionInfer)`

The expanded implementation of the **DECLARE_INFERFUNC** macro is as follows:

```
namespace op {  
class FullConnection;  
}  
static graphStatus FullConnectionInfer(op::FullConnection& op);
```

- Defining the function
`IMPLEMT_INFERFUNC(FullConnection, FullConnectionInfer) {
// Implementation details
}`

The expanded implementation of the **IMPLEMT_INFERFUNC** macro is as follows:

```
static graphStatus FullConnectionInfer(op::FullConnection& op){  
    // Implementation details  
}
```

8.4.10.10 ATTR_ALL_VERIFY

Function Prototype

ATTR_ALL_VERIFY (x)

Function Description

Registers the operator verification function.

Parameter Description

Parameter	Input/Output	Type	Description
x	Input	-	Macro parameter, operator verification function For example, call ATTR_ALL_VERIFY(FullConnectionVerify) to register the operator verification function FullConnectionVerify . FullConnectionVerify is declared in DECLARE_VERIFIER and defined in IMPLEMT_VERIFIER . For details, see Description of Macros DECLARE_VERIFIER and IMPLEMT_VERIFIER .

Return Value

None

Exception Handling

None

Restriction

None

Description of Macros **DECLARE_VERIFIER** and **IMPLEMT_VERIFIER**

Before registering the operator verification function, you need to declare the function in the **DECLARE_VERIFIER** macro and define the function in the **IMPLEMT_VERIFIER** macro.

- Declaring the function

```
DECLARE_VERIFIER(FullConnection, FullConnectionVerify)
```

The expanded implementation of the **DECLARE_VERIFIER** macro is as follows:

```
namespace op {  
class FullConnection;  
}  
static graphStatus FullConnectionVerify(op::FullConnection op);
```

- Defining the function

```
IMPLEMT_VERIFIER(FullConnection, FullConnectionVerify) {  
// Implementation details  
}
```

The expanded implementation of the **IMPLEMT_VERIFIER** macro is as follows:

```
static graphStatus FullConnectionVerify(op::FullConnection op){  
// Implementation details  
}
```

8.4.10.11 **OP_END**

Function Prototype

OP_END ()

Function Description

Ends operator registration.

Parameter Description

None

Return Value

None

Exception Handling

None

Restriction

None

8.4.10.12 List of Built-in Operators

You can check the built-in operators in the header files for operator registration, and set the operator input, output, and attribute during operator registration.

Table 8-29 Operator list

Operator Name	Header File
Data	array_defs.h
Concat	
Flatten	
Reshape	
Split	
Const	const_defs.h
Permute	detection_defs.h
Add	math_defs.h
Mul	
Activation	nn_defs.h
BatchNorm	
Convolution	
Eltwise	
LRN	
ConvolutionDepthwise	
FullConnection	
Pooling	
Scale	
ShuffleChannel	
Softmax	

8.4.11 Model Building APIs

The API calling sequence during model building is as follows:

CreateModelBuff > BuildIRModel > ReleaseModelBuff

8.4.11.1 CreateModelBuff

Function Prototype

```
bool CreateModelBuff(ge::Model& irModel, ModelBufferData& output);
```

Function Description

Creates a model buffer.

Parameter Description

Parameter	Input/Output	Type	Description
irModel	Input	ge::Model&	Model object
output	Output	ModelBufferData&	Offline model struct object struct ModelBufferData { void* data; uint32_t length; };

Return Value

Parameter	Type	Description
-	bool	• true: The model buffer is successfully created. • false: The model buffer fails to be created.

Exception Handling

None

Restriction

None

8.4.11.2 BuildIRModel

Function Prototype

```
bool BuildIRModel(ge::Model& irModel,ModelBufferData& output);
```

Function Description

Builds an offline model, with a model object as the input and an offline model as the output.

Parameter Description

Parameter	Input/Output	Type	Description
irModel	Input	ge::Model&	Model object
output	Output	ModelBufferData&	Offline model struct object struct ModelBufferData { void* data; uint32_t length; };

Return Value

Parameter	Type	Description
-	bool	• true: The model is successfully created. • false: The model fails to be created.

Exception Handling

None

Restriction

None

8.4.11.3 ReleaseModelBuff

Function Prototype

```
void ReleaseModelBuff(ModelBufferData& output);
```

Function Description

Releases a model buffer.

Parameter Description

Parameter	Input/Output	Type	Description
output	Output	ModelBufferData&	Offline model struct object struct ModelBufferData { void* data; uint32_t length; };

Return Value

None

Exception Handling

None

Restriction

None

8.5 TE API Reference

8.5.1 Introduction

Tensor Engine, abbreviated as TE, is a development framework for custom operators based on the Tensor Virtual Machine (TVM). The TVM is an open source project of the community. It aims to further abstract the generation rules of operators by dividing the operators into operation primitives and combining the operators when necessary. According to the definition of the computation process of operators, the TVM uses the Schedule technology and the Codegen technology to generate the operators for the specified hardware.

Schedule is used to describe the computation process for implementing an operator on hardware, which requires profound hardware knowledge. To reduce the difficulty in writing operators, the writing of schedules is simplified based on the TVM. The concept of "auto schedule" is used to provide a group of TE APIs to combine the computation of operators. By combination using APIs, you can define the computation process of an operator and hand over the schedule to "auto schedule". This document describes microprocess APIs defined based on the TVM. You can use these APIs to develop operators.

8.5.2 Description

To improve the usability of custom operators and the development efficiency, some vector operators are encapsulated in a modular manner. For element-wise

operation APIs, you need to use TVM primitives to define the input tensor, call the encapsulated APIs, briefly describe the computation process of the custom operator, and then call the provided **auto schedule** and **build** APIs to compile the custom operator into a binary executable file.

TE provides a group of encapsulated APIs, covering vector calculation. The APIs include the element-wise operation APIs, reduction APIs, broadcast API, index operation APIs, concat API, convolution API, 4D/5D conversion APIs, and matrix multiplication API.

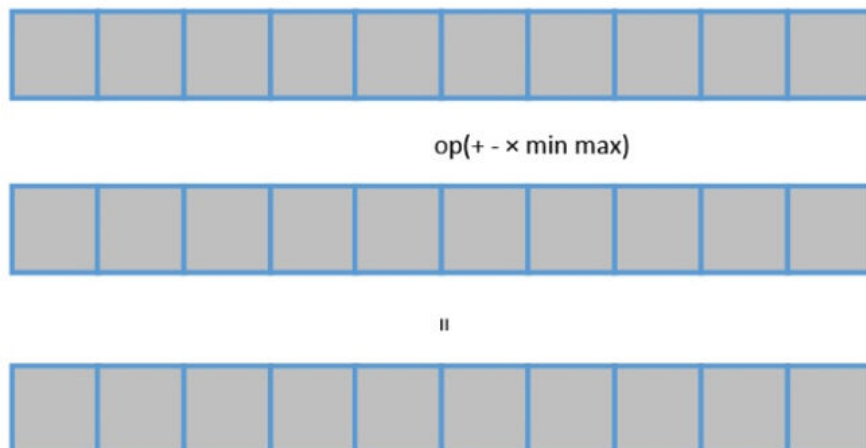
You can view the API definition files in the **ddk/site-packages/te-0.4.0.egg/te/lang/cce/te_compute**, **ddk/site-packages/te-0.4.0.egg/te/lang/cce/te_schedule**, and **ddk/site-packages/topi-0.4.0.egg/topi/generic** directories under the installation directory of the DDK package. If you install Mind Studio and the DDK both in boot installation mode, you can log in to the Mind Studio server as the Mind Studio installation user and view the API definition files under the corresponding directory in **~/tools/che/ddk/ddk/site-packages**. For details about the mapping between the definition files and the APIs, see the following API description. For details about how to use these APIs to develop custom operators and compile and run code, see *TE Custom Operator Development Guide*.

Element-wise Operation APIs

The input data is calculated using different operations by element, and the output usually has the same shape as the input.

- Dual-operand operations. Two tensors are input and calculated by element to obtain the result.
 - Add the corresponding elements of two tensors. For details, see [8.5.3.1 te.lang.cce.vadd\(lhs, rhs\)](#).
 - Subtract an element of one tensor from an element of the other tensor. For details, see [8.5.3.2 te.lang.cce.vsub\(lhs, rhs\)](#).
 - Multiply an element of one tensor and an element of the other tensor. For details, see [8.5.3.3 te.lang.cce.vmul\(lhs, rhs\)](#).
 - Compare the corresponding elements of two tensors and choose the smaller value. For details, see [8.5.3.4 te.lang.cce.vmin\(lhs, rhs\)](#).
 - Compare the corresponding elements of two tensors and choose the larger value. For details, see [8.5.3.5 te.lang.cce.vmax\(lhs, rhs\)](#).
 - Perform the bitwise OR operation on the corresponding elements of two tensors. For details, see [8.5.3.6 te.lang.cce.vor\(lhs, rhs\)](#).
 - Perform the bitwise AND operation on the corresponding elements of two tensors. For details, see [8.5.3.7 te.lang.cce.vand\(lhs, rhs\)](#).

Element-wise (dual operands)



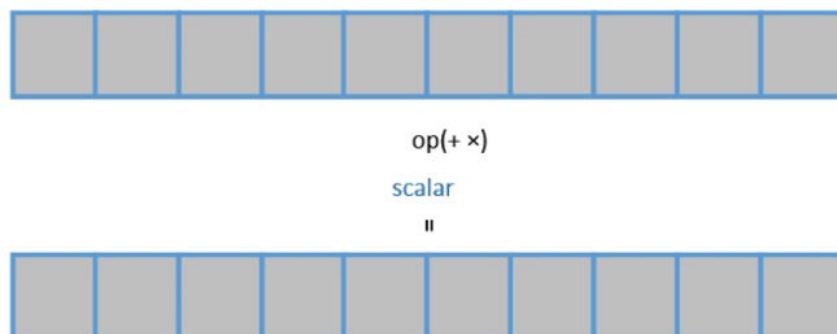
- Single-operand operations. One tensor is input and calculated by element.
 - Natural exponential. For details, see [8.5.3.11 `te.lang.cce.vexp\(raw\_tensor\)`](#).
 - Logarithm. For details, see [8.5.3.10 `te.lang.cce.vlog\(raw\_tensor\)`](#).
 - Absolute value. For details, see [8.5.3.12 `te.lang.cce.vabs\(raw\_tensor\)`](#).
 - Reciprocal. For details, see [8.5.3.13 `te.lang.cce.vrec\(raw\_tensor\)`](#).
 - Round up. For details, see [8.5.3.21 `te.lang.cce.ceil\(raw\_tensor\)`](#).
 - Round down. For details, see [8.5.3.22 `te.lang.cce.floor\(raw\_tensor\)`](#).
 - Banker's rounding. For details, see [8.5.3.23 `te.lang.cce.round\(raw\_tensor\)`](#).
 - Data type conversion. For details, see [8.5.3.14 `te.lang.cce.cast\_to\(data, dtype, f1628IntegerFlag=False\)`](#).
 - ReLU calculation. For details, see [8.5.3.15 `te.lang.cce.vrelu\(raw\_tensor\)`](#).
 - Bitwise NOT. For details, see [8.5.3.16 `te.lang.cce.vnot\(raw\_tensor\)`](#).

Element-wise (single operand)



- Tensor and scalar value operations. Each element of the input tensor is calculated with the same value.
 - Add a scalar to a tensor. For details, see [8.5.3.8 `te.lang.cce.vadds\(raw\_tensor, scalar\)`](#).
 - Multiply a tensor by a scalar. For details, see [8.5.3.9 `te.lang.cce.vmults\(raw\_tensor, scalar\)`](#).

Element-wise (tensor + scalar)



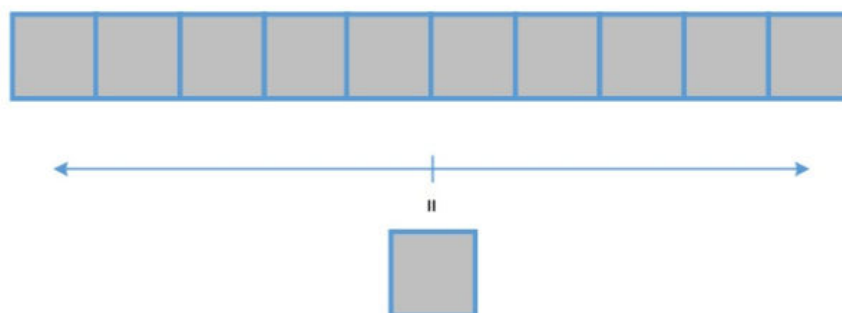
- Three-operand operations. Three tensors are input. The tensors are calculated with scalars by following the preceding calculation rules to obtain the results.
 - Add a scaled tensor to the second tensor. For details, see [8.5.3.17 `te.lang.cce.vaxpy\(lhs, rhs, scalar\)`](#).
 - Calculate three input tensors by following the rule of $x * y + z$. For details, see [8.5.3.18 `te.lang.cce.vmla\(x, y, z\)`](#).
 - Calculate three input tensors by following the rule of $x * z + y$. For details, see [8.5.3.19 `te.lang.cce.vmadd\(x, y, z\)`](#).
 - Calculate three input tensors by following the rule of $\text{relu}(x * z + y)$. For details, see [8.5.3.20 `te.lang.cce.vmaddrelu\(x, y, z\)`](#).

Reduction APIs

Compress the data of a dimension and perform operations such as accumulation and multiplication on the data in the specified direction. The operation output is one dimension less than the input data.

- Accumulate the values along an axis. For details, see [8.5.3.24 `te.lang.cce.sum\(raw\_tensor, axis, keepdims=False\)`](#).
- Calculate the minimum value along an axis. For details, see [8.5.3.25 `te.lang.cce.reduce\_min\(raw\_tensor, axis, keepdims=False\)`](#).
- Calculate the maximum value along an axis. For details, see [8.5.3.26 `te.lang.cce.reduce\_max\(raw\_tensor, axis, keepdims=False\)`](#).

reduction



NOTICE

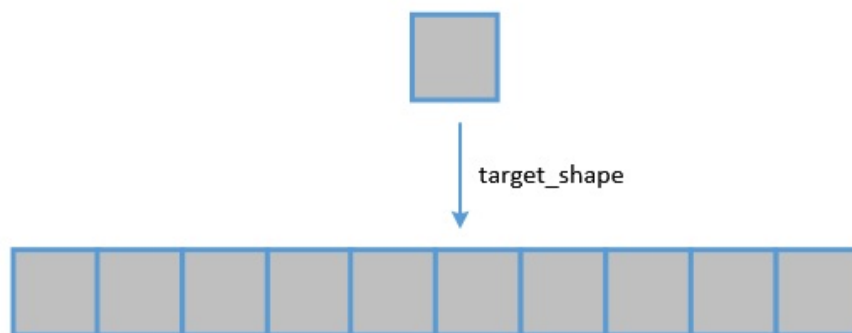
Usage limitation: Due to the data arrangement limitation on the CCE computing platform, the data after the reduction operation needs to be rearranged for subsequent operations. Therefore, when different types of APIs are used, no vector operation can be performed after the reduction operation.

Broadcast API

The broadcast operation is mainly used to process two tensors with different shapes and broadcast an operand with a lower dimension according to a higher-dimension operand. The element-wise calculation is performed after the dimensions of the two operands are the same.

Broadcast a smaller tensor to a larger tensor. For details, see [8.5.3.28 `te.lang.cce.broadcast\(var, shape, output\_dtype=None\)`](#).

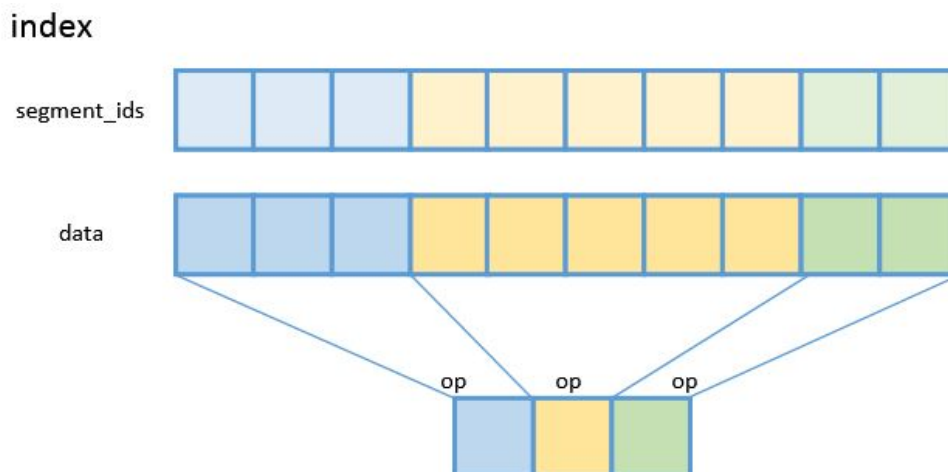
broadcast



Index Operation APIs

Index operations are used to perform calculations on tensors by segment, such as obtaining the sum value, average value, inner product, maximum value, and minimum value.

- Sum up tensors by segment. For details, see [8.5.3.29 `te.lang.cce.unsorted\_segment\_sum\(tensor, segment\_ids, num\_segments, init\_value=0\)`](#).
- Calculate the average value of tensors by segment. For details, see [8.5.3.30 `te.lang.cce.unsorted\_segment\_mean\(tensor, segment\_ids, num\_segments, init\_value=0\)`](#).
- Calculate the inner product of tensors by segment. For details, see [8.5.3.31 `te.lang.cce.unsorted\_segment\_prod\(tensor, segment\_ids, num\_segments, init\_value=0\)`](#).
- Calculate the minimum value of tensors by segment. For details, see [8.5.3.32 `te.lang.cce.unsorted\_segment\_min\(tensor, segment\_ids, num\_segments, init\_value=0\)`](#).
- Calculate the maximum value of tensors by segment. For details, see [8.5.3.33 `te.lang.cce.unsorted\_segment\_max\(tensor, segment\_ids, num\_segments, init\_value=0\)`](#).



Concat API

The concat API is used to concatenate multiple input tensors along an axis.

Concatenate the tensors along an axis. For details, see [8.5.3.34 `te.lang.cce.concat\(raw\_tensors, axis\)`](#).

Convolution API

Convolution APIs are used to implement convolutional operators.

Implement convolution. For details, see [8.5.3.35 `te.lang.cce.conv\(\*args\)`](#).

4D/5D Conversion APIs

These APIs are used for conversion between 4D NCHW and 5D NC1HWC0.

- Convert from 4D to 5D. For details, see [8.5.3.36 `te.lang.cce.compute\_four2five\(input, raw\_shape\_4D\)`](#).
- Convert from 5D to 4D. For details, see [8.5.3.37 `te.lang.cce.compute\_five2four\(input, raw\_shape\_4D\)`](#).

Matrix Multiplication API

This API is used to implement matrix multiplication.

For details, see [8.5.3.38 `te.lang.cce.matmul\(tensor\_a, tensor\_b, trans\_a=False, trans\_b=False, alpha\_num=1.0, beta\_num=0.0, tensor\_c=None\)`](#).

Convergent Operator API

The TE operator convergence function pre-processes information such as the convergence capability and input parameters of a single operator. Therefore, the specified API needs to be called to generate the *.so and *.json files of a single operator.

For details, see [8.5.5.1 `bool BuildTeCustomOp\(std::string ddkVer, std::string opName, std::string opPath, std::string opFuncName, const char \*format, ...\)`](#).

8.5.3 Compute APIs

8.5.3.1 `te.lang.cce.vadd(lhs, rhs)`

Adds two tensors by element. The data types of the elements must be the same. The supported types are float16, float32, and int32. The data types int8 and uint8 are converted to float16.

This API is defined in `elewise_compute.py`.

Parameter Description

- **lhs**: left tensor, `tvm.tensor` type
- **rhs**: right tensor, `tvm.tensor` type

Return Value

res_tensor: `lhs + rhs`, `tvm.tensor` type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vadd(data1, data2)
```

8.5.3.2 `te.lang.cce.vsub(lhs, rhs)`

Subtracts one tensor from the other tensor by element. The data types of the elements must be the same. The supported types are float16, float32, and int32. The data types int8 and uint8 are converted to float16.

This API is defined in `elewise_compute.py`.

Parameter Description

- **lhs**: left tensor, `tvm.tensor` type
- **rhs**: right tensor, `tvm.tensor` type

Return Value

res_tensor: `lhs - rhs`, `tvm.tensor` type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vsub(data1, data2)
```


8.5.3.3 `te.lang.cce.vmul(lhs, rhs)`

Multiplies two tensors by element. The data types of the elements must be the same. The supported types are float16, float32, and int32. The data types int8 and uint8 are converted to float16.

This API is defined in `elewise_compute.py`.

Parameter Description

- **lhs**: left tensor, `tvm.tensor` type
- **rhs**: right tensor, `tvm.tensor` type

Return Value

res_tensor: lhs x rhs, `tvm.tensor` type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vmul(data1, data2)
```

8.5.3.4 `te.lang.cce.vmin(lhs, rhs)`

Compares two tensors by element and chooses the smaller value. The data types of the elements must be the same. The supported types are float16, float32, and int32. The data types int8 and uint8 are converted to float16.

This API is defined in `elewise_compute.py`.

Parameter Description

- **lhs**: left tensor, `tvm.tensor` type
- **rhs**: right tensor, `tvm.tensor` type

Return Value

res_tensor: result tensor, `tvm.tensor` type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vmin(data1, data2)
```

8.5.3.5 `te.lang.cce.vmax(lhs, rhs)`

Compares two tensors by element and chooses the larger value. The data types of the elements must be the same. The supported types are float16, float32, and int32. The data types int8 and uint8 are converted to float16.

This API is defined in `elewise_compute.py`.

Parameter Description

- **lhs**: left tensor, tvm.tensor type
- **rhs**: right tensor, tvm.tensor type

Return Value

res_tensor: result tensor, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vmax(data1, data2)
```

8.5.3.6 te.lang.cce.vor(lhs, rhs)

Performs bitwise OR on two tensor elements. The data types of the elements must be the same. The supported types are int16 and uint16.

This API is defined in **elewise_compute.py**.

Parameter Description

- **lhs**: left tensor, tvm.tensor type
- **rhs**: right tensor, tvm.tensor type

Return Value

res_tensor: bitwise OR between **lhs** and **rhs**, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "int16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vor(data1, data2)
```

8.5.3.7 te.lang.cce.vand(lhs, rhs)

Performs bitwise AND on two tensor elements. The data types of the elements must be the same. The supported types are int16 and uint16.

This API is defined in **elewise_compute.py**.

Parameter Description

- **lhs**: left tensor, tvm.tensor type
- **rhs**: right tensor, tvm.tensor type

Return Value

res_tensor: bitwise AND between **lhs** and **rhs**, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "int16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
res = te.lang.cce.vadd(data1, data2)
```

8.5.3.8 te.lang.cce.vadds(raw_tensor, scalar)

Adds a scalar to each element in **raw_tensor**. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16. If the data type of a scalar is different from that of **raw_tensor**, the data type is converted into the corresponding data type.

This API is defined in **elewise_compute.py**.

Parameter Description

- **raw_tensor**: input tensor, tvm.tensor type
- **scalar**: coefficient to be added to an element in **raw_tensor**, scalar type

Return Value

res_tensor: raw_tensor + scalar, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
scalar = tvm.const(2, dtype=input_dtype)
res = te.lang.cce.vadds(data, scalar)
```

8.5.3.9 te.lang.cce.vmults(raw_tensor, scalar)

Multiplies each element in **raw_tensor** by a scalar. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16. If the data type of a scalar is different from that of **raw_tensor**, the data type is converted into the corresponding data type.

This API is defined in **elewise_compute.py**.

Parameter Description

- **raw_tensor**: input tensor, tvm.tensor type
- **scalar**: coefficient by which the element in **raw_tensor** is multiplied, scalar type

Return Value

res_tensor: raw_tensor x scalar, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
```

```
scalar = tvm.const(2, dtype=input_dtype)
res = te.lang.cce.vmults(data, scalar)
```

8.5.3.10 te.lang.cce.vlog(raw_tensor)

Performs the logarithmic $\ln(x)$ operation on each element in **raw_tensor**. The supported type is float16. The data types int8, uint8, int32, and float32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: $\ln(\text{raw_tensor})$, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.vlog(data)
```

8.5.3.11 te.lang.cce.vexp(raw_tensor)

Performs the natural exponential operation e^x on each element in a tensor. The supported type is float16. The data types int8, uint8, int32, and float32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: $e^{\text{raw_tensor}}$, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.vexp(data)
```

8.5.3.12 te.lang.cce.vabs(raw_tensor)

Performs the absolute value operation $|x|$ on each element in a tensor. The supported type is float16. The data types int8, uint8, int32, and float32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: |(raw_tensor)|, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.vabs(data)
```

8.5.3.13 te.lang.cce.vrec(raw_tensor)

Performs the reciprocal operation $1/x$ on each element in a tensor. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: $1/\text{raw_tensor}$, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.vrec(data)
```

8.5.3.14 te.lang.cce.cast_to(data, dtype, f1628IntegerFlag=False)

Converts a data type, specifically, converts a data type into **dtype**.

This API is defined in **common.py**.

The following data type conversions are supported:

Table 8-30 Supported data type conversions

Source Data Type	Destination Data Type
float32	float16
float32	int8
float32	uint8
float16	float32

Source Data Type	Destination Data Type
float16	int8
float16	uint8
float16	int32
int8	float16
int8	uint8
int32	float16
int32	int8
int32	uint8

Parameter Description

- **data**: input tensor, tvm.tensor type
- **dtype**: destination data type, string type
- **f1628IntegerFlag**: The default value is **False**. If the decimal part of the data before conversion is 0, set **f1628IntegerFlag** to **True**. If the decimal part of the data before conversion is not 0, set **f1628IntegerFlag** to **False**.

Return Value

res_tensor: data after conversion, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.cast_to(data,"float32")
```

8.5.3.15 te.lang.cce.vrelu(raw_tensor)

Performs the ReLU operation each element in a tensor. The supported type is float16. The data types int8, uint8, int32, and float32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: relu(raw_tensor), tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
```

```
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.vrelu(data)
```

8.5.3.16 te.lang.cce.vnot(raw_tensor)

Performs bitwise NOT on each element in a tensor. The supported types are int16 and uint16.

This API is defined in **elewise_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: bitwise NOT on **raw_tensor**, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "int16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.vnot(data)
```

8.5.3.17 te.lang.cce.vaxpy(lhs, rhs, scalar)

Multiplies each element in **lhs** by a scalar and adds the corresponding element in **rhs**. The data types of **lhs** and **rhs** must be the same. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16.

If the data type of the scalar is different from that of the tensor, the data type of the tensor prefers.

This API is defined in **elewise_compute.py**.

Parameter Description

- **lhs**: left tensor, tvm.tensor type
- **rhs**: right tensor, tvm.tensor type
- **scalar**: coefficient by which the element in **lhs** is multiplied, scalar type

Return Value

res_tensor: lhs x scalar + rhs, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
scalar = tvm.const(2, dtype=input_dtype)
res = te.lang.cce.vaxpy(data1, data2, scalar)
```

8.5.3.18 te.lang.cce.vmla(x, y, z)

Multiplies each element in **x** by a corresponding element in **y**, and then adds a corresponding element in **z**. The data types of the three tensors must be the same. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

- **x**: tensor, tvm.tensor type
- **y**: tensor, tvm.tensor type
- **z**: tensor, tvm.tensor type

Return Value

res_tensor: $x * y + z$, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
data3 = tvm.placeholder(shape, name="data3", dtype=input_dtype)
res = te.lang.cce.vmla(data1, data2, data3)
```

8.5.3.19 te.lang.cce.vmadd(x, y, z)

Multiplies each element in **x** by a corresponding element in **z**, and then adds a corresponding element in **y**. The data types of the three tensors must be the same. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16.

This API is defined in **elewise_compute.py**.

Parameter Description

- **x**: tensor, tvm.tensor type
- **y**: tensor, tvm.tensor type
- **z**: tensor, tvm.tensor type

Return Value

res_tensor: $x * z + y$, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
data3 = tvm.placeholder(shape, name="data3", dtype=input_dtype)
res = te.lang.cce.vmadd(data1, data2, data3)
```


8.5.3.20 `te.lang.cce.vmaddrelu(x, y, z)`

Multiplies each element in **x** by a corresponding element in **z**, adds a corresponding element in **y**, and then performs the ReLU operation. The data types of the three tensors must be the same. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16.

This API is defined in `elewise_compute.py`.

Parameter Description

- **x**: tensor, `tvm.tensor` type
- **y**: tensor, `tvm.tensor` type
- **z**: tensor, `tvm.tensor` type

Return Value

res_tensor: `relu(x * z + y)`, `tvm.tensor` type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data1 = tvm.placeholder(shape, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape, name="data2", dtype=input_dtype)
data3 = tvm.placeholder(shape, name="data3", dtype=input_dtype)
res = te.lang.cce.vmaddrelu(data1, data2, data3)
```

8.5.3.21 `te.lang.cce.ceil(raw_tensor)`

Rounds up each element in **raw_tensor**. The supported type is float16. The data type float32 is converted to float16. The output is int32.

This API is defined in `cast_compute.py`.

Parameter Description

raw_tensor: input tensor, `tvm.tensor` type

Return Value

res_tensor: `ceil(raw_tensor)`, `tvm.tensor` type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.ceil(data)
```

8.5.3.22 `te.lang.cce.floor(raw_tensor)`

Rounds down each element in **raw_tensor**. The supported type is float16. The data type float32 is converted to float16. The output is int32.

This API is defined in `cast_compute.py`.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: floor(raw_tensor), tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.floor(data)
```

8.5.3.23 te.lang.cce.round(raw_tensor)

Performs banker's rounding on each element in **raw_tensor**. When the fractional part is 0.5, the element is rounded to the nearest even number. For example, 1.5 rounds up to 2.0 and 2.5 rounds down to 2.0. The supported type is float16. The data type float32 is converted to float16. The output is int32.

This API is defined in **cast_compute.py**.

Parameter Description

raw_tensor: input tensor, tvm.tensor type

Return Value

res_tensor: round(raw_tensor), tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.round(data)
```

8.5.3.24 te.lang.cce.sum(raw_tensor, axis, keepdims=False)

Performs the sum operation based on a certain axis to reduce dimensions. The supported types are float16 and float32. The data types int8, uint8, and int32 are converted to float16.

This API is defined in **reduction_compute.py**.

Parameter Description

- **raw_tensor**: input tensor, tvm.tensor type
- **axis**: axis for the reduce operation. The value range is $[-d, d - 1]$. The parameter **d** indicates the dimension of **raw_tensor**, int or list type
- **keepdims**: The default value is **False**, indicating that the length of the operation axis is 0 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=False, the shape after reduce is (10, 10). If this parameter is set to **True**, the length of the operation axis is set to 1 after

the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=True, the shape after reduce is (10, 10, 1).

Return Value

res_tensor: tensor after the sum value is obtained, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.sum(data, axis=1)
```

8.5.3.25 te.lang.cce.reduce_min(raw_tensor, axis, keepdims=False)

Calculates the minimum value based on a certain axis to reduce dimensions. The supported type is float16. The data types int8, uint8, int32 and float32 are converted to float16.

This API is defined in **reduction_compute.py**.

Parameter Description

- **raw_tensor**: input tensor, tvm.tensor type
- **axis**: axis for the reduce operation. The value range is $[-d, d - 1]$. The parameter **d** indicates the dimension of **raw_tensor**, int or list type
- **keepdims**: The default value is **False**, indicating that the length of the operation axis is 0 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=False, the shape after reduce is (10, 10). If this parameter is set to **True**, the length of the operation axis is set to 1 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=True, the shape after reduce is (10, 10, 1).

Return Value

res_tensor: tensor after the minimum value is obtained, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.reduce_min(data, axis=1)
```

8.5.3.26 te.lang.cce.reduce_max(raw_tensor, axis, keepdims=False)

Calculates the maximum value based on a certain axis to reduce dimensions. The supported type is float16. The data types int8, uint8, int32 and float32 are converted to float16.

This API is defined in **reduction_compute.py**.

Parameter Description

- **raw_tensor**: input tensor, tvm.tensor type

- **axis**: axis for the reduce operation. The value range is $[-d, d - 1]$. The parameter **d** indicates the dimension of **raw_tensor**, int or list type
- **keepdims**: The default value is **False**, indicating that the length of the operation axis is 0 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=False, the shape after reduce is (10, 10). If this parameter is set to **True**, the length of the operation axis is set to 1 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=True, the shape after reduce is (10, 10, 1).

Return Value

res_tensor: tensor after the maximum value is obtained, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.reduce_max(data, axis=1)
```

8.5.3.27 te.lang.cce.reduce_prod(raw_tensor, axis, keepdims=False)

Performs multiplication based on a certain axis to reduce dimensions. The supported type is float16. The data types int8, uint8, int32, and float32 are converted to float16.

This API is defined in **reduction_compute.py**.

Parameter Description

- **raw_tensor**: input tensor, tvm.tensor type
- **axis**: axis for the reduce operation. The value range is $[-d, d - 1]$. The parameter **d** indicates the dimension of **raw_tensor**, int type
- **keepdims**: The default value is **False**, indicating that the axis length is 0 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=False, the shape after the reduce operation is (10, 10). If this parameter is set to **True**, the axis length is set to 1 after the reduce operation. For example, if the original shape is (10, 10, 10) and keepdims=True, the shape after the reduce operation is (10, 10, 1).

Return Value

res_tensor: tensor after multiplication is performed based on a certain axis, tvm.tensor type

Calling Example

```
shape = (1024,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.reduce_prod(data, axis=1)
```

8.5.3.28 te.lang.cce.broadcast(var, shape, output_dtype=None)

Broadcasts **var** to the tensor whose size is **shape**. The data type of the output is specified by **output_dtype**. **var** can be a scalar or tensor. The shape of **var** must

have the same length as the second parameter **shape**. The size of each dimension must be either the same as that of **shape**, or be 1. When the size is 1, the dimension is broadcast to be the same as **shape**. For example, if the dimension of **var** is (2, 1, 64) and the value of **shape** is (2, 128, 64), the calculated dimension of **var** is (2, 128, 64). The supported types are float16, float32, and int32.

This API is defined in **broadcast_compute.py**.

Parameter Description

- **var**: data to be broadcast, scalar or tensor type
- **shape**: target shape for the broadcast operation
- **output_dtype**: output data type. The default value is **var.dtype**.

Return Value

res_tensor: tensor obtained after **var** extension. The shape is specified by the parameter **shape**. The data type is **output_dtype**.

Calling Example

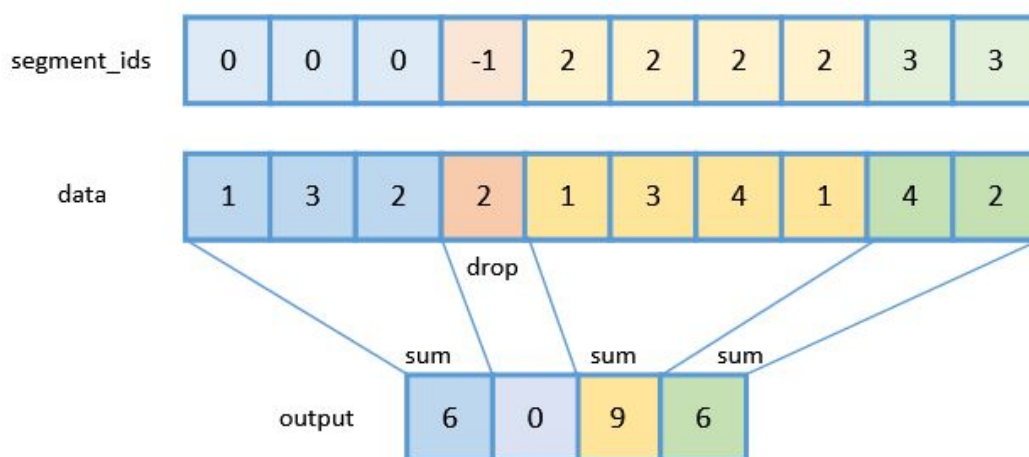
```
outshape = (1024,1024)
shape = (1024,1)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data", dtype=input_dtype)
res = te.lang.cce.broadcast(data, outshape)
```

8.5.3.29 te.lang.cce.unsorted_segment_sum(tensor, segment_ids, num_segments, init_value=0)

Uses the array **segment_ids** to sum up tensors by segment. Assuming that the input is **data** and the output is **output**, then $\text{output}[i] = \text{sum}(\text{data}[j...])$, where **j...** is an array, and element **j** meets the following requirement: $\text{segment_ids}[j] == i$.

If a subscript **i** does not appear in **segment_ids**, then $\text{output}[i] = \text{init_value}$. For example, in the following figure, if **1** does not appear in **segment_ids**, then $\text{output}[1] = 0$.

If a value in **segment_ids** is a negative number, the value of **data** in the corresponding position is discarded. For example, in the following figure, $\text{segment_ids}[3] = -1$, indicating that the value of **data[3]** is discarded and is not involved in the calculation.



The length of **segment_ids** must be the same as the length of the first dimension of **data**. The value of **num_segments** must be greater than or equal to the value of **segment_ids** plus 1.

The supported types are float16, float32, and int32.

This API is defined in **segment_compute.py**.

Parameter Description

- **tensor**: input tensor, which is of the float16, float32, or int32 type
- **segment_ids**: one-dimensional array. This array is used to segment the input tensor. The length of the array must be the same as the length of the first dimension of the input tensor. The array can be sequenced or unsequenced.
- **num_segments**: length of the first dimension of the output tensor. Its value must be greater than or equal to the value of **segment_ids** plus 1.
- **init_value**: default value of the output when a subscript in **segment_ids** does not exist. It is determined according to the implementation of the operator. The default value is **0**.

Return Value

res_tensor: tensor after calculation

Calling Example

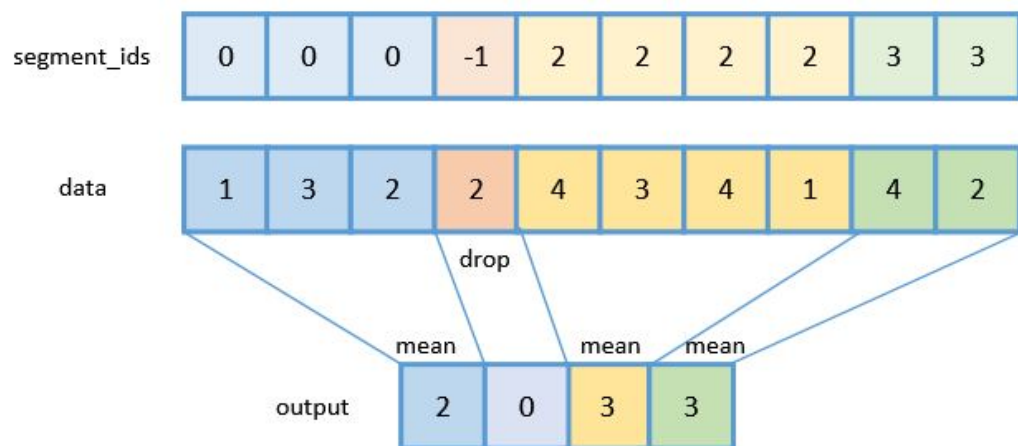
```
import tvn
import te.lang.cce
shape = (5,1024)
input_dtype = "float16"
data = tvn.placeholder(shape, name="data1", dtype=input_dtype)
segment_ids = [1,1,4,5,5]
num_segments = 6
res = te.lang.cce.unsorted_segment_sum(data, segment_ids, num_segments)
res.shape = (6,1024)
# res[0] = 0
# res[1] = data[0] + data[1]
# res[2] = 0
# res[3] = 0
# res[4] = data[2]
# res[5] = data[3] + data[4]
```

8.5.3.30 te.lang.cce.unsorted_segment_mean(tensor, segment_ids, num_segments, init_value=0)

Uses the array **segment_ids** to calculate the average value of tensors by segment. Assuming that the input is **data** and the output is **output**, then $\text{output}[i] = (1/\text{len}(j...)) \sum(\text{data}[j...])$, where **j...** is an array, and element **j** meets the following requirement: $\text{segment_ids}[j] == i$.

If a subscript **i** does not appear in **segment_ids**, then $\text{output}[i] = \text{init_value}$. For example, in the following figure, if **1** does not appear in **segment_ids**, then $\text{output}[1] = 0$.

If a value in **segment_ids** is a negative number, the value of **data** in the corresponding position is discarded. For example, in the following figure, $\text{segment_ids}[3] = -1$, indicating that the value of **data[3]** is discarded and is not involved in the calculation.



The length of **segment_ids** must be the same as the length of the first dimension of **data**. The value of **num_segments** must be greater than or equal to the value of **segment_ids** plus 1.

The supported types are float16, float32, and int32.

This API is defined in **segment_compute.py**.

Parameter Description

- **tensor**: input tensor, which is of the float16, float32, or int32 type
- **segment_ids**: one-dimensional array. This array is used to segment the input tensor. The length of the array must be the same as the length of the first dimension of the input tensor. The array can be sequenced or unsequenced.
- **num_segments**: length of the first dimension of the output tensor. Its value must be greater than or equal to the value of **segment_ids** plus 1.
- **init_value**: default value of the output when a subscript in **segment_ids** does not exist. It is determined according to the implementation of the operator. The default value is **0**.

Return Value

res_tensor: tensor after calculation

Calling Example

```
import tvn
import te.lang.cce
shape = (5,1024)
input_dtype = "float16"
data = tvn.placeholder(shape, name="data1", dtype=input_dtype)
segment_ids = [1,1,5,5,5]
num_segments = 6
res = te.lang.cce.unsorted_segment_mean(data, segment_ids, num_segments)
# res.shape = (6,1024)
# res[0] = 0
# res[1] = (data[0] + data[1]) / 2
# res[2] = 0
# res[3] = 0
# res[4] = 0
# res[5] = (data[2] + data[3] + data[4]) / 3
```

8.5.3.31 te.lang.cce.unsorted_segment_prod(tensor, segment_ids, num_segments, init_value=0)

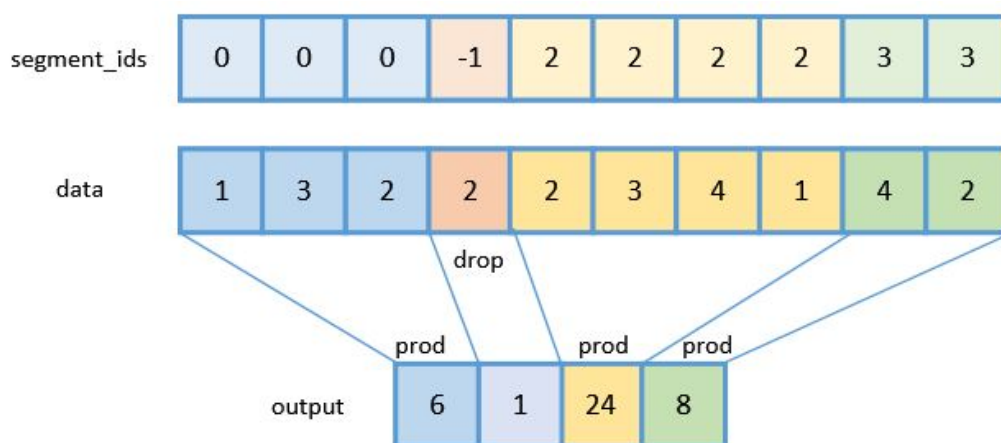
Uses the array **segment_ids** to calculate the inner product of tensors by segment. Assuming that the input is **data** and the output is **output**, then $\text{output}[i] = \text{product}(\text{data}[\mathbf{j}...])$, where **j...** is an array, and element **j** meets the following requirement: $\text{segment_ids}[\mathbf{j}] == i$.

NOTE

The parameter **product** indicates the inner product, that is, all elements in **data[j...]** are multiplied by each other.

If a subscript **i** does not appear in **segment_ids**, then $\text{output}[i] = \text{init_value}$. For example, in the following figure, if **1** does not appear in **segment_ids**, then $\text{output}[1] = 0$.

If a value in **segment_ids** is a negative number, the value of **data** in the corresponding position is discarded. For example, in the following figure, $\text{segment_ids}[3] = -1$, indicating that the value of **data[3]** is discarded and is not involved in the calculation.



The length of **segment_ids** must be the same as the length of the first dimension of **data**. The value of **num_segments** must be greater than or equal to the value of **segment_ids** plus 1.

The supported types are float16, float32, and int32.

This API is defined in **segment_compute.py**.

Parameter Description

- **tensor**: input tensor, which is of the float16, float32, or int32 type
- **segment_ids**: one-dimensional array. This array is used to segment the input tensor. The length of the array must be the same as the length of the first dimension of the input tensor. The array can be sequenced or unsequenced.
- **num_segments**: length of the first dimension of the output tensor. Its value must be greater than or equal to the value of **segment_ids** plus 1.
- **init_value**: default value of the output when a subscript in **segment_ids** does not exist. It is determined according to the implementation of the operator. The default value is **0**.

Return Value

res_tensor: tensor after calculation

Calling Example

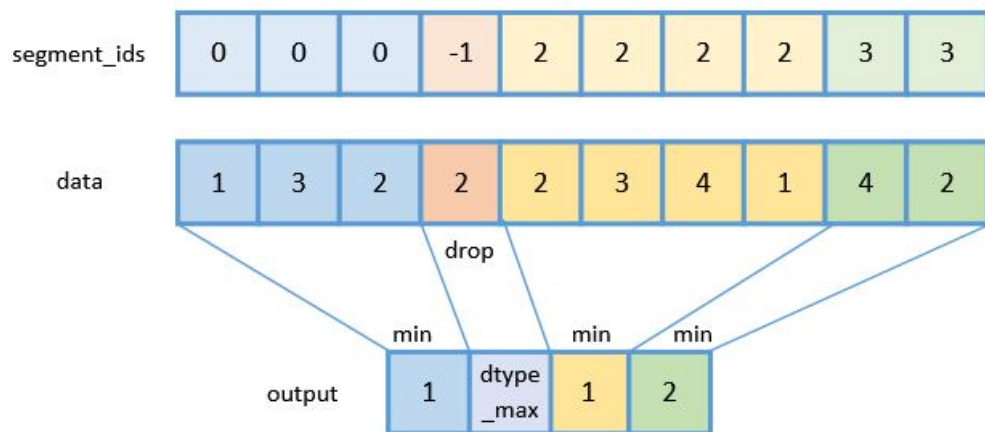
```
import tvm
import te.lang.cce
shape = (5,1024)
input_dtype = "float16"
data = tvm.placeholder(shape, name="data1", dtype=input_dtype)
segment_ids = [1,1,4,5,5]
num_segments = 6
res = te.lang.cce.unsorted_segment_prod(data, segment_ids, num_segments)
# res.shape = (6,1024)
# res[0] = 1
# res[1] = (data[0] * data[1])
# res[2] = 1
# res[3] = 1
# res[4] = data[2]
# res[5] = (data[3] * data[4])
```

8.5.3.32 te.lang.cce.unsorted_segment_min(tensor, segment_ids, num_segments, init_value=0)

Uses the array **segment_ids** to calculate the minimum value of tensors by segment. Assuming that the input is **data** and the output is **output**, then $\text{output}[i] = \min(\text{data}[j...])$, where **j...** is an array, and element **j** meets the following requirement: $\text{segment_ids}[j] == i$.

If a subscript **i** does not appear in **segment_ids**, then $\text{output}[i] = \text{init_value}$. For example, in the following figure, if **1** does not appear in **segment_ids**, then $\text{output}[1] = 0$.

If a value in **segment_ids** is a negative number, the value of **data** in the corresponding position is discarded. For example, in the following figure, $\text{segment_ids}[3] = -1$, indicating that the value of **data[3]** is discarded and is not involved in the calculation.



The length of **segment_ids** must be the same as the length of the first dimension of **data**. The value of **num_segments** must be greater than or equal to the value of **segment_ids** plus 1.

The supported types are float16 and float32.

This API is defined in **segment_compute.py**.

Parameter Description

- **tensor**: input tensor, which is of the float16, float32, or int32 type
- **segment_ids**: one-dimensional array. This array is used to segment the input tensor. The length of the array must be the same as the length of the first dimension of the input tensor. The array can be sequenced or unsequenced.
- **num_segments**: length of the first dimension of the output tensor. Its value must be greater than or equal to the value of **segment_ids** plus 1.
- **init_value**: default value of the output when a subscript in **segment_ids** does not exist. It is determined according to the implementation of the operator. The default value is 0.

Return Value

res_tensor: tensor after calculation

Calling Example

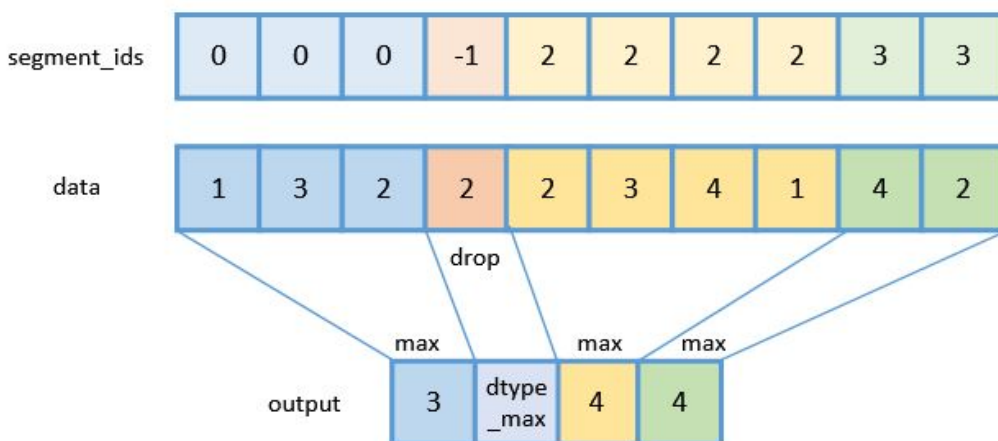
```
import tvn
import te.lang.cce
shape = (5,1024)
input_dtype = "float16"
data = tvn.placeholder(shape, name="data1", dtype=input_dtype)
segment_ids = [1,1,4,5,5]
num_segments = 6
res = te.lang.cce.unsorted_segment_min(data, segment_ids, num_segments)
# res.shape = (6,1024)
# res[0] = 65504 (maximum value of float16)
# res[1] = min(data[0], data[1])
# res[2] = 65504
# res[3] = 65504
# res[4] = data[2]
# res[5] = min(data[3], data[4])
```

8.5.3.33 te.lang.cce.unsorted_segment_max(tensor, segment_ids, num_segments, init_value=0)

Uses the array **segment_ids** to calculate the maximum value of tensors by segment. Assuming that the input is **data** and the output is **output**, then $\text{output}[i] = \max(\text{data}[j...])$, where **j...** is an array, and element **j** meets the following requirement: $\text{segment_ids}[j] == i$.

If a subscript **i** does not appear in **segment_ids**, then $\text{output}[i] = \text{init_value}$. For example, in the following figure, if **1** does not appear in **segment_ids**, then $\text{output}[1] = 0$.

If a value in **segment_ids** is a negative number, the value of **data** in the corresponding position is discarded. For example, in the following figure, $\text{segment_ids}[3] = -1$, indicating that the value of **data[3]** is discarded and is not involved in the calculation.



The length of **segment_ids** must be the same as the length of the first dimension of **data**. The value of **num_segments** must be greater than or equal to the value of **segment_ids** plus 1.

The supported types are float16, float32, and int32.

This API is defined in **segment_compute.py**.

Parameter Description

- **tensor**: input tensor, which is of the float16, float32, or int32 type
- **segment_ids**: one-dimensional array. This array is used to segment the input tensor. The length of the array must be the same as the length of the first dimension of the input tensor. The array can be sequenced or unsequenced.
- **num_segments**: length of the first dimension of the output tensor. Its value must be greater than or equal to the value of **segment_ids** plus 1.
- **init_value**: default value of the output when a subscript in **segment_ids** does not exist. It is determined according to the implementation of the operator. The default value is **0**.

Return Value

res_tensor: tensor after calculation

Calling Example

```
import tvn
import te.lang.cce shape = (5,1024)
input_dtype = "float16"
data = tvn.placeholder(shape, name="data1",type=input_dtype)
segment_ids = [1,1,4,5,5]
num_segments = 6
res = te.lang.cce.unsorted_segment_max(data,segment_ids,num_segments)
# res.shape = (6,1024)
# res[0] = 65504 (maximum value of float16)
# res[1] = max(data[0], data[1])
# res[2] = 65504
# res[3] = 65504
# res[4] = data[2]
# res[5] = max(data[3], data[4])
```

8.5.3.34 te.lang.cce.concat(raw_tensors, axis)

Reconcatenates multiple input tensors based on the specified axis.

raw_tensors indicates multiple input tensors. The data types are the same.

If `raw_tensors[i].shape = [D0, D1, ... Daxis(i), ...Dn]`, the shape of the output after the concatenation is established based on **axis** is: as follows: `[D0, D1, ... Raxis, ...Dn]`.

Where, `Raxis = sum(Daxis(i))`.

For input tensors, the dimensions of other axes must be the same except for **axis**.

For example:

```
t1 = [[1, 2, 3], [4, 5, 6]]
t2 = [[7, 8, 9], [10, 11, 12]]
concat([t1, t2], 0) # [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
concat([t1, t2], 1) # [[1, 2, 3, 7, 8, 9], [4, 5, 6, 10, 11, 12]]

# The shape of tensor t1 is [2, 3].
# The shape of tensor t2 is [2, 3].
concat([t1, t2], 0).shape # [4, 3]
concat([t1, t2], 1).shape # [2, 6]
```

The parameter **axis** can also be a negative number, indicating the **axis + len(shape)** axis, which is calculated from the end of the dimension.

For example:

```
t1 = [[[1, 2], [2, 3]], [[4, 4], [5, 3]]]
t2 = [[[7, 4], [8, 4]], [[2, 10], [15, 11]]]
concat([t1, t2], -1)
```

The output is as follows:

```
[[[ 1, 2, 7, 4],
  [ 2, 3, 8, 4]],

 [[ 4, 4, 2, 10],
  [ 5, 3, 15, 11]]]
```

The supported data types are as follows: int8, uint8, int16, int32 float16, and float32.

This API is defined in **concat_compute.py**.

Parameter Description

- **raw_tensors**: tensor list, list type. The element is `tvm.tensor`, and the last dimension of tensor shape must be 32-byte aligned.
- **axis**: axis based on which the concat operation is performed. The value range is `[-d, d-1]`. The parameter **d** indicates the dimension of **raw_tensor**.

Return Value

res_tensor: tensor after reconcatenation is implemented, `tvm.tensor` type

Calling Example

```
import tvm
import te.lang.cce
shape1 = (64,128)
shape2 = (64,128)
input_dtype = "float16"
data1 = tvm.placeholder(shape1, name="data1", dtype=input_dtype)
data2 = tvm.placeholder(shape2, name="data1", dtype=input_dtype)
data = [data1, data2]
res = te.lang.cce.concat(data, 0)
# res.shape = (128,128)
```

8.5.3.35 te.lang.cce.conv(*args)

Computes 2D convolution when the 4D input and **filter** are specified.

The formats of the input tensor and the filter tensor must be NCHW.

The interface supports **bias**, and the supported data type is float16.

This API is defined in **conv_compute.py**.

Parameter Description

***args**: list. The value is variable.

- The parameter list for the bias scenario is as follows: (**hasBias** must be set to **True**)

A, W, B, res_dtype, padh, padw, strideh, stridew, hasBias

- The parameter list for the non-bias scenario is as follows: (**hasBias** must be set to **False**)

A, W, res_dtype, padh, padw, strideh, stridew, hasBias

Where,

- **A**: input tensor, that is, feature map for convolutional calculation
- **W**: filter tensor, that is, convolutional core for convolutional calculation
- **B**: bias tensor, that is, offset of convolution calculation
- **res_dtype**: data type of the output tensor, that is, data type of the convolution calculation result.
- **padh**: height of padding, that is, number of padding times in the H direction of the feature map during convolutional calculation
- **padw**: width of padding, that is, number of padding times in the W direction of the feature map during convolutional calculation

- **strideh**: stride in the H direction, that is, movement stride of the convolutional calculation filter in the H direction on the feature map
- **stridew**: stride in the W direction, that is, movement stride of the convolutional calculation filter in the W direction on the feature map
- **hasBias**: whether **bias** is contained
- Constraints: Assume that the shape of the feature map is (Fn, Fc, Fh, Fw), the shape of **filter** is (Wn, Wc, Wh, Ww), and the output shape of the convolution result is (On, Oc, Oh, Ow). **padh** is **Ph**, **padw** is **Pw**, **strideh** is **Sh**, and **stridew** is **Sw**. These parameters must meet the following requirements:
 - $F_c = W_c$
 - $O_n = F_n$
 - $O_c = W_n$
 - $O_h = ((F_h + 2P_h - W_h) / S_h) + 1$
 - $O_w = ((F_w + 2P_w - W_w) / S_w) + 1$

Return Value

res_tensor: tensor for convolutional calculation, that is, output of convolutional calculation

Calling Example

```
import tvn
import te.lang.cce
shape_in = (64,64)
shape_w = (3,3)
in_dtype = "float16"
A = tvn.placeholder(shape_in, name='A', dtype=in_dtype)
W = tvn.placeholder(shape_w, name='W', dtype=in_dtype)
b_shape = (shape_w[0], )
B = tvn.placeholder(b_shape, name='B', dtype=res_dtype)
padh = 0
padw = 0
strideh = 1
stridew = 1
res = te.lang.cce.conv(A, W, B, in_dtype, padh, padw, strideh, stridew, True)
# res = A * W + B
```

8.5.3.36 te.lang.cce.compute_four2five(input, raw_shape_4D)

Converts the 4D data format **NCHW** to the 5D data format **NC1HWC0**. The supported type is float16.

This API is defined in **dim_conv.py**.

Parameter Description

- **input**: input tensor, 4D (N,C,H,W), tvn.tensor type
- **raw_shape_4D**: dimension of the input tensor

Return Value

res_tensor: tensor after the data format is converted into the 5D format (N,C1,H,W,C0), tvn.tensor type

Calling Example

```
import tvm
import te.lang.cce
raw_shape = (N,C,H,W)
in_dtype = "float16"
input = tvm.placeholder(raw_shape, name='input', dtype=in_dtype)
res = te.lang.cce.compute_four2five(input, raw_shape)
# res.shape = (N,(C+15)//16,H,W,16)
```

8.5.3.37 te.lang.cce.compute_five2four(input, raw_shape_4D)

Converts the 5D data format **NC1HWC0** to the 4D data format **NCHW**. The supported type is float16.

This API is defined in **dim_conv.py**.

Parameter Description

- **input**: input tensor, 5D format (N,C1,H,W,C0), tvm.tensor type
- **raw_shape_4D**: dimension of the tensor after conversion

Return Value

res_tensor: tensor after the data format is converted into the 4D format (N,C,H,W), tvm.tensor type

Calling Example

```
import tvm
import te.lang.cce
raw_shape = (N,C,H,W)
input_shape = (N,(C+15)//16,H,W,16)
in_dtype = "float16"
input = tvm.placeholder(input_shape, name='input', dtype=in_dtype)
res = te.lang.cce.compute_five2four(input, raw_shape)
# res.shape = (N,C,H,W)
```

8.5.3.38 te.lang.cce.matmul(tensor_a, tensor_b, trans_a=False, trans_b=False, alpha_num=1.0, beta_num=0.0, tensor_c=None)

Performs matrix multiplication as follows: $\text{tensor_c} = \alpha_num * \text{trans_a}(\text{tensor_a}) * \text{trans_b}(\text{tensor_b}) + \beta_num * \text{tensor_c}$.

For **tensor_a** and **tensor_b**, the last two dimensions of **shape** (after transposition) must meet the following condition: Matrix multiplication $(M, K) * (K, N) = (M, N)$. Only one batch is supported. The **tensor_a** data layout must comply with the fractal structure of L0A, and the **tensor_b** data layout must comply with the fractal structure of L0B. In mini mode, only the float16 data type is supported.

This API is defined in **mmad_compute.py**.

Parameter Description

- **tensor_a**: matrix A, tvm.tensor type
- **tensor_b**: matrix B, tvm.tensor type
- **trans_a**: whether matrix A is transposed, Boolean type

- **trans_b**: whether matrix B is transposed, Boolean type
- **alpha_num**: matrix A*B coefficient. Only 1.0 is supported.
- **beta_num**: matrix C coefficient. Only 0.0 is supported.
- **tensor_c**: matrix C, tvm.tensor type. Since **beta_num** supports only 0.0, this parameter is used as a reserved interface.

Return Value

tensor_c: tensor calculated based on the relational calculation, tvm.tensor type

Calling Example

```
import tvm
import te.lang.cce
a_shape = (1024, 256)
b_shape = (256, 512)
a_fractal_shape = (a_shape[0] // 16, a_shape[1] // 16, 16, 16)
b_fractal_shape = (b_shape[0] // 16, b_shape[1] // 16, 16, 16)
in_dtype = "float16"
tensor_a = tvm.placeholder(a_fractal_shape, name='tensor_a', dtype=in_dtype)
tensor_b = tvm.placeholder(b_fractal_shape, name='tensor_b', dtype=in_dtype)
res = te.lang.cce.matmul(tensor_a, tensor_b, False, False)
```

8.5.4 Build APIs

The build APIs are used to generate the schedule object for the defined computation process and build the CCE operator file.

8.5.4.1 te.lang.cce.auto_schedule(outs)

Generates the schedule of DSL. This API is defined in **cce.py**.

Parameter Description

outs: computational graph description of the operator, that is, DSL

Return Value

schedule: computation schedule of the operator

Calling Example

```
import te.lang.cce
from te import tvm
import topi

shape = (28,28)
dtype = "float16"
# Define input.
data = tvm.placeholder(shape, name="data", dtype=dtype)
# Describe the computation process of the operator.
res = te.lang.cce.vabs(data)
with tvm.target.cce():
    # Generate the schedule object.
    sch = topi.generic.auto_schedule(res)
```


8.5.4.2 `te.lang.cce.cce_build_code(sch, config_map = {})`

Prints lower code or performs build for the schedule. This API is defined in `cce_schedule.py`.

Parameter Description

- **sch**: `tvm.schedule`, schedule to build or to print lower code
- **config_map**: build parameter configuration, which is a dictionary. The default value is `{}` and is used. The following keys are included:
 - **print_ir**: whether to print lower IR code. The default value is **True**.
 - **need_build**: whether build is performed. The default value is **True**.
 - **name**: operator name. The default value is **cce_op**.
 - **tensor_list**: list of input and output tensors of the operator. The input is the tensor object returned by the placeholder interface. The output is the calculated tensor object. The value is mandatory. Otherwise, an error is reported. In addition, this list determines the sequence of the parameters of the kernel function for operator generation, which is the same as the sequence of the input and output in the list.

Return Value

None

Calling Example

```
import te.lang.cce
from te import tvm
from topi import generic
# Define the input placeholder.
data = tvm.placeholder(shape, name="data", dtype=dtype)
with tvm.target.cce():
    # Describe the computation process of the operator.
    res = te.lang.cce.vabs(data)
    # Generate the schedule object.
    sch = generic.auto_schedule(res)
# Define build parameters.
config = {"print_ir" : True,
         "need_build" : True,
         "name" : "abs_28_28_float16",
         "tensor_list" : [data,res]
        }
# Build an operator.
te.lang.cce.cce_build_code(sch, config)
```

8.5.5 Convergence API

The TE operator convergence function pre-processes information such as the convergence capability and input parameters of a single operator. Therefore, a specified API needs to be called to generate a single operator. The following describes the usage and restrictions of the API in detail.

8.5.5.1 bool BuildTeCustomOp(std::string ddkVer, std::string opName, std::string opPath, std::string opFuncName, const char *format, ...)

Generates the *.o and *.json files of a single operator. You can view the API definitions in the **ddk/include/inc/custom/custom_op.h** file under the installation directory of the DDK package.

Parameter Description

- **ddkVer**: product version number
Version number rule: **x.y.z.patch.B*****
x: corresponding to the VR version
y: corresponding to the C version
z: number of releases
patch: patch number (optional)
B***: internal B version number
For the specific version number, see the configuration in the **ddk_info** file under the installation directory of the DDK package.
- **opName**: name of a single operator node on the network. The value cannot be empty and can contain only letters, digits, underscores (_), and hyphens (-).
- **opPath**: path of the operator file. The directory contains slashes (/) and file names without filename extensions. The value cannot be empty. The file name can contain only letters, digits, underscores (_), and hyphens (-).
- **opFuncName**: operator function name in the operator file. The function name can contain only letters, digits, underscores (_), and hyphens (-).
- **format**: input parameter of an operator, which has a variable length and must be consistent with the parameter sequence of the operator

Return Value

- **true**: A single operator is successfully generated.
- **false**: A single operator fails to be generated.

Calling Example

```
std::string FilePath = "topi/cce/caffe_reduction_layer";  
std::string FuncName = "caffe_reduction_layer_cce";  
std::string KernelName = "cce_reductionLayer_5_10_5_5_float16__1_SUMSQ_1_0";  
  
// i => int; s => string; f => double; O => bool, and bool value is Py_True or Py_False  
te::BuildTeCustomOp(1.1.1.patch.B001, "reduction", FilePath, FuncName, "(i,i,i), s, i, s, f, s", 5, 5, 5, 5,  
"float16", 1, "SUM", 1.0, KernelName.c_str());
```

8.5.6 APIs that Compilation Depends on

The following APIs are used only during operator compilation, and are not directly called during operator writing: You can view the API definitions in the **ddk/include/inc/tensor_engine/cce_aicpu_3rd_party.h** file under the installation directory of the DDK package.

- `__aicpu__ void *aicpu_malloc(unsigned int size)`

This API is used to allocate memory. The input parameter **size** indicates the size of memory to be allocated. The pointer to memory allocation is returned.

- `__aicpu__ void aicpu_free(void * ptr)`

This API is used to release memory. **ptr** is the memory pointer.

- `__aicpu__ double log(double x)`

This API is used to obtain the log value of **x** (double type).

- `__aicpu__ double exp(double x)`

This API is used to obtain the EXP value of **x** (double type).

- `__aicpu__ double round(double x)`

This API is used to round off the value of **x** (double type).

- `__aicpu__ double floor(double x)`

This API is used to round down the value of **x** (double type).

- `__aicpu__ double ceil(double x);`

This API is used to round up the value of **x** (double type).

- `__aicpu__ double trunc(double x);`

This API is used to obtain the truncation value of **x** (double type).

- `__aicpu__ double sqrt(double x);`

This API is used to obtain the square value of **x** (double type).

- `__aicpu__ float logf(float x)`

This API is used to obtain the log value of **x** (double type).

- `__aicpu__ float expf(float x)`

This API is used to obtain the EXP value of **x** (float type).

- `__aicpu__ float roundf(float x)`

This API is used to round off the value of **x** (float type).

- `__aicpu__ float floorf(float x)`

This API is used to round down the value of **x** (float type).

- `__aicpu__ float ceilf(float x);`

This API is used to round up the value of **x** (float type).

- `__aicpu__ float truncf(float x);`

This API is used to obtain the truncation value of **x** (float type).

- `__aicpu__ float sqrtf(float x);`

This API is used to obtain the square value of **x** (float type).

8.5.7 Instructions

8.5.7.1 Usage Example

The following example shows how to concatenate the preceding APIs. It implements a simple operator that supports the float16 type to obtain an absolute value.

```
import te.lang.cce
from te import tvn
import topi
```

```
shape = (28,28)
dtype = "float16"
# Define input.
data = tvm.placeholder(shape, name="data", dtype=dtype)

# Describe the computation process of the operator.
res = te.lang.cce.vabs(data)

with tvm.target.cce():
    # Generate the schedule object.
    sch = topi.generic.auto_schedule(res)

# Define build parameters.
config = {"print_ir" : True,
          "need_build" : True,
          "name" : "abs_28_28_float16",
          "tensor_list" : [data,res]}
# Build an operator.
te.lang.cce.cce_build_code(sch, config)
```

8.5.7.2 Exception Handling

If an exception occurs when an API is executed, the error is usually caused by incorrect input parameters. The following example shows the error information when **tensor_list** is incomplete.

Code example:

```
data = tvm.placeholder(shape, name="data", dtype=inp_dtype)
with tvm.target.cce():
    res = te.lang.cce.vabs(data)
    sch = generic.auto_schedule(res)
    config = {"print_ir": need_print,
              "need_build": need_build,
              "name": kernel_name,
              "tensor_list": [res]}
te.lang.cce.cce_build_code(sch, config)
```

The following error information is displayed:

```
Traceback (most recent call last):
  File "/home1/repotvm/tensor_engine/ut/testcase_python/tf_abs/test_tf_abs_cce.py", line 71, in test_cce_tf_abs_99991_fp16
    tf_abs_cce((99991,), dtype = "Float16", need_build = False, need_print = False, kernel_name =
"cce_tf_abs")
  File "/home1/repotvm/tensor_engine/topi/python/topi/cce/tf_abs.py", line 68, in tf_abs_cce
    te.lang.cce.cce_build_code(sch, config)
  File "/home1/repotvm/tensor_engine/python/te/lang/cce/te_schedule/cce_schedule.py", line 381, in
cce_build_code
    _build(sch, tensor_list, local_config_map["name"])
  File "/home1/repotvm/tensor_engine/python/te/lang/cce/te_schedule/cce_schedule.py", line 338, in _build
    mod = tvm.build(sch, tensor_list, device, name=name)
  File "/home1/repotvm/tensor_engine/python/te/tvm/build_module.py", line 432, in build
    binds=binds)
  File "/home1/repotvm/tensor_engine/python/te/tvm/build_module.py", line 353, in lower
    stmt = ir_pass.StorageFlatten(stmt, binds, 64)
  File "/home1/repotvm/tensor_engine/python/te/tvm/_ffi/function.py", line 280, in my_api_func
    return flocal(*args)
  File "/home1/repotvm/tensor_engine/python/te/tvm/_ffi/_ctypes/function.py", line 183, in __call__
    ctypes.byref(ret_val), ctypes.byref(ret_tcode)))
  File "/home1/repotvm/tensor_engine/python/te/tvm/_ffi/base.py", line 66, in check_call
    raise TVMError(py_str(_LIB.TVMGetLastError()))
TVMError: [17:12:02] /home1/repotvm/tensor_engine/src/pass/storage_flatten.cc:249: Check failed: it !=
buf_map_end() Cannot find allocated buffer for placeholder(data, 0x27d7290)
```

The problem is solved after the parameter is modified as follows:

```
"tensor_list" : [data, res]
```

9 Command Reference

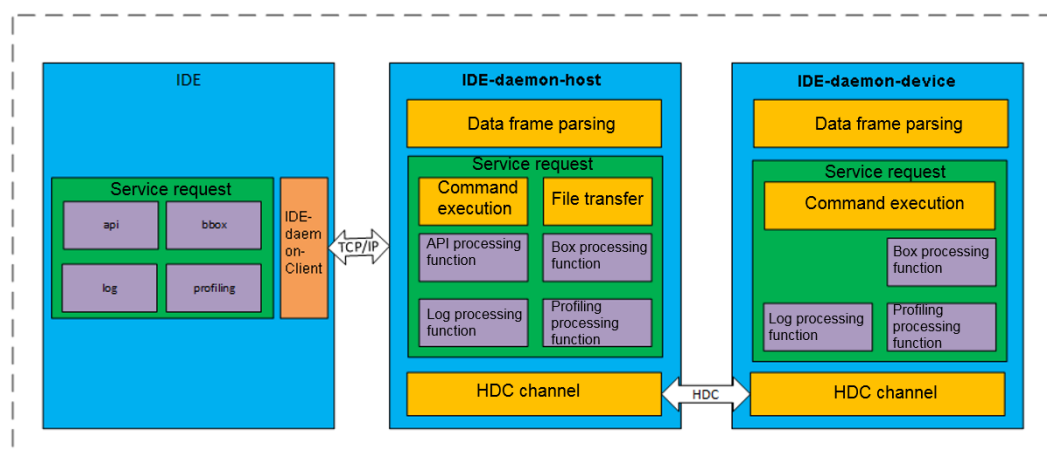
9.1 IDE-daemon-client Command Reference

9.1 IDE-daemon-client Command Reference

9.1.1 Introduction

After the **IDE-daemon-client** command is run, the specified command is sent to the host. If the command needs to be run on the host, run it directly on the host. If the command needs to be run on the device, connect the host to the device and run the command on the device. For details, see the *IDE-daemon-client Command Reference*.

Figure 9-1 Overall architecture of IDE-daemon



Currently, the **IDE-daemon-client** command can be run on the host server in PCIe mode or on the server where the DDK is located.

- To run the command the host server, you can log in to the host server as the **HwHiAiUser** user and run the **IDE-daemon-client** command. This document uses this scenario as an example.

- If the DDK is installed, run the **IDE-daemon-client** command on the DDK server.
 - The DDK is deployed on an independent server and is not shared with the host server. You can log in to the DDK server as the DDK installation user, run the following commands to set the environment variables, and run the **IDE-daemon-client** command: (replace *DDK installation directory* with the actual directory):


```
export LD_LIBRARY_PATH=DDK installation directory/ddk/uihost/lib
export PATH=$PATH:DDK installation directory/ddk/uihost/bin
```
 - If the DDK is deployed on the host server and the DDK installation user is **HwHiAiUser**, you can log in to the server as the **HwHiAiUser** user, and run the **IDE-daemon-client** command without setting the environment variables.

Commands that exceed one line will be automatically wrapped due to the restriction of the PDF document format. Therefore, if you want to use commands in this document directly, you need to manually merge the lines into one line and separate the parameters with spaces.

9.1.2 Sending a Command to the Device by Using IDE Daemon

Function

Sends a command to the device by using IDE Daemon, such as a command for querying the date information on the device.

Syntax

```
IDE-daemon-client --host <host_ip>[:<port>] [--device <deviceId>] --cmd <command>
```

Parameter

Table 9-1 Parameter description

Parameter	Description	Mandatory or Not	Default Value
--host	• <i>host_ip</i>: specifies the IP address of the host. • <i>port</i>: specifies the port number.	Yes	The default value of <i>port</i> is 22118 .

Parameter	Description	Mandatory or Not	Default Value
--device	If the host is connected to multiple devices, you can specify a device by using --device deviceID . If the device is not specified, the device with <i>deviceID</i> of 0 is used by default.	No	The default value of <i>deviceID</i> is 0 .
--cmd	Specifies the command to be executed on the device. For details about the command list, see Command List . Except the date command, other commands must be placed in single quotation marks or double quotation marks. If a command contains double quotation marks, put the command in single quotation marks. If the command contains single quotation marks, place the command in double quotation marks.	Yes	-

Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to view the date information on the device:
IDE-daemon-client --host xx.xx.xx.xx:22118 --cmd date

NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.

Command List

If a command has variables, replace the variables such as *xxxx* or *yy* with the actual directory name or file name.

In the commands, ~ indicates the home directory of the user who starts the IDE-daemon-host or IDE-daemon-device process.

Cat ego ry	Command	Remarks
Co mm and wit hou t vari abl es	kill cce-gdbserver	-
	ide_cmd.sh --reboot	The sudo command is used to escalate the user's operation permission. The actually executed command is reboot mini . Used to restart the developer board.
	date	-
	mkdir -p /tmp/profiler-tools/	-
	rm /opt/mini/mini_developerKit*.rar	-
	chmod +x /opt/mini/minirc_install_phase1.sh	-
	chmod +w ~/ide_daemon/rollback_idedaemon.crt.sh	-
	chmod +w ~/ide_daemon/backup_idedaemon.crt.sh	-
	/opt/mini/minirc_install_phase1.sh	-
	tar -xvf ~/host/EVB.tar -C ~/host/	-
	rm -rf /var/dlog/*	-
	rm -rf ~/host	-
	mkdir -p ~/host	-
	rm -rf ~/HIAI_PROJECTS/dlog.tar	-
	tar -cPf ~/HIAI_PROJECTS/dlog.tar /var/dlog/	-
	grep -rn "firmware upgrade finish, start to reboot" /var/davinci/log	-
	grep -rn "fail" /var/davinci/log/firmware_upgrade_progress.log	-

	<pre>python -c \"import future;import sys;sys.exit(future.__version__ < 0.15)\"</pre>	Checks the version number of the future library on the host. The actually executed command is as follows: <pre>python -c "import future;import sys;sys.exit(future.__v ersion__ < 0.15)"</pre>
	<pre>python -c \"import numpy;import sys;sys.exit(numpy.__version__ < 1.14)\"</pre>	Checks the version number of the numpy library on the host. The actually executed command is as follows: <pre>python -c "import numpy;import sys;sys.exit(numpy.__ version__ < 1.14)"</pre>
	<pre>"python -c \"\n\"import enum\n\"import sys \n\"try:\n\" if enum.__version__ >= 0.4:\n\" sys.exit(0)\n\"except AttributeError:\n\" if enum.version >= 1.0:\n\" sys.exit(0)\n\"\""</pre>	Checks the version number of the enum library or enum34 library on the host. The actually executed command is as follows: <pre>python -c \" import enum import sys try: if enum.__version__ >= 0.4: sys.exit(0) except AttributeError: if enum.version >= 1.0: sys.exit(0) \"</pre>
	<pre>cat /etc/sys_version.conf</pre>	Obtains the Ascend 310 software version of the developer board.

Com mand and with h vari abl es	<code>rm -rf ~/hdc; rm -rf ~/HIAI_PROJECTS/xxx; rm -rf ~/HIAI_DATANDMODELSET/xxx; rm -rf ~/profiler-app/xxx; rm -rf ~/ide_daemon/xxx</code>	-
	<code>rm ~/HIAI_PROJECTS/xxx; rm ~/HIAI_DATANDMODELSET/xxx; rm ~/profiler-app/xxx; rm ~/ide_daemon/xxx</code>	-
	<code>wc -l ~/hdc/xxx; wc -l ~/HIAI_PROJECTS/xxx; wc -l ~/HIAI_DATANDMODELSET/xxx; wc -l ~/profiler-app/xxx</code>	-
	<code>mkdir -p ~/hdc/xxx; mkdir -p ~/HIAI_PROJECTS/xxx; mkdir -p ~/HIAI_DATANDMODELSET/xxx; mkdir -p ~/profiler-app/xxx; mkdir -p ~/ide_daemon/xxx</code>	Runs an executable file.
	<code>mkdir ~/HIAI_PROJECTS/xxx; mkdir ~/HIAI_DATANDMODELSET/xxx; mkdir ~/profiler-app/xxx; mkdir ~/ide_daemon/xxx</code>	-
	<code>tar -cf ~/hdc/xxx/yy.tar -C ~/HIAI_PROJECTS/xxx xxx; tar -cf ~/HIAI_PROJECTS/xxx/yy.tar -C ~/HIAI_PROJECTS/xxx xxx; tar -cf ~/HIAI_DATANDMODELSET/xxx/yy.tar -C ~/HIAI_DATANDMODELSET/xxx xxx; tar -cf ~/profiler-app/xxx/yy.tar -C ~/profiler-app/xxx xxx</code>	Compress the directories and files specified by -C into packages specified by -cf . Change xxx to the actual directory name and yy to the actual .tar package name.
	<code>tar -xvf ~/hdc/xxx/yy.tar -C ~/HIAI_PROJECTS/xxx; tar -xvf ~/HIAI_PROJECTS/xxx/yy.tar -C ~/HIAI_PROJECTS/xxx; tar -xvf ~/HIAI_DATANDMODELSET/xxx/yy.tar -C ~/HIAI_DATANDMODELSET/xxx; tar -xvf ~/profiler-app/xxx/yy.tar -C ~/profiler-app/xxx</code>	-
	<code>mv ~/hdc/xxx ~/HIAI_PROJECTS/xxx; mv ~/HIAI_PROJECTS/xxx ~/HIAI_PROJECTS/xxx; mv ~/profiler-app/xxx ~/profiler-app/xxx</code>	-
	<code>chmod +x ~/hdc/xxx; chmod +x ~/HIAI_PROJECTS/xxx; chmod +x ~/profiler-app/xxx; chmod +x ~/ide_daemon/xxx</code>	-
	<code>chmod -w ~/hdc/xxx; chmod -w ~/ide_daemon/xxx</code>	-
	<code>cp ~/HIAI_PROJECTS/xxx ~/HIAI_PROJECTS/xxx; cp -af ~/HIAI_PROJECTS/xxx ~/HIAI_PROJECTS/xxx</code>	-

sha512sum ~/hdc/xxxx; sha512sum ~/HIAI_DATANDMODELSET/xxxx	-
find ~/HIAI_DATANDMODELSET/xxxx -type f xargs sha512sum sort	-
~/HIAI_PROJECTS/xxxx; ~/ide_daemon/xxxx	Runs an executable file.
~/HIAI_PROJECTS/xxxx ~/HIAI_DATANDMODELSET/xxxx yyy ddd zzz	Runs a binary file with fixed parameters. Replace ~/ HIAI_DATANDMODELSET/xxxx , <i>yyy</i> , <i>ddd</i> , and <i>zzz</i> with the actual values.
cce-gdbserver <i>ip_address:port</i> --attach <i>xxx</i> 2>&1; cce-gdbserver <i>ip_address:port</i> ~/ide_daemon/debug/xxxx	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process ID.
pgrep -x -f "cce-gdbserver <i>ip_address:port</i> --attach <i>xxx</i> "; pgrep -x -f "cce-gdbserver <i>ip_ddress:port</i> ~/ide_daemon/debug/xxxx"	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process ID.
pidof xxxx	Replace <i>xxxx</i> with the actual process name.
ps ux awk '{print}' sed 1d; ps ux awk '{print \$x \t\$y}' sed 1d	-
ps -ef awk '{print}' sed 1d; ps -ef awk '{print \$x \t\$y}' sed 1d	-

	<i>ide_cmd.sh --eth0_up ip_address</i>	The sudo command is used to escalate the user's operation permission. The actually executed command is ifconfig eth0 ip_address. This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.
	<i>ide_cmd.sh --setup_eth0 ip_address netmask_address gw_address</i>	The sudo command is used to escalate the user's operation permission. The actually executed command is ifconfig eth0 ip_address netmask netmask_address && route add default gw gw_address. This command applies only to developer boards. Replace <i>ip_address/netmask_address/gw_address</i> with the actual value.
	<i>sed -i '/iface.eth0/{n;s/address.*/address ip_address/;}' /etc/network/interfaces</i>	The sudo command is used to escalate the user's operation permission. sudo command This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.

	<code>sed -i '/iface.eth0/{n;n;s/netmask.*/netmask ip_address/;} /etc/network/interfaces</code>	The sudo command is used to escalate the user's operation permission. sudo command This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.
	<code>sed -i '/iface.eth0/{n;n;n;s/gateway.*/gateway ip_address/;} /etc/network/interfaces</code>	The sudo command is used to escalate the user's operation permission. sudo command This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.
	<code>del /f/s/q D:\profiler-app; del /f/s/q D:\profiler-app \xxx</code>	Deletes a file on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
	<code>dir D:\profiler-app; dir D:\profiler-app\xxx</code>	Lists all files in a directory on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
	<code>md D:\profiler-app; md D:\profiler-app\xxx</code>	Creates a folder on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.

<code>rd /s/q D:\profiler-app; rd /s/q D:\profiler-app\xxx</code>	Deletes a folder on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
<code>tar -xvf D:\profiler-app\xxx.tar -C D:\profiler-app\yyy</code>	Creates a compressed package on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
<code>kill \$(pidof ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill \$(pidof -x ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill -9 \$(pidof ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill -9 \$(pidof -x ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill `pgrep -x -f "cce-gdbserver ip_address:port --attach xxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process number.
<code>kill -9 `pgrep -x -f "cce-gdbserver ip_address:port --attach xxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process number.
<code>kill `pgrep -x -f "cce-gdbserver ip_address:port ~/ide_daemon/debug/xxxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values.
<code>kill -9 `pgrep -x -f "cce-gdbserver ip_address:port ~/ide_daemon/debug/xxxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values.
<code>cd ~/HIAI_PROJECTS/xxxx/out;./yy</code>	Runs an executable file in the directory. This command has no parameter.

	<code>cd ~/HIAI_PROJECTS/xxxx/out;./yy ~/HIAI_DATANDMODELSET/xxxx yyy ddd zzz</code>	Runs a binary file in the directory. The parameters are fixed. Replace ~/HIAI_DATANDMODELSET/xxxx, yyy, ddd, and zzz with the actual values.
	<code>cd ~/HIAI_PROJECTS/xxxx/out;~/HIAI_PROJECTS/xxxx/out/yyy ~/HIAI_DATANDMODELSET/xxxx yyy ddd zzz</code>	Runs a binary file in the directory. The parameters are fixed. Replace ~/HIAI_DATANDMODELSET/xxxx, yyy, ddd, and zzz with the actual values.

9.1.3 Sending a Command to the Device by Using IDE Daemon

Function

Sends a command to the host by using IDE Daemon, such as a command for querying the date information on the host.

Syntax

```
IDE-daemon-client --host <host_ip>:<port> --hostcmd <command>
```

Parameter

Table 9-2 Parameter description

Parameter	Description	Mandatory or Not	Default Value
--host	<i>host_ip</i>: specifies the IP address of the host. <i>port</i>: specifies the port number.	Yes	The default value of <i>port</i> is 22118 .

Parameter	Description	Mandatory or Not	Default Value
--hostcmd	Specifies the command to be executed on the host. For details about the command list, see Command List . Except the date command, other commands must be placed in single quotation marks or double quotation marks. If a command contains double quotation marks, put the command in single quotation marks. If the command contains single quotation marks, place the command in double quotation marks.	Yes	-

Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to view the date information on the host:
IDE-daemon-client --host xx.xx.xx.xx:22118 --hostcmd date

NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.

Command List

If a command has variables, replace the variables such as *xxxx* or *yy* with the actual directory name or file name.

In the commands, ~ indicates the home directory of the user who starts the IDE-daemon-host or IDE-daemon-device process.

Cat ego ry	Command	Remarks
Co mm and wit hou t vari abl es	pskill cce-gdbserver	-
	ide_cmd.sh --reboot	The sudo command is used to escalate the user's operation permission. The actually executed command is reboot mini . Used to restart the developer board.
	date	-
	mkdir -p /tmp/profiler-tools/	-
	rm /opt/mini/mini_developerKit*.rar	-
	chmod +x /opt/mini/minirc_install_phase1.sh	-
	chmod +w ~/ide_daemon/rollback_idedaemon.crt.sh	-
	chmod +w ~/ide_daemon/backup_idedaemon.crt.sh	-
	/opt/mini/minirc_install_phase1.sh	-
	tar -xvf ~/host/EVB.tar -C ~/host/	-
	rm -rf /var/dlog/*	-
	rm -rf ~/host	-
	mkdir -p ~/host	-
	rm -rf ~/HIAI_PROJECTS/dlog.tar	-
	tar -cPf ~/HIAI_PROJECTS/dlog.tar /var/dlog/	-
	grep -rn "firmware upgrade finish, start to reboot" /var/davinci/log	-
	grep -rn "fail" /var/davinci/log/firmware_upgrade_progress.log	-

	<pre>python -c \"import future;import sys;sys.exit(future.__version__ < 0.15)\"</pre>	Checks the version number of the future library on the host. The actually executed command is as follows: <pre>python -c "import future;import sys;sys.exit(future.__v ersion__ < 0.15)"</pre>
	<pre>python -c \"import numpy;import sys;sys.exit(numpy.__version__ < 1.14)\"</pre>	Checks the version number of the numpy library on the host. The actually executed command is as follows: <pre>python -c "import numpy;import sys;sys.exit(numpy.__ version__ < 1.14)"</pre>
	<pre>"python -c \"\n\"import enum\n\"import sys \n\"try:\n\" if enum.__version__ >= 0.4:\n\" sys.exit(0)\n\"except AttributeError:\n\" if enum.version >= 1.0:\n\" sys.exit(0)\n\"\""</pre>	Checks the version number of the enum library or enum34 library on the host. The actually executed command is as follows: <pre>python -c \" import enum import sys try: if enum.__version__ >= 0.4: sys.exit(0) except AttributeError: if enum.version >= 1.0: sys.exit(0) \"</pre>
	<pre>cat /etc/sys_version.conf</pre>	Obtains the Ascend 310 software version of the developer board.

Com mand and with h vari abl es	<code>rm -rf ~/hdcd; rm -rf ~/HIAI_PROJECTS/xxxx; rm -rf ~/HIAI_DATANDMODELSET/xxxx; rm -rf ~/profiler-app/xxxx; rm -rf ~/ide_daemon/xxxx</code>	-
	<code>rm ~/HIAI_PROJECTS/xxxx; rm ~/HIAI_DATANDMODELSET/xxxx; rm ~/profiler-app/xxxx; rm ~/ide_daemon/xxxx</code>	-
	<code>wc -l ~/hdcd/xxxx; wc -l ~/HIAI_PROJECTS/xxxx; wc -l ~/HIAI_DATANDMODELSET/xxxx; wc -l ~/profiler-app/xxxx</code>	-
	<code>mkdir -p ~/hdcd/xxxx; mkdir -p ~/HIAI_PROJECTS/xxxx; mkdir -p ~/HIAI_DATANDMODELSET/xxxx; mkdir -p ~/profiler-app/xxxx; mkdir -p ~/ide_daemon/xxxx</code>	Runs an executable file.
	<code>mkdir ~/HIAI_PROJECTS/xxxx; mkdir ~/HIAI_DATANDMODELSET/xxxx; mkdir ~/profiler-app/xxxx; mkdir ~/ide_daemon/xxxx</code>	-
	<code>tar -cf ~/hdcd/xxxx/yy.tar -C ~/HIAI_PROJECTS/xxxx xxxx; tar -cf ~/HIAI_PROJECTS/xxxx/yy.tar -C ~/HIAI_PROJECTS/xxxx xxxx; tar -cf ~/HIAI_DATANDMODELSET/xxxx/yy.tar -C ~/HIAI_DATANDMODELSET/xxxx xxxx; tar -cf ~/profiler-app/xxxx/yy.tar -C ~/profiler-app/xxxx xxxx</code>	Compress the directories and files specified by -C into packages specified by -cf . Change <i>xxxx</i> to the actual directory name and <i>yy</i> to the actual .tar package name.
	<code>tar -xvf ~/hdcd/xxxx/yy.tar -C ~/HIAI_PROJECTS/xxxx; tar -xvf ~/HIAI_PROJECTS/xxxx/yy.tar -C ~/HIAI_PROJECTS/xxxx; tar -xvf ~/HIAI_DATANDMODELSET/xxxx/yy.tar -C ~/HIAI_DATANDMODELSET/xxxx; tar -xvf ~/profiler-app/xxxx/yy.tar -C ~/profiler-app/xxxx</code>	-
	<code>mv ~/hdcd/xxxx ~/HIAI_PROJECTS/xxxx; mv ~/HIAI_PROJECTS/xxxx ~/HIAI_PROJECTS/xxxx; mv ~/profiler-app/xxxx ~/profiler-app/xxxx</code>	-
	<code>chmod +x ~/hdcd/xxxx; chmod +x ~/HIAI_PROJECTS/xxxx; chmod +x ~/profiler-app/xxxx; chmod +x ~/ide_daemon/xxxx</code>	-
	<code>chmod -w ~/hdcd/xxxx; chmod -w ~/ide_daemon/xxxx</code>	-
	<code>cp ~/HIAI_PROJECTS/xxxx ~/HIAI_PROJECTS/xxxx; cp -af ~/HIAI_PROJECTS/xxxx ~/HIAI_PROJECTS/xxxx</code>	-

sha512sum ~/hdc/xxxx; sha512sum ~/HIAI_DATANDMODELSET/xxxx	-
find ~/HIAI_DATANDMODELSET/xxxx -type f xargs sha512sum sort	-
~/HIAI_PROJECTS/xxxx; ~/ide_daemon/xxxx	Runs an executable file.
~/HIAI_PROJECTS/xxxx ~/HIAI_DATANDMODELSET/xxxx yyy ddd zzz	Runs a binary file with fixed parameters. Replace ~/ HIAI_DATANDMODELSET/xxxx , <i>yyy</i> , <i>ddd</i> , and <i>zzz</i> with the actual values.
cce-gdbserver <i>ip_address:port</i> --attach <i>xxx</i> 2>&1; cce-gdbserver <i>ip_address:port</i> ~/ide_daemon/debug/xxxx	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process ID.
pgrep -x -f "cce-gdbserver <i>ip_address:port</i> --attach <i>xxx</i> "; pgrep -x -f "cce-gdbserver <i>ip_ddress:port</i> ~/ide_daemon/debug/xxxx"	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process ID.
pidof xxxx	Replace <i>xxxx</i> with the actual process name.
ps ux awk '{print}' sed 1d; ps ux awk '{print \$x \t\$y}' sed 1d	-
ps -ef awk '{print}' sed 1d; ps -ef awk '{print \$x \t\$y}' sed 1d	-

	<i>ide_cmd.sh --eth0_up ip_address</i>	The sudo command is used to escalate the user's operation permission. The actually executed command is ifconfig eth0 ip_address. This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.
	<i>ide_cmd.sh --setup_eth0 ip_address netmask_address gw_address</i>	The sudo command is used to escalate the user's operation permission. The actually executed command is ifconfig eth0 ip_address netmask netmask_address && route add default gw gw_address. This command applies only to developer boards. Replace <i>ip_address/netmask_address/gw_address</i> with the actual value.
	<i>sed -i '/iface.eth0/{n;s/address.*/address ip_address/;}' /etc/network/interfaces</i>	The sudo command is used to escalate the user's operation permission. sudo command This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.

	<code>sed -i '/iface.eth0/{n;n;s/netmask.*/netmask ip_address/;} /etc/network/interfaces</code>	The sudo command is used to escalate the user's operation permission. sudo command This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.
	<code>sed -i '/iface.eth0/{n;n;n;s/gateway.*/gateway ip_address/;} /etc/network/interfaces</code>	The sudo command is used to escalate the user's operation permission. sudo command This command applies only to developer boards. Replace <i>ip_address</i> with the actual value.
	<code>del /f/s/q D:\profiler-app; del /f/s/q D:\profiler-app \xxx</code>	Deletes a file on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
	<code>dir D:\profiler-app; dir D:\profiler-app\xxx</code>	Lists all files in a directory on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
	<code>md D:\profiler-app; md D:\profiler-app\xxx</code>	Creates a folder on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.

<code>rd /s/q D:\profiler-app; rd /s/q D:\profiler-app\xxx</code>	Deletes a folder on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
<code>tar -xvf D:\profiler-app\xxx.tar -C D:\profiler-app\yyy</code>	Creates a compressed package on the Windows OS. Replace <i>D</i> in the command with the actual drive letter on the Windows PC.
<code>kill \$(pidof ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill \$(pidof -x ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill -9 \$(pidof ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill -9 \$(pidof -x ~/HIAI_PROJECTS/xxxx/out/yyy)</code>	-
<code>kill `pgrep -x -f "cce-gdbserver ip_address:port --attach xxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process number.
<code>kill -9 `pgrep -x -f "cce-gdbserver ip_address:port --attach xxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values. Replace <i>xxx</i> with the actual process number.
<code>kill `pgrep -x -f "cce-gdbserver ip_address:port ~/ide_daemon/debug/xxxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values.
<code>kill -9 `pgrep -x -f "cce-gdbserver ip_address:port ~/ide_daemon/debug/xxxx"`</code>	Replace <i>ip_address</i> and <i>port</i> with the actual values.
<code>cd ~/HIAI_PROJECTS/xxxx/out;./yy</code>	Runs an executable file in the directory. This command has no parameter.

	<code>cd ~/HIAI_PROJECTS/xxxx/out;./yy ~/HIAI_DATANDMODELSET/xxxx yyy ddd zzz</code>	Runs a binary file in the directory. The parameters are fixed. Replace ~/HIAI_DATANDMODELSET/xxxx, yyy, ddd, and zzz with the actual values.
	<code>cd ~/HIAI_PROJECTS/xxxx/out;~/HIAI_PROJECTS/xxxx/out/yyy ~/HIAI_DATANDMODELSET/xxxx yyy ddd zzz</code>	Runs a binary file in the directory. The parameters are fixed. Replace ~/HIAI_DATANDMODELSET/xxxx, yyy, ddd, and zzz with the actual values.

9.1.4 Transferring a File to the Device by Using IDE Daemon

Function

Transfers a file to the device by using IDE Daemon

Syntax

```
IDE-daemon-client --host <host_ip>:<port> [--device <deviceId>] --file <source_dir> <dest_dir>
```

Parameter

Table 9-3 Parameter description

Parameter	Description	Mandatory or Not	Default Value
--host	<i>host_ip</i>: specifies the IP address of the host. <i>port</i>: specifies the port number.	Yes	The default value of <i>port</i> is 22118 .

Parameter	Description	Mandatory or Not	Default Value
--device	If the host is connected to multiple devices, you can specify a device by using --device <i>deviceID</i> . If the device is not specified, the device with <i>deviceID</i> of 0 is used by default.	No	The default value of <i>deviceID</i> is 0 .

Parameter	Description	Mandatory or Not	Default Value
--file	• <i>source_dir</i>: specifies the directory of the source file on the IDE Daemon server. The value includes the file name. • <i>dest_dir</i>: specifies the directory of the destination file on the device, including the file name. <i>dest_dir</i> must be specified. Otherwise, the command fails to be executed. The file path must be a directory in / home/HwHiUser/hdcd. You only need to specify the relative path in the command. For example, if the destination file path is /home/HwHiUser/hdcd/ide_daemon/B.java, you only need to set file to ide_daemon/B.java in the command.	Yes	-

Example

1. Log in to the server on the host as the **HwHiAiUser** user.

2. Run the following command to transfer the **A.java** file to the device and save the file as **B.java**:
- ```
IDE-daemon-client --host xx.xx.xx.xx:22118 --file /tmp/A.java "ide_daemon/B.java"
```

 **NOTE**

Replace *xx.xx.xx.xx* with the actual IP address of the host.

## 9.1.5 Transferring a File to the Host by Using IDE Daemon

### Function

Transfers a file to the host by using IDE Daemon.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --sync <source_dir> <dest_dir>
```

### Parameter

**Table 9-4** Parameter description

| Parameter | Description                                                                                                                                           | Mandatory or Not | Default Value                                      |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"><li><i>host_ip</i>: specifies the IP address of the host.</li><li><i>port</i>: specifies the port number.</li></ul> | Yes              | The default value of <i>port</i> is <b>22118</b> . |

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Mandatory or Not | Default Value |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------|
| --sync    | <ul style="list-style-type: none"> <li>• <i>source_dir</i>: specifies the directory of the source file on the IDE Daemon server. The value includes the file name.</li> <li>• <i>dest_dir</i>: specifies the directory of the destination file on the host, excluding the file name. For example, the directory of the destination file is <code>~/ide_daemon</code> or <code>~/ide_daemon/</code>, both of which indicate that the destination file is stored in the <b>ide_daemon</b> directory. <i>dest_dir</i> must be specified. Otherwise, the command fails to be executed. Files can be stored only in the following directories: <ul style="list-style-type: none"> <li>- <code>~/ide_daemon</code></li> <li>- <code>~/HIAI_PROJECTS</code></li> <li>- <code>~/HIAI_DATANDMODELSET</code></li> </ul> </li> </ul> | Yes              | -             |

| Parameter | Description                                                                                                  | Mandatory or Not | Default Value |
|-----------|--------------------------------------------------------------------------------------------------------------|------------------|---------------|
|           | <ul style="list-style-type: none"><li>- /opt/mini</li><li>- /var/log/hisi_logs</li><li>- /var/dlog</li></ul> |                  |               |

## Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to transfer the **A.java** file to the specified directory on the host:

```
IDE-daemon-client --host xx.xx.xx.xx:22118 --sync /tmp/A.java "~/ide_daemon"
```

### NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.

## 9.1.6 Obtaining a File from the Host by Using IDE Daemon

### Function

Obtains a file from the host by using IDE Daemon.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --get <dest_dir> <source_dir>
```

### Parameter

**Table 9-5** Parameter description

| Parameter | Description                                                                                                                                               | Mandatory or Not | Default Value                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"><li>• <i>host_ip</i>: specifies the IP address of the host.</li><li>• <i>port</i>: specifies the port number.</li></ul> | Yes              | The default value of <i>port</i> is <b>22118</b> . |

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Mandatory or Not | Default Value |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------|
| --get     | <ul style="list-style-type: none"> <li>• <i>source_dir</i>: specifies the directory of the source file on the host, including the file name. Files can be stored only in the following directories: <ul style="list-style-type: none"> <li>- ~/ide_daemon</li> <li>- ~/HIAI_PROJECTS</li> <li>- ~/HIAI_DATAN_MODELSECT</li> <li>- /opt/mini</li> <li>- /var/log/hisi_logs</li> <li>- /var/dlog</li> </ul> </li> <li>• <i>dest_dir</i>: specifies destination directory used to store the file. The value includes the file name. <i>dest_dir</i> must be specified. Otherwise, the command fails to be executed.</li> </ul> | Yes              | -             |

## Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to obtain the **B.java** file from the host and save it as **A.java** by using IDE Daemon:

```
IDE-daemon-client --host xx.xx.xx.xx:22118 --get /tmp/A.java "~/ide_daemon/B.java"
```

 NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.

## 9.1.7 Detecting the Heartbeat Between IDE Daemon and the Host

### Function

Detects the heartbeat between IDE Daemon and the host to check whether the host server is running.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --detect
```

### Parameter

Table 9-6 Parameter description

| Parameter | Description                                                                                                                                           | Mandatory or Not | Default Value                                      |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"><li><i>host_ip</i>: specifies the IP address of the host.</li><li><i>port</i>: specifies the port number.</li></ul> | Yes              | The default value of <i>port</i> is <b>22118</b> . |
| --detect  | Detects the heartbeat. If the heartbeat message is detected, <b>SUCCESS</b> is returned. Otherwise, no information is returned.                       | Yes              | -                                                  |

### Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to check the heartbeat between IDE Daemon and the host:

```
IDE-daemon-client --host xx.xx.xx.xx:22118 --detect
```

 NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.

## 9.1.8 Setting Log Levels

### Function

Sets the log levels for the host and device.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> [--device <deviceId>] --log '<params>'
```

### Parameter

**Table 9-7** Parameter description

| Parameter | Description                                                                                                                                                                                                       | Mandatory or Not | Default Value                                      |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"><li>• <i>host_ip</i>: specifies the IP address of the host.</li><li>• <i>port</i>: specifies the port number.</li></ul>                                                         | Yes              | The default value of <i>port</i> is <b>22118</b> . |
| --device  | If the host is connected to multiple devices, you can specify a device by using <b>--device</b> <i>deviceId</i> . If the device is not specified, the device with <i>deviceId</i> of <b>0</b> is used by default. | No               | The default value of <i>deviceId</i> is <b>0</b> . |



| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Mandatory or Not | Default Value |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------|
| --log     | <p><i>params</i> includes the following items:</p> <ul style="list-style-type: none"> <li>• <b>SetLogLevel:</b> sets the global log level. The value <b>0</b> indicates the global log level, the value <b>1</b> indicates the module-level log level, and the value <b>2</b> indicates the event log level. When the event log level is set, <b>enable</b> indicates that the event log level is enabled, and <b>disable</b> indicates that the event log level is disabled.</li> <li>• <b>SetLogLevel[<i>Module name:Level</i>]:</b> sets the log level of a specified module. <ul style="list-style-type: none"> <li>– <i>Level:</i> <b>error</b>, <b>info</b>, <b>warning</b>, <b>debug</b>, <b>null</b> (<b>null</b> indicates that no log is recorded.)</li> <li>– <i>Module name:</i> <b>dlog</b>, <b>slog</b>, <b>cce</b>, and other values</li> </ul> </li> </ul> | Yes              | -             |

## Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following commands to set the log level for the host and device:  

```
IDE-daemon-client --host xx.xx.xx.xx:22118 --log 'SetLogLevel(0)[error]'
IDE-daemon-client --host xx.xx.xx.xx:22118 --log 'SetLogLevel(1)[slog:error]'
IDE-daemon-client --host xx.xx.xx.xx:22118 --log 'SetLogLevel(2)[enable]'
```

## NOTE

- Replace *xx.xx.xx.xx* with the actual IP address of the host.
- The value of *Module name* is defined in the enumerated format. The options are as follows:

```
enum {
DLOG = 0, // Dlog
SLOG, // Slog
IDEDD, // IDE daemon device
IDEDH, // IDE daemon host
LOGAGTH, // log agent host
HCCL, // HCCL
FMK, // Framework
HIAENGINE, // Matrix
DVPP, // DVPP
RUNTIME, // Runtime
CCE, // CCE
#if (OS_TYPE == LINUX)
HDC, // HDC
#else
HDCL, // HDCL windows has a def with the same name HDC, so change HDC to
HDCL
#endif
DRV, // Driver
MDCCONTROL, // Mdc control
MDCFUSION, // Mdc fusion
MDCLOCATION, // Mdc location
MDCPERCEPTION, // Mdc perception
MDCMOP,
MDCFSM,
MDCCOMMON,
MDCMONITOR,
MDCBSWP, // MDC basesoftware platform
MDCDEFAULT, // MDC UNDEFINE
MDCSC, // MDC spatial cognition
MDCBP,
MDCTF,
MLL,
DEVMM, // Dlog memory managent
KERNEL, // Kernel
MDCSMCMD, // sm_control_cmd
MDCSCREEN, // parking_spot_screen
LIBMEDIA, // Libmedia
CCECPU, // ai cpu
ASCENDDK, // AscendDK
ROS, // ROS
```

```
HCCP,
RoCE,
TEFUSION,
PROFILING, // Profiling
DP, // Data Preprocess
APP, // User Application call HIAI_ENGINE_LOG
INVALID_MOUDLE_ID
};
```

## 9.1.9 Sending Profiling Items to the Host by Using IDE Daemon

### Function

Sends profiling items to the host by using IDE daemon and start profiling.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --profile "<params>"
```

### Parameter

**Table 9-8** Parameter description

| Parameter | Description                                                                                                                                              | Mandatory or Not | Default Value                                      |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"> <li><i>host_ip</i>: specifies the IP address of the host.</li> <li><i>port</i>: specifies the port number.</li> </ul> | Yes              | The default value of <i>port</i> is <b>22118</b> . |
| --profile | Specifies profiling items in <i>params</i> .                                                                                                             | Yes              | -                                                  |

### Prerequisites

Mind Studio has been installed and project files compiled on Mind Studio have been copied to the directory on the host in advance.

For example, if the project name on Mind Studio is **testProject** and files generated after the project is compiled are in *Home directory of the Mind Studio installation user/tools/projects/testProject/out*, you need to upload files in *Home directory of the Mind Studio installation user/tools/projects/testProject/out* to the **/home/HwHiAiUser/HIAI\_PROJECTS/workspace\_mind\_studio/testProject/out** directory on the host and rename the executable file as **workspace\_mind\_studio\_Project name** (for example, **workspace\_mind\_studio\_testProject**). In addition, you need

to run the **chmod +x workspace\_mind\_studio\_testProject** command to grant the execute permission to the executable file.

## Example

1. Log in to the Mind Studio server as the Mind Studio installation user.
2. Run the following commands to set the environment variables:

```
export LD_LIBRARY_PATH=~/.tools/che/ddk/ddk/uihost/lib
export PATH=$PATH:~/.tools/che/ddk/ddk/uihost/bin
```

3. Run the **IDE-daemon-client** command.

The following is only an example. You need to change the parameters to actual values.

- Replace *xx.xx.xx.xx* after **--host** with the actual IP address of the host server.
- */home/ascend* indicates the home directory of the Mind Studio installation user. Change it to the actual value.
- Replace */xxx/xxx* after **app\_dir** with the path of the app on the host, for example, **/home/HwHiAiUser/HIAI\_PROJECTS/workspace\_mind\_studio/testProject/out**.
- Replace *app* with the app name in the directory specified by **app\_dir**, for example, **workspace\_mind\_studio\_testProject**.
- Replace the path after **result\_dir** with the actual directory on the host server. After running the command, you can view the profiling result in the directory specified by **result\_dir**.

```
IDE-daemon-client --host xx.xx.xx.xx:22118 --profile "{ \"ddk_dir\": \" /home/ascend/tools/che/ddk/ddk \", \"app\": \"workspace_mind_studio_testProject\", \"app_dir\": \" /home/HwHiAiUser/HIAI_PROJECTS/workspace_mind_studio/testProject/out\", \"umodel\": \"MIND\", \"result_dir\": \" /home/ascend/tools/projects/test_ycm\", \"peripheral_profiling\": \" /\", \"ts_cpu_profiling\": \"off\", \"ai_cpu_profiling\": \"off\", \"rts_profiling\": \"on\", \"ai_core_profiling_mode\": \" /\", \"ai_core_profiling\": \"off\", \"hiai_engine_profiling\": \"off\", \"framework_profiling\": \"off\", \"ctrl_cpu_profiling\": \"off\", \"profiling_mode\": \"online\", \"llc_profiling\": \"off\", \"ddr_profiling\": \"on\", \"deviceid\": \"0\", \"ddr_profiling_events\": \"read,write\", \"hiai_engine_profiling\": \"on\", \"analysis_target\": \"Launch Application\"}"
```

### NOTE

For details about the meanings of profiling items, see the "Profiling" section in *Ascend 310 Mind Studio Auxiliary Tools*.

## 9.1.10 Collecting Dump Information Recorded by the Black Box on the Host

### Function

Collects dump information recorded by the black box on the host by using IDE Daemon.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --bbox <params>
```

Parameter

Table 9-9 Parameter description

| Parameter | Description                                                                                                                                               | Mandatory or Not | Default Value                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"><li>• <i>host_ip</i>: specifies the IP address of the host.</li><li>• <i>port</i>: specifies the port number.</li></ul> | Yes              | The default value of <i>port</i> is <b>22118</b> . |

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Mandatory or Not | Default Value |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------|
| --bbox    | <p><i>params</i> includes the following items:</p> <ul style="list-style-type: none"> <li>• <b>--all</b>: obtains the list and directory of files on all abnormal devices.</li> <li>• <b>--status</b>: obtains the number of exceptions on all devices. If there is only one device, the value, for example, <b>{{0,3}}</b>, is returned, indicating that three exceptions occur on device 0. If there are multiple devices, the value, for example, <b>{{1,3};(0,3)}</b> is returned, indicating that three exceptions occur on device 0 and three exceptions occur on device 1. Information about multiple devices is separated by a semicolon (;).</li> <li>• <b>--latest</b>: obtains the list and directory of the latest files of</li> </ul> | Yes              | -             |

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Mandatory or Not | Default Value |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------|
|           | <p>abnormal devices. If <b>FAILED</b> is returned, no exception occurs.</p> <ul style="list-style-type: none"> <li>'--filter --device <i>deviceID</i> --except-num <i>n</i>': obtains the lists and directories of files from the first <i>n</i> abnormal devices among the specified devices. If multiple devices need to be specified, separate them with a comma (,). <b>--filter --device <i>deviceID</i> --except-num <i>n</i></b> must be placed in single or double quotation marks.</li> </ul> |                  |               |

## Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to collect abnormal file lists and directories from all devices on the host:

```
IDE-daemon-client --host xx.xx.xx.xx:22118 --bbox --all
```

### NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.



## 9.1.11 Obtaining the Information About the Device Connected to the Host

### Function

Obtains the information about the device connected to the host by using IDE Daemon.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --api <params>
```

### Parameter

**Table 9-10** Parameter description

| Parameter | Description                                                                                                                                               | Mandatory or Not | Default Value                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"><li>• <i>host_ip</i>: specifies the IP address of the host.</li><li>• <i>port</i>: specifies the port number.</li></ul> | Yes              | The default value of <i>port</i> is <b>22118</b> . |

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Mandatory or Not | Default Value |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------|
| --api     | <p><i>params</i> includes the following items:</p> <ul style="list-style-type: none"> <li>• <b>device_info</b>: indicates the list of devices connected to the host.</li> <li>• <b>device_status</b>: Only the status of the camera on the developer board can be queried. The value <b>0</b> indicates that the camera does not exist, and the value <b>1</b> indicates that the camera exists. If the value <b>100</b> is returned, <b>1</b> indicates the Hi3559 chip, and the last two 0s indicate that the two cameras do not exist.</li> <li>• <b>board_id</b>: Only the device type on the developer board can be queried.</li> <li>• <b>sys_version</b>: Only the product version of the developer board can be queried.</li> </ul> | Yes              | -             |

### Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to obtain the list of all devices connected to the host by using IDE Daemon:  
IDE-daemon-client --host xx.xx.xx.xx:22118 --api device\_info

 **NOTE**

Replace xx.xx.xx.xx with the actual IP address of the host.

## 9.1.12 Generating a Character String Encryption Files

### Function

Generates character string encryption files on the IDE Daemon server, including the .secu and .store files. These two files are used during certificate replacement.

### Syntax

```
IDE-daemon-client --key
```

### Parameter

**Table 9-11** Parameter description

| Parameter | Description                                   | Mandatory or Not | Default Value |
|-----------|-----------------------------------------------|------------------|---------------|
| --key     | Generates a character string encryption file. | Yes              | -             |

### Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to generate a character string encryption file in the current directory:  
Enter the password as prompted. Ensure that the **HwHiAiUser** user has the write permission on the directory where the command is run.  
IDE-daemon-client --key

## 9.1.13 Synchronizing the Time Between IDE Daemon and the Host

### Function

Synchronizes the time of the host server with the time of the IDE Daemon server.

### Syntax

```
IDE-daemon-client --host <host_ip>:<port> --time
```

## Parameter

**Table 9-12** Parameter description

| Parameter | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Mandatory or Not | Default Value                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------|
| --host    | <ul style="list-style-type: none"> <li><i>host_ip</i>: specifies the IP address of the host.</li> <li><i>port</i>: specifies the port number.</li> </ul>                                                                                                                                                                                                                                                                                                              | Yes              | The default value of <i>port</i> is <b>22118</b> . |
| --time    | <p>If both <b>--host</b> and <b>--time</b> are used, the time of the IDE Daemon server prevails, that is, the time on the host server is synchronized to be consistent with that of the IDE Daemon server.</p> <p>The IDE-daemon-client commands with the <b>--host</b> parameter can include the <b>--time</b> parameter, which is optional. The following provides an example:</p> <pre>IDE-daemon-client --host &lt;host_ip&gt;:&lt;port&gt; --detect --time</pre> | Yes              | -                                                  |

## Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the command to check the heartbeat and synchronize the time:  
IDE-daemon-client --host xx.xx.xx.xx:22118 --detect --time

### NOTE

Replace *xx.xx.xx.xx* with the actual IP address of the host.

## 9.1.14 Viewing the Help Information

### Function

Displays the help information about IDE-daemon-client commands.

### Syntax

```
IDE-daemon-client --help
```

### Parameter

**Table 9-13** Parameter description

| Parameter | Description                    | Mandatory or Not | Default Value |
|-----------|--------------------------------|------------------|---------------|
| --help    | Displays the help information. | Yes              | -             |

### Example

1. Log in to the server on the host as the **HwHiAiUser** user.
2. Run the following command to view help information:  

```
IDE-daemon-client --help
```

# 10 Appendix

---

[10.1 Getting Help](#)

[10.2 General Disclaimer](#)

## 10.1 Getting Help

### 10.1.1 Preparing for Contacting Huawei

To better solve the problem, you need to collect troubleshooting information and make debugging preparations before contacting Huawei.

### 10.1.2 Contacting Huawei Technical Support

If a fault persists after troubleshooting, contact Huawei technical support by visiting <https://e.huawei.com>. Before you report a fault to Huawei engineers, collect the following information:

- Name and address of your organization
- Contact person and telephone number
- Time when the fault occurred
- Detailed fault symptom
- Device types, hardware models and software versions
- Any measures taken and effects
- Fault severity and expected rectification deadline

#### NOTE

If a fault described in this document occurs and cannot be rectified by the recommended solutions, contact the local Huawei branch office or Huawei Customer Service Center for technical support.

### 10.1.3 Preparing for Debugging

When you seek Huawei technical support, Huawei technical support engineers may assist you in performing some operations to further collect fault information or rectify the fault.

Before contacting technical support engineers, prepare the spare parts for boards and port modules, screwdrivers, screws, serial cables, and network cables.

## 10.1.4 Using Product Documentation

Huawei provides the documents delivered with the equipment. This document provides guidance for you to solve common problems that occur during routine maintenance or troubleshooting.

To better rectify the fault, you are advised to use the guide before contacting Huawei technical support engineers.

## 10.1.5 Getting Help from Website

Huawei provides users with timely and efficient technical support through the regional offices, secondary technical support system, telephone technical support, remote technical support, and onsite technical support.

### Technical support website

See technical documents on one of the following technical support websites:

- Huawei enterprise business website: <http://e.huawei.com>
- Huawei carrier business website: <http://carrier.huawei.com>

### Huawei Technical Support

If a fault persists after taking the above measures, contact technical support at your local Huawei office. If a local Huawei office is not available, contact Huawei technical support as follows:

- Enterprise customers  
Send emails to [support\\_e@huawei.com](mailto:support_e@huawei.com) or visit [Global Service Hotline](#).
- Carriers  
Send emails to [support@huawei.com](mailto:support@huawei.com) or visit [Global TAC Information](#).

### Knowledge Base and Self-Service Platform

If you want to further learn server knowledge and communicate with experts, perform the following operations:

- [HUAWEI Server Information Self-Service Platform](#) for specific server product documentation.
- [Huawei server intelligent Q&A system](#) for quick learning about products.
- [Huawei Enterprise Support Community \(Sever\)](#) for learning and discussion.
- You can learn server cases from [Knowledge Base](#).

## 10.1.6 Ways to Contact Huawei

Huawei Technologies Co., Ltd. provides customers with comprehensive technical support and service. For any assistance, contact our local office or company headquarters.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's  
Republic of China

Website: <http://e.huawei.com>

## 10.2 General Disclaimer

- This document may include the third-party information covering products, services, software, components, and data. Huawei does not control and assumes no responsibility for the third-party content, including but not limited to the accuracy, compatibility, reliability, availability, legitimacy, appropriateness, performance, non-infringement, and status update, unless otherwise specified in this document. Huawei does not provide any guarantee or authorization for the third-party content mentioned or referenced in this document.
- If the user needs the third-party license, the user should obtain it in an authorized or legal way, unless otherwise specified in this document.